

Legacy data defects

Every defect found in the 2026-08-21 dump, with its size and what to do about it. Reproduce with `./tools/legacy/audit.sh` — every number here comes from that script.

Severity: **BLOCKER** stops a reconciliation gate · **HIGH** loses or corrupts data · **MEDIUM** degrades a feature · **LOW** cosmetic.

Summary

#	Defect	Size	Severity
1	<code>v_id</code> is not the voucher key	1,200 vouchers would merge	BLOCKER
2	8 vouchers with blank scope columns	₹21,025.82	BLOCKER
3	One voucher out of balance by ₹0.01	1 voucher	BLOCKER
4	Detail scope columns disagree with headers	597 rows	HIGH
5	<code>br_code</code> encoded two ways (<code>1</code> vs <code>B001</code>)	495 rows	HIGH
6	Duplicate barcode — phantom stock	1 barcode, ₹900	HIGH
7	Stock items with no purity, item or quantity	8 items	HIGH
8	Purity codes missing from the master	<code>SLVPC</code> , 5 items	HIGH
9	Zero dates	20 columns, ~180k values	HIGH
10	UTF-8 bytes inside latin1 columns	5 names	HIGH
11	Voucher dated outside its own financial year	1 voucher	MEDIUM
12	Voucher dated after the dump was taken	1 voucher	MEDIUM
13	Invalid mobile numbers	838 of 39,429	MEDIUM
14	Invalid email addresses	57 of 7,295	MEDIUM
15	144 metal items with zero gross weight	144 items	MEDIUM
16	12 stock items valued at zero	12 items	MEDIUM
17	Names differing only by whitespace	571 names	LOW
18	4 dates of birth in the future	4 parties	LOW
19	Birthdays with no year (<code>0000-09-02</code>)	13 parties	HIGH

#	Defect	Size	Severity
20	Implausible birth years (1, 1000, 199...)	317 parties	MEDIUM
21	Credentials in the dump: base64 passwords, a live API token, plain-HTTP login	24 users + 2 services	SECURITY
22	Duplicate item: <code>SILVER PICHKARY</code> recorded twice	1 item	LOW

Checked and clean: no orphan voucher lines, no orphan ledger references, no negative amounts or weights, no line both debit and credit, no line with neither, no net weight exceeding gross weight, no blank party names, `ldgr_code` unique across all 40,519 parties.

1. `v_id` is not the voucher key — BLOCKER

Key	Distinct	vs 1,848 rows
<code>v_id</code>	648	✗ merges 1,200
<code>(v_id, v_typ)</code>	1,840	✗ merges 8
<code>(v_id, v_typ, ac_yr)</code>	1,848	✓

`v_id` restarts per voucher type, so the same id exists as a PMNT and a JRNL. Joining on it alone silently merges unrelated vouchers.

Correction. `docs/migration-profile.md` first reported the key as five columns including `br_code` and `rt_code`. That is wrong — those two are dirty (defects 4 and 5) and cannot be part of a key. Grouping by them splits vouchers apart and reports 838 false imbalances. **Three columns, not five.**

Do: key on `(v_id, v_typ, ac_yr)` everywhere; carry all three into `legacy_id_map`.

2. Eight vouchers with blank scope columns — BLOCKER

Eight JRNL headers carry blank `ac_yr`, `comp_code = 0`, blank `rt_code`. They are real: each has its own detail lines, each balances exactly, and their dates and amounts differ from the well-formed JRNL 1–8 beside them. Together: **₹21,025.82**.

Any filter on `ac_yr = '2026-2027'` drops all eight and the trial balance never ties.

Do: derive `ac_yr` from `date1`, set `comp_code = 1`, `rt_code = 'R001'`.

3. One voucher out of balance by ₹0.01 — BLOCKER

JRNL 312, 2026-05-16 — a Paytm settlement

credit	PAYTM PAYMENTS SERVICES LIMITED	214.14
credit	PAYTM PAYMENTS SERVICES LIMITED	295.00
debit	CREDIT CARD EXPENSES	431.48
debit	INPUT IGST	77.67

debits 509.15 credits 509.14 out by 0.01

Corrected. This first named `v_id 312 PMNT, 2026-06-23, 1571.15 / 1571.14`. That came from grouping on `v_id` alone, which merged the payment and the journal sharing that number and reported the wrong one's figures — the same trap as defect 1. Confirmed against the migrated data: 1,847 of 1,848 vouchers balance, and this is the one that does not.

1,847 of 1,848 balance. This one does not. It is the float problem made visible — exactly what `rupees.Amount` exists to prevent going forward.

Do: repair explicitly with accountant sign-off. Never round it away silently: reconciliation check #4 requires every voucher to balance.

4. Detail scope columns disagree with their headers — HIGH

597 `voucher_detail` rows have `ac_yr`, `br_code` or `rt_code` differing from their own header. 22 rows have blank `ac_yr`, 58 blank `br_code`, 22 blank `rt_code`.

Do: treat the header as authoritative. Never read scope from a detail line.

5. Branch encoded two different ways — HIGH

`voucher_detail.br_code` holds `B001` (the code), `1` (the numeric `brid`), and blank. **495 rows carry 1**. `voucher_main.br_code` is always `B001`.

Do: normalise `1` → `B001` and blank → header value during the transform.

6. Duplicate barcode — phantom stock — HIGH

Barcode `86065435` exists twice in `barcode_main` (ids 96399, 96409), both `gw = 0`, both `total = 900`, and twice in `retail_all_stock` as Opening.

A barcode is the physical tag on an item; two rows means **₹900 of stock that does not exist**. This is why `retail_all_stock` shows 2,820 rows against 2,816 items.

The other two repeated barcodes in stock are **not** defects — `retail_all_stock` is a movement log, and those are legitimate hallmark issue/receive pairs.

Do: delete one copy, with sign-off. Add a unique constraint on the new table.

7. Eight stock items with no purity, item or quantity — HIGH

The same 8 rows fail three checks: blank `purity`, orphan `item_id`, `qty <= 0`. They are unidentifiable stock.

Do: quarantine and review individually. Do not migrate silently.

8. Purity code missing from the master — HIGH

Five items use purity `SLVPC`, which does not exist in `gold_master`. The master has 19 codes; stock references 20.

Do: add `SLVPC` with its correct fineness — **ask the business, do not guess**. An invented fineness misprices every one of those items.

9. Zero dates — HIGH

`0000-00-00` appears in **20 columns**, ~180,000 values. MySQL 5.x accepted it; MySQL 8 needs `ALLOW_INVALID_DATES` to even load it; **PostgreSQL has no such value**.

Six columns are **100% zero** and were never used: `address_book_master.rec_dt` and `.op_bal_dt`, `ledger_master.rec_dt` and `.op_bal_dt`, `customer_order.checked_date` and `.re_checked_date`.

Do: map `0000-00-00` → `NULL`. Drop the six dead columns entirely.

10. UTF-8 bytes inside latin1 columns — HIGH

Five party names contain non-ASCII bytes, and they are **not** latin1:

```
C2A0 20 53555348...  → U+00A0 (UTF-8 no-break space) + " SUSHMITA BHARTI"
...4B415552 09      → "TARVINDER KAUR" + TAB
```

`C2A0` is a UTF-8 sequence sitting in a latin1 column. **A naive latin1→UTF-8 conversion turns it into Å mojibake.**

Do: detect valid UTF-8 sequences first and pass them through; convert only the genuinely latin1 remainder. Strip embedded tabs and no-break spaces.

11. Voucher dated outside its own financial year — MEDIUM

One voucher is dated `2026-03-09` while labelled `ac_yr 2026-2027`. March 2026 is FY **2025-2026**. It is one of the eight from defect 2.

Do: either open the prior financial year or reassign the date, with sign-off. The `finyear` value-object tests already pin this case.

12. Voucher dated after the dump was taken — MEDIUM

One voucher is dated 2026-12-05 , nearly four months after the 2026-08-21 dump. Almost certainly a keying error (05-12 vs 12-05).

Do: confirm the intended date with the business. Add a future-date guard.

13. Invalid mobile numbers — MEDIUM

838 of 39,429 populated mobile numbers are not ten digits: eleven-digit values, numbers with a trailing tab, and genuine international numbers.

***Corrected.** This entry first said 163 of 4,414, and concluded that messaging could reach "about 4,250 parties, not 40,519". That was wrong, and wrong in the direction that would have shrunk the SMS/WhatsApp integration to nothing.*

*The legacy table has **two** mobile columns. `mobile` holds 4,414 values; `mobile1` holds **39,429**, and every row with `mobile` also has `mobile1` — it is a strict subset. I profiled the wrong column.*

***Real coverage is 97.3%**, and 38,591 numbers are cleanly ten digits.*

Do: migrate `mobile1` as the mobile number and ignore `mobile` . Strip whitespace, keep valid international numbers, quarantine the rest.

14. Invalid email addresses — MEDIUM

57 of 7,295 populated emails contain no @ .

Do: null them out rather than migrating unusable values.

15. Metal items with zero gross weight — MEDIUM

144 items carry a metal purity (non-zero fineness) but `gw = 0` . Inspection shows most are flat-priced goods — 16 are GOLD RAKHI at `rate = 0` , `gw = 0` , `total = 2000` .

This is **not** necessarily corruption: it is a **pricing mode the data model must support** — priced per piece, not by weight. The billing screen needs it.

Do: confirm which are intentionally flat-priced and mark them explicitly, so the distinction survives instead of looking like missing data.

16. Stock items valued at zero — MEDIUM

12 items have `total = 0` . They occupy stock but contribute nothing to valuation.

Do: review before migrating; they may need pricing rather than deletion.

17. Names differing only by whitespace — LOW

571 party names contain double spaces. These inflate the duplicate-name count (3,342 names across 11,829 rows) because "A SHARMA" and "A SHARMA" read as different parties.

Do: normalise whitespace *before* the dedupe review, not after.

18. Dates of birth in the future — LOW

4 parties have a DOB after 2026-08-21. Harmless except that they will trigger the Due Date Reminder screen.

19. Birthdays with no year — HIGH

Thirteen `address_book_master.d_o_b` values have a **real month and day but year** `0000` — `0000-09-02`, `0000-08-14`, `0000-11-27`. Someone recorded a birthday without the year.

This is dangerous rather than merely untidy: **PostgreSQL has no year zero** (it runs 1 BC straight to 1 AD), so it does not reject these — it silently coerces them into year 1. My first extract run did exactly that, turning 13 birthdays into `0001-...` dates indistinguishable from defect 20. Caught by comparing staged NULL counts against the source.

The day and month are **real information** — birthdays drive the Due Date Reminder screen — but a `date` column cannot hold them.

Do: NULL them in staging (done — `isZeroDate` now catches any `0000-` prefix) and give the domain model a partial-date concept so the day and month can be recovered from the dump.

20. Implausible birth years — MEDIUM

317 parties have a birth year between 1 and 1899:

Year	Parties
1	247
1000	60
199, 200, 935, 975, 979, 996, 997, 1899	1 each

These are valid PostgreSQL dates, so they load cleanly and stay wrong. Only **7,360 of 40,519 parties (18%) have a usable date of birth.**

Do: carry them into staging faithfully, then quarantine at the domain transform. Do not feed them to the Due Date Reminder.

21. Passwords stored base64-encoded, not hashed — SECURITY

All 24 user passwords in `legacy.users` decode cleanly from base64 into printable text. Base64 is an encoding, not encryption — anyone holding the dump holds every password.

Ten columns store credentials this way, including `branch_master.br_psw` (`b001`) and `retail_master.rt_psw` (`r001`).

Two third-party integrations are worse than the passwords:

- `api_details` — a Maytapi WhatsApp gateway whose API token sits in plain text, both inside the endpoint URL and again in `key_name` . Anyone with the dump can send WhatsApp messages as the shop. (Its `password` column is empty; the token is the credential.)
- `whatsapp_profile` — a second WhatsApp provider at `http://223.196.94.133` , a bare IP over plain HTTP, with username `mcjewl` and a six-digit password.

Do: carry no password column into the new schema — none is. Ask anyone who reused their ERP password elsewhere to change it, and rotate the two service credentials. See `docs/blockers.md` B15.

22. A duplicated item — LOW

`SILVER PICHKARY` appears twice in the catalogue (item ids 1063 and 1156). The two rows are **identical in every attribute** — same group, subgroup, type, unit, rate and HSN — and neither carries a barcode. It is a genuine double entry.

Worth contrasting with `COIN` , which also appears twice and is **not** a defect: those two rows differ in group, subgroup and HSN (silver coin versus pearl), and one carries 114 barcodes. They are two products that share a name.

That distinction is why item names are not unique in the new schema. Identity is the item id; a shop naming two products the same way is ordinary.

Do: confirm with the business, then delete one `SILVER PICHKARY` . Deleting a catalogue row is their call, not a migration's.

23. Opening balances: the ledger's side is not a side — HIGH

`ledger_master` carries `op_bal` (the amount, stored as text) and `op_bal_db_cr` , whose name reads as a debit/credit indicator. It is not one. Across 785 ledgers it holds:

Value	Ledgers
Yes	389
(blank)	282
No	114

Never `Dr` , never `Cr` . It is a yes/no flag whose meaning is not recorded, and it does not correlate with account type -- `No` covers 45 asset, 64 liability and 3 expense ledgers; `Yes` covers 49 asset and 11 liability.

This was migrated as if it were a side. The first ledger load read the side from it, and the counts prove the mapping survived intact: 389 `Yes` became the 389 `credit` ledgers, the remaining 396 became `debit` . The load's checksum summed opening balances by magnitude, ignoring side, so it tied exactly and the error

passed unnoticed.

Neither reading of the flag balances the books -- debits and credits come to ₹24.55 crore against ₹8.93 crore whichever way round it is mapped.

The real source is `ldgr_open_balance` : one row per ledger per account year, separate debit and credit numeric columns, no row carrying both, no duplicates. 353 rows for 2026-2027. Ledgers absent from it opened at zero.

Cash proves which source is right, because cash is the one ledger whose correctness is independently checkable -- a drawer cannot hold less than nothing:

Source	Cash opening	Negative day-end balances
<code>ledger_master.op_bal</code> + flag	4,88,217.19 credit	354 of 460 lines run negative
<code>ldgr_open_balance</code>	2,51,071.19 debit	none, across all 123 trading days

Repaired by `tools/legacy/repair_opening_balances.sh`, which restates every ledger from `ldgr_open_balance` and asserts the result matches the source per ledger rather than in total. After the repair 190 ledgers carry a non-zero opening: 74 debit totalling ₹10,80,37,319.15 and 116 credit totalling ₹26,12,65,922.64.

Two things the repair does not fix, both needing the accountant:

- **The opening trial balance does not balance.** Credits exceed debits by **₹15,32,28,603.49**. The missing debit is closing stock, which this ERP does not carry as a ledger -- the only stock-named account, `COUNTER STOCK`, is filed under *Customer* with a zero balance. Stock in hand comes to ₹14,23,61,188.20 across 2,575 items (20,038.936 g), covering 93% of the gap. Barcode values are at current rates rather than opening cost, so an exact tie was never available. Roughly ₹1.09 crore is unaccounted for.
- `mcj_ledgers` **holds one opening balance per ledger, not one per year.** The legacy table is keyed by account year. This is correct while only 2026-2027 is live; the first year-end rollover needs a per-year table.

`ldgr_open_balance_final` holds 186 rows for **2027-2028** -- a future year that cannot be final, and its gap is wider at ₹16.44 crore. Treated as stale rollover output and not used.

24. Forty-seven voucher numbers were never recorded — MEDIUM

Found by building the Missing Bills screen (44). Voucher numbers are issued in sequence, so an absent number is either a voucher that was deleted or one that was never entered.

Series	Gap runs	Numbers missing
JRNL 2026-2027	23	32
RCPT 2026-2027	9	11
PMNT 2026-2027	4	4
Total	36	47

The sales bills are complete. SALE-GST runs 1 to 224 with no gaps at all, and the same holds for CNTR, all four purchase series and SALE-RET. That is the finding that matters most: a missing tax invoice number is a GST exposure, and there are none. What is missing is internal — journals, receipts and payments.

Individual gaps are mostly single numbers (JRNL 47, 82, 131 and so on) with a few short runs (112–113). That pattern reads like ordinary deletions during data entry rather than anything systematic, but the screen only points; it cannot know why a number is absent and does not guess.

Needs: somebody who was there to say whether these were deleted deliberately. Not urgent, and not blocking anything.

25. 1,537 customers are called "CUSTOMER" — MEDIUM

Found by building the Incomplete Data screen (50).

12,026 of the 40,519 address book records share a name with at least one other, across 3,399 groups. Most of that is one entry: **1,537 records named simply CUSTOMER**, with USER accounting for another 45.

These are not duplicates. They carry distinct mobile numbers — a walk-in sale entered against a placeholder name rather than 1,537 rows for one person. The relationship survived; the name did not. They are reachable and they are real customers; they just cannot be greeted by name.

The rest of the address book's gaps, from the same screen:

Missing	Records
Anniversary date	38,594
Email address	33,224
Date of birth	32,842
Any address	11,201
Mobile number	1,090

Mobile coverage is the one that matters and it is strong: **39,429 of 40,519 records, 97.3%**, are reachable.

Needs: nothing technical. This is a counter-staff habit, and the screen now makes the cost of it visible.

26. Two days where gold was bought back above its selling price — LOW

Found by building the Metal Rates screen (56) and checking the spread across all 143 days rather than the week that was on screen.

Date	Purity	Sells at	Buys at	Spread
15 Jun 2026	995	15,327.00	16,560.00	–1,233.00
16 Jun 2026	995	15,327.00	16,560.00	–1,233.00

The shop was quoting to buy 995 gold for ₹1,233 a gram *more* than it sold it. Two consecutive days with identical figures reads like a keying error that stood until somebody noticed rather than a deliberate position.

These are the only two rows in 2,820 where the purchase rate exceeds the sale rate. No stock is priced from a historical rate — a bill keeps the rate of its own day — so the exposure is limited to whatever 995 was actually bought back on those two days.

Needs: somebody to check whether any old-gold purchase was booked on 15 or 16 June against purity 995. Not blocking.

27. Ten per cent of sales bills carry a non-standard GST rate — MEDIUM

Found by building the sales register (59) and checking the tax on every bill against the goods it was charged on.

Rate charged	Bills
3% — the standard rate on jewellery	202
1.5%	20
0.25%	2

0.25% is a real rate — it is the GST on rough and unworked diamonds — so those two bills are very likely correct.

The twenty at 1.5% are less obvious. They run from April to June across different customers and amounts, so they are deliberate rather than a slip. Both halves are present on each bill (CGST and SGST at 0.75% apiece), so it is the rate that differs and not a missing line.

This changed the sales bill. It had hard-coded 3%, which would have overcharged every one of these. The rate is now chosen on the bill, defaulting to 3%, with 1.5% and 0.25% offered because those are what the shop actually used.

Needs: somebody to say what the 1.5% bills were for, so the screen can suggest the right rate rather than leaving it to the biller to remember.

28. Sixty-one live customer orders are past their delivery date — HIGH

Found by building the order book (65) and comparing each live line's delivery date with today.

	Lines	Estimate
Live work (Process, Urgent, On Hold)	67	49,77,590.00
— of those, past their due date	61 (as at 22 Aug 2026)	
— of those, more than a year past	12	17,10,000.00

The oldest is order `RP/2223/00194`, due **9 August 2022** and still marked *Urgent* — **1,474 days** (as at 22 Aug 2026), more than four years.

Two of these move on their own. Counted against 23 August the past-due figure is **63**, not 61 — two more orders came due — and the age is **1,475 days**. The 67 live lines, the 12 over a year, ₹49,77,590.00 and ₹17,10,000.00 are stable. Anything measured against *today* carries the date it was taken; see defect 101.

This is not a migration artifact: the dates and statuses are as the shop recorded them. Either these pieces were delivered and never marked off, or they are genuinely outstanding. Both matter, and they matter differently — the first is untidy bookkeeping, the second is ₹17 lakh of work a customer may still be waiting for.

Needs: somebody to walk the twelve oldest and say which they are. The order book now shows them, oldest first, which it did not before.

Separately, **142 of the 284 orders name a customer ledger that is not in the address book**, so the order book shows a bare code where a name should be. The order is kept regardless: an order whose customer record has been lost is still work the shop promised.

29. Barcodes are reused — a tag number is not a piece — HIGH

Found by building the stock history (68) and looking at a barcode with ten movements and no tag on file.

The audit trail records a `NEW-BAR` event each time a barcode is issued. Across its 3,021 distinct barcodes:

	Barcodes
Issued once	1,047
Issued more than once	1,974
Most times one barcode was issued	11
Reused barcodes on the shelf today	1,663

A barcode does not identify a piece of jewellery across time. Two movements under the same number may be two different objects, and the majority of the shop's numbers are in that position.

This is probably ordinary practice — tags get reprinted and numbers recycled — rather than a fault. But it has a consequence that matters: **the stock history is the history of a number, not of a piece**, and anything that traces an item backwards through it can reach a confident wrong answer.

It also puts defect 8 in proportion. That recorded a single barcode appearing twice in the stock table at the same moment, worth ₹900. This is the same phenomenon across time and at a scale three orders larger.

The screen says so. Ask it for one barcode that was issued more than once and it tells you how many times, at the foot of the rows.

Needs: nothing urgent. But if the shop ever wants per-piece traceability, the tag number cannot be the key.

30. The old permission system was very nearly all-or-nothing — MEDIUM

Found by building the user rights report (70) over the legacy permission matrix.

81.4% of the 2,755 permission entries are "full control". The legacy system offered six levels and the shop used one:

Level	Entries	Share
full control	2,242	81.4%
entry only	446	16.2%
read only	31	1.1%
none	29	1.1%
without delete	6	0.2%
update / delete	1	0.0%

Seven of the 24 people had full control on every screen they could reach. The widest was 241 screens, all of them full.

Alongside that, from the user flags:

Able to	People
Post into a closed period	9 of 24
Change the gold rate	6
Edit a barcode	4

Back-dating and rate changes are the two powers that let somebody alter what the books say happened. Nine people held the first.

None of this is evidence that anything was done. It is the shape of the control environment, which is what an audit asks about, and it was permissive.

The new system starts differently — every screen already gates read, create, update and delete separately, and the sale, purchase and clearance screens each refuse to a read-only user by test.

Needs: nothing technical. Worth an eye when deciding who gets what now.

31. 1,114 secondary contacts point at a party that no longer exists — MEDIUM

`address_book_sub_name` holds 1,247 additional contacts — a second name, phone and email against a customer record, typically a spouse or a relative. **1,114 of them (89%) name a ledger code that exists nowhere:** not in the address book, not in the chart of accounts.

Sub-contact points at	Rows
A party in the address book	107
A ledger in the chart of accounts	32
Nothing	1,114

399 of the 1,247 carry a mobile number, so this is not empty data — it is detached data. Somebody's relative, reachable, with no record of whose relative they are.

Not migrated. Carrying 1,114 contacts whose parent is gone into a table that implies a relationship would assert a link that does not exist. The 139 that do resolve are not worth a table of their own.

Needs: nothing. Recorded because 399 reachable people is the kind of thing somebody might come looking for, and this says where they went.

32. Bank IFSC codes carry the branch name inside them — LOW

`ledger_bank_details` stores the IFSC and the branch in one column:

HDFC0000543 (AHURA CENTRE-ANDHERI EAST, MUMBAI-400093)

296 of the 389 populated values are a clean 11-character IFSC and the rest have the branch appended, up to 54 characters.

Fixed on the way in. The code is extracted and the branch kept beside it, which recovered **372 valid IFSCs where a plain copy would have kept 296** — and a naive 20-character column, which is what the width suggested, would have cut valid codes into invalid ones and lost every branch name.

33. The retail stock ledger counts metal four times on 219 tags — MEDIUM

`cla_retail_trans_bar_detail` is a per-tag stock transaction ledger — one row per tag at creation, carrying its own `amount`. On **219 tags it records exactly four times the metal value**, while counting the making charge once.

Barcode `89501233`, a 22 ct gold chain, is the pattern in miniature:

26.740 g at ₹9,300	2,48,682.00
Making at 12% of metal	29,841.84
<code>barcode_main.total</code>	2,78,523.84
<code>cla_retail_trans_bar_detail.amount</code>	10,24,569.84

$10,24,569.84 = 2,48,682.00 \times 4 + 29,841.84$. The multiplier is exactly 4.0000 on all 219, so this is a loop or a repeated accumulate in the legacy write path, not a rounding artefact or a data-entry slip.

The inflation is ₹1,82,88,182.61 on tags whose true value is ₹1,41,21,421.30 — those 219 are recorded at roughly 3.3× what they are worth. Across the whole ledger it overstates stock by ₹1,83,55,013.61 against a recorded ₹17,12,31,075.39, or **10.7%**.

`barcode_main` is the trustworthy side. Removing the inflation reconciles the two to the rupee:

<code>cla_retail_trans_bar_detail</code> total	17,12,31,075.39
less the inflation	-1,83,55,013.61
	15,28,76,061.78
<code>barcode_main</code> total	15,28,65,165.78

difference	10,896.00 — the single SALE-RET row
------------	-------------------------------------

What to do: never value stock from this table. Nothing in the rebuild reads it, and nothing should start — `barcode_main` reconciles against the pricing formula and against this ledger once corrected, which is two independent checks that the tag master is right and this one is wrong.

Note this does **not** move B19. The 241 tags carrying `stk_status = 0` are *sold* — 232 of them appear on a sales invoice — so they are correctly outside closing stock, and the ₹1.09 crore that B19 cannot explain is unaffected.

34. The monthly payroll sheet does not carry the components — MEDIUM

***Corrected 22 Aug 2026.** This entry first said payroll ran for 16 months **with no statutory deduction at all**, and advised against building PF and ESI on the grounds that nothing showed them in use. **That was wrong**, and the advice with it. The evidence was `staff_salary_sheet` and its detail table; the components live in a third table, `ledger_wise_payroll_detail`, which the first table sweep never saw because it had never been analysed. The original text follows, corrected.*

The shop does run statutory components. `ledger_wise_payroll_detail` holds them per staff member, and every one of its 46 rows carries a real amount:

Component	Staff	Total
D.A	12	84,200.00
HRA	12	33,600.00
PF	6	14,040.00
P.TAX	12	1,600.00
ESI	4	466.00
Total	12	1,33,906.00

They reach the ledgers too. `DEARNESS ALLOWANCE` carries ₹3,22,800 and `HOUSE RENT ALLOWANCE` ₹1,26,400 over the four months the vouchers cover, which is the right order against ₹84,200 and ₹33,600 a month.

So PF and ESI should be built. Not doing so would leave out something the shop configures for every member of staff.

`staff_salary_sheet` survived intact: **246 sheets, 19 staff, 16 months** from April 2025 to July 2026, **₹49,49,100 gross**. `staff_salary_sheet_payroll_detail` carries a further 209 rows.

What IS true: the monthly sheet ignores them

Every statutory and allowance field on the monthly sheet and its detail table is **zero**, while the per-staff configuration above is not:

Field	Rows	Non-zero
pf_amount , esi_amount , overtime_amount (header)	246	0
payroll_amount (detail)	209	0
pf_amt , esi_amt , ptax_amt , hra_amt , da_amt (detail)	209	0

They are in the sheets after all — as the gap

A further correction, found while building the screen. gross_salary minus paid_amount on the monthly sheet is **the deductions**, not arrears:

Month	Gross	Paid	Gap
Apr 2026	2,64,200.00	2,48,224.00	15,976.00
May 2026	2,85,300.00	2,69,194.00	16,106.00
Jun 2026	2,64,200.00	2,52,124.00	12,076.00

Against **₹16,106** of configured PF, ESI and professional tax. The gap runs about ₹14,500 a month across the history, and falls short of the configured figure because PF covers 6 of the 19 staff while the sheets cover 15 to 17.

Read the obvious way — as arrears — this says the shop owes its people **₹15,71,288**. It does not. The salary screen therefore labels the column **Deducted**, and marks the **96 sheets across 14 months that record no payment at all** separately, because those are a different thing and adding them to the same total produces a number that is wrong twice over.

July 2026 is one of them: ₹2,89,200 gross and nothing paid, the month the shop went dark.

So the components are set up per person and posted to ledgers, but the **monthly sheet does not record them line by line** — it carries a single basic_pay figure with nothing computed on top, and its 209-row detail table is filled with zeroes.

A rebuilt payroll therefore has to compute each month's components from the per-staff configuration, because the monthly history does not hold them. The 16 months of sheets give basic pay and nothing else.

Three of the six components point at ledgers that do not exist — P.TAX, I.TAX and D.A, referencing codes 2677, 2657 and 3373. That is blocker B14, and it has to be answered before the postings can be reproduced. A fourth oddity is worth raising at the same time: PF, ESI and HRA all point at ledger **3372, which is DIRECTORS REMUNERATION** — plausible for a director's own PF, odd for twelve people's.

What is posted to the ledgers is the salary itself, split across component accounts — EMPLOYEE BASIC PAY , DEARNESS ALLOWANCE , HOUSE RENT ALLOWANCE — which is where the empty hra_amt / da_amt columns were evidently meant to go.

Don't read a shortfall into this

At first look the sheets appear not to reconcile: ₹49,49,100 of gross payroll against roughly ₹11 lakh in the component ledgers. That comparison is invalid, and the reason matters well beyond payroll.

The accounts cover one financial year, not two. 1,847 of the 1,848 vouchers fall in FY 2026-2027 and run from **1 April 2026 to 5 December 2026**. The single earlier voucher is the misdated one from §7 above. There is no prior-year transaction history in this capture at all, so 12 of the 16 payroll months — April 2025 through March 2026 — have no vouchers to reconcile against.

Compare only the four months both sources cover and they agree almost exactly:

Payroll sheets, Apr–Jul 2026	11,02,900.00
Posted to BASIC PAY + DA + HRA	11,06,800.00
Difference	3,900.00

So payroll **does** reconcile to the ledgers. The apparent ₹38 lakh hole is the edge of the capture window, not money that went missing — and any figure derived from these vouchers is a figure about April to December 2026.

Also worth knowing before building a payroll screen: **7 of the 19 staff have no ledger in the chart of accounts**, 96 of the 246 sheets record no payment against them, and the component configuration covers **12 staff of the 19**.

35. Twenty-five undelivered orders are priced at rates up to five years old — HIGH

`order_rate_master` was never read by any loader. It holds, per order, the rate for **each of ten purities as at the day the order was taken** — 2,520 rows across 284 orders.

Its `date1` is misleading: every row is stamped 2026-04-01, the year-rollover date. The rates themselves are not. Grouping by the financial year in the order number shows what is really stored:

Order FY	Orders	22 ct rate on the order
2021-22	2	4,802.50
2022-23	5	5,077.60
2023-24	6	5,954.83
2024-25	14	7,371.21
2025-26	56	12,733.41
2026-27	201	14,194.90
Today's card		14,239.00

25 of those orders are still marked undelivered (`delivery_status = 0`): 2 from FY 2021-22, 5 from 2022-23, 5 from 2023-24 and 13 from 2024-25. Each carries the rate card of the year it was taken, and for the oldest that is **a third of today's gold rate**.

Whether the shop treats that rate as binding is a business question, not a data one — but it is worth asking before the order book is worked through, because fulfilling a 2021 order at a 2021 rate means buying metal at ₹14,239 to sell at ₹4,802. This is the same order book as §28, cut a different way: that entry counts repair work past its delivery date, this one counts customer orders whose *price* has aged.

What it costs. The rate each order was struck at is already on the order line, so the shortfall against today's metal is arithmetic. Across the live order book — undelivered work marked Process, Urgent or On Hold:

Order FY	Lines	Extra metal cost to honour
2022-23	1	.00
2023-24	2	6,21,530.00
2024-25	6	8,50,902.66
2025-26	23	25,85,206.15
2026-27	35	1,27,534.61
Total	67	42,85,173.42

Note the shape. The 35 lines taken this year cost ₹1,27,534.61 extra between them, because gold has barely moved since April. **The 31 older lines account for ₹40,57,638.81** — the exposure is almost entirely in work that has been waiting, which is the same work \$28 flags as overdue. An order ageing is not a tidiness problem; it is a priced one.

The order book now carries three columns for this — the rate on the order, the rate today, and what the difference costs on that line's weight — so the question can be asked of a screen instead of a query.

`amount_received` **cannot be used, and was rightly not loaded**

The same table's neighbour, `customer_order.amount_received`, looks like the customer money held against each order — 273 of 284 orders carry a figure, and they sum to **₹14,47,10,446**, of which ₹13,79,74,556 sits on undelivered orders.

It cannot be money received. The **entire order book is ₹2,68,52,217**, so the field exceeds what was ordered by more than five times. Against each order's own estimate the ratio is not constant either — 1.08, 1.27, 1.52, 1.71, 2.89, and on `C0/2627/00041` exactly **10.00**, which has the shape of a decimal-place fault rather than a payment.

`load_orders.sh` loads `advance` instead, which is zero on all 284 rows. That is the right call — the order book shows no customer money because **none can be recovered from this capture**, and a wrong figure there would be worse than a blank one. If customers have paid advances, that record is not in this data.

36. Eighteen tags carry a placeholder metal rate, not a real one — MEDIUM

Found while checking whether stock still on the shelf was tagged at older rates than stock that has sold. It was — but the tail turned out not to be ageing.

Thirteen tags were created on a single day, 2025-09-16, at a round ₹3,000 a gram for 18 ct, when 18 ct was nowhere near that. Three more, created 2026-04-10, carry exactly ₹1,000 a gram for 9 ct against today's ₹6,199. Round numbers entered in a batch are not prices; they are a placeholder somebody meant to come back to.

Tags	18
------	----

— of them on 2025-09-16 alone	13
Understated against today's rates	₹6,33,930.93

One of them is a different fault. Barcode 16237400, a GOLD CHAIN, carries a rate of **3.60** and a net weight of **3.600** — the weight was typed into the rate field. Its metal values at ₹12.96 instead of about ₹44,600.

One of the 18 is probably fine and is left in with its date showing: a 22 ct GOLD LOCKET at ₹3,000 created in **2016**, when 22 ct really was near that.

The near miss: 159 silver tags that look wrong and are not

The first version of the report behind this defect used "far below today's rate" as the finding, and it was **90% noise — 159 of 177 rows**. They were silver, tagged between November 2025 and January 2026 at ₹46 to ₹63 a gram against today's ₹253.50, which looks indefensible until the shop's own tags are read in order:

Silver rate on the tag	Tags	Tagged
60.00	141	Nov 2025 – Jan 2026
115.00	37	Nov 2025 – Jan 2026
75.00	36	Nov 2025 – Jan 2026
100.00	22	Oct 2024 – Jan 2026
260.00	11	May 2026

Silver genuinely moved roughly **four times in a few months**, and the tags trace it. Nothing is wrong with any of them.

The captured rate history only reaches back to 1 April 2026, so a tag made before that cannot be checked against the rate of its own day. Being far below today's rate is therefore not evidence of anything at all. The report now flags only what is wrong regardless of what the metal cost — a rate that is a whole multiple of a hundred, or a rate equal to the item's own net weight.

What this does and does not affect

Selling these is safe. The bill re-prices metal at the rate for the day of sale, not the tag's, so a piece with a placeholder rate still bills correctly — and the formula self-check passes it, because the check asks whether the formula reproduces the tag's own recorded price and it does.

Stock valuation is what suffers. These tags are on the books ₹6,33,930.93 below what their metal is worth, and that flows into every figure derived from book value. Screens 79 and 83 show it.

Needs: somebody to re-rate those 18 tags at what the metal actually cost on the day they were tagged.

37. One catalogue item was merged away in 2023 and never removed — LOW

item_merge is an archive: when two catalogue items are merged the loser is copied there in full, with a pointer to the winner, the date, the user and the IP. **40 merges, the latest on 11 August 2026** — ten days before the shutdown — so this is a feature the shop uses rather than one it forgot.

The merges were done properly. Every winner still exists, and **no stock tag, sale line or order line anywhere points at a merged-away item**. Whoever ran them repointed the references.

One exception: SILVER KOPPOR STAND (item 1139) was merged into SILVER DIPAK (item 215) on **20 September 2023** and **left in the catalogue**. It is inert — no tags, no sale lines, no order lines — but it is still offered in every item list, including the tag-creation screen, where somebody could pick it and start the duplicate again.

It is **not** B17. That is SILVER PICHKARY, a genuine double entry under one name. This one has a unique name, so a duplicate-name check does not see it — the only thing that reveals it is the merge archive.

Needs: delete item 1139, or confirm it is deliberately kept. Deleting a catalogue row is the shop's call, not the migration's.

38. fine_wt on old gold is not the fine weight — LOW

old_gold_purchase records what came back over the counter: 25 purchases, 534.540 g, ₹50,71,370.00 — tying to the PURC-OG vouchers **exactly**.

Its fine_wt column does not hold the fine weight. Of the 25 rows:

fine_wt = net × tunch, as it should be	3
fine_wt = net weight, with tunch below 100	22
Rows with tunch of 100 or more	0
Fine weight if derived	437.561 g
Fine weight as recorded	529.998 g
Overstatement	92.437 g

No money is affected, and that is the useful part of the finding. On all 25 rows $\text{total} = \text{rate} \times \text{NET weight}$ — the purity is carried in the **rate**, not applied to the weight. Dividing each rate by its own touch gives implied pure-metal rates of ₹13,786 to ₹15,051, which is where the 24 ct buy-back rate should sit. The shop's arithmetic is sound; fine_wt simply plays no part in it and was filled inconsistently.

What was done: mcj_old_gold_lines carries gross, net, tunch, rate and amount, and **deliberately has no fine_weight column**. Fine weight is derived where it is wanted, from net and touch. A column that disagrees with itself on 22 of 25 rows should not be stored as though it were a fact.

Anyone reading fine_wt as the metal content would overstate the metal taken in by **92.437 g**.

Correction — that is not ₹14 lakh. This entry originally priced the whole 92.437 g at the gold rate. **54% of it is silver**, and valuing silver as gold overstates the overstatement by more than double:

	Overstated	Rate today	Value
OLDGOLD (22 rows)	41.522 g	16,482.00	6,84,365.60
OLDSLV (2 rows)	50.305 g	253.50	12,752.32

	Overstated	Rate today	Value
No purity stated (1 row)	0.609 g	16,482.00	10,037.54
Correct total	92.437 g		7,07,155.46
<i>All 92.437 g valued as gold</i>			<i>15,23,546.63</i>

The silver line is the whole of the difference: 50.305 g of it is worth **₹12,752.32**, not the ₹8.29 lakh the gold rate would imply. The severity is unchanged — no money moves on `fine_wt` either way — but a figure quoted at roughly twice the truth is the thing this record exists to prevent. Found by [defect 163]'s sweep of columns whose values disagree with their names, which led back here.

How it happened: the original figure treated a weight as though weight alone determined value. It is the same mistake in miniature as [defect 135], where 21 carat-less rows of silver sold by weight were counted as stone.

39. `b2b_purchase` is an account split, not a registration flag — MEDIUM

`stock_inward` — the purchase invoices, 43 of them — carries a `b2b_purchase` flag, and on the sales side the matching flag means *the other party is GST-registered*. It does not mean that here.

It maps **exactly** to which purchase account the goods post to:

Flag	Invoices	Posts to	Ledger total
1	7	PURCHASE_B2B	39,61,244.85
0	36	PURCHASE	1,94,74,168.96

Not one exception either way, and both tie to the ledgers.

But 17 of the 36 unflagged invoices carry GST, averaging 1.26%. A supplier who charges GST is registered. So those 17 are B2B in the sense a return uses and B2C in the sense the shop's chart of accounts uses, and the two senses are not the same thing.

Reading the flag as a registration status would put **₹4,02,980.57 of input tax** on the wrong part of a GSTR-2 — understating what can be set off.

What was done: the inward supplies report offers the filter in the shop's own terms — *Posted to PURCHASE_B2B* and *Posted to PURCHASE* — and names the account on every row, rather than asserting a registration status the data does not support. Which invoices belong in which part of a filed return is a question for whoever files it; the account split is evidence towards the answer, not the answer.

Needs: the accountant to say which of the 43 are registered-supplier invoices. The GST charged on each is in the report, which is the evidence they would use.

What the audit log says actually happened

`barcode_wise_all_transaction` (8,561 rows) spans **2024-04-01 to 2026-08-19** — about 2.5 years — while every transactional table holds only FY 2026-27.

This corrects a claim in IMPLEMENTATION_PLAN.md §1, which said there is no multi-year history to migrate. There is: 2.5 years of barcode audit trail survived, while the vouchers behind it were purged.

Workflow	Movements in 2.5 years	First → last
NEW-BAR — tagging	6,395	2024-04-03 → 2026-08-18
OP-STK — opening stock	1,770	2024-04-01 (one day)
SALE-GST — sales	324	2024-05-09 → 2026-08-19
CST-ORD — customer orders	44	2024-06-12 → 2026-07-26
SALE-RET — sale returns	17	2024-08-18 → 2026-06-16
ISS-HM / REC-HM — hallmarking	4 / 4	2024-10-01 → 2025-03-01
EST — estimates	2	2025-02-02
PURC-IN — purchase inward	1	2026-07-04

This bears on decision #2 (manufacturing pulled into scope). Hallmarking did run — 8 movements — so the empty hallmark tables reflect purged history, not an unused feature. But across 2.5 years of complete audit trail, hallmarking appears **8 times**, and karigar issue/receive leaves **no trace at all**.

That does not contradict the decision — the business knows its own plans, and the subsystem may be manual today or about to start. It does mean **Tier F is building for a workflow with no observed volume**, which is worth confirming before the ~10–14 weeks are spent.

40. `item_adjustment` is not a stock adjustment — and it contradicts itself — LOW

The name says stock adjustment. It is a **per-item making override keyed by rate code** — `per_of_making`, `rt_mk_pr_gm`, `pl_minus_gold` — and it holds no quantities at all. The seventh name in this dump that points at the wrong thing.

Of its 922 rows:

	Rows
Total	922
Distinct items	460
Referring to an item that does not exist	152
Carrying any non-zero value	5

So 917 rows are zeros, one in six points at nothing, and every row carries the same rate code, `R001`.

The five live rows do not agree with each other. Item 1066, GOLD ORNAMENTS, appears three times:

Making %	Rate/gm	Plus-minus
8	250	10
15	—	—
8	250	10

The remaining two point at items 657 and 735, neither of which is in `item_master`.

What was done: nothing loaded. A making-override table that disagrees with itself on its only real item cannot be the source of a price. Making comes from the tag, as it does everywhere else in this rebuild. Recorded so the table is not later mistaken for the stock adjustment screen its name suggests.

41. The address book and the ledgers both key on `ldgr_code`, and it means different things — BLOCKER

The single worst defect found so far, because nothing about it looked wrong.

Two legacy tables hold people:

Table	Rows	What it is
<code>ledger_master</code>	785	the chart of accounts — 772 of them real parties
<code>address_book_master</code>	40,519	the marketing contact list

Both name their key column `ldgr_code`. **The two code spaces are unrelated.**

Code	In the ledgers	In the address book
1011	PURCHASE	ARINA INTERNATIONAL
1014	SALE_B2B	ASIAN PHOTO STUDIO
1016	SALE	ASIT BRACELETS
3346	REPAIR AND MAINTENANCE COMPUTER	BIJOY NAHA

368 code values appear in both tables. **Not one of them names the same person.** The same customer exists in both under different codes and is duplicated in the address book:

Person	Ledger code	Address-book codes
RASHMI SARAF	12480	52025, 91666
POOJA PRASAD	12877	38624, 92087
P. K. MUNDRA	4781	41953

What it broke

Every loader and the customer picker joined the two on that column.

The picker on the sale, credit note and old gold screens offered 368 customers and each one posted to a stranger's account. Choosing BIJOY NAHA debited REPAIR AND MAINTENANCE COMPUTER. Choosing ASIT BRACELETS debited the SALE control account — a bill that credits and debits the same nominal.

547 stored rows were attached to the wrong person:

Table	Wrongly linked	Of
mcj_bank_accounts	193	398
mcj_customer_orders	142	284
mcj_sales_invoices	107	224
mcj_scheme_enrolments	105	239

The order register displayed those names. `mcj_sales_invoices` also stores `party_name` captured at the time, and it disagreed with the link on **all 107** — which is what made this visible at all.

Why the tests did not catch it

Every seed created a party **and** a ledger with the same code *and the same name*, so the join worked in the test and only ever failed on real data. The due dates test seeded a customer name that no assertion read.

What was done

- Migration `mcj-1787384000` adds `ledger_id` to the three tables that lacked it, backfills from the ledger code each row already carried, and **drops** `party_id` **from all four** so the wrong join is not available to write again.
- All four loaders now join `mcj_ledgers`.
- `billableParties` reads ledgers in the Customer, Dealer and Other Debtor groups — 567 accounts a bill can legitimately be raised against — and `partyLedger` no longer joins anything.
- `taxBasisFor` takes the state from the ledger.
- Tests: `TestSale_DoesNotBillTheAddressBook` seeds a decoy contact on the SALE ledger's code and fails if the picker offers it; `TestSale_TakesTheStateFromTheLedgerNotTheAddressBook` and `TestDueDates_NamesTheCustomerFromTheLedger` do the same for tax and for the reminder list.

Verified after reload: 284 of 284 orders, 239 of 239 enrolments and 224 of 224 invoices name exactly what `ledger_master` names. The single remaining difference is `DSTECHNOLOGY` against `DS TECHNOLOGY` — defect 17 whitespace, not a wrong link.

`mcj_parties` stays as what it is: a 40,519-name contact list, used by the customer data-quality report and nothing that posts money.

42. One voucher is dated three and a half months after the capture — LOW

PMNT 576 , dated **5 December 2026**, in a dump taken on 21 August 2026.

It is a post-dated cheque and the narration says so: *Ch. No.435590, Dt.05.12.2026 Issued Against Their Dues*. The voucher is dated to the cheque rather than to the day it was written, which is a legitimate way to record one — it is the only such voucher in 1,824.

Amount	₹1,35,370
Credit	UNION BANK OF INDIA
Debit	DAYANIDHI TRADE PVT LTD (Other Loan)

Why it is worth writing down: any balance computed without a date filter already has that ₹1,35,370 out of the bank. A cash book or bank book asked for a range ending today excludes it and disagrees with the raw ledger total by exactly that amount. Neither is wrong; they answer different questions.

It touches no customer, so the customer-facing registers are unaffected.

What was done: nothing changed. The vouchers that carry date ranges already handle it correctly. Recorded so the discrepancy is recognised rather than investigated from scratch.

43. Two tags on the shelf carry metal and are priced at nothing — MEDIUM

The shelf is otherwise sound. Across **2,816 tags in stock** there is not one negative weight, piece count or value, and not one tag whose net weight exceeds its gross. The legacy screen for this is called *Negative Stock* and against this data it would report nothing at all.

Two tags would bill as free:

Tag	Item	Purity	Net wt	Priced
43264560	SILVER BABY BANGLE	SLV	14.990 g	a fixed amount — of zero
46318302	GOLD LOCKET	22	1.350 g	percentage of the metal — of zero

Roughly **₹13,000 of metal** at the rates in the capture, depending on the day. Scanned onto a bill, each adds nothing to the total.

Two more are priced per gram of net weight and carry none, so they can only ever come to zero — 42210478 and 96904004, both **C GOODS**. Those are customer-goods placeholders rather than stock the shop owns, which is why they are named separately.

What is deliberately NOT a defect

359 tags on the shelf have no weight because they carry a printed price instead — gold rakhis at ₹2,000 among them. Flat pricing is how this shop sells imitation, silver articles and seasonal lines, and reading those as unweighed stock would bury the two that matter under 359 that are correct. A further 8 are priced per piece, which also needs no weight.

What was done: the screen reports the faults and names the flat-priced stock it is deliberately leaving alone. Repricing the two tags is a decision about what the pieces are worth, which is the shop's to make.

44. The year-end carry-forward was run on 14 August, not at the year end — LOW

ldgr_open_balance_final holds 186 ledger balances stamped **2027-2028** — the shop's own assertion about where it finished the year that was running when it went dark. It is the only forward-looking record in the capture.

It was not produced on 31 March. Reconciling each carried figure against opening + movement and sliding the cutoff date, agreement peaks sharply:

Movement counted to	Ledgers reconciling exactly
every voucher in the dump	164 of 186
13 Aug 2026	174
14 Aug 2026	176
15 Aug 2026	173

So it is a snapshot taken **a week before the shop closed**, and any ledger touched after 14 August will differ from it. That is not an error in the carry-forward; it is what the carry-forward is.

The residue at that cutoff is 10 ledgers and under ₹6,000 in total, the largest gap being ₹4,655.56. INPUT CGST and INPUT SGST differ by the identical ₹1,096.18, which is what a consistent tax pair looks like.

One of the differences at the full-dump cutoff is **exactly defect 42**: the post-dated cheque of ₹1,35,370 to DAYANIDHI TRADE PVT LTD, dated 5 December 2026, which the 14 August snapshot could not have known about.

It does not balance, and that is B19 rather than a fault

Carried forward — debits	10,62,76,061.42
Carried forward — credits	27,06,82,293.35
Out by	16,44,06,231.93
The 2026-27 openings, out by	15,32,28,603.49 (same direction)
The vouchers between them, out by	0.01 (defect 3)

An imbalance present at both ends and absent from the movement between them is a missing **asset** side, which is exactly what blocker B19 says: the stock was never in the ledgers. The shelf is worth ₹14,23,61,188.20 — the same order of magnitude as the gap, though not equal to it.

What was done: loaded as recorded and reconciled, not corrected. A carry-forward is the shop's own assertion about its position, and **176 of 186 of those assertions are confirmed by its own vouchers**.

45. Ten of the eighteen code-list loaders reported success without loading — HIGH

Not a defect in the legacy data. A defect in the rebuild's own loader, found by adding a nineteenth list to it.

`load_codelists.sh` reloads each reference list as `DELETE` then `INSERT` inside a transaction, and then checks its work by comparing the source row count against the destination row count.

Ten of the lists are referenced by other tables — colours by packets, item groups by items, shapes by stones — so once those tables are populated the `DELETE` violates a foreign key, **the transaction rolls back, and the old rows stay exactly where they were.** The counts then match, because nothing changed, and the loader printed:

OK	mcj_colors	30 rows
----	------------	---------

having done nothing at all. The check could not fail: it compares the count against itself.

Affected: `mcj_colors`, `mcj_cuts`, `mcj_sieves`, `mcj_stone_purities`, `mcj_card_types`, `mcj_ledger_types`, `mcj_item_subgroups`, `mcj_item_groups`, `mcj_shapes` and `mcj_categories` — 10 of 18.

The loaded data is not wrong. `load_all.sh` truncates everything first, so a full run empties the referencing tables before the code lists are rebuilt and every list loads correctly. What was broken is reloading a single list afterwards, which is exactly what somebody does when a reference list needs correcting — and it would have reported success.

What was done: the loader now checks the exit status of the write itself rather than inferring it from row counts, and says so:

FAIL	mcj_colors	not reloaded – something references it; the old rows are untouched
------	------------	--

A row-count check that compares a number against itself is worth looking for elsewhere in these scripts.

46. Reloading the masters alone would have cut 13,282 rows loose, and reported success — BLOCKER

Found by auditing the loaders after defect 45. Not triggered — the guard below was written before anything ran.

`load_masters.sh` rebuilds `mcj_items` with `DELETE` then `INSERT`. Ten columns across eight tables reference `mcj_items`, and every one of them is `ON DELETE SET NULL`:

Referencing table	Column
<code>mcj_stock_items</code>	<code>item_id</code>
<code>mcj_sales_invoice_lines</code>	<code>item_id</code>
<code>mcj_customer_order_lines</code>	<code>item_id</code>
<code>mcj_stock_conversion_pieces</code>	<code>item_id</code>
<code>mcj_customer_goods</code>	<code>item_id</code>
<code>mcj_designs</code>	<code>item_id</code>

Referencing table	Column
mcj_stock_stones	stone_item_id
mcj_design_stones	stone_item_id
mcj_barcode_edits	old_item_id , new_item_id

So the delete does not fail. It **succeeds**, silently blanking the item on **13,282 rows** — every tag on the shelf, every invoice line, every order line, every conversion piece and every stone. The items are then reinserted with fresh uuids and **nothing relinks them**. The check that follows counts 322 items against 322 and reports OK, because the item table itself is perfectly fine; it is everything pointing at it that has been cut loose.

This is worse than defect 45. That one did nothing and said it worked; this one destroys the links and says it worked.

`load_all.sh` is safe: it empties the referencing tables before the masters are rebuilt, so nothing is pointing at `mcj_items` when the delete runs. The hazard is the standalone run — which is what somebody does when one item name needs correcting.

What was done: `load_masters.sh` now counts what points at `mcj_items` and refuses:

```
REFUSED: 13282 rows currently point at mcj_items.
...
Run ./load_all.sh, which empties the referencing tables first.
Set MCJ_FORCE_MASTERS=1 only if you mean to cut those links.
```

The rest of the loaders were checked

`mcj_ledgers` has the same six `SET NULL` references — including the four repaired in defect 41 — but `mcj_voucher_lines` references it `RESTRICT`, so the delete fails, `set -e` aborts, and `load_ledgers.sh` exits 3 with the data intact. It fails loudly, which is correct.

Every other loader is a single write guarded by `set -e` and aborts the same way. `load_codelists.sh` was the exception because its writes sat inside a loop (defect 45); `load_stock.sh` 's loop only verifies, comparing legacy sums against loaded sums, which is a real cross-check rather than a number against itself.

47. The making ladder records five different rates for the same weight band — HIGH

`item_wise_sale_rate_master` — the Item Wise Wstg and Rate master — encodes a real and sensible ladder. A small piece costs proportionally more to make than a heavy one:

GOLD ORNAMENTS	Making
under 1 g	26%
1 – 2 g	22%
2 – 3 g	20%
3 – 6 g	18%

GOLD ORNAMENTS	Making
over 6 g	14%

It cannot price anything, because it does not say which row wins.

95 rows describe only **3 items** and 27 distinct bands. The same band carries up to five different rates:

Item	Band	Rates recorded
GOLD ORNAMENTS	1 – 2 g	15 / 18 / 20 / 22 / 26
GOLD ORNAMENTS	2 – 3 g	12 / 15 / 18 / 20 / 22
GOLD ORNAMENTS	3 – 6 g	10 / 12 / 15 / 18
EXTRA ORNAMENTS	1 – 2 g	18 / 20 / 22

There is no date, no branch and no status column to choose between them. The table has eight columns and none of them is a discriminator.

Taking the highest legacy id — the only thing that could mean "most recently set" — produces the ladder above. It is coherent for the light bands and **still ambiguous over 6 g**, where three overlapping bands give three answers:

Band	Making
6 – 100 g	12%
6 – 250 g	15%
6 – 300 g	14%

14 of the 27 bands overlap another band for the same item.

What is clean

Wastage. All **91 retail (ID=SALE) rows carry zero wastage**; all **4 trade (B2B=SALE) rows carry 7–10%**. No exceptions either way. The shop charges wastage to the trade and not at the counter.

What was done: every row loaded, duplicates included — collapsing them would hide the ambiguity. The screen shows the ladder, marks superseded rows, and names each band that carries more than one rate or crosses another. **It does not price.** Making comes from the tag, as everywhere else in this rebuild.

Ask: what is the making percentage for a gold ornament over 6 g? Three answers are recorded and nothing in the data chooses.

48. Ornament purchase components do not add up to the bill — MEDIUM

Two tables that had never been read hold what each purchase actually contained: `stock_inward_main` (21 rows, the metal) and `stock_inward_detail` (39 rows, the stones). Of the 43 purchase invoices, **11 bought metal and stones, 9 bought metal only, and 23 bought stones only.**

The invoice's taxable value equals `stock_inward_main.value` **exactly** on every bill that has a metal row. But the components of that value do not reproduce it:

Bills with a metal row	20
Where <code>net × rate + stones + making</code> equals the bill	5
Where it does not	15

Across those 20 bills the bill value is ₹1,35,68,049.00 while net weight × rate comes to ₹58,61,360.85, the stones on those same bills to ₹28,94,996.21 and making to ₹7,50,160.70 — leaving **₹40,61,531.24 unaccounted for**.

Something in the composition is not recorded. Candidates: the `loss` column on the detail rows, an agreed adjustment, or a rate applied to a weight that is not the net one. Nothing in the capture settles it.

What is certain

521.329 carats of loose stones worth ₹1,27,31,698.71 were bought across 34 bills, and no screen has ever shown it. The purchase register carries one taxable figure per bill and no breakdown at all.

What was done: both tables loaded, every row tied to its invoice — the loader fails if any content row cannot reach one. The register shows the components and the bill side by side and states that they do not agree. **Neither figure is adjusted to fit the other:** the bill is the authority on what was paid, the components on what arrived.

49. Two customers have their history split across duplicate ledgers — HIGH

Five names appear twice in the 785-ledger chart of accounts. Three are the same name on consecutive codes — 12777/12778, 12830/12831, 12874/12875 — so somebody created the account, failed to find it, and created it again. Two differ only by a trailing full stop, which is why a literal name comparison finds three of the five and not all of them.

Three are harmless. **Two are not, because both halves carry history:**

MADHUMITA BISWAS — a savings scheme in two accounts

Ledger	Scheme	Instalments paid	Paid
7042	Gold Plus	10 of 10	10,000.00
8091	Gold Plus	10 of 10	10,000.00
8091	Gold Plus	7 of 10	7,000.00

₹27,000 of one customer's contributions across two accounts, one scheme still running. Whoever serves her sees one account and not the other.

ASHOK AGARWAL — orders under both

One order under 10990 and two under 8129.

The other three

PARMESHWAR SINGH has two voucher lines and a bill under 12777 and nothing under 12778. SARKAR ORNAMENTS and SUNITA SHARMA have nothing on either side. All three merge without moving anything.

What was done: the Duplicate Ledgers screen lists all ten accounts, spells out what points at each — *1 bill, 2 scheme enrolments* — and gives a verdict that depends on the other half of the pair. **It merges nothing.** Moving a person's financial history is a decision with their savings attached, and a test fails if the screen ever writes.

50. The hallmark flag is set by hand and 739 gold pieces carry none — MEDIUM

739 of the 1,023 gold jewellery tags on the shelf carry no hallmark record, and they are worth ₹11,54,63,861.86.

Purity	On the shelf	No hallmark recorded	Value
18kt	505	411	5,95,69,420.89
22kt	319	217	4,20,34,082.23
14kt	165	104	1,37,93,189.74
9kt	34	7	67,169.00

(The "On the shelf" column originally read 539 / 352 / 196 / 38, which summed to 1,125 against the 1,023 in the sentence above it and matched neither the tags in stock nor all tags ever made for those purities. Corrected from the data in defect 342; every other figure here reproduced to the paisa.)

Bullion is excluded: 112 of the 114 pure-gold (999) tags carry no hallmark and that is normal, so counting them would inflate the figure and weaken the real one.

The number needs its caveat, and there are two readings

Either the pieces were never hallmarked, or they were and the system was never told. **The second is the more likely**, and the movement trail is why:

Tags flagged as hallmarked	432
Hallmark movements in 2.5 years	12 (6 issues, 6 receipts)
Hallmark charge collected across all 432	₹2,915.00

Correction. This first read 8 movements, from `barcode_wise_all_transaction` alone. `retail_all_stock` — opened later — holds **4 more, none of them overlapping** (see defect 57). The real figure is 12. It does not change the conclusion: a flag set 432 times by a process that recorded six issues is still a flag being set by hand. Its absence therefore records very little, and a screen that presented ₹11.5 crore as unhallmarked stock would be asserting something the data does not support.

What was done: the report shows the figure with the movement count and states in terms that it is **about the record, not about the pieces**. Whether the shelf is actually hallmarked is a question for somebody holding a loupe. A test fails if the caveat is separated from the number.

51. ₹8.8 crore of supplier accounts sit in debit, and ₹2 crore of it never moved — HIGH

A supplier account in debit means the shop has paid ahead, or goods are owed to it. Across the Dealer, Other Creditor and Sundry Creditors groups:

Accounts the shop owes	45, totalling 6,15,02,878.56
Accounts in debit — the shop is owed	17, totalling 8,80,84,874.22
Of those, with no posting at all this year	10, totalling 2,01,01,051.00

Two things about the ₹8.8 crore matter more than its size.

One account is 77% of it. SHREE SAROJ JEWELSS opened the year at ₹6,98,45,346.00 and closed at ₹6,79,37,571.00 across 13 postings.

₹2,01,01,051.00 is opening balances that were carried in and never touched — PALSANI JEWELLERS PVT LTD at ₹99,86,295.00, VASU DEV JEWELLERS at ₹64,69,284.00, MEHANDI DIAMONDS PVT LTD at ₹12,63,478.00 and seven more, every one with zero voucher lines in the whole capture.

For a shop whose shelf is worth ₹14.24 crore, ₹8.8 crore owed *by* its suppliers is a large enough position to need an explanation, and nothing in the capture gives one.

A method note, because I got this wrong first

Summing only the year's postings on this group reports the **opposite** — dealers owing the shop ₹12,06,180 — because the opening balances are most of the answer and the movement is a rounding error against them. **A balance is the opening plus the movement.** The screen shows all three columns and a test fails if the opening is ever dropped from the sum.

What was done: the Supplier Balances screen shows every account with a balance, which way it runs, and how many postings it saw. Nothing is adjusted. Why ₹8.8 crore is owed to the shop is a question for the shop.

52. The monthly sales average falls 52%; the typical bill does not — MEDIUM

Not a defect in the data. A defect in the obvious reading of it, recorded because the obvious reading is what a monthly sales report normally shows.

Across the four full months in the capture the **average bill** falls steeply:

Month	Bills	Average bill	Typical (median) bill	Bills over ₹5 lakh
Apr 2026	47	1,98,695.91	30,863.00	3
May 2026	44	1,23,284.75	14,400.00	2
Jun 2026	55	70,940.42	54,709.00	0
Jul 2026	54	94,712.63	45,759.00	2

The average falls 52% from April to July and reads as customers spending less every month. **The median does the opposite** — it rises from ₹30,863 to ₹45,759 — and the bill count barely moves at 47, 44, 55, 54.

The averages are carried by a handful of large bills: three over ₹5 lakh in April, the largest **₹35,20,906.00**, two in May, and **none at all in June**. Remove those and the months look alike.

The typical customer's bill did not fall. A report showing only the average would have described a collapse that did not happen.

What was done: the Sales by Month screen shows the average, the median and the count of large bills on every row, and its summary says to read the typical bill rather than the average. August is marked partial — the last bill is dated 19 August 2026 — so it is not compared against full months.

53. `barcode_create_from_purchase_detail` records which tags came from a purchase, but not which purchase — LOW

The last plausible source for `purchase_during_receive.php`, and it cannot support that screen.

21 rows, 21 distinct barcodes, **2 distinct items**. Every measurement column is empty on 20 of the 21 — quantity, gross weight, carats, both rates, amount, making and total are all zero — because the real figures live on the tag the row points at, not on the row.

What it does carry is real:

Item	Tags	On the shelf	Value
STONE	20	20	31,07,115.60
DIAMOND	1	1	13,41,446.10

All 21 barcodes resolve to tags on file and all 21 are still in stock, worth ₹44,48,561.70 between them.

There is no bill number, invoice id or supplier on the table. It records that 21 tags were created from a purchase and not which one, so it cannot be joined to the 43 purchase invoices and cannot answer what a purchase register asks.

What was done: nothing loaded, and the screen is not built. Recording the contents so the table is not re-investigated, and so that `purchase_during_receive.php` is understood as unbuildable from this capture rather than merely unfinished.

54. The item master's coin rates are frozen about five years behind the daily list — HIGH

`coin_rate_master` — 5,966 rows, the largest table in the dump nobody had opened — is the shop's **daily price list for coins**: 52 items across 115 trading days, 3 April to 19 August 2026, every day the shop traded.

It explains a figure that looks wrong everywhere else. **₹26,11,606.00 of coins were billed, 17% of all sales, against ₹1,70,345.00 of tagged coin stock.** Coins are priced and sold as products from this list, not by weight off the shelf.

The defect

All eleven coins that carry a rate in `item_master` are priced far below their current daily rate, and by almost exactly the same factor:

Coin	Item master	Latest daily (19 Aug 2026)	Times
COIN-1 GM	3,100.00	15,200.00	4.9
SILVER COIN GANESH LAXMI	520.00	2,500.00	4.8
SILVER COIN-5 GM	260.00	1,240.00	4.8
SILVER COIN-20 GM	1,040.00	4,940.00	4.8
SILVER COIN-10 GM	520.00	2,470.00	4.8
SILVER COIN-100 GM	5,200.00	24,680.00	4.7
SILVER COIN-50 GM	2,600.00	12,340.00	4.7
SILVER COIN-25 GM	1,300.00	6,170.00	4.7
SILVER COIN 250 GM	13,000.00	61,700.00	4.7
SILVER COIN 500 GM	26,000.00	1,23,400.00	4.7
SILVER COIN OUNCE	2,500.00	7,750.00	3.1

A consistent 4.7–4.9× is not eleven mistakes. It is one price list that stopped being updated.

The silver coins imply about ₹52 a gram, which is roughly where silver sat several years ago.

Anything pricing a coin from the item master undercharges by about 79%. A coin the daily list puts at ₹1,23,400.00 carries ₹26,000.00 in the master.

Also in the table

19 of the 52 items are never priced on any of the 115 days — the yantras, the Raja Rani sets, SILVER MOHOR, SILVER NOTE, OUNCE, GOLD ORNAMENTS and several unqualified names like COIN and SILVER COIN. Those are the 2,171 zero rows: not missing prices, but items nobody prices.

What was done: the list is loaded and the Coin Rates screen shows the latest rate, the range over the period and the item master's figure beside it, flagging every coin where the two disagree by more than half. Nothing is corrected — which rate is right is the shop's to say, though the daily list is plainly the live one.

55. The design library is 5,221 pieces from 2016-17 and 69 of them were ever used — MEDIUM

`design_no_status` — 5,242 rows, the second-largest unopened table — turned out to be a status flag over the design library, and opening it made the library itself worth looking at.

Designs on file	5,221
Total estimated value	14,95,13,508.73
Created between	5 July 2016 and 20 January 2017
Designers	3
Designs that ever appear on a tag	69

The whole library was entered in a six-month window nine years ago and 1.3% of it has ever been made. Nothing has been added since January 2017. It is a catalogue the shop stopped using rather than one it maintains.

The status flag

5,066 rows carry status 0 and **176 carry status 1**. The flag does not track anything obvious — the two groups are indistinguishable on pictures (98% either way), estimates and tagging. What separates them is their origin: **all 176 were designed on a single day, 5 July 2016, by one of the three designers**, and they are heavier and dearer than the rest — 13.636 g against 8.591 g, ₹49,287 against ₹28,619.

So it marks one batch entered on one day. What the mark MEANS — selected, approved, discontinued — is not recorded anywhere in the capture.

A smaller thing found alongside

109 tags carry a design code, referencing 93 distinct codes, and **24 of those codes are not in the design library**. Nineteen of the 24 are placeholders — 0, . — on 34 tags. Only **5 tags** name something that looks like a real design code and has no design on file.

What was done: the status flag is not loaded. It distinguishes one day's batch from the rest and nothing in the capture says why that matters; carrying it forward would give it an authority it has not earned. The library itself is already loaded and reachable through the design screens.

56. Three quarters of the shelf is photographed only on someone else's server — BLOCKER (B24)

`barcode_image` — 3,965 rows, the third-largest unopened table — is the tag photography, and it makes blocker B24 a much larger problem than it looked.

Photograph records	3,965
Tags photographed	1,023

Of those, tags still on the shelf	889
Shelf value of those 889 tags	10,73,39,445.03
Shelf value in total	14,23,61,188.20
Share of the shelf that is photographed	75%

The paths point at **two** hosts, neither of which the shop controls:

Host	Photos	Tags
http://tag09.com/mcj2/...	3,923	1,012
https://testworkz.in/mtest/...	42	11

```
http://tag09.com/mcj2/images/bar_pic/23635013_30683902_2.JPG
https://testworkz.in/mtest/images/bar_pic/33537049_10060840_0.JPG
```

3,923 of the 3,965 URLs are plain HTTP. The 42 secure ones are the `testworkz.in` set — and that host's path segment is `mtest`, which reads like a test deployment. Eleven tags, all still on the shelf, have their only photographs on what appears to be somebody's staging server.

Photographs average 3.9 per tag; one tag carries 93. Nine paths are recorded twice. 96 of the 1,023 photographed barcodes have no tag on file at all.

The shop has no copy of any of it. Three quarters of its stock by value is photographed nowhere else, on a third party's server, over an unencrypted connection, and the ERP those images belonged to has already gone offline once.

What was done: nothing, deliberately. `tag09.com` serves a bot check, and working around a third party's access control is not something this rebuild will do — retrieving the shop's own pictures is reasonable, defeating somebody else's protection to get them is not. The ask stands and is now quantified: **open one of these URLs in a browser today** and confirm whether the images are still there. If they are, they need copying while that is still possible.

57. The stock trail is two tables, and one of them was never read — HIGH

`retail_all_stock` — 2,820 rows, the fourth-largest unopened table — is a **second stock movement log** covering the same three accounting years as the one already loaded. Every one of its rows resolves to a known tag.

475 of them are events the first log does not have:

Rows	What	Weight
278	the FY 2025-26 opening, all dated 1 April 2025	358.490 g
192	tag creations, 28 Jul 2024 to 12 Aug 2026	963.237 g
5	hallmark and sale-return events	

The 192 creations are the serious ones. 161 of those tags are still on the shelf and worth ₹57,41,225.14, and the log that exists to record every tag being made does not record them being made.

The two logs are not equally rich and neither is a superset:

	barcode_wise_all_transaction	retail_all_stock
Rows	8,561	2,820
Carries	the user and the IP	the weight and purity at the time
Records the 2025-26 opening	no	yes

It corrects an earlier finding. Data defect 50 reported **8** hallmark movements in two and a half years, counted from the first log alone. The second holds **4 more, none overlapping:** the real figure is **12**. The conclusion there is unchanged — 432 tags flagged by hand against six issues — but the number was wrong and is now right in both the defect and the screen.

What was done: the movement loader now takes both, matching on barcode and day so nothing is counted twice, and stamps each row with the table it came from. The trail is **9,036 rows**, up from 8,561. The Hallmark Records screen reads its count from the trail rather than carrying a literal, so it followed the data on its own.

58. 266 tags are worth less now than the creation log says, and nothing explains it — MEDIUM

`cla_retail_trans_bar_detail` — 2,816 rows, the fourth-largest unopened table — is a **value view of the stock trail**. It carries no event the trail does not already have: all 2,816 rows match one on barcode and day. So it is not loaded.

What it does carry is the value at the moment each tag was created, and on **281 rows that figure disagrees with the tag's price today**. 266 of those tags are still on the shelf.

Rows where creation value ≠ current total	281
Of those, still on the shelf	266
Difference across them	1,82,39,030.71
Direction	every one downward — not a single tag rose
Ratio, creation ÷ current	1.00 to 3.81, median 3.542
Net weight changed on	none of the 266

What it is not

Four explanations were tested and all fail:

- **Not stones.** 214 of the 266 tags carry no stone at all, and the average gap (₹68,567.78) is three times the average stone value (₹21,272.59).
- **Not a weight change.** The net weight is identical on all 266.

- **Not a constant factor.** The ratio spans 1.00 to 3.81. An earlier reading of "about 5×" was a **ratio of averages** — the same artefact that produced the bogus "578% below today's rate" recorded elsewhere in this file. Per tag it does not cluster.
- **Not the market rate of the day.** The creation values imply about ₹22,000 a gram across mixed purities, which for the 222 eighteen-carat tags would put pure gold near ₹29,300 a gram — far above any rate in this data. The rate history does not cover those dates either, so it cannot be checked directly.

What can be said

Nothing in the capture explains it, and the figure must not be read as ₹1.82 crore lost. Two records of the same tags disagree, consistently in one direction, with the weight untouched. Which is right is a question for the shop.

A related fact that may or may not bear on it: **154 of the 266 carry a rate of exactly ₹4,000.00 a gram at 18kt**, and others sit on round numbers — ₹3,300, ₹3,000, ₹7,500. Round rates on that many tags look like placeholders, which is what data defect 36 found on a smaller set.

What was done: the table is not loaded, since it adds no events. Recorded as an open question with the tests that were run against it, so the next person does not repeat them.

59. The order book locks a rate per order, and honouring it costs ₹60,14,200.75 — HIGH

`order_rate_master` — 2,520 rows, the last of the large unopened tables — answers the question data defect 35 asks.

Every one of the 284 customer orders has a rate locked against it, per purity. 2,520 rows is 284 orders across nine or ten metal grades. The system was built to honour the rate an order was taken at, and it recorded one every time.

What honouring the live book costs

152 live order lines carry a weight; **150 of them match a locked rate** and a current one.

Purity	Lines	Net weight	Locked rate (avg)	Rate on 21 Aug	Difference
22kt	53	933.040 g	12,814	15,153	38,50,685.32
18kt	44	458.520 g	10,894	12,398	8,38,805.52
Silver	31	13,024.210 g	227	254	8,40,163.48
995	10	209.000 g	13,139	15,482	4,23,099.00
14kt	10	28.570 g	8,280	9,643	59,493.43
9kt	2	6.000 g	5,916	6,199	1,954.00
	150				60,14,200.75

Metal at the locked rates	2,06,58,955.08
Metal at the rates of 21 August 2026	2,66,73,155.83
Cost of honouring the book	60,14,200.75

22kt alone is ₹38.5 lakh of it, on 933 grams. Silver looks large by weight — 13,024 g, 88% of the live book — and contributes ₹8.4 lakh, because silver is ₹254 a gram against gold's ₹15,153.

***Note on an earlier figure.** The asks list carried **₹42.85 lakh** for this. That was computed before `order_rate_master` was opened and without the locked rates. **₹60,14,200.75** is the figure computed from the rates the orders actually carry, per purity, against the last rates in the capture.*

What was done: recorded, not loaded — the rates belong to a decision the shop has not made. Whether to honour them is defect 35 and remains an ask, but it is now an ask with the right number on it.

60. 1,445 vouchers were transferred in one year, 82% of them by one person — MEDIUM

`voucher_transfer_check` (1,445 rows) is an audit log of vouchers being moved or renumbered: the old (`v_typ`, `v_id`), a new id, a timestamp and a user.

It covers only the four money voucher types, and not all of them:

Type	In the log	Vouchers on file
PMNT	631	626
JRNL	541	624
RCPT	242	269
CNTR	31	31

Who did them

User	Transfers	Voucher types
BAPI	1,186	all four
KESHAV	153	3
ADMIN	69	3
RAJENDRA KICHER	10	2
(no user recorded)	9	2
PRIYA, MAHESH, ASHOK, SHIBANANDA, SUBHAM	18 between them	

One person made 82% of every voucher transfer in the year, and 9 transfers record no user at all. Neither is wrong on its face — BAPI is in the CASH department and this may simply be the job — but a movement log where one account does four fifths of the work and some entries are anonymous is worth the shop knowing about.

The new id cannot be used

`new_vid` looked like the global voucher key that data defect 1 says is missing. It is not one. 1,445 rows carry only **949 distinct values**, and the collisions are not merges:

- `new_vid = 0` **appears 56 times**, all RCPT — a null marker, not an id.
- `new_vid = 1` **appears 23 times** across CNTR, JRNL, PMNT *and* RCPT, which no single voucher could be.

What was done: not loaded. It is an audit trail of an operation the rebuild does not perform, and its target id is unusable. Recorded for the concentration finding and so `new_vid` is not mistaken later for the key defect 1 is looking for.

61. The diamond price grids record what a stone sells for and never what it cost — MEDIUM

`diamond_rate_master` (774 rows) and `dealer_wise_diamond_rate` (642) were both unread. For a shop holding **₹6,04,28,535.00 of stones — 42% of its shelf** — they are how those stones are priced, and nothing in the rebuild could see them.

The retail grid

774 rows: one item, 4 shapes, 17 colours, 7 qualities, 9 carat bands.

Rows carrying a retail rate	774
Rows carrying a wholesale rate	3
Rows carrying a cost	3

The shop records what it sells a diamond for and not what it paid. That is the same gap the marked-against-sold screen hits from the other side: there is no cost recorded against a piece anywhere in this capture, which is why no screen in the rebuild can report a margin, and why gross profit needs B19 answering rather than a better query.

The dealer grid

642 rows across **two ledgers, and they are one grid**: 321 of the 323 comparable rows carry an identical rate, differing on two. Rates run ₹35,500 to ₹60,500 a carat across five bands.

One of the two ledgers is **MANIK CHAND JEWELLERS** — the shop's own name, which also stands as a Dealer in the chart of accounts carrying a credit balance of ₹64,38,009.10 (see the supplier balances screen). The other, ledger 4986, is not in the chart at all.

What was done: both grids loaded into `mcj_diamond_rates` with the grid they came from, and a Diamond Rates screen built under Diamond Setup — where the old menu has seven screens for shape, colour, purity, sieve, cut, mm and packet, and none for the rates those attributes combine into. An unrecorded wholesale rate or cost shows as **empty rather than zero**, and a test fails if that changes: 771 rows reading "o" would say the shop buys diamonds for nothing.

62. The stone picture is complete, and three tables that looked promising are empty — LOW

The stones sold

`sales_vou_detail` (203 rows) was unread and holds the stone content of what the shop sold. It completes a picture the rebuild had two thirds of:

	Value	Carats
Bought	1,27,31,698.71	521.329
Held on the shelf	6,04,28,535.00	2,645.661
Sold	36,44,121.33	131.946

203 rows across 94 bills and 136 tags, **every one tying to an invoice on file**.

Stone value is part of the bill and not extra to it. Not one of the 94 bills has stone value exceeding its own total, and across those bills the stones average **57.4% of what the customer paid**. For a shop whose shelf is 44% stones, that is coherent: it sells what it holds.

The table keeps both the carats a tag claimed and the carats actually found. **On all 203 rows they agree** — 21 record no actual weight at all and the rest match exactly. The screen shows a difference where one exists; in this data none does.

Three tables that hold nothing

Table	Row s	What is in it
<code>voucher_hsn</code>	756	every row has <code>hsn_no = 0</code> — one distinct value
<code>ledger_ref_details</code>	1,071	2 rows carry a name, 1 a mobile — and one of those two "names" is a postal address pasted into the field
<code>barcode_history</code>	3,083	a third view of the stock trail that adds zero events — every row already matches one on barcode and day

None loaded. Recorded so they are not opened again.

63. Fourteen order rows carry stones against an order with no lines — MEDIUM

`customer_order_detail` (115 rows) was unread and holds the stone a customer has asked for. With it the stone picture is whole:

	Value	Carats
Bought	1,27,31,698.71	521.329
Held on the shelf	6,04,28,535.00	2,645.661
Sold	36,44,121.33	131.946
On order	24,69,792.15	402.997

All 115 rows tie to an order on file. Split by what became of that order:

Rows	Carats	Value	The order
58	162.358	16,36,662.00	still live
43	197.600	4,05,236.00	every line cancelled
14	43.039	4,27,894.15	has no lines at all

Those fourteen are the defect. An order header carrying 43 carats of stone and no items — nothing on the order says what to make, so it cannot be fulfilled and cannot be quoted. ₹4,27,894.15 of somebody's expectation with no work attached to it.

A trap in the grain

The stones attach to the ORDER, not to a line. Joining them to lines fans them out: doing so reports **545 carats against a true 403**, because an order with three lines multiplies its one stone row three times. That was caught by the totals not tying, and both the loader and the screen keep the order-level grain the legacy uses. A test seeds an order with three lines and fails if its stone is counted more than once.

What was done: loaded into `mcj_ordered_stones`, and a Stones on Order screen built under Customer Report. The order's condition is computed once per order and joined, never derived by joining stones to lines.

64. The payroll configuration is a copy of the last month, not a check on it — MEDIUM

`staff_salary_sheet_payroll_detail` (209 rows) was unread and holds sixteen months of statutory payroll — what was actually deducted and allowed, per person per month.

PF	2,01,600.00
ESI	8,728.00
P.TAX	27,720.00
HRA	5,29,100.00

D.A	14,02,300.00
Total	21,69,448.00 across 16 staff and 16 months

Data defect 34 said PF and ESI should be built and pointed at `ledger_wise_payroll_detail`, the **configuration**. This is the **history** that configuration describes, and it had never been loaded.

The trap

The configuration holds a monthly entitlement of **₹1,33,906.00**. The last month in this history — July 2026 — comes to **₹1,33,906.00**.

They are the same number because **the configuration is a snapshot of the final month**, not an independent record of it. Reading the two as corroborating each other would be reading one number twice. The screen says so and a test fails if that warning is removed.

What the history does say on its own

Headcount fell from 15 to 11 between April 2025 and July 2026, and the monthly statutory cost fell with it:

	Staff	Monthly statutory
April 2025	15	1,54,392.00
December 2025	13	1,37,332.00
March 2026	11	98,402.00
July 2026	12	1,33,906.00

`payroll_amount` is zero on all 209 rows and is not carried.

What was done: loaded into `mcj_payroll_history` with a loader check that compares the last month against the configured entitlement and prints both, so a future divergence is visible rather than silent. A Payroll History screen shows each month with its headcount, and a component nobody pays reads **empty rather than zero** — sixteen rows of "0.00" would look like a charge levied and come to nothing.

65. 104 customer identity documents were held on the ERP server, and the files are gone — HIGH

`ledger_wise_document_upload` (104 rows) records documents uploaded against customer ledgers between **4 June 2019 and 20 January 2026**.

Documents	104
Customers	81
Types	95 images, 4 PDFs
Paths that are relative	104 of 104

Every path is of the form `../ledger_doc/<hash><name>.jpeg` . They are **relative to the ERP's own web root**, not URLs to a host — so the files lived on the server that went offline on 21 August 2026, and **the dump contains the names and paths but none of the files**.

What they are

Named explicitly in the filename:

Kind	Documents	Customers
PAN	6	4
Visa	4	1
Aadhaar	2	1
Named only after the person	92	79

The 92 carry a customer's name and nothing else, so what they are cannot be told from the capture. For a jeweller collecting KYC on high-value transactions, most will be identity documents of some kind.

Two separate problems

The files are unrecoverable. 104 documents for 81 customers, held for up to seven years, on a server that is gone. If the shop needs to prove a KYC check was performed, this dump cannot help it.

The capture still names them. It does not hold the images, but it does pair **customer names with document types** — that a named individual's visa or Aadhaar was on file is itself information about that person, and it is sitting in the same dump as blocker B18 covers.

What was done: not loaded — there are no files to load and the paths point nowhere. Recorded, and added to the asks: **if the old server or its backups still exist, these 104 files are on it**, and whoever holds that hardware is holding customers' identity documents.

66. The same bill number identifies two different sales — BLOCKER

`sale_bill_link` (135 rows) maps a bill number in the shop's own system to a bill number in a second one. It was unread, and what it says is serious.

Links	135
Distinct MCJ bills linked	132 of the 224 on file
Links where the two numbers are the same	2
Links where they differ	133
Links whose "other" number is also a real MCJ bill	85
...for a genuinely different sale	83

The numbering drifts apart and then overlaps:

MCJ bill	Other system	The sale that MCJ number actually belongs to
GST/2627/00004	GST/2627/00007	4 Apr 2026, ₹32,886.00
GST/2627/00005	GST/2627/00008	5 Apr 2026, ₹7,500.00
GST/2627/00006	GST/2627/00009	5 Apr 2026, ₹2,46,000.00

A customer holding invoice GST/2627/00007 from the second system is holding a number that, in the shop's own records, belongs to somebody else's sale.

Two MCJ bills map to two different numbers each, so the mapping is not even one-to-one in that direction.

Why it is a blocker rather than a curiosity

Any reconciliation that joins on a bill number across the two systems matches the wrong sale 83 times in 135. That includes matching a customer's copy of an invoice against the ledger, chasing a payment, or answering a tax query. The number on the paper is not sufficient to identify the sale.

The second system is not hypothetical: 42 of the tag photographs (defect 56) are hosted on `testworkz.in` under a path segment `mtest`, and this table names its bills.

What was done: not loaded — a mapping this ambiguous should not be given the authority of a table in the new system until somebody says which numbering is authoritative. **Ask: which of the two numberings does the shop consider its own, and do customers hold invoices carrying the other one?**

67. The reorder levels contradict themselves and cannot be checked against the shelf — MEDIUM

`item_wise_tagging` (158 rows) holds a minimum stock quantity per item and category — how few of something the shop is willing to have before making more. It cannot be used, for two independent reasons.

It disagrees with itself. 158 rules cover only 115 distinct (item, category) pairs. **43 pairs carry more than one rule and 41 of those contradict:**

Item	Category	Minimums recorded
DIAMOND BANGLE	2	30 / 6
DIAMOND BANGLE	5	30 / 6
DIAMOND LOCKET	12	45 / 5
DIAMOND BANGLE	1	14 / 6
DIAMOND BANGLE	49	2 / 4

There is no date, status or branch on the table to choose between them — the same shape as the making ladder in data defect 47 and `item_adjustment` in defect 40.

And the shelf cannot answer it anyway. The rules are stated per item AND category, but **only 1,461 of the 2,575 tags on the shelf carry a category at all** — 43% do not. A reorder report would compare a rule against 57% of the stock and call the rest missing.

The one thing that can be said without the rules is that the levels are of a plausible size: GOLD RING at 60 against 38 on the shelf, DIAMOND LOCKET at 45 against 43. If the rules were trustworthy the shop would be short on several lines.

What was done: not loaded and no screen built. A reorder report that is wrong about which rule applies and blind to 43% of the shelf would be worse than none. Recorded so the table is not mistaken for a working reorder policy.

68. The last of the unread tables: one orphaned contact list and two that are not the shop's — LOW

`address_book_sub_name` — **1,247 names pointing at nobody**

Additional contacts against a customer: a name on every row, a mobile on 399, an email on 84, an anniversary on 9, WhatsApp on none.

1,140 of the 1,247 reference a party code that exists in neither table.

Where the code was looked for	Found
<code>address_book_master</code> (40,519 contacts)	107
<code>ledger_master</code> (785 accounts)	32
Neither	1,140

Both code spaces were tried, because they are the two that data defect 41 showed are unrelated. Neither accounts for it. The category is no better: **1,223 of the 1,247 carry a category id that is not in the category master.**

So it is 1,247 names, mostly attached to nobody the shop still has on file — the same shape as `dept_user`, where 78 of 103 rows named users who no longer exist. Not loaded.

`user_online_status` — **1,190 rows of session state**

Who was logged in, which page they had open, and a lock flag, for 25 users. `last_seen_time` is an integer rather than a timestamp. It is the ERP's own session housekeeping and describes a running server, not the business. Not loaded.

`currency_master` — **108 world currencies**

One row per currency, 108 of them, no rates and no usage anywhere in the capture. Reference data that shipped with the software rather than anything the shop entered. Not loaded.

With these three, every legacy table holding more than 100 rows has been opened.

69. ₹2.6 crore of stone has passed through the stock journal, and nothing can say which way — HIGH

`stock_conversion_details` (816 rows) was unread. It is the stone content of the stock journal — the stones inside the pieces that were melted down or remade.

Rows	816
Conversion vouchers	393
Tags	411
Carats	2,554.341
Value	2,60,68,682.20
Rows tying to a voucher on file	816 of 816

That is more stone value than the shop bought or sold in the whole capture:

	Value	Carats
Bought	1,27,31,698.71	521.329
Sold	36,44,121.33	131.946
Through the stock journal	2,60,68,682.20	2,554.341
Sitting on the shelf	6,04,28,535.00	2,645.661

2,554 carats through conversions against 2,731 on the shelf. Whatever a conversion is in this shop, a great deal of stone goes through it.

Why no screen was built

The stock journal's direction is the legacy `add` column, and **which value means tagged and which means untagged is unresolved** — it is one of the open asks, and the migration that loaded the conversions says the data and the column name disagree.

Until that is answered a report cannot say whether these 2,554 carats came *out* of pieces or went *into* them, and the two readings mean opposite things: stones recovered from broken stock, or stones committed into new. **A screen that picked one would be asserting the very thing in question.**

What was done: loaded into `mcj_conversion_stones`, with the loader failing if any row cannot reach its voucher. No screen. When the `add = 1` question is answered, the data is already in place to build on.

70. The chart of accounts was missing its third level, leaving 84% of ledgers unrooted — HIGH (now fixed)

`sub_subgroup_master` (36 rows) is the third level of the ledger group hierarchy and had never been loaded. The rebuild carried two levels — 15 groups and 47 subgroups — and that was not enough to place most of the chart.

54 distinct group names appear on the 785 ledgers. 36 are groups or subgroups. **18 exist only at the third level, and they carry 662 of the 785 ledgers:**

Group name	Ledgers
Customer	485
Dealer	76
Sundry Creditors Expenses	53
Liabilities For Statutory Dues	6
Other Loan	6
Other Debtor	6
...and twelve more	

So **84% of the chart of accounts had a group name with nothing above it.** No profit and loss or balance sheet can roll up a ledger whose group has no parent, which made this a structural obstacle independent of blocker B19.

The id is not a key

The obvious load fails. **36 rows carry 30 distinct ids, and six ids each name two unrelated groups:**

id	Names it carries	Their parents
60	Security Deposit · Interest On Bhagalpur	7 · 13
64	Other Creditor · Short Term Provision	14 · 14
65	Liabilities For Statutory Dues · Shareholders Fund	17 · 18
61	Legal,professional & Consultancy Charges · Bhagalpur Loan	28 · 13
63	Interest & Penalty Charges Statutory · Goods And Services Tax	28 · 17
66	Advances · Utilities	7 · 28

The same shape as data defect 1, where `v_id` is not the voucher key. Carrying it collides on the unique index; picking one of each pair would silently drop six groups. **No legacy id is carried at this level,** and the loader says why.

After loading

34 groups added, and ledgers with no parent went from 662 to 0. The chains resolve as an accountant would expect:

Customer	→ Sundry Debtors	→ Current Assets
Dealer	→ Sundry Creditors	→ Current Liabilities
Other Loan	→ Unsecured Loans	→ Loans (Liability)

96 groups over three levels under 15 roots. **This does not answer B19** — the stock is still outside the ledgers — but the hierarchy is no longer a second reason the statements cannot be produced.

71. GOLD ORNAMENTS is a bucket 25 items were poured into — LOW

`item_merge` (40 rows) records items being merged into others between **20 September 2023 and 11 August 2026**, by two users. **40 items were merged away into just four targets:**

Merged into	Items absorbed
GOLD ORNAMENTS	25
SILVER ORNAMENT	12
SILVER DIPAK	2
PLATINUM MIX ORNAMENTS	1

The merges were applied cleanly — **no tag on the shelf points at any merged-away item**, and 39 of the 40 no longer exist in the item master.

Why it is worth recording

It explains a shape seen twice already. **GOLD ORNAMENTS is not an item so much as a bucket**, and it is exactly the item that behaves incoherently elsewhere:

- data defect 40 — `item_adjustment` names GOLD ORNAMENTS three times with contradictory making values (8% + ₹250/gm, then 15%, then 8% again)
- data defect 47 — the making ladder records GOLD ORNAMENTS at five different rates for the same weight band

Twenty-five different products merged into one code will do that. The contradictions are not corruption; they are the settings of twenty-five items collapsed onto one.

One id was reused

The single surviving merged-away item is **id 1139**, recorded in the merge as `SILVER KOPPOR STAND`. The item on file at that id today is `Rudraksha` — the id was released by the merge and later handed to something else.

The same shape as barcode reuse in data defect 29 and the duplicate ids in defect 70: **an id in this system identifies a row, not a thing.**

What was done: not loaded. The merges are already reflected in the item master and nothing references what they removed. Recorded because it explains two existing defects rather than adding a new one.

72. The load pipeline had not been run end to end, and three faults were waiting in it — HIGH (now fixed)

Eleven loaders were added over the course of the rebuild and each was tested on its own — against a database that already held everything it depended on. `load_all.sh` **had never been run from a truncated database since.** It failed three times before it passed.

1. A stale assertion

`load_masters.sh` checked `mcj_ledger_groups` for **62** rows. The third level added in data defect 70 makes it **96**. The loader's own check caught it and stopped the run — which is the check working, but it should have been updated with the change rather than by the pipeline.

2. Backticks inside an unquoted heredoc

`load_stock.sh` writes its SQL through `psql_do <<SQL` — **unquoted**, because it interpolates the tenant id. Bash therefore expands everything else in the body too. A comment added for the tag-size fix mentioned a table name in backticks, and bash tried to **run it as a command**:

```
./load_stock.sh: line 41: barcode_size: command not found
```

The loader aborted before writing a single tag. A second instance was found in `load_masters.sh`, where a comment carried ``code`` — and `code` is a command that exists on many machines, so that one would have launched an editor rather than failing loudly.

Both comments are now plain text, and the heredoc carries a note saying why prose inside it must avoid backticks and bare `$`.

3. Two loaders running before their dependencies

`load_purchase_contents.sh` ran **eighteen steps before** the purchase invoices it joins to, and `load_sold_stones.sh` before the sales invoices. On a populated database both worked, because the invoices were already there from an earlier run. From empty, every row reached no invoice.

Both loaders assert that every row reaches its bill, so the pipeline stopped rather than loading 263 orphaned rows. **The guards were the only reason this did not pass silently.** Both are now ordered after their dependencies, with a comment saying what they need.

After

`=== all loads complete ===`, **85 OK lines, no errors**, and the figures rebuild exactly: 2,575 tags on the shelf worth ₹14,23,61,188.20, 1,848 vouchers, 96 ledger groups with **0 ledgers unrooted**, 9,036 stock movements, 203 sold stones and 60 purchase content rows.

The lesson is the ordering one. A loader tested against a database that already contains its dependencies is not tested. The only defence that worked was that each loader asserts its own joins resolved.

73. 559 of the shop's 593 payment references were never migrated, and the bank could not be reconciled — HIGH (now fixed)

`mcj_vouchers` carries `cheque_number` and `cheque_date`, filled from `legacy.voucher_main`. That table holds **34** of them. The real data is on `legacy.voucher_detail`: **593 references across 435 vouchers, carrying ₹4,86,12,771.04**. None of it was migrated, and `mcj_voucher_lines` had no column to put it in.

The consequence is not cosmetic. The bank reconciliation screen reads the reference off the voucher header, so **for 1,814 of 1,848 vouchers it showed a blank cheque column** — and a bank statement cannot be reconciled against the books without the instrument reference. The screen was built, tested and passing the whole time.

How it surfaced

Not by looking for it. A voucher dated **2026-12-05**, four months into the future, turned up while checking a report's default date range. It was worth ₹1,353.70 — trivial — but a future date can mean a **day/month swap in my own loader**, which would be systematic. It was not: the narration itself reads `Ch. No.435590, Dt.05.12.2026 Issued Against Their Dues , DD.MM.YYYY`, and the loader had stored it correctly. A genuine post-dated cheque.

That narration suggested a check I had not thought of — **198 narrations embed the cheque date as text**, an independent second source for the parsed dates. Comparing them found no parsing error (and **112 of the 198 have a first field above 12**, which settles DD.MM.YYYY conclusively). What it did find was that `cheque_date` was **NULL for all 198**.

Why it goes on the line

The header cannot hold it. **109 of the 435 vouchers carry two or three different references** — 96 with two, 13 with three — one payment settling several bills, each with its own instrument.

The field is also not always a cheque number. Real values include `Gpay` and `NEFT AXNGG1663185928` beside numbers like `416331`, so it is stored as text and never parsed. It sits on the bank or wallet line, which is the line the reconciliation screen already reads: **UNION BANK OF INDIA 446, Google Pay 81, Paytm 30**.

Done

Migration `mcj-1787404000` adds `cheque_number` and `cheque_date` to `mcj_voucher_lines`; the loader fills them and now **asserts 593 and 680**, so this cannot silently regress. The screen reads the line first and falls back to the header for the 34 that have it there, joining multiple references with a comma rather than picking one.

Three tests cover it, and **two of them fail against the old query** — verified by reverting it, not assumed.

74. A systematic search for the kind of gap defect 73 was — MEDIUM

Defect 73 — 559 missing payment references — was found by accident, following a future-dated voucher. That is not a method. The same class of fault is findable directly: **a legacy column that holds real data and no loader reads**.

Every column of the 70 legacy tables the loaders touch was measured for how many rows carry a value — **1,438 columns** — and cross-referenced against the text of all 40 loader scripts. **126 well-filled columns are named in no loader**.

Most of the 126 are not data

The largest looked alarming: `address_book_master.rate_tds`, `rate_of_interest`, `credit_amt`, `calc_tds` and `brokerage_percentage`, each **100% filled across 40,519 rows**. Writing that up as "202,595 dropped values" would have been wrong. Every one of them holds **a single distinct value** — a column with a default, never used. The same is true of 13 columns on `ledger_master` (TDS, brokerage, making and certificate rates) and of `item_master.percentage`, `min_sale_rate` and `min_purc_rate`.

Fill rate is not evidence. Distinct count is. The check that separates them costs one query and belongs in front of every claim of this shape.

What was genuinely missing

Only one thing needed fixing, and it is small: `ac_name` **is capped at 50 characters** and one ledger is cut mid-word — SONANI JEWELS UNIT OF SONANI INDUSTRIES PRIVATE LI, a supplier whose full legal name is wanted on GST paperwork. `ledger_display_name` holds the full 55.

It is taken **only** where `ac_name` sits at the cap and the display name extends it. The two names differ on 9 ledgers and the other 8 are unrelated variants — URVASHI TALWAR is recorded there as MAMTA TALWAR, AVA CHATTERJEE as SHYAMALI CHATTERJEE — which must not be adopted. Whether those 8 are corrections or errors is a question for the shop, not the loader.

Still open from the audit

`sales_vou_main.actual_gw` and `actual_nw` (251 and 236 distinct over 328 rows), `voucher_detail.subgrp` (27 distinct over 4,931) and `dept_code` on four tables (7 distinct) are real values no loader reads. None is money and none blocks a screen, so they are recorded rather than acted on.

`design_transaction_main.picture_sketch` was already known — and checking it corrected a figure. See below.

A number corrected while checking

`docs/screens.md` said **5,104** designs name an image file. The true count of rows carrying a path is **5,105**, pointing at **5,045 distinct files**, 60 of them shared by two designs each. **Four references are malformed in the source** and would not have resolved even while the server was up: `../image/467903` and `../image/561676` have no extension, `../image/84348720jpg` and `../image/83792824jpg` are missing the dot before `jpg`. Corrected in place.

75. The coverage figure was wrong in both the numerator and the denominator — MEDIUM (now fixed)

The status doc read **112 of 188 screens, 60%**. The true figure is **111 of 187, 59%**. Both halves were wrong, in opposite directions, which is why the percentage looked stable.

The denominator counted the CSV's header row. `legacy_menu.csv` opens with `Top Menu, Sub-Group, Item, Link` — column names. Counting distinct values in the fourth column picks up `Link` as though it were a screen. The application does not make this mistake: `navigation.go` reads `records[1:]`. The fault

was in the counting, not the product.

The numerator counted test files. Scanning `modules/mcj/**/*` for `LegacyFile:` matches the navigation tests as well as the registry — `color_master.php` appears **seven** times, six of them in `navigation_test.go`. Excluding `_test.go` gives 117 distinct claims, of which **111 name a menu row**.

The six that name no menu row

`day_book.php`, `label_print.php`, `add_gold_rate_fix.php`, `set_rate_gold_opening.php`, `add_sales_voucher_combined.php` and `print_sales_vocher_register_gst.php` are all real legacy pages — four have their own captured page and `day_book.php` is linked from **255** of them. They are reached from other screens rather than the menu, so they are legitimately built and legitimately add nothing to coverage.

Whether the menu is the whole inventory — it is

Worth testing, because if real screens existed outside the menu the denominator would be meaningless. **270 pages were captured; only 178 are menu entries**, so **92 captured pages sit outside it**, several of them substantial: `add_sample_issue_to_karigar.php` at 829 KB, `add_cust_order.php` at 676 KB, `add_party_scheme.php` at 500 KB.

All 92 are linked from a menu page. Every one is an add form, a drill-down or a print view belonging to a screen already counted; none is standalone. Writing them up as ~85 more screens to build would have been a large error in the other direction.

The ten menu entries with no captured page are `?type=`, `?bill_type=` and `?rec_type=` variants whose base page **is** captured — not missing references.

Guarded

`TestNavigation_EveryLegacyFileNamesAPageThatExisted` now enforces the rule the navigation tests already state — that an invented `.php` name "reads as evidence about the old system, and would be a fiction". A claim must name a menu row, a captured page, or a page a captured page links to.

The first version was too strict and failed two screens: `/mcj/invoice` and `/mcj/sales/new` rest on **link evidence alone** — `add_sales_voucher_combined.php` is referenced by 254 captured pages and was never captured itself. Those names are real; the test was wrong, and was widened rather than the claims being changed. It also asserts that a fabricated name is still rejected, so the widening did not empty it out.

76. 90% of the shelf names a maker who exists nowhere in the database — HIGH (needs the shop)

`barcode_main.karigar_code` is filled on **1,743 of 2,816 tags**, carrying **110 distinct codes** — `D599` on 470 tags, `D142` on 113, `DL236` on 104, `d76` on 99, and so on. On the shelf specifically it is **1,625 tags worth ₹12,77,72,008.04 — 90% of the ₹14,23,61,188.20 stock**.

Not one of the 110 resolves to anything.

Tested and rejected, in order:

- `mcj_ledgers.code` — **0 of 110**
- `legacy.ledger_master.ldgr_code` — 0
- `legacy.ledger_master.ac_code` — 0
- the same match case-insensitively (the codes are mixed case: `D599` beside `d76`) — 0
- `legacy.address_book_master.ldgr_code` — 0
- the `design_code` on the same tag — 0

Then every text column in the legacy schema was searched for the literal value `D599`. It occurs in **exactly one place**: `barcode_main.karigar_code`. There is no karigar master table in the dump — the only karigar-named tables are `karigar_gold_stock` (2 rows) and `karigar_wip_bal` (1 row) — and the chart of accounts holds **76 Dealer ledgers and exactly one Karigar ledger**.

So the field is free text against no master. It is the same shape as defect 41, where `mcj_parties.ledger_code` was joined to `mcj_ledgers.code` and 368 codes collided without naming a single same person — except that here there is nothing to collide with.

What this settles

The 31 unclaimed MANUFACTURING and Karigar Report screens are not waiting on effort.

`receive_amount` (141 rows, a payments table) aside, every issue and receive table is empty — **except** `karigar_gold_stock`, **whose 2 rows issue 869.850 g of fine gold to MA TARA JEWELLERS and are the subject of [defect 164]**: `karigar_gold_stock 2`, `karigar_wip_bal 1`, `dealer_wise_making 7`, `customer_own_goods_receive 6`, `estimate_main 2`. There are **no karigar transactions, no work orders, no stone issues and no hallmarking movements to report on**, and no master to name the maker even where a tag remembers one.

Two exceptions were found later and both are recorded: `karigar_gold_stock`'s two rows issue 869.850 g of fine gold to a named goldsmith ([defect 164]), and `retail_all_stock` carries 4 hallmarking rows covering 2.230 g, issued and received back ([defect 166]). The first materially contradicts this paragraph; the second only makes it absolute where it should read "almost none".

A Karigar Ledger built on this would render 110 headings, each an unexplained code, with no opening balance, no issues and no receipts under any of them.

The question for the shop

What is `D599`? If it is a workshop shorthand kept on paper or in someone's head, a two-column list — code against name — turns 90% of the shelf from anonymous into attributed, and is the whole of what these screens need to become buildable. If the makers were tracked in a system outside this ERP, that system holds the other half of the manufacturing record.

77. The retail voucher variants have no data, and one table's name is a trap — MEDIUM

Fourteen unclaimed RETAIL screens were checked against what the shop actually recorded. Most cannot be built, and the reasons are worth stating so they are not re-examined.

Approval sales were never configured, let alone used

Across the **entire** legacy schema the word "approval" appears in exactly two columns:

`sales_vou_main.appr_sale` and `default_settings.def_sale_on_appr_ldgr`.

`appr_sale` is **0 on all 328 sale lines**. `def_sale_on_appr_ldgr` — the ledger an approval sale would post to — is **empty** on the single settings row. There is no purchase approval table at all.

That settles seven screens: Sales Approval Voucher, Sales Approval Return Voucher, Purchase Approval Voucher, Purchase Approval Return Voucher, Sales Approval Register, Purchase Approval Register and Approval Pending Report.

There is only one kind of sale

All **224** bills are `SALE-GST`; `bill_type` is `local` on 221 and `central` on 3. Nothing distinguishes an exhibition, an export or a repair sale, so Exhibition Sale, Export Sale and Repair Sale have no rows to show — three more.

Of the line flags, `ret_status` is set on **3** lines (the three sale returns, which reconcile with `sale_return`) and `mrp_status` on **38**, which the MRP sales register already reports.

`sales_quotes` **is not quotations**

The table is named as though it holds customer quotations, and an Estimate screen built on it would have looked right. It holds **motivational quotes for the dashboard**:

"Customer Satisfaction is worthless .. Customer Loyalty is priceless" "The purpose of a Business is to create a customer who creates Customers."

Ten rows of them. The real estimates are in `estimate` and `estimate_main` — **two** of them, `EST/2526/00001` and `EST/2526/00002`, with full line detail: barcode, item, gross and net weight, purity, rate and making, totalling ₹12,294.00 and ₹5,01,000.00.

A table's name is not evidence of what it holds. This is the same lesson as `COUNTER STOCK` being filed under *Customer* with a zero balance.

78. 12.30 kg of gold is recorded as raw stock, unvalued and never migrated — HIGH (needs the shop)

`barcode_main_rawstone` holds **10 rows** the loaders have never read. It is not scrap notes. The metal is broken down by purity the way a jeweller records it:

Item	Purity	Net weight
GOLD ORNAMENTS	9	82.156 g
GOLD ORNAMENTS	14	1,073.049 g
GOLD ORNAMENTS	18	5,566.291 g

Item	Purity	Net weight
GOLD ORNAMENTS	22	4,973.818 g
COIN	22	84.490 g
COIN	995	524.720 g
SILVER ORNAMENT	SLV	9,971.290 g + 2,998.751 g
SILVER COIN	SLV	2,052.044 g
SILVER ARTICLE	ARTIC(PCS)	0.000 g

12,304.524 g at gold purities and 15,022.085 g of silver. Every item id resolves in `item_master`. Not one row carries a barcode, a rate or a value.

Correction — this entry said 11,779.804 g of gold and 15,546.805 g of silver. Both were wrong by the same row: the **524.720 g COIN at purity 995**, listed in the table above, was left out of the gold total and added to the silver one. 995 is **99.5% fine gold**. Valued as silver it is worth ₹1,33,016.52; valued as what it is, **₹81,23,715.04** — an ₹80 lakh difference on one line. The title's "11.78 kg" should read **12.30 kg**.

`barcode_detail_rawstone` holds 9 stone rows totalling 475.433 carats, and they are **not part of this holding: 8 of the 9 carry** `main_id = 0`, which matches no raw stock row at all. Only **0.380 ct** attaches to a parent. Those 8 orphans describe 28,603.429 g of stone gross weight — more than the entire raw metal above — hanging off nothing. Counting them as part of the raw stock, as an earlier draft of this entry did, overstates it.

For scale, the entire tagged shelf is **20,038.936 g**. This is more than half as much metal again, held outside it.

It cannot be current stock, on the accounts' own evidence

The tagged shelf blends to **₹7,104.23 per gram**, and this entry used to value the raw gold at that rate — **₹8,36,86,423.98**.

That rate cannot be applied to gold. It is the whole shelf divided by the whole shelf's weight, and **48% of that weight is silver**: 9,600.508 g of silver against 10,238.962 g of gold. Split by metal the shelf blends to

	Net weight	Value	Per gram
Gold	10,238.962 g	13,77,71,338.90	13,455.60
Silver	9,600.508 g	30,57,474.70	318.47
Purity 95	199.466 g	15,32,374.60	7,682.38

At **today's metal rates**, purity by purity, the raw holding comes to:

	Net weight	Value
Raw gold (9, 14, 18, 22, 995)	12,304.524 g	16,46,39,828.53
Raw silver	15,022.085 g	38,08,098.55

	Net weight	Value
Total	27,326.609 g	16,84,47,927.08

So the raw metal is worth **around ₹16.5 crore of gold**, not the ₹8.37 crore this entry carried — a figure produced by pricing gold at a rate half of it is silver. It **exceeds the entire tagged shelf**, which sharpens rather than softens the question this defect asks.

The trial balance is short by ₹15,32,28,603.49 and the tagged shelf explains ₹14,23,61,188.20 of it, leaving **₹1.09 crore** unexplained. If this metal were also held and unrecorded, the books would have to be short by roughly **₹8.4 crore more than they are**.

The shelf rate carries making and certification, so it overstates raw metal — but even at half of it the figure is several times the residual. **The arithmetic says this is a historical opening that was since tagged or consumed, not metal sitting in the shop today**. It does not say so conclusively, and this is not a question to answer by inference.

Why it was invisible

`barcode_main_rawstone` has **10 rows**, and the tables around it are of a size that reads as noise. It was skipped for the same reason `old_gold_purchase` was — a small table is not an unimportant one. This one describes more metal than the shelf.

What this settles

Of the seven SETUP Opening Stock screens, six cannot be built: **no** `studded_opening_stock` **table exists at all** (Own Studded Stock, Party Sample Stock, Karigar/Dealer Studded Stock), and raw stock exists only as `type='sh'` — there are **no co or kr rows**, so Party Raw Stock and Karigar/Dealer Raw Stock have nothing. Own Tag Stock is already built.

The question for the shop

Is this metal still in the shop? If it is, the accounts are short by around ₹8 crore more than anyone has counted and the balance sheet's difference line is badly understated. If it was tagged into the shelf years ago, it is a historical opening and can be loaded as one — but it should not be loaded as current stock on a guess either way.

ANSWERED, and the accounting inference above was wrong — see defect 133

The shop's own raw-stock movement report survived in a scratch table excluded from staging. It shows this metal **opening, being received and being issued through to 17 June 2026**, and closing at **11,998.005 g at gold purities** — more than it opened with. The inference that the books "have no room" for it was sound arithmetic about the ledgers and wrong about the world: **the metal is real, it is moving, and it sits outside the accounts exactly as the tagged shelf does**.

79. There is no cost basis anywhere in the data, so gross profit cannot be reported — HIGH

The Gross Profit Report (`gross_profit_report.php`) and Ratio Analysis (`ratio_analysis.php`) both need to know what the goods cost. Nothing in this database records it. Three independent routes were tried and all three close.

1. The tag's value is a selling price, not a cost

232 of the 328 sale lines match a tag by barcode. Those tags are valued at **₹1,05,03,577.58** and the pieces sold for **₹1,05,56,997.00** — a difference of **0.5%**. The tag carries the price the piece is billed at, which is what a price tag is for. Subtracting one from the other gives a half-percent "margin" that is really just rounding, and no jeweller's margin looks like that.

(The average of the per-line ratios is 1.110 while the ratio of the totals is 1.005. Both are arithmetically correct and they answer different questions — the per-line average is pulled about by small lines. The ratio of totals is the one that describes the trade. This is the same ratio-of-averages trap that once produced a "constant factor of 5×" here.)

2. Purchases cannot be traced to tags

`barcode_create_from_purchase_detail` links a purchase to the tag it created. It holds **21 rows**. There are **2,816 tags**. That is **0.75%** — enough to cost less than one piece in a hundred.

3. The one place that separates cost from price never used it

`barcode_details` — the stone detail behind each tag — carries **both** `rate_stock` and `rate_sales`, which is exactly the cost-versus-selling distinction a gross profit needs. **727 rows have both populated, and the two values are identical on every one of them.** Not close: equal, on all 727. The field was there and was filled with the same number twice.

What this means

A gross profit screen could be made to render. It would subtract a selling price from a selling price and call the remainder margin. That number would be quoted in a meeting.

It also explains the shape of the profit and loss account: with no cost of goods sold, the P&L nets income against purchases and shows a **₹1,10,87,494.08 shortfall the shop did not make**. The missing figure is the same one in both places.

Both screens are recorded as unbuildable rather than built. If the shop prices from a cost sheet kept outside this ERP, that sheet is the missing input and both become possible.

80. A stone figure was labelled "on the shelf" in seven places and was not — MEDIUM (now fixed)

Every figure repeated across the four documents was re-checked against the database. Most verify to the paisa. One did not, and it was repeated seven times.

₹6,21,95,953.60 is every stone record in mcj_stock_stones — 819 lines over 692 tags, 2,731.821 carats. It was written as "Held on the shelf", "Sitting on the shelf" and "44% of its shelf".

The stones actually on the shelf are ₹6,04,28,535.00: 672 lines, 562 tags, 2,645.661 carats, 42.4% of the ₹14,23,61,188.20 shelf. The difference — ₹17,67,418.60 and 86.160 carats — sits on tags that have already been sold.

How it happened

Not a bad query. A scope slip inside one sentence:

819 stone lines over 692 barcodes, and covers every one of the 486 in-stock tags that carries stone value. 2,731.821 carats, ₹6,21,95,953.60.

The first clause counts the whole table, the middle clause counts in-stock tags, and the total belongs to the first. Read quickly it says the shelf holds ₹6.22 crore of stones. It was then quoted onward into a blocker, a defect and two status entries, and the "44% of the shelf" was derived from the wrong numerator.

Every count in a claim has to answer the same question the money in it does. Where a sentence needs both scopes, both get named.

Verified alongside it

The shelf value ₹14,23,61,188.20 and 20,038.936 g, the opening imbalance ₹15,32,28,603.49, the raw stock 27,326.609 g and its silver 15,546.805 g, and the one-paisa difference across all voucher lines — all exact.

81. The status doc misstated a blocker by 3.4× and mislabelled who it affects — MEDIUM (now fixed)

Continuing the figure audit into the "Waiting on you" table — the numbers most likely to be acted on.

B9 is 165 items whose stored price cannot be reproduced. docs/STATUS.md described it twice as 566 certified pieces:

Bill the 566 certified pieces B9 — Were the certified pieces priced by hand? 566 pieces cannot be billed

Both halves are wrong. The count is 165, not 566 — 3.4× overstated — and they are not the certified goods. blockers.md and pricing-formula.md both say so plainly: the 165 cluster by certification type (by_val 98, per_ct 35, so 133 of 165) which is why certification was suspected, but the shop holds 549 certified pieces in stock and 678 across all tags. Neither is 566, and none of those counts is what B9 blocks.

Read as written, the table says a fifth of the shelf cannot be billed. What is actually true is that 9% of computed-price items carry a pricing component this rebuild cannot see, off by −₹66,585 to +₹10,731.

Corrected in both places, with the cluster stated so the certification link is not lost.

The rest of the table verified

- **40,519** customer records — exact
- **1,023** photographed tags over **3,965** image rows — exact
- **889 tags worth ₹10,73,39,445.03, 75.4% of the shelf** — exact
- **1,625 tags worth ₹12,77,72,008.04** naming an unidentifiable maker — exact
- **135** bill links, **284** orders, **2,520** order rate rows — exact
- All **25** documented legacy table row counts — exact

(One near miss while checking B24: `sum(DISTINCT total)` reports ₹10,63,93,293.99 because it sums distinct **values** and drops tags that happen to be worth the same. The figure is ₹10,73,39,445.03. `DISTINCT` inside an aggregate answers a different question than `DISTINCT` on the rows.)

82. My own tenant fix scoped the list and left the detail page open — HIGH (now fixed)

Defect-in-progress, found by re-reading `tenant_scoped_repository.go` an hour after writing it.

The wrapper built for B11 scoped `List`, `Count` and `GetAll`. It passed `Get`, `Exists`, `Update` and `Delete` **straight through to the unscoped repository**.

All four take a key, and a **key from another tenant is exactly as valid as one of ours unless it is checked**. So anyone holding another tenant's row id could read that row through the ordinary detail page, and `Update` and `Delete` accept the same id — editing and removing it.

That is worse than the leak it replaced. The original defect exposed rows in a list; this exposed rows to modification. And the clean list makes it **harder to notice, not less true**: the row never appears anywhere the user can see, so nothing suggests the id would work.

The fix

`ownsKey` counts rows matching the key **with the tenant filter applied**, and `Get`, `Exists`, `Update` and `Delete` all consult it first. A counting query rather than fetch-and-compare, because the entity is generic here — there is no `TenantID()` to read off it — and a count cannot accidentally return the row it is checking.

`Exists` returns **false** rather than an error for another tenant's row. Answering "it exists but is not yours" leaks the thing being withheld. For the same reason the error is `ErrNotOurs = errors.New("no such record")`, worded as absence rather than refusal.

Two tests cover it: one that `Get` cannot reach another tenant's row, one that `Update` and `Delete` cannot touch it — and the second re-reads the row afterwards to confirm it is still there and unchanged.

And the write side, checked rather than assumed

`Create` and `Update` were left passing values through. The repository **honours a supplied** `tenant_id` — measured: creating a row with another tenant's id in the values lands it in that tenant.

This is not reachable from the web. The crud controller skips readonly fields when parsing a form (`crud_controller.go:432`) and `tenant_id` is declared readonly, so a crafted POST cannot aim a write. Reported as the defence-in-depth gap it is, not as a live hole.

Closed anyway: `forceTenant` replaces any supplied `tenant_id` with the caller's own, on both `Create` and `Update` . Replaced rather than rejected — a caller passing the right tenant is doing nothing wrong, and one passing the wrong tenant should not learn whether that tenant exists.

Five tests now cover the wrapper: reads, the unwrapped leak, `Get` , `Update` and `Delete` , and this.

The lesson

A partial fix to an isolation bug is not a partial fix. Scoping the front door and leaving the side door open reads, in a diff, exactly like closing both. Nothing failed; every test still passed; the list was demonstrably clean.

83. `legacy_id` is not an identity, and ten tables prove it — MEDIUM

Reviewing the Own Raw Stock screen written earlier today: it looks an item up by `legacy_id` to copy the name, which silently assumes one match. That assumption turned out to be worth checking generally.

58 of the 63 tables carrying `legacy_id` have no unique constraint on it, and 10 already hold duplicates:

Table	Duplicated values
<code>mcj_voucher_lines</code>	984
<code>mcj_design_stones</code>	690
<code>mcj_purchase_contents</code>	21
<code>mcj_ordered_stones</code>	20
<code>mcj_purchase_invoices</code>	19
<code>mcj_salary_sheets</code>	17
<code>mcj_ledger_groups</code>	13
<code>mcj_customer_order_lines</code>	10
<code>mcj_bank_accounts</code>	7
<code>mcj_scheme_payments</code>	4

None of them is a bad load

The first look suggested rows had been inserted twice. They had not. `mcj_voucher_lines` holds **5,229** rows against a source of **5,229** — exact. What the source holds is **1,186 distinct `id` values across those 5,229 rows**. `legacy.voucher_detail` reuses its own `id`, and the loader copies it faithfully.

`mcj_ledger_groups` is the clearer illustration. `legacy_id 1` is both *Capital* — a group, from `main_group` — and *Credit Card*, a subgroup from `subgroup_master`. Two source tables, each numbering from 1, merged into one table. Its third level carries **NULL**, having no source row at all.

`legacy_id` **is only unique within a source table**. Several of these tables merge more than one, so it was never going to be an identity, and any reconciliation keyed on it alone will mismatch.

What was done

One constraint, on `mcj_items` only, where the source genuinely is single and genuinely is unique: `item_master` 322 rows, 322 distinct `item_id`, 322 loaded. That is the table the raw stock lookup reads, so the assumption it makes is now enforced rather than merely true.

Deliberately not generalised. Applied as a rule this constraint would reject a correct load of nine other tables. The full pipeline was re-run after adding it: exit 0, 87 OK lines.

84. A screen I wrote today let "may add" mean "may delete" — MEDIUM (now fixed)

The Own Raw Stock screen puts adding a line and removing one on the **same POST route**. A route gate can therefore only ask for one permission, and it asked for `ItemCreate`.

So a user trusted to record metal could **delete** records of it. Not through a crafted request — through the Remove button, which the screen renders for everyone who can reach it.

Fixed by checking `ItemDelete` inside the handler when a `remove` is submitted, which is the only place the two actions are distinguishable. A test asserts a user holding `ItemRead` and `ItemCreate` can add a line, is refused **403** on removal, and that the line is still there afterwards. Verified to fail with the check taken out.

Checked whether it was a pattern

Four other controllers branch on a submitted action — credit note, tag, sale and old gold, all on `action != "save"`. They are safe, and for a reason worth stating: their two actions are *price* and *save*, and both sit behind `VoucherCreate`. Gating a preview behind the create right is **stricter** than it needs to be.

The dangerous direction is the opposite one — the **more** privileged action sharing the **less** privileged gate — and this was the only instance. It was in code written today.

85. The load pipeline said "safe to re-run at any time", and it is about to stop being true — HIGH (now guarded)

`load_all.sh` opened with:

Idempotent: safe to re-run at any time.

That was true while the database held only migrated data. **It stops being true the moment the shop enters anything.** Every loader clears its tables and rewrites them from the legacy dump, so a sale, a voucher or a raw stock line entered through the application is deleted — and the capture cannot put it back, because it never contained it.

The sentence is the dangerous part. "Safe to re-run at any time" is exactly what somebody reads before running it on a live database, and this project has already lost 40,519 parties, 1,848 vouchers and 785 ledgers once to a `migrate down` that looked equally harmless.

The Own Raw Stock screen built today is the first screen whose whole purpose is to write rows the dump does not contain. It made the wording actively wrong rather than merely fragile.

The guard

Every migrated row carries the id it had in the legacy system. **A row with none was created in the application** — so it is precisely what the script would destroy and could never restore.

`load_all.sh` now counts those rows across `mcj_vouchers`, `mcj_sales_invoices`, `mcj_stock_items`, `mcj_raw_stock`, `mcj_estimates` and `mcj_customer_goods` before clearing anything, and refuses if it finds any, naming the tables and counts. `MCJ_FORCE_RELOAD=1` overrides it for the case where rebuilding from the dump really is the intent.

Verified all three ways: a clean run still completes (exit 0, 87 OK lines); a single hand-inserted raw stock line makes it refuse with **exit code 1**, which matters because a caller reads the status rather than the message; and the override completes and restores the migrated figures exactly — 10 raw stock rows, 2,575 tags.

The header now says what is actually true: idempotent against the capture, and **not safe on a database the shop is using**.

The same guard one level down

Protecting only `load_all.sh` protects the careful path. Somebody debugging runs a single loader — `./load_rawstock.sh` — and that is exactly where losing a few rows is least likely to be noticed. The five loaders whose tables the application writes to (`load_rawstock`, `load_vouchers`, `load_invoices`, `load_customer_goods`, `load_estimates`) now carry the same check through a shared `guard_app_data.sh`.

The first version of that was broken and only the full pipeline showed it. The loaders `cd` to the repository root before the guard is sourced, so `$(dirname "$0")` no longer points at the script: standalone runs worked, and `load_all.sh` died with `./guard_app_data.sh: No such file or directory` after 38 successful steps. Resolved from the repository root instead, and verified from all three invocation paths — `bash tools/legacy/load_rawstock.sh`, `cd tools/legacy && ./load_rawstock.sh`, and through `load_all.sh` — each refusing with **exit 1** and each completing under `MCJ_FORCE_RELOAD=1`.

86. A test failed about one run in three, and had done since before today — MEDIUM (now fixed)

`TestCategorise_FiltersToUntaggedOnly` failed in a full-package run, then passed on its own, then passed three times in a row. Running the whole package twice more reproduced it: **run 1 FAIL, run 2 ok.**

The assertion was:

```
if strings.Contains(body, "T3") { ... "a tag that already has a category appeared" }
```

T3 is the seeded barcode of the one categorised tag, and the test asserts it is **absent** from the rendered page. T3 is also **two characters**, and the page carries a session token and other generated values — any of which can contain T3 by chance. When one did, the test failed and the screen was innocent.

An absence assertion on a short string is the fragile direction. `Contains` on a present string only fails when the page is wrong; `!Contains` fails whenever anything else on the page happens to spell it.

Fixed by seeding barcodes that cannot occur by accident — `TAGUNCATEGORISED1`, `TAGUNCATEGORISED2`, `TAGCATEGORISED3`. Three full-package runs clean afterwards.

Two things it caught on the way

Renaming the barcodes broke two other tests, which is how the seed turned out to be read in three places: the insert, a `WHERE barcode IN ('T1','T2')` that collects the ids to post, and the assertions. The id query still named the old barcodes, so nothing was submitted and the tags came back uncategorised — **a passing test would have been possible only because both halves used the same wrong list**.

The failure messages also still read `T1 is "", want RINGS` after the rename, so they now describe the tag rather than quote a barcode that no longer exists.

This flakiness predates today. It is recorded rather than quietly fixed because an intermittent failure that nobody writes down is indistinguishable from an environment problem, and gets re-run instead of investigated.

87. Some of my own "suite green" checks never ran — MEDIUM (now fixed)

```
just test-e2e ran go test -tags=integration ./... with no -count=1.
```

Go's test cache keys on the source tree. **It cannot see the database.** So applying a migration, reloading the legacy data, or adding a constraint — none of which touches a `.go` file — leaves the cache valid, and the next run reports `ok (cached)` for tests that never executed against the new state.

Demonstrated directly: two consecutive integration runs with no source change, both `ok (cached)`, neither having run.

This happened repeatedly during the rebuild. Several times today a migration was applied and the suite immediately reported `ok ... (cached)` for `presentation/controllers` — an integration package. **Those confirmations were weaker than they were reported to be**, and any of them could have been hiding a schema change that broke a screen.

Nothing appears to have been missed: the final runs were uncached and clean, five consecutive full passes. But that is luck rather than method.

`just test-e2e` now passes `-count=1`, as `just test-smoke` already does after the same trap caught a deliberately broken report "passing". The plain `just test` recipe keeps the cache, correctly — those tests read only the source tree.

A test whose inputs live outside the source tree must never be cached. This is the second instance of that rule today, in a different recipe.

88. `migrate down` says it rolls back one migration and rolls back all of them — HIGH (SDK defect)

This project has lost its entire database twice to `migrate down`, and both times it was recorded as carelessness. It is not. **The command's own documentation is wrong**, and here is the line.

`pkg/commands/cli_migrate.go:`

```
Short: "Rollback the last migration",
Long: `Rolls back the most recently applied database migration.`,
```

`pkg/application/migrations.go:153:`

```
plannedMigrations, dbMap, err := ms.PlanMigration(db, "postgres", migrationSource,
migrate.Down, 0)
```

The final argument is `max`. In `sql-migrate@v1.7.0`, `migrate.go:75:`

```
} else if max > 0 && max < toApplyCount {
    toApplyCount = max
}
```

The cap applies only when `max > 0`. Passing `0` leaves `toApplyCount` at every applied migration — so `migrate down` rolls back **all 119**, dropping every table and everything in them.

`just migrate down` reaches the same command through the `direction` parameter.

Why this matters more than a wrong help string

Someone reaching for `migrate down` is usually trying to undo one bad migration. The help confirms that is what will happen. What they get is an empty database, and on 21 August that meant 40,519 parties, 1,848 vouchers and 785 ledgers that existed nowhere else, because the original migration had been run as ad-hoc SQL and never version-controlled.

The load pipeline exists because of that day. It can now rebuild everything from the capture — but only what the capture contains, which is nothing the shop has entered since.

Recorded, not worked around

`redo` has the same shape: it calls `Rollback()` and then `Run()`, so it is "roll everything back and reapply everything", not "redo the last one". It happens to end in the same schema, but every table is dropped in between.

Nothing here is patched. The Justfile now warns at the point of use, and docs/blockers.md B25 carries it for upstream. Rolling back one migration is `psql` and the `Down` block from the file, by hand.

89. The balance sheet's difference line was absorbing ₹13,197 that is not stock — MEDIUM (now fixed)

The difference line is the one figure on that screen whose **size is the evidence** — it is what justifies saying the missing asset is stock, because it is the same order of magnitude as the shelf. Anything else quietly added to it weakens exactly the argument it exists to make.

Chasing a gap left open earlier: the opening trial balance is out by **₹15,32,28,603.49**, but the balance sheet's difference line read **₹15,32,16,805.48**. **₹11,798.01** apart, and nothing named it.

It reconciles exactly:

Opening imbalance	15,32,28,603.49
less expense opening balances the sheet excluded	-13,197.00
plus pre-year trading counted on one side only	+1,398.99
= the difference line	15,32,16,805.48

Three expense ledgers carry an opening balance — ₹13,197.00 credit. That is an earlier year's trading, which a balance sheet would have closed into retained earnings. This ERP has no retained earnings account, so the sheet had nowhere to put it and left it out, whereupon it reappeared inside the difference and read as missing stock.

The second component is the single pre-1-April voucher line found while making the profit and loss agree with the balance sheet: its asset side is inside the closing balances, its income side is outside the year's result.

Done

The openings now appear as their own row — *"Opening balances on income and expense ledgers"*, filed under *"An earlier year's result, with no account to hold it"*. The sheet still ties, and the difference line is now **₹15,32,30,002.48**, which is the opening imbalance plus that ₹1,398.99 and nothing else.

A test seeds an expense ledger with a credit opening, asserts the row carries it, and asserts the sheet still ties with it in.

(The reconciliation closes to within one paisa: the pre-year component is ₹1,399.00 by direct sum and ₹1,398.99 by subtraction, a rounding artefact of the paisa conversion. Stated rather than smoothed over.)

90. The trial balance and the balance sheet disagreed about what does not tie — MEDIUM (now fixed)

Both screens describe the same books, so **the amount that fails to tie must be the same figure on each**. It was not:

Trial balance suspense	15,32,28,603.48
Balance sheet difference	15,32,30,002.48
	₹1,399.00 apart

The same ₹1,399 as before — one voucher line dated before 1 April. This is its third appearance: it split the profit and loss from the balance sheet, then it sat inside the difference line unnamed, and now it splits the balance sheet from the trial balance.

The cause is structural. The balance sheet's result for the year is **bounded at 1 April**; the trial balance has no such split and carries the line inside the ledger's closing balance. So a pre-year income or expense line is counted on the asset side by both and on the other side by only one.

Defect 89 added a row for income and expense **opening balances**. That was half the category. The other half is income and expense **movement dated before the year** — the same thing arriving by a different route.

Done

The row now carries both, and is named for what it holds: *"Income and expense from before this year"*, ₹11,798.00 on the real books — ₹13,197.00 of openings less the ₹1,399.00 line. **Both screens now report ₹15,32,28,603.48**, and a test asserts they agree, treating an absent difference and a zero one as the same claim.

What the fix did to the tests

Three failed, and all three were the tests being wrong rather than the code:

- The seeded books now tie **with no difference line at all** — the ₹400 pre-year sale is properly placed rather than unexplained. The test had asserted the ₹400 gap, so it now asserts there is none.
- The openings row shows **900**, not 500: the 500 brought forward plus the 400 dated 31 March. Both are last year's.
- The agreement test had to accept "neither screen reports a gap" as agreement.

A fix that makes a seeded ledger balance completely will break a test written against the imbalance. That is the test doing its job.

91. A post-dated cheque is booked on its future date, so the debt it settles still shows as owed — LOW (recorded, for the shop to decide)

Cross-checking one bank account across four screens turned this up. The screens themselves are sound; what they revealed is a modelling question.

Every figure agreed. For UNION BANK OF INDIA (code 3137) as on today:

Screen	Closing
Trial balance	4,91,62,109.87 Cr

Screen	Closing
Ledger book	4,91,62,109.87 Cr
Bank reconciliation, per the books	4,91,62,109.87 Cr
Direct SQL	4,91,62,109.87 Cr

The bank book shows **4,91,12,150.78 Cr**, which is not a disagreement: it carries all four bank accounts together, and $-4,91,62,109.87 + 0.11 + 11,460.00$

- 38,498.98 is exactly $-4,91,12,150.78$.

What it exposed

The reconciliation showed the statement balance **equal** to the book balance, which would make the screen useless if it were always so. It is not. Of the vouchers touching that account, **497 are cleared and exactly one is not** — and that one is `PMNT 576`, the post-dated cheque from defect 73, dated **5 December 2026**. It sits outside a reconciliation run to today, so the two balances correctly coincide.

Following it through: the cheque pays **DAYANIDHI TRADE PVT LTD**, an *Other Loan* account.

Balance as on today	25,00,000.00 Cr
Balance including the December voucher	23,64,630.00 Cr

The shop has **issued a cheque for ₹1,35,370.00** and its books will not recognise it until December. The debt still reads as ₹25 lakh, and the cheque appears as an outstanding item on no reconciliation run before then.

Not changed

Booking a post-dated cheque on its own date is defensible — it is not a payment until it clears — and it is what the shop did. The alternative, recording it on issue with `cheque_date` carrying the future date, is what the `cheque_date` column exists for and what makes the cheque visible as outstanding.

One voucher and ₹1.35 lakh today. It is written down because the *practice* matters more than the amount: every post-dated cheque the shop writes from now on lands the same way, and only they can say which treatment they intend.

92. The invoice columns are not additive, and I nearly reported ₹8.96 lakh of nonsense — MEDIUM (a trap, not a fault)

Cross-checking sales across three sources. The three totals differ, and they are supposed to:

Source	Amount	Rows
<code>mcj_sales_invoices.bill_total</code>	2,47,65,386.00	224
<code>mcj_sales_invoice_lines.amount</code>	1,51,40,109.00	328

Source	Amount	Rows
Income ledgers, net credit	2,20,03,143.00	227

`goods_amount` sums to **₹2,40,66,454.00**, which is the income ledgers' credits to the rupee — 2,20,03,143.00 net, plus the 20,63,311.00 of sale returns the ledgers net off. That is the tie that matters, and it is exact.

The trap

`mcj_sales_invoices` carries `goods_amount`, `making_charge`, `other_making`, `certification`, `discount` and `round_off` beside `bill_total`. They read as components of the total. Adding them up gives ₹2,56,61,020.13 against a `bill_total` of ₹2,47,65,386.00 — **₹8,95,634.13 apart**, which looks exactly like a real defect and was almost written up as one.

They are not components. Making, other making and certification are already inside `goods_amount`. The relationship is simply:

```
bill_total = goods_amount + GST
```

Checked on the five largest bills first — 1,02,551/34,18,355, 95,890/31,96,335, 28,064/9,35,462 and the rest all come to **3.000%** — then across all 224.

And the rate spread is legitimate

Rate	Bills	Goods
3.00%	196	—
1.50%	19	12,05,520.00
0.25%	2	1,80,451.00
other	6	₹246 to ₹1,033 each

The three rates are the real GST slabs. The six stragglers are **rounding on bills under about ₹1,000**, not errors. Total tax ₹6,98,932.00, an overall 2.904%.

Nothing is wrong here. It is recorded because the column names invite the mistake, and the ₹8.96 lakh it produces is large enough to be believed.

93. 14% of the stock movement trail refers to tags the tag master does not contain — MEDIUM (source property, not a migration fault)

Cross-checking the stock screens. `mcj_stock_items` holds **2,816** tags; `mcj_stock_movements` references **3,314** distinct barcodes. **499** of them exist on no tag, carrying **1,298** movements — **14% of the 9,036**.

It is not the migration

Checked before writing it up as a gap:

- `legacy.barcode_main` holds **2,816** rows and **2,816** were loaded. Complete.
- `legacy.barcode_wise_all_transaction` references **3,314** distinct barcodes of its own accord.

The legacy tag master already held 498 fewer barcodes than its own movement trail. Both were copied faithfully. Formatting was ruled out too: of the 499, only **2** match when trimmed and case-folded, and only **3** are suffix variants (94751337R1) of a barcode that exists.

What the orphaned movements are

Movement	Count
NEW-BAR	1,099
OP-STK	134
SALE-GST	42
CST-ORD	15
SALE-RET	5
PURC-IN	1

Dated **1 April 2024 to 14 August 2026** — up to nine days before the capture, so not an ancient tail. The bulk is tag creation for tags the master no longer carries, which reads as the legacy system removing a tag row at some point in its life while keeping the trail.

The sales are traceable, but not through the tag

31 distinct barcodes were sold with no tag behind them. **24 appear on a sales invoice line**, so the sale is fully recorded. The remaining 7 name a bill in the movement's own reference — and those bills exist and carry lines, except `GST/2526/00308` , **which is in no invoice table at all**. One bill from the 2025-26 year, referenced by a movement, with no invoice behind it.

Why it is recorded rather than fixed

Nothing can reconstruct a tag the source never kept. What this changes is what may be *claimed*: any statement of the form "every movement belongs to a tag" is false by 14%, and a stock report that inner-joins movements to tags silently drops 1,298 rows. The registers already outer-join, which is why none of them is wrong today.

94. The books are one financial year deep; the stock trail is three — HIGH

Following `GST/2526/00308` — a bill named by a stock movement and present in no invoice table. It is not a missing row. **The source contains no 2025-26 sales at all.**

Every financial table in the dump holds one year:

Table	Coverage
<code>sales_vou</code>	224 bills, all 2026-2027

Table	Coverage
voucher_main	1,840 in 2026-2027, plus the 8 orphans of blocker B4
old_gold_purchase	25, all 2026-2027
customer_order	284, all 2026-2027

The stock movement trail does not:

barcode_wise_all_transaction	Rows
2024-2025	2,138
2025-2026	4,317
2026-2027	794
blank	1,312

6,455 of its 8,561 rows — 75% — predate the only year the books cover.

(That 8,561 plus 475 rows drawn from retail_all_stock is the 9,036 in mcj_stock_movements . Checked, because a count that nearly matches is worth less than one that does.)

What it means concretely

38 tags were sold between 9 May 2024 and 21 October 2025, and not one has a bill on file — the movement is the only evidence the sale happened. The remaining 286 SALE-GST movements are in 2026-27 and reconcile to invoices.

The legacy system evidently purged or archived transactional tables at year end and left the stock trail alone. What survived is a shop with a detailed three-year stock history and a single year of accounts.

What may not be claimed

- **No year-on-year comparison is possible.** There is one year of revenue. Any "last year" figure would have to be invented.
- **The opening balances are the only trace of everything before April 2026** — which is why blocker B19's ₹15.32 crore imbalance cannot be investigated by looking at prior transactions. There are none to look at.
- A stock report joining movements to sales will find nothing for 75% of the trail, correctly.

Recorded, not fixed. Nothing can recover a year the dump does not contain.

95. Purchases reconcile exactly, and the ₹2.98 I first saw was my own arithmetic — LOW (a trap, not a fault)

Cross-checking purchases the way sales were checked.

Source	Amount
mcj_purchase_invoices.total	2,39,56,798.00
Credited to suppliers on the vouchers	2,39,56,798.00
	gap 0.00, on all 43 bills

The false start

The first comparison put invoice `total` against **total voucher debits** and found ₹2.98 across the 43 — small enough to look like accumulated rounding, and the per-bill pattern reinforced it: `GDP/2627/00004` showed 7,98,202.00 against 7,98,202.36, with a `round_off` of -0.36 sitting right there.

The voucher tells a different story:

Ledger	Dr	Cr
ZM LIFESTYLE PVT. LTD.		7,98,202.00
ROUNDED OFF		0.36
INPUT IGST	23,248.61	
PURCHASE_B2B	7,74,953.75	

The supplier is credited the **rounded** total, and the rounding sits as its own credit line. Debits exceed the bill total by exactly the round-off **because the round-off is on the other side**. Comparing a bill total to one side of a double-entry voucher counts it twice.

This is the second instance today of the same shape — after treating `making_charge` and `certification` as additions to a sales `bill_total` when they were already inside `goods_amount` (defect 92). Both produced a plausible figure from a wrong comparison rather than from wrong data.

Nothing is wrong with the purchases. Recorded because the wrong comparison is the natural one to reach for.

Noted while there

`PURCHASE%` ledgers carry a net debit of **₹2,76,65,794.29** against the ₹2,39,56,798.00 of purchase invoices. That is not a discrepancy either: those ledgers also carry old gold (`PURC-OG` , 25 vouchers) and purchase returns (`PURC-RET` , 3), which have no purchase invoice. Not decomposed to the rupee here — flagged so the next person does not read it as one.

96. `amount_received` on an order is not money received — HIGH (trap; the loader is correct)

Cross-checking orders. They migrate cleanly: **284 orders and 366 lines against 284 and 366 in the source, no orphaned lines**. But `advance` is **₹0.00 on all 284**, which is worth a second look on an order book worth lakhs.

The source has another column beside it: `amount_received`, populated on **273 of 284 orders and totalling ₹14,47,10,446.00.**

That is ₹14.47 crore of apparent customer money, on a shelf worth ₹14.24 crore and an order book worth ₹60,14,200.75. Loaded as an advance — which its name invites — every order screen would show money the shop has not got.

What it actually is

The largest single value is **₹7,38,00,000.00**, on order `C0/2526/00402` :

pcs	net weight	rate	estimate	status
3	3,000.000 g	24,600.00	0.00	Cancelled

$3,000 \times 24,600 = \mathbf{73,800,000}$ exactly. It is a computed goods valuation, on a cancelled order carrying three kilograms of gold.

Tested across all 267 orders where both sides exist:

Formula	Orders matching
net weight $\times$ rate + making	178
net weight $\times$ rate	157
gross weight $\times$ rate	157
the line <code>estimate</code>	16

Two thirds land on goods-plus-making. **Nothing about it is a receipt.**

The loader is right, and was right by reading rather than by luck

`load_orders.sh` takes `o.advance` and ignores `amount_received`. `advance` is genuinely zero on every row, so **the order book correctly shows no advances.**

The real question underneath

284 orders and not one advance recorded. Either the shop takes no money up front — unusual against ₹60,14,200.75 of live orders — or advances are receipted as ordinary `RCPT` vouchers with no link back to the order. If it is the second, the order screen cannot show what a customer has already paid, and the shop would have to find it in the ledger by name.

Recorded for the shop. Not guessed at.

97. `item_adjustment` looked like a lead on B9 and holds nothing usable — LOW (candidate closed)

B9 is 165 items whose stored price cannot be reproduced, suspected to carry a pricing step this rebuild cannot see. `item_adjustment` — **922 rows, read by no loader** — carries `per_of_making`, `rt_mk_pr_gm` and `pl_minus_gold`, which is exactly the shape of a missing per-item making rule.

It is not one.

Only 5 of the 922 rows are populated. The other 917 are zeros — the same pattern as the abandoned default columns of defect 74, where fill rate looked like evidence and distinct count was.

The five:

item	making %	rate/gm	pl_minus_gold	in item_master ?	tags on shelf
657	10.0	—	12.0	no	—
735	8.5	—	—	no	—
1066 GOLD ORNAMENTS	8.0	250.0	10.0	yes	o
1066 GOLD ORNAMENTS	15.0	—	—	yes	o
1066 GOLD ORNAMENTS	8.0	250.0	10.0	yes	o

- **Two reference items that do not exist** in `item_master`.
- The other three are the **same item, with contradictory rules** — 8% and 15%.
- That item is `GOLD ORNAMENTS`, the raw stock item of defect 78, which **carries no tags at all**, so no priced piece could be affected by any of it.

B9 is not explained by this table, and the candidate is closed rather than left open. Recorded because a 922-row table with the right column names is exactly what a future search for B9's missing step will find again.

98. A third stock log exists and adds nothing — LOW (candidate closed)

`barcode_history` — **3,083 rows, read by no loader** — is a tag transaction log with the same shape as the two that *are* loaded: barcode, transaction type, bill number, date. Spanning 1 April 2024 to 19 August 2026 over 2,824 barcodes, it looked like a third source and therefore like a gap in the movement trail.

Every one of its 3,083 rows is already in the loaded trail, matched on barcode, movement type and date. It is a duplicate view, and the loaders are right to ignore it.

The hallmark figures re-checked while there

It carries `ISS-HM` and `REC-HM` rows, and the hallmark count has been wrong once already — reported as 8 from a single log before the second was found. Re-derived from the loaded trail:

	barcode_wise_all_transaction	retail_all_stock	total
ISS-HM	4	2	6
REC-HM	4	2	6

Twelve, as corrected. And the headline reproduces exactly: **1,023 gold jewellery tags on the shelf, 739 with no hallmark record, ₹11,54,63,861.86**, counting purities 22, 18, 14 and 9 and excluding bullion.

One figure needed its scope confirmed rather than assumed. The screen contrasts those 12 movements with **"432 tags flagged as hallmarked"**, and an in-stock gold-jewellery count gives 284. Both are right:

Scope	Tags
All tags, flag set	432
In stock	329
In stock, gold jewellery	284

432 is every tag ever flagged, which is the correct comparison against a movement trail that also covers sold tags. Left as it is.

99. The voucher renumbering log cannot be used as a map, and 1,411 vouchers were renumbered — MEDIUM (candidate closed, context for B4)

`voucher_transfer_check` — **1,445 rows, read by no loader** — maps a voucher's old id to a new one per type and year, with a timestamp and a user. It looked like the authoritative answer to blocker B4's eight orphaned vouchers, which the loader currently handles by matching on `legacy_v_id` and hoping.

1,411 of the 1,445 rows record an actual renumbering (`v_id` $\neq$ `new_vid`). Voucher renumbering was not an eight-row exception in this system; it was routine.

But it does not resolve to a map

Rows	1,445
With a blank financial year	5
Old ids pointing at several different targets	2
Target numbers claimed by more than one source voucher	123

JRNL alone shows `v_id` 3 $\rightarrow$ 6 and `v_id` 4 $\rightarrow$ 6, and `v_id` 4 $\rightarrow$ 6 and $\rightarrow$ 1.

The table carries a `datetime`, so the obvious reading is append-only with the latest entry winning. **Tested: taking the latest row per type, year and old id gives 1,442 mappings and leaves all 123 collisions standing.** It is a log of transfer events, not a reconciliation, and nothing in it says which entry prevailed.

The load does not need it

`load_vouchers.sh` ignores it and matches on `legacy_v_id`. That produces 1,848 vouchers and 5,229 lines exactly, **0 lines with an unresolved ledger**, and 1,847 of 1,848 balancing — the one exception being the known `JRNL 312`. Adding a map with 123 ambiguous targets could only make that worse.

What it does explain

B4's eight orphans are not a curiosity. Legacy `JRNL` ids 1–8 became 649–655 and 1, and the same reassignment happened 1,411 times across the system. It is also the backdrop to defect 66, where **83 of 135 bill links point at a number that is also a real bill for a different sale** — a shop that renumbers vouchers this freely will produce exactly that.

Candidate closed. Recorded so B4 is not re-investigated through this table.

100. Row count is not information: three more tables that are full and empty — LOW (candidates closed)

Working through the legacy tables no loader reads — **54 of them, 25,674 rows between them**. The large ones have each been examined and written up already (`design_no_status`, `barcode_image`, `cla_retail_trans_bar_detail`, `order_rate_master`, `voucher_transfer_check`, `barcode_history`, `address_book_sub_name`, `item_wise_tagging`, `sale_bill_link`). These three were the remaining ones big enough to be worth opening:

Table	Rows	Populated
<code>ledger_ref_details</code>	1,071	2 have a reference name, 1 a mobile
<code>item_adjustment</code>	922	5 , and those contradict each other (defect 97)
<code>voucher_hsn</code>	756	0 — <code>hsn_no</code> is 0 on every row

`voucher_hsn` is the cleanest case: 756 rows, one distinct value, no HSN recorded anywhere. A feature switched on, written to on every voucher, and never given anything to record.

The pattern, now seen five times

Counting the earlier ones — `address_book_master`'s TDS and brokerage columns (**100% filled across 40,519 rows, one distinct value each**, defect 74) and `sales_quotes` (**a table named like customer quotations holding motivational sayings**, defect 77) — this is the fifth instance today of a table that is full of rows and empty of data.

Row count is not information. The checks that separate them are cheap and should come before any claim about a table's contents:

- **distinct count**, not fill rate — 40,519 filled rows with one distinct value is a default, not data
- **look at the values**, not the column name — `sales_quotes`, `amount_received` and `item_merge` are all named for something they do not contain

Each of these was opened because a name or a row count suggested it mattered. None did. Recorded so the next sweep does not open them again.

101. A figure that drifts on its own, in a document that does not — LOW (now dated)

Cross-checking blocker B23. Every figure reproduces from the database exactly:

	Documented	Recomputed
Enrolments, all active	239	239
Instalments paid	2,144, ₹87,90,900.00	same
Gold promised at maturity	₹1,03,10,400.00	same
Enrolments already matured	235	235
Gold due on those matured	₹99,74,900.00	same
Oldest maturity	25 July 2017	same

One did not: **"matured more than a year ago — 215"** recomputes as **216**.

It is not an error. It was 215 when written on 22 August and 216 today, because one enrolment — party 10262, maturing 23 August 2025 — crossed the one-year line overnight. Verified by asking the database for both dates: 215 as at 22 August, 216 as at 23 August.

Why it is worth writing down

A static document holding a figure defined *relative to today* is wrong from the day after it is written, and the person who notices has no way to tell drift from a mistake. They will either correct a correct document or — worse — start distrusting the figures around it, which here include a ₹99.7 lakh liability that is exactly right.

The row now carries **"(as at 22 Aug 2026)"** and a note that it moves. The other B23 figures are stable until a scheme matures or is settled, and are left alone.

Worth a sweep of the other documents for the same shape: anything counted "so far", "to date" or "more than a year ago" needs the date it was taken.

102. The sweep for drifting figures found two more, in the same defect — LOW (now dated)

Following defect 101 through the other documents. **41 statements combine a number with a time-relative phrase**; most are "at today's rates" or narration and are fine. Three are counts defined against *now*, and they had already drifted:

Figure	Written 22 Aug	Recomputed 23 Aug
Scheme enrolments matured over a year ago (B23)	215	216

Figure	Written 22 Aug	Recomputed 23 Aug
Live orders past their due date (defect 28)	61	63
Age of the oldest overdue order	1,474 days	1,475

Everything stable in those entries reproduces exactly: 67 live order lines, 12 over a year past, ₹49,77,590.00 of live work, ₹17,10,000.00 of it over a year old, oldest due 9 August 2022 — and B23's 239 enrolments, 2,144 instalments, ₹87,90,900.00 paid and **₹99,74,900.00 due**.

So in one day, one defect entry gained two wrong numbers without anybody touching it. The order one is worse than the scheme one because **the count is in the title** — *"Sixty-one live customer orders are past their delivery date"* — where it will be quoted from.

All three now carry (*as at 22 Aug 2026*) and a note of what they read today, in `blockers.md`, `data-defects.md` and `screens.md`.

The rule this leaves: a number measured against *today* belongs with the date it was taken, or it belongs in a screen rather than a document. The screens recompute; the documents do not.

103. The status table is now taken rather than typed, and taking it found another 28-screen drift — MEDIUM (now automated)

Every figure in `docs/STATUS.md`'s summary table was hand-maintained, and every one of them has been wrong:

- **"20 of 188 screens"** stood while the true figure reached 117 — and the denominator counted the CSV's header row as a screen (defect 75)
- **"11 open blockers"** while 18 were open
- **migrations and loaders wrong twice in one day**, the second time within hours of correcting the first — `guard_app_data.sh`, a helper, counted as a loader

`tools/status_figures.py` now prints the table from the filesystem, the navigation registry and the database. `--check` compares it against what is written and exits 1 on a difference.

It immediately found one more

"Screens written up in `screens.md` : 41" — the true figure is **69**. The headings cover ranges (`## Screens 34–35` is two screens, `## Reference lists (screens 1–10)` is ten), so counting headings gives 36 and counting them by hand gave 41. Distinct screen numbers: 69, spanning 1 to 82.

And the first version of the check passed when it should not have

It searched the whole document for each value. `docs/STATUS.md` said 41, the script computed 69, and **the check reported agreement** — because "69" appeared somewhere else on the page.

Now it locates the row by its label and compares that row's value, allowing a trailing note (`**91 of 149**` — 2 more examined and deliberately not loaded) to count as agreement rather than drift.

Third vacuous check written today, after a field-label test that covered only schemas with no UUID selects and a smoke test that reported `ok (cached)` against a deliberately broken server. The pattern is the same each time: a check that cannot fail is indistinguishable from one that passes, and only trying to make it fail tells them apart.

104. Three tests would have passed while checking nothing — MEDIUM (now guarded)

Defect 103 ended on the observation that a check which cannot fail is indistinguishable from one that passes. Applying that to the test suite.

84 test files. Of the tests whose every assertion sits inside a `range` loop, most iterate a hard-coded table of cases and cannot be empty — `TestWords`, `TestAcceptsEveryRealLegacyCode`, and so on. Those are fine. The ones that matter loop over something **fetched**: a registry, a query, a spec.

Checked individually rather than by the heuristic, because the heuristic over-flagged badly — 80 candidates, then 24, and most of those turned out to assert on accumulated state *after* the loop, which fails correctly on an empty result. `TestHolidays_ShowsWhichYearsAreRecorded` is the pattern: it collects into a map, then asserts 2016 and 2017 are present.

Three were genuinely vacuous-on-empty:

Test	Loops over	Passes empty because
<code>TestAuth_EveryBusinessRouteRefusesAnonymous</code>	routes built from <code>mcj.CodeLists</code>	an empty registry means no route is checked
<code>TestReconcile_TickingRecordsTheState mentDate</code>	voucher ids from the database	a seed that produces nothing clears nothing
<code>TestRawStock_CarriesNoValuation</code>	the report's declared columns	a spec with no columns has no money column

The first is the one that matters. **It asserts that no business route serves an anonymous visitor** — and it would have reported success having tested zero routes.

Each now fails with a message saying what it failed to prove. Verified on the security one by emptying the registry: it fails with *"no routes to check, so this proves nothing about anonymous access"*, and passes again when restored.

The other two were guarded without a live failure being staged, because their emptiness would require a code change that would fail other tests first.

105. The pipeline audit found nothing, and my checking was wrong twice in the finding of it — LOW (clean result, method noted)

Applying the vacuity question to the loaders: does every one verify what it actually wrote, or do some just check that a tenant exists?

All 37 do. There are 95 guard statements across the pipeline, and every loader compares what it loaded against the source — `load_vouchers.sh` asserts 1,848 and 5,229, `load_codelists.sh` compares source to destination for each of its lists, `load_masters.sh` runs every count through a `check` function that exits on a mismatch.

Nothing to fix.

The part worth keeping is how badly the checking went

Two crude patterns, two false alarms:

- Grepping for `FAIL` reported **17 loaders with no assertion**. They use `ERROR:` and `exit 1` instead. `load_coin_rates.sh` compares source rows to loaded rows and exits — it simply does not contain the word.
- The corrected pattern still flagged three, all artefacts: `load_all.sh`'s guard is an `exit 1` inside an `if` rather than after a `||`, and the other two compute their counts into shell variables that no longer look like `mcj_` table references.

Fourth time today the check was wrong rather than the thing checked — after querying `status='Cancel'` where the data says `Cancelled`, joining on `category_id` where the column is `category`, and comparing a bill total against one side of a double-entry voucher.

Each produced a confident, wrong result that looked like a finding: *17 loaders have no assertion* would have been a serious claim about this pipeline. The only thing that caught it was opening one of the seventeen and reading it.

A negative result from a pattern is a claim about the pattern until one case is read by hand.

106. Any user could delete the shop's own particulars — HIGH (now fixed)

Auditing permissions across every write route. Two crud screens declare fewer permissions than they expose operations:

Screen	Declared	Omitted
<code>company.go</code>	Read, Create, Update	Delete
<code>tagprint.go</code>	Read, Update	Create, Delete

An omitted permission is not a closed door. `CrudController.accessDenied` :

```
if perm == nil { return false }
```

A permission that was never set denies nothing, and the route is registered on a separate flag — `enableDelete`, which **defaults to true**.

`tagprint.go` is safe, but not for the reason its permissions suggest: it also calls `WithoutCreate` and `WithoutDelete`, so those routes never exist. Remove either line and the screen opens to anybody.

`company.go` **did not**. A user holding only `CompanyRead` issued `DELETE /mcj/company/{id}` and received **303 with the row deleted** — verified, not inferred. That row is the shop's **name, GSTIN and address**: the particulars printed on every invoice it issues.

Fixed

`WithoutDelete` on the company screen. The shop's own particulars should not be deletable through the interface at all — there is exactly one such row and every bill depends on it. The `DELETE` route is now unregistered rather than merely guarded, so no future permission slip can reopen it.

Two tests hold it: the read-only user now gets **405** where they got 303 and a deletion, and a second records that `tagprint`'s safety comes from `WithoutCreate` rather than from its permission list.

The general shape

A missing permission reads like a restriction and behaves like an open door. The other 16 crud screens declare all four and are unaffected; the nine non-crud write routes were checked too, and the two POSTs gated on a read permission — tag label and stock take — write nothing, so a read right is correct there.

107. Every report was served to any logged-in user, whatever their role — HIGH (now fixed)

Following defect 106 into the report screens. `navigation.Screen.Permissions` carries a declaration for each of the 70 reports, and its own comment says what it does:

Permissions gate visibility. A menu entry nobody can open should not show.

Visibility. `ReportController.Register` mounted every report behind authentication middleware and **no permission gate at all**. So the declarations hid menu entries from users who were still served the page at its URL.

Measured, with a user holding one unrelated permission (`ColorRead`):

Report	Response
<code>/mcj/reports/salary-sheet</code>	200
<code>/mcj/reports/trial-balance</code>	200
<code>/mcj/reports/user-rights</code>	200

The salary sheet carries **basic pay, gross, paid and deducted, per person**, defaulting to everyone — 246 rows across 19 people. The trial balance is the whole of the books. The user-rights report is who may do what.

How it was nearly missed

The first thing found was an inconsistency: the salary sheet declared `ReportRead` while the sibling payroll master declared `PayrollRead`. Changing the declaration was a one-line fix and it would have achieved **nothing** — because the declaration only hides a menu entry. The real question was whether the route was gated at all, and it was not.

Tightening a declaration that turns out to be decorative is worse than leaving it: it looks like the hole was closed.

Fixed

Each report route is now gated on the permissions its navigation entry declares, with `ReportRead` as the floor for anything that declares none — so a report added without a menu entry is closed rather than open. `gateAll` requires every listed permission rather than any, because a screen declaring two is declaring both are needed. The salary sheet's declaration is now `PayrollRead` and, being enforced, means something.

Nobody loses access today: there is one role, Administrator, holding all 171 permissions — which is exactly why it is worth fixing before a second exists. Full suite green, and the admin still reaches all 136 pages.

108. The rest of the route surface is gated, checked rather than assumed — LOW (clean result)

Completing the sweep that found defects 106 and 107. Every `Register` in the module was read for routes mounted without a gate.

One is ungated, and correctly so: `home.go` serves `/` to any authenticated user. It is 81 lines with no SQL, no money and no party data — it reports rebuild progress against the legacy menu. Checked rather than assumed, because a landing page that had grown a shelf figure would have been exactly the same shape of leak as the reports.

The generic crud handlers were read in the SDK rather than trusted:

Handler	Checks
List	readPerm
Details	readPerm
GetEdit	updatePerm

All three consult the right permission — which is why defect 106 mattered so much and defect 107 so much more. The crud screens were **correctly gated on permissions that were mostly declared**; the reports were **not gated at all**, so their declarations never ran.

Where that leaves the route surface

- **70 report routes** and their `/export.xlsx` twins: gated on the navigation entry's declared permissions, floor of `ReportRead` (defect 107)

- **18 crud screens:** gated by the SDK on declared permissions, all four now declared or the operation disabled outright (defect 106)
- **20 hand-registered routes:** gated, and the two write routes behind a read permission write nothing
- **1 landing page:** open to any authenticated user, by design

Nothing further outstanding on this surface. What is *not* covered by any of this is authorisation **within** a screen — a user who may read the ledger book may read every ledger in it. That is how the legacy system worked and is not a regression, but it is worth stating that no screen filters rows by who is asking.

109. SQL is built by concatenation in 29 files, and none of it takes user input — LOW (clean result, verified twice)

Report parameters reach SQL. Searching for query strings built by concatenation rather than binding found **33 sites across 29 files**, which is the sort of number that looks like a finding.

Every one is safe, and for four distinct reasons:

- **Package constants.** `openingSQL` in `trialbalance.go` and `balancesheet.go` is a `const` in this repository.
- **Fixed literals chosen by a branch.** `returns.go` sets `goodsSide`, `taxSide := "l.debit"`, `"l.debit"` and swaps both to `"l.credit"` in one place. The value can only ever be one of two strings written here.
- **Slice iteration.** `negativestock.go` ranges over `stockFaults`; `dataquality.go` selects from `dataChecks` by index with a nil guard.
- **An explicit whitelist.** `movements.go` interpolates a column name and guards it with `switch column`, defaulting to `%q is not a filterable column` — with a comment saying the name comes from the file and never from a request.

The `where` clauses in `estimates.go`, `customergoods.go`, `categorise.go` and `voucherprint.go` append a fixed fragment when a select parameter is not its default. The parameter chooses **which literal**, never the text.

Verified against the running server, not only by reading

Four payloads through report parameters:

Parameter	Result
<code>ledger=' OR 1=1--</code>	200, no rows, nothing executed
<code>as_on=2026-08-23';DROP TABLE mcj_ledgers;--</code>	refused by the date parser
<code>show=all' UNION SELECT null--</code>	200, value ignored
<code>from=' OR 'a'='a</code>	200, no rows

785 ledgers still here. The one that reached a message was rejected at parameter validation rather than at the database, which is the right place.

Recorded because "SQL built by string concatenation in 29 files" is true, alarming and wrong as a conclusion — the next person to grep for it deserves the answer rather than the count.

110. The template layer escapes, and nothing bypasses it — LOW (clean result, verified live)

templ escapes interpolated values by default, so the question is only what bypasses it.

templ.Raw **appears nowhere in the module.** There is no hand-written HTML path at all.

templ.SafeURL **appears 18 times**, and it does bypass URL sanitisation — a javascript: URL passed through it would render as written. So every assignment to the props it wraps was traced: **24 assignments to Action, CancelURL, NewURL and Href, and none comes from a request.** They are path constants (RawStockPath, CategorisePath), constants joined to a compiled slug (TradePath + "/" + kind.Slug), or navigation entries built at start-up.

Checked live as well

Reading tells you what the code does today. The Own Raw Stock screen has a free-text purity field — added this morning — so it was posted with <script>alert(1)</script> and the page read back:

Raw <script>alert(1)</script> in the response	0
Escaped <script>	1

Escaped, as templ promises. The test row was deleted afterwards; the table holds its 10 migrated rows and 0 app-entered rows.

Nothing to fix. Recorded so the next audit knows the SafeURL count was looked at rather than counted, and that the escaping was confirmed against a running page rather than assumed from the framework's documentation.

111. The admin password is published in the repository, and nothing said so — MEDIUM (now warned)

Scanning the 706 tracked files for credentials. .env is gitignored and untracked, and there is exactly one hit:

```
Justfile:19:  seed-admin email="admin@mcj.local" password="wingold123":
```

README.md repeats it — "creates admin@mcj.local / wingold123" — and so does the just start banner. That is three places, all of them the first things a person reads.

On a laptop it is a convenience. On a server reachable by anyone else it is a **published credential on an ERP holding 40,519 customer records**, and nothing in the tooling or the documentation said so. just start runs seed-admin with no arguments, so the default path creates it.

Now

`seed-admin` compares the password it was given against the documented one and prints to **stderr** when they match:

WARNING: "wingold123" is the documented development password. It appears in the Justfile and the README, so anyone who can read this repository knows it. Before this server is reachable by anyone else, re-run with -password.

The constant is declared once, next to the check, so the warning cannot drift away from the recipe that uses it. Verified both ways: the default triggers it, `-password smoketest123` does not.

The README now carries the same warning at the point where it tells you to sign in.

Not removed. A default that lets someone start the system in one command is worth keeping; a default nobody is told about is not. This is a deployment question, and it belongs beside blocker **B1** — which is still open on whether this ever faces the internet at all.

112. Database errors are shown to the user, verbatim — MEDIUM (recorded as a trade-off, not fixed)

62 places across 14 files put `err.Error()` in front of the person using the screen. Where the error came from the database, that is the database's own text.

Seen during the injection testing, when a report query was deliberately broken:

This report could not be run: estimates: ERROR: relation "mcj_estimates_typo" does not exist (SQLSTATE 42P01)

A jeweller reads that and learns nothing. Anyone else reads a table name, a schema convention and a SQLSTATE code.

Why it has not been changed

It is a real trade-off rather than an oversight, and both sides are worth stating:

- **For keeping it.** The shop can quote the message when reporting a problem, which is worth a great deal for a system with one administrator and no support desk. It is also what made the pipeline's failures diagnosable all day.
- **Against.** It describes the schema to whoever is looking, and the `TestEveryMenuPageAnswers` smoke test greps for `could not be run:` — so the test currently depends on the leak.

The standard remedy is to log the detail server-side and show the user a short message with a reference to quote. That is a **62-site change** touching every error path in the module, and it would break the smoke test's detection until it is rewritten alongside.

The condition that decides it

While this runs on the shop's own machine, the exposure is to the shop. If blocker **B1** resolves toward a cloud VPS reachable from the internet, this becomes information disclosure to strangers and should be changed before that happens rather than after.

Recorded rather than done, because it is a decision about deployment and support rather than a fault in the code — and because a 62-site refactor made without that decision would be the wrong kind of thorough.

113. Every money value exported to Excel was text, so nothing could be summed — MEDIUM (now fixed)

The `/export.xlsx` routes had never been exercised. All four tested produce valid `Microsoft Excel 2007+` files with the right contents — the estimates export carries both bill numbers and both customers, matching the screen.

But every money cell was a shared string. The trial balance exported **1,205 cells, 964 of them text and not one numeric money cell**. Its Debit column read `4262.51` as *text*.

An accountant exports a trial balance to add it up. Selecting that column in Excel gave no total, and every cell carried the "number stored as text" warning.

The mechanism

`rowSource.GetRows` did `out[j] = row[c.Field]`, handing the raw value to the writer. A `rupees.Amount` is a struct with no case in that writer, so it arrived as its string form. Columns already holding a Go `int64` — like the raw stock `Lines` count — came out as real numbers, which is why the behaviour looked inconsistent between reports.

`Money: true` and `Numeric: true` affect the **screen**. They never reached the export.

Fixed

`exportValue` converts a `rupees.Amount` to `float64(a.Paisa()) / 100`. Paisa are `int64` and a `float64` holds every integer to 2^{53} , so no real amount loses anything. Divided rather than formatted — a formatted string is what caused this.

Numeric cells in the trial balance went from **241 to 481**, and the exported file now totals **₹29,58,46,599.44 on both sides, difference 0.00** — the same figures as the screen, including the suspense line.

Noted, not fixed

Weights are still text. They arrive as strings from `to_char(...)` in the query rather than as numbers, so the same conversion cannot reach them. Money was the column with a reason to be summed; weights are worth revisiting if anyone totals them.

(Two of my own counts were wrong on the way here: "241 numeric cells" in the trial balance were style and index cells, not money, and a first regex reported zero numeric cells at all. Fifth time today the check was wrong rather than the thing checked.)

114. The exports are correct, and my counting was wrong for the sixth time — LOW (clean result)

Following defect 113: does an export honour the filters applied on screen, or dump the whole report? An accountant who filters to one supplier and exports everything would not necessarily notice.

It honours them, and matches the screen row for row:

View	Screen data rows	Export rows
Designs, default	500	500 + header
Designs, <code>?show=all</code>	500	500 + header
Designs, filtered to one item	<i>No designs match.</i>	the same message

The export URL carries the request's query string, so the filtered export runs the filtered query. Even the empty state travels: the sheet contains the row reading "*No designs match.*" rather than a bare header.

The sixth wrong check

The first comparison used `grep -oE '<tr'` **across the whole page** and reported 502 screen rows against 501 exported — a consistent off-by-one that looked exactly like the export dropping a totals row. It was counting `<tr>` in the parameter form and the page furniture along with the report table.

Counting only the rows inside `<tbody>` that contain a `<td>` : **500 and 500.**

That is the sixth time today the check was wrong rather than the thing checked, and the second time this exact mistake was made — the categorise flakiness (defect 86) was also `grep -c` counting lines rather than rows.

The pattern is specific enough to name: counting markup with a text pattern over a whole document. Every instance produced a plausible discrepancy in a place where a real one would have mattered.

115. The 40,519-row lists are paginated, searchable and fast — but never say how many matched — LOW

`mcj_parties` holds **40,519 rows**. A screen that renders them all would be unusable, so the list screens were checked.

They are fine:

Screen	Rows served	Response
/mcj/address-book	25	0.06 s
/mcj/ledgers	25	0.05 s
/mcj/items	25	0.05 s

Paging is **infinite scroll**, not links: the last row carries `hx-get="...?limit=25&page=2"` with `hx-trigger="intersect once"` and `hx-swap="afterend"`, so the next 25 append as you reach the bottom.

Search works. The parameter is `Search`, capitalised — `?Search=TRISHANA` returns one row, `?Search=SHAW` twenty-seven. That matters more than paging here: reaching row 40,000 by scrolling would take about **1,600 fetches**, so search is the only practical way to find a customer, and it is there.

What is missing

No total is displayed. Searching "SHAW" fills a screen and gives no indication whether that is 27 matches or 2,700 — and with infinite scroll there is no last page to arrive at either. A shop looking up a customer cannot tell whether to refine the search or keep scrolling.

The list is rendered by the SDK's crud table, so this is not a line in this repository. Recorded rather than attempted: it is a small usability gap in a component we do not own, and it sits behind a real question — whether the shop searches by name often enough for the count to matter.

116. A sale made in the new system appears in neither the sales register nor the GST return — HIGH

The billing screen had only ever been smoke-tested for a 200. A real sale was put through it: customer `KIRAN ASWANI`, one tag, `64091403` — a silver locket.

The pricing is right. 0.250 g net at today's rate is 63.38, making 25.00, goods **88.38**; CGST 1.33 and SGST 1.32 (3% split in half, uneven by a paisa because rounding is honest), rounded off –0.03, **91.00 to the customer**. It prices at today's metal rate rather than the tag's stored 43.75, which is correct for metal.

The accounting is right. A balanced `SALE-GST` voucher:

Ledger	Dr	Cr
KIRAN ASWANI	91.00	
SALE		88.38
CGST		1.33
SGST		1.32
ROUNDED OFF	0.03	

It reaches the day book, the ledgers and the trial balance. The tag leaves the shelf.

What it does not do

Row in <code>mcj_sales_invoices</code>	none
Row in <code>mcj_stock_movements</code>	none
Appears in the sales register	no — 0 rows for the day
Appears in the GST return (<code>gstr-sales</code>)	no

The registers and the returns read `mcj_sales_invoices` , which only the migration fills. So a bill raised in this system collects GST from a customer and **does not appear in the return that reports it**, and the day's takings cannot be reviewed on the screen built for that.

The stock trail is the second half: `in_stock` flips, but no movement is written, so the tag's history ends at the day it was created. That is the same shape as defect 93 — a trail that does not account for every disposal — except this one we are creating ourselves.

Not fixed here

This is the Tier 0 billing path and the fix is a design decision, not a patch: either the sale writes an invoice row alongside the voucher, or the registers and returns are rebuilt to read vouchers. The first is smaller and keeps the legacy shape; the second is truer to double entry. **It should be decided before the shop bills anything real**, because every bill raised before it lands is a bill missing from a GST return.

*(The test sale was reversed: voucher and its five lines deleted, `in_stock` restored. Verified afterwards — 1,848 vouchers, **0 app-created**, 2,575 tags, ₹14,23,61,188.20.)*

117. Nothing the application does writes a detail row, so the registers freeze at migration — HIGH

Defect 116 found the sale screen posting a voucher and no invoice. That is not one screen's omission. **The entire module has four insert paths:**

File	Writes
<code>infrastructure/persistence/voucher_repository.go</code>	<code>mcj_vouchers</code> , <code>mcj_voucher_lines</code>
<code>presentation/controllers/partysample.go</code>	<code>mcj_customer_goods</code>
<code>presentation/controllers/rawstock.go</code>	<code>mcj_raw_stock</code>
<code>presentation/controllers/tag.go</code>	<code>mcj_stock_items</code>

Nothing writes `mcj_sales_invoices` , `mcj_sales_invoice_lines` , `mcj_purchase_invoices` , `mcj_purchase_contents` , `mcj_old_gold_lines` or `mcj_stock_movements` . Confirmed against the data — every one of those tables is entirely migrated:

Table	Rows	Written by the app
<code>mcj_sales_invoices</code>	224	0

Table	Rows	Written by the app
mcj_purchase_invoices	43	0
mcj_old_gold_lines	25	0
mcj_stock_movements	9,036	0

(475 movements carry no legacy_id , which looked like app-written rows until checked: they come from retail_all_stock , a migration source with no id of its own.)

What that means after go-live

- **GST returns** report the 224 migrated bills forever; every new sale is missing (defect 116). *(The sales register is NOT affected — it reads vouchers. That half of defect 116 was a 404, corrected in defect 137.)*
- **Purchase register** the same for the 43.
- **Old gold** posts the money and never records the metal — the gross, net, touch and rate that defect 40's loader existed to recover.
- **The stock trail stops.** Its last entry is 19 August 2026. Tags will keep leaving the shelf, because in_stock flips, but nothing records that they did. Within a month the movement history and the shelf disagree, with no way to reconstruct the difference.

The accounting is unaffected throughout: vouchers and lines are written properly, so the day book, the ledgers, the trial balance and the balance sheet stay correct. **The books will be right and the registers will be empty.**

Not fixed

Same decision as defect 116, one level up: either each posting screen writes its detail rows alongside the voucher, or the registers are rebuilt to read vouchers and the movement trail is written from the same place stock is flipped.

This is the largest open item in the rebuild and it is a design choice, not a bug to patch quietly. **It belongs in front of the shop before the first real bill.**

118. The voucher screens are complete, and defect 117 does not reach them — LOW (clean result)

Defect 117 says posting screens write a voucher and no detail row. The voucher screens themselves are the highest-volume path in the shop — **626 payments and 269 receipts** in the migrated year — so they were put through end to end too.

A payment was posted through /mcj/vouchers/payment/new : two lines, 500.00 against an expense ledger and 500.00 out of a bank account.

Voucher created	PMNT 631 , dated 23 Aug 2026
Lines	GENERAL CHARGES Dr 500.00 , CASHFREE PAYMENTS Cr 500.00

Balances	yes
Appears in the day book	yes

Defect 117 does not apply here, and the reason is worth stating: a voucher is its own detail.

There is no separate table a payment should also have written. The sale, purchase and old gold screens post a voucher *in addition to* something the registers read; the receipt, payment, contra and journal screens post the only record there is.

So the money-in and money-out path — the one the shop uses most — is complete today.

*(The cash book did not show the voucher, which looked like a second finding for a moment. It was my ledger choice: name LIKE 'CASH%' matched **CASHFREE PAYMENTS PVT.LTD**, not the cash account, so the cash book was right to omit a payment that never touched cash. Eighth wrong check of the day.)*

The test voucher was reversed — two lines and the voucher deleted. Verified: **1,848 vouchers, 0 app-created, 5,229 lines, movement difference one paisa**, as before.

119. The same tag cannot be sold twice, and the refusal says why — LOW (clean result)

A jeweller's worst billing error is selling one piece to two customers. The sale screen was tested for it: the same tag, 64091403 , billed twice in succession.

Two attempts, one voucher. The second was refused.

It is refused at **pricing**, before the save guard is reached, and the message names the piece in plain language:

*These tags are not on the bill and must be priced by hand: 64091403 SILVER LOCKET — **not on the shelf — already sold or issued***

There is a second guard behind that one, which matters because pricing and saving are separate requests and a tag can be sold in between:

```
UPDATE mcj_stock_items SET in_stock = false
WHERE tenant_id = $1 AND id = ANY($2) AND in_stock
```

It then compares `RowsAffected()` to the number of tags on the bill and refuses the whole sale if they differ —

N of M tags were no longer on the shelf — someone else may have sold them

The `AND in_stock` is what makes it safe: the update cannot clear a tag twice, so two tills racing on the same piece produce one sale and one refusal rather than two bills. The code says so in its own comment: *"better than billing a piece twice."*

Nothing to fix. Recorded because it is the failure a jeweller would care about most, and because it holds under the race rather than only in sequence.

Both test sales were reversed — **1,848 vouchers, 0 app-created, 2,575 tags, ₹14,23,61,188.20.**

120. The one duplicate barcode is benign, and the schema does not stop the next one — MEDIUM

Blocker **B8** records a duplicate barcode. It is still live, and it is the only one: **2,575 tags on the shelf carry 2,574 distinct barcodes**, and across all 2,816 tags there are 2,815.

86065435 is on two in-stock tags. The sale resolves a barcode with:

```
WHERE s.tenant_id = $1 AND s.barcode = $2
ORDER BY s.in_stock DESC LIMIT 1
```

— which picks one of the two arbitrarily. That would matter a great deal if the pair differed. **It does not.** Every priced field is identical:

gross	net	rate	amount	making	total	item	purity	pieces
0.000	0.000	.00	.00	.00	900.00	same	same	1
0.000	0.000	.00	.00	.00	900.00	same	same	1

Two identical **MALA** pieces at ₹900.00, created the same day. Scanning the barcode sells one; the second stays on the shelf and can be sold on its own merits. **Nobody is billed the wrong amount and no piece is sold twice** — the arbitrary choice costs nothing because there is nothing to choose between.

What is not protected

The tag screen refuses a barcode *"already on a tag that is still on the shelf"*, which is the right rule — a barcode belonging to a sold tag may legitimately be reused, and this shop reuses them.

But **there is no database constraint**. `mcj_stock_items` carries a primary key on `id` and nothing on `barcode`, so the rule lives only in one controller. A loader, a script, or a second screen written later can create a duplicate without meeting it — and the next pair may not be two identical ₹900 malas.

A partial unique index on `(tenant_id, barcode) WHERE in_stock` would express it in the schema. **Not added here:** it would reject the existing pair, so it needs that pair resolved first, which is a question for the shop — are there two malas on the shelf, or one counted twice? That is B8, and it is still the right question.

121. The credit note posts a correct reversal — LOW (clean result)

The refund path, untested until now. A wrong credit note gives money away, and the direction is the whole of it.

Priced: goods 1,000.00, CGST 15.00, SGST 15.00, **payable 1,030.00** — 3% split in half, as on a sale.

Posted as SALE-RET 4 , continuing from the three migrated sale returns:

Ledger	Classification	Dr	Cr
KIRAN ASWANI	asset		1,030.00
SALE_RETURN	income	1,000.00	
CGST	liability	15.00	
SGST	liability	15.00	

Every direction is the mirror of a sale. The customer is **credited**, so what they owe falls. SALE_RETURN is an income account **debited** — contra-revenue, which is why the profit and loss nets returns off rather than adding them. The tax ledgers are **debited** to give back the liability the sale created. It balances at 1,030.00 both ways.

Worth stating because the opposite would look almost identical on screen and would quietly inflate revenue while crediting the customer twice.

Defect 117 applies here too: no row is written to any register table, so the return will not appear in the sales register or the GST return either. The accounting is right and the register is silent, as with the sale.

Test voucher reversed — **1,848 vouchers, 0 app-created, 3 SALE-RET**, as migrated.

122. Old gold records the money and loses the metal — HIGH (the sharpest case of defect 117)

The shop buying gold back over the counter. Priced: 10.000 g at 22kt, today's rate ₹14,253.00 a gram, **₹1,42,530.00 payable to the customer.**

Posted as PURC-OG 26 :

Ledger	Classification	Dr	Cr
PURCHASE_OLD GOLD	expense	1,42,530.00	
KIRAN ASWANI	asset		1,42,530.00

Balanced, and the direction is right — the shop owes the customer for metal it has taken in.

mcj_old_gold_lines **stayed at 25**. The ten grams were not recorded anywhere. Not the weight, not the purity, not the rate that produced the price.

Why this is the sharpest case

load_oldgold.sh exists because the original migration recovered the money and lost the metal. Its own header says so:

The money for these was already here — the PURC-OG vouchers tie to 50,71,370.00 exactly. What was missing is the METAL: gross and net weight, the touch test, and the rate paid. For a jeweller taking gold back over the counter that is the number the trade runs on.

That loader was written to fix exactly this gap in the migrated data — **and the live screen reproduces it for every new purchase**. A shop that buys back a kilogram over a month will have paid for it correctly and have no record of receiving it: the metal does not reach stock, the weight is not stored, and the touch that justified the price is gone.

Recorded under blocker **B26** with the rest of defect 117. Of the six affected screens this is the one where the missing row is not merely a register entry but **the goods themselves**.

Test voucher reversed — 1,848 vouchers, 0 app-created, 25 PURC-OG, 25 old gold lines.

123. The purchase screen behaves exactly as predicted, which is worth having checked — LOW (clean result, prediction confirmed)

The insert audit said the purchase screen would post a voucher and no invoice row. Predictions have been wrong eight times today, so it was tested rather than assumed.

Posted as PURC-GSC 25 , continuing from the 24 migrated:

Ledger	Classification	Dr	Cr
JAI KRISHNA JEWELLERS CO.PVT.LTD.	liability		5,150.00
PURCHASE	expense	5,000.00	
INPUT CGST	liability	75.00	
INPUT SGST	liability	75.00	

Balanced, and correct in a detail worth naming: the tax posts to **INPUT** CGST and SGST, not the output accounts a sale uses. Input tax is recoverable and output tax is owed; a screen that confused them would file a GST return claiming credit it is not entitled to, and the voucher would still balance.

`mcj_purchase_invoices` **stayed at 43** — defect 117 confirmed here too, so the purchase register will not show it.

Why record a confirmed prediction

Because the prediction being right is a fact about this codebase, not a foregone conclusion. Of the eight wrong checks today, several began as confident readings of code that the data then contradicted. **Reading the insert paths told me what should happen; posting a purchase told me what does.**

Five of the six B26 screens have now been exercised end to end. Only the debit note and purchase-against-an-order remain, and they share the credit note's and the purchase's controller respectively.

Test voucher reversed — 1,848 vouchers, 0 app-created, 24 PURC-GSC, 43 purchase invoices.

124. The debit note and purchase order are correct, and the ninth wrong check was one I had warned myself about — LOW (clean result)

The last two B26 screens, completing the set of six.

Debit note — PURC-RET 4, the exact mirror of the purchase:

Ledger	Dr	Cr
JAI KRISHNA JEWELLERS	2,060.00	
PURCHASE_RETURN		2,000.00
INPUT CGST		30.00
INPUT SGST		30.00

The supplier is **debited**, so what the shop owes falls. PURCHASE_RETURN is credited as contra-expense, and the input tax is **given back** rather than claimed — which is the half a wrong implementation would get wrong while still balancing.

Purchase against an order — PURC-GCO 20 : supplier credited 2,000.00, PURCHASE debited 2,000.00, and **no tax lines at all**.

That looked like an omission. It is not:

Legacy voucher type	Vouchers	Carrying input GST
PURC-GCO	19	0
PURC-GSC	24	24

The screen reproduces the shop's own pattern exactly.

The ninth wrong check, and this one I had warned myself about

The first query said **all 19** PURC-GCO vouchers carry input GST — the opposite answer. It joined voucher_detail to voucher_main on v_id alone:

```
WHERE d.v_id = m.v_id -- and nothing about the type
```

v_id is not unique across voucher types. Defect 83 measured that: legacy.voucher_detail holds 5,229 rows with **1,186 distinct ids**, reused freely. Defect 99 found 1,411 renumberings on top. I wrote both entries today and then joined on the key they describe as ambiguous.

Adding d.v_type to the grouping gives 0 and 24. **A key documented as unsafe is not safe because the person joining on it wrote the documentation.**

All six B26 screens are now exercised end to end. Every test voucher reversed — **1,848 vouchers, 0 app-created, 5,229 lines, 3 PURC-RET, 19 PURC-GCO, 2,575 tags.**

125. Every way the checking went wrong today, in one place — REFERENCE

Nine investigations produced a confident wrong answer because of how the question was asked, not because of the data. They are scattered through the entries above; gathered here because the next person will reach for the same shortcuts.

#	What was asked	What it returned	Why
1	<code>status = 'Cancel'</code> on order lines	0 of 366	the value is <code>Cancelled</code>
2	<code>join</code> <code>address_book_sub_name</code> on <code>category_id</code>	error, then 0	the column is <code>category</code>
3	invoice total against voucher debits	a ₹2.98 gap on 43 bills	the round-off sits on the credit side; comparing a document total to one side of a double entry counts it twice (defect 95)
4	<code>grep FAIL</code> in the loaders	17 with no assertion	they use <code>ERROR:</code> and <code>exit 1</code> (defect 105)
5	<code>grep -c '<tr'</code> for report rows	every report looked empty	templ minifies a table onto one line, so it counted lines , not rows (defect 86)
6	<code>grep '<tr'</code> across a whole page	screen 502 against export 501	it counted the parameter form and page furniture too (defect 114)
7	<code>sum(DISTINCT total)</code> for B24	₹10,63,93,299 instead of ₹10,73,39,445	it sums distinct values , dropping tags worth the same (defect 81)
8	<code>name LIKE 'CASH%'</code> for the cash ledger	a payment missing from the cash book	it matched CASHFREE PAYMENTS PVT.LTD (defect 118)
9	<code>join voucher_detail</code> on <code>v_id</code> alone	all 19 PURC-GCO carry GST — the opposite of the truth	<code>v_id</code> is reused across types: 5,229 rows, 1,186 distinct ids (defects 83, 124)

Three more of the same family, caught before they became claims:

- **Fill rate read as evidence.** `rate_tds` is 100% filled across 40,519 rows and holds **one distinct value**. Reporting "202,595 dropped values" would have been badly wrong (defect 74).
- **A whole-document search in a `--check . status_figures.py --check`** reported agreement while `STATUS.md` said 41 and the truth was 69, because "69" appeared elsewhere on the page (defect 103).
- **A regex counting style cells as money.** "241 numeric cells" in the trial balance were nothing of the kind (defect 113).

What they have in common

Every one produced a **plausible** figure, in the place where a real defect would have mattered, and several were large enough to be believed: ₹8.96 lakh unexplained, 17 unguarded loaders, an export dropping a totals row.

Three defences worked, and only these three:

1. **Read one case by hand.** Opening a single "unguarded" loader ended #4.
2. **Reconcile against a second source.** The narration text settled the cheque dates; the weight settled the shelf column; the voucher lines settled #3.
3. **Try to make the check fail.** Emptying the registry exposed a vacuous security test; breaking a report exposed a cached smoke run.

A pattern that returns a number has not answered the question. It has answered a different question that may happen to have the same shape.

126. The record itself checks out — REFERENCE (clean result)

The documentation is now half the deliverable, so it was checked the way the code is.

Defects defined	125 , numbered 1–125
Gaps in the numbering	none
Blockers defined	26
Broken <code>defect N</code> references	0
Broken <code>BN</code> references	0

Every cross-reference between `STATUS.md`, `data-defects.md`, `blockers.md`, `screens.md`, the README and the plan resolves to something that exists.

*(The checker flagged `B001` four times as an unknown blocker. It is a **branch code** in the legacy data — defect 5 is about `br_code` being written both as `1` and as `B001`. A `B\d+` pattern cannot tell a blocker from a branch. Tenth false positive of the day, and the mildest.)*

Alongside it, the state of the build at this point:

- `go build ./... clean, gofmt clean`
- **Full integration suite passing, uncached**
- **117 of 187 screens**, 69 written up
- **77 migrations, 37 loaders, 85 test files**
- **91 of 149 legacy tables read**
- `tools/status_figures.py --check` agrees with the filesystem

The pipeline rebuilds the whole database from the capture in one command, exit 0, 87 OK lines, reproducing 2,575 tags at ₹14,23,61,188.20, 1,848 vouchers and 593 cheque references exactly.

127. Stock take is correct, and it matters more because of defect 117 — LOW (clean result)

Defect 117 leaves the movement trail frozen at 19 August 2026: tags will keep leaving the shelf and nothing will record that they did. **Physical verification then becomes the only way to know what is actually there**, which raises this screen from a periodic chore to the fallback.

It was given four barcodes — two real tags on the shelf, one belonging to a tag already sold, and one invented:

Scanned	4
On the books	2,575
Not found	2,573 — ₹14,23,34,155.40
Not on the books	2 — the sold tag and the invented one

The arithmetic ties to the paisa: the shelf is ₹14,23,61,188.20, the two tags actually scanned are worth ₹27,032.80, and the remainder is **₹14,23,34,155.40** exactly.

A sold tag and a barcode that never existed are both reported as *not on the books*, which is the honest classification — the screen cannot tell a mis-scan from a piece that came back.

What it refuses to do

*2,573 tag(s) on the books but not on the shelf — ₹14,23,34,155.40. Each is either mislaid, out with a karigar or on approval, or gone. **Nothing here is written off; somebody has to walk this list.***

It adjusts nothing. A stock take that silently wrote off ₹14.23 crore of unscanned tags would balance the books and lose the shop its stock record — and that is exactly what a partial count looks like on the first pass.

Nothing to fix.

128. The tax invoice omits the tax rate and the reverse-charge statement — MEDIUM (for the shop's accountant)

The invoice is what the customer physically receives, so it was rendered and read rather than pattern-matched. Two bills: one without item detail, one with.

It is comprehensive. Supplier name, address, **GSTIN** 19AADCM7443D1ZH and PAN; the customer with mobile, state and GSTIN field; invoice number and **date in full** ("18 April 2026"); **place of supply**; the goods value, the tax, the rounding and the total; **amount in words** — "*Rupees Thirty Five Lakh Twenty Thousand Nine Hundred Six Only*" — a declaration, CIN, terms, bank details and a signatory block. On a bill with lines, the item table carries # · Description · HSN · Purity · Pcs · Gross · Net · Rate · Making · Amount .

Two things are not there:

Missing	Where it is required
The rate of tax — the bill shows IGST 1,02,550.65 and never @ 3%	CGST Rules, rule 46(i)
Whether tax is payable on reverse charge	rule 46(m)

No percentage appears anywhere on either bill.

The legacy print screen (`sales_bill_print.php`) was **never captured** — the app generated it in JavaScript — so there is nothing to compare against and no way to tell whether the shop's own bills carried these.

Recorded rather than added. Both are one line of template each, but an invoice is a legal document and the wording is not mine to invent: "3%" against IGST is easy, and a reverse-charge line has to say the right thing. This is a question for the shop's accountant, alongside blocker **B3**, which already asks for a real printed bill to compare the layout against.

(Reaching this took four failed attempts to read the page — patterns that reported the GSTIN missing when it was 222 characters from the heading. Eleventh and twelfth false positives; the answer only came from printing the lines and reading them.)

129. The voucher print is correct, and it guards the trap that caught me three times — LOW (clean result)

The receipt a customer gets for money paid. Rendered `RCPT 1` of 2026-2027 and read it:

MANIK CHAND JEWELLERS & SONS PVT.LTD.
25A,CAMAC STREET, KOLKATA-700016
GSTIN: 19AADCM7443D1ZH

Receipt Voucher

No: RCPT 1
Financial year: 2026-2027
Date: 01 April 2026
With: TRISHANA SHAW

Account	Debit	Credit
TRISHANA SHAW		6,150.00
CASH	6,150.00	
Total	6,150.00	

Matches the database exactly: the customer credited 6,150.00, cash debited 6,150.00.

The detail worth noticing

It prints the **financial year**, and the controller says why:

A voucher number repeats across types and years, which is the same key trap that broke the migration's voucher load and the bank clearing dates. Without the year, "RCPT 3" is several different vouchers.

That is the same trap as defect 83 — `legacy.voucher_detail` holding 5,229 rows with **1,186 distinct ids** — and defect 99's 1,411 renumberings. **It is also the trap I fell into myself**, joining `voucher_detail` on `v_id` without the type and getting the opposite answer about input GST (defect 124).

So a receipt handed to a customer identifies the voucher unambiguously, while the person auditing the system wrote a query that could not. The product is more careful here than its author was.

Nothing to fix.

130. The tag label carries what a tag must and deliberately omits the price — LOW (clean result, B3 still open)

The last print screen. Printed a label for 31595399 :

31595399	Purity	18
[barcode image]	Gross	158.500
DIAMOND SET	Net	147.242
	Pcs	3

Every figure matches the database — gross 158.500, net 147.242 — and the barcode is a real Code 128 image embedded as a data URI, not a font that has to be installed on the till.

The geometry is a butterfly tag: **62mm × 16mm**, a 30mm left wing, a 2mm dashed bridge, a 28mm × 7mm barcode. That is not invented — tag_print_setup survived the capture with the shop's own panels, margins, fold centres, six-point type and a Zebra printer, and the controller says plainly which half is the shop's and which is reconstruction:

The label's GEOMETRY survived... What did not survive is which FIELDS sat where, so the arrangement below is a reconstruction from what a jewellery tag has to carry.

No price is printed, and that is right for this trade: the shelf is valued at today's metal rate, so a printed price is wrong the morning after. The piece above is worth ₹32,86,591.00 today and the label says nothing about it.

B3 stays open — a physical tag still has to be held against this to confirm the fields sit where the shop expects. Nothing here is wrong; it is unconfirmed, which is a different thing and already recorded as such.

The comment checks out against the data

The controller's description of the surviving setup was verified rather than taken on trust, because a comment can drift from the table it describes:

Column	Value
left_panel_margin	1
right_panel_left_margin	100
top_margin / right_panel_top_margin	15 / 3
tag_centre / long_tag_centre	130,34 / 130,24
font_size	6
printer_type	zebra

Exactly what the comment claims — "a left panel at 1, a right panel at 100, top margins of 15 and 3, fold centres at 130,34 and 130,24, six point type, a Zebra printer." The geometry on the label is the shop's own, to the millimetre.

(The first query guessed the column names and failed. Thirteenth wrong check, and the mildest — but it is the reason the values above were read rather than assumed to match.)

131. The shop's own system also computed a loss, and the working survived — HIGH (corroborates B19)

Both migration reconciliations pass — **154 tables against the ERP's own backup manifest, 0 mismatches; 149 staged tables against MySQL, 0 mismatches** — so the chain from backup to staging is verified row for row.

Five manifest tables are not staged, all `temp_*` scratch. Correctly excluded, but one of them is worth reading. `temp_pl` is the legacy system's own profit and loss working, six rows, left where it last ran:

Group	Type	Amount
Sales Account	income	2,11,39,143.00
Direct Income	income	0
Indirect Income	income	0
Purchase Account	expense	2,62,33,491.29
Indirect Expenses	expense	42,66,733.51
Direct Expenses	expense	11,458.75

Income **₹2,11,39,143.00** against expenses **₹3,05,11,683.55** — the shop's own ERP computed a **loss of ₹93,72,540.55**.

Why this matters

The profit and loss screen built here reports a shortfall of **₹1,10,87,494.08** and refuses to call it a profit or a loss, because a jeweller's cost of sales needs closing stock and this ERP carries none as a ledger (defect 79, blocker B19). That reasoning was inference from the data.

It is no longer only inference. The legacy system, given the same ledgers, produced the same shape of answer: a nine-figure sales line, a larger purchase line, and a net that reads as a substantial loss. The shop was not running at a ₹93 lakh loss; its own P&L said so because stock sits outside the accounts there too.

The two figures differ by **₹17,14,953.53**, which is expected and not reconcilable: `temp_pl` is a scratch table with no date stamp, written whenever the screen was last opened, while the figure here is bounded to 1 April–23 August 2026. What matches is the **shape**, and the shape is the argument.

For B19 this is the strongest evidence yet that closing stock was always held outside the ledgers rather than lost in migration — the previous system had the same hole and printed the same misleading total.

132. The legacy register carried a TCS column that was never used, and the rebuild has no provision for it — LOW (correct today, a threshold question for later)

Reading the rest of the excluded `temp_*` tables after `temp_pl`. They are the shop's own report scratch, and their columns describe the registers it ran. `temp_sale_register` and `temp_purchase_register` both carry:

```
... amount · discount · sgst · cgst · igst · total · **tcs** · total_1 · rnd_off ...
```

TCS — tax collected at source — which appears nowhere in this rebuild.

It was never used. Checked at every level:

Sales bills with a TCS amount	0 of 224
Sales bills with a TCS rate	0 of 224
Total TCS collected	₹0.00
Stock inward rows with TCS	0 of 43
Ledgers named TCS	0

`legacy.sales_vou` and `legacy.stock_inward` both have `tcs_per` and `tcs_amt` columns and both are empty throughout. So there is nothing to carry forward and nothing was lost.

Why it is worth writing down anyway

The provision exists in the old system because a jeweller can need it. The largest buyers in the migrated year:

Buyer	Bills	Total
SHISH JEWELS LTD	1	35,20,906.00
SHREE SAROJ JEWELSS	1	32,92,225.00
MANIK CHAND JEWELLERS	18	29,88,286.00
MANISH KUMAR SHARMA	1	20,65,000.00

No buyer reaches ₹50 lakh, so on these figures the question does not arise — which is presumably why the columns sit empty. But two of them are within striking distance on a single bill, and **the rebuilt system has no TCS field, no TCS ledger and no line on the invoice for it**. If a buyer crosses whatever threshold applies, that is a schema change and an invoice change, not a setting.

Recorded as a fact and a threshold, not as advice: whether TCS applies, and at what point, is for the shop's accountant. What is stated here is that **the old system could record it, this one cannot, and nobody has needed it yet**.

(`MANIK CHAND JEWELLERS` appearing as a buyer on 18 bills is presumably a branch or internal transfer. Not pursued.)

133. The raw metal is real, still moving, and my accounting inference about it was wrong — HIGH (corrects defect 78)

temp_tag_untag_report_new — 171 rows, one of the five scratch tables excluded from staging — is the shop's own **raw stock movement report**. Its opening rows are exactly the barcode_main_rawstone figures of defect 78: 82.156 g at 9, 1,073.05 at 14, 5,566.29 at 18.

It covers **1 April to 17 June 2026** and carries three document types:

Purity	Opening	Receive	Issue	Balance
9	82.156	86.498	19.402	149.252
14	1,073.050	448.003	129.687	1,391.366
18	5,566.290	82.377	148.830	5,499.837
22	5,058.310	1.900	102.660	4,957.550
o / blank	95.010	16.630	1.396	110.244
Total	11,874.816	635.408	401.975	12,108.249

11,998.005 g at gold purities, after 146 issues and 14 receipts — worth **₹15,76,50,889.76** at today's metal rates, purity by purity:

Purity	Balance	Rate today	Value
9	149.252 g	6,199.00	9,25,213.15
14	1,391.366 g	9,643.00	1,34,16,942.34
18	5,499.837 g	12,398.00	6,81,86,979.13
22	4,957.550 g	15,153.00	7,51,21,755.15
Total	11,998.005 g		15,76,50,889.76

The metal was not a historical remnant. It was being worked with — issued out and taken back in — eleven weeks before the system died.

What I got wrong

Defect 78 argued this could not be current stock, from the accounts:

At the shelf's blended ₹7,104.23 per gram the raw gold alone is ₹8,36,86,423.98... the trial balance's unexplained residual after tagged stock is ₹1.09 crore. If this metal were also held and unrecorded the books would be short by roughly ₹8.4 crore more than they are.

The conclusion does not follow — and **the ₹8,36,86,423.98 was itself wrong**, which was not known when this entry was written. ₹7,104.23 is the whole shelf divided by the whole shelf's weight, and **48% of that weight is silver**; it cannot be applied to gold. Priced purity by purity at today's metal rates the 11,998.005 g comes to **₹15,76,50,889.76**, not ₹8.37 crore. See the correction in [defect 78].

The reasoning fault stands on its own regardless of the figure. It assumed the books would show the metal if the shop had it. **They do not, and that is the whole of B19:** this ERP holds no stock in its ledgers, tagged or raw. Reasoning from an absence in accounts that are known not to record stock was circular, and I did not notice because the arithmetic was clean.

The honest reading is simpler and worse: **the shop holds roughly 12 kg of gold in raw stock that appears in no ledger, on top of ₹14.24 crore of tagged stock that appears in no ledger.** B19's hole is larger than the shelf, not smaller.

What follows

- Defect 78's inference is retracted; its measurements stand.
- The raw stock screen shows the **opening** figures, which is what `barcode_main_rawstone` holds. The 146 issues and 14 receipts are in a scratch table, not a real one, so there is nothing to load them from — the movement history exists only as a report someone happened to run.
- **The question for the shop is unchanged but sharper.** Not *"is this metal still there?"* — the report says it was in June — but **"where is it now, and who has it?"**

134. The migration's scope is provably complete, and the check that proved it only looked one way — MEDIUM (now fixed)

`verify_rowcounts.sh` compares every manifest entry against MySQL and reported **154 tables, 0 mismatches**. That is a real check and it was passing honestly.

It says nothing about a table that is **in MySQL and not in the manifest**. Such a table is never looked at, and the check passes.

The database holds **451 base tables** against **154 manifest entries** — so **297 are unlisted**. Every one was counted:

Unlisted tables	297
Of those, non-empty	0
Rows between them	0

So the manifest is exactly the set of tables that hold data, and nothing was missed. That is worth knowing precisely, because it closes the question that has been open all day in the background — *is there anything we have not looked at?*

The whole chain, verified:

Stage	Count
Base tables in the legacy MySQL	451
— holding any data	154
Row counts matching the ERP's own backup manifest	154, 0 mismatches

Stage	Count
Staged into Postgres	149 (the 5 omitted are temp_* scratch)
Staging matching MySQL row for row	149, 0 mismatches
Read by a loader	91
Examined and deliberately not loaded	the remainder, each written up

Fixed

The script now walks the other direction too: every base table absent from the manifest is counted, and a populated one fails the run with its name and row count. Today that finds nothing, which is the point — **it is a guarantee about the next dump, not a discovery about this one.**

(The five temp_* tables excluded from staging turned out to carry the shop's own P&L working and its raw-stock movement report — defects 131 and 133, the two most consequential findings of the day. "Excluded as scratch" was right for loading and wrong for reading.)

135. A third of "stones sold" is silver sold by weight — HIGH (figure corrected)

Following the excluded scratch tables into temp_sale_register, which listed 55 June bills. **17 of them show no item detail on the invoice** — the screen says so: "This bill carries no item detail in the migrated data."

The detail exists. sales_vou_main — the line table the loader reads — has **0 rows** for those 17 bills, while sales_vou_detail has 17. Reading one:

bill	barcode	item	gw	purity	fine wt	metal	rate	amount
GST/2627/00092	(none)	660	200.000	92.50	185.000	SLV	262.14	52,427.00

200 grams of silver sold by weight, untagged. sales_vou_main covers tagged pieces only, so bulk metal sales appear nowhere as invoice lines.

The consequence is worse than a missing line

load_sold_stones.sh loads **all 203 rows** of sales_vou_detail into mcj_sold_stones with no filter. So those metal rows are counted as stones:

	Rows	Carats	Value
Genuine stones	182	131.946	36,44,121.33
No carats — metal sold by weight	21	0.000	75,39,341.00
Total as loaded	203	131.946	1,11,83,462.33

By metal purity the metal rows are unmistakable: SLV 14 rows, 995 2 rows, SLV999 1 row — **17 rows with a metal purity and not one carat between them, ₹75,39,341.00.**

The figure published as "stones sold — ₹1,11,83,462.33" is three times the truth. Stones sold is ₹36,44,121.33. The carat figure, 131.946, was always right: metal rows carry no carats, so they inflated the money and left the weight alone — which is exactly why it survived being written down four times.

Corrected, not yet fixed in the loader

Every occurrence in the record now reads ₹36,44,121.33 with the correction noted. The loader is left alone deliberately: whether those 21 rows belong in `mcj_sold_stones` at all, or as invoice lines for the 39 bills worth ₹89,26,345.00 that currently show no detail, is the same question as blocker **B26** — what a posting writes and what a register reads. Splitting the table on a guess would move the error rather than remove it.

136. The purchase side already makes the distinction the sale side missed — LOW (the fix has a pattern to follow)

Defect 135 found metal sold by weight counted as stones. The symmetric question: does the purchase side make the same mistake?

It does not, and it never did. `mcj_purchase_contents` is fed from `stock_inward_main` and `stock_inward_detail`, and **28 of its 60 rows carry no carats** — the same shape that went wrong on sales. The report handles it:

- a kind **parameter** offering *Everything · Stones · Metal*
- a kind **column** on every row
- a footer that counts them apart — *"6 stone row(s), 1 metal row(s)"*
- a test, `TestPurchaseContents_KeepsCaratsAndGramsApart`, asserting stone rows are measured in `ct` and metal rows in `g`

Live, filtered three ways: **6 stone rows and 1 metal row**, and the unfiltered view shows exactly those 7.

Why this matters for fixing 135

The distinction between metal and stones is not something this codebase failed to understand. It is implemented, tested and surfaced on the purchase side, and absent on the sale side. **That makes defect 135 an oversight with a pattern to copy rather than a design question to settle** — whoever splits `mcj_sold_stones` has a working example twenty files away, including how to word the filter and how to measure each kind.

(The three filtered views first counted 9, 8 and 3 rows, which do not add up. Each count included the two footer rows. Fifteenth counting artefact of the day, same family as the rest: a row count taken from markup rather than from the data.)

137. Defect 116 was half wrong: the sales register does show new sales — HIGH (correction)

Defect 116 reported that a sale made in this system *"appears in neither the sales register nor the GST return"*, and blocker **B26** was raised partly on it.

The sales register half is wrong, and the cause was a 404.

The report key is `sales-register`. I queried `sale-register`:

URL	Response	Rows
<code>/mcj/reports/sale-register</code>	404	0
<code>/mcj/reports/sales-register</code>	200	224

An empty page from a wrong URL reads exactly like an empty register. Re-tested with a real sale posted:

Report	Rows for the day	The new sale
<code>sales-register</code>	1	present
<code>gstr-sales</code>	1	absent

`salesregister.go` **reads** `mcj_vouchers` **and** `mcj_voucher_lines`, so it sees anything the shop posts. The GST return reads `mcj_sales_invoices`, so it does not.

What is actually affected

Thirteen reports read one of the three migration-only tables (`mcj_sales_invoices`, `mcj_purchase_invoices`, `mcj_old_gold_lines`):

`gstr-sales` · `mrp-sales` · `mrp-purchases` · `sale-cost` · `sales-comparison` · `sales-targets` · `sold-stones` · `statistics` · `customer-metal` · `old-goods-register` · `order-to-barcode` · `purchase-contents` · `ledger-merge`

The GST return is the one that matters — a bill raised in this system still does not reach it, which is a compliance exposure and B26 stands on that alone. The day book, cash book, ledger book, trial balance, balance sheet, P&L **and the sales register** all read vouchers and are unaffected.

What I should have done

The claim rested on a single empty page. Two things would have caught it immediately: **checking the HTTP status** — a 404 was right there in the response — and **listing the report keys** rather than guessing one from the screen's name. Sixteenth wrong check of the day, and the most consequential: it escalated into a blocker and a caveat on the coverage figure.

138. Every other report path in the record is real — REFERENCE (audit after defect 137)

Defect 137 turned on a URL that did not exist. The obvious question is whether anything else in the record rests on one.

69 report keys are defined. Nineteen distinct report paths are cited across `STATUS.md`, `data-defects.md`, `blockers.md` and `screens.md`. **Eighteen of the nineteen are real keys.** The exception is `sale-register`, and it now appears in exactly three places — all of them describing the mistake:

- the defect 137 entry, as the 404 it was
- the B26 correction note
- the status log entry

Nothing else cites it as a working path.

That the rest hold up is partly luck and partly method: **most of the day's sweeps harvested their URLs from the rendered menu rather than typing them.** The 136-page smoke sweep, the 140-run parameter sweep and the export checks all took their paths from `href` attributes, so a wrong key was impossible by construction. The two hand-typed ones went wrong — `sale-register` (404, defect 137) and `gstr1` (404, caught at the time and corrected to `gstr-sales`).

Both hand-typed URLs were wrong. Neither harvested one was. That is the argument for `TestEveryMenuPageAnswers` harvesting its list rather than carrying one, and it applies to investigation as much as to tests: **ask the system what its paths are, do not tell it.**

139. All 69 reports run and none is silently empty — REFERENCE (clean result, harvested)

Applying defect 138's lesson to a sweep: the report keys were **harvested from the registry**, not typed, so a wrong path was impossible.

Reports defined	69
Returning HTTP 200	69
Failing to run	0
Awaiting a parameter by design	2
Returning data	67
Silently empty	0

The last row is the one worth having. A report that returns 200 with no rows and no explanation is indistinguishable from a broken one, and there are none.

Busiest: `tag-wise-stock` 2,575 rows, `incomplete-data` 1,090, `diamond-rates` 775, `stone-ledger` 673, `stock-journal` 500.

Leanest with data: `day-book`, `gstr-sales`, `gstr-purchases` and `mrp-purchases` at one row each, `estimates` at two — all small because the data is, not because anything failed. The day book defaults to a single day; the GST returns to a single period.

(This is the third full sweep of the day and the first taken from the registry rather than the rendered menu. The two agree: 69 report keys against 70 report pages in the menu — the extra is `order-book` appearing twice, once as itself and once as Customer Repairing with `?kind=RP`.)

140. The quote report reprices correctly, and shows how stale a tag's stored value is — LOW (clean result)

The last of the 69 reports never seen with data. Quoted 31595399 , a diamond set:

Metal	147.242 g net at ₹12,398.00 per g	18,25,506.32
Stones	carried from the tag	24,76,760.00
Making	₹1,500.00 per gram of net weight	2,20,863.00
Price before tax		45,23,129.32

Every line checks: $147.242 \times 12,398 = 1,825,506.32$; $147.242 \times 1,500 = 220,863.00$; the three sum to 4,523,129.32 exactly.

What it shows about the shelf

The same tag stores total ₹32,86,591.00 at a stored rate of ₹4,000.00 per gram. Today's quote is ₹45,23,129.32 — ₹12,36,538.32 more, because the 18kt rate has gone from ₹4,000 to ₹12,398 since the tag was priced.

That is a 3.1× move in the metal rate, and it is the clearest illustration of why several figures in this record carry the phrase "at today's rate":

- the shelf's ₹14,23,61,188.20 is the tags' stored values, priced when each tag was made
- a quote, a sale and the stock-at-today's-rates screen all reprice the metal
- so the shelf's book value and what the stock would fetch are different numbers, and neither is wrong

Nothing to fix. Recorded because a reader comparing ₹32,86,591.00 on the tag against ₹45,23,129.32 on the quote would reasonably suspect one of them — and both are right.

141. Most of the "shelf is worth half as much again" comes from an opening-book rate, not a price rise — MEDIUM

Ran Stock at Today's Rates for the first time with the totals reconciled:

	Book value	Value today	Difference
Rate-bearing tags (1,607)	13,74,96,960.20	20,67,35,594.36	+6,92,38,634.16
Fixed price (968)	48,64,228.00	48,64,228.00	0.00
Shelf	14,23,61,188.20	21,15,99,822.36	+6,92,38,634.16

The book column totals ₹14,23,61,188.20, matching the shelf figure this record has carried all along — an independent confirmation of it.

The report's arithmetic is sound

Worth stating, because I doubted it and was wrong. The tag's metal value is `rate x net_weight` and the report reprices exactly that, leaving stones and making alone. On stone-set tags `amount` already contains the stone value, so the identity is `amount - stones = rate x net_weight` — which holds for **478 of 478** such tags, to the paisa. No tag is mispriced by this report.

But the baseline is not what the screen implies

358 of the 360 tags carrying a stored rate of exactly **₹4,000.00** are flagged `is_opening`. That is an opening-stock valuation — a round number the shop's books opened at — and it sits across **five different purities** (18, 22, 14, 9, 95) over **19 months** (2024-04-01 to 2025-11-21). One rate cannot be the market price of 18kt and 22kt on the same day, let alone for a year and a half. It was never meant to be a market rate.

Those tags are **34%** of the rate-bearing book value but produce **57% of the entire uplift**:

	Tags	Book value	Uplift produced
Stored rate = ₹4,000.00	360	4,69,44,332.28	3,94,01,031.93
Every other tag	1,247	9,05,52,627.92	2,98,37,602.23

So the larger half of "the shelf is worth half as much again" is **the distance between an opening book entry and today's gold price**, not the gain on those pieces since they were bought. The remaining ₹2,98,37,602.23 is the part that reflects an actual price movement against a real tagging rate.

Nothing here can be repaired from the data: `mcj_metal_rates` **begins 2026-04-01**, and every one of these tags was created in 2024 or 2025. There is no rate history reaching back to when they were tagged, so what gold actually cost on those dates cannot be recovered from this database.

A visible symptom

107 tags carry a stored rate **above** today's rate for their purity, so the report marks them **down by ₹7,28,869.28**. 91 of them are 14kt and 9kt, and **every one of those 91 is stone-set**, with implied fine-gold rates reaching ₹26,333/g (14kt) and ₹38,755/g (9kt) against today's ₹16,531/g. A historic *market* rate cannot sit 1.6x-2.3x above today's when gold has risen throughout. Those are loaded in-house rates for stone-set mounts, and repricing them to a market rate is not meaningful. It is why the **9kt line reads as a loss of ₹1,55,823.65** on screen — an artefact, not a fall in value.

What to do

The figure is not wrong, but "the shelf is worth half as much again" should not be quoted without the split above. Ask Rahul what the 2024 opening rate was meant to represent before this screen is shown to anyone valuing the business. Relates to [defect 140] and [defect 79] — the tag's stored value is a stale price, and this is the same fact at portfolio scale.

142. There is no accounting history behind 87% of the stock, and it is not recoverable — HIGH

Chasing [defect 141] into the ledgers produced the reason B19 has never tied out.

The books begin in March 2026

The legacy `voucher_main` holds **1,848 vouchers in total**. Every one is filed under `ac_yr` **2026-2027** (eight carry a blank year), and the **earliest voucher of any kind is dated 2026-03-09**.

That is the whole accounting history the shop possesses. It is not a migration gap — our `mcj_vouchers` holds **1,848** rows, matching the legacy table exactly, so nothing was dropped on the way in. The source system simply does not retain vouchers beyond the current financial year.

The shelf begins in 2016

Shelf stock	Tags	Book value	Net weight
Created before the first voucher (2026-03-09)	2,305	12,43,38,210.80	18,777.098 g
Created after it	270	1,80,22,977.40	1,261.838 g
Total	2,575	14,23,61,188.20	20,038.936 g

87.3% of the shelf by value predates the first voucher in the books. Those 2,305 tags cannot have a purchase document behind them, because no document from that period exists anywhere in the system.

The tags themselves say so. Their `voucher_no` values reference financial years **1617, 1920, 2021, 2122, 2223, 2324, 2425** and **2526** — years for which both databases hold **zero** vouchers. And **83% of shelf value** (2,207 tags, **₹11,84,27,092.95**) entered through `created_from='stk'`, a direct stock entry citing an `STK/NNNNN` number that carries **no financial year at all** and matches nothing in the ledger set.

Why this matters

- **B19 is explained.** Stock is not merely held outside the ledgers; for 87% of it the accounting record that would place it inside them **no longer exists**. The opening trial balance cannot be reconstructed from documents.
- **It is unrecoverable.** Not a load to re-run. The rows are absent from the shop's own MySQL, and `newgst.in/mcj` went to 404 on 2026-08-21, so there is no second copy to go back to.
- **Provenance stops at the tag.** For ₹12.43 crore of goods, what the shop paid and who it bought from cannot be established from this data. Only the tag's own stored value survives — and [defect 141] shows that value is an opening-book figure for 360 of those tags.

What to do

Ask Rahul whether purchase history exists **outside** this ERP — paper invoices, a previous system, an accountant's books. If it does, that is the only route to provenance for ₹12.43 crore of stock. If it does not, the opening stock figure has to be asserted rather than derived, and B19 closes as *by design* rather than as a defect to fix.

143. Eight reports print a footer that counts their own summary row a second time — HIGH

The `report.Spec` comment warns about exactly this: *"a report containing both groups and their totals would count each figure twice and print a footer nobody can reconcile."* That is `NoTotals`, and eight reports that emit their own summary row never set it. The shell treats the summary row as data and adds it to the detail rows it already summarised.

Every one of these was read off the running screen:

Report	Its own summary says	Footer prints	
stone-ledger	6,04,28,535.00	12,08,57,070.00	2x
hallmarking	11,54,63,861.86	23,09,27,723.72	2x
opening-stock	8,51,58,772.85	17,03,17,545.70	2x
customer-metal	50,71,370.00	1,01,42,740.00	2x (4 columns)
ordered-stones	16,36,662.00	32,73,324.00	2x
sold-stones	4,17,856.40	8,35,712.80	2x
mrp-sales	1,800.00	3,600.00	2x (3 columns)
purchase-contents	14,38,899.00	42,76,377.00	2.97x

`purchase-contents` is the worst because it carries **two** trailing summary rows ("*6 bill(s)*" and "*Billed for these purchases*"), so its value column is counted three times: $14,38,899.00 + 14,38,899.00 + 13,98,579.00 = 42,76,377.00$.

`opening-stock` is the useful cross-check. Its summary row, **₹8,51,58,772.85**, is exactly the opening-stock book value computed independently against the database in [defect 142]. The summary row is right; the footer is wrong.

`order-to-barcode` **is NOT affected** and was checked: it also emits a summary row, but leaves the money cell blank, so its footer (₹66,11,878.00) equals the sum of its 35 detail rows. Correct as it stands.

Fix: `NoTotals: true` on the eight. Applied — see below.

Incomplete — see [defect 160]. Two further reports, `sale-cost` and `mrp-purchases`, had the same fault and were missed here, because the sweep read only the *last* body row and `sale-cost` carries two summary rows.

144. Three reports total a per-unit rate — MEDIUM

Distinct cause, same footer. `Column.Money` makes a column both money-formatted and *summable*, and three reports mark a per-unit rate as money:

Report	Column	Footer prints
stone-ledger	Rate a carat	2,67,79,476.58

Report	Column	Footer prints
sold-stones	Rate a carat	5,73,301.92
purchase-contents	Rate	1,63,234.11

₹2,67,79,476.58 is 673 per-carat rates added together. It is not a quantity of anything — no stone costs it, no total contains it — and it sits in the footer in the same style as the genuine value total beside it.

Rates are averaged, not added, and an average of rates weighted by nothing is also misleading. The right answer is to print nothing.

Fix: a `NoTotal` flag on `report.Column` that keeps money formatting but excludes the column from the footer. Applied — see below.

143 / 144 — fixed and verified

- `NoTotals: true` on the eight reports that carry their own summary row.
- A new `report.Column.NoTotal` keeps a money column out of the footer while leaving its money formatting intact; set on the three per-carat rate columns.
- Re-fetched all 69 reports from the running app: the eight wrong footers are gone, **no doubled footer remains anywhere**, and the 32 reports with a legitimate footer still have one — including `order-to-barcode`, which was correct before and is untouched.
- Three unit tests in `report_totals_test.go`. They were checked by removing the `NoTotal` guard and confirming the rate test fails with the summed `"50,000.00"` — they catch the regression rather than passing vacuously.
- The Excel export writes no totals row, so it never had either defect.
- Full suite green, integration included.

145. Three reports stop at 500 rows without saying so, and two of them total the truncated set — HIGH

Found by grepping the report queries for `LIMIT` after [defect 143]. Three carry `LIMIT 500`. Nothing on the screen mentions it, and each renders **exactly 500 rows** — the tell that the cap is binding, not coincidental.

Report	Rows in the data	Shown	Footer says "Total"	True total	Missing
designs	5,221	500	9,58,73,534.80	14,95,13,508.73	5,36,39,973.93
stock-journal	1,242	500	3,50,96,512.36	10,88,46,402.18	7,37,49,889.82
stock-history	9,036	500	(no money column)	—	8,536 rows

Both footers were reproduced exactly by re-running each query's own `ORDER BY` with `LIMIT 500` against the database, so this is the cap and nothing else. The `designs` footer shows **64%** of the real figure; `stock-journal` shows **32%**.

The word on the screen is **"Total"**. There is no "showing 500 of 5,221", no truncation notice, and because `designs` orders by estimate descending it shows the 500 *most valuable* designs — which makes the partial total look convincingly large.

The one that matters most

`stock-history` hides **8,536 of 9,036 movements**. That is the register that answers "*where did this tag go*" — the exact question [defect 78] and [defect 133] leave open about the 12 kg of raw gold. Anyone opening it today sees the most recent 500 movements and no sign that 94% of the history is missing.

Fix: each of the three now runs a second aggregate over the *whole* set and ends with a summary row carrying the true count and true totals, with `NoTotals` set so no partial total is printed. Applied — see below.

145 — fixed and verified

Each of the three now runs a second aggregate over the **whole** selection and ends with a summary row carrying the true count and totals, with `NoTotals` set so no partial total can be printed:

Report	Now says	Checked against
<code>designs</code>	5,221 design(s) · estimate 14,95,13,508.73 · making 1,30,80,066.59 · stones 6,21,12,124.33	database, to the paisa
<code>stock-journal</code>	1,242 piece(s) · 10,88,46,402.18	database, to the paisa
<code>stock-history</code>	9,036 movement(s) · "8,536 earlier movement(s) are not on this page"	database

`designs` also reports stones at **₹6,21,12,124.33**, which matches the "₹6.21 crore of stones" written in that file's own header comment — an independent corroboration of both.

146. `barcode` is used as a join key but is not unique, and one duplicate exists — MEDIUM

Caught while checking the fix for [defect 145]: the new `stock-history` count came back **9,040** against a table holding **9,036** movements.

The cause is a fan-out. `mcj_stock_items` has a **non-unique** index on (`tenant_id`, `barcode`) — only `legacy_id` is unique — and one barcode is on the shelf twice:

legacy_id	barcode	description	value	voucher	in stock
96399	86065435	MALA	900.00	STK/12518	yes
96409	86065435	MALA	900.00	STK/12519	yes

Two separate stock entries on consecutive vouchers, same day, same item, same value — two identical MALA pieces that were given **the same barcode**. That barcode has 4 movements, and $4 \times 2 - 4 =$ the 4 extra rows.

Why it is worth recording

- **Scanning 86065435 is ambiguous.** Two pieces answer to it. A sale, a transfer or a stock check on that number cannot say which one it touched.

- **Every join on barcode inflates.** Reports join stock on barcode rather than on id, so any of them can double-count this tag. Today the impact is 4 rows; it grows with every future duplicate, and nothing in the schema prevents one.
- The **shelf value is not affected** — these really are two pieces worth ₹900 each, so ₹1,800 is correct. The defect is the key, not the money.

Fixed here: the `stock-history` count is now `count(DISTINCT m.id)` and reports 9,036. **Not fixed:** the underlying duplicate, and the absence of a unique constraint. A unique index cannot simply be added — it would reject this row, and the shop may legitimately need to keep both pieces until someone re-tags one.

Needs Rahul: re-tag one of the two MALA pieces, after which `(tenant_id, barcode)` can be made unique for goods still in stock.

This now blocks a second defect. [Defect 181] found that `tag.go`'s check-then-insert cannot prevent two tills creating the same barcode, and the partial unique index that would close it is rejected by **this row** — the only barcode with more than one in-stock copy. One re-tag unblocks both.

147. The category tagging screen stops at 200 tags, and 80 of them cannot be reached at all — MEDIUM

Same class as [defect 145], found by sweeping the controllers for `LIMIT`. The cap itself is deliberate and the code says so — 2,575 checkboxes is not a page anyone can work through — but the screen never mentioned it.

What it showed before the fix:

Filter	Tags matching	Rows shown	Page said
none	2,575	200	"1114 tags on the shelf have no category"
GOLD CHAIN	280	200	the same 1114 line
SILVER TOPS, untagged	241	200	the same 1114 line

The one count on the page is the **global** uncategorised total. It does not respond to the filter, so a user who filters to GOLD CHAIN sees 200 rows and a number, 1,114, that has nothing to do with them.

80 tags that no filter can reach

The list is ordered by value descending, and the screen offers exactly two filters: **item** and **"only tags with no category"**. For GOLD CHAIN:

- 280 tags in stock, 200 shown, **80 past the cap**
- **all 80 already have a category**, so the untagged filter excludes them
- there is no purity filter, no search, no paging

So those 80 tags — worth **₹7,54,587.96** — cannot be brought onto the screen by any combination of controls it offers. **Their category can never be corrected here.** SILVER TOPS is the milder version: 241 uncategorised against a 200 cap, so 41 are hidden on the first pass, though those do surface once the first 200 are assigned and drop out of the filter.

Fixed

The screen now runs a count over the *same* filter and says what it is hiding:

- no filter → "Showing the 200 dearest of 2575 matching tags — 2375 are not on this page."
- GOLD CHAIN → "...200 dearest of 280... — 80 are not on this page."
- SILVER TOPS untagged → "...200 dearest of 241... — 41 are not on this page."

All three counts verified against the database. The notice is amber, not grey, because it changes what the page means.

Not fixed — needs Rahul: the 80 unreachable tags still need either a purity filter, a barcode search, or paging on this screen. Saying so is not the same as fixing it, and re-categorising those tags is impossible until one of the three exists.

148. A reload silently unlinked seven customers' goods, leaving a screen unusable — HIGH

The party-sample **issue** screen offered an empty customer list. Nothing could be selected, so no outstanding holding could be recorded as returned.

The shop is holding **7 lines of customers' own goods** — 233 pieces, 14.43 carats of stones and diamonds belonging to five named people:

Receipt	Received	Customer	What	Pieces	Carats
CG/2627/00003	28 Jun 2026	RASHMI SARAF	stone	1	2.750
CG/2627/00005	22 Jul 2026	POOJA PRASAD	diamond + stone	203	4.150
CG/2627/00006	26 Jul 2026	TAHA.M. TAMBAWALA	stone	3	6.250
CG/2627/00007-8	18 Aug 2026	PRABHA BHANSALI	stone + diamond	18	1.100
CG/2627/00009	19 Aug 2026	P. K. MUNDRA	diamond	8	0.180

Every row carried the customer's **name and code**, and all five codes resolve to a ledger exactly — same code, same name. But `ledger_id` was **NULL on all seven**, and the screen lists customers by ledger, so it showed nobody.

The cause is a class, not a typo

Migration `mcj-1787385000` adds `ledger_id` **and backfills it**. That migration runs once.

`load_customer_goods.sh` runs whenever the data is reloaded, and it does `DELETE FROM mcj_customer_goods` followed by an `INSERT` whose column list never mentioned `ledger_id`. **Every reload undid the migration's backfill.**

The loader was already joining `mcj_ledgers` — it used `lg.name` for `party_name` — and simply never selected `lg.id`.

Four tables have a migration backfill that a loader can wipe this way: `mcj_customer_goods`, `mcj_customer_orders`, `mcj_sales_invoices`, `mcj_scheme_enrolments`. The other three loaders set `ledger_id` in their own insert, so **customer_goods was the only one exposed** — checked, all three are fully linked (284, 224 and 239 rows, none NULL).

Fixed

- `load_customer_goods.sh` now selects `lg.id` into `ledger_id`.
- The seven live rows were backfilled; all 7 now linked.
- The loader now **fails** if any row has a party code but no ledger, so this cannot regress quietly.
- Verified on the running screen: the issue page now lists all five customers with the right counts — $1 + 2 + 2 + 1 + 1 =$ the 7 outstanding lines.

149. The open-order dropdown stops at 200, hiding the 43 oldest — MEDIUM

Found in the same file. `openOrderOptions` caps at `LIMIT 200` and there are **243 undelivered orders**. Confirmed on the running screen: the order select renders **201 options** — 200 orders plus the placeholder.

It is ordered `ordered_on DESC`, so the ones that fall off are the **43 oldest open orders** — the ones most likely to be chased, and the least likely to be noticed as missing. There is no notice and no search on that control.

Not fixed. Unlike a report, a `<select>` cannot carry a "43 more" line without changing the control. The honest options are a searchable input, or raising the cap and accepting a 243-option list. That is a UI decision.

Needs Rahul: which of the two. Until then, a sample against one of the 43 oldest open orders cannot be recorded against its order.

150. No account holder can be telephoned — the cost of defect 41, quantified — MEDIUM

[Defect 41] established that `mcj_parties.ledger_code` and `mcj_ledgers.code` are unrelated code spaces, and the wrong join was correctly removed from `posting.go`. Re-confirmed here from the data: **368 codes appear in both tables and not one names the same person** (277 of them on billable ledgers, checked name by name — **zero** agree). Nothing to reopen there.

What was never written down is what that costs.

Every ledger contact column is empty

All **785** ledgers carry `phone`, `mobile`, `address`, `gstn` and `aadhaar_number` as empty strings — not sparsely filled, **zero rows** in every one. Contact detail lives only in the 40,519-row address book, and after defect 41 there is no key joining the two.

So for a customer with an outstanding balance, this system holds a name, a balance, and no way to reach them.

How much is recoverable

Of the **567 billable ledgers** (Customer, Dealer, Other Debtor), 277 collide with an address-book code and must not be linked. The remaining **290** break down as:

Group	Unmatched	Unique name match	Ambiguous	No entry
Customer	250	158	44	48
Dealer	36	8	0	28
Other Debtor	4	0	0	4
Total	290	166	44	80

166 match exactly one address-book entry by name, and 145 of those carry a mobile number.

That is a real route back to the customer — but a shared name is not proof of a shared person, and this record has already been burned once by a key that looked right (defect 41). It is a candidate list to confirm, not a migration to run.

Needs Rahul: confirm whether name is a safe match for these 166. If it is, 145 customers get their phone number back. The 44 ambiguous and 80 absent need the shop's own records either way.

Three routes have now been checked and rejected — see [defect 170] for the third, `address_book_sub_name`, whose 399 mobiles sit on a code space where **923 of 1,039 codes match nothing** and the 31 that match a ledger name a different person every time.

One other route was checked and does not work. `ledger_ref_details` (1,071 rows, never loaded) holds `party_ldgr`, `reference_name` and `ref_mob1` — a reference contact per party, which would have been the obvious place to look. **Exactly 1 of its 1,071 rows carries a mobile number.** 349 of its party codes resolve to a ledger, so the linkage is sound; the phone column is simply empty. It is not a second address book.

Deliberately not done: no automatic name-based backfill. Linking the wrong person to an account is the exact failure defect 41 describes, and doing it on a weaker key than the one that already failed would be worse, not better.

151. The design catalogue cannot be repriced, and 31 designs are worth minus twelve lakh — MEDIUM

Followed the [defect 150] sweep into `mcj_designs`, the 5,221-row catalogue whose ₹14.95 crore of estimates was published in [defect 145].

Nothing in it identifies the metal

Three columns are empty for **every one of the 5,221 rows**: `purity`, `metal_rate` and `making_rate`. Checked against the legacy source — MySQL `design_transaction_main` has `purity` blank, `rate` 0 and `making_rate` 0 on all 5,221 rows too, so **the migration is faithful and there is nothing to reload.**

The consequence is structural. A tag carries purity *and* the rate it was priced at, which is what lets [defect 141]'s revaluation work at all. A design carries neither. So:

- a design cannot be said to be 18kt or 22kt
- its estimate cannot be recomputed at today's gold rate
- the catalogue's ₹14.95 crore is frozen at rates nobody recorded

Weights and stone detail *are* there — gross, net, and 3,416 stone lines — so a design could be repriced **if someone supplies the purity**. Without it the catalogue is a reference list, not a price list.

31 designs carry a negative estimate

Designs with a negative estimate	31
What they total	-12,86,354.00
Worst single design (DBNG0100)	-3,97,488.00

DBNG0100 has 33.488 g net, ₹20,100 of making and ₹51,080 of stones — all positive — and an estimate of minus ₹3.97 lakh. No arithmetic over the other columns reproduces the figure, on that row or any of the eight examined, so it is a sign error in the shop's data rather than a computed result.

Legacy has the same 31 rows totalling the same -1,286,354, so again the migration is faithful.

They net off the catalogue total. The ₹14,95,13,508.73 reported in defect 145 is the *net* figure. The positive side is **₹15,07,99,862.73**, and 199 further designs carry an estimate of exactly zero.

Because the catalogue sorts by estimate descending, all 31 sit in the tail — invisible before defect 145's cap was addressed, and now folded into a summary total without comment.

Needs Rahul: whether those 31 estimates should be positive, zero, or the rows retired; and whether design purity is recorded anywhere outside this ERP. Neither is recoverable from the data.

152. 173 customers have paid a savings scheme to term and nothing records that any of them was settled — HIGH

The `mcj_schemes` sweep from [defect 151] led here. There is exactly **one** scheme, *Gold Plus* — 11 months, 10 instalments of ₹1,000 — so the 239 enrolments all sharing one `scheme_id` is correct, not a linkage fault. Checked before going further.

The money is not small.

	Enrolments	Customers paid in	Payable	Maturity value
Fully paid	174	72,11,000.00	70,66,000.00	78,58,300.00
Part paid	64	15,79,900.00	22,15,000.00	24,30,100.00
Nothing paid	1	0.00	20,000.00	22,000.00
Total	239	87,90,900.00	93,01,000.00	1,03,10,400.00

₹87,90,900.00 collected from customers across 2,144 payments, the earliest in April 2017 and the latest **1 June 2026**.

Every single enrolment is still open

235 of the 239 have already matured — the oldest on **25 July 2017** — and `is_active` is **true on all 239**. Not one scheme has ever been marked closed.

That is faithful to the source: legacy `party_wise_scheme` has `activate_status = 1` on all 239 rows and the same 235 matured. The migration is correct; the shop's system simply has no record of a scheme ever completing.

What that leaves open

173 enrolments are both fully paid and matured. Those customers put in **₹71,01,000.00** and are owed **₹77,37,300.00** at maturity. The system does not say whether a single one has been settled.

It cannot be checked from the data either. [Defect 142] established that the books begin **9 March 2026**, and these maturities run back to 2017, so there are no vouchers to look for a payout against. The one thing that is visible — the last scheme payment landing on **1 June 2026** — says the scheme was still being collected on well after most of it had matured.

What to do

This is the largest unquantified obligation found so far, and unlike [defect 19] it is owed to **named individuals** rather than sitting in a suspense line.

Needs Rahul, before any of this is treated as settled:

1. Were the 173 matured, fully-paid enrolments paid out? If so, where is that recorded — it is not in this ERP.
2. Is `activate_status` simply never maintained, or does *active* genuinely mean these are all still owed?
3. The 64 part-paid enrolments have taken in ₹15,79,900.00 against ₹22,15,000.00 payable. Are those lapsed, refundable, or still collecting?

Nothing here should be written off or closed programmatically. Recording the question is the correct outcome until the shop answers it.

153. Payroll records four months where nobody was paid, and the loader reads a column that is always zero — HIGH

Four months show a full salary bill and nothing paid

Month	Staff	Gross salary	Paid
August 2025	15	3,37,200.00	0.00
September 2025	15	3,58,300.00	0.00
October 2025	15	3,37,200.00	0.00

Month	Staff	Gross salary	Paid
July 2026	12	2,89,200.00	0.00
		13,21,900.00	0.00

Not one person in any of those months. Across all 246 sheets the shortfall is **₹15,71,288.00**: **59** carry a salary and nothing paid, **149** are part paid, and exactly **one** was paid in full.

Correction — statutory deductions do exist, and explain 15% of it. This entry first said deduction was not the answer, on the strength of `pf_amount` and `esi_amount` being zero on all 246 rows of `staff_salary_sheet`. They are zero there, but the deductions are kept in a **different table**, `staff_salary_sheet_payroll_detail`, which was found later by checking a figure in a code comment (see [defect 157]):

Over the same 16 months	
Provident fund	2,01,600.00
ESI	8,728.00
Professional tax	27,720.00
Total deductions	2,38,048.00
Payroll shortfall	15,71,288.00
Explained by statutory deduction	2,38,048.00 (15%)
Still unexplained	13,33,240.00

The finding stands — 85% of the gap has no explanation in the data — but the original wording was wrong and is corrected here rather than left to mislead.

The four zero-paid months are not explained away either: **August, September and October 2025 all carry payroll-detail rows** (14, 15 and 14 staff), so payroll was being computed in the months nothing was recorded as paid. And [defect 142] puts the first voucher at 9 March 2026, so for the 2025 months there are no payment vouchers to check against.

This is faithful to the source; the totals tie to legacy exactly (gross 4,949,100, paid 3,381,582). The question is what it means, not whether it loaded correctly.

The loader reads the wrong column

Separately: `days_worked` is **0.00 on all 246 rows**. The loader takes `s.no_of_days`, and that column is **zero on every legacy row**. Four similarly-named columns sit beside it, and the only populated one is `total_days`.

But `total_days` must not be used. Its values are 28, 30 and 31 — 136 rows at 31, 95 at 30, 15 at 28. That is the **length of the month**, not attendance. Mapping it would state that every member of staff worked every day of every month, including the four months nobody was paid.

The honest finding is that **attendance was never recorded at all**. Of the legacy payroll columns, `no_of_days` , `extra_days` , `total_extra_days` , `total_working_time` , `total_overtime` and `overtime_amount` are **zero on all 246 rows**. Only `basic_pay` (209 rows), `gross_salary` , `paid_amount` and the month length carry anything. The shop used this module to record amounts, never attendance or overtime.

Deliberately not fixed. Pointing the loader at `total_days` would turn an empty column into a wrong one. `days_worked` stays 0 because that is the truth, and the column is now documented rather than quietly misleading.

Needs Rahul:

- 1. Were August–October 2025 and July 2026 salaries actually paid? **₹13,21,900** of gross with nothing recorded against it.
- 2. Is the ₹15,71,288 shortfall real, or is `paid_amount` simply not maintained?
- 3. Should `days_worked` be dropped, given attendance was never kept?

154. ₹14.47 crore sits in an order column nobody reads, and the one we do read is empty — MEDIUM

`mcj_customer_orders.advance` is **0.00 on all 284 rows**. The loader takes legacy `customer_order.advance` , and that column is **zero on all 284 rows too**, so the load is faithful and the field is simply dead. Nothing in the module reads it — `advance` appears in no Go file outside the loader.

Beside it in the same legacy table sits `amount_received` , populated on **273 of 284 orders** and totalling **₹14,47,10,446**. Nothing reads that either.

What it is not

Before treating that as ₹14.47 crore of unrecorded customer advances — which is what it looks like — I compared it against each order's own line estimate:

Relationship	Orders
Equal to the line estimate	28
Some other ratio (0.85 to 2.89)	104
Exactly 10x	1
Order has no line estimate to compare	140

It is **not** the order value, and it is not a clean multiple of anything. A single order, **CO/2526/00402 dated 1 April 2026, carries ₹7,38,00,000** — more than half the total on one line, against a line estimate of zero.

Ratios below 1 (0.85, 0.90, 0.92) read like part payments; ratios of 1.32 and 2.89 do not. **The column cannot be interpreted from the data**, and this record has already been burned twice this week by a column that looked obvious — `total_days` in [defect 153], which was really the length of the month, and `ldgr_code` in [defect 41].

Why it matters anyway

If any part of ₹14.47 crore is money customers have handed over against undelivered orders, it is a liability, and **243 of the 284 orders are still undelivered** ([defect 149]). The order screens currently show no advance at all, because the only column wired up is the empty one.

Deliberately not loaded. Mapping `amount_received` into `advance` would put an uninterpreted figure — including a ₹7.38 crore outlier — onto a screen labelled as customer money. That is the failure mode this record keeps catching, not a fix.

Resolved — see [defect 169]. `amount_received` is the order's **metal value**, matching `metal_rate` × `net_weight` exactly on 157 of 273 orders and metal plus making on 21 more. It is not cash. The ₹7,38,00,000 on CO/2526/00402 is 3,000 g of silver booked at **₹24,600 a gram**, about 97 times its real rate, and the "exactly 10x" order is a 995 rate entered ten times too high. **There is no evidence of ₹14.47 crore of customer advances**, and the decision not to load the column was right. If it is an advance, the order book needs it and the outstanding-liability picture changes.

155. 37% of sales by value has no line detail, and the GST return says so itself — HIGH

The GSTR-1 HSN summary already prints a warning row for this, but it has never been recorded as a defect, and the figure had never been read across the whole period — the screen defaults to the current month, where it looks like a rounding note about **4 bills**.

Run over the full period it is not a rounding note:

GSTR-1 HSN Summary, 2016 → 2026	Taxable	Tax	Total
711319 Jewellery, gold/diamond/platinum	—	—	—
711311 Silver jewellery	—	—	—
711411 Silver articles	—	—	—
39 bill(s) with no line detail	89,26,345.00	2,44,729.79	91,71,074.79
Total	2,40,66,454.00	6,98,933.87	2,47,65,387.87

₹91,71,074.79 of ₹2,47,65,387.87 — 37% of everything sold — sits on bills whose contents are unknown. Confirmed independently against the database: 39 of the 224 invoices have no row in `mcj_sales_invoice_lines`, and their taxable value comes to the same ₹89,26,345.00.

It is faithful to the source. Legacy `sales_vou` holds 224 bill headers and `sales_vou_main` holds 328 lines spanning only **185 distinct bill numbers** — 224 minus 185 is exactly the 39. Both tables loaded completely (224 = 224, 328 = 328, checked). The line detail was never captured.

What it costs

- **The GST return cannot be filed from this system.** The report says so in plain words on the screen: *"This summary is short by the value shown and is NOT complete for filing."* Over a third of turnover has no HSN.

- **Nobody can say what was sold on those 39 bills** — no item, no purity, no weight, no tag.
- Those bills cannot be tied to stock movement, so they are invisible to any check that reconciles what left the shelf against what was billed.

What to do

The report's behaviour is right: it shows the gap rather than quietly filing a short return, and the total still reconciles. Nothing to fix in the code.

Needs Rahul: whether the line detail for those 39 bills exists anywhere else — the printed invoices, the accountant's copy, a GSTR-1 already filed with the portal. If a return was filed for this period, it was filed from something other than this data.

156. A live screen told users the HSN summary could not be produced, months after it was built — MEDIUM

Found by reading the GST registers over the full period for [defect 155]. The GSTR-1 B2B screen ended every run with:

The HSN summary cannot be produced from this data: no HSN is recorded against a sale line, and the item master has no HSN column. See blocker B21.

Every clause of that is now false, and B21 itself says so. The blocker was narrowed twice and records the correction in full: `item_master.hsn_no` is populated, on **318 of 322 items**, and its values are a foreign key into `hsn_master` rather than codes — which is exactly what made it look absent. Read as codes they would have produced a return of ones and sevens.

Checked against the running database:

Items carrying an HSN code	318 of 322 (98.8%)
Distinct HSN codes	21
Invoice lines resolving to an HSN	all 328
GSTR-1 HSN Summary	built, and produces 711319, 711311 and 711411

So one screen told the user the thing could not be done while the screen next to it did it. Anyone preparing a return would have gone looking outside the system for data that was already in it — the opposite of what this record is for.

Fixed. The row now says what is true and points at the report that does the job:

This is the B2B section only. For table 12 of GSTR-1 see the GSTR-1 HSN Summary, which covers every bill and states what it cannot account for.

Verified on the running screen.

Worth noting as a class. The stale text survived because the correction was written into `docs/blockers.md` and the code comment above the *other* function, but never into the string the user actually reads. A note that names a blocker should be checked whenever that blocker changes; nothing enforces it, and only reading the screen caught it.

A second instance, and a sweep of the rest

Acting on that, every user-facing string naming a blocker was checked — **26 of them**. One more was stale, in the same way and from the same blocker.

`gst-register` ended with:

*This register reconciles the books. It is not a return: the invoice detail a GSTR-1 needs — party, taxable value and HSN per bill — **did not survive the capture**. See blocker B21.*

The invoice detail did survive: **224 bills** in `sales_vou`, party and place of supply on **every one**, and **all 328 lines** resolving to an HSN. B21 records exactly that. The row now reads:

This register reconciles the books. It is not a return — for that see GSTR-1 B2B, B2C and the HSN Summary, which are built from the 224 invoices. 39 of those bills carry no line detail, so the HSN summary names what it cannot account for.

That also carries [defect 155]'s real limitation instead of a false one.

A test was pinning the stale text in place. `TestGSTRegister_SaysItIsNotAReturn` required the disclaimer to contain both *"not a return"* and *"B21"* — so the false claim was not merely unnoticed, it was asserted. The test now checks the disclaimer says it is not a return, **points at the GSTR-1 screens that exist**, and **fails if the words "did not survive" come back**. Its original intent is kept; what it no longer does is require a citation that had gone wrong.

The other 24 were checked and are accurate. The notes citing B19 (stock is not a ledger), B9 (165 items whose stored price cannot be reproduced) and B3 (the label template was never captured, so the geometry is a reconstruction) all match those blockers' current text. **Both stale notes came from B21** — the one blocker on this list that was substantially narrowed after the screens were written, which is exactly where to look next time a blocker changes.

157. Seven staff codes name nobody, and ₹7,14,430 was paid to four of them — HIGH

Found by checking a figure written in a code comment — *"21,69,448.00 across 16 staff and 16 months"* in `payrollhistory.go`. The figure is exactly right (PF 2,01,600 + ESI 8,728 + P.TAX 27,720 + HRA 5,29,100 + D.A 14,02,300 = 21,69,448.00, over 209 rows, 16 staff, 16 months). Checking it turned up two things the comment was not about.

The staff cannot be named

67 of the 246 salary sheets carry no staff name, across seven staff codes. **None of the seven exists in the ledger master**, so there is nowhere to look the name up:

Staff code	Sheets	Gross	Paid	In the ledgers
4023	15	4,68,000.00	3,10,800.00	no
8827	15	2,58,000.00	1,92,330.00	no
8855	15	1,89,900.00	1,25,820.00	no
8828	15	1,50,500.00	85,480.00	no
12679	4	0.00	0.00	no
12475	2	0.00	0.00	no
12509	1	0.00	0.00	no
Total	67	10,66,400.00	7,14,430.00	

₹7,14,430.00 was paid out to four people this system cannot identify — a payroll code and an amount, and no name anywhere in the data.

The same four account for **37 of the 209 payroll-history rows**, which are likewise unlinked. Unlike [defect 148], this is not a loader omission: the loader joins `mcj_ledgers` correctly and there is simply no ledger with those codes. Nothing to repair in the code.

It is not a code fault, so it needs the shop

The remaining three codes carry no money at all — 7 sheets, all zero — and look like abandoned entries.

The names are not recoverable, and the codes are staff codes

Checked against the **legacy** `ledger_master`, not just our copy: **4023, 8827, 8828 and 8855 exist in neither**. Their ledgers are not missing from the migration — they were already absent from the shop's own chart of accounts when the dump was taken.

That they are staff is not in doubt. Each sits inside a run of ledgers that are:

Code	In ledger_master	Group
4023	absent	—
4025	BAPI DAS	Staff
4027	DIPAK SONI	Staff
4028	SRITI KHANNA	Staff
8827, 8828	absent	—
8830	RAJENDRA PRASAD JAT	Staff

`ledger_wise_payroll_detail` (46 rows, 12 staff, all named) was also checked and holds none of the four.

So the payroll is real and the people are unidentifiable from this data by any route: no ledger, no payroll-detail row, and by [defect 142] no voucher history.

Needs Rahul: who staff codes **4023, 8827, 8855 and 8828** are. Four people were paid ₹7,14,430 between April 2025 and July 2026 and **nothing in the shop's own database says their names** — this is a question only someone at the shop can answer, not one a better query will settle.

158. The sold-stones screen went on showing the figure defect 135 had already corrected — HIGH

[Defect 135] established, months ago, that "stones sold ₹1,11,83,462.33" is roughly three times the truth: sales_vou_detail carries bulk metal sold by weight alongside stone lines, load_sold_stones.sh takes all 203 rows without a filter, and **21 carat-less rows worth ₹75,39,341.00** are silver sold by the gram, not stone.

The defect was written up. **The screen was never changed.** Run over the full period it still summarised:

94 bill(s), **203 stone line(s)** ... **1,11,83,462.33** ... 63.4% of the 1,76,29,343.00 those bills came to

Every figure there is the uncorrected one, and the 63.4% was worse than the total: a claim *about the shop* — that stone is nearly two thirds of what a customer pays — derived entirely from the error.

Re-confirmed against the database:

	Rows	Tags	Carats	Value
Genuine stone	182	134	131.946	36,44,121.33
No carats — metal by weight	21	5	0.000	75,39,341.00
Total as loaded	203		131.946	1,11,83,462.33

Fixed

The report now separates them. Verified on the running screen:

94 bill(s), **182 stone line(s)** ... **36,44,121.33** ... 20.7% of the 1,76,29,343.00 those bills came to. A further 21 line(s) worth 75,39,341.00 carry no carats — that is bulk metal sold by weight, not stone, and it is NOT in the total above.

The metal rows are **named rather than dropped** — they are real sales, and losing them would hide the untagged silver trade that [defect 135] found. The share is now computed on genuine stone only.

The file's own header comment carried the same wrong figures — SOLD 1,11,83,462.33 and "stones average **57.4%** of what the customer paid" — and has been corrected to 36,44,121.33 and 20.7%, with a note saying what it used to claim.

This is the third stale-claim defect in a row ([156], its sweep, and this). The pattern is consistent: the finding gets written into docs/ , and the screen and the comment that state it are left alone. Reading the screen is the only thing that has caught any of them.

A systematic sweep, rather than a fourth accident

Three in a row is a process failure, so instead of waiting to trip over the next one every money figure asserted in module code was extracted — **87 distinct values** — and cross-checked against the defects record for any that sit in a correction.

It found one more instance of this same figure. orderedstones.go carries a four-line summary of the whole stone picture, and its sold line was the uncorrected ₹1,11,83,462.33. Now corrected, with a note saying what it was.

The other three lines of that table were re-verified against the database in the same pass and are exact:

Claim in orderedstones.go	Database	
bought 1,27,31,698.71 · 521.329 ct	39 rows, 34 bills, 521.329 ct, 1,27,31,698.71	✓
held 6,21,95,953.60 · 2,731.821 ct	819 rows, 2,731.821 ct, 6,21,95,953.60	✓
on order 24,69,792.15 · 402.997 ct	115 rows, 402.997 ct, 24,69,792.15	✓

Other claims checked and found exact in this pass: supplierbalances.go 's **45 dealer accounts owed ₹6,15,02,878.56** (the screen's wider 62 / ₹6,23,56,784.52 covers three ledger groups, which is why they differ — both right), and payrollhistory.go 's **₹21,69,448.00 across 16 staff and 16 months**, which is what led to [defect 157].

Every remaining mention of ₹1,11,83,462.33 in the module now names it explicitly as the value that **was** wrong, so the figure survives only as its own correction.

159. The opening stock is not selling — 0.1% of it in five months, not the 1.7% quoted — MEDIUM

Found by continuing [defect 158]'s sweep past the defects record and checking code figures **directly against the database**, which is the gap that sweep could not cover.

openingstock.go carried this comparison:

	tags	value	sold
opening stock	1,571	8,52,67,768.16	1.7%
tagged during the year	1,245	6,75,97,397.62	17.3%

Every tag count and every value is **exactly right** — confirmed against the database, including the split that had briefly looked like a discrepancy:

	Tags	Value
Opening, still on the shelf	1,545	8,51,58,772.85
Opening, sold	26	1,08,995.31
All opening tags	1,571	8,52,67,768.16

The percentages are a different unit from the column beside them

Those **sold** figures are **shares of tag count**: 26 of 1,571 is 1.7%, and 215 of 1,245 is 17.3%. They sit immediately to the right of a **value** column, and the sentence underneath argues in rupees. A reader takes them for a share of value.

By value the same rows read:

	Sold, by tag	Sold, by value
Opening stock	1.7%	0.1%
Tagged during the year	17.3%	15.4%

The passage concluded that newer stock "sells ten times more readily". By tag count that is right ($17.3 \div 1.7 = 10.2$). **By value it is 154 times.**

The finding is much stronger than the one that was written down

₹8,52,67,768.16 of opening stock produced ₹1,08,995.31 of sales in five months — one tenth of one percent. The 26 opening tags that did sell were the cheap ones, which is why they are 1.7% of the tags and 0.1% of the money.

That sits directly alongside [defect 141]: the opening stock is the same 360-tag-and-more group carried in at a round ₹4,000/g, and it is both the least saleable stock on the shelf and the part whose book value is least defensible. It is also the stock [defect 142] can find no purchase document for.

Fixed: the comment now gives both columns and says which is which. The screen itself never showed this comparison — it lists on-shelf tags only — so nothing user-facing was wrong. This is a record-keeping defect in the reasoning, and the reasoning is what the next person will read.

160. Two more doubled footers, and defect 143's sweep was not as complete as it claimed — HIGH

[Defect 143] found eight reports whose footer counted their own summary row twice, fixed them, and recorded that **"no doubled footer remains anywhere"**. That claim was wrong. Two reports still had one.

Found while verifying `salecost.go`'s figures against the database for the [defect 159] sweep — the comment's numbers were right, and the screen totalling them was not.

Report	Its own summary says	Footer printed	
sale-cost	1,05,03,577.58 marked / 1,05,56,997.00 sold	2,10,07,155.16 / 2,11,13,994.00	2x

Report	Its own summary says	Footer printed	
mrp-purchases	40,16,651.60	80,33,303.20	2x

sale-cost 's difference column was worse than doubled: the footer read **₹19,68,409.86**, which is the 232 detail rows (₹53,419.42) plus **both** summary rows (₹53,419.42 and ₹18,61,571.02) added together.

Why the original sweep missed them

The detector looked for a footer equal to **twice the last body row**, then for trailing rows whose first cell matched a count like "39 line(s)".

sale-cost defeats both. It carries **two** summary rows, and the final one begins "*The net is cancellation, not agreement*" — no count, no match — so the scan stopped before reaching the row that actually holds the totals. Reading the last row alone was never a sound test, and the earlier sweep's clean result was a property of the test, not of the reports.

The detector that works

A subtotal row is one whose value **equals the sum of every row above it**. That is checkable without knowing anything about labels or position:

- run against all 69 reports, requiring at least four preceding rows so a row matching the single row above it is not mistaken for a subtotal
- it found exactly these two, and nothing else

Both now set NoTotals . Re-fetched all 69 reports: **both footers gone, and the detector now reports zero.**

The lesson is about the check, not the code. Defect 143's fix was right; its *verification* used the same shape of test as the bug it was fixing, so it agreed with itself. A claim of completeness needs a test that could have failed for a different reason than the one that motivated it.

161. The quote screen refuses a fifth of the shelf, and nobody had counted it — MEDIUM

Found by extending [defect 156]'s sweep to the gap that sweep left: it checked strings **naming a blocker**, so a stale or unquantified claim that cites nothing would have passed. Twenty-one user-facing strings make a definitive negative claim; most are field validation. Two make the "*did not survive the capture*" assertion that has already been wrong twice.

One of them is right, and is worth a number.

Quoting refuses every certified tag

quote.go refuses certified goods outright, citing [blocker B9] — which was checked and **is still open and accurate**: 165 items' stored price cannot be reproduced, and they cluster by certification type (by_val 98, per_ct 35). Refusing is the correct behaviour; approximating a certified piece would be wrong by a large amount rather than a small one.

What had never been established is how much of the shelf that removes:

Certification	Tags	Value
by_val	472	1,54,27,841.49
per_ct	91	1,30,44,293.01
(charge but no type)	3	3,660.00
Total refused	566	2,84,75,794.50

566 of the 2,575 tags on the shelf — 22% by count and 20% by value — cannot be quoted at all. Confirmed on the running screen: barcode 53710364, a DIAMOND BANGLE and the dearest certified tag, returns "*Cannot be priced here*".

That is the practical cost of B9, and it is larger than B9's own description suggests. B9 counts **165 items whose price cannot be reproduced**; the screen refuses **566**, because it will not price *any* certified tag, not merely the ones known to be wrong.

Not changed. Refusing is right while the pricing step is unknown, and narrowing the refusal to the 165 would mean quoting 401 certified tags on a formula that is unverified for certified goods generally.

Needs Rahul: what the certification pricing step is. It is the difference between a quote screen that covers the shelf and one that covers four fifths of it, and B9 has been open longest of any blocker on this list.

Checked and found sound

- oldgoods.go's "*a purchase whose detail did not survive*" branch is defensive rendering, not a claim: **all 25 old-gold vouchers have detail**, so it never fires today.
- schememaster.go and salarysheet.go's "never recorded" wording matches [defect 152] and [defect 153] exactly.

162. The balance sheet did not balance on screen, and neither did the P&L — HIGH

The two statements an accountant opens first were each printing totals that did not tie, because both end with a **memo row** — the shelf value, shown for comparison — and the shell's footer sums every money cell it finds.

	Printed	Printed	Gap
Balance Sheet	Assets 41,33,12,142.12	Liabilities 27,09,50,953.92	14,23,61,188.20
P&L Account	Expense 18,65,39,319.36	Income 2,20,03,143.00	—

The gap on the balance sheet is exactly ₹14,23,61,188.20 — the stock memo row, to the paisa. On the P&L the same row inflates the Expense column by the same figure.

Both statements were correct underneath

This is presentation, not accounting. Take the memo row out and each balances:

- **Balance Sheet:** $41,33,12,142.12 - 14,23,61,188.20 = \mathbf{27,09,50,953.92}$, equal to the liabilities side.
- **P&L:** expenses $3,30,90,637.08$ against income $2,20,03,143.00$, with the shortfall of **1,10,87,494.08** closing it — and that shortfall is the figure both screens already agree on.

The suspense line does its job in both cases. The memo row was simply never meant to be added to anything: on the balance sheet it is labelled *"for comparison with the difference"*, and it was being added to the difference instead.

Fixed

Each statement now prints its **own balancing Total row** before the memo, and sets `NoTotals` so the shell adds nothing after it. The memo is relabelled **"FOR COMPARISON — not part of the total above"**.

Verified on the running app:

	Total	
Balance Sheet	$27,09,50,953.92 = 27,09,50,953.92$	balances
P&L Account	$3,30,90,637.08 = 3,30,90,637.08$	balances
Trial Balance	$29,58,46,599.44 = 29,58,46,599.44$	already correct — no memo row

Why the earlier sweeps missed it

[Defect 160]'s detector looks for a row equal to **the sum of the rows above it**. A memo row is not a subtotal — it is an unrelated figure from another module — so it matches nothing and the detector was silent. Correctly silent: it was built to find double-counted subtotals, and this is a different fault with the same symptom.

A test existed, and it passed throughout

`TestBalanceSheet_TiesWithNothingLeftOver` has been green all along. It sums the query's rows and **skips the "Stock in hand" row itself**, with the comment *"stated for comparison, never added into a side"*. So the test knew the rule the screen was breaking — it just checked the **query output**, and the fault was in the **rendering**, where the shell totals every money column it is handed.

Two new tests now check the layer the bug lived in: each statement must emit its own `Total` row, that row must balance, and the memo must come **after** it and be labelled as excluded.

Both were confirmed by breaking the code on purpose:

- relabelling the memo back to *"for comparison with the difference"* → *"balance sheet does not mark the stock memo as excluded from the total"*
- renaming the P&L's `Total` row → *"p&l prints no Total row, so the shell will total it and add the memo"*

The first version of these tests was vacuous and said so. Seeded books carry no stock, so `shelfValue` was zero, the memo row was never emitted, and the loop checking it ran over nothing — the label mutation passed. The tests now seed one tag and **fail outright if the memo row is absent**, because a test that cannot see the row proves nothing about it.

163. The barcode edit trail records that a tag changed, but not what it changed from — LOW

`mcj_barcode_edits` holds 156 edits across 154 tags, April to August 2026. Seven of its columns carry a single value throughout, which is what drew attention.

The **flags are exactly right**. Checked against legacy `barcode_edit_tracking` row by row:

Change	Flagged in our data	Genuinely differs in legacy
Item	31	31
Units	119	119
Making	7	7
Gross weight	0	0
Net weight	0	0
Stone	0	0

Every flag matches. The loader is reading the right columns and the empty ones are empty because nothing of that kind was ever edited.

What was not carried across

Legacy keeps **before and after values** for each kind of change — `old_qty / new_qty` , `old_gw / new_gw` , `old_making / new_making` , `old_stone_wt / new_stone_wt` . Our schema keeps only `old_item_id / new_item_id` .

So for the **119 unit changes** and the **7 making changes**, the trail says a tag was edited and on what date, but not from what to what. On the making edits that is a real loss: the legacy values run from **– ₹300 to + ₹1,683**, including two tags taken from ₹0 to ₹1,155 and ₹1,120.

Not urgent, and not corrected here. It needs a migration to add six columns and a loader change, and the underlying rows are intact in MySQL — unlike [defect 142]'s vouchers, this is recoverable whenever someone wants it.

A wrong reading, recorded because it was nearly written down

The 119 unit changes are all downward — 101 from 2 to 1 and 18 from 4 to 1, a net of **–155**. That reads as 155 pieces removed from tags by editing rather than by selling, which would have been a serious finding.

It is not what happened. quantity is the number of items on a tag and pieces is the number of physical objects, and they are meant to differ: **761 in-stock tags — ₹6,76,78,194.42, 47% of shelf value — have pieces greater than** quantity , 679 of them a pair of two pieces on one tag. Every edited tag now reads pieces 2, quantity 1, which is a pair correctly described.

The edits were corrections bringing quantity to 1 on multi-piece tags. Nothing left the shelf. The number was real and the conclusion would have been wrong.

164. ₹1.35 crore of fine gold was issued to a named goldsmith, and the record dismissed it as an empty table — HIGH

[Defect 76] surveyed the manufacturing tables and concluded there are "no karigar transactions, no work orders, no stone issues and no hallmarking movements to report on", listing the evidence as "every issue and receive table is empty: karigar_gold_stock 2, karigar_wip_bal 1".

Those two rows are not an empty table. Read, they say:

Bill	Date	Karigar	Item	Weight	Purity	Rate	Type
C4/00001	16 Sep 2025	ledger 11699	GOLD	369.850 g	995	11,417.00	ISS-RAW
C4/00002	16 Sep 2025	ledger 11699	GOLD	500.000 g	995	11,417.00	ISS-RAW
				869.850 g			

Ledger **11699** is **MA TARA JEWELLERS**, group **Karigar** — and defect 76 notes it is the *only* Karigar ledger in the chart of accounts. So the shop has one goldsmith on its books, and both rows issue metal to him.

869.850 g of 995 fine gold, worth **₹99,31,077.45** at the rate on the issue and **₹1,34,67,017.70** at today's 995 rate of ₹15,482.

Nothing records it coming back

receive_from_karigar	0 rows
receive_from_karigar_details	0 rows
receive_from_karigar_main	0 rows
issue_to_karigar and its 8 variants	0 rows
karigar_wip_bal	1 row, 0.000 at every purity

And in the accounts, MA TARA JEWELLERS carries **an opening balance of ₹0.00 and not one posting**. The issue of ₹99 lakh of metal to him appears in no voucher, no ledger and no report.

karigar_gold_stock is **not migrated** — there is no mcj_karigar table — so the rebuild does not carry it either.

What it means, and what it does not

This is a **separate record** from the raw stock of [defect 78] and [defect 133]. Different table, different date — September 2025 against April–June 2026 — and defect 133's movement table carries no 995 row at all. Whether the 869.850 g overlaps with the raw holding, was returned as finished goods, or is still out **cannot be told from this data**. It should not be added to the ₹15.77 crore without the shop confirming it.

The lesson is about the survey, not the shop. Defect 76 judged the table by its row count and wrote it off with tables that genuinely hold nothing. Two rows held more than a crore of gold and a name.

load_oldgold.sh already carries the note that "a row estimate is not a row count"; this is its companion — **a row count is not materiality**.

Needs Rahul, and this one has a name attached: did MA TARA JEWELLERS return the 869.850 g issued on 16 September 2025? If it came back as finished stock it should be on the shelf; if it did not, it is the single largest identified item of metal outside the accounts that can be attached to a person.

165. 334 tags are re-issues of earlier tags, and the link back is not loaded — MEDIUM

Applying [defect 164]'s lesson — *a row count is not materiality* — to the rest of the small legacy tables. Most turned out to be already loaded: stock_inward (43 rows, ₹2,39,56,798) is mcj_purchase_invoices to the rupee, and stock_inward_detail (39 rows, ₹1,27,31,698.71) is the stone side of mcj_purchase_contents . Both check out exactly.

The return tables do not.

What they hold

sale_return_detail carries a column our schema has no equivalent for:

bill	barcode	new_barcode	pcs	ct	gw
SR/2627/00001	70630587	70630587R1	588	10.280	2.056

A returned tag is re-issued under a new number, the old one with an R1 suffix. That is the convention behind the suffixed barcodes across the shelf:

Barcode shape	Tags in stock	Value
Plain numeric	2,241	—
R<n> re-issue	245	57,86,486.50
Plain letter suffix	89	59,76,162.48

The link is not recoverable

The tables that state it explicitly are **not read by any loader** — sale_return_detail (4 rows), purchase_return_detail (2), purchase_return_main (3, carrying 59.650 g gross and 32.767 g fine worth ₹8,40,989.52). The returns report reads mcj_vouchers only, so a return is a money total with no record of

what came back.

And the suffix cannot substitute for the link. Of the **245** R-numbered tags:

Parent barcode still on the shelf	1
Parent barcode with any movement history	5
Parent with no record of any kind	240

So for ₹57,86,486.50 of stock, the tag says it replaced something and nothing in the system says what. The six rows that would name it were never loaded.

What to do

Loading `sale_return_detail` and `purchase_return_detail` is small and worth doing — it recovers 6 links exactly and gives returns their line detail, the weights included. It will **not** explain the other 239: those re-issues predate the captured returns, and by [defect 142] the documents are gone.

Needs Rahul: whether an `R1` suffix always means a return, or is also used for a repair, a resize or a re-tag after splitting. The convention is doing the work of a foreign key on ₹57.86 lakh of stock, and only he can say what it means.

166. `retail_all_stock` is our stock table, and it reconciles to the gram — LOW (clean result)

Continuing [defect 164]'s lesson through the larger unopened tables. Two were worth the look and neither is a defect, which is worth recording as plainly as a fault would be.

`item_adjustment` — 922 rows, five of them meaningful

The name suggests stock adjustments. It is a **making-rate master**: `item_id`, `per_of_making`, `rt_mk_pr_gm`, `pl_minus_gold`. Of its 922 rows across 460 items, **5** carry a making percentage, **2** a rate per gram and **3** a gold-minus figure — and rows repeat verbatim (item 1066 appears twice, identical).

The inverse of the karigar lesson: **a high row count is not materiality either**. 917 of 922 rows hold nothing.

`retail_all_stock` — 2,820 rows, and it is our own stock table

	<code>retail_all_stock</code>	<code>mcj_stock_items</code>
Rows	2,820	2,816
Distinct barcodes	2,815	2,815
Net weight	21,380.564 g	21,376.104 g

It reconciles exactly. The **4 extra rows** are hallmarking movements, and their **4.460 g** is the whole of the weight difference:

- $21,380.564 - 21,376.104 = 4.460 \text{ g}$
- $2,820 - 2,816 = 4 \text{ rows}$

The matching distinct-barcode count on both sides — **2,815 against our 2,816 rows** — is [defect 146]'s duplicate barcode `86065435` collapsing to one, seen from the other side.

So this is not a second stock record and nothing is missing from the load.

A small correction to defect 76

[Defect 76] states there are *"no karigar transactions, no work orders, no stone issues and no hallmarking movements to report on"*. On hallmarking that is very nearly right rather than exactly right: **4 rows exist**, `HMISS/2425/00014` and `HMREC/2425/00014` and one more pair, covering **2 tags** on 1 April 2025 — **2.230 g issued and 2.230 g received back**.

They net to zero and amount to two grams. Unlike [defect 164], where the same wording hid ₹1.35 crore of gold, here the conclusion stands and only the wording is absolute where it should be nearly so. Recorded so the next reader does not have to re-derive it.

167. A second stock table values 281 tags differently, and on 208 of them the metal is counted four times — MEDIUM

`cla_retail_trans_bar_detail` — 2,816 rows, never loaded — turns out to hold one row per tag with an amount . It is the same population as our stock table:

	cla_retail_trans_bar_detail	mcj_stock_items
Rows	2,816	2,816
Distinct barcodes	2,815	2,815
Barcodes present in one only	0	0
Sum of amount	17,12,31,075.39	15,28,65,165.78

Same tags, **₹1,83,65,909.61 apart**. On **2,534 of 2,815** the two agree to the paisa, so this is not a units or rounding difference. It is 281 specific tags.

208 tags: the metal counted four times

For 208 of them the relationship is exact and identical:

$$\text{cla amount} = (\text{metal} \times 4) + \text{making}$$

Worked through on the largest, **89501233**, a GOLD CHAIN:

Net weight	26.740 g
Rate	9,300.00

Metal (26,740 × 9,300)	2,48,682.00
Making, <code>per_gld_val</code> at 12%	29,841.84
Our total	2,78,523.84
cla amount	10,24,569.84 = 2,48,682.00 × 4 + 29,841.84

Our figure is internally consistent — the metal is rate × weight and the making is 12% of it. The cla figure is the same tag with its metal counted four times. **Across the 208 that accounts for ₹1,81,45,176.21 of the gap.**

What the 208 are not. Three explanations were tested and each fails:

Hypothesis	Result
The multiplier is the tag's piece count	No — 202 of the 208 carry <code>pieces = 1</code> , six carry 2. None carries 4
A particular making type	No — they span <code>per_gm_nw</code> (133), <code>per_gld_val</code> (39), <code>per_pcs</code> (8), <code>val</code> (4), <code>per_gm_gw</code> (1)
A date cutoff in the old system	Nearly — all 208 are New Create dated on or before 25 Oct 2025 , but 939 other tags in that same window agree to the paisa

So the window is necessary and not sufficient: 208 of the 1,147 tags created up to 25 Oct 2025 are affected, and the 1,669 created after it are all correct. Every one of the 208 is `item_type = M` and 176 of them are 18kt.

The relationship itself is exact — × 4, not × 3.98 — on all 208. Something computed it deliberately. What, cannot be told from this data.

73 tags: a certification charge our total leaves out

The remaining 73 differ by small amounts — ₹225, ₹300, ₹315, ₹700 — and each is exactly that tag's **certification charge**:

Barcode	Metal	Making	Certification	Our total	cla
12774913	4,885.96	1,500.00	300.00	6,385.96	6,685.96
22759867R1	32,088.00	3,000.00	315.00	35,088.00	35,403.00
45336681	4,83,388.00	4,599.00	700.00	4,87,987.00	4,88,687.00

This is faithful. Legacy `barcode_main.total` omits the certification charge too — checked row by row, its own `amount + making_sales + certification_charge` exceeds its own `total` by exactly that charge. The migration copied what was there.

Across the whole shelf, **193 in-stock tags carry ₹1,32,937.25 of hallmark and certification charges that their stored `total` excludes.**

What this means

Neither figure should be adopted over the other without the shop. Our ₹14,23,61,188.20 follows `barcode_main`, which is the tag master and the right source. But two of the shop's own tables disagree about 281 tags, one of them by counting metal four times, and that is worth knowing before either number is put in front of an accountant.

Needs Rahul: whether the certification charge belongs in a tag's value — if it does, the shelf is understated by ₹1,32,937.25 — and what `cla_retail_trans_bar_detail` was for, since something in the old system was producing a four-times-metal valuation on 208 tags.

168. The 8 orphaned vouchers are identified, and they all balance — MEDIUM (narrows B4)

[Blocker B4] has carried "8 JRNL vouchers worth ₹21,025.82 have blank scope columns" as an open item awaiting an accountant. Reading `voucher_transfer_check` (1,445 rows, never loaded) led to them.

They are legacy `voucher_main` **JRNL v_id 1 to 8**, and the identification is exact — their amounts sum to **₹21,025.82**, matching B4 to the paisa:

v_id	Date	Amount	comp_code	rt_code	ac_yr
1	2026-04-01	411.82	0	(blank)	(blank)
2	2026-03-09	1,399.00	0	(blank)	(blank)
3	2026-04-19	12,572.00	0	(blank)	(blank)
4	2026-06-19	13.00	0	(blank)	(blank)
5	2026-07-01	34.00	0	(blank)	(blank)
6	2026-06-30	4,000.00	0	(blank)	(blank)
7	2026-07-10	1,711.00	0	(blank)	(blank)
8	2026-07-20	885.00	0	(blank)	(blank)

v_id 2 is the ₹1,399 of 9 March 2026 — the same line that leaked into the balance sheet's year result before it was bounded, and the earliest voucher in the entire capture ([defect 142]).

They are not broken, only unscoped

All eight balance. Loaded, each carries equal debits and credits — 411.82 against 411.82, 12,572.00 against 12,572.00, and so on for all eight. Whatever is wrong with them, it is not the double entry.

What is missing is scope: `comp_code = 0`, and `rt_code` and `ac_yr` blank. Our loader derives the financial year from the date, so seven landed in 2026-2027 and `v_id 2`, dated 9 March, correctly in 2025-2026.

The consequence, which is latent rather than live

Because their `ac_yr` is blank they share (`type`, `legacy_v_id`) with the ordinary FY 2026-2027 JRNL vouchers numbered 1 to 8. In `mcj_vouchers` that is a collision:

Colliding (legacy_v_id, type) pairs	8
Extra rows they create	8
All of type	JRNL

There is no unique index on legacy_v_id , so nothing rejects it. **Nothing is currently wrong**, because every loader that joins on it constrains the type — load_oldgold.sh to PURC-OG (and says why id and type suffice), load_purchase_invoices.sh to si.v_typ, load_brs.sh to type and financial year — and no collision exists outside JRNL. It would bite the first join that omits the type.

What this gives B4

B4 asks for "a named accountant to approve both repairs". This narrows the second repair to a specific and much smaller question: **the eight are arithmetically sound and need a company, retail point and financial year stamped on them, not a correction to their amounts**. The ₹0.01 JRNL 312 imbalance is untouched by this and remains the harder half.

169. amount_received on an order is the metal value, not money received — HIGH (resolves defect 154)

[Defect 154] found ₹14,47,10,446 in customer_order.amount_received , could not interpret it, and deliberately refused to load it into a field labelled "advance". Reading order_rate_master (2,520 rows, never loaded) settled it.

Every order carries a **rate snapshot per purity**, and our mcj_customer_order_lines already holds it as metal_rate — populated on 361 of 366 lines. With that, amount_received resolves:

amount_received equals	Orders	
metal_rate × net_weight	157	58%
the same plus making	21	8%
the line estimate	3	1%
none of these	92	34%

It is a computed metal value, not cash. 58% match the metal exactly and another 8% match metal plus making. A third still fits no formula tried, so this is not fully closed — but the reading that mattered is refuted: **there is no evidence of ₹14.47 crore of customer advances**, and the order book showing no advance is not hiding a liability of that size.

Both of defect 154's puzzles were two bad rates

That entry flagged two things it could not explain. Both are order lines carrying an impossible metal rate, and both orders are **still undelivered**:

Order	Purity	Net weight	Rate booked	Plausible rate	amount_received
CO/2627/00041	995	20.000 g	1,52,000.00	~15,200	30,40,000.00
CO/2526/00402	SLV	3,000.000 g	24,600.00	~253	7,38,00,000.00

- CO/2627/00041 is defect 154's "exactly 10×" case. Its rate is **ten times** a plausible 995 rate, and its own estimate of ₹3,04,000.00 is right — $20.000\text{ g} \times ₹15,200$. The estimate is sound and the rate is not.
- CO/2526/00402 is the **₹7,38,00,000 outlier** that was more than half the whole figure. Silver booked at **₹24,600 a gram**, about **97 times** its real rate, against a line estimate of zero.

So the single row that made `amount_received` look like a fortune is a mistyped silver rate on one undelivered order.

What is left of it

Two live orders carry a metal rate that is wrong by a factor of 10 and 97. If either is priced from `metal_rate` at delivery, the customer is quoted a figure with no relation to the metal. The estimates on those lines are the trustworthy figures.

Needs Rahul: correct the booked rate on **CO/2627/00041** and **CO/2526/00402**, both still open. And `amount_received` should not be surfaced as an advance anywhere — [defect 154]'s decision not to load it was right, for a reason now known.

170. A third contact table, a third `ldgr_code` space, and it cannot reach a customer either — MEDIUM

`address_book_sub_name` (1,247 rows, never loaded) looked like the answer to [defect 150], which established that **no account holder in this system can be telephoned**. It carries `mobile1`, `mobile2`, `whatsapp`, `email`, `home_phone`, `office_phone` and `res_adrs`, and it is keyed on `ldgr_code`.

What it holds:

Rows	1,247
Distinct <code>ldgr_code</code>	1,039
With a mobile	399
With an email	84
With an address	116

It is a third code space

Following [defect 41]'s lesson, the codes were tested against **both** existing tables, by code and then by name:

	Codes matched	Same person by name
<code>ledger_master</code>	31	0

	Codes matched	Same person by name
address_book_master	90	20
Neither	923	—

923 of 1,039 codes match nothing at all, and of the 31 that match a ledger, **not one names the same person**. It is defect 41 exactly, in a third table:

Code	Name here	Name in ledger_master
3225	SHIV BAGLA	RAMNIKLAL AND SONS
4025	AKSHAY OBEROI	BAPI DAS
4909	NEETI GOURAV PAREKH	RELIANCE JIO INFOCOMM
11653	MANISH GARODIA	RANI AGARWAL
12795	NITU	NAFISA MUSTAFA

Joining on `ldgr_code` would file SHIV BAGLA's mobile under RAMNIKLAL AND SONS, and give a person's number to a telecom company.

So defect 150 stands

Three tables in this dump carry contact details — `address_book_master` (40,519 rows), `ledger_ref_details` (1,071, of which **exactly 1** has a mobile) and this one. **None of them can be attached to a ledger**. The 399 mobiles here are real and belong to real people; there is nothing in the data that says which account, if any, each belongs to.

Deliberately not loaded. A contact table that cannot be keyed is worse than no contact table, because it looks usable. This is the third apparent route to customer contactability to be checked and rejected, and the reason is the same every time: `ldgr_code` **means something different in every table that has it**.

Needs Rahul — the same ask as defect 150, now with one more table's worth of evidence that it cannot be answered from the data.

171. The shop's photography is not lost — 3,965 images have addresses, and none was captured — MEDIUM

`barcode_image` (3,965 rows, never loaded) is an inventory of tag photographs. Its `path` column is a `longtext`, which raised the possibility of embedded image data. It is not: every value is 60 to 74 characters and holds a URL.

```
http://tag09.com/mcj2/images/bar_pic/25929279_65910625_SR_SR_SR_SRA_2.jpeg
```

What exists

Photographs referenced	3,965
------------------------	--------------

Tags photographed	1,023 (3.9 images each)
Of those, still on the shelf	889
Sold	38
Shelf value of the photographed tags	10,73,39,445.03 — 75.4% of the shelf

Separately, `mcj_designs` carries a picture for **5,104 of 5,221 designs** across 5,044 distinct paths — but those are **relative** (`../image/14017640.jpg`), so they name a file without naming a host.

About 9,000 images are referenced across the two, and not one was captured.

Why this is worth recording

The record has said the pictures are gone. That is true of the files and not of the **addresses**: 3,965 of them are absolute URLs on `tag09.com`, so unlike [defect 142]'s vouchers — which are absent from the shop's own database and unrecoverable — these are merely **unfetched**. 889 of them show stock that is on the shelf today — **₹10,73,39,445.03, or 75.4% of the shelf by value**. The photographed tags are the dear ones: 35% of the tags carry three quarters of the money.

Deliberately not fetched. [Blocker B24] records the decision not to work around the bot-check on `tag09.com`, and that stands: it is a third-party host and retrieving several thousand files from it is the shop's call, not a step to take unasked. Nothing here changes that.

Also not loaded. The URL list itself exists only in the MySQL dump. Carrying it into the rebuild would preserve *which* tags were photographed even if the files never come back — a small job worth doing if the images matter, and pointless if they do not.

Needs Rahul: whether the shop still controls `tag09.com` and wants the images. If it does, this is 889 photographs of current stock, recoverable while that host answers. The design pictures need a host name before they can be looked for at all.

172. 104 customer documents are referenced and none survives, including PAN and Aadhaar — MEDIUM

`ledger_wise_document_upload` (104 rows, never loaded) attaches documents to ledgers. Classified by name pattern rather than by reading them:

Documents	104 across 81 ledgers
Format	100 images (jpeg/jpg), 4 PDF
Named as a PAN	6
Named as Aadhaar	2
Named as a photo or image	56
Dated	4 Jun 2019 to 20 Jan 2026

None of the files was captured, and unlike [defect 171]'s photographs the paths are **relative** — 0 of 104 is a URL — so they name a file without naming a host. There is nowhere to fetch them from even in principle.

The contents were not opened. Document *names* were pattern-matched for PAN , AADH , GST and similar; no personal data was read or printed, on the same footing as [blocker B15].

Why it matters: 81 ledgers had identity documents attached, at least eight of them government ID. If those were the shop's KYC evidence, the evidence is gone — the references survive and the files do not. That is a question for whoever is responsible for the shop's compliance, not a data-migration matter.

Needs Rahul: whether KYC documents are held anywhere outside this ERP. If they are not, 81 customer files exist only as filenames.

173. The design catalogue looks unrealised and is not — a fill rate misread, caught — LOW

design_no_status (5,242 rows, never loaded) flags **176 of 5,218 designs** with status = 1 and the rest 0. There is no legend, and the flag does not correlate with anything checkable: of the 176, exactly **1** ever became a tag. It adds nothing interpretable and is not worth loading.

Testing that turned up a figure worth recording, and a conclusion worth **not** recording:

*Only **69 of 5,221 designs** have ever been made into a tag — 1.3%.*

Read alone that says the catalogue is almost entirely unrealised, which would be a substantial finding about ₹15 crore of designs. **It is not what the number means.**

Tags carrying a design code at all	109 of 2,816 — 3.9%
Distinct design codes on those tags	93
Of those 93, resolving to a design	69 — 74%

The catalogue is not unrealised; **tags simply do not record which design they came from**. Where a tag does carry a design code, it resolves three times in four, which is a good match rate. The 1.3% measures the fill rate of design_code , not the productivity of the design room.

This is the same trap as [defect 163]'s -155 pieces and the fill rate read as evidence note in this record's early entries. Logged because the wrong version was one sentence from being written down.

174. The dealer rate cards cannot be B9's missing step — LOW (narrows B9)

[Blocker B9] — 165 items whose stored price cannot be reproduced — lists what has been tried and rejected: stone total against stone amount , hm_charge , certification_charge in all combinations, per-tag stone lines, barcode_main.making_sales , diamond_rate_master and item_wise_sale_rate_master .

Two rate cards are **not** on that list, both never loaded:

Table	Rows	Covers
dealer_wise_diamond_rate	642	2 dealers, 1 item
dealer_wise_making	7	4 dealers

dealer_wise_diamond_rate is 321 carat-banded rates each for two dealers against a single item.

dealer_wise_making holds one making rule per dealer per purity — a rate below a 2.000 g net weight and another above it, with a touch percentage:

Dealer	Purity	Under 2 g	Over 2 g	Touch
4986 MANIK CHAND JEWELLERS	14 / 18	2,000.00 per pc	1,000.00	60 / 76
7713	14 / 18	1,500.00 per pc	750.00	60 / 76
3465, 4780	14 / 18	(zero)	(zero)	60 / 75 / 76

Neither can explain 165 items. Between them they cover **4 of the shop's 76 Dealer ledgers**, one of them the shop's own name, and two of the four carry zero rates. A rate card for four suppliers cannot account for a gap spread across the catalogue and clustering by certification type.

This does not reopen B9 — it closes the last obvious rate-card avenue. B9 has already converged on the reading that these are **hand-priced items**, supported by the invoice evidence that making matched the tag on only 39 of 232 captured lines. Eliminating the two remaining rate cards makes that reading stronger, not weaker.

B9's ask is unchanged and remains a yes/no: did the shop price these by hand?

175. The screen coverage denominator was six short, and nearly twenty-three too many — MEDIUM

Coverage has been reported against legacy_menu.csv , captured from the live menu: **187 distinct screens**. The database carries its own menu in three tables, none ever loaded, and they do not agree with it.

Table	Rows	Links
main_menu	12	0 — headings only
sub_menu	104	80
exception_sub_menu	135	134

Together **210 distinct links**, and our CSV is a strict **subset**: nothing in the CSV is missing from the tables, and **23 links are in the tables and not the CSV**.

Adding all 23 would have been wrong

The first attempt added all of them and the navigation test rejected it:

legacy menu has 212 entries, want 189 legacy menu has **17 top-level headings, want 12**

That second line is the real check. `main_menu` holds the shop's twelve headings — SETUP, RETAIL, MANUFACTURING, INVENTORY, ACCOUNTS, MIS REPORTS, REPORTS, DSS REPORT, GST REPORTS, HR, IMP REPORTS, AUDIT — and every heading in `exception_sub_menu` is one of those twelve. `sub_menu` **is not**: its headings are *Exhibition*, *KARIGAR REPORTS*, *PENDING REPORTS*, *Report* and *STOCK REPORTS*, none of which exists in `main_menu`, and all 104 of its rows belong to a single user, `admin`. It is a separate custom menu, not the standard one.

So 17 of the 23 come from a menu the shop's own `main_menu` does not describe, and adding them would have inflated the denominator with someone's private shortcuts.

The six that are real

Six links sit in `exception_sub_menu` under headings that already exist:

Top	Sub-group	Screen
REPORTS	Karigar Repair Report	<code>issue_from_user_to_diff_karigar.php</code>
REPORTS	Karigar Repair Report	<code>karigar_issue_receive_register_repair.php</code>
REPORTS	Karigar Repair Report	<code>shape_wise_issue_receive_repair.php</code>
REPORTS	Manual Stock Tally	<code>back_date_subgroup_stock.php</code>
REPORTS	Manual Stock Tally	<code>packet_wise_stock_summary_report.php?tally_status=1</code>
REPORTS	Manual Stock Tally	<code>raw_stock_tally.php</code>

Added. **Coverage is now 117 of 193 — 61%, not 63%.**

Five of the six are karigar or raw-stock screens, which [defect 76] shows have almost no data behind them; `raw_stock_tally.php` is the shop's own view of the holding in [defect 78]. They are unlikely to be buildable, but they are part of the menu and the denominator should say so.

The guard test is what made this safe. `TestLegacyMenuLoads` asserts both the row count and that there are exactly twelve headings. The row count would have been updated without thought; **the heading count is what caught the seventeen.** It now records why, so the next person adding menu rows meets the same check.

176. The unopened tables, closed out — 59 of 451 hold anything, and what they hold is now known — LOW

`STATUS.md` has carried "54 never opened" as an open edge for weeks. Measured properly against every loader:

Tables in the dump	45¹
--------------------	-----------------------

Not read by any loader	356
Of those, non-empty	59

The other 297 are empty. That is the real shape of the gap, and it is much smaller than 356.

What the 59 turned out to hold

This session opened eighteen of them. The results are recorded in their own entries and split three ways:

Material — four became defects:

Table	Rows	Finding
karigar_gold_stock	2	₹1.35 crore of gold issued to a named goldsmith ([defect 164])
cla_retail_trans_bar_detail	2,816	281 tags valued differently, 208 with metal counted 4× ([defect 167])
order_rate_master	2,520	resolved [defect 154] — amount_received is metal value, not cash ([defect 169])
ledger_wise_document_upload	104	104 customer documents gone, incl. PAN and Aadhaar ([defect 172])

Checked and correctly excluded — the loaders were right: **address_book_sub_name** (a third ldgr_code space, [defect 170]), **ledger_ref_details** (1 mobile in 1,071 rows), **item_adjustment** (5 meaningful rows in 922), **design_no_status** (no interpretable meaning), **dealer_wise_diamond_rate** and **dealer_wise_making** (2 and 4 dealers, [defect 174]), **sub_menu** (a private admin menu, [defect 175]).

Verified already loaded, and matching exactly: **barcode_create_from_purchase_detail** — its 21 tags' stone detail is in **mcj_stock_stones** at **20.850 carats and ₹11,88,450.00**, the same to the paisa. **retail_all_stock** reconciles to the gram ([defect 166]).

One route checked and rejected today

sale_bill_link (135 rows) maps a bill number to an alternate — and three of [defect 155]'s 39 detail-less bills are mapped:

Bill with no lines	Maps to	Lines there
GST/2627/00009	GST/2627/00015	2
GST/2627/00043	GST/2627/00079	1
GST/2627/00058	GST/2627/00109	1

That looked like a recovery for 3 of the 39. **It is not.** Each alternate is itself a separate invoice that already owns those lines, and the totals are unrelated — GST/2627/00009 is ₹2,46,000.00 against GST/2627/00015's ₹45,154.00. Whatever the mapping records, it is not the missing detail. **Defect 155 stands at 39 bills and ₹91,71,074.79.**

What is left, and what will not be opened

Of the 59, **five are deliberately not opened** — `api_details`, `email_master`, `sms_profile`, `whats_app_url`, `whatsapp_profile` — under [blocker B15], because they carry live credentials. That is a standing decision, not an outstanding task.

The rest are session state (`user_online_status`, 1,190 rows), per-user report preferences (`user_wise_order_report_setup`, 401), small masters (`currency_master` 108, `dept_user` 103) and scratch tables already read for [defect 131] and [defect 133]. None is a store of money, metal or people.

The "never opened" line can stop being read as risk. It is 59 tables, 18 examined, 5 sealed by policy, and the remainder are configuration.

177. What B26 actually costs: a real bill would balance the books and vanish from fifteen screens — HIGH

[Blocker B26] says the posting screens "write a voucher and no detail row". That is right, and it understates what follows. Measured rather than described.

The insert paths

The whole module contains **five** `INSERT INTO mcj_*` statements outside tests:

Table	Written by
<code>mcj_vouchers</code>	<code>voucher_repository.go</code>
<code>mcj_voucher_lines</code>	<code>voucher_repository.go</code>
<code>mcj_stock_items</code>	<code>tag.go</code>
<code>mcj_raw_stock</code>	<code>rawstock.go</code>
<code>mcj_customer_goods</code>	<code>partysample.go</code>

(An earlier count of "four" was taken before the raw stock and party sample screens existed. It is five now, and the shape is unchanged.)

Every posting screen — **sale, old gold, credit note** — does exactly two things: `vouchers.Save(...)`, and for a sale `UPDATE mcj_stock_items SET in_stock = false`. None writes an invoice, a movement or a stone line. `invoice.go` is **read-only** — 0 inserts, 0 updates — so the screen that displays bills cannot create one.

What a sale saved tomorrow would do

Correctly: appear in the cash book, day book, trial balance, ledger book, balance sheet, P&L and GST register, because all of those read the voucher. The shelf value would drop, because `in_stock` flips.

And be invisible to every screen below, because nothing writes the table each one reads:

Unwritten table	Screens that go blind
mcj_sales_invoices	9 — GSTR-1, sold stones, sale cost, MRP sales, sales comparison, sales targets, order-to-barcode, ledger merge, statistics
mcj_sales_invoice_lines	6 — GSTR-1 HSN, sale cost, MRP sales, sales targets, order-to-barcode, statistics
mcj_stock_movements	2 — stock history, hallmarking
mcj_sold_stones	1 — sold stones
mcj_old_gold_lines	3 — old goods, customer metal, statistics
mcj_purchase_invoices	4 — purchase contents, MRP purchases, GSTR-1, statistics

Counting distinct screens, **fifteen registers would not show a bill raised today**, including **the GST return**.

Why this is worse than it sounds

The books would balance. The trial balance would tie, the cash book would foot, and the shelf would go down by the right amount — so nothing on the accounting side would signal that anything was wrong. The failure is silent, and it is silent in exactly the places [defect 155] already shows the shop cannot afford gaps: **37% of historical sales have no line detail, and a new sale would add to that at 100%**.

What to do

This is B26 and it is unchanged: the screens must write their detail rows before the first real bill. The contribution here is the **list of what to write** — five tables, and the fifteen screens that prove each one is load-bearing.

Needs Rahul: nothing. This one is engineering, and it is the single most consequential piece of work left in the rebuild.

178. The one screen that creates stock cannot record stones, certification or hallmarking — HIGH

[Defect 177] established that the posting screens write a voucher and nothing else. The screens that *do* write were then read, and the tagging screen — `tag.go`, the only path that creates a `mcj_stock_items` row — has the same shape of problem.

Its insert names 24 columns. **Ten columns the existing shelf uses are not among them**, and the tag form has no input for any of them:

Column	Tags on the shelf using it
(stone lines in <code>mcj_stock_stones</code>)	562

Column	Tags on the shelf using it
certificate_no / certification_type / certification_rate / certification_charge	566
hallmarked / hallmark_charge	329
category_id	1,461
wastage_percent	22
design_code	109
has_mrp	—

Stones are the sharpest of these

The screen **does** take a stone value — `stone_value` on the form — and pricing carries it: `Amount = GoldValue + StoneValue (pricing.go:82)`. So a new stone-set tag stores the stone money **inside** `amount` and writes **no stone rows at all**: there is no `INSERT INTO mcj_stock_stones` anywhere in non-test code.

That breaks the identity [defect 167] verified on **478 of 478** legacy stone-set tags:

```
amount - stones = rate × net_weight
```

With no stone rows, `amount - 0` is metal *plus* stones, and the identity fails by exactly the stone value. A new tag would also be invisible to the **stone ledger**, and on sale its stones would be missing from **sold stones** — compounding [defect 177] rather than being separate from it.

Certification has a second-order effect

`quote.go` refuses to price any certified tag, keyed on `certificate_no` and `certification_charge` ([defect 161] — 566 tags, ₹2,84,75,794.50). A tag created by this screen can never carry either, so a genuinely certified piece **would be priced by the quote screen** rather than refused. That is the exact case [blocker B9] says cannot be priced safely.

What this means

The tagging screen produces a **plain metal tag**: barcode, item, purity, weights, rate, making, total. For the 40% of the shelf that is plain metal that is complete and correct. For a stone-set, certified, hallmarked or categorised piece — **which is most of the shelf by value** — it silently records less than the piece is.

Needs Rahul: nothing. Like B26 this is engineering, and it belongs with it: the same work that makes a sale write its detail rows should make a tag write its stones. Recorded separately because they are separate screens and the tagging one is quieter — it produces a row that looks complete.

179. Two smaller write-path gaps, and two that turned out fine — LOW

Completing the read of the four screens that can write, after [defect 178] found the tagging screen omits ten columns. The other two are much better, and the differences are worth recording precisely so this is not re-derived.

`rawstock.go` — **complete, but hardcodes who holds the metal**

Its insert covers every meaningful column of `mcj_raw_stock`. Two are constants:

```
VALUES ($1,$2,$3,$4,$5,$6,$7,'sh','Dr')
```

`holder` and `side`. The loader maps them from legacy `barcode_main_rawstone.type` and `.dr_cr`, and **all 10 legacy rows are** `sh / Dr` — so the constants match every row that exists, and nothing is wrong today.

They are not decoration, though. `holder` is what distinguishes *the shop holds this* from *someone else holds this*, and [defect 164] found **869.850 g of fine gold issued to MA TARA JEWELLERS** with nothing recording its return. The one screen that can record raw metal cannot express that the metal is out. Whoever implements B26's detail rows should widen this at the same time.

`partysample.go` — **one column short**

Its receive path writes 11 columns of `mcj_customer_goods` and omits `item_id`. All 7 existing rows carry it, set by the loader from the item master; a receipt taken on the screen would carry NULL.

Nothing reads it today — `customergoods.go` and `ledgermerge.go` both use `description` — so this is latent rather than live. It is a one-line fix and should not wait for a report to need it.

Checked and correct

- **The return path is complete.** `partysample.go` sets `returned = true`, `returned_on` and `return_note` together, which is exactly what migration `mcj-1787385000` added those columns for: *"a stone set into an order and a stone handed back over the counter are different events"*.
- `voucher_repository.go` writes both `mcj_vouchers` and `mcj_voucher_lines`. The voucher side of every posting screen is sound; [defect 177] is about what is missing **beside** the voucher, not the voucher itself.

So of the five write paths: **one is materially incomplete** ([defect 178]), one hardcodes a field that will matter ([defect 164]), one is a column short, and two are correct.

180. No permission says "may create stock", so the tagging screen borrows one named for printing — MEDIUM

Following the write-path audit ([defect 177] to [defect 179]) into the gates on those same paths. **Nothing is ungated** — every POST requires a permission, which is the fix [defect 107] put in. The problem is which permission.

Screen	What it writes	Gated on
sale	voucher + lines	VoucherCreate
old gold	voucher + lines	VoucherCreate
credit note	voucher + lines	VoucherCreate
tag	a stock item — value onto the shelf	TagPrintUpdate
categorise	the category on up to 2,575 tags	VoucherUpdate
raw stock	a raw metal holding	ItemCreate
party sample	customers' goods held	VoucherCreate (no voucher written)

The three voucher screens are right. The other four are gated on a permission from a different domain, because **the permission vocabulary has no word for stock**. Of 144 permissions in the module, **not one** is named `StockCreate`, `BarcodeCreate` or `TagCreate` — the set runs `ItemCreate` (the item *master*), `TagPrintRead` / `TagPrintUpdate` (tag *printing*) and `VoucherCreate` / `VoucherUpdate` / `VoucherRead`.

So:

- **Whoever may print tags may create stock.** `TagPrintUpdate` reads as a printing right and grants the act that puts a priced item on the shelf.
- **Whoever may amend a voucher may re-categorise the shelf.** `categorise.go` can rewrite the category on any of the 2,575 tags, gated on `VoucherUpdate`.
- **Party sample writes no voucher** yet requires `VoucherCreate`, so the gate is both wrong in name and stricter than needed.

This is least-privilege, not an open door

Nothing here is reachable by an unauthenticated user, and nothing is unprotected. The risk is that a grant made for one reason silently confers another — and the shop's legacy access matrix shows this is not hypothetical: **24 roles across 2,755 grants**, with tag and barcode screens granted to at least six of them (Ashok 21 grants, ankush 19, GKS 19, rps 18, Bapi 17, anand 15). Whoever maps those roles onto these permissions will be choosing, without knowing it, who can create stock.

Not changed. Adding `StockCreate` and `StockUpdate` means renaming gates that [defect 107] has only just made correct, and the mapping from legacy roles is not yet built — doing it now would be guessing at an allocation the shop has not made. It belongs with that work.

Needs Rahul, when roles are set up: who may create a tag, and who may re-categorise stock. Those are two separate answers today given by one grant each for the wrong thing.

181. Two tills tagging the same barcode would both succeed — MEDIUM

`tag.go` guards barcode uniqueness carefully, and says why:

The uniqueness check is repeated here rather than trusted from the preview. Between pricing and saving somebody else may have used the number, and two pieces sharing a barcode is exactly the defect this rebuild inherited.

The intent is right and the implementation cannot deliver it. `barcodeInUse` runs

```
SELECT count(*) FROM mcj_stock_items
WHERE tenant_id = $1 AND btrim(barcode) = btrim($2) AND in_stock
```

— a plain read, no `FOR UPDATE`, nothing to lock because the row does not exist yet. No isolation level is set anywhere in the module, so PostgreSQL's default **READ COMMITTED** applies: two concurrent saves of the same barcode each see zero, each insert, and both commit.

Nothing in the database stops them. The only barcode index is not unique:

```
CREATE INDEX idx_mcj_stock_items_barcode ON mcj_stock_items (tenant_id, barcode)
```

So the check-then-insert is the whole of the protection, and it is exactly the pattern that does not hold under concurrency.

The fix is blocked by the defect it was written to prevent

The right guard is a partial unique index matching the check's own semantics — unique among tags **in stock**, which leaves the shop's deliberate practice of reusing a sold tag's barcode alone (1,974 of 3,021 barcodes have been reused this way):

```
CREATE UNIQUE INDEX ... ON mcj_stock_items (tenant_id, btrim(barcode)) WHERE in_stock
```

It cannot be created today. [Defect 146]'s duplicate is barcode **86065435** — two MALA pieces at ₹900 each, **both** `in_stock` — and it is the only one. Confirmed: exactly one barcode has more than one in-stock copy.

So the chain is: **defect 146 must be resolved before this race can be closed**, and defect 146 needs Rahul to re-tag one of the two pieces. Until then the index would be rejected by the very row that proves it is needed.

Interim, if the shop starts tagging from two terminals before that: a transaction-scoped advisory lock keyed on the barcode would serialise the check and insert without needing an index. Not applied — it is a workaround for a race that a single till cannot hit, and the real fix is one re-tag away.

Severity is bounded by how the shop works. One person tagging at one screen can never trigger it. Two people tagging simultaneously, or any future import running beside the screen, can.

182. Requests are not transactional, and a rejected save can leave half its rows behind — HIGH (proven)

Chasing transaction boundaries through the write paths found that **there are none**, and it was confirmed against the running application rather than inferred.

The proof

A party-sample receipt was posted with two lines: the first valid, the second carrying a carat value too large for `numeric(14,3)`.

```
http 422
receipt_no | description          | pieces | carats
TXTEST/001 | TX PROBE LINE ONE |      1 | 1.000
```

The request was rejected and the first line was written anyway. The probe row was deleted immediately; `mcj_customer_goods` is back to its 7 rows, verified.

Why

`composables.UseTx(ctx)` returns the transaction from the context **if one is there**, and otherwise falls back to `UsePool(ctx)`:

```
func UseTx(ctx context.Context) (repo.Tx, error) {
    tx := ctx.Value(constants.TxKey)
    if tx == nil {
        return UsePool(ctx)    // <- a pool, not a transaction
    }
    ...
}
```

Nothing puts one there. **No middleware in the SDK sets** `TxKey` — checked every file in `pkg/middleware`, none references `TxKey` or `pool.Begin` — and **no controller in this module calls** `composables.InTx`. So every `tx.Exec` in a hand-written controller runs on the pool and commits on its own.

What that costs, screen by screen

Screen	Statements per save	Partial outcome
sale	voucher, then N line inserts, then the stock UPDATE	a committed voucher whose tags are still on the shelf
old gold / credit note	voucher, then N line inserts	a voucher with some of its lines — an unbalanced voucher
party sample	one insert per line	proven above
tag, raw stock	one insert	safe by having nothing to divide

The sale case is the sharpest. Its stock update is written to refuse the sale if a tag was sold in the meantime:

```
"%d of %d tags were no longer on the shelf — someone else may have sold them"
```

That refusal is correct and now cannot help: **the voucher is already committed** when it fires. The books would carry a sale the shop declined, with the goods still on the shelf.

The tests cannot catch this

`itf` wraps every request in a real transaction — `suite.go:195` does `ctx = composables.WithTx(ctx, s.env.Tx)` — and rolls it back between tests. So in tests `UseTx` returns a transaction and everything is atomic; in production it returns a pool and nothing is. **The suite is green precisely because it does not reproduce the condition.** That is why 86 test files and a full integration run have never shown it.

What to do

Wrap the mutating handlers in `composables.InTx`. The SDK's generic CRUD service already does exactly this (`crud/service.go:157` and `:218`), which is why the 18 CRUD screens are safe and the seven hand-written ones are not.

Fixed and proven

An `atomic` wrapper now runs each mutating handler inside one `composables.InTx`, rolling the transaction back when the handler answers 4xx or 5xx. The response is already written by then, which is the wanted outcome: **the client is told no and the database agrees**. Applied to all seven hand-written mutating routes — sale, old gold, credit note, tag, raw stock, party sample, categorise.

Verified against the running app with the same probe that proved the defect, and with its counter-test:

	Before	After
Two-line receipt, second line invalid	422 — line one persisted	422 — 0 rows left , table unchanged at 7
Valid single-line receipt	<i>(not tested)</i>	200 — row written and present

The counter-test matters as much as the fix: a wrapper that rolled back everything would also return 200 and lose the sale. Both probe rows were deleted; `mcj_customer_goods` is back to 7, verified.

Integration suite green.

Needs Rahul: nothing. It was engineering, it was small, and it belonged **before** [B26]'s detail rows — writing more rows per save without a transaction would have widened the failure rather than narrowing it.

183. My own fix for defect 182 covered seven routes of ten — MEDIUM

[Defect 182]'s fix was applied to "all seven hand-written mutating routes". That list came from [defect 177]'s write-path audit, which counted `INSERT INTO statements` — and so counted the voucher repository once, as one path, without asking how many screens reach it.

Checking the fix by a different measure — **every non-GET route in the module**, which is the question the fix is actually answering — found three more:

Route	Writes	Was wrapped
<code>voucher.go</code> → create	<code>repo.Save</code> — voucher + N lines	no
<code>voucher.go</code> → update	<code>repo.Save</code> — UPDATE, DELETE all lines , re-insert	no
<code>reconcile.go</code> → update	UPDATE <code>mcj_vouchers</code> in a loop	no

The voucher update path was the worst one left. `voucher_repository.go` handles an edit by deleting every line of the voucher and inserting the new set. Without a transaction, a failure between the delete and the inserts leaves a voucher with **no lines at all** — not merely unbalanced, empty. That is the general journal, payment and receipt entry screen.

`reconcile.go` marks cheques cleared one UPDATE at a time, so a failure part-way through leaves some cheques reconciled and some not, with the screen reporting an error over a partly-applied change.

All three are now wrapped. **Ten of the module's twelve POST routes are atomic**; the two that are not — `stocktake.go` and `taglabel.go` — were checked and write nothing, being a report filter and a label preview.

The lesson is the same one as defect 160

That entry recorded a sweep whose clean result was "a property of the test, not of the reports". This is the same shape: the fix was verified against the list that produced it. **Counting inserts answers "what writes?"; counting routes answers "what can a user submit?" — and the second is the question a transaction boundary belongs to.**

Full integration suite green after the change.

184. Voucher numbers are race-safe; almost nothing else is — MEDIUM

Following [defect 181]'s barcode race to its parallel: does the same check-then-write pattern threaten voucher numbers?

It does not, and the reason is instructive. `nextNumber` is exactly the vulnerable shape —

```
SELECT COALESCE(MAX(number), 0) + 1 FROM mcj_vouchers
WHERE tenant_id=$1 AND type=$2 AND financial_year=$3
```

— two concurrent saves would read the same maximum and choose the same number. But the database refuses the second:

```
CREATE UNIQUE INDEX mcj_vouchers_tenant_id_number_type_financial_year_key
ON mcj_vouchers (tenant_id, number, type, financial_year)
```

The race exists and cannot produce a duplicate. **The constraint is doing the work the query cannot**, which is the whole argument of defect 181 stated the other way round.

A unique index is not the same as a protected table

The schema carries **133 unique indexes**, which reads as thorough until they are read for what they cover. The two that matter most are keyed on the **migration** id, not on anything a user enters:

Table	Unique index	Constrains a new row?
mcj_stock_items	(tenant_id, legacy_id) WHERE legacy_id IS NOT NULL	No — an app-created tag has legacy_id NULL, so the index does not apply to it at all
mcj_sales_invoices	(tenant_id, legacy_id)	No — same

So the tag table looks constrained and is not, which is precisely how [defect 146]'s duplicate barcode survived and why [defect 181] can happen.

Tables with no business key at all

Of the transactional tables, **eight** have no unique index beyond the primary key: mcj_voucher_lines , mcj_stock_movements , mcj_stock_stones , mcj_sold_stones , mcj_old_gold_lines , mcj_purchase_invoices , mcj_customer_goods , mcj_raw_stock .

Most are line tables where duplication is legitimate — a voucher may debit the same ledger twice, a tag may hold two identical stones — and [defect 168] records that the legacy line table had no key either, one id being shared by 51 rows. So this is mostly correct.

mcj_purchase_invoices is the odd one: it is a **header** table with a bill number, and nothing stops the same bill being recorded twice. Its sales counterpart is only protected on legacy_id , which no new row will have.

What to do

Nothing here is a live fault — no screen writes purchase invoices at all today ([defect 177]). It becomes one the moment [B26] makes them writable, and the cheapest time to add (tenant_id, bill_no) is before there is data to conflict with it.

Recorded rather than fixed, because adding constraints to tables nothing writes yet is guessing at the shape B26 will give them.

185. The transaction wrapper would have committed a panicking handler — MEDIUM

Auditing [defect 182]'s own fix. Two things were checked and one was wrong.

Checked and safe: the response is rendered inside the transaction

atomic writes the response before the transaction closes, so a handler whose error page reads the database could show a row that the rollback is about to remove. Every error path was read: they render from **props already in memory**, and the only database read on a failure path is billableParties — the ledger list, which no save touches. Nothing displays a doomed write.

Wrong: a panic committed

`composables.InTx` has **no deferred cleanup**:

```
if err := fn(WithTx(ctx, tx)); err != nil {
    ... tx.Rollback(ctx) ...
}
return tx.Commit(ctx)
```

If `fn` panics, neither branch runs — the transaction is abandoned mid-flight, holding its pooled connection, with whatever it had written neither committed nor rolled back. The SDK recovers panics further up the stack (`middleware/auth.go`, `middleware/logging.go`), so the request answers 500 and the data is left in limbo.

Fixed. The wrapper now catches the panic, sets the named return so `InTx` rolls back, and raises it again **after** the transaction is closed — leaving the SDK's recovery middleware to answer the request exactly as before:

```
var panicked any
_ = composables.InTx(r.Context(), func(ctx context.Context) (err error) {
    defer func() {
        if p := recover(); p != nil { panicked = p; err = errRolledBack }
    }()
    ...
})
if panicked != nil { panic(panicked) }
```

Recovering without re-raising would have been worse than the bug — a panicking save would answer 200 with an empty page.

And a test for the rule itself

The wrapper's whole behaviour is one comparison: `status >= 400` unwinds, anything else commits. **A redirect is how every successful save answers** — the sale screen ends with `http.StatusSeeOther` — so a wrapper that treated 3xx as failure would roll back every completed sale and report success. That is now asserted directly, along with implicit 200, 422 and 500.

Three unit tests, no database needed.

186. The report gate was tested on three reports of sixty-nine, and never on the export — MEDIUM

[Defect 107] found every report served to any authenticated user and gated them. The test written alongside it, `TestReports_RequireAPermissionAndNotJustALogin`, checks **three** reports — `salary-sheet`, `trial-balance`, `user-rights` — and only the screen, never `export.xlsx`.

The gate itself is right. `report.go` registers both routes through the same call:

```
router.HandleFunc("/"+key, gateAll(required, c.show))
router.HandleFunc("/"+key+"/export.xlsx", gateAll(required, c.export))
```

So nothing was exposed. But the export is **the same data in a more portable form** — a spreadsheet a user can keep — and it was protected by a line nobody had ever exercised.

Widened

The test now sweeps **every key in the registry**, both routes, and refuses to run at all if the registry holds fewer than 60 reports — so trimming the registry cannot quietly shrink the sweep:

	Before	After
Reports covered	3	69
Routes per report	screen only	screen and export
Assertions	3	138

Confirmed non-vacuous. Removing `gateAll` from the export route produces **67 failures**, each naming the report it served. Restored, green.

Why three was not enough

The three chosen are the obviously sensitive ones — salaries, the trial balance, user rights. That is the instinct that picks a sample, and it is the wrong instinct for a gate: defect 107 was not "three screens were exposed", it was **every screen in a class**, found only because the whole class was checked. A sample proves the three; a sweep proves the rule.

187. Nothing asserted that the CRUD screens use the tenant guard — MEDIUM

[Blocker B11] found that `crud.DefaultRepository` does not scope reads to the tenant, and `persistence.TenantScoped` was written to fix it. Four isolation tests prove **the wrapper works**: it filters List and Count, refuses Get and Exists on another tenant's row, and cannot be told which tenant to write to.

Nothing proved the screens use it. A new CRUD controller written without the wrapper would leave all four isolation tests green and serve every tenant's rows. That is B11 reintroduced one file at a time, and it would look like a normal controller.

The 18 call sites were checked by hand while investigating [defect 180]. A count taken by hand is true on the day it is taken.

Now a test. `TestEveryCrudRepositoryIsTenantScoped` reads every controller source and fails if any `crud.DefaultRepository[` is not immediately preceded by `TenantScoped(` . It also refuses to pass if fewer than 15 generic repositories are found, so removing the CRUD screens cannot make the guard vacuously true.

Proven non-vacuous, and not by mutation. Unwrapping a real controller orphans the `persistence` import and fails the build — which proves nothing about the assertion, only that Go noticed an unused import. The rule is therefore split into `unwrappedRepositories(src string)` and exercised directly against three synthetic sources: a wrapped call (0 flagged), a bare call (1 flagged), and both together (exactly 1). That is the check testing itself rather than testing the compiler.

A note on the attempt

The first mutation attempt restored from a backup path, `/tmp/color.bak`, that already existed from unrelated work. It wrote a 137-line `package color` file into the controllers directory and broke the build with *"found packages controllers and color"*. Removed, build verified clean, and the retry used a session-scoped scratch path. Recorded because a backup filename that is not unique is a trap, not a mishap.

188. The transaction fix was held in place by a grep, not a test — MEDIUM

[Defect 182] made the mutating routes transactional; [defect 183] found the first attempt had covered seven of ten. Both were verified by reading the source at the time. Neither left anything behind that would notice a POST route added tomorrow without `atomic`.

That matters more than the tenant guard of [defect 187], because the failure is silent in both directions: an unwrapped route looks identical, behaves identically until something fails part-way, and then leaves half its rows committed — which is exactly what defect 182 demonstrated against the running app.

Now a test. `TestEveryPostRouteRunsInATransaction` walks the controller sources and requires every `Methods(http.MethodPost)` registration to be wrapped:

```
12 POST routes, 10 wrapped, 2 exempt by name
```

The two exemptions are named individually with the reason — `stocktake.go` is a report filter and `taglabel.go` a label preview, neither writing anything — so adding a third is a deliberate act rather than an oversight. The test also fails if fewer than 10 POST routes are found, so deleting the posting screens cannot make it pass by having nothing to check.

Proven non-vacuous: unwrapping the raw-stock route makes it fail by name —

```
1 POST route(s) are not wrapped in atomic() and are not exempt:
  rawstock.go: router.HandleFunc("", gate(permissions.ItemCreate, c.post)).
```

Restored and green.

The first version of the guard was wrong, and said so

Reading one line at a time, it reported `creditnote.go` as unguarded. That file registers across four lines and closes with a bare `}})).Methods(http.MethodPost)` — the `atomic` sits three lines above. The check now takes the whole statement, from its `router.HandleFunc` back to the `Methods` call.

Worth recording because the failure was a **false positive on correct code**, and the tempting response is to add an exemption. The right response was to fix the check: an exemption would have hidden `creditnote.go` from the guard permanently, so the day someone unwrapped it nothing would have said so.

189. The doubled-footer rule cannot be a unit test, so it is a script — LOW

[Defect 186] to [defect 188] replaced hand checks with tests. The same was attempted for [defect 160]'s rule — that no report's footer may count a summary row the query already produced — and it does not fit in a test.

A source-level rule does not hold. The obvious one, *"a report that appends its own report.Row must set NoTotals"*, is wrong: many reports correctly append a note row carrying no money at all — `audit.go`, `daybook.go`, `dataquality.go`, `gst.go` among them. Requiring `NoTotals` of those would be requiring them to suppress a footer they should have.

A seeded test would pass over nothing. The property only appears when a report has enough rows to produce a subtotal. Against `itf`'s fixtures most reports return two or three rows, so the detector would examine nothing and go green — the same vacuity caught in [defect 162] and [defect 187].

So it stays a sweep, and the sweep is now a **script that can be re-run** rather than something reconstructed by hand each time:

```
python3 tools/check_report_footers.py --port 3250
```

It fetches every report from a running instance, applies defect 160's detector — a subtotal is a row equal to the sum of every row above it, with at least four rows above so a coincidence is not mistaken for one — and exits non-zero on a find.

Clean tree	69 reports checked, 0 doubled footer(s), exit 0
<code>NoTotals</code> removed from <code>sale-cost</code>	3 doubled columns named, exit 1

Two things this cost, both mine. The first check of the exit code piped the script into `tail`, so the shell reported `tail`'s status and not the script's. The second read exit 1 on what should have been a clean tree — because restoring the source and running `go build ./...` does not rebuild the binary the server runs from; the scratch build still held the mutation. Both are the same mistake: **verifying a thing through something that is not the thing.**

190. ₹69 lakh of customer debt has not moved, and the shop holds ₹21 lakh of customers' money — MEDIUM

The customer ledger position, which no entry in this record had yet stated. Of **485 customer accounts**, 457 are square and 28 are not:

	Accounts	Amount
Owe the shop	10	73,96,553.00
Shop owes them	18	21,41,852.00

The debt is dormant, not current

Splitting by whether the account saw any posting in the captured period:

	Accounts	Amount
Owes — no posting at all	6	69,28,065.00
Owes — active	4	4,68,488.00
Shop owes — no posting at all	4	11,52,636.00
Shop owes — active	14	9,89,216.00

94% of the receivable is an opening balance that has not moved. The three largest are entirely inherited:

Customer	Opening	Movement	Balance	Postings
HARSIT ANAND SULTANIA	32,30,681.00	0.00	32,30,681.00	0
AMIT SINGH YADAV	25,55,000.00	0.00	25,55,000.00	0
SHEJAN TAJ	7,81,837.00	0.00	7,81,837.00	0

Two customers account for **₹57,85,681.00** — 78% of everything owed to the shop — and neither has a single transaction in the period the books cover.

It cannot be aged. [Defect 142] puts the first voucher at 9 March 2026, so there is nothing to say whether these balances are three months old or ten years old. An opening balance with no history is a number, not a debt with a date.

The other side is customers' money

₹21,41,852.00 sits on 18 customer accounts in credit, the largest being **SHARON WINES at ₹10,50,000.00 with no postings at all**. A credit balance on a customer is money taken and not yet turned into goods — the same shape as [defect 152]'s matured schemes and, unlike [defect 169]'s order figures, this one **is** money: it is in the ledgers.

What to do

Needs Rahul, and this is a collections question rather than a data one:

1. Are **HARSIT ANAND SULTANIA (₹32,30,681)** and **AMIT SINGH YADAV (₹25,55,000)** live debts, or written-off balances carried forward? Together they are 78% of the shop's receivables.
2. What is **SHARON WINES' ₹10,50,000 credit** — an advance against goods, or a deposit to return?

Nothing here is a fault in the data or the rebuild. It is a position the shop should see stated, and no screen states it: the supplier side has one ([defect 145]'s supplier balances) and the customer side does not.

191. Twelve cheque payments cannot be reconciled, and the first count of them was wrong — LOW

The bank reconciliation screen matches on a cheque number. Counting where those numbers live:

Vouchers naming a cheque in the narration text	163
Of those, carrying the number on a voucher line	151
Invisible to reconciliation — narration only	12

The reconcile query reads the line-level number first, then the voucher-level one, so 151 of the 163 are perfectly reconcilable. **Twelve are not:** their cheque number exists only inside a sentence such as "*Ch. No.435590, Dt.05.12.2026 Issued Against Their Dues*", and no query can match on that.

The first reading of this was wrong and is recorded as such. Counting only

`mcj_vouchers.cheque_number` gave "*34 structured, 163 narration-only*" and implied 83% of cheque payments were unreconcilable. That ignored `mcj_voucher_lines.cheque_number`, which holds **593** numbers. The real figure is 12 — a twelfth of what the first count suggested, and the difference was one table.

Also checked and correct: the one voucher dated in the future — PMNT 576, **5 December 2026**, ₹1,35,370.00 to DAYANIDHI TRADE PVT LTD — is a **post-dated cheque**, exactly as its narration says. It is properly balanced and correctly excluded from an as-on-today balance sheet, which is why the sheet reads ₹4,91,62,109.87 against ₹4,92,97,479.87 counted across all dates.

Not fixed. Extracting a cheque number from prose is guesswork, and twelve vouchers is a smaller job by hand than a parser is to trust.

192. A sale now records what was sold — B26's sales half, done and proven — HIGH (fixed)

[Defect 177] measured what [blocker B26] costs: every posting screen wrote a voucher and nothing else, so a bill balanced the books and was invisible to fifteen registers including the GST return. The sales half is now written.

`writeSaleDetail` runs inside the same transaction as the voucher and the stock update, and writes four things in the order a bill is read:

Table	What it records
<code>mcj_sales_invoices</code>	the bill — number, customer, place of supply, goods, making, rounding, total
<code>mcj_sales_invoice_lines</code>	one row per line, with <code>item_id</code> , purity, weight, rate and making
<code>mcj_stock_movements</code>	the piece leaving the shelf, referencing the bill
<code>mcj_sold_stones</code>	the stones, copied from the tag so the sold record matches the shelf

The bill number follows the shop's own format, taken from its 224 captured bills: `GST/2627/00225`.

Proven against the running app

A real sale was posted for a stone-set 18kt nose pin and every count moved by exactly one:

	Before	After
Vouchers	1,848	1,849
Invoices	224	225
Invoice lines	328	329
Stock movements	9,036	9,037
Sold stones	203	204
Tags in stock	2,575	2,574

And it appears in the registers that were blind. The GSTR-1 HSN summary moved by exactly the sale:

HSN 711319	Before	After	Difference
Quantity	34.956 g	35.206 g	0.250 g — the tag
Taxable	5,64,137.00	5,67,836.50	3,699.50 — the goods
Tax	16,924.16	17,035.15	110.99 — CGST + SGST

Stock history, sold stones and sale cost all show the bill by name. `gstr-sales` and `mrp-sales` correctly do **not**: the first is the B2B section and this is a counter sale, the second is MRP goods only.

The test sale was reversed in full — all six counts and the shelf value ₹14,23,61,188.20 verified back to where they started.

Two mistakes worth keeping

Four columns that do not exist. The first version inserted `taxable`, `cgst`, `sgst` and `igst` onto the invoice header. Go compiles SQL as a string, so the build passed; the first real sale would have failed. The table deliberately has no tax columns — the GST is read from the voucher's own tax lines, which is how the GSTR screens already compute it for the captured bills, and storing it twice would let the two drift.

A line with no item. The second version wrote the line but not its `item_id`, and the HSN summary joins line → item → HSN code. The sale recorded perfectly and was still absent from the GST return. That is [defect 148] and [defect 179]'s lesson a third time: **a detail row without its foreign key is a row that reports cannot see.**

And one wrong test. The check "is the bill number in the report?" reported the HSN summary as blind when it was not — that report groups by HSN code and never prints a bill number. The figures had to be compared before and after instead.

What is left of B26

Old gold and credit note still write a voucher only. Their detail tables — `mcj_old_gold_lines` and the return side — are next.

193. Old gold now records the metal, not just the payment — HIGH (fixed)

The second of [blocker B26]'s three posting screens. Before this, buying back a customer's chain wrote a PURC-OG voucher and nothing else: the money was in the books and **the metal was in no record at all**. `mcj_old_gold_lines` is read by the Old Goods Register, Customer Metal and Statistics, and nothing wrote it ([defect 177]).

The screen already computed each lot — purity, net weight, buy-back rate, amount — and then threw the detail away, passing `save` only a total and a **count**. It now passes the lots.

`writeOldGoldLines` runs inside the same transaction as the voucher and writes one row per lot, numbered in the shop's own format: `OGP/2627/00026`.

Proven against the running app

A 5.000 g lot of 22kt was bought back at the buy-back rate:

OGP/2627/00026		party 12843		22		5.000 g		rate 14,253.00		71,265.00		PURC-OG 26
----------------	--	-------------	--	----	--	---------	--	----------------	--	-----------	--	------------

$5.000 \times 14,253.00 = 71,265.00$, exactly. It appears in both registers that read the table:

- **Old Goods Register** — 23 Aug 2026 | PURC-OG 26 | ABDUL RAHMAN | 22 | 5.000 | 14,253.00 | 71,265.00 | Set against their account
- **Customer Metal** — the same lot, closing *"the shop owes this customer"*

Reversed in full: 25 old gold lines, 1,848 vouchers, 5,229 voucher lines and the shelf at ₹14,23,61,188.20, all verified back.

Tunch is written as zero, deliberately

The screen does not ask for touch and none is invented. [Defect 38] established that this shop prices old gold as **rate x NET weight with the purity carried in the rate** — tunch plays no part in the money — and that its legacy `fine_wt` column was so unreliable it was left unloaded. A fabricated touch figure would be a guess printed as a fact, which is the failure this record exists to prevent.

Two wrong turns, both mine, both the same shape

A purity that has no buy-back rate. The first attempt used `OLDGOLD`, which is what the 22 captured lots carry — and the screen correctly refused: *"there is no buy-back rate for OLDGOLD on or before 23 Aug 2026."* Buy-back rates exist for the numeric grades (9, 14, 18, 22, 95, 995, SLV), not for the legacy bucket name. The screen was right and the test was wrong.

A report key that does not exist. Checking visibility, `old-goods` returned zero rows and looked like a failure. The key is `old-goods-register`. The report was working the whole time.

Both are the error of [defect 189]: **verifying a thing through something that is not the thing**. Three times now in one session.

And a third: `go build` passed on a broken format string

Changing `save` to take the lots rather than a count left the narration reading `fmt.Sprintf("... %d lot(s)", lots)` — a slice formatted as a number. `go build ./...` **reported success**. It is a `go vet` finding, not a compile error, and `vet` runs as part of `go test` — so the integration suite caught it and the build did not.

Worth keeping beside [defect 192]'s four non-existent columns: that was SQL the compiler cannot see, this is a format string it will not check. **Two different ways a green build meant nothing, on consecutive screens.**

B26 remaining

Credit note — the sale-return side — still writes a voucher only.

194. B26 is closed on the trade path: all three posting screens now record their detail — HIGH (fixed)

The credit note was the last of the three. Returning a customer's goods wrote a voucher and set `in_stock = true` — the tag reappeared on the shelf with nothing saying when or why, which is [defect 192]'s silence seen from the other side.

It now writes a `SALE-RET` movement in the same transaction, taking the weight and purity from the tag rather than re-entering them. The same code serves all four trade kinds, so a purchase return records `PURC-RET` without further work.

Proven against the running app. A ₹5,000 credit note against a 76.264 g 18kt diamond bracelet, and the tag's own history now reads in order:

23 Aug 2026		70630587		DIAMOND BRACELET		returned by customer		SALE-RET 4
15 Apr 2026		70630587		DIAMOND BRACELET		returned by customer		SR/2627/00001
02 Apr 2026		70630587		DIAMOND BRACELET		sold		GST/2627/00004

The new return sits beside the shop's own captured one from April, in the same form. Reversed in full — 1,848 vouchers, 5,229 lines, 9,036 movements, 2,575 tags and the shelf at ₹14,23,61,188.20, all verified back.

Where B26 now stands

Screen	Writes
Sale	voucher, invoice, invoice lines, stock movement, sold stones ([defect 192])
Old gold	voucher, old gold lines ([defect 193])
Credit note / trade returns	voucher, stock movement

A bill raised today is no longer invisible. The fifteen registers [defect 177] listed now see a sale, and the GST return moves by exactly the sale's own figures.

What B26 does not yet cover: there is still no screen that records a **purchase invoice**.

`mcj_purchase_invoices` and `mcj_purchase_contents` hold the shop's 43 captured bills and nothing writes them, so buying stock from a supplier remains voucher-only. That is the next piece, and it is the one [defect 184] warned to add a `(tenant_id, bill_no)` constraint to **before** it becomes writable.

195. Buying from a supplier now records the bill — the trade path is complete — HIGH (fixed)

The last gap in [blocker B26]. `mcj_purchase_invoices` held the shop's 43 captured bills and nothing created a forty-fourth, so every purchase was voucher-only and the purchase-side registers were frozen at migration ([defect 177]).

Purchases run through the same trade controller as returns — the kinds `purchase` (PURC-GSC) and `purchase against an order` (PURC-GCO). Both now write a header in the same transaction, numbered in the shop's own format `GDP/2627/00025`. The other two kinds are deliberately excluded: a **sale return is not a purchase**, and a **debit note reverses one** rather than making one — which matches the data, where 43 bills sit against 25 PURC-OG and 3 PURC-RET vouchers carrying none.

Unlike the sales header, this one **does** carry the tax split — the table has `taxable`, `cgst`, `sgst` and `igst` and the captured bills fill them. That is the opposite of [defect 192]'s mistake, and it is opposite for a reason: the two tables genuinely differ.

Proven. A ₹1,00,000 purchase from A 1 SILVER:

```
GDP/2627/00025 | 12835 | A 1 SILVER | interstate | taxable 1,00,000.00 | total 1,03,000.00  
| PURC-GSC 25
```

Reversed in full — 43 invoices, 1,848 vouchers, 5,229 lines, shelf ₹14,23,61,188.20, all verified back.

The constraint went in first

[Defect 184] warned that `mcj_purchase_invoices` is a header table with a bill number and no uniqueness, and that the cheapest moment to fix it is before anything writes to it. Migration `mcj-1787408000` adds

```
CREATE UNIQUE INDEX ... ON mcj_purchase_invoices (tenant_id, btrim(bill_no))  
WHERE btrim(bill_no) <> ''
```

applied before the first write, with all 43 existing bills verified distinct beforehand. Unlike [defect 181]'s barcode index — which cannot be created because a duplicate already exists — this one went in unopposed because it was done at the right time.

A wrong expectation, checked before it became a defect

The test purchase recorded **interstate** with the tax in IGST, while the form said `interstate=0`. That looked like the flag being ignored. It is not: the screen overrides a stale form value when the party has changed, and **A 1 SILVER is in Maharashtra against the shop's West Bengal**. IGST is correct.

The check also settled something the record had left vague: **562 of 785 ledgers carry a state**, including 433 of 485 customers. [Defect 150] found the contact columns empty and it would have been easy to assume `state` was among them. It is not, which is why the sale writer takes the party's own state and falls back to the shop's only for the 52 customers without one.

B26 is closed

Screen	Records
Sale	invoice, lines, movement, sold stones
Purchase / purchase against order	purchase invoice
Old gold	old gold lines
Credit note / debit note	stock movement

A day's trade — buy, tag, sell, take goods back, buy old gold — now leaves a record on every screen built to read one.

196. Two screens built, and each one found something in the data — LOW

Starting the 42 screens that have data behind them. Both of these came with a finding, which is the usual pattern: building a screen is how the data gets read properly.

Subgroup Stock Tally — `tag_untag_subgroup.php`

The shelf grouped the way it is counted, leading with tags and weight rather than value, because a tally is a sheet somebody carries around with a pen. Ten subgroups, **2,575 tags, ₹14,23,61,188.20** — tying to the shelf exactly.

369 tags carry no weight at all and the screen says so on the row rather than showing a blank: 218 imitation pieces are counted by the piece and never weighed, and 16 22kt and 5 silver tags have no weight recorded when their neighbours do. The first is correct; the second is worth someone's eye.

User Master — `user_master.php`

Who could sign in to the old system and what each was trusted with. **24 logins, 11 of whom could change something after the fact** — back-date a voucher, change a rate, edit a barcode, move a due date. Those four were worth a column each in the legacy master because every one alters something already recorded.

Read-only and deliberately so: these are the shop's OLD logins as the dump found them, not accounts in this rebuild. **No password or security-code column is read** — they are in the dump and [blocker B15] keeps them shut.

`role_code` is not a role code

The first version joined `mcj_legacy_access.role_code` to `mcj_legacy_users.role_code` and every user rendered *"no screens granted"*. The two columns share a name and hold different things:

Table	role_code holds
mcj_legacy_users	a number — 2, 5, 6, 8, 12
mcj_legacy_access	a login name — ankush, Ashok, dipak, GKS, Priyanka

Grants in this ERP are per **person**, not per role. Joined on the login instead, the screen reads properly: ANKUSH 231 screens, ANAND 175, ANURADHA 22, ARVIND 11.

This is [defect 41]'s shape for the fourth time — a column named for one thing holding another — and it is worth stating that **the wrong join produced a plausible screen**: every row said "no screens granted", which for a shop with one admin would have looked entirely believable.

And a self-inflicted one

The comment explaining that join was written with **backticks around a column name** — inside a Go backtick raw string, which it silently terminated. Eleven compile errors from one pair of quotes. Fixed by writing the name plainly.

Coverage **119 of 193**. Integration suite green.

197. Two more screens, and the i18n guard earned its keep — LOW

Quarterly Report — quarterly_report.php

Trading by quarter, read from the vouchers rather than any quarterly table — the dump has none, and a quarter is an arrangement of dates rather than a thing to store.

Quarter	Vouchers	Income	Expense	Gap
2026 Q1	1	0.00	1,399.00	-1,399.00
2026 Q2	1,196	1,61,16,345.00	2,22,63,668.57	-61,47,323.57
2026 Q3	650	58,86,798.00	1,08,26,968.51	-49,40,170.51
2026 Q4	1	0.00	0.00	0.00
Total	1,848	2,20,03,143.00	3,30,92,036.08	-1,10,88,893.08

Income ties to the P&L exactly. **The expense does not, by ₹1,399.00** — and the screen says so rather than leaving two figures to disagree in silence. That is the journal of 9 March 2026, which falls before the financial year the P&L is bounded to: the same ₹1,399 of [defect 162] and [defect 168], now visible from a third angle. The quarters holding one voucher are labelled *"the edge of what was captured"*, because a quarter with a single entry reads as a quiet trading period and is not one.

Raw Stock Tally — raw_stock_tally.php

One of the six screens the database's own menu listed and the capture missed ([defect 175]). It groups the loose metal by purity and shows **weight only** — the Raw Stock screen values the holding; a person walking round with a sheet is weighing metal and has no use for a rupee figure while doing it.

It encodes [defect 78]'s correction. Gold and silver are totalled separately and never added:

```
Gold 12,304.524 g, silver 15,022.085 g
```

with the **995 COIN of 524.720 g on the gold side** — the single row that entry had counted as silver, worth ₹81,23,715.04 rather than ₹1,33,016.52. The summary says why the two are kept apart, so the next reader does not add them.

The i18n guard caught what I did not run

Registering the tally under a new sub-group produced a **500 on the screen**:

```
panic: i18n missing translation: MCJMenu.Group.reports__manual-stock-tally
```

`TestNavigation_EveryMenuTitleHasATranslation` catches exactly this — and it passed for the two screens before it, because **I ran it then and not after this one**. The guard was right; the process was wrong. Its own comment warns that a missing key "does not fail where it is introduced" and surfaces in whichever test renders the menu first, which is precisely how it behaved: silent at build, silent in the report tests, a panic on the page.

Two keys added. Coverage **121 of 193**, full suite green.

198. Two stock-tally screens, and the shelf two months ago — LOW

Subgroup Stock, Back Dated — `back_date_subgroup_stock.php`

A tally taken today answers *what is here*. This answers *what was here on the day we counted*, which is the question anybody reconciling a physical count a week later actually has.

There is no stored snapshot, so the shelf is rolled back from two facts: the tag existed by that date, and its **last** stock movement on or before it was not a sale. Taking the last rather than the first matters — a piece sold in April and brought back in May is on the shelf on 30 June, and reading the sale alone loses it.

As at **30 June 2026**: 11 subgroups, **2,486 tags, ₹13,57,58,680.29**. Against today's 2,575 tags and ₹14,23,61,188.20, the shelf has since taken on 89 tags and ₹66,02,507.91.

Untagged Item Stock — `untag_item_stock.php`

The same loose metal the Raw Stock Tally shows, read the other way. The tally groups by purity because that is how it is weighed; this groups by item because that is how it is asked for — somebody wants to know how much GOLD ORNAMENTS is loose, not how much 18kt.

Item	Purities	Lines	Net wt
SILVER ORNAMENT	SLV	2	12,970.041
GOLD ORNAMENTS	14, 18, 22, 9	4	11,695.314
SILVER COIN	SLV	1	2,052.044

Item	Purities	Lines	Net wt
COIN	22, 995	2	609.210
SILVER ARTICLE	ARTIC(PCS)	1	0.000

It does not total the weight column, and says why: the five items mix gold and silver, and a single figure across both is neither. That is [defect 78]'s correction built into a screen rather than written beside it.

The i18n guard, run this time

Both registrations needed a locale key and the guard named each before the screen was opened — `MCJMenu.Item.subgroup-stock-tally-back-date` for the first, none for the second. Run after every registration now, which is what [defect 197] should have taught the first time.

Coverage **123 of 193**.

199. Ratio Analysis, and a first draft that flattered the shop threefold — MEDIUM

`ratio_analysis.php`, under MIS REPORTS. Built, and the building found the mistake.

What the books can support

Ratio	Value	From	Over
Quick ratio	0.07	78,97,531.19	11,16,54,264.39
Cash to overdraft	0.01	5,00,978.19	4,92,97,479.87
Receivable to payable	0.12	73,96,553.00	6,23,56,784.52
Borrowing to stock	0.44	6,19,93,420.47	14,23,61,188.20

Three more are **refused by name** rather than printed — gross margin, stock turnover, return on stock. Every one needs closing stock inside the ledgers and [blocker B19] is exactly that it is not there. A margin computed without it is arithmetic on an absence, and the screen says so where the number would have been.

The first draft counted goods as cash

It read "money owed to the shop" as every debit balance across Customer, Dealer and Other Debtor — **₹9,54,35,175.00**, and the ratios that followed:

	First draft	Corrected
Quick ratio	0.86	0.07
Receivable to payable	1.53	0.12

Twelve times out on the second. The ₹8.8 crore difference is [defect 145]'s suppliers sitting in debit — SHREE SAROJ JEWELSS and others, where the shop has **paid ahead or is owed goods**. That is a real asset and it is not cash, and a quick ratio exists precisely to exclude things that are not.

The corrected receivable, **₹73,96,553.00**, is [defect 190]'s customer figure to the rupee.

Why it matters more than a wrong number usually does

A quick ratio of 0.86 reads as a shop that is tight but solvent. **0.07 reads as a shop living on its overdraft**, which — with ₹5,00,978.19 of cash against ₹4,92,97,479.87 owed to Union Bank — is what the books actually show. The first draft would not have looked wrong to anyone; it would have looked reassuring.

Both the excluded figure and the reason are now stated on the screen, so nobody re-adds it thinking it was an oversight.

200. Gross Profit, which refuses to state one — and three screens that now tie to the paisa — MEDIUM

`gross_profit_report.php` , under DSS REPORT. The screen a shop opens expecting a margin, and the one this data cannot give.

Gross profit is sales less cost of goods sold, and cost of goods sold is opening stock **plus** purchases **less** closing stock. [Blocker B19] is exactly that no account carries stock, so the term that turns purchases into cost is absent. The screen sets the calculation out and marks the missing line rather than printing a figure with a hole in it:

Line	Amount
Sales	2,40,66,454.00
Less returns	20,63,311.00
Net sales	2,20,03,143.00
Purchases	2,34,35,413.81
Old gold bought in	50,71,370.00
Bought in total	2,85,06,783.81
Opening stock	NOT KNOWN — blocker B19
Closing stock	14,23,61,188.20 — the stock module's, never in the books
Gross profit	not computable
What is knowable	-65,03,637.43

The last line is labelled as **not a margin**: it ignores stock at both ends, so a month of heavy buying reads as a loss and a month of selling old stock reads as a profit. It is a figure to carry to an accountant **with** the opening stock, not instead of it.

Three screens now agree to the paisa

Figure	This screen	Agrees with
Sales	2,40,66,454.00	GSTR-1 HSN summary's taxable total
Net sales	2,20,03,143.00	the P&L's income
Bought in total	2,85,06,783.81	the purchase register's goods column

The first draft was 25.62 adrift, and the reason is worth keeping

It summed every credit on a SALE-GST voucher as sales. A sale voucher also carries a **ROUNDED OFF** line — classified as an *expense* — and its 26.44 of credits went into the sales figure, leaving net sales 25.62 away from the P&L.

Small, and it would have been easy to wave through as rounding. It was not rounding: it was **rounding counted as revenue**. Each figure is now taken from its own side of the ledger rather than from the voucher type alone, and the query says why.

201. Every screen with data behind it is now built — the remaining 67 are classified, one by one

The last of the buildable screens, and the survey that says so.

Packet Wise Stock Summary — `packet_wise_stock_summary_report.php`

Diamond stock arranged by the packet it came out of. **10 packets in use of the 205 in the master**, holding **107 stones across 107 tags, ₹13,80,534.60** — the largest being DSOLMIX at ₹6,67,550.00 for a single 1.690 ct stone, and DMIX at ₹6,15,580.00 spread across 96 tags.

The 195 unused packets are not a fault: a packet master catalogues the grades a shop buys, not the stones it holds. The screen says so rather than leaving a reader to wonder where the rest went.

The 67 that remain, and why none can be built

Each was checked against the shop's own tables:

Why	Screens	Evidence
Karigar and manufacturing	34	every issue and receive table empty; no karigar master
Approval, merge, split, discount	12	<code>sales_vou_appr</code> , <code>purchase_approval</code> ×4, <code>barcode_merge</code> , <code>barcode_split</code> , <code>discount_master</code> — 0 rows each
No source table at all	10	<code>raw_opening_stock</code> , <code>studded_opening_stock</code> , <code>set_rate_sub_item</code> , <code>tds_on_job_charge</code> , <code>broken_stone</code> , <code>work_order</code>
Sale and purchase variants	6	only <code>local</code> (221) and <code>central</code> (3) bill types; no PURC-CO or PURC-SC voucher ever raised
Utility and credential	5	backup, change password, SMS/WhatsApp — blocker B15

Nothing was left unclassified. Two that looked like exceptions were checked and are not:

`ledger_setting.php` has no settings columns beyond the master fields the Ledger Master screen already shows, and `sales_voucher_estimate.php` is an entry screen for estimates the Estimates report already covers.

Why this survey is by row count, not by table name

The first version of this classification would have been by name, and it would have been wrong in one place that matters: `karigar_gold_stock` sounds like the emptiest table in the manufacturing set and holds **₹1.35 crore of gold issued to a named goldsmith** ([defect 164]). Counting rows rather than reading names is the only reason that was found.

Five areas are complete — ACCOUNTS 13/13, DSS 3/3, GST 7/7, HR 9/9, MIS 6/6.

`docs/STATUS.md` now carries this classification in place of the older estimate of "about half of what is left should not be built", which was written when the figure was 106 and is now measured rather than estimated.

202. The screen classification, verified individually — and a scare from a truncated shell pipe — LOW

[Defect 201] classified the 67 unbuilt screens into five buckets by pattern. Two of those buckets were matched by regex rather than checked one at a time, so they were checked.

The sale and purchase variants have no discriminator

Six screens — Credit Sale, Estimate Sale, Exhibition Sale, Export Sale, Repair Sale, and the CO and SC purchase vouchers — would each need a way to tell one kind of bill from another. There is none:

<code>sales_vou.sale_type</code> distinct values	<code>1</code> — SALE-GST
<code>sales_vou.bill_type</code>	<code>local</code> 221, <code>central</code> 3
Voucher types ever raised	10, and no PURC-CO or PURC-SC among them

Every captured sale is the same kind of sale. The existing sale screen already handles the local/central split through its interstate flag, which is the only distinction the data actually carries.

The utility screens

Backup and Change Password are actions, not reports. `ledger_setting.php` was checked against the table: `mcj_ledgers` carries no settings columns beyond the master fields the Ledger Master screen already shows. The two SMS screens stay shut under [blocker B15].

Migration fidelity, confirmed in passing

Voucher types match the source **exactly**, all ten, count for count:

PMNT 626 · JRNL 624 · RCPT 269 · SALE-GST 224 · CNTR 31 PURC-OG 25 · PURC-GSC 24 · PURC-GCO 19 · PURC-RET 3 · SALE-RET 3

The scare, and why it is worth writing down

A `SELECT DISTINCT v_typ` appeared to return **8** types where our data has 10, which read as the migration having invented `PURC-GC0` and `PURC-RET` — 22 vouchers from nowhere. It had not. Three queries were piped through `head -14` and the later rows of the first were cut off.

The output was truncated, not the data. That is [defect 189]'s error a third time — reading a thing through something that is not the thing — and it is the cheapest possible way to manufacture a serious-looking finding. Re-run as a `GROUP BY` with no pipe, both sides agree exactly.

203. A tag that could not be priced fell off the bill and out of the door — HIGH (fixed)

Testing [defect 192]'s sale writer on a **multi-line** bill — it had only ever been proven on a single line — found something the single-line test could not have found.

Three tags were scanned. The bill saved **two**.

```
scanned: 88106644 DIAMOND RING      18    1.460 g
          90169090 DIAMOND NOSEPIN  14    0.792 g
          77378547 SILVER LOCKET    SLV   9.568 g

saved:    2 lines · 2 stock movements · 2 tags cleared
shelf:    90169090 still showing as in stock
```

The nose pin is certified, and certification pricing is [blocker B9] — the step is not in the captured data, so the screen is right to refuse to price it. The refusal was correct, was recorded, and **was shown**: an amber panel above the bill named the tag and the reason. That part worked exactly as designed.

What did not work is what happened next. `props.CanSave` was set to `true` the moment a single line priced, and the save branch never looked at the refusals at all. So the clerk sees the amber panel, presses Save, and gets a bill for two pieces while three sit on the counter. The third goes into the customer's bag unbilled and stays on the shelf in the system: **the shop loses the goods and the stock report never notices**. The takings are short and nothing disagrees.

The sale template already carried a comment stating the opposite intent —

a tag that cannot be priced must not quietly vanish from a sale somebody is about to make

— which was true of the pricing step and false of the saving step.

The fix

Save is now blocked while any scanned tag is unpriced, both by hiding the button and by refusing the POST, since a form can be replayed:

```

props.CanSave = len(refusals) == 0
...
if len(refusals) > 0 {
    props.Errors = append(props.Errors,
        "This bill cannot be saved while a scanned tag is unpriced. "+
        "Remove those barcodes or bill them as untagged items.")
    c.render(w, r, props)
    return
}

```

The clerk clears it by taking the barcode out of the box or by billing that piece as an untagged line — the screen already supports both, and an untagged line is exactly how the shop bills the 72 of its 328 captured lines that carry no barcode.

The other three posting screens were checked, and are fine

Screen	Gate
Sale	was broken — fixed above
Credit note	blocks on problems before CanSave
Old gold	blocks on problems before CanSave
Tagging	blocks on problems before CanSave

Sale was the only one that let a refusal through, because it is the only one that prices tags **individually** — the others refuse the whole document.

Proven against the running app, then reversed

A mixed bill was posted to the running server:

```

1 good tag + 1 certified tag, action=save
http 200 (not 303)
"cannot be saved while a scanned tag is unpriced"
save button present: 0
invoices: 225 → 225
good tag still in stock: t

```

Then a clean two-tag bill, to prove the normal path still saves:

```

invoices  225 → 226      lines  330 → 332
movements 9038 → 9040    stock  2573 → 2571
sold stones 207 → 207    (both tags genuinely carry no stones)

```

Both test bills — the earlier three-tag one and this one — were then reversed in full, and every count returned to baseline:

```

invoices 224 · lines 328 · movements 9036 · sold stones 203 · in stock 2575
shelf value ₹14,23,61,188.20

```

The test, proven by breaking it

`TestSale_WillNotSaveWhileAScannedTagIsUnpriced` asserts no invoice is written and the good tag stays on the shelf. The existing `TestSale_ListsRefusedTagsRatherThanDroppingThem` only ever posted `action=price` — which is why a screen with a test named *"rather than dropping them"* was dropping them at save.

Reverting the guard fails the new test exactly as it should:

```
sale_e2e_test.go:248: the bill did not say why it refused to save
sale_e2e_test.go:260: 1 bill(s) were saved with a tag unpriced, want 0
```

Worth noting

This is the second time a fix has been proven on the simplest possible case and found wanting on the next one up. [Defect 192] was proven on a one-line sale; [defect 199]'s ratio analysis was proven against a total that happened to agree. **One line is not a multi-line test**, and a screen that handles items individually needs a case where the items disagree with each other.

204. Old gold was priced off the customer's stamp, not off what the metal tested at — HIGH (fixed)

Applying [defect 203]'s lesson to the next posting screen — test it on more than the simplest case — a three-lot old gold purchase went through cleanly and the arithmetic was right. Then the record it wrote was compared against the shop's own 25 lots, and the screen turned out to be buying gold by a method the shop has never used.

What the shop actually does

Every one of the 25 captured lots is booked against the scrap grades `OLDGOLD` or `OLDSLV` — **not one names a karat** — and carries an assessed **tunch**:

OLDGOLD	22 lots	tunch 63.1% – 92.0%
OLDSLV	2 lots	tunch 50.0% – 66.0%
(blank)	1 lot	tunch 71.0%

The tunch is the assessment of what the metal *is*, judged by the person holding it. The spread — 63% to 92% — is the whole point: **the stamp on a customer's ornament is not evidence**.

And the price is not computed at all. On all 25 lots:

rate = round(amount / net_weight, 2)	25 of 25
amount is a whole rupee	25 of 25
amount is a multiple of 100	9 of 25 (two of them exactly ₹6,00,000 and ₹10,00,000)
rate is a whole rupee	0 of 25

The price is **agreed across the counter** and typed in; the rate is worked back from it so the lot can be checked later. It is the record of a negotiation, not the output of a rate table.

What our screen did instead

It offered the clerk a dropdown of **stamped** purities — 0, 9, 14, 18, 22, 95, 995, SLV, SLV999 — looked up that grade's buy-back rate, multiplied by weight, and wrote `tunch_percent = 0`.

The scrap grades were in `mcj_purities` all along (`OLDGOLD`, `OLDSLV`, `OLDPLT`), but `purityOptions` selected `DISTINCT purity_code FROM mcj_metal_rates WHERE purchase_rate > 0` — and no rate feed carries a row for a scrap grade, so the three codes the shop actually uses were the exact three the dropdown filtered out.

What it would have cost

Re-pricing the shop's own 25 lots the way our screen priced them — 22ct buy-back for gold, SLV for silver:

The shop paid	₹50,71,370.00
Our screen would have paid	₹52,53,510.57
Overpaid	₹1,82,140.57 — 3.6%

The 3.6% average is the wrong number to look at. The damage is per lot, and it lands hardest exactly where the assessment matters most:

Date	Grade	Tunch	Net	Shop paid	We would pay	Over
19 Jul 2026	OLDSLV	50.0%	2.650	300.00	581.41	93.8%
10 Apr 2026	OLDSLV	66.0%	144.060	23,326.00	35,582.82	52.5%
8 May 2026	OLDGOLD	63.1%	1.380	12,400.00	18,875.64	52.2%
14 Apr 2026	OLDGOLD	65.0%	6.790	64,300.00	94,251.99	46.6%
31 May 2026	OLDGOLD	71.0%	0.570	5,936.00	8,059.80	35.8%

A low-tunch lot is precisely the one a customer presents as 22ct. **The screen was at its most wrong exactly when the assessment was most needed.**

The total was checked against a second source: the sum of `PURC-0G` voucher credits is **₹50,71,370.00**, agreeing with the line amounts to the paisa.

Also lost: the deduction

The writer inserted `VALUES (...,$7,$7,0,...)` — the net weight into **both** the gross and net columns. A clerk who deducted 1.000 g from a 10.000 g lot got a record saying the customer handed over 9.000 g and nothing was deducted. The schema has separate columns; the writer collapsed them. Gross and net do happen to be equal on all 25 captured lots, which is what the code's comment relied on — but the screen offers a deduction field, and anything typed into it vanished.

The fix

The screen now works the way the counter does:

Column	
Purity	the scrap grades — <code>OLDGOLD</code> , <code>OLDSLIV</code> , <code>OLDPLT</code>
Gross wt / Less / Net wt	both weights now recorded as themselves
Tunch %	required, $0 < t \leq 100$ — the assessment behind the price
Amount agreed	typed, the negotiated price
Rate	derived, <code>amount ÷ net</code> , shown back as the record

A **guide** sits under the amount — the lot valued at its tunch against the day's pure rate (995 for gold, `SLV` for silver) — as a sanity check on a figure typed in a hurry. It never replaces the typed amount. Platinum has no rate feed, so a platinum lot simply gets no guide.

Proven by reproducing a real lot

The shop's lot **OGP/2627/00005** was re-entered through the rebuilt screen:

	bill_no	bought_on	purity	gross	net	tunch	rate	amount
shop's	OGP/.../00005	2026-04-14	OLDGOLD	6.790	6.790	65.000	9469.81	64300.00
ours	OGP/.../00026	2026-04-14	OLDGOLD	6.790	6.790	65.000	9469.81	64300.00

Identical in every column. The old screen would have paid **₹94,251.99** for that same metal. The test lot was then reversed — 25 lots, 25 vouchers, ₹50,71,370.00.

The tests

`TestOldGold_PricesAtTheBuyBackRate` and `TestOldGold_RefusesAGradeItDoesNotBuyBack` were **testing the defect**: they asserted that a lot stamped 22 gets the 22ct buy-back rate, which is the behaviour that overpays. They are replaced by tests of what the shop does:

- `TestOldGold_TakesTheAgreedAmountAndWorksTheRateBack`
- `TestOldGold_DoesNotOfferStampedPurities` — asserts no `value="22"` on the page
- `TestOldGold_RequiresTheTunch` — rejects blank, 0, negative and >100
- `TestOldGold_RequiresTheAmount`
- `TestOldGold_ShowsAGuideWithoutOverridingTheAgreedPrice`
- `TestOldGold_RecordsGrossAndNetSeparately` — catches the `$7,$7` collapse
- `TestOldGold_WorksTheRateBackFromNetNotGross`

Worth noting

[Defect 203] was found by testing a screen on more than one line. This one was found by going one step further: checking what the screen *wrote* against what the shop's own records look like. The arithmetic was correct throughout — every figure the old screen produced was internally consistent. **It was answering the wrong question, and no amount of testing its own logic would have said so.**

A passing test suite is not evidence that a screen does the shop's job. Two of the tests here asserted the defect and passed for it.

205. The next gold-coin purchase would have failed outright — HIGH (fixed)

Same method as [defect 204], applied to the purchase side: compare what the screen writes against what the shop's own records look like.

The shop runs **two purchase bill series**, and its 43 captured bills split between them without a single exception:

GDP/...	24 bills	PURC-GSC	₹1,80,38,003.00	stock purchases
GCP/...	19 bills	PURC-GCO	₹59,18,795.00	bought against an order

Both series number from 1, and on all 43 the number in the bill is the voucher's own — which `nextNumber` also allocates **per type per year**.

`writePurchaseInvoice` hard-coded the prefix:

```
// GDP/2627/00025 -- the format on the shop's own 43 captured bills.  
billNo := fmt.Sprintf("GDP/%02d%02d/%05d", ..., v.Number)
```

So every gold-coin purchase was numbered into the **GDP** series. GDP had reached 24; GCO stood at 19. The next gold-coin purchase would take voucher number **20** and build `GDP/2627/00020` — a bill number already held by a real **₹9,20,000.00** purchase from ANAND SILVER on 1 July 2026.

Posted against the running app, that is exactly what happened:

```
POST /mcj/trade/purchase-order → http 500  
purchase invoices 43 → 43 · PURC-GCO vouchers 19 → 19
```

The shop could not have recorded its next gold-coin purchase at all. The unique index added for [defect 195] is what turned a silent overwrite into a loud failure, which is the outcome it was there for.

Two more columns the writer never filled

`due_on` was never written, though **all 43** captured bills carry one, so every new purchase would have been invisible to any ageing report. It is now set to the bill date, which matches 32 of the 43; the remaining 11 scatter from five days before the bill to one after, and nothing in the captured data says whether the shop wants a credit period entered per bill.

`is_b2b` stays `false`. It is true on 7 captured bills — D.N. JEWELS, ZEST CORPORATION, ZM LIFESTYLE, ANJALI LABTECH, S K CHAINS, ANUPAM TRADERS — every one a registered dealer. There is nothing to derive it from: **no ledger in the database carries a GSTIN**, 0 of 785 ([defect 150]).

Fixed and proven

GCP/2627/00020	2026-08-23	due 2026-08-23	₹50,000.00
----------------	------------	----------------	------------

The correct series, continuing from GCP/2627/00019, with a due date. Reversed after the test — 43 bills, 24 GDP and 19 GCP, ₹2,39,56,798.00.

Checked in passing, and it holds without exception: **all 24 GDP bills carry tax, all 19 GCP bills carry none**. The tax-free flag on Purchase Against an Order is right.

`TestTrade_EachPurchaseKindKeepsItsOwnBillSeries` posts one of each and asserts the prefixes and the due date.

206. A save that failed told the wrapper it had succeeded — HIGH (fixed)

Chasing [defect 205] surfaced something worse than the bug being chased.

The failing purchase returned this, as the entire page:

```
ERROR: current transaction is aborted, commands ignored until end of
transaction block (SQLSTATE 25P02)
```

That is not the reason. The reason — a duplicate bill number — had been found, wrapped in a sentence a clerk could read, and then thrown away.

How the reason was lost

1. `writePurchaseInvoice` hit the unique index. The Postgres transaction is now aborted.
2. `save` returned the error; `post` put "This credit note could not be saved: ..." into `props.Errors`.
3. `render` then called `billableParties(ctx)` to fill the customer dropdown — **a query on the aborted transaction**.
4. That query failed with `25P02`, and `render` answered it with `http.Error(w, err.Error(), 500)` — replacing the whole page, and the real reason with it.

The explanatory path was unreachable in precisely the case it existed for.

The worse half

`atomic` commits on any `2xx` and rolls back from `400` up. The save-failure path rendered the page at **200**.

So a screen whose save had failed was telling the wrapper the request had succeeded, and **whatever the failed save had already written was committed**.

Here the damage was contained by luck: the failure had aborted the Postgres transaction, and Postgres turns `COMMIT` on an aborted transaction into `ROLLBACK`. A failure that does *not* abort the transaction — a lookup returning no rows, an error raised between two writes — has no such protection. It would commit a purchase voucher with no bill behind it, or a sale voucher with no invoice: exactly the split [blocker B26] was closed to prevent.

All four posting families shared it — sale, old gold, trade, and the general voucher form.

The fix

A new `failedSaveStatus` (500) is returned by every save-failure render, so `atomic` rolls back. On that path the lookups that dress the page are allowed to come back empty rather than replace the page and the reason:


```

parties, err := billableParties(ctx)
if err != nil && !tolerant {           // tolerant on the failure path only
    http.Error(w, err.Error(), http.StatusInternalServerError)
    return
}

```

Two message defects were fixed alongside it. `TradeController` serves four screens and told all of them they were a credit note, and the raw SQLSTATE went straight to the clerk. Both now read:

```

before  ERROR: current transaction is aborted ... (SQLSTATE 25P02)
after   This purchase against an order could not be saved: that number
        is already taken for this year. Try saving again.

```

`humanise` was already doing this job on the voucher form; it now serves all four, with the sentence prefix moved to the caller so each screen names itself.

Proven

Against the running app, with the next bill number deliberately occupied:

```
http 500 · reason on the page · PURC-GCO vouchers 19 → 19
```

Then with the blocker removed, the same request saved normally at 303.

`TestTrade_AFailedSaveDoesNotReportSuccess` pins the status and asserts the reason is not replaced by a transaction message; `TestTrade_AFailureNamesTheScreenItHappenedOn` pins the wording and that no raw SQLSTATE reaches the clerk. Reverting either fix fails them.

`TestAtomic_ClassifiesTheHandlersAnswer` already pinned the other half of the chain — 2xx commits, 400 and up rolls back. What was missing was anything asserting which side of that line a failed save lands on.

Worth noting

The end-to-end rollback cannot be asserted from the integration harness: it runs the request inside the test's own transaction, so a poisoning failure takes the test's own queries down with it. That is why the "nothing was committed" half was proven against the running app instead, and the test pins the status. A test that cannot observe the thing it claims to prove is [defect 189]'s error in another costume — and the way out was to check the claim somewhere it is observable, not to weaken the claim until the harness agreed with it.

207. The salesperson master was loaded with its two columns the wrong way round — MEDIUM (fixed)

Continuing the column-by-column comparison against the shop's own records, this time on the 224 captured sales bills.

`salesperson` is filled on **all 224**. Following it into the master found this:

```

mcj_salespeople.name      S039
mcj_salespeople.short_code NARENDER SEVADA

```

All 24 rows the same way round: `name` matches `^[0-9]+$`, and `short_code` holds a person. So the codelist screen headed **Name** listed staff codes, and the one headed **Short code** listed people.

What makes this worth writing down is that it was known. Both the migration and the codelist entry carry the same warning:

The legacy columns are SWAPPED relative to their names: `s_name` holds the code (S004) and `s_code` holds the person's name, in all 24 rows. The seed reads them the right way round.

The seed did not. `tools/legacy/load_codelists.sh` line 44 read:

```
mcj_salespeople|salesperson_master|s_name|s_code
```

— straight across, preserving the legacy swap. **The comment describing the correction and the code performing it disagreed, and the comment was believed.**

Fixed in the loader, and corrected in place by migration `mcj-1787409000`, which is conditional on the name still looking like a code so it is idempotent and harmless against a fresh load. All 24 person names are distinct and none is blank, so the swap is safe. One row reads `DELETE (S061)` — a tombstone left in the legacy master.

208. A bill could not say who made the sale — MEDIUM (fixed)

Every one of the 224 captured bills names a salesperson. **The sale screen had no such field, and `writeSaleDetail` never wrote one**, so sales attribution stopped dead at migration.

131 of the 224 name a real person; the other 93 carry `0`, which is no salesperson at all — there is no row with that id in the master. So it is optional, and stays optional: making it required would refuse a sale the shop has always allowed.

The column held something no new bill could use

`mcj_sales_invoices.salesperson` holds the **legacy row id** as text — `"3"`, `"7"`, `"45"` — which resolves only by joining `mcj_salespeople.legacy_id`. A salesperson hired tomorrow has no legacy id, so a new bill had nothing valid to put in that column at all.

A real reference was added beside it rather than rewriting the shop's own records:

```
ALTER TABLE mcj_sales_invoices ADD COLUMN salesperson_id uuid
REFERENCES mcj_salespeople (id) ON DELETE SET NULL;
```

Backfilled from `legacy_id`: **131 linked, 93 null, and 0 named-but-unlinked** — every bill that named somebody now resolves. The captured text column is left exactly as it was. Rewriting 224 historical rows to tidy a vocabulary is not a trade worth making; the same reasoning as [defect 204], reached the other way.

Proven

A bill posted through the rebuilt screen:

bill_no	sold_by	code	legacy_col
GST/2627/00225	SHREE CHAND SHARMA	S004	(blank)

The dropdown lists people **by name** — which it could only do after [defect 207] was fixed. Reversed after the test; 224 bills, 131 attributed.

TestSale_RecordsWhoMadeTheSale , TestSale_SalespersonIsOptional and
TestSale_RefusesAnUnknownSalesperson cover it.

Two columns still not written, and why

party_mobile is filled on 196 of the 224. Nothing in the sale screen collects it and the ledgers cannot supply it — **0 of 785 carry a phone number** ([defect 150]).

is_b2b is true on 20 of the 224. Same obstacle as on the purchase side ([defect 205]): no ledger carries a GSTIN, so there is nothing to derive it from.

certification is non-zero on **41 of the 224** — 18% of the shop's bills. Those cannot be raised at all until [blocker B9] is answered, which is the same wall [defect 203] ran into from the other direction.

209. The migration chain could not be rolled back — MEDIUM (fixed)

Found by accident while checking that [defect 208]'s migration reversed cleanly. It did. The chain underneath it did not:

```
Error: failed to rollback migrations:  
ERROR: value too long for type character varying(10) (SQLSTATE 22001)  
handling mcj-1787403000.sql
```

mcj-1787403000 widens mcj_ledger_groups.legacy_level from varchar(10) to varchar(24), because the third level of the group hierarchy is called sub-subgroup — twelve characters — and it sits on **34 of the 96 groups**.

Its Down narrowed the column straight back:

```
ALTER TABLE mcj_ledger_groups ALTER COLUMN legacy_level TYPE varchar(10);
```

which **fails every time the Up has actually been useful**. The migration exists precisely to hold values longer than 10 characters, so undoing it always trips over them.

Why that is worse than a failed command

A rollback runs migration by migration. This one failed *part way through*: five later migrations had already been undone when it stopped, leaving the schema half-rolled-back. Only running up again recovered it — which it did, cleanly, with no data lost (checked against every baseline, including the ₹14,23,61,188.20 shelf).

Discovering this during an actual emergency rollback is the bad version.

The fix

The Down now narrows the column only when it can do so honestly, and otherwise leaves it wide — which is harmless to the code that ran against the narrow one:

```
DO $$
BEGIN
  IF NOT EXISTS (
    SELECT 1 FROM mcj_ledger_groups WHERE length(legacy_level) > 10
  ) THEN
    ALTER TABLE mcj_ledger_groups ALTER COLUMN legacy_level TYPE varchar(10);
  END IF;
END $$;
```

Every Down section across all migrations was scanned for the same shape — a narrowing `ALTER COLUMN ... TYPE`. This was the only one.

Proven on a scratch database, not this one

A full `migrate down` against the live database would drop the migrated tables and everything in them, so the fix was verified on a throwaway copy: 121 migrations up, a `sub-subgroup` row seeded to trigger the overflow, then down.

The rollback now runs **past every mcj migration** — 0 mcj tables left — and stops much later, in an IOTA SDK core migration:

```
ERROR: role "bichat_agent_role" cannot be dropped because some objects
depend on it (SQLSTATE 2BP01) handling changes-1769853356.sql
```

That one is the SDK's, not ours. Recorded here because a full rollback still cannot complete, and the reason is now somebody else's to fix.

Worth noting

Three defects this round came from the same move: read what the shop's own rows contain, then check the code actually does that. [Defect 207] is the sharpest case — **a comment stated the correction, the code never made it, and the comment was trusted for both**. A comment is a claim about the code, not evidence about it.

210. Every bill line sold exactly one piece, whatever the tag said — MEDIUM (fixed)

The header of a sales bill was checked in [defect 208]. This is its 328 lines.

Four columns disagreed with the shop's own records, and three of them were being written as constants:

Column	Captured on 328 lines	What we wrote
pieces	221 sell one, 78 sell more (one sells 40), 29 sell none	hard-coded 1
gross_weight	filled on 316; differs from net on 141	0

Column	Captured on 328 lines	What we wrote
<code>fine_weight</code>	filled on 316 of 328	<code>0</code>
<code>wastage_pct</code>	filled on 14	<code>0</code> — correctly, see below

pieces

A tag is one label, not one piece. A pair of tops is one barcode and two pieces. On the shop's shelf, **766 of the 2,575 tags carry more than one** — 30% of everything on display — and every one of them would have been billed as a single piece.

Nothing about the money is wrong: the tag's price is the tag's price. What is wrong is the count, so any quantity a report states — pieces sold, pieces on hand, pieces per bill — is understated on nearly a third of the stock.

gross and fine weight

Gross is the metal plus whatever is set into it, and it differs from the net on 141 of the 328 captured lines. A line carrying only the net loses the weight the customer actually handled.

Fine weight is the pure metal in the line, and the rule turned out to be exact once the outliers were separated out:

22ct lines	92.00% of net	(21 at 92.00, 14 at 92.01, 11 at 91.99 — 3dp rounding)
18ct	75.00%	
14ct	58.33%	
SLV	92.50%	

— which is `mcj_purities.fineness` to the digit. The lines that do not fit are the 14 carrying a **wastage** percentage, which this screen has no field for. It does not invent one: `wastage_pct` stays 0, and that is the one constant among the four that is right.

An earlier reading put the ratio at ~93% for 22ct and looked like a systematic 1% drift. It was not systematic — it was **fourteen wastage lines pulling an average**. Grouping by the ratio rather than averaging it showed the rule is exact.

And the untagged rows were already asking

72 of the 328 captured lines carry no barcode, and the screen's untagged row already collects `u_pieces_` and `u_gross_` from the clerk. Both were being dropped between the row and the bill: `untaggedLine` carried them, and the priced line copied only the description, purity and net weight. **The screen asked for two numbers and then threw them away.**

Proven

A two-line bill of multi-piece, stone-set tags, posted against the running app:

barcode	description	purity	pieces	gross	net	fine
32348536	DIAMOND PENDANT SET	18	3	7.440	6.104	4.578
33969968	SILVER SET	SLV	3	20.060	18.786	17.377

Exact on all six figures against the tags. Reversed after the test — 224 bills, 328 lines, 9,036 movements, shelf ₹14,23,61,188.20.

211. A piece left the shelf with no record of who took it — LOW (fixed)

The same comparison, one table over. `mcj_stock_movements.done_by` is filled on **322 of the shop's 324 captured sale movements** — BAPI, ADMIN, RAJENDRA KICHER, PRIYANKA — and our writer left it blank on every one.

A stock trail that cannot say who moved a piece is most of the way to no trail at all. It is now the logged-in user, which is a **different question** from [defect 208]'s salesperson: one is who operated the till, the other is who made the sale, and the shop's own data records both, in different tables.

It fails soft — an empty string rather than an error — because a bill must not be refused over an audit column.

Checked and deliberately left as it is

Our sale movement also writes `net_weight` and `purity`, which the shop's own 324 sale movements leave blank (0 of 324 carry either). That is ours being fuller than the original rather than different from it, and the weight is the same piece's weight either way, so it stays.

The tests

`TestSale_RecordsThePieceCountGrossWeightAndFineWeight`,

`TestSale_UntaggedLineKeepsWhatTheClerkTyped` and `TestSale_TheStockMovementNamesWhoDidIt`.

Breaking each fix in turn fails them with exactly the figures above — 1 piece where 2 was wanted, 0.000 fine where 7.360 was wanted, and a movement naming nobody.

Worth noting

Six defects have now come out of one habit: **open the shop's own rows and count what is in each column, then check the code puts that there.** Not one of these would fail a test written against the code's own logic — the arithmetic was right every time. `pieces = 1` is a perfectly correct statement about a bill for one piece; it just was not a statement about this shop.

212. A tag was priced on one day and dated on another — MEDIUM (fixed)

The tagging screen is where every stock item comes from, so it was compared against the 2,816 tags the shop already has.

The screen asks **"Tagged on"**. That date is parsed, used to pick the metal rate — deliberately, because a tag's stored price is meant to age ([defect 35]) — and then discarded. The row was written with `created_date = now()`.

So a piece tagged as at 2 June was **priced at 2 June's rate and stamped with whatever day it was entered.** The tag disagrees with itself, and the back-dated stock tally built in [defect 198] reads the stamp, so the piece was priced into one month and counted into another.

tagged on	2026-06-02	→	22ct rate 14,775.00	(the June rate, correctly)
stamped	2026-08-23			(today, incorrectly)

Now both are the form's date:

barcode	created_date	last_received_date	retail_code	pieces	rate
ZZTEST01	2026-06-02	2026-06-02	R001	2	14775.00

`last_received_date` was never written either, though it is filled on **2,810 of 2,816** tags. For a tag being created it is the same day — it has just been received — and it equals `created_date` on 2,688 of the 2,816, differing only where a piece went out and came back. `retail_code` was blank where the shop fills `R001` on 2,808 of 2,816.

`TestTag_IsDatedTheDayItWasTaggedNotToday` and `TestTag_CarriesTheRetailCode`. Reverting the fix fails the first with, precisely, *"the tag is dated 2026-08-23, want the 2026-04-10 on the form"*.

What a new tag still does not carry

Recorded rather than quietly left. Fill rates are across all 2,816 captured tags:

Column	Filled	What it is
<code>voucher_no</code>	2,797	the stock-inward voucher the piece arrived on — a new tag has no link back to how it entered
<code>lot_no</code>	2,059	free text, and untidy: <code>UTENSILS</code> and <code>UTENSIL</code> both appear
<code>category_id</code>	1,629	the design category
<code>has_mrp</code>	978	
<code>certificate_no</code>	678	tied to [blocker B9]
<code>remarks1/2</code>	452	
<code>hallmarked</code>	432	see [defect 50] — 739 gold pieces carry no hallmark record
<code>ref_no</code>	260	
<code>design_code</code>	109	
<code>size</code>	99	
<code>wastage_percent</code>	26	

Each needs a field on the screen, and `voucher_no` needs a stock-inward concept that does not exist yet. None of them is money.

213. A verified non-defect: `making_sales` is not a second price — LOW

Worth writing down because the column names invite exactly the wrong conclusion, and I nearly drew it.

mcj_stock_items carries both making_value and making_sales , and they disagree on **987 of the 2,575 tags on the shelf** — 38%. The sale prices from making_value . Read as *cost* against *selling* making, that is a screen billing customers at cost on more than a third of the shelf.

It is not. Grouping by making_type settles it:

making_type	tags	sales<>value
val	1197	0
per_gm_nw	599	597
per_pcs	393	4
per_gld_val	384	384

They differ **only where the making is a rate rather than an amount**. making_value is the rate per unit; making_sales is that rate already multiplied out:

PLATINUM LOCKET	1.976 g	value 2,000.00/g	→ sales 3,952.00
GOLD CHAIN	11.960 g	value 720.00/g	→ sales 8,611.20
SILVER DIPAK	30.110 g	value 15.00/g	→ sales 451.65

Exact in every case, which is why they coincide on val tags — there the rate is the amount.

Our pricing engine takes the rate and multiplies. Confirmed against the running app rather than argued: barcode 43221199 billed making of **8,611.20**, to the paise the same as the tag's own making_sales .

Two columns disagreeing on 38% of the shelf is exactly the shape of a serious money defect, and the difference between finding one and inventing one was grouping by a third column before believing the first two.

214. 27% of the shop's vouchers could not be edited at all — HIGH (fixed)

The voucher screens carry the most traffic in the system — 626 payments, 624 journals, 269 receipts, 31 contras — so they got the same treatment: read what the captured rows hold, then check the code.

bank_cleared is true on **497 vouchers**, and bank_cleared_on carries a date on **466 of them**. The voucher form offered a "**Cleared by the bank**" checkbox that set the flag. Nothing on that form has ever written the date.

Clearing is a **pair** of columns, and a migration says so out loud:

```
CONSTRAINT mcj_vouchers_cleared_date_agrees CHECK (bank_cleared OR bank_cleared_on IS NULL)
```

Uncleared items have no clearing date, and a dated item is a cleared one. Enforced rather than assumed, because a reconciliation that disagrees with itself about which items are outstanding is worse than no reconciliation.

A checkbox can only ever set one half of that pair. Open any reconciled voucher, untick the box, save:


```
POST /mcj/vouchers/payment/fd92bc70-... → http 500
ERROR: new row for relation "mcj_vouchers" violates check constraint
"mcj_vouchers_cleared_date_agrees"
```

497 of 1,848 vouchers — 27% — could not be corrected through the screen built to correct them. Not a typo, not a date, not a narration.

The fix is about ownership, not about the constraint

There is already a reconcile screen, and it does this properly: it writes the flag and the date together. The voucher form was a second, worse path to the same fact.

So the repository no longer touches either column on an edit, and the checkbox is gone. One screen owns one fact. A newly entered payment is no longer born reconciled either — that happens when the bank says so, on the screen for it.

The old comment was right about the danger and wrong about the cure

Both the props field and the test that guarded it carried the same reasoning:

a screen that hid the box would silently clear the flag the next time somebody edited one of those vouchers

That was a real risk, and it came from the UPDATE setting `bank_cleared` from whatever the form happened to post. Removing the box alone would indeed have cleared the flag on every edit. **Removing it from the UPDATE removes the danger at its source**, and then hiding the box is safe — which is what makes this a fix rather than a trade.

`TestVoucher_ClearingCheckboxIsOnEveryKind` was asserting the defect, exactly as two tests did in [defect 204]. It is replaced by `TestVoucher_DoesNotOfferBankClearing` and `TestVoucher_EditingAReconciledVoucherLeavesTheReconciliationAlone`, which reconciles a voucher the way the reconcile screen does, edits it, and checks the narration changed while the flag and the 2026-04-12 date did not. Restoring the old UPDATE fails it.

Proven against the shop's own data

Voucher **PMNT 5**, cleared on 2026-04-03, edited through the form:

```
before  true / 2026-04-03 / Being Charges PORC Customer (UBINH26092190603)
after   true / 2026-04-03 / edit test          ← http 303
        number still 5, lines still 4
```

Narration restored afterwards; 1,848 vouchers, 497 cleared, 466 dated.

215. A failure named a voucher that does not exist — LOW (fixed)

Visible in [defect 214]'s error message, and worth its own number because it would have misdirected anybody reading it:

```
updating voucher 0/PMNT/2026-2027: ERROR: ...
```

There is no voucher 0. `Key()` is the legacy natural key — number, type, year — and a voucher arriving from an **edit** form carries its id but no number, because the form has no reason to post one back. So every failure on an edit named `0/TYPE/YEAR`.

The number itself was never at risk: the UPDATE does not set it, and PMNT 5 came back as PMNT 5 with all four lines. But an error that misidentifies the record it is about is worse than a vague one — it sends the reader to the wrong voucher, or to look for one that was never there.

Save failures now use `Ref()`, which falls back to the type, year and id when there is no number to give.

Checked while in there, and left alone

`mcj_voucher_lines` has its own `cheque_number` and `cheque_date` columns. They are empty on **all 3,935 captured lines** across all four kinds — the cheque lives on the voucher header, where the shop fills it on 34 payments. Nothing writes them and nothing should.

216. A returned tag's movement named no document and nobody — MEDIUM (fixed)

Same comparison on the return path.

Every one of the shop's **9,036** stock movements references a document number:

SALE-GST	324	GST/2425/00119 ... GST/2627/00220
SALE-RET	18	SR/2425/00017 ... SR/2627/00002
PURC-IN	1	GDP/2627/00023

Not one carries anything else. Our credit note wrote:

```
fmt.Sprintf("%s %d", kind.Type, v.Number) → "SALE-RET 4"
```

which would have been **the only reference in the whole register naming no document**. A stock register is read by looking a document up; a row that names none is a dead end. It also left `done_by` blank, where 17 of the 18 captured returns say who handled it — [defect 211] on the other screen, in the same place.

Both fixed. A return now writes `SR/2627/00004` and the logged-in user, proven against the running app:

barcode	movement	reference	done_by
32348536	SALE-RET	SR/2627/00004	Admin User

The number is the voucher's, deliberately

The shop's SR series does **not** track its SALE-RET voucher numbers. There are three SALE-RET vouchers in 2026-27 and two SR references, and only the first pair agree:

voucher 1	2026-04-15	SR/2627/00001	2026-04-15	✓
voucher 2	2026-06-16	SR/2627/00002	2026-05-06	x
voucher 3	2026-07-25	—		

Continuing the SR sequence independently would mean maintaining a second counter with nothing behind it. Tying the reference to the voucher at least names a document that exists, and it is how every other number in this system is built — GST, OGP, GDP, GCP all take the voucher's own number.

217. Sold stones recorded half their weight — LOW (fixed)

`mcj_sold_stones.actual_carats` is filled on **182 of the shop's 203** rows and equals `carats` on **every one of them**. Ours wrote zero.

The shelf carries no separate figure to draw it from — `mcj_stock_stones` has no `actual_carats` column — so it is now the same weight rather than a zero, which is exactly what the 182 captured rows say it should be.

Proven on a stone-set tag: 5.670 and 1.010 carats, both columns agreeing.

Checked and deliberately not "fixed"

A returned sale keeps its sold-stone rows. The obvious-looking bug is that returning a tag leaves `mcj_sold_stones` claiming its stones were sold. The shop's own data settles it: of its three returned invoices, `GST/2627/00004` still carries **3** sold-stone rows and `GST/2627/00099` carries **1**. The rows survive a return. Ours behave the same way, so nothing was changed.

`mcj_sales_invoices.is_returned` **is never set by the credit note**. It is true on 3 of the 224 captured bills. It stays unset here because **a credit note does not know which bill it reverses** — the screen takes barcodes, and its own note says so: *"nothing links a tag back to the bill that sold it."* The shop had a `sale_bill_link` table for this, and [defect 66] found 83 of its 135 links pointing at a number that is also a real bill for a different sale. Marking the wrong invoice returned would be worse than marking none.

Something else the captured data shows and we do not do: the returned tags carry a suffix — `57808299R1`, `10864120R1`, `95831681R1`. The shop re-barcodes a piece that comes back. Whether a returned piece should keep its number or take a new one is a question for the shop, not something to infer; recorded rather than guessed.

And a third: the two tags on the shop's returned invoices are still `in_stock = false`. A sale marked returned did not put those pieces back on the shelf. That may be a legacy inconsistency or it may mean those returns were refunds without goods — it is not enough to act on, and it is listed here so it is not rediscovered as new.

218. The payroll screen showed a Days column of zeros, and totalled them — LOW (fixed)

A different sweep this round: instead of auditing what the screens write, look for columns that are **empty in every row**, then check which reports read them. 39 such columns exist across the `mcj` tables.

`mcj_salary_sheets.days_worked` is zero on all **246** rows, and the Staff Salary Sheet displayed it as a **numeric** column — so it also totalled, and the footer read **0 days** across the whole payroll. Beside a Paid column of real money, that reads as people being paid for no work.

Nothing in this system writes a salary sheet — only the legacy loader — so the column could never have been anything but zero.

The loader was right; the report was wrong

The legacy table has **four** day columns, and the loader's own comment already had this settled:

no_of_days	0 on all 246
extra_days	0 on all 246
total_extra_days	0 on all 246
total_days	136 rows of 31, 95 of 30, 15 of 28

`total_days` is the **length of the month**, not days worked. Using it would assert everyone worked every day including the four months nobody was paid, so the loader writes zero and says why ([defect 153]). That decision is right.

What was wrong was showing the result. **An empty column is an honest answer; an empty column that totals is a claim.** The column is gone and the report's comment now carries the reasoning.

tools/check_empty_columns.py

The sweep is now re-runnable, because the answer changes as data is loaded:

```
python3 tools/check_empty_columns.py --read
```

29 of the 39 empty columns are named somewhere in the report package. **Most of those are false hits and the tool says so in its own output**, because it greps for the column NAME across all reports and column names repeat across tables. Two of the first three leads were exactly that:

Lead	Reality
<code>orders.go</code> subtracts an always-zero <code>metal_rate</code> from today's rate, overstating exposure	It reads <code>mcj_customer_order_lines.metal_rate</code> — filled on 361 of 366. The empty one is <code>mcj_designs.metal_rate</code> , which it never touches
<code>makingbands.go</code> counts <code>DISTINCT making_rate</code> over an always-zero column	It reads <code>mcj_item_making_bands.making_rate</code> . The empty one is <code>mcj_designs.making_rate</code>

Both would have been written up as money defects on the strength of a grep. Confirming the **table** before believing the **name** is the whole of the difference, and the tool now prints that warning on every line.

Three more, checked and left alone

The tag-edit audit trail is frozen. `mcj_barcode_edits` is written only by the legacy loader; no screen in the new system adds a row. Of its eight change flags only three were ever true — `item_changed` 31, `units_changed` 119, `making_changed` 7 — so the report's "Weight or stones" filter can never match anything. Left as it is, deliberately: an **empty result set is truthful** ("nothing changed a weight"), where a zero total is not. That is the same distinction that decided the Days column, applied the other way.

No screen here edits a tag's weight, item or making, so there is nothing yet to audit. The three screens that touch `mcj_stock_items` change `category_id` (Categorise) and `in_stock` (sale, return) — and the latter two already record themselves in `mcj_stock_movements`, which is the right place. The legacy kept no trail for category changes either: `barcode_category_tagging` is a two-row barcode-to-category mapping, not an audit.

The shop is holding 233 pieces and 14.430 carats of customers' own stones.

`mcj_customer_goods.returned` is false on all 7 rows, which looked like the same shape as [defect 164]. It is not: every lot is against an **undelivered** order and was received the same day the order was taken.

RASHMI SARAF	C0/2627/00072	1 pc	2.750 ct	28 Jun
POOJA PRASAD	C0/2627/00094	99 pcs	1.300 ct	22 Jul
POOJA PRASAD	C0/2627/00094	104 pcs	2.850 ct	22 Jul
TAHA.M. TAMBAWALA	C0/2627/00101	3 pcs	6.250 ct	26 Jul
PRABHA BHANSALI	C0/2627/00111	15 pcs	0.930 ct	18 Aug
PRABHA BHANSALI	C0/2627/00112	3 pcs	0.170 ct	18 Aug
P. K. MUNDRA	C0/2627/00113	8 pcs	0.180 ct	19 Aug

Not returned because the work is not finished — the data is consistent and nothing was changed. Recorded because it is **five customers' property sitting in the shop**, the oldest for 56 days, and the last lot came in on 19 August, the day the system went down.

219. Two of the eight migration gates had been waiting on work that was already done — MEDIUM (fixed)

`cmd/reconcile` runs the migration's acceptance gates, and its own header says why they matter:

Checks that depend on work not yet done report PENDING rather than passing vacuously — a gate that cannot fail is not a gate.

Two of the eight were literal stubs:

3. Trial balance ties to zero PENDING needs the finance domain transform (Tier B)
8. New reports match legacy output PENDING needs the first migrated reports (Tier 0 #9, Tier B)

Both dependencies exist. The finance domain is 785 ledgers, 1,848 vouchers and 5,229 lines; the reports are 126 screens. **The gates were not waiting for anything — they had simply stopped being looked at.** A gate that never runs is no more a gate than one that cannot fail.

Both now run, and the tool reports **4 passing, 4 failing, 0 pending**.

Gate 3 — the trial balance

It computes the openings and the transactions separately:

openings out by -15,32,28,603.49 (debits 10,80,37,319.15, credits 26,12,65,922.64)
transactions out by 0.01

The opening figure matches [blocker B19] to the paisa, which is the point — a gate is worth having when it reproduces the known answer without being told it. The 0.01 is [blocker B4]'s single unbalanced voucher, and the gate deliberately does **not** fail on it: gate 4 already owns that finding, and counting it twice would make one problem look like two.

Gate 8 — parity against the shop's own printout

This one needed something to compare against, and the source system has been 404 since 21 August 2026. Of the 354 captured screens, **254 are password walls** — the capture ran unauthenticated, so 72% of it is a login page. Of the 100 with content, exactly one is a printed document carrying figures:

```
print_sales_vocher_register_gst_php_billno_sale_type.html
GST/2627/00215 · 17/08/2026 · NANDA KUMAR GUPTA
Taxable 92,400 · SGST 693.00 · CGST 693.00 · Gross Total 93,786.00
```

The gate reads those figures **out of the capture** rather than from numbers typed into the source file, and holds the database to them. Every one agrees:

taxable	printed	92,400.00	migrated	92,400.00
tax	printed	1,386.00	migrated	1,386.00
total	printed	93,786.00	migrated	93,786.00

Party, date and place of supply match too, and the voucher debits NANDA KUMAR GUPTA the full 93,786.00 against SALE 92,400.00, CGST 693.00 and SGST 693.00.

Then it fails, correctly. The printout has one line — DIAMOND Wt 0.880 Ct, HSN 710239, net 0.176 — and the migrated invoice has none. That is [defect 155]: **39 of the 224 bills carry no lines at all, ₹91,71,074.79** — 37% of the sales value. The gate found it from the outside, without being told where to look.

That is worth stating plainly: the money is right and the detail is missing, and a check that reported only the first half would have been the more flattering half of one answer.

Proven

The bill's total was moved by one rupee and the gate said so precisely —

```
GST/2627/00215 total: printed 93786.00, migrated 93787.00
```

— then returned to the line-count finding when it was put back.

What this does not do

One invoice is not report parity. It is the only external evidence that survives of what the shop's own screens printed, and it exercises the header arithmetic of one bill. The other 123 reports have nothing to be checked against and will not have, unless more of the legacy output turns up. Recorded so that a green line on gate 8 is never read as "the reports match the old system".

220. Defect 155 is recoverable in full — the missing detail was never missing — HIGH (fixed)

[Defect 219]'s new parity gate found that `GST/2627/00215` ties to the paisa and carries no lines. That is [defect 155]: **39 of the 224 bills with no detail at all, ₹91,71,074.79, 37% of the sales value.** It has stood open since it was found, and the one recovery route tried before — `sale_bill_link` — was correctly rejected.

Following the printed invoice into the dump found the detail immediately. `sales_vou_detail` has a row for that bill:

```
billno GST/2627/00215 · item_id 33 · ct 0.880 · gw 0.176 · rate 105000.00 · amount 92400.00 · uom Ct
```

$0.880 \times 105,000 = 92,400$ exactly, and the printout reads *"DIAMOND Wt 0.880 Ct · HSN 710239 · 0.176 · 92,400"*. **The shop sells loose stones as well as tagged pieces, and a loose-stone line is keyed by carat, not by barcode** — so it lives in `sales_vou_detail`, which our loader read only into `mcj_sold_stones`.

All 39 of the line-less bills have rows there.

Why this is a recovery and not a double-count

`sales_vou_detail` covers 94 bills, and the two halves behave differently — provably, not plausibly:

	bills	sales_vou_detail	existing lines	invoice goods
already had lines	55	22,57,117.33	82,11,912.00	82,11,912.00
had no lines	39	89,26,345.00	0.00	89,26,345.00

For the 55, the existing lines already equal the goods to the paisa, so their detail rows are stone **components inside** those amounts — inserting them would double-count. For the 39, the detail equals the goods to the paisa, and **every one of the 39 ties individually**. There the detail is the line.

So the recovery is restricted to invoices that have none, which also makes it idempotent: a second run finds lines and inserts nothing. Proven by running the loader twice.

The result

```
invoice lines      328 → 367
bills with no lines 39 → 0
lines against the goods on the bills, difference: 0.00
```

That last line is a stronger invariant than anything the loader asserted before: **every rupee of goods on all 224 bills is now accounted for by a line.** Both checks are now gates in `load_invoices.sh`, so a silent return to 39 fails the load.

The recovered line for the printed bill, against the printout:

printed	DIAMOND Wt 0.880 Ct	HSN 710239	0.176	105000	92,400
recovered	DIAMOND	HSN 710239	0.176	105000	92,400

HSN 710239 now carries 20 lines across 20 bills and ₹12,06,553.00 in the GSTR-1 summary — sales that previously contributed nothing to the return, because that summary joins line → item → HSN and there was no line to join.

Reconciliation gate 8 now passes: GST/2627/00215 matches the printed invoice: 93786.00. **5 of 8 gates passing, 3 failing, 0 pending** — the three are B19, B4 and B8, all known and all open for reasons of their own.

Two things that could have gone wrong quietly

A carat is not a gram. gw 0.176 is grams and ct 0.880 is carats; the line stores the gram figure, which is what the column means, and the carat figure is not lost because mcj_sold_stones carries it from this same table. rate is per carat on these lines, which is the shop's own rate and consistent with how rate already behaves: **on none of the 328 pre-existing lines does rate × weight equal amount**, because amount carries making and stones too. Nothing here should be multiplied out.

Reloading the invoices empties the sold stones. mcj_sold_stones_invoice_id_fkey deletes on cascade, so load_invoices.sh takes all 203 rows with it. Inside load_all.sh that is harmless — load_sold_stones.sh runs next — but this script is also run on its own, and it was: the count went to 0 and nothing said so. It says so now, on stderr, naming the script to run. A loader that silently empties another table is how a register goes quiet.

221. An order rate that is not a rate per gram — LOW (marked, and a question for the shop)

Following [defect 220]'s method outward: which staged legacy tables does no loader read? **53 of the 149.** Most are already accounted for in [defect 176] as configuration, session state or the five sealed under [blocker B15]. Two looked transactional and were checked:

voucher_hsn (**756 rows**) — **nothing to migrate.** Its hsn_no is 0 on every row. An empty shell.

order_rate_master (**2,520 rows**) — **a second record of the rate promised on an order**, per order and per purity. The shop keeps the same promise twice: once on customer_order_main.rate, which we load as mcj_customer_order_lines.meta_rate, and once here.

They agree on 340 of the 366 lines and are within 6% on 17 more — ordinary drift, the lock taken on a different day. **Using the locked rate throughout moves the order book's exposure by – ₹18,457.92 on ₹85,48,154.20 — 0.2%**, which is not worth loading a table for.

Two lines are not drift, and both are **coins**:

COIN-10 GM	2 pcs	20.000 g	1,52,000	9.8x the day's rate
SILVER COIN 1KG	3 pcs	3000.000 g	24,600	97.0x

The first **is correct, as a rate per coin**: ₹1,52,000 for a 10 g coin is ₹15,200 a gram against the day's ₹15,482, and rate × pieces is exactly its ₹3,04,000 estimate. The second is a cancelled order with no estimate and a figure nothing reconciles — ₹24,600 for a kilo of silver is ₹24.60 a gram.

The unit is not consistent even among coin orders. C0/2627/00108 is also COIN-10 GM and quotes 15,568, which is per gram. Nothing in the row says which is meant, and order_rate_master is per gram for both.

What was done, and what was not

The report showed the per-coin figure as "Rate on order" beside a per-gram "Rate today", and computed a shortfall from the difference — which came out as zero, reading as *nothing to make up*. It now marks the rate and **computes no shortfall**: subtracting a per-gram market rate from a per-coin quote is not a smaller number, it is a meaningless one.

```
C0/2526/00402 · VIJAY MITTAL · SILVER COIN 1KG · 3,000.000 · 24,600.00 ? · 253.50 ·
```

Which unit was meant on a coin order is a question for the shop, not something to infer. Nothing was rewritten.

Two things I nearly got wrong, and how

A zero rate is already handled. Five lines carry metal_rate = 0 , and read straight from SQL they produce ₹3,181.43 of phantom exposure — the whole current value, as though the customer had been promised the metal for nothing. That number never reaches the screen: the Go layer guards on orderRateP > 0 and shows no rate and no shortfall. **I measured the query and not the report**, and the report was right. [Defect 189]'s error again — reading a thing through something that is not the thing.

The first version of this fix suppressed a real figure. I flagged both lines as a units error on the strength of a ratio, wrote a comment asserting "a quote copied across without its unit", and only then noticed that rate × pieces reproduces the ₹3,04,000 estimate exactly — the gold line is a correct per-coin quote, and I had been about to bury a genuine ₹5,640 shortfall under a "?". The marker survives because the arithmetic genuinely cannot be done; the reasoning behind it does not, and has been rewritten.

And the purchase side, checked and clean

The same question was asked of purchases: do the 43 invoices carry their detail? They do — 60 rows across all 43. The first check said contents exceeded the bills by ₹36,45,156.91, which was **my formula, not a defect**: taxable is not the sum of the content rows. Grouping by kind settles it —

20 bills with a metal row	taxable = the metal row's value	₹1,35,68,049.00
23 stone-only bills	taxable = the stone values	₹98,36,702.50

43 of 43 tie. Stones and making are components recorded beside the headline, not additions to it — the same shape as the sales side in [defect 220], reached the other way round.

222. A piece could be sold before it had ever arrived — MEDIUM (fixed)

The last several rounds audited screens one at a time. This one ran **a whole day's trade** instead, which is a different question: every screen can be right and the day still not work.

```
1. tag a piece in      DAY00001  12.500 / 12.000 g at 15,153    1,81,980.00
2. sell it            GST/2627/00225  sold by SHREE CHAND SHARMA  1,87,439.00
3. buy old gold       OGP/2627/00026  8.200 / 8.000 g at 88.5% tunch  1,05,000.00
4. take the balance   RCPT 281  CASH 82,439.00
5. customer returns it SR/2627/00004  back on the shelf
```

It works. All four vouchers balance to the paise, the customer's account nets to **exactly 0.00** across the sale, the old gold and the receipt, the day book lists all three, and the shelf ends at baseline plus the new tag. The books' only imbalance is still the pre-existing 0.01 of [blocker B4] — the day added nothing to it.

PURC-OG	26	dr	1,05,000.00	cr	1,05,000.00	0.00
RCPT	281	dr	82,439.00	cr	82,439.00	0.00
SALE-GST	225	dr	1,87,439.40	cr	1,87,439.40	0.00
SALE-RET	4	dr	1,87,439.40	cr	1,87,439.40	0.00

Then the piece's own history was read, and it had none:

DAY00001	SALE-GST	GST/2627/00225	Admin User
	SALE-RET	SR/2627/00004	Admin User

The trail begins at the sale. Nothing said when the piece arrived on the shelf or who put it there. Asked "where did this come from", the stock register had no answer for anything the shop had made since the migration.

The shop's own trail does not start there. **NEW-BAR is the largest movement type it keeps — 6,587 of 9,036, 73% of everything** — and 6,395 of those name a person and the machine they were sitting at:

89200048	NEW-BAR	New Create	PRIYANKA	122.163.127.22	18 Aug 2026
----------	---------	------------	----------	----------------	-------------

The reference is the literal **"New Create"**, on all 6,395. It is the one movement in the register that names no document, and rightly: **making the tag is the event**. There is no bill behind it to look up.

Fixed

Tagging now writes its NEW-BAR movement — reference, person and address — dated the day on the form, not the day it was typed ([defect 212]):

DAY00003	NEW-BAR	New Create	Admin User	127.0.0.1	23 Aug 2026
----------	---------	------------	------------	-----------	-------------

The address is trimmed to the bare host, because the 6,395 captured rows hold **122.163.127.22** and not a port.

`TestTag_RecordsThePieceArrivingOnTheShelf` checks all four — movement type, the literal reference, that somebody is named, and that it carries the form's date. Removing the write fails it with *"the tag arrived on the shelf with no movement"*.

The whole rehearsal was then reversed: **1,848 vouchers · 5,229 lines · 224 invoices · 367 invoice lines · 2,575 in stock · 25 old gold lots · 9,036 movements**, shelf ₹14,23,61,188.20.

Why the per-screen tests could not have found this

Every screen involved was individually correct and individually tested. The tagging screen created a tag; the sale sold it; the return brought it back; each wrote what its own tests asked for. **The gap was between them** — a table that three screens write and only two of them were writing to.

That is the argument for rehearsing a day rather than testing a screen: the question "does this screen work" and the question "can this shop trade" are not the same question, and the second one is the one the shop is asking.

223. A purchase records the money but not the goods — B26's purchase half is still open — HIGH

[Defect 222] rehearsed a selling day. This one rehearsed a buying day, and the buying day also works — up to a point.

1. buy from a supplier	GDP/2627/00025	ANAND SILVER PVT LTD	4,00,000 + 12,000 tax
2. pay by cheque	PMNT 631	778812	HDFC BANK LTD 4,12,000.00
3. reconcile the cheque	cleared,	dated 25 Aug	
4. correct the payment	narration changed,	reconciliation untouched	

Everything ties. The purchase debits **INPUT** CGST and SGST rather than the accounts that hold tax collected from customers; the supplier's account nets to **exactly 0.00** for the day; the payment is **not** born reconciled and only becomes so on the reconcile screen, which sets the flag and the date together; and editing the reconciled voucher afterwards changes the narration and leaves the clearing alone — [defect 214] confirmed in a real flow rather than a unit test.

Then the bill was read for what it bought, and it had bought nothing.

GDP/2627/00025	4,00,000.00	contents: 0
GDP/2627/00003	650,500.50	contents: 2
GDP/2627/00002	1,871,518.15	contents: 3

All 43 of the shop's captured purchase bills carry contents. Ours carries none, and nothing in the system writes that table — `mcj_purchase_contents` has two readers and no writer.

The purchase is therefore **invisible** to the Purchase Contents register. Asked for August, the report lists nine rows of real purchases and silently omits the ₹4,00,000 one.

What is missing

`mcj_purchase_contents` is what the bill actually bought, and the shop's own data is mostly stones:

metal	21 rows	20 bills	983.557 g	₹1,35,68,049.00
stone	39 rows	34 bills	521.329 ct	₹1,27,31,698.71
11 bills bought metal and stones · 9 metal only · 23 stones only				

More than half the bills by count are loose stones. A purchase screen that cannot record them cannot describe most of what this shop buys.

Why this is a build and not a fix

[Defect 195] closed what it could: it added the bill **header**, so a purchase now produces a `mcj_purchase_invoices` row with supplier, tax split, due date and total. [Defect 194] recorded B26 as closed on the trade path on that basis. **That was true of the header and not of the contents**, and the distinction was not drawn at the time.

The screen takes a party, a date, one goods amount, a tax rate and a reason. There is nowhere to put an item, a weight, a purity, a carat count or a rate, so there is nothing to write contents *from*. Closing this means giving the purchase screen per-line entry of the same shape old gold got in [defect 204] — metal lines with weight and purity, stone lines with carats — and writing both kinds.

That is the largest remaining piece of work on the trade path, and it is recorded here rather than half-done: **a purchase screen that wrote one empty content row per bill would satisfy the table and describe nothing.**

What the buying day does prove

Nothing above is a regression, and three earlier fixes held under a real sequence rather than a test:

- the GDP/GCP series ([defect 205]) — the bill came out `GDP/2627/00025`, continuing the stock-purchase series
- `due_on` is filled, so the bill is visible to an ageing report
- bank clearing stayed the reconcile screen's alone ([defect 214]), including through an edit

224. B26's purchase half is closed — a purchase now records what it bought — HIGH (fixed)

[Defect 223] found it and sized it rather than half-doing it. This closes it.

The purchase screen now takes the breakdown the shop's own 43 bills carry: up to two **metal** lines with item, purity, gross, net, rate and making, and up to three **stone** lines priced by the carat.

What the shop's own rows had to teach first

Three things, and two of them contradicted my first reading:

The metal row's value is a line value, not the bill total. It looked like the total, because 19 of 21 metal rows equal their bill's taxable exactly. The two that do not are the same bill — `GCP/2627/00001` carries **two** metal lines, 25,319.25 and 26,640.00, which sum to its 51,959.25. So the rule is that metal values **sum** to the goods, and a screen offering only one metal line could not have recorded that bill.

A stone is measured twice. `gross_weight` is `carats × 0.2` on 32 of the 40 stone rows — a carat is a fifth of a gram — and the shop stores both. Our first attempt wrote the carats and left the gram weight at zero, which the comparison against `GDP/2627/00003` caught.

Metal lines name an item too. 21 of 22 do — GOLD ORNAMENTS, SILVER ARTICLE, DIAMOND RING. The first version of the screen offered an item on the stone line only.

What has to add up

```
metal lines present → their values sum to the goods on the bill
no metal at all     → the stone values sum to it instead (23 of the 43)
```

The stones on a bill that also has metal are what is **set into** that metal and are already inside its value. Adding them again would overstate every studded purchase — the same shape as [defect 220]'s 55 bills, reached from the other side. The screen enforces this and says which total it used:

```
The metal lines come to 90,000.00 and the goods on the bill are 1,00,000.00. They have to agree.
```

A purchase with no breakdown at all is refused rather than filed blind:

```
Say what the bill bought. A purchase with no contents is a payment,
and it will not appear in the purchase register at all.
```

Proven by reproducing one of the shop's own bills

GDP/2627/00003 was re-entered through the rebuilt screen as GDP/2627/00025 :

bill	kind	item	purity	gross	net	carats	rate	value
making								
GDP/2627/00003	metal	GOLD ORNAMENTS	14	47.401	46.241	0.000	8789.94	
650500.50								39304.85
	stone	DIAMOND		1.160	0.000	5.800	34500.00	
200100.00								0.00
GDP/2627/00025	metal	GOLD ORNAMENTS	14	47.401	46.241	0.000	8789.94	
650500.50								39304.85
	stone	DIAMOND		1.160	0.000	5.800	34500.00	
200100.00								0.00

Identical in every column. The Purchase Contents register went from 9 rows in August to 11, ours among them — the register that could not see a purchase at all in [defect 223]. Then reversed: 43 bills, 60 content rows, 1,848 vouchers.

Seven older tests had to be given a breakdown

Every existing purchase test posted a bill with no contents, and every one is now correctly refused. They are about other things — the input tax accounts, the voucher type, the bill series, the failure messages — so they take a one-line `metalFor` helper rather than six form fields each. **A test that has to be changed by a fix is worth looking at twice:** here they were not asserting the old behaviour, they simply predated the requirement.

`TestTrade_PurchaseRecordsWhatItBought` compares every column against the shop's own row.

`TestTrade_PurchaseContentsMustAgreeWithTheBill`, `TestTrade_PurchaseCanBeStonesAlone` and

`TestTrade_PurchaseWithNoContentsIsRefused` cover the rest.

225. Seven purchase bills held a total their own figures could not reach — MEDIUM (fixed)

Following the chain from a tag back to where it came from, `legacy.stock_inward` turned out to BE the purchase bill — `billno GDP/2627/00001`, the same 43 rows — and it carries two columns this schema had nowhere to put.

adl_charge — 7 bills, ₹30,662.56, and it was inside the total

GDP/2627/00004	taxable	751766.19	+	tax	23248.61	-	0.36	=	775014.44
	total	798202.00							23187.56 short

Seven of the 43 stored a total their own components could not explain. The missing figure is the supplier's additional charge — freight, hallmarking — and it is not a footnote:

$(751766.19 + 23187.56) \times 3\% = 23248.61$	exact
$(650500.50 + 4410.00) \times 3\% = 19647.32$	exact

The tax is charged on the goods PLUS the charge, on all 7, and the shop debits its purchase account with the sum of the two as a single line — ₹654,910.50 on `GDP/2627/00003`. So the charge is part of the taxable base and part of the cost, and storing only the goods left the arithmetic unresolvable.

before	36 of 43 bills add up
after	43 of 43

ref_no — the supplier's own invoice number, on 22 of 43

`DNS/26-27/00001`, `ZCMH001426-27`, `TIS/26-27/0263`. Nothing in this system tied a bill to the piece of paper it came from, so reconciling a supplier statement was done by hand.

And the goods of a registered dealer go to their own account

The 43 split on it **without exception**: all 7 marked b2b post to `PURCHASE_B2B`, all 36 others to `PURCHASE`. Our screen always used `PURCHASE`. It cannot be inferred — no ledger carries a GSTIN ([defect 150]) — so the form now asks, which is also the only way `is_b2b` has ever been set here.

Proven by reproducing the bill in full

`GDP/2627/00003` re-entered with its charge, its reference and its b2b flag:

PURCHASE_B2B	debit	654,910.50	(650,500.50 goods + 4,410.00 charge, as one line)
INPUT CGST	debit	9,823.66	
INPUT SGST	debit	9,823.66	19,647.32 together — the shop's own tax figure
ROUNDED OFF	debit	0.18	
supplier	credit	674,558.00	

`taxable` holds the goods and `adl_charge` sits beside it, as the shop's own bills read. Both are now gates in `load_purchase_invoices.sh`.

Two mistakes made getting here, both mine

The backfill was put in the migration, where it can never run. Migrations run before the loaders, so on a fresh setup there are no invoices to update yet — the live database came back with 0 charges and 0 references after a reload. The backfill belongs in `load_purchase_invoices.sh`, and now lives there.

The migration referenced `legacy.stock_inward` unguarded, and broke every test. That schema is staged on this machine and nowhere else — not in the test harness, not on a deployment. It is [defect 209]'s shape again: a migration that cannot run in a state the system is legitimately in. Now wrapped in a check for the table.

And a third, worse than either: I ran `migrate down` to test the fix and rolled back 122 migrations, emptying every table. The warning against exactly this is written in [defect 209] — *"a full `migrate down` against the live database would drop the migrated tables and everything in them, so the fix was verified on a throwaway copy"* — and I did it anyway on the live one.

Nothing was lost: the staged legacy schema is the source and survived intact, so `load_all.sh` rebuilt everything, and all twelve baselines came back exact — 1,848 vouchers, 5,229 lines, 2,816 tags, 2,575 on the shelf, 785 ledgers, 224 invoices, 367 invoice lines, 43 purchase bills, 60 content rows, 9,036 movements, 25 old gold lots, 203 sold stones, shelf ₹14,23,61,188.20.

That it was recoverable is the migration's doing, not mine. The lesson is the one already written down and not followed: verify a rollback on a copy.

226. A piece on the shelf could not be traced back to the money that bought it — MEDIUM (fixed)

[Defect 224] gave a purchase its contents and [defect 225] gave it its charges and the supplier's own reference. Nothing joined any of that to the piece that ended up on the shelf.

`mcj_stock_items.voucher_no` is the document a piece arrived on, filled on **2,797 of the shop's 2,816 tags**, and its series follows `created_from` exactly:

stk	2,401	STK/...	re-tagged out of existing stock
purc	316	GDP/ GCP/ DP/	bought from a supplier
ret	45	SR/...	returned by a customer
rec	33	RV/...	received from a karigar
og	2	OGP/...	taken in as old gold

Our tagging screen wrote none of it. A piece created since the migration stood on the shelf naming no document, so the chain from **money paid** → **what was bought** → **what is on the shelf** stopped at the second link.

Fixed

The screen asks **"Came in on"**, and offers the documents that actually exist for the source chosen — 43 purchase bills for `purc`, 25 old gold bills for `og`, the sale returns for `ret`. A piece re-tagged out of existing stock is offered **nothing**, deliberately: an STK number is the shop's own counter with no document behind it, and inventing one would be inventing a record.

A tag saved against GDP/2627/00003 now resolves the whole way:

barcode	from	came in on	supplier	goods	charge	their invoice
ARR00001	purc	GDP/2627/00003	ZEST CORPORATION	650500.50	4410.00	ZCMH001426-27

TestTag_NamesTheDocumentThePieceArrived0n checks that the bills are offered for a bought piece, that a re-tagged one is not asked, and that the saved tag joins to its bill.

A user had to be recreated first

The migrate down in [defect 225] rolled back the core tables too, so load_all.sh restored the shop's data but not the login it is used through, and the first attempts here were redirected to a login page rather than failing — which read as "the field is not rendering". **A 302 is not a rendering bug**, and the answer came from reading the server log rather than the HTML.

cmd/seed-admin recreated it. Worth writing down because the reload documented in defect 225 was not complete after all: load_all.sh **restores the shop, not the system.**

What a tag still does not carry

Unchanged from [defect 212] and still recorded rather than quietly dropped: lot_no (2,059 of 2,816), category_id (1,629), has_mrp (978), certificate_no (678 — [blocker B9]), remarks (452), hallmarked (432), ref_no (260), design_code (109), size (99), wastage_percent (26). voucher_no, the largest of them at 2,797, is now written.

227. The shop could not tag a piece it sells at a fixed price — MEDIUM (fixed)

has_mrp is true on 978 of the shop's 2,816 tags, and it turns out to mean something quite specific. Every one of those 978 carries:

rate 0 · making_value 0 · amount 0 · total = the price

The price is in total and **nowhere else**. These are silver murtis, silver watches, a gold watch case — pieces sold at their own price whatever the metal does that morning. **949 are on the shelf, worth ₹21,56,285.00.**

The sale already handled them, and was checked rather than assumed

Selling one bills correctly:

78960405	GOLD WATCH CASE	14	43.910	metal 0.00	stones 0.00	fixed price	85,000.00
				Goods 85,000.00 · CGST 1,275.00 · SGST 1,275.00 ·			
Payable 87,550.00							

That was the first thing tested, because a pricing engine that computes metal + stones + making would have billed this piece ₹0. It does not — the stored total is used when the tag is fixed-price.

The tagging screen could not make one

It priced the metal and added the making, so the shop's own ₹35,000 murti came out of our screen at **₹50,018.90** — 78.970 g of silver at 253.50 plus 30,000 of making. And a fixed-price piece in a metal with no rate that day was refused outright, which is precisely the case where the metal rate is irrelevant.

So the shop had 949 such pieces on the shelf and no way to add a 950th.

Fixed, and proven the whole way round

The screen now takes a fixed price, and a piece made that way is indistinguishable from the shop's own:

barcode	description	has_mrp	net_weight	rate	making	amount	total
MRP00001	SILVER MURTI	t	78.970	0.00	0.00	0.00	35000.00
86829935	SILVER MURTI	t	78.970	0.00	0.00	0.00	35000.00

Then sold: billed at exactly **₹35,000.00**, ₹36,050.00 with tax — and it **appears in the MRP Sales report**, which filters on `has_mrp`. That report could never have shown a piece made here before, because nothing wrote the flag.

`TestTag_CanPriceAPieceAtAFixedPrice` checks the shape the shop's 978 carry — rate, making and amount all zero, the price in `total` alone. `TestTag_AFixedPricePieceNeedsNoMetalRate` checks that a metal with no rate is not a reason to refuse, and that a fixed-price piece with no price still is.

Worth noting

This was found by asking what a column *means* rather than whether it is filled. `has_mrp` was on [defect 212]'s list of ten unwritten tag columns, filed under "needs a field on the screen" with `lot_no` and `size`. It is not the same kind of thing: the others describe a piece, and this one **prices** it. A list of missing columns flattens that distinction, and reading the rows behind one of them is what brought it back.

228. Blocker B9 is not a missing formula — a certified piece's price is negotiated — HIGH (answered)

[Blocker B9] has stood as *"the certification pricing step is not in the captured data"*, and it blocks a great deal: the sale screen refuses every certified tag outright ([defect 203]), which is **549 pieces on the shelf worth ₹2,33,45,417.46 — 21% of the shelf value — that cannot be sold at all.**

Applying [defect 227]'s lesson — ask what a column *means* — the data does answer it, and the answer is not the one the blocker was looking for.

The certification charge is settled

It is **not** part of the tag's price. Of the 350 certified tags carrying a charge, **not one** has `total = amount + making + certification_charge`.

It appears on the **bill** instead. `mcj_sales_invoices.certification` equals the sum of the sold tags' `certification_charge` on **32 of the 41** bills that carry one. That much is a rule, and the shelf is owed ₹1,41,871.00 of it.

The piece's price is not settled, because there is no formula

129 certified pieces have been sold. Against each line:

billed at the tag's stored total	1 of 129
billed at what our engine computes	0 of 129
billed at neither	128 of 129

And the ratio of what was billed to what the tag says does not cluster anywhere:

0.44 – 0.60	13 lines	1.00 – 1.20	28 lines
0.60 – 0.78	36 lines	1.20 – 1.40	14 lines
0.81 – 1.00	23 lines	1.41 – 3.47	15 lines

A piece is sold at anything from **44% to 347%** of its tag. And the bill re-states the piece completely: of the 129 lines, **125 carry a different metal rate from the tag and 107 a different making**. Only 4 carry both unchanged.

22070670	tag 49,004.57	→	billed 1,69,838.00
50784726	tag 1,64,815.76	→	billed 97,457.00

The tag is a starting point, not a price. There is no step missing from the captured data; the price is agreed at the counter, as it is for old gold ([defect 204]) and for a customer's coin order ([defect 221]).

What that means for the fix

The blocker as posed — find the pricing rule — has no answer, and looking harder will not produce one. What the screen needs is a **per-line price**, the way old gold now takes the amount agreed rather than computing one. That is a change to the sale screen, and it is worth stating separately from this finding because it is a decision about how the counter works, not a fact about the data.

Nothing was changed here. **A blanket refusal is still the right behaviour until that field exists** — billing a certified piece at a computed price would be wrong 128 times in 129.

The three things B9 can now be told

1. **The certification charge is a bill line**, not part of the tag, and the rule holds on 32 of 41 bills.
2. **The piece's price is negotiated**. 128 of 129 sold pieces were billed at neither the tag's price nor a computed one, at $0.44\times$ to $3.47\times$.
3. **The fix is a price field, not a formula** — and until it exists, 549 pieces worth ₹2.33 crore stay unsellable.

Checked along the way, and worth keeping

The sale's pricing engine reproduces the stored price on **97.7% of ordinary tags** (1,146 of 1,173) — so the engine is sound and this is not a general pricing problem. It matches only **72%** of certified ones, which is the same finding from the other side: certified pieces are not priced by the formula.

The 853 MRP tags match 0% by design, and that is correct — their price is the stored total, which [defect 227] made the screen able to produce.

229. B9 closed — 549 certified pieces worth ₹2.33 crore can now be sold — HIGH (fixed)

[Defect 228] established what [blocker B9] actually is: not a missing formula but a negotiated price. This acts on it.

What the screen does now

A certified tag is no longer refused. It is **held back for its price**, with a box to answer in and the tag's own figure as the opening number:

```
41841396  DIAMOND BANGLE  certified — enter the price agreed with the customer
[1231268.56]
```

Name a price and it goes on the bill at that price:

```
41841396  DIAMOND BANGLE  18  58.648  fixed price  11,00,000.00
          Goods 11,00,000.00 · CGST 16,500.00 · SGST 16,500.00 · Payable 11,33,000.00

bill_no      goods      certification  total      line      line_cert
GST/2627/00225  1100000.00  19089.00      1133000.00  1100000.00  19089.00
```

The certificate is charged for, and not charged on

That distinction is the shop's, and it is exact. **On all 41 captured bills carrying a certification charge, the bill total EXCLUDES it** — `goods + round_off + tax = total`, 41 of 41. The charge is recorded on the bill and on the line, and the customer does not pay it.

So it is written to both `mcj_sales_invoices.certification` and the line's own column, and left out of the total. Getting this backwards would have overcharged every certified sale by the cost of its certificate — ₹1,41,871.00 across the shelf.

Why an agreed price is the right mechanism and not a shortcut

It is what the shop does. Of its 129 certified sales, **one** matched the tag's stored price and **none** matched a computed one; the rest ran 0.44× to 3.47× of the tag, with 125 of 129 lines carrying a different metal rate and 107 a different making. A screen that computed a price would have been wrong 128 times in 129.

It is also the mechanism already accepted twice on the evidence of the same kind of data: old gold takes the amount agreed ([defect 204]) and a coin order quotes per piece ([defect 221]). **Three of this shop's trades are priced by agreement rather than by formula, and all three now say so.**

What is still refused, and rightly

Everything else the refusal panel ever held: a tag scanned twice, a barcode that does not exist, a piece already off the shelf. A certified piece is the only refusal a biller can answer on the spot, because it is the only one where the missing thing is a decision rather than a fact.

TestSale_CertifiedPieceIsBilledAtThePriceAgreed checks the hold-back, the box, the agreed price reaching the line, the certificate charge on the bill, and the total excluding it.

TestSale_ListsRefusedTagsRatherThanDroppingThem was asserting the old blanket refusal and now asserts the hold-back instead.

The shelf, before and after

before	549 certified pieces, 2,33,45,417.46, unsellable
after	549 certified pieces, 2,33,45,417.46, sellable at the price agreed

That is **21% of the shelf value** that the system could not put on a bill.

230. A sale took no money — HIGH (fixed)

The trade path could bill, but not take payment for the bill.

mcj_sale_payments is the shop's record of **how** a bill was settled — 159 entries against 141 of its 224 bills, ₹52,52,946.00 in all:

cash	101	46,19,519.00
card	50	5,73,846.00
bank	8	59,581.00

Nothing in the system wrote it. The table had one reader — the Payments report — and no writer, so a sale made here was invisible to it and to card commission.

What the shop actually does, and it is two things

Not one of the 224 sale vouchers carries a cash line. A bill always leaves the customer owing; the money moves on a **receipt** voucher afterwards, and a matching RCPT exists for the paid bills. Separately, the mode and instrument are noted against the bill.

So a bill and its settlement are two records, and this screen did the first and neither of the others. The [defect 222] rehearsal only looked settled because I posted the receipt voucher by hand as a second step — which is what the shop would have had to do for every sale.

Fixed

The sale screen now has a till: **paid by**, amount, which account it goes into, a reference and a card type. Filled in, it does both things the shop does —

mcj_sale_payments	GST/2627/00225	cash	5000.00	2026-08-23
RCPT 281	Received against GST/2627/00225			
	CASH	debit	5000.00	
	ABDUL RAHMAN	credit	5000.00	

Left empty, the bill goes out on credit and **no receipt is invented**, which is what 83 of the shop's 224 bills did.

Doing only the voucher would leave the books right and the till silent.
Doing only the register would say the money came in and never move it.

What it refuses

A half-entered till. A mode with no amount, or an amount with nowhere to go, is refused rather than guessed at — the shop's own records are equally definite, 141 bills with a payment and 83 without, and none with half of one.

More than the bill. *"That is 9,99,999.00 against a bill of 13,405.00. A payment cannot be more than the bill."* Less is fine and is a part payment, which **26** of the shop's bills are.

`TestSale_TakesThePaymentAndPostsTheReceipt` checks both records,
`TestSale_ACreditSaleTakesNoMoney` checks that an empty till invents nothing, and
`TestSale_RefusesAHalfEnteredOrOversizedPayment` checks both refusals.

Worth noting

This is the fourth thing found by asking what the shop records that we do not, rather than by testing what we built. The others were the salesperson ([defect 208]), the piece's arrival ([defect 222]) and what a purchase bought ([defect 224]). **All four were tables with readers and no writer**, and none would have failed a test of the code as written.

231. Nothing could set the day's gold rate — HIGH (fixed)

The last four defects all had one shape: a table the shop fills, that our reports read, and that no screen writes. Rather than find a fifth by hand, the question was asked of every table at once —

`tools/check_unwritten_tables.py`, which narrows 60-odd tables to the **36** a report reads and the app never writes.

One of them stops a real day tomorrow.

<code>mcj_metal_rates</code>	2,820 rows	141 days x 20 grades	latest 2026-08-21
------------------------------	------------	----------------------	-------------------

2026-08-21 is the day the shop went down. Nothing in the system could add a rate, and there was no screen for it, so every sale would have priced gold at that morning's figure for as long as the system ran — silently, because a stale rate is not an error, just an old number. On 22ct alone the feed moved on 141 of 141 days.

Why it was missed

`gold_master.php` is **two screens in one table**. `legacy.gold_purity_master` holds nineteen **grades** and 141 days of **rates** — $20 \times 141 = 2,820$ rows. The grade half is the Purity Master, which was built and which claimed the legacy file. **The screen was therefore counted as covered, and the rate half of it had never been built.**

Coverage is measured by legacy file, and a file that is two screens counts once either way. That is not wrong as a measure; it is just not a measure of whether the shop can work.

The screen

Pick a date, see every grade with the rates in force, set them. A day with no rates of its own **shows the last ones set and marks them** —

14	sale 9,643.00	buy-back 8,743.00	carried from 21 Aug 2026
22	sale 15,153.00	buy-back 14,253.00	carried from 21 Aug 2026

— rather than showing blanks, because the carried figure is what a sale would actually charge and a blank grid leaves somebody guessing at it.

Proven the whole way round

```
set 22ct to 15,600.00 for 23 Aug
sell 9.950 g of 22ct    ->  1,55,220.00    = 9.950 x 15,600
                        at the old rate it would have been 1,50,772.35
```

Setting the same day twice leaves **one** row per grade, not two: a rate is a fact about a day, and the pricing lookup takes the most recent rate on or before the date, so a duplicate would make which one it finds a matter of chance. A blank grid is refused rather than written as a day of zero rates that every sale would then price from.

The rates were reversed afterwards — 2,820, latest 21 August.

Where it lives

Beside the Purity Master under SETUP → Item Management, registered with **no legacy file of its own**. The registry allows one screen per legacy file and is right to; `gold_master.php` is taken by the grades. Inventing a plausible `.php` name to claim would read as evidence about the old system and would be a fiction — which the navigation package says in as many words.

tools/check_unwritten_tables.py

Re-runnable, and it reports who writes each table: **app**, **crud** (a master screen, which builds its SQL at runtime and so has no `INSERT` to grep for), **loader** (frozen at migration), or **nobody**.

Being on the list is not automatically a defect — a tag-edit audit trail with nothing in the new system that edits a tag is legitimately historical ([defect 218]), and five tables are sealed under [blocker B15]. The tool cannot tell those apart. It narrows the question to the handful worth reading, which is all it should do.

35 remain on the list. The ones that look like this defect rather than like history: `mcj_stock_stones` (819), `mcj_scheme_payments` (2,144), `mcj_customer_orders` (284) with its 366 lines, `mcj_estimates`, and `mcj_parties` (40,519).

232. A stone-set piece could be tagged with no stones in it — MEDIUM (fixed)

Second from [defect 231]'s list. `mcj_stock_stones` is what is set into a tag: **819 rows over 692 tags, 2,731.821 carats worth ₹6,21,95,953.60**, and the app never wrote one.

The tagging screen took a **stone value** — one figure — and nothing else. That prices the tag correctly and tells the stone registers nothing, because they read the stones and not the figure. It also meant a piece sold later recorded **no stones sold**: that writer ([defect 217]) copies them from here, so it would have copied nothing.

Every tag on the shelf whose price includes a stone value has its stones itemised — 486 of 486.
A tag made here would have been the first without.

Fixed

The screen takes stone lines — stone, pieces, carats, rate a carat — and the tag's stone value becomes their sum rather than a figure typed beside them, so a tag cannot disagree with what is set into it.

```
STN00001  metal 5.200 g at 12,398.00    64,469.60
          DIAMOND 3 pcs 1.250 ct at 80,000.00  1,00,000.00
          amount 1,64,469.60 · making 3,000.00 · total 1,67,469.60

stored:    pieces 3 · carats 1.250 · gross 0.250 · unit Ct · rate 80,000.00
```

`gross_weight` is the carats at a fifth of a gram each, and the unit is `Ct` — the shape of all 819 captured rows. Then sold, and the stones went with it:

```
GST/2627/00225 STN00001 DIAMOND 3 pcs 1.250 ct actual 1.250 80,000.00 1,00,000.00
```

Reversed after: 2,816 tags, 819 stones, 203 sold stones, 224 bills.

The parameter-type trap, twice in one file

The first save failed with

```
inconsistent types deduced for parameter $2 (SQLSTATE 42P08)
```

`$2` is the barcode, selected into a varchar column and compared against `btrim()`'s text. Postgres deduces a parameter's type from how it is used, so casting one occurrence and not the other is worse than casting neither.

The same file already carries a comment about this, on `$6::int`, `$6::numeric` a hundred lines above, written the last time it happened. Knowing a rule and applying it are different things; the comment is now on both.

What is left on the list

34 tables a report reads and the app never writes, most of them legitimately historical. The ones still shaped like this defect: `mcj_scheme_payments` (2,144), `mcj_customer_orders` (284) with its 366 lines, `mcj_estimates`, and `mcj_parties` (40,519).

233. Nobody can pay a scheme instalment — MEDIUM (open, and one question for the shop)

Third from [defect 231]'s list, and the first that cannot simply be built.

mcj_scheme_payments holds **2,144 instalments across 238 enrolments, ₹87,90,900.00 collected between April 2017 and 10 August 2026** — Bank 1,287, Cash 528, Card 329. Nothing in the system writes it, so a customer walking in with this month's instalment cannot be recorded.

That is not a dormant register. **18 instalments were collected since the books open on 1 April 2026**, the most recent eleven days before the shop went down:

2026-04	5	32,000.00	2026-07	3	28,000.00
2026-05	4	31,000.00	2026-08	2	17,000.00
2026-06	4	31,000.00			

Reconciled with what was already known

[Defect 152] states 173 enrolments both fully paid and matured, ₹71,01,000.00 in and **₹77,37,300.00** owed. Recomputed independently here: **173** and **₹77,37,300.00**, to the rupee. It stands.

(A looser reading — matured, without the fully-paid condition — gives 235 and ₹99,74,900.00. Two different questions, and the difference is worth naming so the larger figure is never quoted as the debt.)

Why this is not just another screen to build

Taking money should move money, which is what [defect 230] settled for a sale. Here there is nowhere obvious for it to go: **no scheme, chit, deposit or instalment ledger exists among the 785**. The nearest groups are Advances (3 ledgers) and Security Deposit (2), neither of them about schemes.

What does exist: **all 239 enrolments have a customer ledger** — 239 of 239 — so an instalment could be credited to the customer, who is after all owed goods for it. That is defensible bookkeeping and it is not evidence that it is what the shop does.

So this is a question, not an inference: when a customer pays their monthly instalment, which account should it be credited to — their own, or a scheme liability account that would have to be created? Building a screen that takes money and books it nowhere would repeat the defect that [defect 230] just fixed.

A query of mine that was wrong, and the one that replaced it is not proof

The first check asked whether recent instalments appear in the ledger and answered **3 of 18**, which reads as money collected and not booked. That query filtered to RCPT vouchers alone. Widening it to any voucher type matched **18 of 18** — 17 of them journals debiting **Google Pay** (13) and **PAYTM** (3), which is what a UPI collection would look like.

Neither number is evidence. A voucher on the same day for the same amount is a coincidence in a shop that takes money all day, and I have no way to tie an instalment to a voucher. **The honest answer is that I cannot tell from this data whether scheme collections are booked**, and the near-miss is recorded because "3 of 18" would have been a striking and false finding.

What building it needs

The screen itself is straightforward — pick an enrolment, show what is paid and what is due against instalments and instalment_amount, take the next one with its mode. It is the ledger side that is blocked, and only on one answer.

234. Nobody could take an order — MEDIUM (fixed)

Fourth from [defect 231]'s list, and unlike [defect 233] nothing blocked it: an order takes no money. `mcj_customer_orders.advance` is **zero on all 284**, so there is no ledger question to answer first.

284 orders and 366 lines were on the books and nothing could add a 285th. That is the front half of a trade the rest of the system already handles — the order book values the shop's exposure at ₹85,48,154.20, the tagging screen can make the piece, the sale can bill it — and nobody could take the order.

CO	179	custom orders
RP	105	repairs, the customer's own goods

The screen

Customer, date, kind, who took it, remarks; then lines of item, metal, pieces, gross, net, making, estimate, wanted-by date and status. Taken:

CO/2627/00114	23 Aug 2026	ABDUL RAHMAN	C BANGLE	Process	05 Oct 2026
	24.000 g	estimate 4,00,000.00	rate 15,153.00		

The rate is locked at the day's, which is the whole point of the order book: a customer promised today's gold for a piece delivered in six weeks, and all 366 captured lines carry the rate they were struck at. 15,153.00 is what 22ct stood at on 23 August. It appears in the order book report immediately.

Each series numbers from 1 within its financial year, so the next custom order took **CO/2627/00114** against the shop's last CO/2627/00113.

What it refuses

A line with no wanted-by date. All 366 captured lines have one, and an order nobody has promised a date for is a conversation rather than an order — the order book's whole job is saying what is late. And an order with nothing on it is not an order.

The guard caught me inventing a filename

The screen was first registered against `LegacyFile: "customer_order.php"`, and the navigation guard refused it:

```
/mcj/orders/new claims LegacyFile "customer_order.php", which is neither a
menu row nor a captured page -- an invented name reads as evidence about the
old system
```

There is no such page. The menu row is `customer_order_list_new.php`, which the order book **report** already claims, and the shop took orders from inside that list. So this screen registers with **no legacy file**, placed beside it under MANUFACTURING — the same treatment the rate screen got in [defect 231], and for the same reason.

That guard exists because a plausible `.php` name in this codebase reads as evidence about a system nobody can check any more. It was right and I was wrong, which is the second time this session a guard has caught something I had already written down as a rule elsewhere.

235. A full day's trade, rehearsed end to end after twelve builds — no defect found

[Defect 222] rehearsed a day and found that a piece had no beginning. [Defect 223] rehearsed the buying side and found a purchase bought nothing. Twelve builds later the same test was run again, this time exercising everything added since, to see whether it still holds together.

1. morning	set the day's rates	22ct to 15,800.00
2. an order	C0/2627/00115	bangle for 10 Oct, rate locked at 15,800.00
3. a new piece	DAY9001	bought in on GDP/2627/00003, diamond-set
4. the sale	GST/2627/00225	sold by SHREE CHAND SHARMA, 1,00,000.00 cash

It holds. The piece priced at the rate set that morning —

metal	14.500 g at 15,800.00	2,29,100.00
stones	2.400 ct at 95,000.00	2,28,000.00
making		7,000.00
		4,64,100.00
		-> bill 4,78,023.00 with tax

— carried its arrival movement and its purchase bill, sold with its salesperson, its piece count, its gross and fine weights, its stone going with it, the cash recorded at the till and a receipt posted for it. Both vouchers balanced to the paisa, the customer was left owing exactly the ₹3,78,023.00 unpaid, and the books' only gap was still [blocker B4]'s 0.01 — the day added nothing to it.

Everything then reversed: 1,848 vouchers, 2,816 tags, 2,575 on the shelf, 224 bills, 284 orders, 2,820 rates, 159 payments, 819 stones, shelf ₹14,23,61,188.20.

What that one piece touched

Twelve defects' worth of work, in one transaction:

rate set that morning	231
order taken, rate locked	234
arrival movement	222
linked to its purchase bill	226
stones recorded in it	232
piece count and weights	210
salesperson on the bill	208
who took it off the shelf	211
stones recorded as sold	217
cash taken at the till	230
receipt posted for it	230
purchase it came from	224, 225

None of those was found by testing the screen it belongs to. **Eleven of the twelve were found by reading what the shop's own rows contain and asking whether our code puts that there**, and the twelfth by walking a day rather than a screen.

Recorded as a non-defect on purpose

This entry finds nothing, which is worth a number of its own. A rehearsal that passes is the only evidence that the ones before it were fixed rather than moved, and the numbers above are the baseline the next one is measured against.

236. The rules this shop keeps, made checkable — tools/check_invariants.py

Twenty-eight defects this session came from one move: read what the shop's own rows contain, then check whether the code puts that there. The rules found that way were scattered across twenty-eight write-ups and nothing enforced them, so a twenty-ninth would have been found the same slow way.

They are now twelve checks, each naming the defect that found it:

every bill's lines add up to its goods	220
every bill carries at least one line	155, 220
every purchase bill adds up from its own figures	225
a purchase says what it bought	223, 224
a tag priced with stones has its stones	232
every piece has a beginning	222
a stock movement names a document	216
no sale voucher settles itself	230
a till entry never exceeds its bill	230
every voucher balances	B4
clearing a cheque sets both columns	214
an order line says when it is wanted	234

None of these is a schema constraint. A database will happily hold a bill whose lines do not add up; **this shop never has.**

Written the wrong way round first

The first version reported **four failures**, and every one was history:

```
57 tags from 2016–2023 that left the shelf with no arrival recorded
470 movements with no reference -- 278 opening stock, 192 legacy tag creations
1 voucher that does not balance -- blocker B4
31 payments cleared with no date, which arrived from the legacy reconciliation already so
```

Not one was caused by anything this system did. **A checker that cries wolf about the past gets switched off, and then it is not checking anything.**

So each rule now carries the number of exceptions the migration itself contains, and fails only when the count **grows**. That is the only version of the question that says anything about the code — the past will not change, and what matters is whether today adds to it.

every piece has a beginning	holds (57 from the migration, unchanged)
a stock movement names a document	holds (470 from the migration, unchanged)

Proven by breaking it

Three faults were planted — a bill whose lines do not sum, a movement naming no document, a cheque ticked without a date — and it named all three with the figure that matters:

```
x a stock movement names a document 471 ... -- 1 more than the migration's 470
x clearing a cheque sets both columns 32 ... -- 1 more than the migration's 31
```

Then removed, and 12 of 12 hold again.

Where it sits among the other tools

cmd/reconcile	did the MIGRATION land -- eight gates, run once and after a reload
check_invariants.py	do the shop's own rules still hold -- run after a day's trade
check_unwritten_tables.py	is there a register nothing writes -- 32 left, mostly historical
check_empty_columns.py	is a report reading a column nobody fills
check_report_footers.py	does any report total itself twice
status_figures.py	does STATUS.md agree with the filesystem

The first asks whether the past arrived intact. This one asks whether the present is being kept the way the shop keeps it, which is a different question and the one that matters now that the system can trade.

237. An order could be taken but never completed — MEDIUM (fixed)

[Defect 234] built the taking of an order. Nothing could finish one: no screen set `delivered`, so an order stayed outstanding for good and the order book's overdue list could only grow.

```
85 orders outstanding · 101 overdue lines · ₹1,14,01,568.00
oldest promise: 1,475 days
```

An overdue list that can only lengthen is not a list anybody reads, which is a quiet way for a real obligation to become invisible.

What the list shows

Work still owed, oldest promise first, with what was ordered, when it was wanted, how late it is, and whether the piece has been made yet:

RP/2223/00194	I. AHMED	C GOODS	wanted 09 Aug 2022	1,475 days
RP/2324/00106	SASHI CHOUDHARY	C NOSEPIN	wanted 11 Jun 2023	1,169 days
C0/2324/00621	ERFAN HOSSAIN	GOLD TOPS	wanted 05 Nov 2023	1,022 days
₹59,136.00				

Tick and hand over. Only an order still outstanding can be handed over — ticking one twice is a double-click, not a second delivery — and ticking nothing is refused rather than treated as a delivery of everything.

Worth your attention

The four oldest are repairs — customers' own goods. RP orders are work on pieces the customer brought in, and three of the four oldest are C GOODS or C NOSEPIN with no estimate against them, which is what a repair looks like. Two years and four years is a long time to hold somebody's own jewellery.

This is the list, not the answer: some may have been collected years ago and never marked, which is exactly what an unmarkable flag produces. **The screen can now clear them one by one as you confirm each; it cannot tell you which are real.** That is the same shape as [defect 164]'s ₹1.35 crore of gold issued with no recorded return.

TestOrder_CanBeHandedOver checks that a live order lists, hands over and stops listing;

TestOrder_HandingOverNothingIsRefused checks the empty case.

238. Every bank payment the shop has is against a bill nobody can look up — LOW

Chasing receive_amount — 141 rows on [defect 176]'s never-opened list, set aside there as "a payments table" — turned up nothing new: all 141 of its bills are already in mcj_sale_payments, so it is a duplicate view and no data was missed. That is the answer, and it is worth having.

But counting it produced a number that did not match. **149 distinct bills appear in mcj_sale_payments against only 141 invoices that have a payment.** Eight payments name a bill that is not on file:

GST/2526/00017	05 Apr 2025	bank	18,500.00
GST/2526/00066	29 Apr 2025	bank	1,300.00
GST/2526/00080	03 May 2025	bank	5,100.00
GST/2526/00094	14 May 2025	bank	5,000.00
GST/2526/00100	20 May 2025	bank	1,200.00
GST/2526/00104	21 May 2025	bank	800.00
GST/2526/00246	01 Aug 2025	bank	27,069.00
GST/2526/00622	29 Oct 2025	bank	612.00

All eight are from the previous financial year, and all eight are every bank payment the shop has. sale_pay_bank holds exactly 8 rows and not one of them belongs to a bill we hold.

Not a load error

legacy.sales_vou has **224 rows, all 2026-27** — GST/2627/00001 to GST/2627/00224. The shop's captured invoice history is the current year only, while its payment tables reach back further. The eight are orphans in the source, not something the loader dropped.

What it changes

[Defect 230] reported the till modes as "cash 101, card 50, bank 8" as though all three described current trade. They do not: **the bank mode is entirely historical and every row of it is unlinkable.** A Payments report grouping by mode shows a bank column that cannot be reconciled to a single bill, and will show one until a bank payment is taken through the new screen.

Nothing was deleted. Eight rows of ₹59,581.00 recording money the shop received are worth more on file than tidy.

Added to the invariant checker

```
a payment names a bill that exists    holds (8 from the migration, unchanged)
```

The baseline is 8 for the reason above. A ninth would mean a payment written here against a bill that does not exist, which is a different thing entirely.

239. The unwritten-tables tool overstated by a third — MEDIUM (fixed)

[Defect 231]'s sweep reported **36** tables a report reads and the app never writes, and that figure was quoted again in 232, 233 and 234 as it came down. It was wrong.

The tool looked for two things: a literal `INSERT INTO mcj_x` in the controllers, and a `Table: "mcj_x"` declaration for the codelist screens. There is a third place a table gets written, and eighteen screens use it:

```
crud.NewBuilder[ledger.Ledger](ledger.Schema(), ...)
```

The table name lives inside the domain aggregate's `Schema()`, which the grep never saw. So the **ledger master, the item master, the purity master, parties, packets, schemes** and a dozen others were all reported as never written when each has a working screen.

Corrected, the figure is **24**, and the tables that dropped off are the ones a shop uses most.

```
quoted    36 -> 35 -> 34 -> 32
actual    24
```

Why it matters more than the number

A tool that overstates gets believed once and discounted afterwards. This one was used to pick the last four things to build, and it could as easily have sent me to build a customer master that already existed. **It sent me to look at the ledger master to check exactly that — which is how [defect 240] was found**, so the overstatement was useful once and would not have been twice.

Both detections are now in the tool, with the reason written next to them.

240. A new customer could not be taken on — HIGH (fixed)

Checking [defect 239]'s false positive turned into a real one. **A ledger could not be created unless a ledger type was chosen**, and every sale, purchase, receipt and payment names a ledger, so the shop could not take on a new customer or supplier without knowing that.

```
insert or update on table "mcj_ledgers" violates foreign key constraint
"mcj_ledgers_ledger_type_id_fkey" (SQLSTATE 23503)
```

`Ledger.LedgerTypeID` is a plain `uuid.UUID`, so "no type" is the all-zero uuid, and the mapper wrote that straight into a foreign key. No such row exists and none ever will.

The column is nullable and 14 of the shop's own 785 ledgers carry no type, so leaving it blank is a state the data already has. It goes in as NULL now, which is what "none" means to a database.

What the user saw

```
Field validation failed
```

No field named, no reason given. A screen that refuses a customer and will not say which box is wrong is worse than one that refuses loudly.

It had never been tested

There are e2e tests for the colour master's create, its validation, its permissions — the codelist screens are well covered. The ledger master had none: it is built from a different constructor and was assumed to work because it exists. **The one screen every other screen depends on was the one nobody had driven.**

`TestLedgerMaster_CreatesACustomerWithNoLedgerType` and `TestLedgerMaster_CreatesACustomerWithALedgerType` now cover both, and the pair is deliberate: the second passed before the fix and the first did not, which is what identified the cause rather than the symptom. `TestLedgerMaster_RefusesALedgerWithNoCodeOrName` keeps the required fields required.

241. The same bug in five more masters — fifteen optional references — HIGH (fixed)

[Defect 240] was not one screen's mistake. The question it raised — which other optional references are written as the zero uuid — has one answer per master, and **every one of them was wrong the same way.**

item	subgroup_id, hsn_code_id	2
packet	stone_item_id, shape_id, colour_id, clarity_id, cut_id, sieve_id	6
party	category_id, main_category_id, source_id, religion_id, profession_id	5
retail point	branch_id	1
card commission	card_type_id	1

Fifteen optional references, all declared `uuid.UUID` rather than a pointer, all written straight into a nullable foreign key. "None" arrives as the all-zero uuid, which no row has and none ever will.

Proven on the one that matters most

A new **item** is what a new kind of piece needs before it can be tagged, so the item master gates the stock side the way the ledger master gates the money side. It refused:

```
insert or update on table "mcj_items" violates foreign key constraint ...
```

An item could not be created without naming an HSN code and a subgroup, both of which the shop leaves unset on its own rows. Fixed, and the same `nullUUID` in all five mappers with the reason written beside it.

What this class looks like from outside

Nothing about it is visible until somebody tries. Every one of these screens renders, lists, filters and searches; the failure is only on create, only when a field is left blank, and the message names no field. **A screen that works for everything except the thing it exists for is the hardest kind to notice**, and five of them shipped that way.

Why the sweep was possible at all

The question was answerable in one query — every nullable uuid foreign key in the schema — and then one grep per aggregate for the plain `uuid.UUID` return. Ten minutes for five defects, against the one that took an afternoon to find by hand.

[Defect 239]'s overstating tool sent me to look at the ledger master. That found 240, which asked the question that found this. A wrong number was more useful than a right one would have been, which is luck and not method — the method is that a defect found by hand is worth asking whether it has siblings.

242. The half-cast placeholder, swept and closed — LOW

[Defect 241] closed by asking whether a defect found by hand had siblings. The same question, asked of the other bug that has bitten twice.

A query parameter used more than once must be cast the same way everywhere: Postgres deduces a parameter's type from how it is used, so casting one occurrence and not the other is **worse than casting neither**, because it looks handled.

```
inconsistent types deduced for parameter $2 (SQLSTATE 42P08)
```

It failed that way in [defect 224] on `$4` and again in [defect 232] on `$2` — in the same file, a hundred lines apart, **the second time with a comment about the first sitting directly above it**. Knowing a rule and applying it are different things, which is the argument for checking rather than remembering.

The sweep found nothing, after finding thirty-one

The first pass reported **31** repeated placeholders. Every one was a false positive of two kinds:

Same-type repeats are fine. `WHERE tenant_id = $1 ... AND x.tenant_id = $1` needs no cast at all — both uses deduce the same type. Only a *mixed* state is wrong.

And several of these queries explain the rule in prose that mentions `$6`. Counting comment text as a use inflated the count. Stripping SQL comments first took it to **0 of 31**.

So the class is closed: both instances were the two already fixed, and nothing else in the codebase is cast inconsistently.

Kept as a test, not a memory

`TestSQL_ARepeatedParameterIsCastConsistently` walks every non-test, non-generated Go file. `TestSQL_TheCastRuleCatchesAMixedOne` proves the rule against text that breaks it — including the comment case — because a green test that cannot fail says nothing.

Proven against the real thing too: removing one `::text` from `tag.go` fails it by name and line.

```
tag.go: $2 is used more than once and cast on only some of them
```

A negative result worth a number

Nothing was wrong. That is the finding: the class has no more members, and the next one to appear fails a test rather than a customer's sale. **Three sweeps this session have gone this way — optional references (241, five defects), tables with no writer (231, four defects), and this one (nothing).** A sweep that finds nothing is the cheapest possible outcome and the only way to know.

243. Fourteen of seventeen master screens had never been driven — MEDIUM (fixed), and a correction to 241

[Defect 240] found that the ledger master could not create a customer, and [defect 241] found the same in four more. Both lived in the same blind spot: the screen had no test. So the next question was how big that spot is.

```
17 CRUD master screens
 3 tested  item, ledger (both added yesterday), party
14 never driven at all
```

Fourteen screens that render, list, filter and search, and which nothing had ever asked to save a row.

One test, eleven screens

Rather than fourteen files, one table-driven test: each master, the least it needs to save, and where to look for the row. It runs the one thing a master exists for.

```
HSN code · purity · state · branch · ledger group · sale rate
scheme · payroll component · retail point · card commission · packet
```

All eleven pass. It catches the class it was written for: putting back a single `nullUUID` in the packet mapper fails it immediately.

A correction to defect 241

Defect 241 said fifteen optional references were "all wrong the same way". Two of them were not, and driving the screens is what showed it:

```
a retail point must belong to a branch
card type is required
```

`retail_points.branch_id` and `card_commissions.card_type_id` are nullable columns that the DOMAIN requires. They could never have been left unset, so those two were never broken in practice. The `nullUUID` fix is harmless there and the reasoning behind it was wrong. **Thirteen were broken, not fifteen.**

That distinction only appears when you drive the screen. Reading the schema says the column is nullable; reading the aggregate says the field is a plain uuid; neither says the domain refuses a blank.

What the failures turned out to be

Of the five that failed first time, **three were my own test's fault** — wrong column names in two, and incomplete payloads in the others — and the three that looked like defects were business rules doing their job:

```
a scheme needs between 1 and 120 instalments
a retail point must belong to a branch
a commission has to say which card it is for
```

Recorded because "three master screens are broken" was the reading for about ten minutes, and it was wrong.

244. Editing a master, driven for the first time — nothing wrong

[Defect 243] closed the create blind spot on the master screens. Editing is the same blind spot one step further in: **a create test passes on a screen whose update path is broken**, and an update that fails is exactly how 27% of the shop's vouchers became uncorrectable ([defect 214]).

Nothing had ever driven one. Six masters are now exercised end to end — create, then change a field, then read it back:

HSN code	description
purity	fineness
state	abbreviation
branch	code
sale rate	name
packet	remarks

All six edit cleanly. No defect.

Proven not to be vacuous

A test that only ever passes proves nothing, and this one asserts a value came back changed — which would hold if the update quietly did nothing and the value happened to match. So it was pointed at a column the form never sends:

```
state: kind came back "state" after editing it to "NEVER-SENT"
```

The assertion is live: an update that does not reach the column fails it.

Two negatives in a row, and what that means

[Defect 242] swept the half-cast placeholders and found none. This drove every master's edit and found none.

Two consecutive sweeps that found nothing, after three that found nine defects between them.

That is the shape a search should have when it is working: the classes with members get emptied, and the ones without get closed. A sweep is worth running precisely because you cannot tell which kind it is beforehand — and the ones that find nothing cost an hour and buy the right to stop looking there.

245. Every write route checked for a permission, and the pairing that keeps two of them safe — MEDIUM (fixed)

Three screens are checked by name for permission holes. Nothing asked the question of all of them, which matters because an ungated write is not hidden — `CrudController.accessDenied` reads `if perm == nil { return false }`, so an operation with no permission set is **open to any authenticated user**. On this system that means any member of staff could raise a bill, set the day's gold rate or hand over somebody's order.

Two checks now ask it across the board:

```
15 of 15 hand-written write routes are behind a permission
18 CRUD screens: every operation is either permitted or removed
```

Both pass. The finding is not a hole; it is what keeps two screens out of one.

The pairing nobody was checking

```
company.go    omits WithDeletePermission, calls WithoutDelete
tagprint.go   omits Create and Delete permissions, calls WithoutCreate and WithoutDelete
```

Both are correct, and both are **one deleted line from being open to anybody**. The safety comes from the `Without` call unregistering the route, not from the missing permission — and `tagprint`'s own test comment says so: *"remove the WithoutCreate and the screen is open to anybody."* Nothing paired the two, so either could be removed as tidying-up and the tests would stay green.

The rule is now that each operation must be **permitted or removed, never merely absent**.

The check was blind to comments, and only breaking it showed that

The first version passed when `WithoutDelete` was commented out — the string is still in the file. A screen whose delete had just been opened read as safe.

```
company.go: neither WithDeletePermission nor WithoutDelete –
the operation is registered with no permission
```

That is the same comment-blindness as [defect 242], where prose mentioning \$6 was counted as a use and reported thirty-one problems that were not there. Twice in two days, in checks written a day apart. Comments are stripped in both now.

It is also the argument for breaking every guard deliberately: this one passed, looked right, and would have kept passing through the exact change it exists to catch. Both are proven against a real break — a commented-out `WithoutDelete`, and a `gate(...)` removed from the metal rate screen.

246. My own checkers were blind to comments — MEDIUM (fixed)

Comment-blindness has now appeared three times in three days:

```
242 prose mentioning $6 counted as a use, reporting 31 problems that were not there
245 a commented-out WithoutDelete counted as present, passing a screen whose delete had
just been opened
246 this
```

Twice was a coincidence. Three times is a class, so the question was asked of every check that reads source text.

```
tenant_scoping_test.go    strips
placeholder_casts_test.go strips
check_unwritten_tables.py DID NOT
check_empty_columns.py   DID NOT
```

Proven, not assumed

`check_unwritten_tables.py` decides whether a register has a writer by looking for `INSERT INTO mcj_x`. One line appended to a controller —

```
// A dead thought: INSERT INTO mcj_holidays (tenant_id) VALUES ($1)
```

— and `mcj_holidays` **dropped off the gap list entirely**. A comment was enough to make the tool report that a table is written when nothing writes it.

That is the worst direction for this particular tool to be wrong in. It is the one that found four defects by naming registers with readers and no writer, and [defect 239] already had it overstating by a third in the other direction. A tool that can be wrong both ways, quietly, is worse than no tool.

Both now strip line comments before matching, and both were proven the whole way round: the same comment changes nothing, and a real `INSERT` in live code still counts as a writer.

```
holidays listed before = 1, with the comment = 1    (ignored)
with real code = 0                                     (counted)
```

The rule, written where it will be read

The helper carries the reason rather than the mechanism:

```
A commented-out writer is not a writer, and a table named in prose is not a
read. [...] three times in three days, which is why it is a function and not
a habit.
```

// preceded by `:` is left alone, so a URL is not truncated mid-line.

Worth saying plainly

Four sweeps this session were run with these tools, and their numbers went into four defect write-ups. **The numbers happen to be unaffected** — no controller in this codebase has a commented-out INSERT, which is why the counts did not move when the bug was fixed. That is luck, and it is the second time this week a tool's output has been believed before it was tested.

247. All seventeen master screens driven, and the last unproven tool proven — nothing wrong

[Defect 243] drove eleven master screens and [defect 244] their edits. Three were out of reach of that table and were left: **company** is a singleton from its own constructor, the two **bands** come from a definition list rather than a path constant, and **tag print setup** has no create at all.

All three are now driven.

company	saves the shop's own particulars
value band	saves
weight band	saves
tag print	404 – WithoutCreate, refused by design

Sixteen of seventeen create a row; the seventeenth proves it refuses because the route is absent rather than merely ungated, which is the distinction [defect 245] was about. Every master screen in the system has now been asked to do the thing it exists for.

The last tool without a proof

Five of the six checks had been shown to catch something real. `check_empty_columns.py` had not — it reported 31 columns and nothing had ever confirmed it would notice one *stopping* being empty, which is the direction that matters when data starts arriving.

return_note listed as empty	1
after filling one row of seven	0
reverted	1

It notices. The row was put back.

Where that leaves the checks

cmd/reconcile	proven – a bill moved by ₹1 is caught
check_invariants.py	proven – three planted faults, each named
check_unwritten_tables.py	proven – a comment no longer hides a gap, real code still counts
check_empty_columns.py	proven – a single filled row drops a column off
check_report_footers.py	proven when written – defect 189
status_figures.py	proven repeatedly – it has caught every stale count this week

Every check that guards this system has now been shown to fail when it should. That is worth stating because three of the last eight defects were in the checks themselves, not the product, and a guard nobody has broken on purpose is a guess about its own behaviour.

248. A re-tagged piece could not name where it came from, and defect 226 was wrong about why

Defect 226 gave the tagging screen its arrival-document field, offering the right series for each source: a purchase bill for `purc`, an old-gold bill for `og`, a sale return for `ret`. For `stk` -- a piece re-tagged out of existing stock, and the largest group by far -- it offered nothing, and I wrote the reason into the code as a fact:

```
an STK number, which is the shop's own counter with no document behind it
```

That was wrong. There is a document. `mcj_stock_conversions` holds 859 of them, STK/00001 to STK/00859, each a record of stock being melted or remade, with 1,242 pieces itemised in `mcj_stock_conversion_pieces`. It was sitting in the unwritten-tables list the whole time, which is where I found it.

Why the wrong reason looked right

tags carrying an STK number	2,401	
distinct STK numbers referenced	143	
conversion documents captured	859	(STK/00001 – STK/00859)
of the 143 that resolve	48	<– first written as 55; see defect 249

Only 48 of 143 STK references match a captured conversion. The other 95 run up to STK/15863 and belong to years the conversion history does not reach -- the same current-year-only window as the invoices in defect 238. Checking a handful of the shop's recent tags against the conversion table would have found nothing most of the time, and I took that for absence of the series rather than absence of the older half of it.

(The count in this section first read 55. Seven of those were older documents that happen to carry the same number -- the STK series restarts. Corrected here and explained in defect 249.)

Fixed

`arrivalDocuments` now has a `stk` case offering the conversion vouchers, newest first, each labelled with its date. The comment that asserted the opposite has been replaced with what is actually true, including why it looked otherwise.

Driven end to end -- a piece re-tagged against STK/00859:

POST /mcj/tag action=save source=stk came_on=STK/00859	303
mcj_stock_items.voucher_no	STK/00859
mcj_stock_movements	NEW-BAR

The test row was removed; tags 2,816, movements 9,036, conversions 859, all exact.

What this does not do

It offers the 859 documents that exist. A piece re-tagged out of stock that predates the captured window still has nothing to name, and the field stays empty -- correctly, because the document it would name was never migrated. The conversion screen itself is still unbuilt; this defect makes the conversions *referenceable*, not *enterable*. That remains open.

Method note

Two of the last three defects came from the unwritten-tables list, and this one also corrected a mistaken explanation I had committed to a comment. A table nobody writes is worth reading before concluding what it means -- and an explanation of an absence deserves the same second source as a figure does (see the standing rule on verifying numbers). I asserted this one from a sample and was wrong on 48 counts.

249. What the conversion flag means, settled for half of it — and the STK number is not unique

*THE CENTRAL CLAIM BELOW IS WRONG AND IS RETRACTED. See defect 250. The "gold is conserved" finding compared a document's listed pieces against the tags naming that document, and those are **the same pieces** -- 47 of 48 carry the same barcode on both sides. The weights agreed because it was one set of gold weighed twice. What survives, and what replaces it, is in defect 250. The section on the STK number not being unique is unaffected and still holds.*

Building the conversion screen means knowing what a conversion is. The migration that created the table said, deliberately, that it could not be known:

THE DIRECTION FLAG IS CARRIED UNINTERPRETED, and that is deliberate.
... two readings, one contradicting the column name and one contradicting the data, and nothing in the capture settles it.

That was right when it was written. It is no longer right, because defect 248 built a link that did not exist then: a tag naming the document it arrived on. That gives the output side of a conversion, which the conversion table never had.

Direction 0 is a melt-and-remake, and the gold is conserved [RETRACTED -- see defect 250]

Take the documents where both sides are captured -- the pieces listed on the document, and the tags naming that document as their arrival:

direction 0	51 docs	422 pieces in	2,283.888 g	->	348 tags out	2,294.940 g
direction 1	4 docs	4 pieces in	34.720 g	->	25 tags out	2.480 g

Aggregates are weak evidence, so per document:

direction 0	32 of 51 balance to within 50 mg
direction 1	0 of 4

32 documents where what went in and what came out agree to a twentieth of a gram. That is not a coincidence of totals; it is the same gold, weighed twice. So a direction-0 document is a melt-and-remake: the listed pieces are consumed, the tags naming it are produced.

Note this is the opposite of what the column name says. The legacy column is `add`, and 0 is the value that adds. The migration flagged that contradiction and declined to resolve it; the weights resolve it.

Direction 1 is still not settled, and I am not going to guess

519 documents, every one a single piece
only 4 of 519 ever produced a tag
375 of 519 name a barcode with no tag on file
average net 14.767 g against 7.141 g for direction 0
heavily silver -- SILVER MURTI, SILVER MUKUT -- and
some carry net weight 0 against a piece count of 3

Four documents is not evidence, and none of them balances. The screen will show direction 1 as recorded, without a label, exactly as the migration decided. **Half the flag is now known; the other half is not, and saying so is the point.**

The STK number is not unique, which makes defect 248's figure wrong

The join that produced all of the above matches a tag to a conversion by voucher number. That is unsafe:

the 32 that balance every tag created 2026-04-04 .. 2026-08-18, inside the window
the 19 that do not tags reaching back to 2024-04-03
 3 documents have tags created BEFORE the document existed

A tag cannot arrive on a document that does not exist yet. **The STK series restarts, and those 19 are matching an older year's document carrying the same number.**

STK numbers on tags that resolve, by number alone 55
... that survive a date check 48

Defect 248 said 55. The honest figure is 48, and it has been corrected there and in the code comment. The fix itself is unaffected: a tag being written today can only mean the document of that number that exists today. It is reading the *history* that is ambiguous, not writing the present.

What this changes about the screen

It has to be built knowing three things that were not known this morning: a direction-0 document consumes its listed pieces and produces tags, a direction-1 document is unexplained and must not be labelled, and the voucher number is not a key across years. Building it before establishing these would have baked in a guess about all three.

Method note

Defect 248 corrected a false explanation. This one corrects the number in that same correction, one iteration later, and both were found the same way -- by asking the data a second question rather than accepting the first answer. **The rule about verifying a figure against a second source applies to figures written while fixing a figure.**

250. I proved the same gold weighs the same as itself, and called it conservation

Defect 249, written one iteration ago, claimed the direction flag was settled for direction 0: a melt-and-remake, with the pieces listed on a document going in and the tags naming that document coming out, balancing to within 50 mg on 32 documents. **That is wrong.** The two sides are the same pieces.

listed pieces on the 32 balancing documents	48
that carry the same barcode as a tag naming the doc	47

Forty-seven of forty-eight. The weights agreed because I weighed one set of gold twice and subtracted it from itself. A balance that cannot fail is not evidence, and I presented it as the strongest evidence in the write-up.

What I should have asked, and didn't

The check that breaks it is one line -- *are these the same barcodes?* -- and I had every reason to run it. Defect 29 records that this shop reuses barcodes. Defect 249's own second half is about matching on an identifier that turns out not to be unique. I ran that check on the voucher number, found the collision, corrected defect 248's figure, and wrote it up as a lesson about verifying identifiers -- **without applying it to the barcode in the very same query.**

There was also a signal in plain view that I explained away: 346 of the 486 direction-0 listed pieces are still on the shelf today. I noticed it, attributed it to barcode reuse, and moved on. Pieces that were melted are not on the shelf. The data was saying the pieces were never consumed, and I had a story ready for why it didn't count.

What actually holds

Redone with the date restriction defect 249 established, so no cross-year collisions:

documents with tags naming them	48	all direction 0
tags naming them	317	
of those tags, also listed as a		
piece on the same document	317	every one
direction-1 documents with tags	0	

So a direction-0 conversion document is **a tagging source**: the pieces listed on it are the pieces tagged against it, in every one of 317 cases. It brings stock in. That is the same role a purchase bill plays, which is exactly what defect 248's fix assumed, so **the fix stands and this changes nothing about it.**

Direction 1 has **no tags at all** once dates are respected -- the 4 documents and 25 tags reported in defect 249 were every one a cross-year collision.

What is consistent, and is not being claimed as proven

direction 1	519 docs, 519 pieces, one per document, average net 14.767 g
direction 0	340 docs, 723 pieces, 2.13 per document, average net 7.141 g

One heavy piece out, several lighter finished pieces in, is what melting and remaking looks like, and it fits the shape of both halves. It is **not proven**: no document links a direction-1 piece to a direction-0 one, and the totals do not balance either (7,664.013 g against 5,163.068 g). It is written here as a shape the data is consistent with, and the screen will not label the flag on the strength of it.

The reading does contradict the legacy column name -- `add`, where 1 would be the value that adds -- which is the contradiction the original migration flagged and declined to resolve. **It was right to decline. I was not right to resolve it.**

Consequence for the screen

The conversion screen cannot be built as "choose pieces to melt", because the captured data contains no input side. What a direction-o document actually is, on the evidence, is the document a batch of new tags is raised against. That is buildable and useful; the melting half is not, on this data.

Method note

Three iterations, three corrections: defect 248 corrected a false explanation, 249 corrected 248's number, and 250 retracts 249's headline. The pattern is not that checking is working -- it is that **I keep publishing the first coherent story and checking it afterwards**. 249 even contained the words "aggregates are weak evidence, so per document" immediately before presenting a per-document figure that could not fail. Rigour applied to the wrong question reads exactly like rigour.

251. The melt hypothesis, tested before it was written up, and refused

Defect 250 ended with a shape the data was "consistent with" -- one heavy piece out under flag 1, several lighter pieces in under flag 0 -- and said explicitly it was not being claimed. This entry tests it, before writing anything down as true, because the last three entries each published a story and checked it afterwards.

The test, chosen so it could fail

Defect 250 established that 317 of the pieces listed on the 48 date-safe direction-o documents are the tags raised against those documents. They are not all of them:

listed pieces on those 48 documents	419
that are tags raised against the doc	317
that are not	102

If a conversion melts pieces down and remakes them, the 102 are the input and the 317 are the output, and on a document carrying both the two should weigh about the same. 17 documents carry both. That is the test.

documents with pieces on both sides	17	
balancing to within 50 mg	0	
balancing to within half a gram	0	
totals	1,067.346 g tagged	783.028 g untagged

Zero of seventeen. The hypothesis makes a prediction, the prediction is checkable, and it is wrong. Written down as a refusal rather than a finding, which is what it is.

What the 102 are, as far as the data says

name a barcode with no tag on file anywhere	71
name a tag that exists	31
of those, a tag predating the document	19
of those, a tag no longer on the shelf	8
blank barcode	0

Not explicable from what was captured. The migration already noted that 612 of all 1,242 pieces name a barcode with no tag on file, so the 71 are the same phenomenon and not new information.

The decision this forces

The conversion screen is not being built, and that is the outcome of this entry rather than a postponement. An entry screen has to know what a document does to stock. Three iterations have produced three readings of that -- a melt-and-remake (retracted), a tagging source (true of 317 of 419 pieces), and now a mixed document whose remainder resists explanation. The Stock Journal report already shows all 859 of them, so the shop can read its own history; what is missing is the ability to write one, and **writing one requires deciding what the other 102 pieces mean, which the capture does not support.**

Building it anyway would mean choosing a meaning and then teaching the shop to enter data in that shape, which is how a wrong reading becomes permanent.

What was built instead

A fourteenth invariant, guarding the one thing that is established:

a tag naming a conversion is listed on it	317 of 317, 0 exceptions
---	--------------------------

Proven not to be vacuous -- a planted tag naming STK/00859 without a matching piece is caught, 13 of 14 rules hold, and removing it restores 14 of 14. The rule checks dates, because without that restriction it reports **56 exceptions** that are only the STK series restarting (defect 249). A rule that fails on history gets switched off, which is the point of the allowance column.

Method note

Four iterations on one table, three of them corrections. The difference in this one is only the order: the check that would have refuted the claim was run before the claim was written, not after. That ordering is the whole of the lesson from defects 248, 249 and 250, and it cost nothing here -- the query was no harder to write today than it would have been tomorrow, after it had been published as fact.

252. Three master screens could delete themselves out from under live data

Defects 243 and 244 drove create and edit on every master. Delete was the one operation never driven, and the only one that can destroy something. Driven now, in two parts: can a master be deleted at all, and does it stop when the row is in use.

The first is fine -- all eleven masters in the table delete cleanly. The second was not.

mcj_schemes	-> mcj_scheme_enrolments.scheme_id	239 rows
mcj_payroll_components	-> mcj_staff_components.component_id	46 rows
mcj_packets	-> mcj_stock_stones.packet_id	107 rows

Each foreign key was `ON DELETE SET NULL`. The delete came back **303 -- see other**, the ordinary success redirect, and the rows that pointed at the master survived pointing at nothing. No error, no warning, nothing on the screen to say anything had happened to them.

Deleting the shop's one scheme would have left **239 customers enrolled in nothing**, with no way afterwards to say what any of them had been paying into. The scheme collection screen is still blocked on a question to the owner (defect 233), so those enrolments are the only record of the arrangement that exists.

Fixed

`migrations/mcj-1787411000.sql` changes the three to `ON DELETE RESTRICT`. The delete is now refused by the database.

The columns stay `NULLABLE`, which is a different question and was decided on evidence: 134 of the 239 enrolments name a ledger that was never in the address book, and 5 tags carry a purity with no fineness anywhere (blocker B2). Those rows must be storable. **What no migration argues for anywhere is a master vacating the column on its way out** -- the delete action was carried along with the nullability rather than chosen.

The eleven left alone deliberately

Fourteen masters have a `SET NULL` reference with no `RESTRICT` anywhere, including `mcj_vouchers`, whose deletion would detach sales invoices, old gold lines and purchase invoices at once. Eleven of the fourteen have **no delete route registered anywhere in the application**, so nothing can reach them. They are left as they are: a constraint added for a screen that does not exist protects nothing, and would have to be revisited when the real screen decides its own rule. The list is in this entry so the next person building one of those screens finds it.

The test passed against the bug first

The first version rolled back to a savepoint before checking the state, because a refused delete aborts the transaction. That rollback also undid a *successful* delete, so the test passed on the unfixed schema for all three masters. It was caught by asking the question this doc keeps having to ask -- does this fail when it should? -- and the answer was no.

Now the status is read before the rollback, and a delete that comes back under 400 fails on the spot. Verified by weakening the migration's own Up to `SET NULL` and running it again:

```
scheme:           deleting a master that is in use came back 303
payroll component: deleting a master that is in use came back 303
packet:           deleting a master that is in use came back 303
```

All three fail without the fix and pass with it.

What the refusal looks like, which is not good

The database raises a foreign-key violation and the CRUD builder has no handler for it, so the user gets an error rather than a sentence explaining that the scheme has 239 people in it. **The data is safe and the message is poor**, which is the right way round, but it is not finished. Logged here rather than fixed, because the fix belongs in the shared CRUD builder and touches every master screen at once.

Method note

Third time this week a checker or test has been the defect rather than the product (242, 245, 246, and the savepoint here). The pattern in all four is the same: the check was written, it went green, and green was taken as evidence. **A test's first green run against code that is known to be broken is the only cheap chance to find out whether it tests anything.**

253. The refusal reached the server log and nothing else

Defect 252 stopped three masters being deleted while in use, and said the message was poor and left it. This is that half.

The repository's message could never have reached anybody

The obvious place to explain a refusal is where it is detected, so the tenant scoping decorator now turns a foreign-key violation into a sentence:

```
this cannot be deleted while scheme enrolments still refer to it
```

Driven, it changed nothing on screen. The CRUD controller every master screen is built from does this:

```
if _, err := c.service.Delete(ctx, idFieldValue); err != nil {
    log.Printf("[CrudController.Delete] Failed to delete entity %s: %v", id, err)
    errorMsg, _ := c.localize(ctx, errFailedToDelete, "Failed to delete data")
    http.Error(w, errorMsg, http.StatusInternalServerError)
```

The error is logged and then discarded, and a fixed string is sent instead. No message written in a repository, a service or a domain object can reach a user through this path. That is worth knowing beyond this defect: it is where every master screen's delete errors go.

What was done instead

`Errors.FailedToDelete` is a translation key, and the mcj bundle overrides it:

```
This could not be deleted. It is most likely still in use by other records --
a scheme with people enrolled, a packet holding stones, or a payroll
component on a salary. Detach those first, then delete it.
```

Proven to be the thing doing the work: with the key removed the test fails with `Failed to delete data`, and with it restored all three masters pass. Full suite green, 18 packages.

What this is not

It is **generic**. It cannot say *239 people are enrolled in this scheme*, because the string is chosen before the error is looked at -- the SDK localises a constant, not the failure. It names the three things it is most likely to be, which is a guess printed in advance, and it will read oddly on a master that fails for some fourth reason.

The honest fix is a delete handler of our own that counts the references and says the number. That means intercepting the route ahead of the CRUD controller for all eighteen generated screens, and it was not done here: the risk of rerouting every master screen outweighs a better sentence on an action that is now safe.

Recorded as the known next step rather than left as a vague intention.

The repository translation is kept even though no user sees it. The SDK logs the error it discards, so the sentence lands in the server log, where the shape of the failure is otherwise a bare SQLSTATE.

Method note

The first version of this fix was written, built, and would have been reported as done on the strength of it compiling. It took driving the screen to find that the message went nowhere. **Three of this week's defects were checks that passed without checking; this is the same error one layer up -- a fix that worked everywhere except where anyone could see it.**

254. The one unexplained test failure, and the guess about it that could not have been right

TestCategorise_FiltersToUntaggedOnly failed once under the full suite on 22 August and has not recurred. STATUS.md carried it under "Known unknowns" with a hypothesis:

```
The screen carries a `LIMIT 200`, which is the kind of thing that only
breaks once something leaves enough rows behind
```

That mechanism is impossible, and the harness says so plainly:

```
Parallel harnesses (separate test binaries, and t.Parallel within one)
each provision their OWN per-test database
```

Every suite runs against a database of its own, created empty and migrated. No other test can leave a row behind for this one to trip over, so the count the LIMIT applies to is whatever this test seeded -- three rows. The guess was written without reading how the harness isolates, and it sent the question in a direction it could never have come back from.

Which also means something better

Tests do not run against the shop's data at all. Every baseline check in this document that was taken after a test run -- 2,816 tags, 239 enrolments, 859 conversions -- was measuring a database the tests cannot reach. That is the right answer, and it was being taken on faith.

It also means an experiment run yesterday was meaningless: three constraints were altered on the live database to see whether a test would catch it, and the test never saw them. That was noticed at the time and the migration file was flipped instead, which is what actually proved the point.

Not reproduced

```
43 runs of TestCategorise, including -count=40 in one process    0 failures
```

Not reproduced is not the same as not there, so nothing is being closed.

What was done instead of guessing again

The seed's lookups were scoped to their tenant. Three of them read across the whole table:

```
SELECT id FROM mcj_categories WHERE name='RINGS'
```

With per-test databases this is safe today. It is not safe by design -- it returns whichever row the planner reaches first, and any change that shares a database between suites turns it into a test that categorises its tags against another tenant's category. Fixed because it is a hazard, not because it is the cause.

The failure now records why it failed. The one thing the 22 August run left behind was a sentence with no facts in it -- not the status code, not what the tenant held, not even which of the two assertions fired. Proven by forcing it:

```
an untagged tag is missing from the untagged list.  
status          200  
body            518304 bytes  
seeded TAG%     3  
untagged rows   2    (the screen caps its list at 200)  
rows in tenant  3
```

If it happens again, the run will say whether the list was short, the request failed, or the rows were never seeded.

Method note

This is the second time in two days that a written-down explanation turned out to be the defect rather than a description of one (defect 248 was the first). Both were plausible, both went unchecked because they explained the symptom, and **both were contradicted by a comment in the code they were about**.

255. Nineteen tags were refused with an instruction the screen would not let anyone follow

This started as a suspicion that STATUS.md was stale. `blockers.md` marks B9 **ANSWERED**; STATUS.md still lists it twice, once under "What it cannot do" as *Bill the 165 items whose stored price will not reproduce*. One of the two had to be wrong.

STATUS was right. The sale screen refuses these tags, and refuses them for a good reason -- the formula reproduces 91% of computed prices, and billing a figure it cannot stand behind would be worse than refusing. The message it gave:

```
the formula does not reproduce this tag's recorded price  
(38,760.00 against 42,140.00) – price it by hand
```

There was nowhere to price it by hand. The box beside a refusal is attached in one place only:

```
if making.certified {  
    refusal.PriceField = agreedPriceField + barcode
```

A certified piece gets a box. A tag the formula cannot reproduce got the instruction and no box, on the same screen, three lines apart. Nothing else on the sale screen prices a tagged piece either -- the making inputs belong to priced rows, and a refused tag has no row. The only route left was to bill it as an untagged item, retyping a piece that is already on the shelf and leaving the tag on it.

Measured across the whole shelf

Every one of the 2,575 in-stock tags was put through the screen, in batches:

certified, box offered	565
cannot reproduce, no box	19
scanned twice (B8's duplicate)	1

19 tags, ₹26,89,060.47, could not be sold by any route the screen offers. Not the 165 B9 counts -- that figure covers all items including those long sold; 19 is what is on the shelf today.

Fixed

A tag that fails the reproduction check now returns `handPriced`, which offers the same box a certified piece gets, seeded with the tag's own total. Driven end to end on the real barcode:

```
93150410 refused, box agreed_93150410 offered, suggested 42,140.00
         agreed 42,140.00 entered -> priced row, fixed price, 42,140.00
```

The wording changed with it, from "price it by hand" to "enter the price agreed with the customer", which is what the biller is actually doing and matches the certified case.

Regression test added, and **proven to fail without the fix**: with the box condition put back to `certified` only, it reports *refused with no way to price it — the instruction cannot be followed*.

Two things I got wrong on the way

Read absence of refusals as success. The first sweep reported "all 60 priced". Nothing had priced: the POST was rejected with "*Choose a customer.*" before pricing ran, and the barcodes I found in the response were the textarea echoing my own input back. Checking that a positive marker was present, not that a negative one was absent, is what caught it.

Nearly filed rate movement as a defect. Comparing computed prices against stored ones for 200 tags showed 107 disagreeing, which looks alarming until you notice the screen prices at *today's* metal rate and the stored total was set at the tag's creation-date rate. The self-check in the code gets this right -- it recalculates `atTagRate` before comparing. I had to read it to find that out, having already measured the wrong thing.

Still not billable

Barcode **86065435**, refused as *scanned twice*, because two tags share it. That is blocker B8 and needs the shop to re-tag one of the two pieces. Correct behaviour, unchanged.

256. The quote screen spent a fortnight telling customers the opposite of what the sale screen had learned

Defect 255 fixed a refusal that gave no way forward, so the obvious next question is whether that refusal has siblings. It does, and the sibling is worse: it is not a missing box, it is a **statement the project has already disproved**.

`quote.go` refused every certified tag with this:

This tag is certified, and certified goods carry a pricing step that did not survive the capture. Quoting it would risk being wrong by a large amount rather than a small one. See blocker B9.

There is no such step. Defect 228 established it from the shop's own bills: of the 129 certified pieces sold, ONE went out at the tag's stored price and NONE at a computed one -- 0.44 to 3.47 times the tag, with 125 of the 129 lines carrying a different metal rate. The price is agreed across the counter, the way old gold and coin orders are. `blockers.md` has said **ANSWERED** since.

Defect 229 changed the sale screen accordingly. The quote screen was not changed, and went on citing an answered blocker as an open one.

certified tags on the shelf	566
value	2,84,75,794.50

22% of the shelf by count, 20% by value, could not be quoted -- the figures from defect 161, re-measured today and identical. Defect 161 saw this and recorded it as correct behaviour: *"Refusing is right while the pricing step is unknown."* It was right when written. Two days later the step was known, and nothing went back to the screen that assumed otherwise.

Fixed

A quote is a read-only report, so there is no box to type an agreed price into and nothing has been invented. What it can do is give the counter what it needs to open the conversation:

DIAMOND BANGLE	barcode 53710364 · on the shelf		
Priced at the counter	This piece is certified. Its price is agreed with the customer rather than computed...		
The tag's own price	what the counter starts from	15,97,755.00	
Certificate charge	billed separately and NOT added	5,145.00	

Driven on the exact barcode defect 161 used to demonstrate the refusal.

The same treatment went to the tag-cannot-be-reproduced refusal, which ended "Price it by hand" on a screen with no way to price anything. It keeps its explanation and both figures, because the size of the gap is what lets somebody judge it, and it now also carries the tag's own price as the figure to work from.

The shape of this defect

Neither screen was wrong in isolation. The sale screen learned something, the blocker doc recorded it, and **the third place that had encoded the old belief was never revisited** -- and it was encoded as a sentence shown to a customer, not as a comment. Defects 248 and 254 were the same shape in comments and in `STATUS.md`.

What makes this one worse is that the wrong belief was *cited*: the message pointed at blocker B9 by name, which reads as a claim that has been checked. **A reference to a blocker is a claim about the blocker's current state, and nothing checks those.** There is no tool that finds a screen citing an answered blocker; that is a gap this defect does not close.

The gap this closed

The write-up above ended by naming a gap: *"A reference to a blocker is a claim about the blocker's current state, and nothing checks those."* Leaving that as a sentence would have been the same mistake one layer up, so it is now a tool.

`tools/check_blocker_citations.py` reads every blocker heading in `blockers.md`, then scans the Go and templ sources for **string literals** naming one. A blocker that still wants something -- OPEN, CRITICAL, SECURITY, TIME-SENSITIVE, NARROWED -- may be cited freely. One that has been settled -- ANSWERED, RESOLVED, CLOSED, WORKED AROUND -- may not.

Comments are stripped first and are never flagged. A comment saying "B9 is answered, see defect 228" is history and is exactly right; a sentence shown to a customer is a live claim. That distinction is the whole tool, and getting it backwards is what defect 246 did three times running.

It found one more on its first run, which the sweep by hand had missed:

```
modules/mcj/presentation/reports/ratio.go
    "the shop's purchase cost is not recorded against a tag anywhere
    (blocker B9 and defect 79)"
```

Not merely stale -- **the wrong blocker**. B9 is about 165 prices that will not reproduce; cost of goods sold is defect 79's subject and nothing to do with it. A reader following that citation would have found an answered blocker about a different question. The reference is gone; defect 79 stays, because that part is right.

Proven three ways rather than assumed:

a live string citing B20 (RESOLVED)	caught
the same words in a comment	ignored
B3 (NARROWED) on the label screen	allowed -- it asks for exactly what B3 still wants

That third one was a real false positive on the first version, which counted NARROWED as settled. The label screen tells the user to compare a printed label against a real tag, which is precisely what B3 is narrowed to. **A checker that flags a message doing its job teaches people to ignore it.**

257. A day's trade, run end to end for the first time by a test — and the bill it raised did not add up

The goal this work is measured against is *run a real day's trade end to end*. Every screen in that sequence has its own tests. **Nothing had ever run them in order against one shop.** The sequence was rehearsed by hand twice and then existed only as a memory of those two afternoons.

`daystrade_e2e_test.go` runs it: set the morning's gold rate, tag a piece against it, sell that piece, take the cash, and check the books. Three screens chained, which is where the failures this file can catch actually live - a tag priced at yesterday's rate, a bill that cannot find a piece the tag screen just made, a voucher that balances alone but not against the stock that left.

It found one on the first run.

The header did not agree with its own lines

tag DAY001	12 g at the day's 7,000 = 84,000 metal	
	plus 200/g making	= 2,400
	tag total	86,400
bill line	amount 86,400	making 2,400 correct
bill header	goods_amount 84,000	short by the making
SUM(line.amount)	86,400	out by 2,400
bill_total	88,992.00	correct

The customer is charged correctly -- the taxable value and every ledger posting use the line totals, and 86,400 at 3% is 88,992. What was wrong is the header column, which was adding up `price.Amount` (metal and stones) and leaving the making charge out.

It contradicts the shop's own convention, measured

captured bills	224
where goods_amount = SUM(line.amount)	224
lines carrying a making charge	279 of 367, 14,35,896.00

On the shop's own bills the making charge sits **inside** `goods_amount` and `making_charge` is the memo saying how much of it was making. GST/2627/00001 is goods 49,709.00 with making 4,281.00 within it, not beside it. Every one of the 224 agrees.

So this is not a matter of taste. **Invariant rule 1 -- "every bill's lines add up to its goods" -- would have failed on every bill this screen raised carrying a making charge**, and the rule is right; the screen was wrong.

Why nothing caught it

Only three tests read `goods_amount` from the database, and two of them bill a **fixed-price** piece -- a certified one and a hand-priced one -- where the whole price is one figure and there is no separable making to lose. The third seeds an invoice directly. **The one case that could expose it, an ordinary tag with a making charge billed through the screen, had no test at all.**

The screen tests do check making: `TestSale_PricesAtTodaysRateWithLocalTax` looks for "63,000.00" on the rendered page, and the page was always right. The page is not the books.

Fixed

`goods` now accumulates `l.price.Total`. The lines, the taxable value, the ledger postings and the customer's total are unchanged -- only the header column moves, from 84,000 to 86,400, which is what the shop writes.

Proven: with the old line put back the day's-trade test reports *the bill's lines do not add up to its goods -- out by 2400.00*, and restoring it passes. Full suite green, 18 packages.

What the test does not yet cover

Old gold. The day it runs is a rate, a tag, a sale and a receipt; taking metal in over the counter is a fourth screen and a second voucher, and it needs its own seed. Stated rather than implied, because a test named for a day's trade invites the assumption that it covers the whole day.

The old gold half, added — and nothing wrong with it

Defect 257 ended by naming what the day's-trade test did not cover: taking metal in over the counter, a fourth screen and a second voucher. That is now in the same test, on the same day, against the same shop.

afternoon	10.000 g gross, 1.000 less, tunch 91.6, agreed 1,20,000
booked	one lot, 9.000 g net, 1,20,000.00
vouchers	2 raised, both balancing to the paisa

Nothing was wrong. A sale and a purchase on one day was the case worth checking -- a shared voucher number, or a ledger one posting expects to find as the other left it -- and both postings are clean and separate.

Recorded because a negative result from a test written to find something is worth as much as the entry above it, and because the next person to wonder whether the two halves of a counter day interfere should not have to find out again.

A trap removed from the test itself

Proving the new assertions were not vacuous produced this:

```
the lot's net weight is 9.000, want 9.000
```

The wanted figure was written twice -- once in the comparison, once in the message -- so changing the comparison alone left the message asserting that nothing was wrong. In normal running the two agree and it reads correctly, which is exactly why it would survive. It is named once now and used for both.

A failure message that can disagree with its own test is the same shape as the savepoint in defect 252 and the three comment-blind checkers in defect 246: the check works, and what it reports about itself does not.

258. The report that asks how old the money is aged the shop's oldest stock as its newest

Found while chaining sell → refund → sell again (below, and nothing wrong with that). A comment in tag.go says last_received_date differs from created_date "only where a piece went out and came back". Checking that led somewhere else.

tags where last_received_date > created_date	2
of those, carrying a SALE-RET movement	0
returned barcodes with a tag on file	10
of those, last_received_date updated on return	0

The comment's reason is not supported. One of the two went out to hallmarking and came back (ISS-HM, REC-HM); the other has no round trip at all. And a return does not touch the field -- which is correct, because the legacy system did not either, on none of its 10 returned tags.

But the arithmetic left 120 tags unaccounted for, and those are the finding.

created_date is not when the piece arrived

tags where last_received_date is EARLIER	120
all marked is_opening	120
created on 2024-04-01, a financial-year start	119
their real receipt dates run back to	2017-03-26

For opening stock the two dates answer different questions. created_date is the day somebody keyed the shelf into the books; last_received_date is when the piece actually arrived.

openingstock.go grouped by extract(year from created_date) , under a comment saying "the question is how old the money is", and the whole report is filtered to is_opening -- so this was not an edge case in it, it was the case.

	by created_date	by arrival
three years or older	260 tags	377 tags
	14,40,284.59	74,48,288.10

117 tags worth 60,08,003.51 were counted as current stock when they are three years old or more. 95 of them, ₹52,17,407.82, arrived in 2021. The report exists to find money that has been sitting, and it was hiding five sixths of it.

Fixed

Aged by the earlier of the two dates. Driven on the running screen, and the footer now reads what the SQL predicted, to the rupee:

1571 tag(s) · 377 of them acquired in 2023 or earlier · of which 74,48,288.10

"tagged" became "acquired" in that sentence, because it is no longer the tagging date being counted. The existing test that asserted the old wording was updated with it, and a new one seeds a piece keyed in on the opening day but received five years earlier -- **proven to fail without the fix: a piece bought in 2021 is reported as 2026 stock.**

On the figure in this entry

An earlier draft of it said 59,08,003.51, summed by hand from a listing that had been cut off at twelve rows. The query says 60,08,003.51. The hand figure was wrong by a lakh and would have read just as convincingly.

The chain that started this, which found nothing

sell → refund → sell again now runs as a test. The existing return test puts a tag back on the shelf, but the tag it returns is **seeded** sold -- a row inserted with in_stock false, never carried there by a bill. So a real sale followed by a real refund had never been driven.

```
sold      off the shelf, one SALE-GST movement
refunded  back on the shelf, one SALE-RET movement
sold again accepted, two bills, one piece, off the shelf at the end
vouchers  all balancing
```

Clean. Worth having: a return is the one operation that reverses another, and if a sale left anything a refund does not undo, the second sale is where it would show -- by which point there are two bills and one piece.

259. A table that looks like a bill cross-reference, and would attribute 83 bills to the wrong customer

`sale_bill_link` sits in the dump with 135 rows, never loaded and referenced nowhere. The name and the shape both suggest a useful thing -- a mapping from one bill number to another -- which is exactly why it is worth writing down what it actually contains.

```
rows                      135
where the two numbers differ      133
mcj_bill values that are real bills  135    all of them
test_bill values that are real bills   85

mcj_bill    GST/2627/00001 .. GST/2627/00220
test_bill    GST/2627/00001 .. GST/2627/00338
```

The shop's own bills stop at 00224. The other series runs to **00338**, 114 documents further, and one `mcj_bill` maps to two of them -- GST/2627/00008 to both 00013 and 00014. So `test_bill` is not a renumbering of the same documents; it is a different, longer series from somewhere else.

The part that matters

Of the 133 differing rows, 83 name a `test_bill` that IS a real bill in this shop's books. **All 83 of those belong to a different party.**

```
differing rows                      133
whose test_bill is a real bill      83
and that bill belongs to a different customer  83    every one
```

So a screen or report that loaded this table and used it to cross-reference a bill would, on 83 bills out of 135, hand back a document belonging to somebody else -- with a real number, a real customer, and a real amount on it. Nothing about the result would look wrong.

Left unloaded, deliberately, and now on the record

This is not a defect in the code: nothing loads it and nothing reads it, which is correct. It is written up because *not loading a table* leaves no trace, and the next person to work through the unloaded tables will find a plausible name and 135 tidy rows.

What `test_bill` refers to cannot be recovered from the capture. A parallel installation, a trial run, a second till -- the series is longer than the shop's own and stops partway through the year, which fits something that ran alongside and was abandoned. **A low-priority question for the owner if it ever matters, not a blocker:** the shop's own 224 bills are complete and unaffected.

Method note

This began as a lead on something else entirely -- whether handing over an order raises a bill -- and `sale_bill_link` was the most promising-looking candidate for an order-to-bill link. It is not one. The lead was wrong and the table was worth reading anyway, which is the opposite of how most of these go: usually the lead is right and the table is empty.

260. Ten outstanding orders were invisible on the screen that lists outstanding orders

The Work Still Owed screen joined to the order lines:

```
FROM mcj_customer_orders o
JOIN mcj_customer_order_lines l ON l.order_id = o.id
WHERE o.tenant_id = $1 AND NOT o.delivered AND l.status <> 'Cancelled'
```

An inner join, so **an order carrying no lines never appeared at all**.

undelivered orders	243	
listed by the screen	85	
undelivered with no lines	10	listed: none of them

All ten are carried-forward orders keyed in on 2026-04-01, the day the books opened, from the **2122, 2223, 2324 and 2425** series -- the oldest placed in 2021-22, five financial years ago. Every one names a real customer ledger. **Two are RP repairs**, which is a customer's own gold sitting in the shop with nothing on the one screen that exists to say so.

The screen's own comment says why that matters:

```
An order taken could never be completed: nothing set `delivered`, so every
order stayed outstanding for good and the order book's overdue list could
only ever be empty.
```

That was fixed. This is the same failure from the other end -- work that is owed and cannot be seen.

Fixed

A LEFT JOIN, with the cancelled test moved into the join condition. As a WHERE clause it also dropped an order whose lines were all cancelled -- which is right, that work is not owed -- but it could not tell that case from an order with no lines at all, which is. The two are separated now, and both are tested.

orders listed	85 -> 95
the ten	all shown, marked "(nothing itemised on this order)"

The fix broke the page, and only driving it showed that

The widened join was correct SQL -- run directly it returned 95 rows. The screen showed **nought**, with this in the error box:

```
can't scan into dest[4]: cannot scan NULL into *string
```

Every column on that row comes off the lines -- the date wanted, the days overdue, what was ordered, the estimate -- and an order with no lines has none of them. `to_char(MIN(l.delivery_date), ...)` returns NULL, and NULL does not scan into a string. **One row with no lines killed the whole list**, so the fix for ten invisible orders had briefly made all ninety-five invisible.

Checking the query against the database would have confirmed the fix and missed this entirely. The screen had to be opened.

Proven

Regression test seeds an order with no lines and one whose only line is cancelled: the first must be listed with its marker, the second must not. **With the inner join restored it fails** -- *an order with no lines is not listed as outstanding — the shop owes work it cannot see.*

The sweep for defect 260's siblings, and the tool that came out of it

Defect 260's shape -- correct SQL, dead page -- has an obvious sibling class: any aggregate or LEFT JOIN column that can be NULL and is scanned into a string. Swept for it two ways.

By reading. Every aggregate in a query with no `GROUP BY` and no enclosing `COALESCE`: 23 candidates, all of them already guarded. `itemcatalogue.go` is typical -- `COALESCE(s.tags, 0)` and `COALESCE(ROUND(s.value * 100), 0)` around a lateral that may match nothing. `nextReceiptNo` in `partysample.go` looked like the worst case, a `max()` over a table that is empty before the first receipt, and is safe by construction: it has a `GROUP BY`, so no rows come back rather than one NULL row, and the error branch starts the series.

By provoking it. Reading finds what you thought to look for. Every report was fetched over a date range matching nothing -- 2099 -- which is the state that makes aggregates return NULL. **78 reports, all clean.** Every screen a controller declares, 36 of them, likewise.

So the class has no other members today. That is worth knowing and worth keeping true, which is the part reading cannot do.

tools/check_screens_render.py

Opens every screen and every report, each report twice -- plainly, and over the empty range. A page fails if it does not return 200 or if it renders the error box the layouts use.

Proven against the defect it was written for. Putting back the single uncoalesced

```
to_char(MIN(l.delivery_date), ...):
```

```
x screen    /mcj/orders/outstanding
  can't scan into dest[4]: cannot scan NULL into *string
1 page(s) broken
```

and with it restored, 34 screen(s) and 78 report(s) render, including over a range that matches nothing.

Nothing in this project opened a page before this. `check_report_footers.py` fetches reports but reads only their footers, and the test suite drives controllers it has been told to drive. A screen could be added tomorrow, never tested, and break on a shop with no data in it -- which is every shop on its first day.

261. Opened fresh, the system rendered every screen and could not raise a single bill

The tool from defect 260 opens every page, but it opens them against this shop's own database, which has four months of trade in it. The state it was written to provoke -- no data -- was still only being simulated with a date range. So a database was created from the migrations alone, the way a new shop starts.

tenants	1	
users	1	after cmd/seed-admin
ledgers	0	
purities	0	
items	0	
ledger groups	0	
company	0	

Every screen and every report rendered. 34 screens, 78 reports, twice each. Nothing was broken, which is worth knowing and is not the same as working.

It could not sell anything

The posting screens find their accounts by code, deliberately -- a code is stable where a name is not. There are **twelve** of them:

sale.go	1016 sales · 1036 CGST · 1035 SGST · 1037 IGST · 3336 rounding
creditnote.go	1010 sale returns · 1011 purchases · 1006 purchase returns 11577/11576/11578 input CGST/SGST/IGST
oldgold.go	1018 old gold purchases

Every one exists on the shop's own books, which is why nothing noticed that a new database has none of them: **migrations create the schema, not a chart of accounts.** Nothing seeds one, and no screen mentions these numbers.

A shopkeeper opening the system for the first time could set up their company, their metals, their catalogue and their customers, tag a piece -- and then be told, on trying to bill it:

```
ledger 1016 is missing from the chart of accounts
```

A number that appears on no screen, no report and in no document the shop has.

Fixed, and driven end to end on a database that had never existed before

`cmd/seed-admin` -- the command that opens a shop -- now creates the thirteen accounts the code requires, taking the names and groups from the shop's own ledgers rather than inventing them, `ON CONFLICT (tenant_id, code) DO NOTHING` so an existing shop keeps exactly what it has. CASH is the thirteenth: the till finds it by name rather than code, and a shop with no CASH account cannot take money over the counter either.

Then a full first day, through the screens, on the fresh database:

```
company · purity · customer ledger · item group · item      303 each
the day's gold rate, 7,000                                303
a 10 g chain tagged at that rate                          303
billed                                                     303

invoice  GST/2627/00001  goods 72,000.00  total 74,160.00
vouchers 1, balanced
```

10 g at 7,000 is 70,000, plus 200 a gram making is 72,000, and 3% tax makes 74,160. **A shop that did not exist an hour earlier raised its first bill.**

And proven the other way

A second fresh database, set up identically but with the sales account deleted:

```
bill 500
invoices raised: 0
this bill could not be saved: the sales account (code 1016) is missing from
the chart of accounts – create it under Ledger Master, or run cmd/seed-admin
```

The message names the account's purpose now, not just its number. Both scratch databases were dropped afterwards; the shop's own is untouched -- 2,816 tags, 224 invoices, 785 ledgers, 1,848 vouchers, all unchanged.

Method note

Defect 260's tool said "every shop on its first day" in its own docstring, and the sentence was written without ever having started one. **The gap between simulating an empty shop with a date filter and actually opening one is where this defect lived.** Two silent failures on the way: a `DROP DATABASE` refused because the application still held a connection, which made a "fresh" database the old one and reported *Applied 0 migrations* -- noticed only because 0 was an odd number to see on a new database -- and a `DELETE` that hung on a lock the running application held.

The guard, and what it caught immediately

Defect 261 was fixed by listing the required accounts in `cmd/seed-admin`. That list was built by hand from a `grep`, which is the same method that produced the defect, so it needed a check rather than care.

`TestPostingCodesAreAllSeeded` reads every `ledger...` = "1234" constant out of the controllers, reads the seeded list out of `cmd/seed-admin`, and fails if a code a screen posts to is not created. It refuses to pass vacuously: an empty list on either side is a failure, not a green run.

It failed on its first execution.

```
1 account code(s) a screen posts to are not created by cmd/seed-admin,
so a shop opened fresh cannot post through that screen:
1012 (used by creditnote.go)
```

1012 is PURCHASE_B2B . The list written an hour earlier had twelve accounts and needed thirteen, because the grep output it was read from had been **cut off at twenty lines** -- the same truncation that produced the wrong figure in defect 258. A B2B purchase on a new shop would still have failed, with the fix in place and the write-up already saying it was fixed.

Now:

```
12 posting code(s) across the screens, all seeded
14 mandatory account(s) created in the chart of accounts
```

Fourteen against twelve because CASH is found by name rather than code, and because two codes share the tax accounts. Verified once more on a database created from nothing, then dropped; the live shop still has its 785 ledgers.

Method note

Three times now a figure or a list has been wrong because a listing was truncated and read as complete -- defect 258's lakh, defect 260's join, and this. **The failure is not carelessness about numbers, it is trusting the visible part of a result that was cut off.** The countermeasure that works is the same each time: have something else derive the answer independently, and compare.

262. The guard written to catch incomplete lists was itself an incomplete list

Defect 261's guard found a missing account on its first run and was written up as working. Checking it properly found two more things, both the same shape.

Its own pattern missed a code

```
(?m)^\s*ledger\w*\s*=\s*"(\d+)"
```

That requires the line to begin with the constant's name, which is true inside a `const (...)` block. `oldgold.go` declares its account on its own:

```
const ledgerOldGold = "1018"
```

so the pattern never saw it. The guard reported twelve codes when there are thirteen, and passed -- because 1018 happens to be seeded. **A guard that misses a case and goes green over it is the failure it was written to prevent**, and this one was one line long and shipped an hour earlier.

Widened to `^\s*(?:const\s+)?ledger\w*`, it sees thirteen.

A third hand-built list of the same codes

Defect 261 also added `accountPurpose`, which turns a code into words for the error message. It was written by hand from the same truncated grep, so it has twelve entries and is missing **1012** -- meaning a shop without PURCHASE_B2B would have been told:

```
the posting account (code 1012) is missing from the chart of accounts
```

which is the bare number the message was changed to stop showing. The fix and its own fallback disagreed with each other.

The guard now checks both halves: every posting code must be seeded, **and** every posting code must have a purpose. Proven by removing 1018 from each list in turn -- *1 posting code(s) have no purpose* and *1 account code(s) ... are not created* -- and passing with both restored.

```
13 posting code(s) across the screens, all seeded and all named
```

And the failure message contradicted itself again

The purpose check was a `t.Errorf`, which does not stop the test, so a failing run still logged *"all seeded and all named"* one line later. That is the third time in this document: defect 252's savepoint, defect 258's hardcoded want, and now this. It is a `t.Fatalf`.

Method note

Three lists of the same thirteen codes now exist -- the constants, the seed, and the purposes -- and two of the three were wrong within an hour of being written. The lesson is not to write more carefully. It is that **a list derived by hand from another list should be checked by a machine on the day it is written**, because the moment to catch it is before the write-up says it is done.

263. The financial-year rollover, proven rather than assumed — and nothing wrong with it

A bill number carries the year it belongs to: GST/2627/00224 is the 224th bill of 2026-27, and the count restarts each April. That is a thing which can only go wrong once a year, on a day the shop is open, and this shop reaches it on **1 April 2027**.

The repository scopes numbering to `(type, financial_year)`, and its comment says so. Nothing had ever run a shop across the boundary.

```
31 March 2027  GST/2627/00001
 1 April 2027  GST/2728/00001
vouchers       two, one in each financial year
```

It is correct. The count restarts, the year code advances, and the two vouchers land in separate years -- which matters beyond the number, because a trial balance for either year would otherwise pick up the other's trade.

Proven to fail when it should, at the second attempt

The first attempt at breaking it replaced the year filter with `$3 IS NOT NULL`. The test failed -- and for the wrong reason. Postgres could no longer infer the parameter's type, so the save itself errored and the failure landed on the status assertion rather than the bill number. **A test that fails for the wrong reason proves nothing**, and it looked exactly like proof: a red run, the right test name, immediately after the change.

That is defect 232's parameter-typing problem showing up in a deliberately broken query, which is a fair reminder that a break has to be as carefully made as a fix.

Broken properly -- `(financial_year=$3 OR true)`, which keeps the parameter typed and only stops it narrowing:

```
the first bill of 2027-28 is "GST/2728/00002", want GST/2728/00001 --
the count did not restart with the year
```

Also checked, also fine

```
customers' own goods      7 lots held, 4 to 56 days, every one against a
                           live order, none delivered, none returned
receive / list / return  every step has a screen, and the return path
                           guards with AND NOT returned
the report's comment      "233 pieces, 14.43 carats, five customers, none of
                           it returned" -- matches the data exactly today
```

Customer property is the one thing in the shop that is not the shop's, and the flow around it is complete and consistent. Recorded because the alternative is finding out later that nobody looked.

264. The GST return reconciles to the rupee, and told the reader it was the wrong section

The outward-supplies return is the one report in this system that gets filed with a government, and **nothing drove it**. Reconciled it against the books by hand first, because a report that does not agree with the ledgers is worse than one that does not exist.

		the books		the report	
B2B	20 bills	95,15,937.00	98,01,417.00	95,15,937.00	98,01,417.00
B2C	204 bills	1,45,50,517.00	1,49,63,969.00	1,45,50,517.00	1,49,63,969.00
both	224 bills	2,40,66,454.00	2,47,65,386.00	2,40,66,454.00	2,47,65,386.00

Tax on the whole year is 6,98,933.87 -- CGST 2,27,893.80, SGST 2,27,893.80, IGST 2,43,146.27 -- and the two halves add to the whole. **Exact, in every figure.** The report reads the taxable value from the VOUCHER rather than the invoice header, deliberately, and the vouchers are what the books are.

What was wrong

The closing note said, whichever section was chosen:

```
This is the B2B section only.
```

So a B2C listing -- 204 bills and 1,45,50,517.00 of taxable value -- described itself as B2B, and so did Both, which is not a GSTR-1 section at all. On a document that gets filed, a label naming the wrong section is worse than no label: it is an assertion, and it is false on two of the three settings.

Fixed. The note now names what is shown, and says of Both that it is every bill rather than a section as filed.

Tested, at the second attempt

Two tests: each section reconciles, and the note names the section.

The first version read `rows[len(rows)-2]` as a totals row. **The report does not return one** -- the shell adds totals, and the report returns its bills and the note. So that read the LAST BILL, which is the same figure whenever a section holds one bill. It passed on B2B and on B2C and failed only on Both:

```
all taxable = 40,000.00, want 1,40,000.00
```

Two of three green for the wrong reason. It sums the rows itself now.

Both tests proven against the defects they describe: filtering B2C as B2B gives *b2c taxable = 1,00,000.00 over 1 bill(s), want 40,000.00*, and putting the note back gives *with kind=b2c the note reads "This is the B2B section only."*

Method note

A test that passes on two cases out of three, for a reason unrelated to what it claims to check, is the same failure as defect 262's guard and defect 252's savepoint. What caught it here was **having three cases where two would have done** -- Both was included because it exists, not because it was suspected, and it is the only one that could expose the mistake.

265. The purchase return knew something dangerous and kept it in a comment

Defect 264 tested the sales return, so the inward one was the obvious sibling. It reconciles exactly:

		the books		the report	
PURCHASE_B2B	7 bills	39,33,602.29	40,79,648.00	39,33,602.29	40,79,648.00
PURCHASE	36 bills	1,94,71,149.21	1,98,77,150.00	1,94,71,149.21	1,98,77,150.00
both	43 bills	2,34,04,751.50	2,39,56,798.00	2,34,04,751.50	2,39,56,798.00

The screen also avoids the trap its sibling fell into: its filter is labelled in the shop's own terms -- *Posted to PURCHASE_B2B, Posted to PURCHASE* -- rather than B2B and B2C, and there is a "Posted to" column. Whoever wrote it understood the risk.

And then wrote the reason down where nobody would read it

Above the function:

```
It is NOT "the supplier is registered" ... 17 of the 36 unflagged invoices
carry GST, so those suppliers are plainly registered ... Labelling them B2C
in the sense a return uses would understate recoverable input tax by
4,02,980.57.
```

Verified today and still exact: **17 of 36**. That is the single most important sentence about this screen, it names a figure of over four lakh, and it was visible only in the source. A person filing a GSTR-2 sees a split that looks like the registered/unregistered one and nothing telling them it is not.

The sales return carries a closing note; this one had none at all.

Fixed

The return now ends with what the comment knew:

This is the shop's own split between its two purchase accounts, not the registered-supplier split a GSTR-2 asks for: 17 of the 36 invoices posted to PURCHASE carry GST, so those suppliers are registered. Which invoices belong in which part of a filed return is for whoever files it; this is evidence towards that, not the answer.

Tested on all three sections, and proven by removing the note -- *with kind=b2b the return does not say what its split is: "REGISTERED SUPPLIER"*, which is the last supplier row standing in for a note that is no longer there.

Method note

This is the fourth time this week a correct piece of reasoning existed only as a comment: defect 248 (an explanation that had gone stale), 254 (a hypothesis contradicted by the harness's own docstring), 256 (a screen citing an answered blocker), and now one that was right the whole time and simply never reached the person it was written for. **A comment is a note to whoever edits the code. When the reader who needs it is the shopkeeper, it has to be on the screen.**

266. A comment told the next reader that table 12 could not be filed. It has been filable for a fortnight

Sweeping for the pattern defect 265 named -- reasoning that lives only in a comment -- turned up thirty reports whose doc comments carry a warning or a figure. Most are ordinary documentation. One is the HSN summary, which is table 12 of GSTR-1, and its comment said:

WHAT IS STILL MISSING is line detail for 39 of the 224 bills ... That is 89,26,345.00 of taxable value ... A return filed from a 63% summary would be wrong.

Every one of the 224 bills carries line detail today. Defect 155 recovered those 39 from the capture -- the lines went from 328 to 367 -- and invariant rule 2 has checked it on every run since. The screen agrees: the summary totals 2,40,66,454.00 of taxable value and 6,98,933.87 of tax, matching the books exactly, across twelve HSN rows.

So the report was filable and its own documentation said it was not, on the one screen in this system whose output goes to a government.

What I got wrong while fixing it

The first correction said the paragraph was stale and replaced the lot. It was stale in its figures and **right in its design**, which I only found by trying to break the test I had just written.

A bill whose lines cannot be resolved to an HSN is not dropped. It comes back as a row coded "--" carrying its taxable value, so the table always adds to the books and any gap is visible IN the return rather than missing from it. That is what "states it on the face of the table" meant, and it still works:

```

with one bill's line removed:
row 0  hsn 711319  1,00,000.00
row 1  hsn —      40,000.00
total                1,40,000.00 = the books

```

The corrected comment keeps that and fixes only the figures.

The test I wrote could not fail

It asserted the summary's total equals the books'. **That is true by construction** -- the "—" row exists to make it true -- so the assertion passes whether coverage is complete or nothing is attributed at all. Removing a bill's line left it green.

It took three attempts to see. The first break did not apply, because shell quoting mangled it; the second applied and still passed, which I could not explain; instrumenting it showed why. **A test whose green run I cannot account for is worth no more than one that is red.**

Now it asserts what matters -- that the "—" row is empty -- and refuses to run against empty books. Proven: leaving a bill without lines gives *40000.00 of taxable value has no HSN behind it — it appears in the return under "—", and a return filed with that row is incomplete.*

Method note

The sweep that found this looked for warnings in comments and had to be read by hand, because its own "does the screen say it?" test was unreliable -- it missed the note added in defect 265 an hour earlier, since that note is built by concatenating two short strings. **The heuristic was wrong and the list it produced was still useful**, which is the argument for looking at thirty candidates by eye rather than trusting a filter over them.

The rest of the sweep: 92 claims in report comments, and they hold

Defect 266 came out of a sweep that listed thirty reports whose doc comments carry a figure or a warning. Only the warnings had been read. The figures -- **92 claims** across those comments -- were checked next, because a document that quotes a number is asserting it.

The most-repeated and most consequential, against the database today:

shelf value, in stock	14,23,61,188.20	used by 5 reports	exact
opening stock	8,52,67,768.16		exact
sales goods	2,40,66,454.00		exact
bill totals	2,47,65,386.00		exact
stock stones	6,21,95,953.60	used by 3 reports	exact
sold stones	1,11,83,462.33		exact
old gold taken in	50,71,370.00		exact
purchase register	71 vouchers, 2,85,06,783.81		exact
pricing pending	4 tags; 952 / 951 / 968		exact
negative stock	0, 0, and 0		exact

Every one holds. The one claim that had gone stale is defect 266's, and it went stale because the PRODUCT changed underneath it -- 39 bills were recovered -- not because a number was mis-transcribed.

That is the useful distinction. A comment quoting a total stays true while the data does; a comment describing a GAP is a claim about what the software cannot do, and every fix is a chance to falsify it. Swept for that shape specifically: eight lines, six of them historical narrative or genuine properties of the data ("a stone is bought by the carat and has no ticket price"), and the two live ones both verified above.

Three of the "mismatches" were my own measurements

Every apparent discrepancy in this pass turned out to be me comparing a different quantity from the one stated:

purchase register	compared 43 purchase INVOICES (2,39,56,798.00) against a comment about 71 purchase VOUCHERS (2,85,06,783.81) -- driving the register gave 71 rows and the exact figure
pricing pending	counted every shelf tag with no rate (968) against a comment about those with making_type 'val' (952) -- and the comment's own 968 was sitting in the next sentence
order detail	joined the legacy table on the wrong key, getting "212 of 273 orders have no lines" where our own data says 13 of 284 (defect 260)

Two of the three were within a sentence of being written up as defects.

The pattern is not carelessness with numbers, it is answering a question adjacent to the one asked. The countermeasure that worked every time was reading the claim again before believing the disagreement -- and, where the claim is about what a screen shows, driving the screen instead of querying the table behind it.

267. Three of the four voucher screens had never been driven

The voucher controller mounts four kinds -- receipt, payment, contra, journal -- from one constructor. Ten tests exercise it, and every one of them posts to the **journal** path.

journal	624 of the shop's vouchers	driven
payment	626	never
receipt	269	never
contra	31	never

926 vouchers' worth of screen against the journal's 624, and the payment kind carries a cheque block the journal has not got, so that field's whole path was code no test had executed.

Same blind spot as defects 243 and 244 on the master screens: one of a set is exercised and the rest are assumed to follow because they share a constructor. Here they do follow -- all four save, land under their own type and balance -- but that was not known.

Driven now, including a payment with cheque 004521 dated 2026-04-10, which is read back off the voucher.

Two mistakes in the test, both mine, both caught by driving it

It asserted the wrong table. The first version read the cheque back from `mcj_voucher_lines` , and failed:

```
payment: the cheque details were not written: no rows in result set
```

Both tables carry `cheque_number` and `cheque_date`. The screen writes the voucher's; the shop's own data has 34 on vouchers and **593 on lines**, which is what made the line the obvious place to look. It is not where this screen puts them, and the reconciliation screen already handles both -- it reads the line first and falls back to the header, with a comment explaining the same 34.

It asserted a figure I made up. The test comment said 419 of the shop's 626 payments carry a cheque number. The number is **34**. Nothing had been measured; it was written while looking at 593 lines and reasoning about vouchers.

And the type check could not fail

It compared the saved type against the kind's own declared type -- so retyping contra as a journal moved both sides together and the test stayed green. Proven by doing exactly that.

The kinds are now collected and compared to each other, which is the property that matters: a receipt that posts as a journal is not a receipt, and the day book, the cash book and the voucher list would all file the shop's money under the wrong heading. Retyping contra now gives:

```
contra and journal both post as "JRNL" -- they are one kind wearing two names
4 kind(s) produced only 3 distinct type(s)
```

Method note

Three tests written this week have compared a value against the same source that produced it -- defect 262's guard, defect 266's total, and this. **A test is only worth what its two sides are independent of each other**, and the quickest way to tell is to break the thing it claims to check and watch what happens. All three were caught that way, and none of them by reading.

268. The fourth trade kind was missing from the test that reads as full coverage

Defect 267 found three of four voucher screens undriven, so the sibling was the trade controller, which mounts four kinds from one constructor the same way.

`TestTrade_EachKindPostsItsOwnType` looks like the answer to exactly this question. It drives three:

credit-note	SALE-RET	driven
purchase	PURC-GSC	driven
debit-note	PURC-RET	driven
purchase-order	PURC-GCO	never

Purchase against a customer's order -- 19 of the shop's purchases, and the only trade kind that is tax-free by design -- was absent from the one test whose name promises every kind. A test called *EachKind* that covers three of four is worse than no test, because it answers the question wrongly.

Driven now, and it posts PURC-GCO correctly.

The list is checked against the source

Writing the expected types out by hand is deliberate -- reading them from `TradeKinds()` would compare the code against itself, which is what made defect 267's type check green while contra was retyped as a journal. So the values stay written down, and the SET of slugs is checked against `TradeKinds()` instead:

```
trade kind(s) [purchase-order] are not exercised here
```

That is the failure with purchase-order removed from the expected map. A kind added tomorrow fails here rather than being silently uncovered.

What actually stops a new kind slipping through

Planting a fifth kind to test the guard did not reach the guard. It panicked first:

```
navigation: screen "/mcj/trade/planted-kind" has no LegacyFile,  
so it must give Top and Item
```

The navigation registry refuses a screen that names neither a legacy file nor a menu position, so a trade kind cannot be added without being placed somewhere a person can find it. **That is a stronger guard than the one I was writing**, it was already there, and it caught the plant in under a second.

Worth recording because it is the opposite of this week's usual finding: a protection that existed, worked, and was better than what was being added beside it.

Method note

Three sets of screens built from one constructor have now been checked -- masters (243, 244), vouchers (267), trade (268) -- and every one had members nobody had driven. **The shape to look for is a constructor taking a list**, and the question is not whether the code handles each member but whether anything has ever asked it to.

269. Seventeen reports were named by no test at all

The list-shaped blind spot of defects 267 and 268 has a bigger instance: 79 reports registered from one list, and no count of how many had ever been asserted about. **17 of the 79 are named by no test.**

BackDatedSubgroupStock	ItemCatalogue	SalesRegister	SubgroupStockTally
DesignCatalogue	PacketStockSummary	SchemeBook	SuspectRates
GrossProfit	Quarterly	StockHistory	UntaggedItemStock
RawStockTally	RatioAnalysis	StockJournal	UserMaster
		StockValuation	

They are not unexercised: `check_screens_render.py` opens every one of them, twice, and they render. What none of them has is an assertion about what the figures ARE.

Reconciled by hand, and they are right

sales register	224 bills	2,40,66,454.00	6,98,933.87	2,47,65,386.00
the books	224 bills	2,40,66,454.00	6,98,933.87	2,47,65,386.00
stock valuation	shelf 14,23,61,188.20, matching the tags exactly			

Both agree. The sales register reads from the VOUCHER, like the purchase register and the GST return, so this is the fourth report checked this week that ties to the books to the rupee.

The sales register now has a test, and it took three corrections

The seed made vouchers with no debit side. Two credits and nothing else -- which is not a voucher, and the register lists the customer's name off the debit line, so it returned nothing at all. Adding the customer's leg fixed it and made the seeded vouchers balanced, which they should have been from the start.

The test read the wrong key. The register calls its column `goods` ; the test asked for `taxable` , got nothing from every row, and summed zero.

That would have passed as a green run against an empty sum, except for the one line that saved it:

```
if bills == 0 {  
    t.Fatal("the register listed no bills, so this test proves nothing")  
}
```

That guard is the only reason this was caught rather than shipped. It is the same guard defect 266 needed and did not have.

Proven by zeroing the register's goods subquery: *the register totals 0.00 taxable against the books' 140000.00.*

Method note

Five times this session a query or a test has asked for something adjacent to what it meant: purchase invoices for purchase vouchers, rate-zero tags for val-priced ones, a legacy join on the wrong key, `mcj_voucher_lines` for `mcj_vouchers` , and now `taxable` for `goods` . Every one produced a plausible answer -- usually zero, sometimes a real number belonging to a different question.

A test that can return zero and pass needs a line saying zero is not an answer. That is cheap, and it is the difference between the two outcomes here.

270. Nine tests asserted that a report does not list something, and would have passed if it listed nothing

Defect 269's saving line -- *"the register listed no bills, so this test proves nothing"* -- is the only reason a test that summed the wrong key was caught. So: how many tests are missing it?

Swept for assertions made inside a loop over a report's rows, with no check that there were any. **38 candidates**, most of them fine -- they look up a specific row through a helper and fail on nil. The class that is not fine is the negative assertion:

```

for _, r := range rows {
    if got, _ := r["status"].(string); got == "Cancelled" {
        t.Fatal("a cancelled order line was listed")
    }
}

```

Nine of these. *LeavesCancelledLinesOut*, *DoesNotMixDocumentsUp*, *StaysInsideThePeriod*, *IgnoresACancelledBill*, *CountsOnlyGoldJewellery* and four more -- every one of them satisfied by a report that returns nothing at all. They were testing that a filter excludes what it should, and a report that excludes everything passes.

The first guard I wrote was decoration

if len(rows) == 0 -- which is never true. **This codebase's reports return one placeholder row when they have nothing to show:** "No live order line matches that choice.", "Nothing was bought between those dates.", and so on. That is good behaviour and it is the house style, and it means row count is not evidence of anything having been listed.

I found that out by neutering a report and watching the guards not fire. Two unrelated tests failed and the two I had just "protected" passed.

The guards now count the rows carrying the field each test actually reads, so a placeholder does not count:

```

listed := 0
for _, r := range rows {
    if _, ok := r["status"]; ok {
        listed++
    }
}
if listed == 0 {
    t.Fatal("the report listed nothing, so the check below proves nothing")
}

```

Proven by suppressing the report's rows entirely:

```

TestOrderToBarcode_DoesNotTreatUnmadeWorkAsAProblem
    the report listed nothing, so the check below proves nothing
TestOrderToBarcode_LeavesCancelledLinesOut
    the report listed nothing, so the check below proves nothing

```

Method note

Two guards in two days have been written, shipped green, and turned out to check nothing -- defect 262's regex and this one. Both were caught the same way, by breaking the thing they claimed to guard rather than by reading them.

The rule that keeps holding: a check is not a check until it has been seen to fail. It costs a minute. Neither of these would have been found by any amount of care at the time of writing, because both looked exactly right.

A report that has gone silently empty, and the check that now catches it

Defect 270 is a trap inside the tests; the same trap exists outside them. Every report here answers an empty result with **one placeholder row** -- good behaviour, and it means a 200 and a rendered table prove nothing about whether anything was listed. A report whose query stops matching looks exactly like a report with nothing to say.

Measured across the shop's whole history, 2024-04-01 to 2027-03-31:

reports showing real data	76 of 78
showing none	2 – ledger-book and quote

Both of those take a **subject** rather than a period: the ledger book needs a ledger and the quote needs a barcode, and an empty page without one is the right answer. So nothing is silently empty today.

`check_screens_render.py` now asserts that, with those two named as exceptions in the source and the reason written beside them. Proven by silencing one report's row loop:

```
x report (no data) /mcj/reports/hallmarking
  renders, but shows no data over the whole of the shop's history –
  a report that has stopped matching looks like one with nothing to say

1 page(s) broken
```

and restoring it gives *34 screen(s) and 78 report(s) render, including over a range that matches nothing, and 76 of them show real data.*

This is the check that was missing when the tool was written yesterday: it opened every page and confirmed none of them broke, which is a different question from whether any of them still work.

271. The shop sells 17% of its billing from a price list nothing could set

`mcj_coin_rates` sat in the unwritten-tables list with **5,966 rows** -- read by a report, written by nobody. It is not history.

```
52 coin items, priced across 115 trading days
3 April to 19 August 2026, the day before the system went down
26,11,606.00 of coins sold – 17% of all billing
against 1,70,345.00 of tagged coin stock
```

A coin is a product with a published price, not metal weighed off the shelf. The shop set that list every trading day and the system had no screen to set it.

The screen that looked like it did

Daily Metal Rates has a coin column. It writes `mcj_metal_rates.coin_rate` -- and that column is populated on **none** of the shop's 2,820 rate rows. It is a different figure, one per metal grade, that the shop never used. The list it actually sells from is one price per coin per day, and nothing wrote it.

So the gap was invisible twice over: a table with a reader and no writer, beside a screen that appeared to cover it.

Built

`/mcj/coin-rates`, Daily Coin Prices, modelled on the metal rate screen because it answers the same shape of question. Driven against the shop's own data:

```
52 coins listed
prices carried from 19 Aug 2026 and labelled as carried
a price set for 24 Aug 2026 wrote SILVER COIN 1/2 TOLA at 7,42,500.00
```

The test row was removed and the table is back to 5,966.

The item master's own rate is shown beside each coin and never used as one. Every one of the eleven items carrying a rate there is 4.7 to 4.9 times below its latest daily price -- pricing a coin from the master undercharges by about 79%, a coin the list puts at 1,23,400.00 carrying 26,000.00 there (data defect 54). It is displayed because it is what somebody would otherwise reach for, and it is never filled into the price box.

Seven tests: the list holds coins and not other items, a carried price is labelled, the frozen master rate is shown but not offered, a price saves, setting twice leaves one row, a blank row is not an error -- 19 of the 52 are never priced on any day -- and something that is not a price is refused rather than stored as nought.

One correction

The first test compared the stored value as text and failed:

```
the price came back 126900.00000, want 126900.00
```

The column is `numeric(18,5)`. The price was right; the comparison was against Postgres's rendering of it rather than against money. Formatted before comparing.

```
tables read but never written    24 -> 23
```

272. Which of the unwritten tables are still alive, and the one the shop uses daily that cannot be done

Defect 271 built the coin price list because it turned out to be operational. The obvious question is which of the remaining 23 are. **Asked by last activity rather than by name**, because a table called "holiday calendar" sounds annual and this shop's is not.

stock_conversions	859	18 Aug 2026	live – screen refused, defect 251
barcode_edits	156	15 Aug 2026	live – NOTHING CAN DO THIS
ordered_stones	115	10 Aug 2026	live – customers' stones on orders
scheme_payments	2,144	10 Aug 2026	live – blocked on defect 233
scheme_enrolments	239	11 Feb 2026	blocked on defect 233
estimates	2	23 May 2025	two rows, a year old
holidays	109	26 Dec 2024	abandoned
designs	5,221	20 Jan 2017	nine years old

The holiday calendar is not the annual thing it looks like. 52 Thursdays in 2016, 53 in 2017, then four days in December 2024 and nothing since -- the shop's weekly closing day recorded for two years and given up. Building an entry screen for it would be building for a practice that stopped in 2017. That is exactly the assumption defect 271's method exists to avoid.

The design catalogue's 5,221 rows are from 2017. The largest table on the list by far, and the most obviously dead.

The one that matters: a tag cannot be corrected

`mcj_barcode_edits` is the trail of corrections to a tag. It is live -- 156 of them between 12 April and 15 August 2026, about one a working day -- and every one of the 156 is on a tag **still on the shelf today**.

What the shop corrects:

the unit count	119 of 156	<- said "piece count" first; see defect 273
what the item is	31	
the making charge	7	
gross weight	0	
net weight	0	
stones	0	

So a tag is corrected for what it IS and how many, and never for what it weighs.

The rebuild cannot do any of it. Every `UPDATE mcj_stock_items` in the codebase sets one of three things: `category_id` from the categorise screen, and `in_stock` from a sale and a refund. **There is no screen that changes a tag's item, piece count or making** -- which is why nothing writes the audit trail, and why the trail's absence looked like an ordinary unwritten table rather than a missing operation.

A piece tagged as one ring when it is two cannot be put right. It can be sold wrongly, or left.

Not built here, and why

The screen is well defined by the measurements above -- three fields, plus a row in `mcj_barcode_edits` recording old and new, who and from where, which the table already has columns for. It is the next build.

It is not this one because the coin price list was built in this same pass and a second screen begun now would be a screen half-driven. **The measurements are the useful part and they are recorded**, so the build starts from evidence rather than from a guess about what a tag edit changes.

Method note

Three tables were assessed by name this session and two of the three were wrong: holidays reads as operational and is dead, barcode_edits reads as an audit artefact and is a missing operation. **The one question that separated them was "when was the last row written", and it takes one query.**

273. A pair of bangles was tagged as two units, and defect 272 named the wrong field

Setting out to build the tag-correction screen defect 272 called for, the first question was what the shop's 156 corrections actually change. That answer turned out to be wrong in the write-up, and following it produced a real defect somewhere else.

The correction to defect 272

It said 119 of the 156 edits "fix the piece count". They do not. They fix the **unit** count, and the report's own comment says so:

```
Units, not pieces. old_units/new_units come from barcode_main.qty, which is
not what a bill counts -- that is nag.
```

Compared against nag first, and got a contradiction that looked like a serious data defect: the tag's count matched old_qty on all 119, while item and making matched their new_ side. Against qty -- the column the trail is actually about -- **119 of 119 match** new_qty , consistent with everything else. There is no defect in the trail, and the one I nearly wrote up would have been the sixth thing this session measured one field away from the question.

The real defect, found by following it

pieces and quantity are different numbers, and this tag screen wrote the piece count into both:

```
VALUES (... , $6::int, $6::numeric, ...)
```

The shelf says otherwise:

tags that are ONE unit	2,787 of 2,816
holding more than one piece	803 – 719 pairs, 56 sets of 3, 21 of 4
tags where quantity equals pieces	21

A pair of bangles is **2 pieces and 1 unit**. The screen made it 2 units of 2 pieces, which matches the 21 exceptions and contradicts the 2,787.

Fixed: a tag is one unit. Proven by restoring the old line -- *the tag is 2.000 unit(s), want 1.000 — a pair is one unit.*

What this is and is not

Nothing reads quantity **today**. It is written by this screen and consumed by no report, which is why 803 tags could disagree with a new one and nothing showed it. So no bill is wrong and no figure moves.

It matters because it is the field the shop corrects most -- 119 of its 156 tag edits -- so it is plainly the one they care about, and because a column written wrongly and read by nobody is a column that will be read eventually.

And a self-inflicted one

The comment explaining all this went into the SQL string, and contained backticks around the column names. The SQL is a Go raw string, which backticks terminate:

```
syntax error: unexpected name pieces in argument list
```

Caught by the compiler in a second, and worth a line because the fix was to write the names plainly -- the quoting that reads well in prose is the quoting that breaks the string it is inside.

274. A tag on the shelf can be corrected, and the correction is on the record

Defect 272 measured what the shop corrects and left the screen unbuilt; defect 273 corrected the measurement. This is the screen.

```
the unit count    119 of 156 edits
what the item is  31
the making        7
gross weight      0
net weight        0
stones            0
```

Weights are not offered. The shop has never corrected one in 156 edits, and a weight is what a scale settles rather than a screen.

Changing the making reprices the tag

Checked before building, on the shop's own seven making corrections: `total` equals `amount + making + other making` exactly on all seven afterwards. So the legacy repriced, and so does this.

Leaving the total alone would have been the quiet kind of wrong. A tag whose price no longer follows from its own figures is one the sale screen holds back for a hand price (defect 255) -- correcting a making charge would have made the piece harder to sell, and nothing on the screen would have said so.

Driven on the shop's own data

Barcode 10060840, a real pair, 2 pieces and 1 unit:

```
units 1 -> 2
making and total unchanged, which is right - units are not a price
mcj_barcode_edits: units_changed t, old 1, new 2, item_changed f,
                    named a person and an address
```

That matches the shop's own convention, where `new_units` is what the tag ends up holding -- 119 of 119 (defect 273). The tag was put back and the trail returned to its 156 rows.

Seven tests, two of them proven by breaking the screen

Units change without repricing; making changes and reprices to 51,500.00 from 50,000 of metal and 10 g at 150; the item changes; a sold tag is refused *by name* -- "a piece already sold cannot be corrected" rather than pretending it does not exist; an empty correction is refused rather than written to the trail, because an audit entry that changed nothing is a lie about the shelf; and no weight field is offered.

Proven: writing the audit row against the wrong barcode gives *the correction was not recorded*, and dropping the reprice gives *total came back 51000.00, want 51500.00 — the tag was not repriced*.

One that did not prove anything

The first attempt to break the audit write deleted the INSERT and left `time` imported, so the package did not compile and the test never ran. **A break that does not build is not a break**, and the run that "passed" against it proved nothing -- it was caught only because the compiler said so out loud.

tables read but never written	23 -> 22
screens	35 -> 36

275. An order for a diamond ring was recorded as an order for a ring

`mcj_ordered_stones` is what a customer has asked to be set into the piece they have ordered. It is live -- 115 rows to 10 August 2026 -- and the order screen could not write one.

orders carrying stones	68 of 284
stones asked for	115 rows, 24,69,792.15
of that, on orders still live	16,36,662.00

So a customer ordering a diamond ring was recorded as ordering a ring. The karigar is not told what to set, and the estimate carries the metal and nothing else.

Only the fields the shop fills

Counted before the form was drawn, from its own 115 rows:

the stone itself	115 of 115
weight	113
carats	107
pieces	101
rate and value	86
shape	64
colour	37
cut	4
sieve	0

Cut and sieve are not offered. Four rows and none, against seven Diamond Setup masters in the old menu -- the shop keeps the masters and does not use them on an order. Offering every column the table has would have been drawing the schema rather than the shop.

Order level, not line level

The stones attach to the ORDER. That is how the shop's own rows sit, and joining them to lines fans out: the stone report measured 545 carats that way against a true 403. A part-cancelled order cannot split its stones between what was cancelled and what was not, which is why the report states the order's condition rather than a line's -- and why this screen writes them the same way.

Driven on the shop's own data

```
C0/2627/00114    4 pc · 1.250 ct · 0.250 g · RUBY at 45,000.00 = 56,250.00
```

The order and its stone were removed afterwards; 284 orders, 366 lines and 115 stones, all back where they were.

Three tests: a stone is recorded against the order, stones are optional -- 216 of 284 orders carry none and the ordinary case must not be made harder -- and a carat weight that cannot be read is refused rather than stored as nought. Proven by stopping the write: *the stone was not recorded against the order*.

```
tables read but never written    22 -> 21
```

276. Twelve people read as unpaid on a screen, and the ledgers say they were paid

Defect 153 asked the shop a question worth ₹13,21,900: were August, September and October 2025 and July 2026 salaries actually paid? Each month shows a full salary bill and nought against it. **One of those four months can now be answered from the books, and the answer is yes.**

July 2026 sheets	12 people, 2,89,200.00 gross, 0.00 paid
staff ledgers, 5 Aug	12 people, 2,73,094.00 paid
the month's deductions	16,106.00

2,89,200 less 16,106 is 2,73,094, to the rupee, and every one of the twelve matches a staff ledger by name. **They were paid.** The payment is in the ledgers and was never written back to the sheet.

Wages for a month are paid in the next one -- every month the shop paid, the voucher falls in the month after the sheet -- so the July sheet was still reading nought when the dump was taken on 21 August, sixteen days after the money went out.

The screen said the wrong thing about it

```
July 2026 | ANKUSH KHANDELWAL | 23,000.00 | 40,000.00 | 0.00 | no payment recorded
```

"No payment recorded" is true of the sheet and false of the shop, on the one screen somebody would use to decide whether staff are owed money.

The report was already careful here -- it distinguishes deductions from arrears by name, and says so at length, because the naive reading would have it announce that the shop owes 15,71,288. It just had no way to know the payment existed, because it never looked outside the sheet.

Fixed

The sheet now consults the ledgers for the month after it, and says which case each row is:

12 rows	paid 2,73,094.00 per the ledgers; the sheet was never updated
37 rows	nothing due (gross is nought too)
47 rows	no payment recorded, and none in the ledgers either

96 zero-paid sheets, all three cases named. The 47 are the ones that are actually open.

What that does to the question for the owner

was	4 months, 13,21,900.00
is	3 months, 11,11,800.00

July 2026 is answered and comes out. What remains is August, September and October 2025 -- 10,53,800.00 -- plus 58,000.00 across three sheets in February 2026 and 21,100.00 in April 2025.

The 2025 months still cannot be settled from the books: the voucher history does not reach back that far (defect 142), which is why defect 153 could not check them. **That has not changed. What has changed is that the question is a quarter smaller and no longer includes a month where the money demonstrably moved.**

Method note

This came out of asking which unwritten tables are still alive. Payroll is -- sheets run to July 2026, 12 to 17 people a month -- and the intention was to build the screen that writes one. Reading the data first found that a screen already built was saying something untrue, which is worth more than the screen that is missing.

277. One person was paid more than their salary, and the screen had no branch for it

Following defect 276 into the next column: the salary sheet DERIVED its deducted figure as gross minus paid, while mcj_payroll_history records the real one -- PF, ESI and professional tax, 209 rows against 246 sheets.

Compared like for like, per person per month:

paid sheets	150
where the two agree	149
where they differ	1

RAJENDRA PRASAD JAT, June 2026: 21,100.00 gross, 24,870.00 paid. Paid 3,770.00 MORE than the month's salary, while payroll history records a 130.00 deduction.

The screen said nothing about it

The switch had a case for nothing paid, and a case for gross exceeding paid, and **no case for paid exceeding gross.** That row fell through both: blank deduction column, no note, indistinguishable from an ordinary line.

It is the one row on that screen anybody would want pointed out -- somebody paid above their salary is either arrears, an advance settled with the wages, or an error, and all three are worth a sentence.

Fixed: *paid 3,770.00 more than the month's gross — arrears or an advance settled with it.* The screen names it and leaves the reading to whoever knows.

The deducted column now shows what payroll history RECORDS, falling back to the derived figure only for the 37 sheets that have no history row. It changes almost nothing -- 149 of 150 agree -- which is the point: the number was right by luck rather than by source.

Three wrong measurements on the way to one right one

"3 of 12 months differ by 11,608"	compared paid sheets against ALL history rows, including staff whose sheet was unpaid – filtered on one side only
"the disagreement is per-row"	the note never fired, which said the hypothesis was wrong before any of it was written down
"then it must be unmatched rows"	zero unmatched in those months

Only the fourth attempt -- joining sheet to history per person per month -- gave the real answer, which is one row, not three months.

Seventh time this session a query has answered a question next to the one asked. The pattern each time is a filter applied to one side of a comparison and not the other. What caught it here was the screen: the note I had written for the disagreement never appeared, and a fix whose evidence does not show up where you expect it has not been proven.

278. The check that finds a silent report called an empty shop broken 55 times

Defect 274 and 275 added screens; the obvious question is whether they work on a shop that has just opened, so a database was built from the migrations alone again (defect 261's method) and every page opened against it.

Every screen and every report rendered, including the three built today. No scan errors, no non-200s, nothing broken.

What did happen is that the tool said 55 pages were broken.

55 page(s) broken

All 55 were the emptiness check added one iteration earlier: *renders, but shows no data over the whole of the shop's history.* On a shop that has not traded, showing no data is the correct answer, and the check had no way to know the difference.

It was written against one kind of shop

The check exists because a report whose query stops matching looks exactly like a report with nothing to say. That is true -- and the same sentence, read the other way, says the check cannot tell an empty shop from a broken one either. It was written and proven against the shop's own populated database and never run against an empty one, which is precisely the gap defect 261 opened and this tool was built to close.

Fixed by counting

One quiet report among many that speak is a fault. Nearly all of them quiet at once is an empty shop.

```
against the fresh shop
  55 of 76 reports show no data, so this looks like a shop that has not
  traded yet -- the emptiness check does not apply and was skipped.
  36 screen(s) and 78 report(s) render ... and 21 of them show real data

against the shop's own data
  36 screen(s) and 78 report(s) render ... and 76 of them show real data

with one report silenced on the populated shop
  x report (no data) /mcj/reports/hallmarking
  1 page(s) broken
```

So it still catches the thing it was built for, and no longer accuses a new shop of being broken on its first morning.

Method note

Three tools have now been written this week and two of them were wrong on their first serious use -- defect 262's guard missed a case, defect 270's guard checked nothing, and this one failed the first time it met a database it had not been developed against. **A tool proven against one dataset has been proven against one dataset.** The fix each time was cheap; finding it required running the thing somewhere new, which is not something reading can substitute for.

279. Two more checks that passed against a shop with nothing in it

Defect 278 ended with the observation that a tool proven against one dataset has been proven against one dataset. So the rest of them were run against the empty shop as well.

Two more had the same trap.

```
check_invariants.py      14 of 14 rules hold
check_report_footers.py  78 reports checked, 0 doubled footer(s)
```

Both true. Both examined nothing.

Every invariant counts exceptions and compares against what the migration already contained, so on a database with no bills, no vouchers and no tags the count is nought and nought never exceeds an allowance. A doubled footer can only appear under rows, so a report with no rows cannot have one.

The database was genuinely empty and the tools genuinely ran against it -- the first reading of the invariants output looked like it had ignored `DB_NAME`, because a rule's allowance is printed as prose ("8 from the migration") and reads like a live count. It had not; the figures were checked against both databases side by side before anything was changed.

Both now say what they looked at

```
check_invariants, empty shop
  This shop has no bills, no vouchers and no tags. Every rule below counts
  exceptions, so all of them hold trivially and none of them says anything.
  Nothing was checked.

check_report_footers, empty shop
  78 reports checked, 5 of them with numbers in, 0 doubled footer(s)

check_report_footers, the shop's own data
  78 reports checked, 76 of them with numbers in, 0 doubled footer(s)
```

76 matches the render checker's count of reports showing real data, from a different tool reading different markup, which is the first time two of these have agreed on anything independently.

The two that were fine

`status_figures.py` and `check_blocker_citations.py` read the filesystem and never touch the database, so an empty shop cannot fool them. `check_unwritten_tables.py` reports row counts as -1 when it cannot get them and still lists the tables correctly, because its real subject is the source.

Method note

Three of the six checks guarding this system reported success against a database they had examined nothing in. None of them was wrong; all three were useless in a way their output did not admit. **The question that found all three is the same one: what did you actually look at?** -- and it is worth asking of any check that reports a count of problems rather than a count of things examined.

280. The four unnamed staff stop a month before the payroll does, and how a salary is worked out

Payroll is the largest live gap left: the shop's sheets run to July 2026, twelve to seventeen people a month, and nothing in this system can produce one. Before building that screen, the arithmetic behind a salary had to come from the shop's own records rather than from an assumption about payroll.

Both formulas, verified

<code>gross = basic + HRA + dearness allowance</code>	209 of 209 rows
<code>paid = gross - (PF + ESI + professional tax)</code>	149 of 150 paid rows

The single exception is RAJENDRA PRASAD JAT, June 2026, which defect 277 already found and named on the screen -- paid 3,770.00 above his gross.

`mcj_staff_components` carries exactly those five components, 46 rows over 12 people, and nothing else. So a salary sheet is fully determined by a basic pay and that setup.

Basic pay is not fixed. It changes for 12 of the 19 staff across the sixteen months, so a screen must carry the last one forward and let it be changed rather than read it from a master. That is the same shape as the metal rates and the coin list.

The correction

Defect 157 records four staff codes that name nobody -- 4023, 8827, 8828 and 8855 -- taking 10,66,400.00 gross and 7,14,430.00 paid. Both figures are exact. The period was written as **Apr 2025 – Jul 2026** and is **Apr 2025 – Jun 2026**:

each of the four	15 sheets, last period 2026-06
July 2026	12 sheets, none of them these four

They stop one month before the payroll does. Twelve other people have a July sheet; these four do not. That is not proof of anything, but it is a fact about them that was not known, and it narrows what they might be: four people who left the payroll together in June 2026, or four rows that stopped being written.

It matters because this is a figure the owner is being asked to explain, and the question is better asked about the right sixteen months.

The screen this was for

Not built here. What it needs is now settled and written down: pick a month, list the staff, carry each basic pay forward from the last sheet, take the five components from the per-person setup, compute gross and net by the two formulas above, and write `mcj_salary_sheets` and `mcj_payroll_history` together -- the second is where the deductions live, and defect 277 showed what happens when a screen derives them instead.

281. The month's payroll can be prepared, and the guard against a known bug did not see it

Defect 280 settled the arithmetic; this is the screen. `/mcj/salary-sheets`, Monthly Payroll.

gross	=	basic	+	HRA	+	dearness allowance
pay	=	gross	-	(PF + ESI + professional tax)		

Only the basic is typed. The allowances and deductions come from each person's own setup in `mcj_staff_components` and are shown beside the row so the arithmetic is visible rather than asserted. The basic is carried from that person's last sheet and labelled as carried, because it moved for 12 of the 19 staff over sixteen months and a figure repeated silently is a figure nobody checks.

Both tables are written. `mcj_salary_sheets` carries the pay and `mcj_payroll_history` the deductions. Writing only the first would leave every future month in the position defect 277 found: a report deriving the deduction from gross minus paid, which is right on 149 of 150 rows and wrong on the one that matters.

Driven on the shop's own people, August 2026:

ANKUSH KHANDELWAL	23,000	basic	40,000	gross	39,850	to pay
ANURADHA MALHOTRA	20,000		35,000		34,850	
MADHURIMA DAS	11,000		19,300		19,170	

Each gross matches what the shop itself recorded for July. The sheets were removed afterwards; 246 and 209 rows, back as they were.

Six tests: the basic carries forward and says so, both formulas hold, the deductions are recorded separately, preparing a month twice leaves one sheet, somebody with no basic is left out rather than paid nothing -- the shop's sheets run 12 to 17 people against 19 codes -- and a basic that cannot be read is refused.

The guard that did not catch the bug it exists for

The first save failed:

```
inconsistent types deduced for parameter $2 (SQLSTATE 42P08)
```

That is defect 232's failure exactly, and there is a test for it --

```
TestSQL_ARepeatedParameterIsCastConsistently -- which passed over this query.
```

Its rule is that a repeated parameter must be cast everywhere or nowhere, and its comment explains why nowhere is safe: *both uses then deduce the same type*. **That is not always true**. Here `$2` was uncast in all three places and deduced two different types anyway -- once as a value being inserted into a varchar column, once inside `btrim(col) = $2`.

Widening the rule to flag every repeated parameter would flag the tenant id in nearly every query in this codebase, so the rule stays and the gap is written into it instead. **What caught this was saving a payroll, not reading the SQL**, and the honest thing is for the test to say so rather than to imply a coverage it does not have.

tables read but never written	21 -> 19
screens	36 -> 37

282. Payroll reconciles completely, and the four unnamed staff are documented month by month

Having built the payroll screen, the question was whether anyone can be paid by it. Twelve of the nineteen staff codes have a components setup; seven do not, and **those seven are exactly defect 157's unnamed codes** -- 4023, 8827, 8828, 8855 and three others.

That looked like a finding and it is not. Payroll reconciles, completely:

sheets with a payroll-history row	209	gross = basic + HRA + D.A on all 209
sheets without one	37	gross = basic on all 37, no allowance to explain
	246	every sheet accounted for

There is no allowance anywhere in sixteen months that lacks a record. The four unnamed codes carry 4,34,200.00 of allowances above basic and every rupee of it is in `mcj_payroll_history`, month by month -- 1,98,000.00 for 4023, 1,02,000.00 for 8827, 59,500.00 for 8828 and 74,700.00 for 8855.

The near-miss

That 4,34,200.00 was measured first against `mcj_staff_components` and written down, in a draft, as *allowances with no setup to explain them*. It would have told the owner that four people no one can identify were paid four and a third lakh of unexplained allowances.

`mcj_staff_components` is a **current** setup, one row per person per component. `mcj_payroll_history` is the **per-month** record. Sixteen months of changing allowances cannot match a single current setup, and the proof was already in front of me: only 61 of 172 sheets belonging to people who DO have a setup match it either. The setup explains almost nobody, which should have said at once that it was the wrong source rather than that these four were unusual.

Eighth time this session a measurement has been taken against a source next to the right one.

It is the first that was caught before it reached the document, and what caught it was checking the contrast -- if the four are anomalous, the twelve should be clean, and they were not.

What is actually true of the four

no name on any sheet
no ledger, in our copy or the legacy one (defect 157)
no components setup, alone among the nineteen
last sheet June 2026, a month before the payroll stops (defect 280)
pay fully recorded month by month, 10,66,400.00 gross, 7,14,430.00 paid

A person who has left would have no current setup, which is consistent. What remains unexplained is only the name -- and that is the question already put to the owner, unchanged. **The money is documented; it is the people who are not.**

283. Payroll could run the months the shop already had and could not take on anybody new

Defect 281 built the screen that prepares a month. It reads each person's allowances and deductions from `mcj_staff_components`, and nothing could write that table -- the component MASTER had a screen (six components, PF through D.A) and the per-person amounts did not.

So a shop could re-run August for the people already on file and could not hire anybody. `/mcj/staff-pay`, What a Person Is Paid, is the other half.

the master's six components, every one offered
which way each goes -- PF, ESI, P.TAX and I.TAX come off the pay,
HRA and D.A go on it
a blank is not a component: 8 of the shop's 12 people pay no ESI,
6 pay no PF

Driven on a real person first -- ANKUSH KHANDELWAL comes back D.A 11,500.00, HRA 5,500.00, P.TAX 150.00 and nothing else, which is exactly his setup -- and then on somebody new, whose four amounts were written, linked to the master, and removed again. 46 rows before and after.

Saving replaces rather than merges. A component taken off the form is one the person no longer gets, and an old row left behind would go on paying it every month. Proven by removing the clearing step: 3 row(s), HRA at 9000.00 — HRA should be gone and D.A raised.

The two halves are tested together

The setup screen writes what the payroll screen reads, so a test drives both: set up a new hire with D.A 8,000, HRA 3,000 and P.TAX 130, then run August on a basic of 20,000.

```
gross 31,000.00 = 20,000 + 8,000 + 3,000
to pay 30,870.00 = less 130 professional tax
```

Payroll now works end to end for somebody who did not exist when the shop's data was captured, which is the only version of it that is any use to a shop that is still trading.

```
tables read but never written 19 -> 18
screens                        37 -> 38
```

284. The unwritten tables that are still unwritten, and why each one stays that way

Six screens were built out of this list in two days -- the coin price list (271), tag correction (274), stones on an order (275), the monthly payroll (281) and what a person is paid (283) -- taking it from 24 tables to 18. This is what is left and why, so the next person does not open each one again.

legacy_access, legacy_departments, legacy_users	the OLD system's own users and departments, not the shop's trade
scheme_enrolments, scheme_payments	blocked on defect 233, which needs the owner refused with reasons
stock_conversions, stock_conversion_pieces	(defect 251): the capture has no input side verification data. 176 of 186 reconcile; the imbalance is B19
carry_forward	party bank details, read by the duplicate-ledger screen. 272 of 398 name no ledger at all
bank_accounts	last written 20 Jan 2017 a rate card. The tag screen takes a stone rate as typed, so this is reference, not input
designs, design_stones	95 rows over 3 items, and 14 of the 27 bands overlap another for the same item
diamond_rates	two rows, May 2025
item_making_bands	52 Thursdays in 2016, 53 in 2017, four days in Dec 2024, then nothing
estimates, estimate_lines	22 lines of printed wording on documents
holidays	9 rows over four months with no year on them
declarations	
sales_targets	

Nothing here is a daily operation the shop cannot do. Two are waiting on the owner, two were refused on evidence, three are the old system's plumbing, and the rest are either reference the screens already read or practices this shop gave up years ago.

The rule that decided each one

Not the table's name. `holidays` reads as an annual chore and is nine years dead; `barcode_edits` read as an audit artefact and turned out to be a missing operation the shop performed about once a working day. The question that separated them was **when was the last row written**, and it costs one query.

`bank_accounts` is the one to watch: 398 rows sounds operational and 272 of them name no ledger, so building a screen over it would mostly be building over rubble. If the shop wants supplier bank details, that is a new screen against clean data rather than a loader for this.

What this does not close

The list only counts tables a REPORT reads. A table nothing reads and nothing writes is invisible to it, and so is a screen the shop needs that has no table at all. Neither is measured here, and neither should be assumed empty.

285. Two crore of stone data that no screen, report or test can reach

Defect 284 ended by naming what its own measurement could not see: *a table nothing reads and nothing writes is invisible to it*. Measured, and it was not hypothetical.

tables in the schema	79
named nowhere in any non-test Go source	2
mcj_conversion_stones	816 rows, 393 conversions, 2,554.341 carats, 2,60,68,682.20
mcj_financial_years	0 rows

816 rows of stones, worth 2.6 crore, loaded by a script and reachable from no screen, no report and no test. It could never have appeared in the unwritten-tables list, because that list's entry condition is that something reads the table.

What they are, as far as can be said

barcode is a tag on file	450 rows	1,075.760 ct	1,38,77,247.10
no tag with that barcode	366 rows	1,478.581 ct	1,21,91,435.10

More than half share a barcode with a tag that exists, so those may be the same stones the stone ledger already values -- and **may not**, because this shop reuses barcodes (defect 29) and a shared barcode is not a shared stone. That is the trap defect 250 fell into, comparing the same gold to itself and calling it conservation, so no total is claimed here.

What can be said is that the stone ledger describes 819 rows on the shelf and these 816 are not those, and that nothing in the application shows them.

They are not built into a screen for the same reason defect 251 refused the conversion screen: a conversion's meaning is not settled by the capture, and showing its stones would mean choosing one. The finding is that the data exists and is unreachable, which was not known.

`mcj_financial_years` is the other kind: empty, unread, unwritten. The finyear package computes years in code, so the table serves nothing.

The check now looks for both

`check_unwritten_tables.py` reports them, and the distinction is the useful part -- loaded-and-unread is data the shop has and cannot see; never-touched is a table that should not exist.

The first version missed the case it was written for. It required the table to have no loader either, so `mcj_conversion_stones` -- which a loader does fill -- did not appear. It printed only `mcj_financial_years`, and the tell was that the 816-row table I had just measured was not in the output. **A fix whose evidence does not show up where you expect it has not been proven**, which is the third time that sentence has been earned this week.

286. A screen could name a real legacy file, be placed nowhere, and appear in no menu

Checking STATUS.md's headline -- **126 of 193 legacy menu screens rebuilt** -- against the source. It is right, and the check found something next to it.

distinct LegacyFile values in the source	132	
the captured menu lists	193	
matched	126	<- the headline, correct
claimed but NOT in the menu	6	

The six are real legacy screens that were never menu entries: `day_book.php`, `label_print.php`, `add_sales_voucher_combined.php`, `print_sales_voucher_register_gst.php`, `add_gold_rate_fix.php` and `set_rate_gold_opening.php` -- print actions and "add" forms reached from inside another screen. The navigation code already knows about this case and says so:

A screen WITH one may still need placing, because plenty of real legacy screens were never menu entries.

Nothing required it. The guard fires only when `LegacyFile` is empty, so a screen naming an off-menu file with no Top and Item was accepted -- and then silently dropped by `placedScreens`, which skips whatever it cannot place. It would render at its URL, pass the render check, pass the navigation test, and appear in no menu at all.

All six declare a position today, so nothing is lost. Nothing kept them right.

Fixed

Register now refuses that case, with the menu read once and cached because it is called 126 times at start-up. Proven by taking the position off `day_book.php`:

panic: navigation: screen "/mcj/reports/day-book" names `day_book.php`, which the captured menu does not list, so it must give Top and Item to say where it belongs

The measurement that nearly became a defect

The first count of this was taken from `links.go` alone: 126 registrations, 104 `LegacyFile` values, therefore -- it seemed -- 22 screens with no legacy counterpart and a headline overstated by that much.

`LegacyFile` is declared in five files, not one: 104 in `links.go`, 18 in `codeLists.go`, 4 each in `creditnote.go` and `voucher.go`, 2 in `bands.go`. 132 in all, and `status_figures.py` reads every one of them, which is why its figure was right and mine was not.

Ninth time this session a measurement has come from a narrower source than the claim it was tested against. The tell each time is the same: my number disagreed with a tool that had been right for a fortnight, and the tool was right again.

287. What the shop still has that this system never took, and two of them read

Defect 285 found data loaded and unreachable. The question on the other side is what was never loaded at all.

legacy tables carrying rows	154
named nowhere in the loaders or the module	53

The largest, and what is known about each:

5,242	design_no_status	design workflow; designs stop Jan 2017
3,965	barcode_image	tag photographs – blocker B24, they live on someone else's server
3,083	barcode_history	tag history
2,816	cla_retail_trans_bar_detail	READ HERE – redundant, see below
1,445	voucher_transfer_check	
1,247	address_book_sub_name	defect 170: 923 of 1,039 codes match nothing, no row names the same person
1,190	user_online_status	defect 68: session housekeeping
1,071	ledger_ref_details	
922	item_adjustment	defect 97: the shape B9 needed, holds nothing usable
756	voucher_hsn	READ HERE – all zeros, see below

The two read this time

`voucher_hsn`, **756 rows, and not one carries an HSN**. 164 receipts and 592 payments, `hsn_no` zero on every row. Receipts and payments have no goods, so there is nothing an HSN could describe. It looked like the most consequential table on the list -- table 12 of GSTR-1 is filed from HSN, and if the shop had recorded it per voucher while this system computes it from items, the two could disagree. They cannot: there is nothing there.

`cla_retail_trans_bar_detail`, **2,816 rows -- exactly the tag count**. One row per tag saying how it came to exist: New Create 2,534, Opening 278, SALE-RET 4. Its Opening count of 278 matches our `OPN-STK` movements exactly, and the fact itself is already held twice over -- `created_from` on 2,810 of 2,816 tags, and the movement trail. Redundant.

The count was wrong first, and by a lot

The first pass said **81**. It tested whether a loader contained the literal string `legacy.<table>`, and the code-list loader does not: it reads a manifest of `mcj_colors|color_master|name|` lines and builds the query. Twenty-eight masters -- colours, shapes, sieves, card types, categories -- were reported as never loaded when they are loaded, and are on the shelf in this application today.

The tell was the list itself. `color_master` at 30 rows and `shape_master` at 16 appearing as "never taken" is not a plausible gap in a system whose stone screens plainly know about colours and shapes. **Tenth time this session a measurement has been narrower than the claim, and the second caught by reading the answer rather than checking the method.**

What this leaves

53 tables, and the ten largest are accounted for: two dismissed here, three dismissed by earlier defects, two are dormant workflow, one is a blocker's subject, and two -- `voucher_transfer_check` and `ledger_ref_details` -- have not been read by anybody. They are the next candidates, and neither is large.

288. The shelf could say who sold a ring; the ledger could not say who posted the entry

Defect 287 left two legacy tables nobody had read. One of them is an audit trail the rebuild was missing. `voucher_transfer_check` , 1,445 rows, 1 April to 19 August 2026: a timestamp, a voucher type and id, and a user.

BAPI	1,186
KESHAV	153
ADMIN	69
RAJENDRA KICHER	10
six others	

`mcj_vouchers` had **no column naming a person at all**, which is inconsistent with the rest of the trail: a stock movement records who moved the piece -- 322 of the shop's 324 captured sale movements do, and the sale screen writes it. So the shelf could say who sold a ring while the ledger could not say who posted the entry for it.

The other table, `ledger_ref_details` , is 1,071 rows of party references with **2 names and 1 mobile number filled in**. Nothing to load.

Done

migration loader	entered_by on <code>mcj_vouchers</code> , '' meaning not recorded load_voucher_authors.sh, joined on (type, legacy_v_id), earliest row per voucher – 10 vouchers name two people and the first is who created it
result going forward	1,416 of 1,848 vouchers now say who raised them all five places that raise a voucher name the current user: a sale, its receipt, a credit note, old gold, and the voucher screen itself
on screen	the register has an "Entered by" column, and says "not recorded" where the shop never attributed it

432 vouchers stay unattributed. That is a fact about the old system rather than a gap to fill, and a blank cell would read as a bug -- so the screen says so in words.

Two of my own mistakes, both cheap

The context was wrong in one of the five. Four voucher literals had `ctx` in scope and the fifth did not, so the build failed on `undefined: ctx` -- caught by the compiler in a second.

The list test looked at the wrong month. The register defaults to the current month and the seeded voucher is dated April, so the table had no rows and no headers, and the test reported "the voucher list has no column for who raised it" when the column was there. The tell was that the page contained *Narration* and *Voucher* -- the other headers -- which cannot happen if the header row is missing. **A failing assertion is worth reading twice when the thing it says is absent has a neighbour that is present.**

289. A year-end snapshot was merged as if it were an event log, so four hallmark events happened twice

The stock trail is built from two legacy logs. The first, `barcode_wise_all_transaction` (8,561 rows), is an event log. The second, `retail_all_stock`, was treated as one too, and it is not: it is partly a **year-end snapshot**.

The tell is in its own dates. Eight of its rows carry a voucher number, and every one of those vouchers belongs to financial year 2024-25 -- yet five of the eight are stamped **1 April 2025**, the first day of the next year:

barcode	movement	log 1 (the event)	log 2 (the snapshot)	voucher
94132093	ISS-HM	2024-10-01	2025-04-01	HMISS/2425/00014
94132093	REC-HM	2024-10-01	2025-04-01	HMREC/2425/00014
45148896	ISS-HM	2025-03-01	2025-04-01	HMISS/2425/00023
45148896	REC-HM	2025-03-01	2025-04-01	HMREC/2425/00022
10864120R1	SALE-RET	2025-03-28	2025-04-01	SR/2425/00058

The voucher number is what proves these are one event and not two: a hallmark issue voucher cannot issue the same tag twice, and `HMISS/2425/00014` names barcode 94132093 once.

The loader skipped a second-log row when the first log already had **the same barcode on the same day**. That rule catches the three rows whose dates happen to agree and lets these five through. So the trail said tag 94132093 was sent for hallmarking and came back in October 2024 *and again* eighteen months later -- and because the phantom copy is the later one, "issued for hallmarking, 1 April 2025" was the **last thing the trail said about a tag sitting on the shelf today**.

It reached the owner. The hallmarking report fetches this count rather than asserting it:

```
"The trail holds 12 hallmark movement(s) in two and a half years, so the
flag is set by hand rather than by the process."
```

There are **8**. The sentence is an argument about how little the shop hallmarks, built on a number 50% too big.

Done

`tools/legacy/load_movements.sh` now skips a second-log row on either of two tests: the first log has the same barcode on the same day, **or** it has the same barcode, movement and voucher number. The second test applies only to rows that carry a voucher number -- opening and "New Create" rows do not, and stay on the day rule, which is right, because the 2024 and 2025 openings are two real events over the same tags.

Trail 9,036 -> **9,031**. `retail_all_stock` no longer contributes any hallmark row; the report now says 8. `SALE-RET` 18 -> 17, and where 17 of 18 named a person, all 17 now do -- the row that named nobody was the phantom.

Invariant rule 15, "one voucher moves a tag once", counts events recorded twice under a single voucher number. It is allowed **18**, which is the legacy's own: those sit twice in the *first* log with two row ids each, because re-saving a bill re-logged its tags -- the eight tags on GST/2627/00150 appear on both 3 and 4 July 2026. That is the shop's history. The rule guards the merge, which is ours. Checked by inserting one duplicate: 19 against an allowance of 18, fails.

Six code comments quoted the old total and two quoted "18 sale returns"; all corrected. One said "every one of the 9,036 movements references a document number", which was never true -- 396 carry a document number, 6,395 say "New Create", 1,770 "Opening" and 470 nothing. Corrected to what the trail actually holds rather than to a smaller wrong number.

290. The same event was spelled two ways, so asking for opening stock missed a year of it

The same two logs disagree about how to spell one movement. The first says `OP-STK`; the second says `OPN-STK`, and its own wording column for those rows reads, in words, "Opening". Every other code -- `NEW-BAR`, `SALE-RET`, `ISS-HM`, `REC-HM` -- is spelled identically in both, so this is the only pair.

Both openings are real: 1,770 tags on 1 April 2024 and 278 on 1 April 2025, a financial year apart. The defect is not the events, it is that the rebuild carried both spellings through and then treated them as different kinds of thing.

`movementLabel` in the stock history screen names nine codes in plain English and `OP-STK` is one of them. `OPN-STK` is not, so it fell to the default, which returns the code. Two consequences, both on the screen an audit uses:

- 278 rows read `OPN-STK` sitting among "sold", "returned by customer", "opening stock".
- The Movement filter is built from the distinct codes in the data, so it offered **both** "opening stock" and a raw "OPN-STK" as separate choices. Asking for opening stock returned the 2024 opening and silently dropped the whole of the 2025 one.

Done

The loader spells it `OP-STK` on the way in, with the reason on the line. Nothing is lost: the `source` column already records which log a row came from, so provenance survives normalisation. This follows the loader's own precedent -- it already upper-cases the user column "so a person appears once in a report rather than twice".

`OP-STK` is now 2,048 rows spanning both openings, and the filter offers one "opening stock" that returns both years.

Invariant rule 16, "the trail speaks words, not codes", counts movements the stock history screen cannot name. Rather than pin this one code, it **reads the case labels out of** `movements.go` and checks the data against them, so a new code from the legacy or a renamed label in the screen cannot drift apart silently. Checked by inserting an `OPN-STK` row: 1 against 0, fails.

The runner described every allowance as "the migration's", which was already untrue of the allowances that come from legacy history rather than from a migration; reworded to "allowed".

291. The audit screen showed every event twice for the one tag an auditor was told to look at

Found by checking the fix for defect 290 against the database instead of accepting the page. Asking the stock history screen for the April 2025 opening returned **280 rows**; the database holds **278** movements matching. Two rows came from nowhere.

The join is the reason. The screen joins the stock table only to name the item:

```
FROM mcj_stock_movements m
LEFT JOIN mcj_stock_items s ON s.barcode = m.barcode AND ...
LEFT JOIN mcj_items i      ON i.id = s.item_id
```

`barcode` is not unique in `mcj_stock_items`, so a movement is printed **once per stock row that shares its barcode**. Exactly one barcode in the shop has two: **86065435** — which is blocker **B8**, the duplicate barcode. Its four movements were drawn as eight rows.

So the one tag an auditor has been told to look at was the one tag whose history was doubled, on the screen whose entire purpose is answering where a piece went.

This is defect 146 in the same file, half fixed. That defect found the same join inflating the count under the list, and the count was changed to `count(DISTINCT m.id)` with a comment explaining why. **The query that draws the rows was left alone.** The guard was put on the summary and not on the thing the summary summarises.

Done

The rows now join `LEFT JOIN LATERAL (... ORDER BY s.id LIMIT 1)`, so exactly one stock row can ever name a movement, and the `ORDER BY m.moved_on DESC` the screen needs is untouched. Which of the two pieces names the item is arbitrary but stable; for a barcode held by two pieces there is no right answer, which is what the caveat row already on this screen says.

The summary count lost its joins altogether. It only ever joined so that `count(DISTINCT m.id)` could undo the duplication the join itself caused, and nothing in the filters touches the stock table — so the join was a hazard being corrected for, and removing it is the correction.

Live: the April 2025 opening now returns 278 rows against the database's 278, and barcode 86065435 returns 4 against its 4 movements. Those four are genuine — two pieces share the barcode and each was counted in both the 2024 and 2025 openings.

`TestStockHistory_ListsAMovementOncePerMovement` seeds two stock rows on one barcode and a single movement, and asserts it is listed once. Checked against the bug by restoring the plain join: *"one movement against a barcode held by two tags was listed 2 times"*.

A second test was written and deleted rather than kept. It asserted the truncation footer did not appear — but with one movement the footer is absent whether the rows are doubled or not, so it could not have failed against the bug. That is the same decoration as defects 266 and 270.

Then the same join was looked for everywhere else

Seven queries join the stock table. One joins on `id` and is safe (`stoneledger.go`). The other six all join on `barcode` , so each would print a row twice for a duplicated barcode:

<code>conversions.go</code> (twice)	<code>mcj_stock_conversion_pieces</code>
<code>salecost.go</code>	<code>mcj_sales_invoice_lines</code>
<code>mrpsales.go</code>	<code>mcj_sales_invoice_lines</code>
<code>barcodeedits.go</code>	<code>mcj_barcode_edits</code>
<code>ordertobarcode.go</code>	<code>mcj_customer_order_lines</code>
<code>stocktake.go</code>	the barcodes a person has just scanned

None of them is affected today. 86065435 appears in no table but the movement trail — 0 rows in conversion pieces, 0 in invoice lines, 0 in barcode edits. They are latent, not live, and no change was made to them.

`stocktake.go` was worth checking by hand because a person can put that barcode in by scanning it. Its filter is `WHERE t.id IS NULL OR NOT t.in_stock`, and both of 86065435's rows are `in_stock` and identical (MALA, 1 piece, ₹900.00, no weight), so a scan of it produces no exception row at all.

What makes six latent joins worth a guard is that a **second** duplicate would make all of them live at once, silently. New invariant rule, "a barcode names one tag", allowed **1** — B8 itself, which is a blocker and not news. Checked by copying a stock row: 2 against an allowance of 1, fails.

The first attempt at that check inserted nothing — the copy collided with a unique index on (`tenant_id`, `legacy_id`) — and the rules reported "17 of 17 hold" over a database that had not changed. Recorded because a self-check that silently verifies nothing is the failure mode of defects 278 and 279, and it happened again here, in the act of guarding against it.

292. The two biggest unread legacy tables, read and dismissed — and B24 turns out to be about three quarters of the shelf

Continuing down defect 287's list of tables nothing loads or reads. These were the two largest left.

`design_no_status` — **5,242 rows. Nothing to load.**

Three columns: a design code, a status of 0 or 1, and one retail point. 5,218 distinct codes, of which 176 carry status 1.

all 5,218 codes already exist in <code>mcj_designs</code>	
status 0	5,042 designs, 68 of them ever became a tag
status 1	176 designs, 1 of them ever became a tag

The flag does not mark designs that were used — 1.3% of the unflagged became tags against 0.6% of the flagged, so whatever it means, it is not that. No legacy screen reads the table; the only file in the capture that names it is `back_up_php.html` , which lists every table in the database.

And the catalogue underneath it is nine years dormant: all 5,221 designs are dated **5 July 2016 to 20 January 2017**, and **none** since the trail begins in April 2024. Loading an unexplained flag over a dead catalogue would add a column nobody could interpret. The 24 codes that appear more than once are all status

O, so there is no conflict to resolve either.

`barcode_image` — **3,965 rows. Nothing to load, and it made blocker B24 worse.**

Three columns: a barcode and a URL. Every URL points at `http://tag09.com/mcj2/images/bar_pic/...` — the third-party host of B24, which answers with a bot-challenge page rather than the images, and which I have deliberately not worked around. So there is nothing here to load: the rows are pointers, and what they point at is not retrievable from a script.

But nobody had measured what the pointers are worth. B24 said "1,023 tags' photographs" and stopped there:

tags with a photograph	1,023
of those, on the shelf today	889
their value	₹10,73,39,445.03
the shelf	₹14,23,61,188.20
	= 75.4%

Three quarters of the shelf by value has its only photograph on a domain the shop does not appear to control. That is not a photo archive of things long sold. B24 and its summary line now carry the figure.

The shelf total was checked against the established baseline rather than taken from one query. `SUM(amount)` over the same rows is ₹12,97,04,187.18 — that is the goods before making and tax, not the shelf, and quoting it would have understated the total by ₹1.27 crore. The figure above is `SUM(total)`, which is the number every other shelf figure in these documents uses.

Done

Nothing loaded, by decision, for both tables. B24 gained the measured stake and its summary row now says 889 tags and 75.4%.

293. The shop could say what its departments were, and nothing could say what any of them did

`barcode_history` was the last of defect 287's unread tables. As an event log it is dead: all **3,083** of its rows match a movement the trail already holds on barcode, movement and day — not one event is missing. Nothing to load.

But it has a column the trail does not, and following that column found something much bigger than the table.

`dept_code` **sits on twenty-eight legacy tables.** Every sale, order, estimate, sale return, purchase return, stock inward, old gold purchase, karigar stock row and conversion records the department that raised it. The department master was loaded — `mcj_legacy_departments`, 12 rows, and there is a Departments screen reading it — but **not one transaction carried a department.** The only columns matching `%dept%` on our side were `mcj_packets.depth_mm` and `depth_percent`, which are a physical depth.

So the shop could say what its departments *were* and nothing could say what any of them *did*.

It is not a formality with one dominant value. On the three header tables:

224 bills	CASH 185, ORDER 19, DATAENTRY 11, SUB ADMIN 5, WORKSHOP 4
284 orders	WORKSHOP 117, SALES 103, DATAENTRY 58, RECEPTION 3, CASH 2, Back Office 1
859 conversions	WORKSHOP 427, CASH 347, ORDER 60, ADMIN 21, +3

Done

Migration `mcj-1787413000.sql` adds `department varchar(8)` to `mcj_sales_invoices`, `mcj_customer_orders` and `mcj_stock_conversions`. The **code** is stored and joined to `mcj_legacy_departments.code`, so a department renamed in the master is renamed everywhere. `''` means the legacy row named none — the same choice as `entered_by` in defect 288, and for the same reason a NULL would confuse "not recorded" with "name missing".

The three loaders fill it. **All 224 bills, 284 orders and 859 conversions are attributed; none is blank, and every code resolves to a named department.** That is unusual in this data and worth stating.

New screen, **What Each Department Did** (`/mcj/reports/department-work`). It walks the master so a department that raised nothing says "raised nothing" rather than being left off — three of the twelve are in that state, including Retail (Sales), which defect-era notes already flagged as the retail face nobody was assigned to.

WORKSHOP	4 bills	3,42,669.00	117 orders	83,87,663.00	427 conv	8,856.236 g
CASH	185 bills	1,77,91,355.00	2 orders	8,494.00	347 conv	3,130.227 g
SALES	–		103 orders	78,01,322.00	–	
ORDER	19 bills	61,06,653.00	–		60 conv	707.776 g
DATAENTRY	11 bills	3,52,209.00	58 orders	1,05,79,231.00	2 conv	9.508 g
	224	2,47,65,386.00	284	2,68,52,217.00	859	12,827.081 g

What the screen refuses to be: a league table. A bill is attributed to the department that RAISED it, and CASH raises most of them because CASH is where bills are made. The reading worth having is the other one — ORDER raises 19 bills averaging over three lakh while CASH's 185 average under one, and WORKSHOP takes 117 of the 284 orders. That is how work flows, not who sells well. The screen says so on its face.

The figures were checked against gates that already existed

Three of the four footer totals are numbers something else already asserts, so they are cross-checks rather than restatements:

224 bills	the invoice loader's own count
284 orders	the order loader's own count
859 conversions	the conversion loader's own count
2,68,52,217.00	the order loader's gate: "estimates tie at 26852217.00"
12,827.081 g	the conversion loader's gate: "net weight ties at 12827.081 g"

The weight total was missing on the first render — the per-department weights were right and the footer showed nothing, because I had not accumulated it. Adding it produced 12,827.081 g, which is what made it a cross-check rather than a fourth unverified number.

Two mistakes of mine, recorded

I measured the wrong tables first. The department splits I originally wrote into the migration comment — "sales CASH 198, ORDER 109", "orders WORKSHOP 230, SALES 83" — came from `sales_vou_main` and `customer_order_main`, which are the **line** tables. The columns I was adding sit on the **header** tables `sales_vou` and `customer_order`, whose splits are 185/19 and 117/103. Caught by noticing that the header counts must add to 224 and 284, which the line figures do not. The migration comment now carries the header figures and says that the counts adding up is how the join key was checked. This is the eleventh time in this project that reading a table adjacent to the right one has produced a plausible wrong number.

A second test I wrote was worth keeping only because I broke the code to check it.

`TestDepartmentWork_CountsDoNotMultiplyEachOther` seeds CASH with 2 bills and 3 conversions. Swapping the scalar subqueries for a naive three-way join makes it fail, which is the only reason to trust it — three one-to-many tables hanging off one department is exactly the shape that produced defect 291, and a report that multiplied 2 by 3 would have looked entirely plausible at 6.

The bigger half: which department HOLDS the stock

Carrying the department onto bills, orders and conversions says who RAISED a transaction. The same sweep found the other half, and it covers the whole shelf.

`cla_retail_trans_bar_detail` carries a `dept_code` on **every one of the 2,816 tags**. `barcode_main`, which the stock loader reads, does not — which is why the column was missed. Deduplicated by barcode and joined to the shelf it accounts for all 2,575 tags in stock and every rupee of the ₹14,23,61,188.20:

WORKSHOP	862 tags	₹3,64,42,394.60
ORDER	799	₹3,67,55,414.59
SUB ADMIN	604	₹4,62,20,787.74
CASH	239	₹1,19,44,679.89
Back Office	66	₹1,07,02,187.42
ADMIN	5	₹2,95,723.96

The most tags and the most value are in different departments. WORKSHOP holds 862 tags worth ₹3.64 crore; SUB ADMIN holds 604 worth ₹4.62 crore; Back Office holds 66 worth ₹1.07 crore, which is ₹1.62 lakh a tag against WORKSHOP's ₹42,000. Nobody could ask that before.

Migration `mcj-1787414000.sql` adds `department` to `mcj_stock_items`; the stock loader fills it from the transfer detail, deduplicated by barcode. **All 2,816 tags are attributed, none blank.** The screen gained "Tags held" and "On the shelf", and is now ordered by what a department holds.

Only one barcode has more than one row in that table — 86065435 again — and its two rows agree on the department, so which one the dedup takes does not matter. That was checked rather than assumed: earliest against latest disagreed on 0 of 2,815 barcodes.

I hit defect 291's own mechanism while measuring this. The first version of the query joined the detail table to the shelf without deduplicating, and reported 2,577 tags worth ₹1,800 too much — the duplicate barcode counted twice, in an ad-hoc query, minutes after fixing the same fault in the code. It was caught by the totals failing to equal 2,575 and ₹14,23,61,188.20, which is the only reason the figures above can be trusted.

What is deliberately still uncarried

`dept_code` is on twenty-eight legacy tables and the department is now on four of the things they describe. The rest are small: `estimate` 2 rows, `purchase_return` 3, `sale_return` 3, `customer_own_goods_receive_detail` 7, `karigar_gold_stock` 2, `old_gold_purchase` 25, `stock_inward` 43 — about 85 rows between them. They are left alone rather than done quietly, and this is the note saying so.

Invariant rule 18, "a transaction's department exists", counts transactions whose code names no department in the master. This matters because the screen walks the master and asks each department what it did — so a bad code produces no error and no row, it simply vanishes. Checked by setting one bill's department to '99': 1 against 0, fails; the bill was then restored by re-running the loader rather than by writing a value back by hand.

294. A loader reported OK while emptying a register another loader owned

Found by a guard rather than by looking: `check_screens_render.py` reported `/mcj/reports/stone-ledger` rendering with no data over the whole of the shop's history, and `check_unwritten_tables.py` reported `mcj_conversion_stones` at **0 rows** where it holds 816. I had done it, ten minutes earlier, by re-running one loader.

Three child tables reference their parent **ON DELETE CASCADE**:

<code>mcj_conversion_stones</code>	<code>-></code>	<code>mcj_stock_conversions</code>	<code>CASCADE</code>
<code>mcj_ordered_stones</code>	<code>-></code>	<code>mcj_customer_orders</code>	<code>CASCADE</code>
<code>mcj_sold_stones</code>	<code>-></code>	<code>mcj_sales_invoices</code>	<code>CASCADE</code>

Each parent loader begins by deleting its own rows, so it silently takes the child's with it. Inside `load_all.sh` that is harmless, because the child loader runs next. Run on its own — which is exactly what I did, three times, to fill in the new department column — it leaves a register empty.

And every gate still passed. The loaders' gates count their own rows: `load_conversions.sh` confirmed 859 conversions and 1,242 pieces and printed OK, while 816 stone rows it had just destroyed went unmentioned. A checker that only audits its own output cannot see what it broke next door.

Damage as found: `mcj_conversion_stones` 816 -> 0, `mcj_sold_stones` 203 -> 0, `mcj_ordered_stones` 115 -> 0. All three restored by running `load_all.sh`, which orders them correctly.

The guard was already there for one of the three, and was too quiet

`load_invoices.sh` already knew. A prior defect had put a check in it:

```
NOTE: reloading the invoices cascaded mcj_sold_stones to empty.  
Run ./load_sold_stones.sh to put its 203 rows back.
```

It printed to stderr and **exited 0**. So it did not stop anything, and I lost it anyway — I ran the loader, filtered the output for `OK` and `FAIL`, and the word NOTE was not in either. A warning that a caller can filter out is not a guard. The other two parents had nothing at all.

Done

All three parents now **put the rows back** rather than report them missing. Each ends by checking the cascaded child and, if it is empty, running that child's loader and confirming it worked:

```
echo "-- mcj_conversion_stones was cascaded away; reloading it --"
"${HERE}/load_conversion_stones.sh"
CH=$(...); [ "$CH" != "0" ] || { echo "FAIL: still empty"; exit 1; }
```

Repairing rather than complaining was the second attempt. The first version failed with the remedy named — which read well and **broke** `load_all.sh`, since the parent now exited 1 before the child it was complaining about could run. That is worth recording: a guard that turns a silent data loss into a hard stop of the whole load is not obviously an improvement. Both are idempotent, so `load_all` running the child again straight afterwards costs nothing.

Verified three ways: each loader run standalone now exits 0 with its child intact (203, 115, 816 rows); `load_all.sh` still completes with `=== all loads complete ===`; and the stone ledger has its data back.

295. The screen I had just built would have lost the first bill raised in the new system

Asked of my own work an hour after finishing it: the What Each Department Did screen walks the department master and asks each department what it did — so what happens to a record no department claims?

It disappears. Not from one column: from the table, because there is no row for it, and from the footer, because the footer sums the rows. The screen would have gone on reconciling to the rupee — 224 bills, ₹2,47,65,386.00, 2,575 tags, ₹14,23,61,188.20 — while quietly leaving trade out.

And it is not hypothetical, because nothing in the app writes a department. A grep across every controller and repository finds no writer: the four INSERTs that create a bill (`sale.go:1513`), an order (`order.go:496`), a tag (`tag.go:427`) and a conversion do not mention the column. Every legacy record is attributed, so the totals are whole *today* — and the first bill made through the new screens would have been invisible on a screen that looked exact.

This is the same shape as defect 288, where the shop recorded who raised a voucher and the app recorded nobody. The difference is that here I built the reader and the gap in the same afternoon.

Done

The screen now ends with a **(no department recorded)** row counting everything the master cannot claim — blank department or a code the master does not have — and adds it to the footer, so the totals are the whole shop rather than the attributable part of it. Today that row is absent, because there is nothing unattributed; it appears the moment there is.

`TestDepartmentWork_DoesNotLoseWhatNoDepartmentClaims` inserts a bill exactly as the sale screen writes one — `department = ''` — and asserts it is listed, that the footer counts 3 bills and ₹7,444.00 rather than 2 and ₹3,000.00, and that the summary still says "2 active, 1 raised nothing" rather than counting the

unattributed row as a thirteenth department. That last assertion caught a real mistake: the summary read `len(out)-idle`, and appending the new row to `out` made it call the row a department. Departments are now counted as they are read.

What is NOT done, and why it is a question rather than a task

Which department should a bill raised in the new system be attributed to?

There is no way to answer it from the data. The legacy user table carries the department on `role_code`, keyed by `login_id`; the app's users are SDK users keyed by email, and nothing links the two. `entered_by` stores the app user's display name, and matching that against legacy names — BAPI, KESHAV, ADMIN — would be a guess dressed as a lookup.

So the options are the shop's to pick, not mine: attach a department to each app user, put a department picker on the billing screen, or decide the new system does not track it and let the column describe history only. **This goes on the list of things needing an answer.** Until then the screen says "does not yet record a department" on the face of the row rather than implying the shop has one.

296. A report about a flag, and nothing in the system can set the flag

Corrected by defect 297. The count below is too low. I decided "written by" with a filename grep, and a file that mentions a column is not a file that writes it — `certificate_no` (678 tags) and `certification_charge` (371) are READ by the sale screen when billing, not written, so they belong in the first table here and are missing from it. The real total is 17 columns, not 11. The hallmark finding itself stands.

Defects 288 and 295 are the same shape — a column the shop fills that the app never writes — so the family was swept for rather than waited for. For every column on `mcj_stock_items`, how many tags carry a value, which screens write it, and which screens read it.

The tag creation screen writes 28 of the table's 51 columns. Most of the rest are keys or timestamps. Three are not.

column	tags with a value	written by	read by
hallmarked	432	NOTHING	Hallmark Records
hallmark_charge	76	NOTHING	Hallmark Records, Quotation
wastage_percent	26	NOTHING	Quotation

`hallmarked` appears in exactly one non-test file in the whole codebase — `reports/hallmarking.go` — and every appearance reads it. There is no INSERT, no UPDATE, no form field. The column is `NOT NULL DEFAULT false`, so **every tag created through this system is permanently recorded as not hallmarked, and there is no screen on which anyone can say otherwise.**

The report was saying so, and saying it wrongly

Hallmark Records carries a caveat, and the caveat is the honest part of a ₹11.5 crore headline. It read:

"...so the flag is set by hand rather than by the process."

That was true of the old system. In this one the flag is not set by hand either, because no hand can reach it. The sentence invited the owner to think a clerk could fix the record, and nothing could.

wastage_percent and hallmark_charge have a quieter version of the same problem: the Quotation report reads both off the tag, so a quote for any tag created here silently omits a wastage percentage and a hallmark charge, and looks complete.

Done

Both notes on Hallmark Records now say no screen can set the flag, and the doc comment says the same. The missing > 0 note reads:

"...and NO SCREEN HERE CAN SET THE FLAG – so this is a report about a record that has stopped being kept, not about the pieces."

TestHallmarking_CarriesTheCaveatWithTheNumber asserted the note contained "set by hand" and **failed when the wording was corrected — which is the test working**. The assertion was moved to the new claim rather than deleted: what it exists to prevent is the headline travelling without a caveat, and that is still what it checks.

Not fixed, because it needs a decision rather than a screen

A hallmarking screen is buildable — the document series pattern (HMISS/2425/00014) is the same one four other screens already use. What stops me choosing is that **the shop has recorded 8 hallmark movements in two and a half years, over 4 tags**. Against 432 flagged tags, that says the flag was set somewhere other than the movement trail, and probably not as part of any workflow. Building a send-and-receive screen on that evidence would be inventing a process rather than restoring one.

So the question is the shop's: is hallmarking recorded in the ERP at all, or on paper and in the BIS portal? **This goes on the list of things needing an answer**. Until it is answered, the report says plainly that the record has stopped rather than implying somebody is keeping it.

The other half of the sweep: columns loaded and read by nobody

The same measurement found eight columns carrying legacy data that **no screen writes and no screen reads** — invisible in a different way from defect 285's unreachable tables, because these sit on a table five screens do use:

certification_type	2,801 tags	by_val 2,683, per_ct 118
lot_no	2,059 tags	59 distinct lots
making_sales	1,825 tags	
remarks1	451 tags	
certification_rate	371 tags	
design_code	96 tags	
order_sl	10 tags	
remarks2	2 tags	

`certification_type` is on 2,801 of 2,816 tags and says whether stone certification was charged by value or per carat — the companion to `certification_rate` and `certification_charge`, the second of which the sale screen does write. So the shop's tags record how certification was priced and nothing in the rebuild reads it.

None of these is fixed here. They are recorded so the next screen built on this table knows what is already sitting in it, which is the same reason defect 285 exists. `legacy_user_id` (2,808 tags) is deliberately excluded from that list: it is a legacy key, not shop data.

297. The receipt in the customer's hand said how the money moved; the one this system prints did not

Defect 296 swept one table by hand and got part of it wrong. This is the tool that does it properly, what it found, and the correction it forced.

The correction to defect 296 first

296 said three columns on `mcj_stock_items` were read by reports and written by nothing, and eight more were touched by nobody at all — eleven in total. The real number is **seventeen**, because I decided "written by" with a filename grep, and a file that mentions a column is not a file that writes it.

<code>certificate_no</code>	678 tags	I credited sale.go. sale.go READS it (line 1246).
<code>certification_charge</code>	371 tags	likewise.
<code>ref_no</code>	260 tags	I credited creditnote.go. Its INSERT at line 805 names ref_no on a DIFFERENT TABLE.
<code>order_no</code>	11 tags	likewise, order.go and partysample.go.
<code>size</code>	99 tags	the readers I found were the word "size" elsewhere.

`certificate_no` **and** `certification_charge` **matter more than the three 296 did report.** The sale screen reads both off the tag when billing, so a certified piece created in this system carries no certificate number and no certification charge onto its bill, and looks complete. That belonged in 296's first table and was not there.

The tool

`tools/check_unwritten_columns.py` asks the question 288, 295 and 296 each answered by hand: which columns did the shop fill that no screen here can fill again. It does not grep for the column name. It parses the SQL in non-test Go — the parenthesised list after `INSERT INTO <table>`, and the assignments in an `UPDATE <table> ... SET` — so a column counts as written only if a statement against **that** table names it. Its own docstring lists what it cannot see: runtime-built SQL, and whether a read exists (the "read" column is a grep, and says so).

The tool's first output was wrong, in the way its own note predicted

It reported 44 columns on 12 tables, and seven of them were `mcj_ledgers` — `account_code` and `display_name` on all 785 rows, `country` on 598, `pan` on 177. I wrote that down as "a party created here can be given neither".

A clerk can type into every one of them. `mcj_ledgers` has a full create-and-edit screen built on `pkg/crud`, which builds its SQL at runtime, so there is no INSERT in the source for the parser to find. The docstring warned about exactly this and I believed the output anyway.

The fix was two attempts. Matching `crud.New*Field("col")` was not enough, because each aggregate wraps those in helpers of its own naming — `text("size")`, `pctField("making")` — so `mcj_sale_rates` was still reported as unwritable when every rate on it is editable. **The authority is the aggregate's** `mapper.go`, whose write map names exactly the columns the generic repository sends to the database. That is now what the tool reads.

And the first version silently skipped any table it found no INSERT for, on the reasoning that a loader-only table is another tool's question. That hid whole tables: `mcj_packets` has an editing screen whose mapper writes 6 columns and a loader that fills 27, and none of the difference was reported.

Corrected: 58 columns on 13 tables. (Defect 298 then wrote two of them — a stone's shape and colour — so the tool now reports **56**.) The largest:

```
mcj_stock_items    department 2816, certification_type 2801, lot_no 2059,
                  making_sales 1825, certificate_no 678, remarks1 451,
                  hallmarked 432, certification_rate 371 ...
mcj_voucher_lines  cheque_date 680, cheque_number 593
mcj_stock_stones   shape_id 555, colour_id 416, packet_id 107, cut_id 28
mcj_packets        21 columns – the whole solitaire grading detail: depth %,
                  table %, crown and pavilion angles, girdle, polish,
                  symmetry, fluorescence, Rapaport price
mcj_sales_invoices department 224, party_mobile 196, salesperson 131
```

What the cheque turned out to be

The tool pointed at `mcj_voucher_lines.cheque_number` and the first reading of that was wrong too — `voucher.go` does take a cheque number from its form. It writes it to the **header**, `mcj_vouchers`, and the shop keeps it on the lines:

vouchers recording how the money moved	554
...on the lines only	520
...on both	34
...on the header only	0

`reconcile.go` already reads the lines and falls back to the header. **The printed voucher read the header only**, so on 520 of 554 — 94% — it printed no reference at all, while the counterfoil the customer kept had one. A receipt is signed by the person handing over money; this is the field that lets them check it later.

And the column is not really a cheque number. Of its 593 values:

looks like a cheque number	215	five to eight digits
bank transfer	112	RTGS, NEFT, IMPS
UPI	85	GPAY, PAYTM
something else	181	"Direct", "IMPSAB", ...

Done

`voucherprint.go` now falls back to the lines for both the reference and the date, as the reconciliation screen already did.

The wording is decided in the controller, not the template, because **"Cheque GPAY" is not a sentence**. `paymentInstrument` prints "Cheque 416710" for a run of five to eight digits and otherwise prints what the shop wrote, on the grounds that the shop's own word for it beats a guess at a category. The template's field is named `Instrument` rather than `ChequeNumber` so the next person reading it knows the column's name is not its meaning.

Proven on the running app against real vouchers: PMNT 6 and PMNT 8, whose lines carry 416710 and 416709, print "Cheque 416710" and "Cheque 416709"; JRNL 1, whose line says Gpay, prints "Gpay" and not "Cheque Gpay".

Two tests, both checked against the old behaviour by reverting the query to header-only: *"the receipt does not show the cheque its own line records"* and *"the receipt does not show how the money moved"*.

Still open from this sweep

Nothing else here is fixed. The 58 columns are now listed by a tool that can be re-run rather than by a paragraph that goes stale, and the two biggest untouched groups are the stone attributes — shape on 555 rows, colour on 416, so a stone-set tag made here cannot record what its stones are — and the packet grading detail, 21 columns describing solitaires that the packet screen cannot enter.

`mcj_stock_items.department` heads the list at 2,816 and is defect 295's open question, not a new one. Two columns are worth nothing and are named so nobody spends time on them: `mcj_ledgers.display_name` is a 50-character truncation of `name` (784 of 785 identical, the exception being SONANI JEWELS UNIT OF SONANI INDUSTRIES PRIVATE LIMITED cut short), and `account_code` equals `code` on 784 of 785 — the exception is 1034 SALES GST, whose nominal account number is 34. Those two were the ones I had called the next thing to look at.

298. Pricing a tag threw its stones away, and the price kept their value

Taken from defect 297's named-not-fixed list: the stone attributes. Fixing them uncovered something worse sitting underneath.

What was asked for

`tag.go` writes twelve columns into `mcj_stock_stones` and omits five. A stone added through the tagging screen recorded pieces, carats, weight and rates — and nothing about **what the stone is**. The Sold Stones register joins shape and colour to name them, so a stone added here would sit in that register with two blank columns.

The shop's own rows decide the scope:

shape	555 of 819 stone rows
colour	416
cut	28
sieve	0

Shape and colour are ordinary; cut is on 3% and sieve on none. **Only shape and colour were added.** A form field nobody fills is a field everybody has to skip, and the honest place to record that judgement is beside the code — which is what the comment above `stoneShapeOptions` now says, including that cut belongs there next if it starts being recorded.

Both are optional and never refuse a tag: the shop leaves them off a third of its own rows, so demanding them would stop work the old system allowed.

What was found on the way

The tag screen **prices before it saves.** A post without `action=save` re-renders the form with the price on it, and the clerk then presses Save — which posts whatever the form holds at that moment.

`Props.Stones []StoneRow` was declared, read by the template on every row, and **never assigned anywhere in the controller.** So the grid came back empty, and the second post carried no stones at all.

The comment above `readStones` is what makes that serious, and it was already in the file:

```
"Every one of the 486 tags on the shelf whose price includes a stone value
has its stones itemised -- 486 of 486 -- and the stone registers read the
stones, not the figure. A tag with a stone value and no stones would be the
first in the shop's history."
```

Pricing a stone-set tag and then saving it made exactly that tag: a stone value in the price, and no stones behind it. The invariant was written down, the screen broke it, and nothing checked.

Done

`stoneRowsFromForm` hands the grid back on every render. It returns the **raw** strings rather than the parsed ones, so a carat weight that could not be read comes back as typed for correction instead of vanishing — the same principle as the rest of the form.

`stone_shape` and `stone_colour` are parsed, carried on `tagStone`, and written to `shape_id` and `colour_id`. `check_unwritten_columns.py` now reports only `packet_id` (107) and `cut_id` (28) for that table, which are the two the evidence said to leave.

Three tests. `TestTag_KeepsTheStonesWhenItPricesTheTag` was checked against the bug by removing the one line that fixes it: *"the priced form has lost the stone that was chosen / its carats / its rate"*, all three.

Two mistakes of mine in one test

The test passed by skipping. The first version looked up an item named DIAMOND and called `t.Skipf` when it found none — and nothing seeds one, so `go test` printed `ok` and I had proved nothing. Caught only by re-running with `-v`. That is defects 270 and 278 again — a check that cannot fail — in the one new place I had not looked, and the skip made it invisible rather than merely vacuous. The helper now seeds the item, and a stone row referencing it, because the screen offers only items some stone or purchase already used.

One of the three assertions passed for the wrong reason. Checking that the response contained the stone's id proved nothing: the id appears in the options list whether or not the row remembers which was chosen. It now asserts `value="<id>" selected`, and only then did it fail against the bug.

299. A guard that had never once run, found by asking whether defect 298's mistake had siblings

Defect 298 ended with a test of mine that passed by skipping. The obvious next question is whether anything else in the suite does that, and one thing does — and it is not a test I wrote.

```
$ grep -rn "t.Skip" modules/ --include="*_test.go"
navigation_links_test.go:136   the captured legacy pages are not on disk
navigation_test.go:137         no screens registered in this test binary
```

The first is honest: it skips only if the capture is missing. The second **skips every single time**.

`TestEveryRegisteredFileExistsInTheLegacyMenu` lived in the navigation package and read the screen registry directly. Screens register themselves from `modules/mcj`, so that package's own test binary has an empty registry — the test hit its guard, printed `no screens registered in this test binary`, and reported PASS on every run since it was written.

Its comment records that it caught a real fault once: screen 17 registered against `category_master.php`, which is not in the menu at all — the address category screen is `category_list.php`. It has caught nothing since, silently, and its own comment even says *"An earlier version of this test hardcoded a single filename and so proved nothing"* — the second version proved nothing in a new way.

Done

`InLegacyMenu` is exported, and the assertion now lives in `modules/mcj/navigation_links_test.go`, which imports `mcj` so `links.go` runs and the registry is real. It **fails** where the old one skipped:

```
if len(registered) == 0 {
    t.Fatal("no screens are registered, so this test proves nothing")
}
```

The old location keeps a comment saying what was there, why it could not work, and not to move it back.

It reports its own coverage rather than passing quietly: **126 registered screens are in the legacy menu; 6 name a real legacy file that was never a menu entry and are placed by hand**.

My first revival asserted the wrong rule and "found" six faults

Revived as written, it failed immediately on six screens — the day book, the GST sales voucher register print, two rate screens and two add forms.

They are all deliberate, and the code says so where I had not read it:

```
// A screen WITH one may still need placing, because plenty of real legacy
// screens were never menu entries -- print actions and "add" forms reached
// from inside another screen. Both declare Top and Item.
```

The rule is **in the menu, OR placed by hand with Top and Item** — which is what `Register` enforces at start-up. Had I trusted the revived test I would have logged six defects that are not defects. The assertion now mirrors the real rule, and the count of each kind is printed so the split stays visible.

The failure path is not guesswork: it was exercised, by that wrong first version, which is how I know the loop reports rather than passing over.

No test in the suite skips any more. Running the whole of it with `-v` and counting `--- SKIP` gives **0**, which is the figure that says this defect is closed rather than moved: the only unconditional skip is gone, and the one conditional skip left — the legacy capture being absent from disk — did not fire, because the capture is there.

300. Nothing tagged in this system could ever be a certified piece

Next from defect 297's list: the certification columns on `mcj_stock_items`. Following them found that the missing fields are not descriptive — they choose how a piece is billed.

Certified is a route, not a label

`sale.go` decides it in one line:

```
if certificateNo != "" || certChargeP > 0 {
```

A certified piece is **not priced by the formula at all**. The screen refuses to compute and asks the biller for the price agreed across the counter, and defect 228 is why: of the 129 certified pieces the shop has sold, exactly ONE went out at the tag's stored price and NONE at a computed one — they went at 0.44 to 3.47 times the tag, with 125 of the 129 lines carrying a different metal rate and 107 a different making.

certified for billing	699 of 2,816 tags
...of those, on the shelf	566, worth 2,84,75,794.50
carrying a certificate number	678
carrying a certification charge	371, 2,12,569.15 in total

The tag screen wrote neither column. So a diamond piece with a certificate, tagged in this system, would have been quietly priced by a formula the shop does not use for it — and the biller would never have been asked. The refusal that protects a quarter of the shelf was unreachable for anything new.

This is the sharpest example yet of the defect-297 family. `hallmarked` (296) made a report untrue; this one changes what a customer pays.

Done

`certificate_no` and `certification_charge` are on the tag screen, written by the tag INSERT, and carried on `tagDraft`.

Either field alone makes a piece certified, exactly as the sale screen reads it, so neither is inferred from the other and both are optional. A charge that cannot be read **is** reported as a problem — unlike a stone's shape in defect 298, which is dropped silently, because this one changes the price.

`certification_rate` (371 tags) and `certification_type` are deliberately left out. The type is on 2,801 of 2,816 tags including plain gold — `by_val` on 2,683, `per_ct` on 118 — so it is a default rather than a property of certified goods, and the rate only explains how the charge was reached. The bill uses the charge.

The test is cross-screen, because a one-screen test could not have found this:

`TestDaysTrade_ACertifiedPieceIsBilledAtAnAgreedPrice` sets the day's rate, tags a piece with a certificate through the tag screen, and then puts it on a bill — asserting the sale screen asks for the agreed price. Checked against the bug by making the INSERT write an empty certificate: *"the certificate number was not recorded"*.

The assertion I nearly got wrong again

The first version asserted the response contained the word `"certified"`. It appears in the screen's own explanatory text whether or not this line took that route, so it would have passed against the bug — the same mistake as defect 298's options-list assertion, two defects later. It now asserts the whole phrase the refusal actually prints: *"enter the price agreed with the customer"*.

301. The biller chose a salesperson and the invoice showed none

Continuing the defect-297 sweep into `mcj_sales_invoices`. This one is not an unwritten column — it is two columns for one fact, with the writer using one and the reader the other.

```
all 224 captured bills    name a salesperson in the TEXT column
                        and carry NO salesperson_id
every bill raised here    carries a salesperson_id
                        and leaves the text EMPTY
```

`sale.go` offers a list from `mcj_salespeople` and writes `salesperson_id`. The tax invoice reads `i.salesperson`, the text. So the invoice printed **no salesperson at all** on any bill raised in this system — the template's `if inv.Salesperson != ""` simply never fired — while the biller's choice sat in the next column along.

The bill a customer is handed is where this shows. "Served by: ..." is on the invoice because the shop puts it there on all 224 of its own.

Done

The invoice resolves the name from whichever column holds it, falling back from the text to a lookup on `mcj_salespeople`:

```
COALESCE(NULLIF(btrim(i.salesperson), ''),
          (SELECT sp.name FROM mcj_salespeople sp WHERE sp.id = i.salesperson_id), '')
```

The reader was changed, not the writer: the id is the better record — it references a real table rather than repeating a name — and rewriting history into the text column would lose that. This is the same shape as defect 297's cheque, where the print learned to read where the shop actually keeps the fact.

Nothing else reads that column: a grep across every report and controller finds only this one, now fixed.

Two tests, one for each shape, and both were checked against the old code — which **fails the new-bill test and passes the captured-bill test**, exactly as it should, showing the fix restored one without breaking the other.

The test failed for the wrong reason first

Both tests failed on the first run, including the captured-bill one that the old code should have passed.

`invoiceSuite` builds the harness and does **not** seed; seeding is a separate `seedInvoice` call every other test in the file makes, and I had not. So the UPDATE matched no rows and the assertion failed against an empty page.

Had only the new-bill test failed I would have believed it and moved on with a fix that happened to work. The tell was the test that should have passed failing too — a fix's evidence appearing where it should not is the same signal as defect 285's, where evidence failing to appear where it should was the tell.

The setup now checks itself before asserting: it counts the bills wired the way the sale screen writes them and fails with *"no bill is set up the way the sale screen writes one"* if there are none.

302. The GST return's two halves disagreed about whether to say which tax applied

Swept for defect 301's shape — one fact in two columns — and found only one other pair,

`mcj_staff_components.component / component_id`, which is fine: all 46 rows carry both and `staffpay.go` writes both. So I went looking at the tax basis instead, and the first thing I found was not there.

What I claimed, and why it was wrong

The sale screen lets the biller override the tax basis. The override reaches the voucher — an overridden bill posts to IGST — and the sale writes `place_of_supply` as the party's state, so the bill keeps no record of the decision. The invoice computes an `Interstate` flag from `place_of_supply` against the shop's own state.

From that I concluded the printed invoice would show CGST and SGST while the ledger held IGST, wrote a test asserting the two disagreed, and it failed exactly as predicted:

```
the books charged IGST but the bill records place_of_supply="West Bengal"
against the shop's "West Bengal", so the invoice will print CGST/SGST
```

The invoice prints neither. `readInvoiceTax` takes the tax from the VOUCHER, by ledger, so the invoice already agrees with the books — that is what `TestInvoice_TakesTheTaxFromTheBooks` has been asserting all along. My test proved that a derived boolean was false; it never checked what the invoice prints, so it could not have found a contradiction that does not exist.

The `Interstate` field it derived was assigned to the invoice's props and **used by nothing** — a non-trivial SQL expression with a comment explaining why it mattered, feeding a field no template reads. It looked load-bearing and was not, which is how I came to believe it.

What is actually true

GSTR-2, purchases	prints a "Tax basis" column -- CGST and SGST, or IGST --
	reading <code>mcj_purchase_invoices.interstate</code> , a stored column
GSTR-1, sales	printed no basis at all, because the sales side stored
	nothing to print

The two halves of one return disagreed about whether to say which tax applied. And the sales side could not have said it: the biller's decision was recorded nowhere, so even deriving it would have been wrong for any bill that was overridden.

Done

`mcj_sales_invoices.interstate`, **nullable on purpose**: the 224 captured bills have no such flag and must keep deriving it, which they do correctly — 18 are Bihar against a West Bengal shop. NULL means "not recorded, work it out"; true or false means somebody decided.

The sale records it. GSTR-1 now prints a Tax basis column exactly as GSTR-2 does, preferring the stored flag and falling back to place-of-supply against the shop's state. On the shop's own data: **206 CGST and SGST, 18 IGST**, which is the Bihar count exactly.

The unused `Interstate` field is gone from the invoice, along with the SQL that fed it and the now-pointless join to the company table. A field that looks like it decides something and decides nothing is worse than no field.

`TestSale_RecordsTheTaxBasisItWasBilledOn` asserts the bill records the basis it was billed on, checked against the bug by reverting the write: *"the bill does not record which basis it was raised on"*.

Two things this cost, both worth recording

I built the fix before I checked the claim. The migration and both code changes were written against the invoice-contradicts-ledger story, and only when I went to prove the invoice printed the wrong tax did I find it prints the right one. The column survived because the real finding needed it too; the invoice changes were reverted. Writing the test first would not have helped here — I wrote one, and it passed against a premise I had not checked.

The test skipped, and the skip hid a broken form. Its first version called `t.Skip` when the override had not reached the books, and it took that branch — not because the override failed, but because my form was missing nothing at all and instead had too much: `pay_mode` and `pay_amount` with no `pay_account`, which made the save fail. The skip turned a broken test into a quiet one, three defects after 299 was logged for exactly that. It is now a `t.Fatal`.

303. Five fields computed for screens that never showed them, and a comment that said the opposite of the code

Defect 302 turned on a props field that was computed, assigned, documented, and read by nothing. Go does not warn about that — an unused struct field compiles cleanly — so it was worth asking how many others there are.

Every field on every page struct, checked against whether the package renders it:

<code>taglabel</code>	<code>Label.Total</code>	a price, formatted and never printed
<code>taglabel</code>	<code>Geometry.FoldAt</code>	where the tag folds, parsed and unused
<code>stocktake</code>	<code>Result.MatchedCount</code>	computed and never shown
<code>voucherprint</code>	<code>Voucher.ShopState</code>	read from the company record, unprinted
<code>partysample</code>	<code>ReceiveProps.CanSave</code>	declared, never set, never read

The one that mattered was a comment

Chasing `FoldAt` led to the tag label's header, which said of the shop's stored print setup:

```
"That is the shop's own setup and is used as it stands."
```

It is not. **Only the type size is applied** — six point, through `labelStyle`. The margins, panel positions, fold centres and printer type are printed on screen as a sentence describing the setup, and nothing is laid out by them.

That matters because of how blocker B3 closes: somebody holds a printed label against a real tag. Doing that against this claim, they would measure margins the screen never applied and conclude the reconstruction was wrong.

The neighbouring helper has always said it correctly — *"The rest of the geometry is shown to the user rather than applied blindly, because the unit those numbers are in is exactly what blocker B3 is still missing"* — so the two comments in one feature disagreed, and the wrong one was in the header, where a reader arrives first. The header now says what the code does and why.

`FoldAt` is deleted rather than applied, with `centreX` that fed it. Not applying it is the same deliberate decision the helper documents; a parsed value sitting unused only suggests somebody forgot.

The rest

`Label.Total` **is deleted, and the query no longer reads the price**. A price on the tag would be wrong here rather than merely absent: this shop bills at the day's metal rate, not the tag's stored figure, and of 129 certified pieces sold exactly one went out at the stored price (defect 228). A number on the tag is one the customer can reasonably expect to pay. Whether a real tag carries one belongs to B3, and the struct now says so instead of implying an answer.

`MatchedCount` **is now shown** — a fifth figure on the stock take, "Found: on the books and scanned". It is derivable from two of the others, which is presumably why nobody noticed, but making a stock-taker subtract is not the same as telling them.

`ShopState` **is now printed**. The address stops at the city — "25A, CAMAC STREET, KOLKATA-700016" — so the receipt named no state at all, while the value sat fetched and unused.

`CanSave` **is deleted**. No controller ever set it and no template read it.

The sweep was wrong the first time, in the same way twice

Its first run reported eight fields. Two — `Line.Pieces` and `Line.Number` on the invoice — are used through methods, `PiecesText()` and `NumberText()`, which a search for `.Pieces` does not see.

Worse, I read the templates for the tag label's geometry, found every field appearing only inside one descriptive sentence, and concluded that **none** of the shop's print setup was applied. That was about to become the finding. The font size is applied — by `labelStyle(g)`, a helper in a plain `.go` file in the templates package, which my search of `.templ` files never opened.

Both misses have the same shape as defect 302's: a value reached through something other than a direct field reference. The sweep now reads the package's `.go` helpers alongside its templates, and treats a method of the field's name as a use. Re-run, it reports one field — `Voucher.Kind`, which the controller uses to choose the voucher's label and is legitimately never rendered.

And the verification was wrong before the code was

Checking the receipt afterwards, the shop's state was still missing. The change looked to have failed.

The server I was reading had not been started by me. A new one had been launched in the background and had died immediately —

```
bind: address already in use
```

— leaving an older process serving the port and answering with code from before the edit. Nothing said so: the launch was backgrounded, so its exit was silent, and the page came back 200 with the old markup. **A false negative from talking to the wrong process**, and had I trusted it I would have "fixed" a change that was already correct. Restarted properly, the receipt reads MANIK CHAND JEWELLERS / 25A, CAMAC STREET, KOLKATA-700016 / West Bengal / GSTIN.

Clearing it out cost something too: `kill -f wingold` matches the compiled test binaries as well as the server, so it killed the full suite that was running in the background, which exited 144 and had to be started again.

304. "Set Aside" could only ever mean nothing was set aside

Following the unwritten columns into the order book. `mcj_customer_order_lines` carries a `barcode` on 59 of its 366 rows — the piece put by for the customer who ordered it — and the order screen writes 12 columns, not that one.

Worse than unwritten: **no UPDATE of an order line exists anywhere in the codebase**. A line is inserted with a status and stays that way for ever.

Cancelled	214 lines	0 name a tag
Set Aside	82	58 name a tag
Process	33	0
Urgent	29	0
On Hold	5	0

`Set Aside` is the status that means a piece is reserved. The Order to Barcode screen already flags the gap in the captured data — *"24 are Set Aside with nothing set aside"* — and this system could produce **nothing but** those: it can write the status, because status comes off the form, and it can never write the barcode that gives the status its meaning.

The outstanding-work screen made it plainer still. It **shows** the reserved tag in a column of its own, so it displayed a fact that nothing in the system could put there.

Done

A tag can now be set aside from the outstanding list — a box on each row, which writes the line's `barcode` and moves it to `Set Aside`.

Only where it is unambiguous. A piece is reserved against a LINE and that screen is per ORDER, so an order with one live line takes an input and an order with several shows its line count instead. **74 of the 95 orders the screen lists have exactly one live line, 11 have several, 10 have none.** The refusal is on the screen rather than in a message after the fact.

The tag must be on the shelf. Reserving a piece already sold would put by something that cannot be handed over, and the screen says which of the two is wrong: *"There is no tag X"* against *"Tag X has already left the shelf"*.

Four tests, and the first was checked against the bug by reverting the SET clause: *"the line names "", want RESERVE1"*.

The screen refused its own new feature

Setting a piece aside without handing anything over failed, and the tests said so before I noticed anything else. The handler opened with a guard — no ticks, no work — and returned before reaching the reservation. That was right when the screen did one thing and wrong the moment it did two.

Handing over and setting aside are now both whole actions, and either alone is accepted. The existing `TestOrder_HandingOverNothingIsRefused` asserted the old wording *"Tick the orders"*; its assertion moved to the new refusal rather than being deleted, because what it exists to prevent — an empty submission taken for work — has not changed.

Two figures and one server, all wrong

The figure I put in the code was counted from the wrong population. I wrote "190 of the 243 outstanding orders have exactly one line", from every undelivered order. This screen deliberately leaves off orders whose lines are ALL cancelled, so the number describing it is **74 of 95** — which the page then confirmed exactly, 59 rows offering an input plus 15 already reserved. Counting the wrong population is how a comment comes to describe a screen nobody is looking at.

And I read a stale server again, one defect after logging that lesson. The page showed no input boxes at all. The launch had died on `bind: address already in use`, because `pskill -f cmd/wingold` does not match the temporary binary `go run builds`. Killing by port — `lsof -ti tcp:3210` — is what actually works, and the fresh server showed all 59. Twice now the same false negative, and both times the tell was in a log I had not read before trusting the page.

305. An order could be taken and handed over, and never cancelled

The other half of defect 304. Having found that no UPDATE of an order line existed, the question is what else that made impossible.

Cancelled is the commonest status the shop records — 214 of its 366 lines, more than the other four together — and nothing in this system could set it after an order was taken. The order screen offers it in the status list, which only makes it stranger: a status you can choose when taking an order and never afterwards,

for the one thing that always happens afterwards.

The consequence is on the outstanding list, the screen that says what the shop owes. An abandoned order stays there for ever: it is not delivered, its lines are not cancelled, and there is no third thing to do. **The only way to clear it was to tick Delivered — recording a hand-over that never happened**, in the register a customer's claim would be checked against.

Done

A Cancelled tick beside the Delivered one. It sets every live line on the order to Cancelled, and the outstanding list drops an order once no live line remains — so the order leaves the screen without anybody claiming it was collected. The test asserts exactly that: lines cancelled, order gone from the list, and `delivered` still false.

Order level, unlike setting a piece aside in defect 304: cancelling an order cancels everything on it, so there is no line to choose between and no need to refuse the multi-line case.

Two refusals rather than silent nothings:

- **An order cannot be handed over and cancelled in one submission.** Both boxes are on the same row and both are ticked by hand.
- **An order with nothing itemised cannot be cancelled at all.** The status lives on the line and there is none, so it says so instead of reporting success having changed nothing. Ten of the 95 orders on this screen are in that state, and that is a limit of the data model rather than of the screen: there is nowhere to record the cancellation of an order that itemises nothing.

What this does not settle

Whether a 58.5% cancellation rate is normal, or whether the status is used for tidying up, is still a question for the shop — it is in *Waiting on you* and this does not answer it. What changed is that the rate can now be a record of what happened rather than an artefact of the only status anybody could reach.

The order line's `making` column was examined on the way and left alone: 122 rows carry it, the same 122 that carry `making_amount`, and **nothing reads either one**. It is not a rate the amount is derived from — `making * net_weight` matches the amount on 8 of the 122 — and the order screen takes both making and estimate as figures the biller types, which is consistent with a shop that agrees prices rather than computing them (defect 228). No formula was reconstructed, because the data does not support one.

306. A bill returned in full still printed as a clean sale

Generalising defect 304's discovery. If no UPDATE of an order line existed, what else can this system create and never change? Every table it writes:

19 of the 24 tables the app writes to are create-only.

Most are right. `mcj_barcode_edits` and `mcj_stock_movements` are audit trails and must be append-only; the rate tables are deleted and rewritten. `mcj_ledgers` appears only because its screen is `pkg/crud` and builds its SQL at runtime — the same blind spot defect 297's tool has, met again in a different tool.

`mcj_sales_invoices` is the one that matters. It carries two flags:

```
is_cancelled    0 of 224 bills
is_returned     3 of 224 bills
```

Nothing in this system writes either. The tax invoice prints a banner from each — `CANCELLED`, `GOODS RETURNED` — across the top of the sheet the customer is handed, and six reports filter on `is_cancelled`. A bill returned against through the credit note screen printed as an ordinary sale.

The credit note does everything else: puts the tag back on the shelf, writes the `SALE-RET` movement, raises the purchase-side documents. It never touches the bill.

The shop's own flag does not settle how to set it

```
everything came back    4 bills    2 flagged
part came back          1 bill      0 flagged
nothing came back       219 bills    1 flagged
```

Of the four bills where every piece came back, **two are flagged and two are not**. The single partial return is not flagged. And one flagged bill has **no return movement at all**. Like the hallmark flag in defect 296, this is set by hand and is not a reliable record of what happened.

So the rule implemented is the one that cannot be wrong: **the bill is marked returned when everything on it has come back**. A banner reading `GOODS RETURNED` across a bill of three pieces where one came back would overstate what happened, and the shop's own data gives no warrant for it — its one partial return is unflagged.

`is_cancelled` is left alone. The shop has never cancelled a bill in 224, and a credit note is what it does instead; inventing a cancel action would be inventing a workflow, as in defect 296.

Done

The credit note marks the bill, in the same transaction as the return. Two tests: one asserting a fully-returned bill says so, one asserting a partly-returned bill does not. The first was checked against the bug by neutering the `SET` clause — *"everything on the bill came back and the bill does not say so"*.

The test harness hid a broken save

The first version of the `UPDATE` named `updated_at`, which `mcj_sales_invoices` does not have. The statement failed, the whole transaction aborted, and **the test did not notice**: `credit()` posts and reads the body without asserting a status, so a 500 came back as a string and the test carried on. The first sign was a later read failing with *"current transaction is aborted, commands ignored until end of transaction block"* — an error about the test's own query, pointing nowhere near the cause.

A save that fails should fail the test that made it, where it happens. There is now a `creditSaved` helper that insists on the redirect, and the new tests use it. The nine existing `credit()` calls are left alone: several of them post a save that is *meant* to be refused, so a blanket assertion would be wrong.

307. The shop could see what its suppliers owed it and not what its customers

did

Following defect 306's create-only sweep into `mcj_sale_payments`, which nothing can correct once written. That turned out to be sound — the payments reconcile:

```
149 bills carry a payment
115 paid in full
26 part paid, 13,55,888.00 short
0 paid MORE than the bill
```

Part payment is ordinary in a jeweller and no bill is overpaid, so there is nothing wrong with the record. But asking who is short led somewhere else.

Supplier Balances exists. There is no customer equivalent. Its own note makes the argument: *"the old system had purchase registers, which show what was bought, and a ledger book, which shows one account at a time. Neither puts the supplier side together, and what it shows is worth a screen of its own."* Every word applies to customers, and there are **491 customer accounts against 76 dealer ones** — the side without a screen was the larger one.

```
10 accounts owe the shop      73,96,553.00
20 accounts the shop owes    27,17,486.77
10 balances saw no posting this year
```

The second line is the one nobody could see. A customer account in CREDIT means the shop is **holding that person's money** — an advance against an order, a part-payment on something not yet delivered — and ₹27.17 lakh of it is a liability that appeared on no screen.

Done

Customer Balances, mirroring the supplier screen: opening, movement, postings and balance per account, with the same four views — who owes the shop, everyone with a balance, money the shop is holding, and balances that never moved.

Verified against the database rather than against itself: the screen returns 10 rows totalling **₹73,96,553.00** under "owe the shop", 20 under "holding" and 10 under "never moved", which are the three figures an independent query gives. The largest single debtor is HARSIT ANAND SULTANIA at ₹32,30,681.00.

The screen says on its face what it is not: **an account balance is not a list of unpaid bills**. A customer who paid one bill and left another outstanding nets to one figure here.

Three tests. The one that matters is the sign: debit positive, so a customer who owes reads positive and an advance reads negative. Checked against the bug by making the balance the year's postings alone — the mistake the supplier screen's own notes record its author making — which drops the 50,000 brought forward and reports it as nil: *"an account that owes 50,000 from last year is missing — the year's postings alone are not a balance"*.

Not a defect, checked and left

`mcj_sale_payments` cannot be corrected once written, and that is defensible: a payment is part of a posted sale, and the accounting answer to a wrong posting is a correcting entry, which the voucher screen can make. The payments report reads only its own table, so a correcting journal would not reach it — that is worth knowing before anybody relies on it as a cash-reconciliation tool, and it is why the reconciliation above was run against the bills rather than assumed.

308. Nothing checked that the bill in the customer's hand equals what the books say

Defect 307 left a thread: the payments report reads its own table rather than the ledger, so a correcting journal would never reach it. That raised the general question — which money on a screen comes from a subsidiary table rather than from the books, and does it still agree?

The tax invoice and the GST return already answer it well: both take the tax from the VOUCHER, deliberately, and say why. But **no rule held the bill and its voucher together**. A bill could drift from its own posting and every screen would go on agreeing with itself, because each reads one side.

Checked, and they agree

224 bills, 224 agree, difference 0.00

Exactly. Every bill has a voucher, and every bill total equals the debit side of that voucher, to the paisa — 204 debiting a Customer account and 20 a Dealer, the shop's B2B sales.

That is worth stating plainly because the shop's accounting has three open blockers against it (B19, B4, B8), and this is one large thing that is not wrong: **₹2,47,65,386.00 of sales, and the bills and the books do not disagree by a rupee.**

The rule now guards it

Invariant 19, "a bill agrees with its own voucher". Checked by adding one rupee to GST/2627/00206: *1 bill(s) that disagree with the voucher behind them.*

It looked like a 28-rupee hole first

The first comparison summed **every** debit on the voucher, and reported 115 of 224 bills disagreeing by ₹28.31 in total. That is not a discrepancy, it is the rounding, and one voucher shows it:

THE SATURDAY CLUB	Customer	4000.00	
ROUNDED OFF	Miscellaneous Expenses	0.52	
SALE	Sales Account		3884.00
SGST	Duties & Taxes		58.26
CGST	Duties & Taxes		58.26

The customer is debited 4,000.00 — exactly the bill — and the 52 paise the shop rounded away are an expense. Debits and credits both come to 4,000.52 and the voucher is right. **The bill total was never meant to equal the total debits; it equals the party's debit.**

Reading one voucher settled it in a minute, where the aggregate had suggested a systematic fault across half the bills. The rule therefore excludes the round-off account by name and says why, so the next person to run this comparison does not rediscover the same ₹28.31.

This is the twelfth measurement in this project where the number was right and the question was wrong.

309. The first thing on the list of decisions I was waiting for had already been decided

Defect 308 reconciled the sales side. The purchase side is the same question mirrored — a purchase credits the supplier where a sale debits the customer — so it was asked the same way.

```
43 purchases, 43 agree, difference 0.00, on 2,39,56,798.00
25 old gold lots, 25 agree, on 50,71,370.00 for 534.540 g
```

Both clean. Invariant 20, "a purchase agrees with its own voucher", now holds it.

Old gold also carries a weight and a rate on **every one of its 25 lines**, which is what sent this somewhere else.

Blocker B26 said the opposite, and was the first thing on your list

B26 — *"Posting screens write a voucher and no detail row"* — was the top row of **Waiting on you**, asking for a decision before the first real bill. It says:

- *"nothing writes mcj_sales_invoices, _lines, mcj_stock_movements, mcj_sold_stones, mcj_old_gold_lines or mcj_purchase_invoices"* — **all six are written.**
- *"Old gold ... the metal itself — weight, purity and rate all unrecorded"* — `oldgold.go:365` **writes purity, gross, net, tunch, rate and amount.**
- *"Two ways to close it"* — **the first was taken.** Each screen writes its detail rows beside the voucher.

So the first decision I was waiting for had been made and built, and the document kept asking for it. That costs the one thing this project cannot make more of, which is your attention.

Closed on its own criterion, by test

B26 names what would prove it: *"A bill raised today would balance the books and be invisible to fifteen screens, including the GST return."* So the closing evidence is a bill raised today, looked for on the return.

`TestDaysTrade_ASaleReachesTheGSTReturn` sets the morning's rate, tags a piece, sells it at the counter through the sale screen, and then asserts the bill appears in `gstr-sales` carrying a taxable value — and that the tag has exactly one SALE-GST movement, which is the other half B26 says is lost.

A grep showing six INSERT statements is not that. It shows the statements exist; it does not show a bill reaching a return, and this blocker has twice been closed on evidence of that weaker kind.

It has been declared closed twice before, and reopened

That history is the reason for the test rather than the grep:

- **Defect 194** closed B26 on the trade path, when the credit note began writing a SALE-RET movement.
- **Defect 195** closed it again for purchases, on the bill header. **Defect 224 found that was true of the header and not of the contents** — 23 of the shop's 43 purchase bills are stones only, and none of the contents was written.

Each closure was true of what it measured and wrong about the whole. This one is closed on the blocker's own stated criterion, exercised end to end, which is the narrowest claim that actually settles it.

The blocker is marked RESOLVED with everything it said while open kept below a line, because the measurements in it are the record of what was wrong. Its row is gone from Waiting on you.

310. A second answered question was still on the decision list, and now a check says so

Defect 309 found B26 asking for a decision that had been made. The obvious question is whether it was the only one, and the obvious way to answer it is not by reading.

`check_blocker_citations.py` already reads every blocker heading to catch a SCREEN citing a settled blocker. It now checks the other direction as well: a blocker still sitting on **Waiting on you** in STATUS.md when its own heading says it is settled.

It found one immediately. B9 — *"Were these priced by hand?"* — has been **ANSWERED** since defect 228:

There is no formula to find. Of the 129 certified pieces the shop has sold, one went out at the tag's stored price and none at a computed one.

The answer is in the blocker, the sale screen was rebuilt around it — that is what returned 549 pieces worth ₹2,33,45,417.46 to being sellable — and the question was still on the list. Removed.

Why this is worth a check rather than a habit

That table is the shopkeeper's decision list. A row on it is a request for their attention, which is the scarcest thing this project spends, and a stale row spends it on nothing. Both stale rows had been stale for a while and neither was noticed by reading, including by me reading that table several times this week to write the closing summary of a turn.

The check cannot produce a false alarm: it compares an id on the list against that id's own heading, both of which are written by hand in the same repository. Verified by putting B20 (RESOLVED) back on the list — *"asks about B20, whose status is: The data loads were never version-controlled — RESOLVED"* — and removing it again.

The rest of the list was checked by measurement, and holds

The other rows assert figures. Those that can be re-derived unambiguously were:

1,625 tags naming a maker who exists nowhere	1,625, worth 12,77,72,008.04
and none of the 103 codes matches a ledger	0 of 103
214 of 366 order lines cancelled	214 of 366
14.43 carats of customers' own stones held	14.430 ct on 7 rows

Two others — defect 50's hallmark count and defect 78/133's raw stock weight — were NOT confirmed, because my query is not their query: I picked the purity codes that count as gold jewellery and summed a weight column across every raw-stock row, and got 732 pieces against their 739 and 27,326.609 g against their 11,998.005. **That is a difference in method, not evidence of drift**, and saying otherwise would be the adjacent-measurement error this project has made twelve times. Left alone rather than reported as stale.

311. "No trace at all" was too strong, and 869.85 g of gold is the trace

Defects 309 and 310 both found the documents making stale claims about DECISIONS. The same question applies to the claims about CAPABILITY, so the two sections that make them were checked the same way.

The one that was wrong

What it cannot do carried this:

```
Anything manufacturing | Deliberate. 2.5 years of audit log show karigar
issue/receive leaving **no trace at all**
```

Defensible about the audit log, which is what it was measured from: the stock trail's vocabulary has no karigar movement. Misleading as written, because somebody reading it concludes nothing anywhere records manufacturing.

karigar_gold_stock has two rows:

ldgr	date	gross	bill	type	finish
11699	2025-09-16	369.850	C4/00001	ISS-RAW	1
11699	2025-09-16	500.000	C4/00002	ISS-RAW	1

869.850 g of gold issued to a named maker on a dated bill. Ledger 11699 is MA TARA JEWELLERS, and it sits in a ledger group called **Karigar** — the shop has that concept in its chart of accounts.

karigar_wip_bal is a single row of zeros, which agrees with both rows being marked finished: it went out and came back.

That is a small trace and it is not nothing. The row now says what the trail shows, what the other table shows, and the weight.

What it does NOT do is answer defect 76

Worth stating, because the temptation was there. Defect 76 asks who D599 is — **1,625 tags on the shelf worth ₹12,77,72,008.04 name a maker who exists nowhere**. Finding a Karigar ledger group looks like the answer and is not:

Karigar ledgers in the chart of accounts	1
maker codes on the shelf	103
codes matching that ledger	0

One karigar is known to the books. The 103 codes on the tags match none of them, and defect 76's question stands exactly as it was. The re-check confirmed its figures too — 1,625 tags, ₹12,77,72,008.04, and 0 of 103 matching any ledger at all.

The other direction: the list understated what is built

The shop can do this today had not kept up with the week. Six capabilities built and tested were missing from the one document whose stated job is to be the list of what is done:

certified pieces billed at an agreed price, certificate on the tag	defect 300
a tag recording what its stones ARE -- shape and colour	defect 298
setting a piece aside against an order	defect 304
cancelling an order	defect 305
what each customer owes and what the shop holds for them	defect 307
what each department did	defect 293

All six added. A document that overstates what works is dangerous; one that understates it wastes work already paid for, and this had been doing the second for as long as a week.

312. Two of the eighty reports were exempt from the check that they show anything

Chasing what today's schema changes might have broken, the rebuild was run from scratch — `load_all.sh` clean, every baseline back to the rupee, 20 of 20 rules holding — and then the render sweep, which said:

```
38 screens and 80 reports render ... and 78 of them show real data
```

Two short. Not because two were silent: because two were **excluded**, and the line did not say so.

`NEEDS_A_SUBJECT = {"ledger-book", "quote"}`. Both answer a question about one thing — an account, a barcode — and show nothing until told which, so the emptiness check skipped them. **2 of 80 reports had never been verified to produce anything at all**, and the summary read as though 78 was the total.

That is the shape of defect 278, where a tool could not tell an empty shop from a broken one, and of defect 299, where a guard skipped every run and reported PASS. This one is milder — it under-reports rather than lies — but its own file says a checker that hides its blind spots is worse than none, and this blind spot was hidden in a subtraction.

Both reports are fine

Given a real subject by hand:

```
quote          a 147.242 g DIAMOND SET — 17,15,369.30 of metal, 24,76,760.00 of stones
ledger-book    CASH, 462 rows from the opening balance on
```

Done

The checker now finds its own subjects, over HTTP, from pages it already fetches: the ledger book's own select for an account, and the stock history register for a barcode. **80 of 80 reports are checked**, and the two are named in the output with the subject used, so a reader can see they were checked rather than exempted.

Failing to find a subject is reported as a broken page, not skipped — the whole point being that a skip is what hid this.

Checked by pointing the barcode finder at a tag that does not exist: *"given a real subject (a barcode that does not exist) it still shows nothing"*.

The rest of the sweep found nothing wrong

Recorded because a negative result is worth the same as a positive one when the question was real:

- **The rebuild is sound after three migrations and four changed loaders.** 2,816 tags, 2,575 on the shelf, ₹14,23,61,188.20, 1,848 vouchers, 9,031 movements, 224 bills. Every column added today survived: 2,816 of 2,816 tags carry a department, 224 of 224 bills, 1,416 of 1,848 vouchers name who raised them. `interstate` is recorded on none, which is correct — it is nullable so the captured bills keep deriving it.
- **The three capped reports still say they are capped.** Defect 145's fix is intact on all of them: `conversions` and `designs` each run a second query over the whole selection so the page cannot present 500 rows as the total, and `movements` says how much is not on the page.
- **Schemes are already covered.** 235 of 239 have matured, the oldest in July 2017, and none is closed — which the due-date reminder already reports, in those words.

313. Every legacy screen that has data behind it is built; the 67 that are not have none

The goal says to build every screen that has data behind it. **126 of 193 legacy menu screens are rebuilt** and nobody had asked what is in the other 67 — the figure was a percentage, not an answer. Asked properly, by taking the 67 and looking for their data.

~30	manufacturing and karigar	two rows, 869.850 g -- defect 311
5	sales voucher variants	none: all 224 bills are SALE-GST
1	estimate entry	2 estimates in two and a half years
4	barcode merge and split	no such table in the capture
5	purchase approval	no such table in the capture
7	setup and opening stock	setup tasks for data already loaded
2	SMS and WhatsApp	blocker B15's tables, deliberately unopened
13	the rest	backup, password, discount master, TDS ...

Nothing unbuilt has data behind it, and the one exception is named: the manufacturing screens, whose whole evidence is the two rows defect 311 found, and which STATUS records as a deliberate exclusion.

Three of these are worth stating because the answer was not obvious:

- **The five sales-voucher variants** — credit sale, estimate sale, exhibition, export, repair — sound like real trade. Every one of the shop's 224 bills is `SALE-GST`, 221 local and 3 central. There is no credit sale, no export sale and no repair sale in two and a half years.
- **Estimates** have a register (`estimate_list.php`, built) and no entry screen (`sales_voucher_estimate.php`, not). That asymmetry is right at this size: the shop made **2 estimates** in the whole capture. Building an entry screen for a workflow used twice would be inventing one, which is the argument defect 296 made about hallmarking.
- **Barcode merge and split** have four menu entries between them and **no table in the capture at all**. The screens existed; the shop never used them.

The one table this turned up: `item_merge` , 40 rows, never read

Not barcode merges — the CATALOGUE's own history. Forty items folded into four, between 20 September 2023 and **11 August 2026**, which is recent.

39 of the 40 merged-away items are correctly absent from our catalogue. One is not, and it is the interesting one:

legacy item 1139 was SILVER KOPPOR STAND, merged into 215 SILVER DIPAK
on 20 September 2023
legacy item 1139 in our catalogue today is called Rudraksha

The id was reused. `legacy_id` **is therefore not a stable identity for an item** — 1139 means one thing before September 2023 and another after — which is the same trap the stock history screen carries on its face about barcodes, found for a second kind of key.

Nothing is wrong today. Nought tags, nought bill lines and nought order lines point at that item, and nought tags anywhere name any of the 40 merged-away items. The finding is that a join on `legacy_id` across a date could mislabel history, and that nobody had opened the table that says so.

314. Nine pieces the trail says were sold are still counted on the shelf

Both sides of the book reconcile (defects 308, 309), so the question moved to the stock side: does the shelf agree with the movement trail? Grouping every tag by its LAST movement:

last movement	tags	still on the shelf	gone
NEW-BAR	2332	2299	33
OP-STK	270	261	9
SALE-GST	205	9	196
SALE-RET	6	3	3
CST-ORD	3	3	0

Nine tags have a sale as their last movement and are still marked in stock, worth **₹99,492.23** — counted in the shelf's ₹14,23,61,188.20.

53541166	DIAMOND RING	52,445.17	sold 09 Aug 2026	GST/2627/00207
81846212	GOLD BANGLE	25,668.00	sold 21 Oct 2025	GST/2526/00590
67675269	SILVER RING	9,512.97	sold 26 Jul 2026	GST/2627/00193
41075931	GOLD RAKHI	2,000.00	sold 23 Aug 2025	GST/2526/00308
83267807	GOLD RAKHI	2,000.00	...same bill	
30043849	GOLD RAKHI	2,000.00		
14880124	GOLD RAKHI	2,000.00		
27099313	GOLD RAKHI	2,000.00		
15551589	SILVER RING	1,866.09	sold 18 Aug 2026	GST/2627/00219

It is not barcode reuse, which is the obvious explanation and was checked first

This shop reuses barcodes — 1,974 of 3,021 have been "created" more than once, and the stock history screen carries that caveat on its face. If the tag on the shelf were a NEW piece wearing an old number, there would be no defect.

none of the nine was tagged AFTER its own sale
none of the nine barcodes was created more than once
all three bills still on file are live -- not cancelled, not returned

So the piece the trail sold is the piece the shelf is counting.

The clearest one, which explains the shape

Bill GST/2627/00207, 09 August 2026, ₹98,300. The trail shows **two** diamond rings leaving on it:

69343561R1	68,424.00	in_stock = false	← cleared
53541166	52,445.17	in_stock = true	← still on the shelf

The bill carries **one** line, for ₹95,437, naming 69343561R1. So two pieces left against a bill that itemises one, and the one the line does not name is the one still on the shelf. **Nine bills show more tags leaving in the trail than they have lines.**

This is the shop's own record, faithfully loaded — not something this system did. It sits beside B8's duplicate barcode as a fact about the data.

The other direction, for completeness

42 tags are off the shelf with no movement saying they left — their last movement is a creation or an opening — worth ₹29,90,169.63. That is the safer direction: the shelf is understated rather than overstated, and a piece that left without a movement is what defect 192 fixed for sales made here.

Done

Invariant 21, "a tag the trail sold is off the shelf", allowed **9** — the legacy's own. What it guards is a tenth, which would mean a sale made HERE left its piece on the shelf. Checked by planting a SALE-GST movement against the most valuable tag in stock: *10 tag(s) ... 1 more than the 9 allowed.*

315. The GST return could see every rupee, and a blocker still said it could see 63%

Defect 314 compared the shelf against the trail. The same question, asked of the two records of a SALE, turned up something else.

tagged bill lines	256	
...with a matching sale movement	256	
sale movements naming a bill	324	
...with a matching bill line	271	
...bill not in the capture at all	38	(8 bills, pre-2026-27)
...bill here, no line for that tag	15	(10 bills – defect 314)

The bill never claims a sale the trail missed. **All 256 tagged lines have their movement.** The disagreement runs one way only, which is what defect 314 measured.

But counting bills without line detail to explain the 38 gave a figure that should not have been possible.

B21 said 39 bills have no line detail. There are none

o of 224. B21's remaining scope was HSN completeness, and it recorded:

```
sales_vou_main carries lines for 185 of the 224 bills and no rows at all for
the other 39 ... the HSN summary sees ₹1,51,40,109.00 of taxable value and
cannot see ₹89,26,345.00
```

True of `sales_vou_main`. **The loader also reads `sales_vou_detail`, which carries the other 94 bills** — the two sources together cover all 224, and the loader's own gate has been asserting "bills with no lines: 0" the whole time.

```
sales_vou_main      lines for 185 bills
sales_vou_detail    lines for  94 bills
loaded              367 lines, 224 of 224 bills, 0 lines without an HSN code
```

Checked against the books rather than the bills, because the books are what a return must agree with:

```
invoice lines                2,40,66,454.00
Sales Account credit on SALE-GST 2,40,66,454.00
```

Exactly equal. And B21's own two figures — 1,51,40,109.00 seen, 89,26,345.00 unseen — sum to 2,40,66,454.00, which is the arithmetic proving the gap it described is precisely the part now loaded. The HSN summary reports that total with ₹6,98,933.87 of tax and carries no shortfall note, because there is none.

B21 is RESOLVED. That is the third blocker closed this week by measuring what it asserted rather than by arguing about it — after B26 in defect 309 and B9 in defect 310, both of which were also asking for something that had already been done.

The guard

Invariant 22, "the line detail sees every taxable rupee", compares the two totals directly. Checked by taking ₹100 off one line: it reports the gap, and so does rule 1, which catches the same edit from the bill's side.

A unit test for this already existed and was not enough: `TestHSNSummary_AccountsForEveryRupee` seeds its own bills and proves the report's arithmetic. It cannot say anything about whether the SHOP's line detail is complete, which is the whole of what B21 was about. Arithmetic and coverage are different questions and needed different checks.

316. Two open blockers re-measured, and one of them is ten thousand rupees rather than unbounded

Three blockers closed this week by measuring what they asserted — B26, B9, B21. The same treatment applied to the ones still open does not close any, which is itself the answer: they are open because they need a decision or an action, not because nobody has looked. Two were worth re-measuring anyway.

B2 — SLVPC purity — accurate, and it was reading as bigger than it is

5 stock items use purity code SLVPC, which does not exist in gold_master
Blocks: correct valuation of those 5 items; ANY STOCK TOTAL THAT INCLUDES THEM

The first line is exactly right. **All five carry SLVPC, no purity master has that code, and all five are on the shelf.** They are worth ₹10,000.00 between them — 0.007% of ₹14,23,61,188.20.

The second line is what needed sizing. "Any stock total that includes them" reads as though the shelf figure were suspect. It is ten thousand rupees, and knowing that is the difference between a blocker somebody drops everything for and one answered next time the owner is passing.

A separation worth having: 13 tags have no purity, not 5. The other **8 carry a BLANK purity** and resolve to none either — but **all eight are off the shelf**, so they affect no valuation at all. Anyone measuring "tags with no purity" gets 13 and concludes the blocker is nearly three times its size.

B17 — SILVER PICHKARY — accurate in every particular

item 1063 SILVER PICHKARY metal gm 0 tags
item 1156 SILVER PICHKARY metal gm 0 tags

Two identical rows, neither carrying a single tag, exactly as recorded. Still the shop's call to delete one, and still costing nothing while it waits.

B4 and B8, checked in passing

B4 vouchers that do not balance 1, out by ₹0.01
B8 duplicated barcodes 1

Both exactly as stated. B8's is the barcode defect 291 found doubling rows on the audit screen and defect 314 found among the nine sold-but-shelved tags, so it has now been met three times from three directions and is the same single tag each time.

What this says about the open list

Of the 17 open blockers, the ones carrying a measurable claim have now been measured and every one holds. The list is not stale — B26, B9 and B21 were, and they are closed. What remains genuinely waits on somebody: a fineness from the business, a duplicate confirmed for deletion, an accountant's two repairs, a photograph opened in a browser.

317. The 29.9 lakh that left with no movement is accounted for; 400 rupees of it is not

Defect 314 recorded the other direction of the shelf-against-trail comparison and left it there: **42 tags are off the shelf whose last movement is a creation or an opening — nothing says they left — worth ₹29,90,169.63.** That figure deserved chasing rather than filing.

on a bill AND in a conversion 33 tags 29,89,769.63
neither 9 tags 400.00

Thirty-three of the forty-two are accounted for twice over: they appear on a bill line and among the pieces of a stock conversion. They did leave, and the trail simply carries no exit movement for them — which is the silence defect 192 fixed for sales made in this system.

So the ₹29.9 lakh is not missing. **What is unexplained is ₹400.00.**

49242293	MALA	400.00	tagged 16 Mar 2021
67141215	(blank)	0.00	
45174064	(blank)	0.00	
30223979/SR	(blank)	0.00	
30683902	(blank)	0.00	
19684380/SR	(blank)	0.00	
65910625/SR/SR/SR/SR	(blank)	0.00	
55380544	(blank)	0.00	tagged 22 Jan 2019
69375796	(blank)	0.00	tagged 18 Apr 2023

One real piece and eight empty shells — no description, no value, six with no creation date — all last seen in the opening stock of 1 April 2025.

Two conventions for "this piece came back"

65910625/SR/SR/SR/SR is not a typo. The shop marks a returned piece by changing its barcode, and it does so **two different ways**:

suffixed R<n>	263 tags	245 on the shelf	57,86,486.50
suffixed /SR once	7 tags	4 on the shelf	1,27,936.85
suffixed /SR more than once	1 tag	0 on the shelf	
plain	2,545 tags	2,326 on the shelf	13,64,46,764.85
		2,575 on the shelf	14,23,61,188.20

The R<n> form INCREMENTS — R1, R2 — so a piece returned twice is one barcode. The /SR form APPENDS, so the same history becomes X/SR/SR, and four returns become the barcode above. **263 tags carry the first convention against 8 the second**, so the incrementing form is the shop's practice and the other is a handful of survivals.

Anything matching a returned piece to its original has to know both. Only **one** R-suffixed tag has its plain original still on file, so the two rarely coexist — which is why nothing has tripped over this yet.

Nothing here is broken. It is recorded because the next person to match barcodes across a return will meet it, and because the ₹29.9 lakh in defect 314 should not be left looking like a hole when it is ₹400.

318. Nobody could check this repository without being told how

Three hundred and seventeen defects, eight checkers, a Go suite and a reconciler, and **no way to run them**. Every verification in this project has been a sequence assembled from memory, in a terminal, by whoever ran it last.

That is a real fault and not a tidiness one. The knowledge that mattered was nowhere written:

- Two of the eight checkers need the application answering on a port, and one of them reports `connection refused` as a **failure** if it is not — indexed under the same exit code as a genuine fault.
- Three of the checkers are informational: the reconciler fails three of its eight gates on purpose, because they are blockers B19, B4 and B8 and need an accountant rather than a patch. Anybody running it cold would read red and conclude the migration was broken.
- The Go suite takes twelve minutes and the rest take two, so there is a fast answer and a slow one, and no way to ask for either.

Somebody handed this repository could not tell whether it was sound.

Done

`tools/check_all.sh`, with `--quick` for everything but the Go suite. It starts the application if it is not already up, stops it on the way out, and ends with a list rather than a scroll:

```
ok  it builds
ok  it vets
ok  the shop's own rules
ok  STATUS.md matches reality
ok  blockers and the ask list
ok  every screen renders
ok  no report doubles a total
--  the migration against the backup (informational)
--  tables nothing reads (informational)
--  columns nothing writes (informational)

7 check(s) passed. Nothing is wrong that this repository can detect.
```

Must-pass and informational are separated by design. Making the run red for three blockers that need a person is how everybody learns to ignore red.

It stops the application by PORT, not by name, and says why in a comment: `pkill -f cmd/wingold` does not match the temporary binary `go run builds`, which is how defects 303 and 304 both came to read a stale page and believe a working change had failed.

STATUS.md now opens with how to run it, because a document that says what is done should say how to check it.

Verified, and my own check was wrong first

Planted a false figure in STATUS.md: the run reports `FAIL STATUS.md matches reality`, lists the seven that passed, and **exits 1**. Clean, it exits 0. The application is not left running either way — `lsof -ti tcp:3210` is empty afterwards.

The first attempt to read that exit code piped the script into `tail` and printed `exit was 0 . $?` after a pipeline is the LAST command's status, so I had measured `tail`. This project has recorded that exact mistake before, in the same words, and the fix is to run the thing and read `$?` before piping anything.

319. Defects 304 and 305 walked back into defect 206, and the screen said the save had failed while it committed

Asked of my own work, because 206 is the kind of fault that comes back: does the outstanding-orders screen return a failing status when a save fails?

It did not. `renderOutstanding` ended with

```
templ.Handler(orderpage.Deliver(props)).ServeHTTP(w, r)
```

— no status, so 200. And both of this week's new save paths, setting a tag aside (defect 304) and cancelling an order (defect 305), reported their refusals through it.

`atomic` commits the request's transaction on any 2xx and rolls back from 400 up. So a submission that reserved one tag and then met a tag already sold **told the wrapper it had succeeded** — and the first reservation was committed while the screen said the whole thing had failed.

The comment above `failedSaveStatus` warns about exactly this, and adds that a failure which ABORTS the Postgres transaction is survivable by luck, since Postgres turns COMMIT on an aborted transaction into ROLLBACK. **That luck does not apply here.** These refusals are logical — a tag not on the shelf, an order with no lines — so the transaction is perfectly healthy and commits.

Proved against the running app, because the harness cannot show it

```
with the 200    status 200    reserved lines 59 → 60
                                tag 10016845 reserved on C0/2627/00113
                                and the screen said "has already left the shelf"
with the 500    status 500    reserved lines 59 → 59
```

The itf harness runs the request inside the test's own transaction, so it cannot demonstrate a rollback either way — `TestTrade_AFailedSaveDoesNotReportSuccess` records that for the credit note, and the same applies here. **My first version of this test asserted the row count and failed for that reason**, reporting "1 reservation(s) were written" against a harness that never rolls anything back. It now asserts the STATUS, which is what the screen controls, and the rollback was confirmed on the running app as above.

The reservation the proof wrote into the shop's data was undone by re-running `load_orders.sh`: 60 back to 59.

Done

`renderOutstanding` takes a status, and every caller says which. The two that mean "nothing was asked for" stay 200; the four that mean a save failed — including the malformed-id path, which can run after a reservation has already gone in — return `failedSaveStatus`.

Two tests, one per new save path, asserting the status and the message.

Why this matters more than the money

Nothing was lost: a tag was reserved that should not have been. The failure is that **the screen and the database disagreed about whether anything happened**, and the screen was the one telling the truth as far as the user could see. A clerk told "that tag has already left the shelf" has no reason to check whether the other one went through.

320. Four more screens told the wrapper a failed save had succeeded

Defect 319 found the orders screen rendering a failed save on a 200. The question that matters is not whether that one is fixed but whether it is the only one, and it is not.

Every controller, every branch where a save can fail:

```
coinrate.go:114    if err := c.save(...)    -> c.render(...)    200
metallrate.go:111  if err := c.save(...)    -> c.render(...)    200
salarysheet.go:158 if err := c.save(...)    -> c.render(...)    200
staffpay.go:138    if err := c.save(...)    -> c.render(...)    200
tagcorrect.go:193  if err := c.apply(...)   -> c.render(...)    200
```

None of the ten controllers that write was status-aware. Every one of these five saves writes more than once — coin rates two statements, metal rates two, the salary sheet four, staff pay two, the tag correction two — so a failure partway leaves the earlier writes standing, and `atomic` commits them because the screen answered 2xx.

`tagcorrect` **is the one that matters most.** `apply()` corrects the tag and then writes the row saying what changed. If the second fails, the tag is corrected with nothing in the edit trail — on the one screen that exists *because* the legacy kept `barcode_edit_tracking`. Its own header says so: *"recorded what changed in barcode_edit_tracking — which this system loaded, read with an audit report, and could not add to."*

Done

All five renders take a status; every caller says which. Pre-write validation failures stay 200 and now say so out loud rather than by omission, which is the difference between a decision and an oversight.

`TestTagCorrect_AFailedSaveDoesNotReportSuccess` arms the audit write to fail with a CHECK constraint and asserts the screen answers 500. Checked against the bug by setting it back to 200: *expected: 500*.

Three occurrences is a class, so it has a check now

```
defect 206    four posting families, found by reading
defect 319    the orders screen, reintroduced by my own defects 304 and 305
defect 320    four more, found by the sweep after 319
```

`tools/check_failed_saves.py` looks for a save-shaped call whose failure branch renders without naming a status of 400 or more, and it is wired into `check_all.sh` as **"no failed save says ok"**. Checked by putting the tagcorrect fault back: it names the file, the line, the save and the render.

Its docstring says what it cannot see — a save spelled some other way, and whether the write really happened before the failure — and that it errs towards reporting, so a screen rendering a pre-write refusal must pass `http.StatusOK` explicitly to say that is deliberate.

Two mistakes making the change, both from editing Go with regular expressions

The first attempt spliced a duplicate function and left a stray `{`, which compiled — Go allows a bare block — so `go build` said nothing was wrong while the file was nonsense. Reverted and redone as a parameter on the existing function.

The second broke three files: `c.render\((w, r, [^\n]*)\)` stops at the first `)`, so `c.render(w, r, props, time.Now())` became `c.render(w, r, props, time.Now(, http.StatusOK))`. That one the compiler did catch. Redone with a paren-matching pass rather than a pattern.

321. The check I built to make verification easy ran the test suite twice

Defect 318 gave this repository one command. Read back with the same eye used on everything else, that command had two faults of its own.

It ran the whole Go suite twice

```
go test ... | grep -E "(ok|FAIL|---|panic)" ; ! go test ... | grep -q "FAIL"
```

Once to show the output, once to decide whether it passed. Go caches test results, so this would normally be nearly free — except both runs pass `-count=1`, which exists precisely to defeat that cache. **Six minutes of work took twelve.**

That matters beyond tidiness. The whole point of defect 318 was that nobody could check this repository; a check that takes twelve minutes when it could take six is one people skip, and then it checks nothing at all.

One run now, filtered into a temporary file, read twice from there.

before	about 12 minutes
after	6 minutes 33 seconds, measured

It could not see a skipped test

`go test` prints nothing for a skipped test unless asked. So the one command that claims to say whether this repository is sound was **blind to the failure mode that has hidden more broken tests here than any other**:

defect 299	a guard that skipped every run for months and reported PASS
defect 312	two reports exempted from a check, the exemption hidden in a sum
defect 298	my own test, skipped because nothing seeded what it looked for
defect 302	my own test, skipped because my form was wrong

Four times, and the last two were mine, in this week's work. It runs `-v` now and filters to the four line shapes that matter, so a skip is **reported by name and fails the run**:

```
tests SKIPPED -- a skip is how a broken test hides:
--- SKIP: TestZZSkipProbe (0.00s)
```

Verified by adding a test that does nothing but skip, watching the step report it, and removing it again.

There is one legitimate skip in the suite — a test that stands down when the captured legacy pages are not on disk. It does not fire while the capture is there, and if it ever does, the run failing with that test's name is the right answer: you cannot verify what you cannot read.

Both faults were in the tool, not the shop

Recorded because the tooling deserves the same treatment as the code, and because a verification tool that is slow and partly blind is worse than an honest absence of one — it produces a green tick that nobody has any reason to believe.

322. One person's payroll alternates month by month, and three more unnamed codes exist than the list says

Sales, purchases and old gold all reconcile against the books (defects 308, 309). Payroll was the money area left, so it got the same treatment — independently, without reading defect 282 first, which turned out to have settled the headline already.

Payroll ties, re-derived from the other end

Defect 282 recorded that payroll reconciles. This confirms it by a different route: the salary journals name their month in words, so they can be matched to the sheets exactly.

the April-2026 salary journals credit	2,64,200.00
the April-2026 sheets' gross	2,64,200.00

To the rupee, on 11 named staff.

A near-miss on the way, caught before it was written. I first compared the sheet's gross against the ledger accounts EMPLOYEE BASIC PAY and DEARNESS ALLOWANCE, got ₹2,34,600, and had a ₹29,600 shortfall in every month. That is not a shortfall: the journal also credits provident fund (₹14,040), professional tax (₹1,470) and ESI (₹466), and the staff accounts carry net pay rather than gross. Basic plus DA is not the gross, and the total credited is. **The thirteenth time in this project that the number was right and the question was wrong.**

Three more unnamed staff codes than the open question names

The question on the list says four unnamed codes, ₹7,14,430 paid. **That figure is exactly right** — 4023, 8827, 8855 and 8828, 15 sheets each:

4023	15 sheets	4,68,000.00 gross	3,10,800.00 paid
8827	15	2,58,000.00	1,92,330.00
8855	15	1,89,900.00	1,25,820.00
8828	15	1,50,500.00	85,480.00
			7,14,430.00

There are three more: 12475, 12509 and 12679, with 2, 1 and 4 sheets — and nought gross and nought paid between them. They cost nothing and change no figure, and they are named here so that anybody counting unnamed codes gets seven and is not left wondering which four the question means.

MADHURIMA DAS is paid every other month, exactly

Staff 7369, 16 sheets from April 2025 to July 2026:

202504	21,100.00	gross	0.00	paid
202505	21,100.00		20,970.00	
202506	0.00		0.00	
202507	21,100.00		20,970.00	
202508	0.00		0.00	
202509	21,100.00		0.00	
202510	0.00		0.00	
...alternating to				
202607	21,100.00		0.00	

From June 2025 the pattern is strict: **nought in every even month, ₹21,100 in every odd one.** Nine months of gross, six of payment, seven of nothing at all. She is the **ONLY** named person on the payroll with a zero month — every other zero sheet belongs to an unnamed code.

₹21,100 is a standard band here, the same as two other staff, so the amount is ordinary and the rhythm is not. Whether she works alternate months or whether her pay is entered every second month is not answerable from the data, and it is the kind of thing the shop knows in a sentence.

days_worked is **0.00 on all 246 sheets**, including months that were paid in full, so it cannot distinguish the two. That is defect 218's finding met again from another direction: the column is not a record of anything.

323. The stock journal's flag, asked again from the other side, and still not answerable

One of the questions waiting on the shop is whether add = 1 on a stock conversion means tagged or untagged. It has been open since the conversions were loaded, and it is worth more than it looks: it decides what 859 documents did, and whether **₹2,60,68,682.20 of stone across 2,554.341 carats** went into the shop or out of it. Both figures verify exactly as the list states them.

The screen had reasoned about it from the DOCUMENTS' side and stopped, carefully, at what that could support. This asked from the TAGS' side.

	flagged 0	flagged 1
pieces whose tag names that document	317 of 486	0 of 144
tags created the SAME DAY as the document	328 of 486	3 of 144
pieces with no tag on file at all	237 of 723	375 of 519

Every line points the same way: **0 raises tags, 1 consumes them** — which makes add = 1 mean UNTAGGED, against what the column's name suggests.

And it is still not proof, so the label stays off

The whole case for 1 is an **absence**: no tag names it. That is consistent with consuming tags and also with any number of other things, and this shop reuses barcodes — a barcode pre-dating a document does not prove that piece went into it, because it may have been reused before as easily as after.

The weights make it worse rather than better:

flag 1	7,664.013 g
flag 0	5,163.068 g

Read as 1-in and 0-out, a third of the metal is lost, which is too much for gold. Read as 0-in and 1-out, gold is created from nothing — unless the conversion also draws on raw metal, which the shop certainly does and this capture cannot show. **Two impossible readings and one unmeasurable escape is not an answer.**

The stones do not break the tie either: flag 1 carries more carats (1,364.071 against 1,190.270) and less value (₹1,19,87,286.30 against ₹1,40,81,395.90), which is what sorting looks like and also what nothing in particular looks like.

What changed

Nothing on the screen, deliberately. The flag is still shown as the shop recorded it, because labelling it "In" and "Out" would make the journal readable and possibly backwards, and for a stock journal that is worse than making the reader ask.

What changed is that the three measurements are now written beside the code, so the next person to have this idea can see it has been had, and how far it got. The question on the list now carries its own evidence rather than being a bare ask — **the answer is one sentence from somebody who worked there**, and this is the second time this week that has turned out to be the whole of what is missing.

324. Fourteen coin prices are out by a digit, and one stood for five days

The coin rate list is 5,966 quotes over 115 days — the largest table nobody had examined. Grouping each coin's quotes by how far they sit from that coin's own median:

COIN-30 GM	01 Jul 2026	40,00,110.00	usually	4,30,350.00
SILVER COIN 1KG	25 May 2026	29,33,000.00	usually	2,56,000.00
COIN-8 GM	26 May 2026	15,00,400.00	usually	1,14,860.00
COIN-8 GM	27 May 2026	14,94,800.00		
COIN-8 GM	29 May 2026	14,93,900.00		
COIN-8 GM	30 May 2026	14,86,500.00		
COIN-8 GM	31 May 2026	14,86,500.00		
COIN-4 GM	15 Jul 2026	5,40,400.00	usually	57,580.00

and the same fault the other way:

COIN-100 GM	17 Apr 2026	1,46,310.00	usually 14,32,000.00
COIN-20 GM	30 May 2026	46,270.00	usually 2,86,700.00
COIN-20 GM	31 May 2026	46,270.00	
SILVER COIN 1/2 TOLA	01 Jul 2026	156.00	usually 1,640.00
COIN-50 GM	21 Jul 2026	68,150.00	usually 7,16,750.00

Fourteen quotes, every one with the signature of an extra or a missing zero. Gold's largest real move in this data is a few per cent; these are factors of six to fourteen.

A hundred-gram gold coin was listed at ₹1,46,310 against a usual ₹14,32,000. Anybody selling one at the listed price hands over ₹12.86 lakh of gold for the price of one gram short of ten.

And COIN-8 GM stood at thirteen times its price for five consecutive days — 26, 27, 29, 30 and 31 May. It was not a slip that somebody noticed and corrected; it was typed again each morning for a working week.

Why nothing caught it

The list is a fact about a day and nothing validates it. The screen accepted any number. And the list is ADVISORY — no sale prices from it, which is why the shop survived this — so a wrong quote does nothing at all until somebody reads it to a customer, which is precisely what the list is for.

Done

The screen refuses a price more than three times, or less than a third of, the last one set for that coin, naming both figures: *"COIN-8 GM at 15,00,400.00 is a long way from its last price of 1,14,860.00. Check the figure — a coin price out by a digit is the commonest mistake on this list."*

Three times either way is wide enough for any real move and narrow enough for every one of the fourteen. It compares against the last price rather than the median so it works from the first day onwards, and a coin priced for the first time has nothing to compare and passes.

It refuses rather than warns, because a coin price is read off this screen by somebody about to quote a customer, and a warning on a figure already saved is one nobody sees the second time.

Two tests: a price ten times yesterday's is refused, shows what the price was before, and **writes nothing**; a real eight per cent move goes through. Checked against the bug by removing the call — *"a price ten times yesterday's was accepted without a word"* and *"1 price(s) were written for a day that was refused"*.

Invariant 23 holds the fourteen already in the list at fourteen, so a fifteenth means the refusal has been removed or got around.

What this does not settle

Defect 54 asks which coin price is right, the daily list or the item master. Still open, and this does not touch it: these fourteen are wrong against the list's OWN other days, which needs no view on that question. Worth saying because the two look alike and are not — one is a typing slip, the other is a question about which of two records the shop trusts.

325. A bill dated before the shop set any rate charged nothing for the gold

Defect 324 found bad prices in the coin list, so the metal rates got the same test. **They are clean** — nought quotes more than three times or less than a third of their purity's median, across 1,410 priced rows. The rates the shop sells from are sound.

The other half of that table is where this went. Of 20 purities, **10 have never had a sale rate** — 141 days each of zero. Nine of them hold no stock: scrap grades, stone and article categories that need no rate. The tenth, `IMT`, holds **220 tags**.

The chase, and the wrong turn in the middle of it

210 of the 220 are fixed-price and unaffected. Ten are not, and reading the code I concluded all ten were unsellable — `rateForPurity` fails, the caller returns "no rate available", and that refusal carries neither `certified` nor `handPriced`, so it is not offered the price box defect 255 added.

That was wrong, and the test I wrote to prove the second half of it failed. `rateForPurity` does not fail:

```
err = tx.QueryRow(...).Scan(&paise, &when)
if err != nil {
    return rupees.Zero, on, nil    // <- no error
}
```

On finding no row it returns a **zero rate and no error**. So `if err != nil` never fires for a metal nobody has priced, nothing was ever refused, and the eight per-piece tags were being billed correctly at their making all along.

I had built a fix on that premise. It was a dead branch and was reverted.

What is actually wrong, which is worse

A zero rate is a missing rate, and the tagged path did not say so. The untagged path eighty lines above gets it right:

```
if err != nil || rate.IsZero() {    // untagged lines
if err != nil {                    // tagged lines
```

Two halves of one screen disagreeing about the same question. The tagged half prices metal at nought whenever no rate is found — and **the rates begin on 1 April 2026**, so any bill dated before that finds no row for any purity at all. The making is charged, the gold is given away, and the bill looks entirely ordinary.

Proved: a bill dated 1 January 2020 for a 22kt tag prices without complaint.

Done

```
if err != nil || (rate.IsZero() && makingType != per_pcs)
```

The exception is narrow and earned. Eight tags on the shelf are priced per PIECE with amount 0.00 and the total equal to the making to the rupee — a photo frame and seven malas — and refusing those would make **₹13,620.00** of stock unsellable to close a hole they are not part of. Per-piece is the one basis whose arithmetic cannot involve a rate.

Three tests: the back-dated bill is refused, the metal-free piece still sells at its making, and a per-gram piece with no rate is refused because it would price to nothing. The first was checked against the bug.

The lesson, which is the third time this week

I read a code path, concluded what it did, measured the data to size the consequence, and started fixing — and the premise was false because the function three lines away swallowed the case I was reasoning about. **The test that failed was the one I wrote to confirm the part I was already sure of.** Defects 302 and 319 were the same shape: the check written to confirm the obvious half is the one that catches the reasoning.

326. The bill and the quote no longer answer the missing-rate question the same way

Defect 325 came from two halves of one screen disagreeing. The same shape has a second instance, and this one is written down in the code as an invariant:

```
// The refusals mirror the quote screen exactly, because a bill that prices
// a tag the quote screen refuses would be the more dangerous of the two.
```

That claim was true when written and 325 made it false. Where a grade has no rate:

```
the quote    falls back to the rate stored on the tag, and says so on its face:
              "the rate on the tag -- this grade has no rate feed"
the bill     refuses
```

Which is right depends on the document, so the divergence stays

A quote is a conversation. It can price something at a stale rate as long as it admits to it, and the quote screen does, in the row the customer is looking at.

A bill is what the customer takes away. **A bill that quietly charges a three-year-old rate is worse than one that will not print until somebody sets today's**, and there is no way to make a bill "admit" to a stale rate that survives being folded into a wallet.

So they diverge on purpose, and the comment now says which way and why instead of claiming a mirror that is not there.

The direction is the safe one. The bill refuses what the quote will price, never the reverse — which is exactly the case the original comment was written to prevent. If it ever runs the other way, that is the dangerous one.

The rest of the mirror was checked and holds

The quote's other refusal is the tag whose own recorded price the formula cannot reproduce — *"Cannot be priced here ... a computed quote would be a guess"* — and the bill has the matching route, the hand-priced box defect 255 added, across 1,626 non-fixed-price tags on the shelf. Certified pieces mirror likewise (defect 228 on one side, defect 300 on the other). Weight and making-value failures refuse on both.

One point of difference, deliberate, documented, and in the safe direction. The others hold.

Also checked, and clean

The untagged half of the bill fills a missing weight from the other side (net = gross and back), which the tagged half does not. **No tag needs it:** nought tags on the shelf have one weight and not the other. 369 have neither, and they are accounted for — 359 fixed-price, 8 priced per piece with the total equal to the making, and 2 that defect 325's change now refuses because they are priced per gram with no rate. Nothing there to fix.

327. The stock journal shows the metal a conversion yielded and hides the stones

reports/conversions.go reads mcj_stock_conversion_pieces and stops there. So when the shop melts a piece down, the screen lists the gold that came out of it and says nothing about the stones prised out first.

stone lines loaded	816
conversions carrying stones	393 of 859
pieces	24,478
carats	2,554.341
value	2,60,68,682.20
anything under modules/ that reads them	NONE

2.6 crore of stone detail, loaded faithfully and reachable from no screen, no report and no test. This was already *found* -- check_unwritten_tables.py has been printing mcj_conversion_stones as loaded-and-unread since defect 285 -- and then left there. A finding that sits in a tool's output for six defects is not being tracked, it is being tolerated, and the goal says build every screen that has data behind it.

The number is right

Checked three ways before it was written down. Against the legacy source, on all four figures at once:

legacy.stock_conversion_details	816	24,478	2,554.341	2,60,68,682.20
mcj_conversion_stones	816	24,478	2,554.341	2,60,68,682.20

and again by summing the fourteen stone kinds separately -- DIAMOND 430 lines at 1,45,05,443.00 down to YELLOW SAPPHIRE 2 at 2,59,000.00 -- which adds to the same 2,60,68,682.20 across the same 816 lines.

A separate report, not more columns

The two halves of a conversion do not share a grain or a unit. A document has both pieces and stones; one table would invite a reader to add a gram to a carat, and the value column would total the same document twice -- which is the thing check_no_double_totals.py exists to catch. The new report joins only single-row lookups, and the count comes back 816 in and 816 out, so nothing fans out.

Two absences belong to the capture, not the screen

Sieve is null on all 816 rows and cut on all but 22. That looked like a LEFT JOIN silently dropping matches -- a lookup table that never matches loses data without saying so -- so it was measured on the legacy side rather than assumed:

	legacy carries	loaded
seives_id	0	0
cut_id	22	22
color_id	482	482
shape_id	636	636

The loader dropped nothing. Sieve therefore gets no column at all: a column empty for every row the shop will ever see reads as a fault in the screen.

A zero that means two different things

88 lines carry no value. 24 of them are CUSTOMER DIAMOND and CUSTOMER STONE -- the customer's own stones, worth nothing *to the shop* and correctly zero. The other 64 are the shop's own stones recorded without a price, which is a gap in the capture. Same figure, two different facts, so the row says which rather than showing a bare zero.

Fixed: `reports/conversionstones.go`, registered as `conversion-stones`. 81 reports now render and all 81 show real data.

328. A tag raised in this system belongs to no department, and every one of the shop's own does

`department` is absent from the tag INSERT. The shop's 2,816 tags all carry one -- seven of the twelve departments hold stock -- and a tag raised here carries the empty string.

1014	6	WORKSHOP	71	4	Back Office
837	5	ORDER	6	10	ADMIN
614	9	SUB ADMIN	1	2	DATAENTRY
273	8	CASH			
2,816 tags, none blank					

Nothing goes missing, which is why this took until now to notice. The department report has carried a `(no department recorded)` row since defect 295, built for exactly this: an unattributed piece is visible rather than lost, and the totals still reconcile. So the failure is not a wrong number, it is a breakdown that gets less useful with every tag the shop adds -- the catch-all row grows and the department it belongs to never learns of it.

Fixed: a Department select on the tag screen, written through to the column. Checked on the running app rather than in the markup: the select renders with all twelve departments and the blank.

Blank stays allowed, deliberately

`selectField` offers an empty first option and the save accepts it. A required box the counter cannot answer does not produce a correct department, it produces a wrong one -- and a wrong department is worse than none, because none is already visible in a row built to show it.

Two tests, and the first was made to fail

`TestTag_RecordsTheDepartment` and `TestTag_SavesWithoutADepartment`. The first was run against the un-fixed INSERT before being kept:

```
--- FAIL: TestTag_RecordsTheDepartment
    tag_e2e_test.go:752: the tag records department "", want 6
```

which is the check defects 298 and 302 earned -- a test that passes on both sides of the change is not evidence.

The same sweep, a column that is a trap rather than a bug

`making_sales` is filled on 1,825 tags, named by no INSERT or UPDATE, and read by no non-test Go at all. That looks like a second making rate the bill is ignoring, which would be serious, so it was measured instead of assumed:

```
tags where making = making_sales      2,816 of 2,816
```

They are the same number on every row, so nothing is being ignored and no bill is wrong. **But the tag INSERT writes `making` and not `making_sales`**, so the first tag raised here breaks an equality that has held on every row since the capture. Nothing reads it, so there is nothing to fix today; recorded because the next person to find that column will find it half true, and a column that agrees with its neighbour on 2,816 rows and disagrees on the 2,817th is worse than one that never agreed.

Whether the legacy ever meant them to differ is the shop's to say, and is not worth asking about until something reads it.

329. The certification charge looked like a rate the bill was ignoring, and is not

`certification_type` is filled on 2,801 of the 2,816 tags and named by no INSERT, no UPDATE and no non-test Go at all. It holds `by_val` (2,683) or `per_ct` (118) -- the same shape as `making_type`, which decides how a charge is computed. Since defect 300 made certification a billing route rather than a label, a per-carat RATE being billed as though it were an AMOUNT would be wrong on 118 tags and wrong quietly.

It is not, and the bill is right to read `certification_charge` alone. The stored figure is the computed amount, not the rate:

```
per_ct tags with stones      118
charge = rate x ALL the tag's carats    111
charge = rate x SOME of its stone rows   6
neither                               1
```

The six are exact to the paisa against a subset of the tag's own stone rows -- one row on four of them, two rows on 70733986R1, three on 69492299R1 -- so the charge is computed from real parcels, not typed in. **Which parcel was certified is recorded nowhere**, and the excluded row is not the one you would guess: on 89558954 the 24 small stones totalling 0.420 ct were charged and the 1.250 ct pair was not.

The last one, 70630587, is **90 paise** out: 9,252.90 stored against 9,252.00 for its 10.280 ct row, and no subset of its rows gives 10.281 ct. One tag, under a rupee, in the legacy's arithmetic.

What was nearly claimed here

The first reading of this was from column averages -- `by_val` showed avg rate 31.02 and avg charge 31.02, and an average matching an average proves nothing about any row. Checked properly: **2,683 of 2,683** `by_val` tags have rate equal to charge, none differing. The conclusion survived; the evidence for it did not, and this is the fourteenth adjacent measurement caught before it was written down rather than after.

The gap that is real, and was not built

A certified tag raised in this system records a charge with no stated basis -- neither the type nor the rate has a box on the screen. Nothing reads either column, so no bill and no report is affected today, and adding two fields with no consumer would be building furniture. Recorded rather than built; it belongs with defect 328's `making_sales`, which is the same shape -- a legacy column the new screens quietly stop maintaining.

330. Building the fix made the decision list wrong, and the guard for that covers a quarter of it

Defect 328 put a department picker on the tag screen. A row on the shopkeeper's decision list said, in bold, **"nothing in the app writes it"**. That sentence was true when written and false by the time the tests passed, and nothing complained.

`check_blocker_citations.py` exists for exactly this -- defect 310 wrote it after B26 sat at the top of the decision list asking for something already built and decided. It reads every row keyed to a **blocker** and fails if that blocker is settled. The list is not mostly blockers:

rows on the decision list	28
keyed to a blocker (checked)	7
keyed to a defect (unchecked)	13
no id at all (uncheckable)	8

7 of 28. The guard was written against the one row that had gone wrong and sized to it.

The obvious extension is wrong, and the data said so

Fail a defect-keyed row when the changelog calls that defect fixed. Run against the thirteen, it fires exactly once -- on **defect 295**, which is marked `(fixed)` because the report's `(no department recorded)` row was built, while the question that defect raised (*which* department a new record should belong to) is untouched and genuinely still open.

That is a false positive on the only row it catches, so it was not built. **A defect's number and the question it raised have different lifecycles**, and treating the first as a proxy for the second turns a decision list into a nagging one -- which costs the shopkeeper the scarcest thing this project spends, and is the failure defect 310 was trying to prevent, not repeat.

The row was corrected by hand instead. What can be checked mechanically is already checked; what is left is a claim in prose about the state of the code, and this one is recorded so the next person knows the list is only as fresh as the last person to read it.

A choice was made on the shop's behalf, and it is reversible

The question offered three answers: attach a department to each app user, put a picker on the screen, or decide the column describes history only. **I built the picker for tags without being asked**, because 328 needed something and a picker is the least presumptuous of the three -- it does not preclude defaulting from the user later, and blank remains allowed. Bills and orders still have nothing, and the mechanism is still the shop's to choose. The row now says that rather than claiming the app writes no department at all.

331. The menu was honest about what works and silent about what is missing

Every unbuilt screen was simply absent. `navigation.go` said so as a principle -- *"Nothing renders a link to a screen that has not been built ... never advertises a 404"* -- and it kept the menu truthful about the 126 that work. It also meant a person hunting for a screen they used every day could not tell **not built** from **moved somewhere I have not found**, and the 67 missing ones existed only in a document they will never read.

All 193 legacy screens are now in the menu. The 67 lead to a stand-in that says what the screen would have shown and why it shows nothing, in the evidence's own terms rather than as a category:

```
Karigar Gold Issue          MANUFACTURING > Issue Voucher
Not built
The capture holds no karigar master and no issue-or-receive table. The
only manufacturing rows in the whole of it are karigar_gold_stock (2
rows) and karigar_wip_bal (1) -- which is why this survey counted rows
rather than reading table names.
What would change it: A karigar master and the vouchers behind it.
Whether this work is done on paper is Rahul's to say.
```

The count had to survive it, and did not at first

A stand-in registers through a new `TOD0For` field, never `LegacyFile`, because the coverage figure is counted by grepping non-test Go for `LegacyFile:`. I wrote that reasoning into a comment, then named the field on my own `TOD0Screen` struct `LegacyFile` -- and the figure read **193 of 193**:

```
Legacy menu screens rebuilt: STATUS.md says 126 of 193, taken as 193 of 193
```

Caught by `status_figures.py --check`, which is the whole reason that tool exists (defects 75 and 102: every hand-kept figure here was wrong at some point). The field is now `PHPFile`, and the comment above it records what it is not called and why.

A guard, because two lists that must agree will not

`check_todo_screens.py` fails if a screen is in both lists (built, and still apologising for itself), in neither (invisible again), or stands in for something the menu never listed. Proven by making a screen claim both:

```
1 screen(s) built and still standing in:      colour_master.php
1 screen(s) in the menu, neither built nor stood in for:  back_up.php
```

Now a step in `check_all.sh`. **Building a screen is now the act of deleting its stand-in**, and forgetting is a failed check rather than a stale page.

The menu had a rule about this and I broke it

`TestModule_NavigationIsDeclared` refuses a nav entry with no visibility policy, because one shows to everybody -- including staff who could not open the real screen. All 67 arrived with none, and the whole Go suite failed:

```
nav entry for /mcj/todo/change-psw-php has no visibility policy
-- it would show to users who cannot open it
```

Right rule. A stand-in appearing for people who would never have used the screen is noise dressed as an explanation. Each now takes the permission its section's BUILT screens predominantly use -- `VoucherRead` for RETAIL, where every unbuilt screen is a sale or purchase voucher; `TagPrintRead` for INVENTORY, where the two are barcode merge and split; `ReportRead` for the rest -- so a placeholder is visible to exactly the people who would have used the thing it stands in for.

The fallback would have hidden a bug. An unmatched heading falls through to `ReportRead`, which is a valid policy, so a broken split would still have passed the test with every screen silently mis-gated. Checked separately:

"RETAIL > Sales Voucher"	-> top="RETAIL"	found=true
"INVENTORY > Barcode Management"	-> top="INVENTORY"	found=true
"AUDIT"	-> top="AUDIT"	found=false

What the survey's own numbers turned out to be

Assigning a reason per screen does not reproduce defect 284's five buckets. Mine splits out two the prose already discusses as checked exceptions -- estimates (the Estimates report covers it) and ledger settings (the Ledger Master shows every column it has) -- which 284 folded one into "no source table" and one into "utility". Same 67, better shelved.

One of 284's evidence lines is wrong in kind. It says `sales_vou_appr`, `purchase_approval` x4, `barcode_merge`, `barcode_split` and `discount_master` hold **0 rows each**. None of those tables exists: the capture has 149 tables and not one of them is any of these. The doc insists on that exact distinction two paragraphs earlier -- *"loaded-and-unread is data the shop cannot see; never-touched is a table that should not exist"* -- so the stand-ins say **"no table for this exists among the 149 the capture contains. Not an empty table -- none at all."**

`item_merge` does exist, with 40 rows, and looked for a moment like the merge table the survey had missed -- the `karigar_gold_stock` mistake all over again. It is a 40-row fragment of the item master (`item_id`, `short_code`, `grp`, `uom`, `rate`), not a barcode-merge transaction. Checked before it was written down.

The landing page and the status doc have never agreed

While fixing the count I found they differ by one, and always have:

the landing page	127 of 194
STATUS.md	126 of 193

`statistics_report.php` sits under two top menus. `Progress()` counts screens per heading, so it counts that one twice; `status_figures.py` counts distinct links. Both are defensible and neither is wrong, but a person reading one and then the other has no way to know that. Not caused by this change and not fixed by it -- recorded, because a figure on the front page of the application is the one people quote.

332. Four of the six screens defect 117 flagged now write their detail rows, and the two that do not are the returns

STATUS.md has carried this caveat since defect 117: *"Six of the built screens post accounting and not their detail rows -- the sale, credit note, debit note, purchase, purchase against an order, and old gold purchase."* Checked one by one against what each save path actually inserts:

sale	<code>mcj_sales_invoices + mcj_sales_invoice_lines</code>
purchase	<code>mcj_purchase_invoices + mcj_purchase_contents</code>
purchase against an order	<code>mcj_purchase_invoices + mcj_purchase_contents</code>
old gold purchase	<code>mcj_old_gold_lines</code>
credit note (sale return)	voucher and stock movements only
debit note (purchase ret.)	voucher only

Four of the six were fixed and the caveat was never narrowed. It is the front-page qualifier on the coverage figure, and it has been overstating the gap by two thirds. `WritesInvoice` is true for exactly the two purchase kinds, which is what makes the remaining pair easy to state: **the gap is the returns.**

What that costs, in the one place it is not recoverable

`reports/gstr.go` builds the sales return from `mcj_sales_invoices` alone. A credit note writes no row there, so it appears in no GST return:

SALE-RET vouchers	3
output tax debited on them	61,899.33

That tax is reversed in the ledgers and absent from GSTR-1, so the return over-declares output tax by 61,899.33 while the books are right.

Checked against the legacy's own returns before it was written down, because a figure taken from voucher lines alone is the shape of mistake this file records thirteen times:

SR/2627/00001	20,04,854.00 + 60,145.62 = 20,65,000.00	voucher (0.38 round)
SR/2627/00002	16,505.00 + 495.15 = 17,000.15	voucher (exact)
SR/2627/00003	41,952.00 + 1,258.56 = 43,211.00	voucher (0.44 round)
	61,899.33	

The legacy recorded the thing GST asks for, and this system has nowhere to put it

`sale_return_main` carries a `sale_bill` column, filled on all three rows:

SR/2627/00001	->	GST/2627/00004
SR/2627/00002	->	GST/2627/00099
SR/2627/00003	->	GST/2627/00173

Every return names the bill it reverses. That is precisely what GSTR-1's credit-note table needs, and a credit note raised here records no such link, because it stores no document of its own -- only a voucher. **The old system was better at this than the new one.** Its three returns are also not loaded: no loader mentions `sale_return`, and there is no `mcj` table for them.

And the returned-bill banner cannot reach most bills

Defect 306 made a fully returned bill print GOODS RETURNED, driven by the barcodes coming back. Counted against the bills it has to work on:

bills	224	
with no tagged line at all	95	flag unreachable
tagged and untagged mixed	4	flag overstates
tagged only	125	works as intended

The 95 can never be flagged: the UPDATE requires a returned barcode to match a line on the bill, and an untagged bill has none. **42% of the shop's bills cannot be marked returned by any route.** The 4 mixed ones fail the other way -- the "everything is back" test filters untagged lines out, so returning the tagged half stamps the banner across a bill whose untagged half is still with the customer, which is the overstatement 306's own comment set out to avoid.

Not fixed here. A returns document, the link to the bill it reverses, and a credit-note section in GSTR-1 is a migration and a screen, not an edit, and half-building it at the end of an evening is how the GST return acquires a second wrong number instead of one. It is the next thing worth building.

333. A return is now a document, so it reaches the registers and the GST return

Defect 332 measured the gap and left it. Built:

migration mcj-1787416000	mcj_credit_notes, one table for both directions
creditnote.go	writeCreditNote, gated on a new IsReturn kind
creditnote.templ	an "Against bill" box on the two return screens
load_credit_notes.sh	the shop's own 6 returns, as documents
gstr.go	the credit notes, NEGATED, on the outward return
check_invariants.py	a return agrees with its own voucher (rule 24)

The GST return now nets. Over the whole captured period the outward return showed 6,98,933.87 of output tax; it now shows **6,37,034.54**, which is that figure less the 61,899.33 the three credit notes reverse. The books had it right all along and the return was over-declaring by exactly the tax on the notes.

Each note carries the invoice it reverses, which is what GSTR-1 asks for and what this system had nowhere to put:

SR/2627/00001	-20,04,854.00	-60,145.62	credit note against GST/2627/00004
SR/2627/00002	-16,505.00	-495.15	credit note against GST/2627/00099
SR/2627/00003	-41,952.00	-1,258.56	credit note against GST/2627/00173

SR/2627/00002 is Chandigarh against a West Bengal shop and prints IGST, taken from the note's own recorded flag rather than re-derived -- defect 302's rule, applied to the new table from the start.

A debit note stays off the outward return. It is a purchase return and belongs on the inward side; putting it here would understate what the shop supplied. There is a test that would catch it moving.

Two mistakes worth keeping

I compared the wrong two numbers and nearly filed a defect against the data. Checking each note against its voucher, two of six came out 15 and 32 paise short, and both were notes with a NEGATIVE round-off. The vouchers were right: a negative rounding sits on the opposite side of the voucher, so summing debits alone misses it. The comparison is now against the PARTY LINE, which is the note's face value whichever way the rounding fell -- and that is what rule 24 checks. Fifteenth adjacent measurement, caught before it was written down.

My own loader reported success while losing every party link. It joined ledgers on `legacy_id` where every other loader in this directory joins on `code`, and the two numberings are not the same -- one of the notes points at ledger 12154 and the highest `legacy_id` in the table is 11919. Nought of six resolved. It printed *"loaded 6 return document(s), every one tied to its voucher"*, which was true and beside the point, because the guard checked the voucher join and not the ledger one. **A loader that checks one of its two joins reports success for half a load** -- defect 294 in a different costume. There is now a second guard, and the message says both.

What is still not done

The debit note has a document but no inward-return section reads it yet: the purchase register and GSTR purchases still read `mcj_purchase_invoices` alone. The exposure is 25,229.67 of input tax across 3 notes, and it is overstated input rather than understated output -- the shop claiming back more than it should, which is the same size of error pointing the other way. Next.

334. The inward return claimed back input tax on goods the shop had sent away

Defect 333 closed the outward half and named this one. A debit note reverses the input tax claimed on goods returned to a supplier, and `gstrPurchasesQuery` read `mcj_purchase_invoices` alone, so the shop's three debit notes counted for nothing:

input tax on the purchase bills	5,21,364.93
less the three debit notes	25,229.67
the inward return now shows	4,96,135.26

The same error as 332's, pointing the other way. That one declared more output tax than was due; this one claimed back more input tax than was owed. One overpays the government and the other is the kind that gets noticed.

PR/2627/00001	SHISH JEWELS LTD	IGST	-5,45,749.83	-16,372.49
PR/2627/00002	HASTMILAP PVT LTD	IGST	-1,60,219.39	-4,806.58
PR/2627/00003	SWARN SHILP CHAINS AND JEWELLERS	CGST/SGST	-1,35,020.30	

Each is labelled *debit note -- goods sent back* in the "Posted to" column, so a negative line on a filed return says what it is rather than looking like a mistyped bill.

The two halves stay on their own sides, and each has a test that fails if it moves: a credit note on the inward return would understate recoverable input tax, a debit note on the outward one would understate what was supplied. Both were run against the un-fixed report first --

```
the debit note moved input tax by 0.00, want 300.00
the debit note row is posted to "", want it named as a debit note
```

-- because a test that passes before the change is not evidence of it.

What defect 117's list is now

Both return screens write a document, and both documents are read by the return that has to account for them. The list that defect 117 opened -- six screens posting accounting and not their detail rows -- is empty, four of the six having been quietly fixed long before anyone narrowed the sentence (defect 332).

335. The two registers disagreed about whether a return is trade

The purchase register has listed its own returns since it was written -- `v.type LIKE 'PURC%'`, with "Returned to a supplier" in its own filter. The sales register asked for `v.type = 'SALE-GST'` and nothing else. Same shop, same kind of document, two different answers.

And the sales register disagreed with the GST return printed beside it, by exactly the tax on the notes that defect 333 had just made count:

sales register	6,98,933.87	of tax
GST outward return	6,37,034.54	
the difference	61,899.33	the three credit notes

Listing a return and showing nothing for it is worse than leaving it out

The purchase register was not right either. It read `SUM(l.debit) FILTER (WHERE lg.name LIKE 'INPUT %')` -- debits alone -- and a return CREDITS the input tax it reverses, so all three appeared as rows of nothing:

03 Jun 2026	PURC-RET 1	PURCHASE_RETURN	0.00	0.00	0.00
29 Jun 2026	PURC-RET 2	PURCHASE_RETURN, ROUNDED OFF	0.00	0.00	0.00
01 Jul 2026	PURC-RET 3	PURCHASE_RETURN, ROUNDED OFF	0.00	0.00	0.00

8,66,219.00 of returns, present and worth nothing. A reader who sees the return on the screen concludes it has been accounted for, which is the one thing that cannot be concluded here. The "From" column names the ACCOUNT rather than the supplier for the same reason: it asked for credited ledgers, and a return debits the party.

Both are now debit-less-credit, and the party is taken from whichever side it sits on:

03 Jun 2026	PURC-RET 1	SHISH JEWELS LTD	-5,45,749.83	-16,372.49
29 Jun 2026	PURC-RET 2	HASTMILAP PVT LTD	-1,60,219.39	-4,806.58
01 Jul 2026	PURC-RET 3	SWARN SHILP CHAINS...	-1,35,020.30	-4,050.60

The registers and the returns now agree to the rupee

	goods	tax	total
sales register	2,20,03,143.00	6,37,034.54	2,26,40,175.00
GST outward	2,20,03,143.00	6,37,034.54	2,26,40,175.00
purchase register input tax	4,96,135.26		
GST inward	input tax	4,96,135.26	

A third thing the sign broke

The sales register decides the tax basis with `igstP > 0` and `localTaxP > 0`. A return carries the same tax negated, so every one of them would have been labelled "-", as though no tax had been charged on a document whose whole purpose is to give tax back. Tested against nought instead. **The bug and its fix arrived in the same change**, which is the only reason it is written down: nothing would have failed, and the column would have quietly been wrong on every return the shop ever raises.

336. Swept for the one-sided sum, and found two more — one of them in the total itself

Defects 334 and 335 were both the same mistake: reading one side of the ledger for a figure a RETURN posts to the other. Two instances is a class, so the rest of the reports were swept for it rather than waited for.

Most were already right -- the balances, the ratios and the carry-forward all read `SUM(debit) - SUM(credit)`; the cash flow takes both sides separately and the statistics one correctly-chosen side per figure. **Two were not.**

(This paragraph originally named the day book among those checked. It had not been opened, and the claim about it was wrong in both directions -- see defect 337, which corrects it and fixes what the correction found.)

The gross profit netted its sales returns and not its purchase returns

The screen has subtracted sale returns since it was written. Its purchases line names `v.type IN ('PURC-GSC', 'PURC-GC0')` and stops, so goods the shop had sent back still counted as bought:

	before	after
Purchases	2,34,35,413.81	2,34,35,413.81
Old gold bought in	50,71,370.00	50,71,370.00
Less returned to suppliers	--	8,40,989.52
Bought in total	2,85,06,783.81	2,76,65,794.29
What is knowable	-65,03,640.81	-56,62,651.29

The corrected bought-in figure is **exactly** the purchase register's own goods total after defect 335, which is how it should have been all along and was not: one screen said the shop bought 2,85,06,783.81 and the other 2,76,65,794.29.

And the HSN summary's caveat was adding itself to the total

Table 12 is gross, because a credit note carries no line detail and so no HSN to subtract from. That is defensible; leaving a person to discover it by comparing two screens is not, so the summary now says so. **The row that said it made the total wrong.**

total before the new row	2,40,66,454.00	
total with it	2,61,29,765.00	
the difference	20,63,311.00	the returns, ADDED

A disclosure row carrying money in the taxable and tax columns is summed into the footer like any other. **The line reading "this summary is SHORT by X" made the total X too BIG** -- and that fault was already there, on the shortfall row this report has carried since defect 156, waiting for a period with a detail-less bill. Both rows now state their figures in the sentence and carry no columns.

`check_no_double_totals.py` did not catch it, and could not: it looks for a footer that double-counts its own rows, and this footer added a row that was never trade. The test that catches it is the one that now exists.

The test encoded the bug

`TestHSNSummary_NamesTheBillsItCannotSee` asserted the shortfall row carried `taxable` and `tax` as amounts -- the exact thing that inflated the total. It passed for as long as the fault existed and would have failed the fix. Rewritten to assert the opposite: the figures appear in the sentence, and the row carries no money columns at all.

337. I described a report I had only skimmed, and the sentence was wrong

Defect 336 published this, about the sweep for one-sided sums:

the cash flow, the day book and the statistics take both sides separately

Checked, one at a time, because a claim about code is worth exactly what was read to make it:

cash flow	money_in = SUM(credit), money_out = SUM(debit)	accurate
statistics	one side per figure, the RIGHT one each time	loose
day book	SUM(l.debit) alone, and deliberately	WRONG

The day book has a comment above that line saying so: *"The amount of a voucher is the sum of ONE side, not of both. A balanced voucher's debits equal its credits, so adding the two reports every entry at twice its value."* It is correct, it is the opposite of what I wrote about it, and the sweep that produced the sentence never opened it.

The correction found a real one

If debits always equal credits, then BOTH sides carry the rounding, and one side is either

the transaction + a positive rounding = the document's face value
the transaction + |a negative rounding| = a rupee over it

So a voucher rounded DOWN reads high. Measured, and confirmed against a second query:

vouchers carrying a rounding CREDIT	129
overstated in total	35.30
the largest single one	1.00

Pennies, and the exact kind of difference that costs an afternoon when somebody holds a day book against a cash book and cannot find the last rupee. The amount is now the debit side less any rounding sitting on the credit side, which leaves a positively-rounded voucher untouched and brings a negatively-rounded one back to its face value.

Why this is filed as a defect and not a typo

The sentence was in docs/data-defects.md , which is where the next person looks to find out what has already been checked. **A wrong entry there is worse than a missing one:** it retires a question that was never asked. The sweep it described was real and its other findings stand -- but it covered two reports it named and not the third.

338. The purchase register's Total was not the bill's total

Found by carrying defect 337's question -- *where else does the rounding fall on the wrong side of a sum?* -- to the next report along. The register builds its Total in Go as g.Add(t) , goods plus tax, and the rounding is neither, so it never arrived:

	register	the supplier's bill	out by
GDP/2627/00001	29,99,998.60	30,00,000.00	1.40
GDP/2627/00013	1,46,361.41	1,46,380.00	18.59
bills carrying a rounding	23 of 43		
out by in total	25.47		
the worst single one	18.59		

A register whose Total is not the document's total is one nobody can tie to a supplier statement. It is small money and a whole afternoon: the figures agree on every bill that happens to round to zero and disagree on the 23 that do not, which is the pattern that makes a person check the arithmetic three times before doubting the screen.

The rounding now goes into the Total and stays out of the goods and the tax, which are its two parts. Checked against the books rather than only in a test:

register footer, purchases from a supplier	1,80,38,003.00
mcj_purchase_invoices, the same 24 bills	1,80,38,003.00

The same question, asked of the rest

The sales register's Total was already right, and for a reason worth writing down: it reads the PARTY line (SUM(debit - credit) over the non-tax, non-sales groups) rather than adding its own parts up. **A total taken from the document cannot drift from the document.** Where the register above assembles a figure out of the pieces it happens to display, that one asks the voucher what the customer owed -- and the rounding is already inside the answer.

That is the better shape, and it is why this defect exists in one register and not the other.

339. The 8.8 crore the suppliers owe is an opening balance, not a debt this system watched build

Why is 8.8 crore owed to the shop by its own suppliers? has sat on the decision list unchanged since defect 51. The figure is right -- it reproduces exactly -- but as asked it cannot be answered, because it does not say WHERE the money came from. Split:

17 supplier accounts in debit	8,80,84,874.22
from opening balances	8,80,38,622.00
from trade this system recorded	46,252.22

The trade half is not a mystery and never was. Six accounts, all of them expense suppliers with a small prepayment:

INTERGLOBE AVIATION (WB)	20,873.00	SBS LEGAL	10,000.00
GROWTH IS LIFE FOUNDATION	11,330.00	BHARTI AIRTEL	2,976.04
ARPITA MERCHANDISE	600.00	VODAFONE	473.18

Airline tickets, phone bills and a legal fee. Nobody needs to be asked about those.

What is actually being asked

One account carries three quarters of it:

SHREE SAROJ JEWELSS	opening	6,98,45,346.00	debit
	13 postings	-19,07,775.00	
	balance	6,79,37,571.00	

and ten more carry 2,01,01,051.00 between them with **not one posting** -- their balance is their opening figure and nothing else has touched it since.

The opening is the shop's own, carried in as it was found. Checked against the legacy's own file rather than assumed:

legacy.ldgr_open_balance	ldgrcd 9797	debit 69845346.00	acc_year 2026-2027
mcj_ledgers	code 9797	opening_balance 69845346.00	debit

So the question is not "why do your suppliers owe you 8.8 crore". It is "your books opened the year saying SHREE SAROJ JEWELSS owed you 6,98,45,346.00 -- is that real, and what is it?" That is answerable in a sentence by somebody who knows the shop, and the old one was not. It also belongs beside **B19**, whose opening trial balance is short 15.32 crore: both are questions about what the year opened with, not about anything the new system did.

Nothing is fixed here and nothing was wrong. The report reproduces, the loader is faithful, the arithmetic is standard. What changed is that the ask now names one account and one figure.

340. B14 asks the shop for three ledgers its own vouchers already name

B14 has been open since screen 19: three of the six payroll components point at ledger codes absent from the chart -- 2677 (P.TAX), 2657 (I.TAX), 3373 (D.A) -- and it asks whoever runs payroll which ledgers those should be. **Two of the three are answered by the shop's own postings, and the third has no money in it.**

First, a hypothesis tested and refused

The obvious explanation is that those numbers are legacy `id` s rather than `ldgr_code` s and the loader matched the wrong column. Checked:

```
3372 exists as a CODE (id 2372) = DIRECTORS REMUNERATION
2677, 2657, 3373 exist as neither id nor code
```

So they are missing, exactly as B14 says. Recorded so nobody spends the same half hour.

P.TAX -- answered to the rupee

```
ledger 3242 PROFESSIONAL TAX (group: Payable)
  51 postings, 6,140.00 CREDITED
salary sheets, April to July 2026
  1,470 + 1,600 + 1,470 + 1,600 = 6,140.00
```

Credited, which is what a deduction withheld from staff should be. The choice is not arbitrary:

PROFESSIONAL TAX (COMPANY) (3340) also exists, with one posting of 2,500 debited -- the employer's own PT, a different thing -- and the shop's vouchers pick 3242 for the payroll deduction.

D.A -- answered within 600

```
ledger 4291 DEARNESS ALLOWANCE (group: Employee Benifit Expences)
  47 postings, 3,22,800.00 debited, across three payment runs
salary sheets for the same four months          3,22,200.00

posted 2026-05   11 rows   77,200.00
posted 2026-06   24 rows  1,61,400.00  <- two months in one run
posted 2026-07   12 rows   84,200.00
```

Agreeing within **600.00**, and the 600 sits in the June run that covers two months at once. Close enough to name the account and not close enough to call it reconciled.

I.TAX -- blocks nothing at all

No employee has ever had an I.TAX amount: no rows for it in `ledger_wise_payroll_detail`, and `staff_salary_sheet_payroll_detail` has no column for it. It is a component the shop configured and never used.

What is left of B14, and what it turned up

P.TAX	1,470-1,600 a month	the vouchers say 3242
D.A	77,200-84,200 a month	the vouchers say 4291
I.TAX	nil	nothing to decide

A different question came out of it. The three components that DO resolve -- PF, ESI and HRA, together 7,39,428.00 over sixteen months of sheets -- point at 3372 DIRECTORS REMUNERATION. That account carries 12 postings totalling 14,40,000.00, which is 1,20,000 a month and reads like one director's pay; and all twelve people on the payroll sit in the chart's own **Staff** group, not a directors' one. Whether staff PF, ESI and HRA belong in a directors' account is a smaller question than B14 asked and a real one.

Nothing changed here. The mapping is the shop's chart of accounts and naming it from the evidence is not the same as deciding it. B14 goes from *"tell me three ledgers"* to *"confirm two your vouchers already use, ignore the third, and say whether staff benefits belong in DIRECTORS REMUNERATION"*.

341. No coin in the capture was ever billed from the coin price list

Defect 54 asks which coin rate is right, the daily list or the item master, and rests on a sentence: *"Coins are priced and sold as products from this list, not by weight off the shelf."* **The shop's own bills say otherwise, and they say it without a single exception.**

coin lines billed	32
priced by WEIGHT (a rate and a weight)	23
flat-priced (rate 0, a figure typed in)	9
matching the coin list's per-piece price	0

Not one. And the 23 weight-priced lines track the day's METAL rate:

within 5% of that day's rate for that purity	23 of 23
within 1%	10
exactly equal to it	8
average, billed rate over metal rate	0.9826

Worked through on one line, because a ratio is not an explanation:

SILVER COIN-5 GM, 21 July 2026, 40 pieces, 46,900.00	
the day's SLV999 sale rate	232.50 per gram
the line's rate	232.50 per gram
5 g x (232.50 + 2.00 making)	1,172.50 per piece
the coin list that day	1,220.00 per piece

The bill used the metal rate to the paisa and the coin list not at all.

So the question was the wrong one

Nothing in this system prices a sale from either source. `mcj_coin_rates` is read by the coin-rate screen and its own report and by nothing else; `mcj_items.rate` is read by nothing at all. Coins are sold from a tag (14 of the 32 lines) or as a typed untagged line (18). The item master's five-year-old rates undercharge by 79% only if something prices from them, and nothing does.

That does not make the stale rates harmless -- a screen built tomorrow could reach for `items.rate` and be 4.8x low -- but it is a trap, not a leak, and it has never cost the shop a rupee.

What is worth asking instead

The shop kept `coin_rate_master` **daily**: 5,966 rows, 52 items, 115 trading days, every day it traded between 3 April and 19 August 2026. Somebody typed those in. And no coin was billed from them.

What is the daily coin list for? It may be the counter display, or what the shop pays to buy coins in, or a price it quotes and then discounts to the metal rate at the till -- the average of 0.9826 is consistent with the last. This measured SALES only, and cannot see the rest.

Defect 54 stays open, because the list being unused for billing is not the same as the item master being right, and neither is a thing this repository can decide. But it is no longer HIGH: there is no undercharging happening.

342. The shop was getting better at recording hallmarks every year, and this system could not record one at all

Defect 296 noted that nothing here can set the hallmark flag. What that costs was not measured until now, and the measurement is the argument:

gold jewellery tagged in	tags	hallmarked	%
2024	198	12	6.1
2025	570	159	27.9
2026	387	203	52.5

Not an abandoned practice -- an adopted one. The share has climbed nine-fold in two years and stands at better than half. A tag raised in this system carried `hallmarked = false` whatever the piece was, so the first one made here would have started that climb again from nought.

Fixed: a Hallmarked box on the tag screen, written through to the column. Confirmed on the running app, not in the markup. Two tests, and the first was run against the un-fixed INSERT before being kept:

```
--- FAIL: TestTag_RecordsAHallmark
tag_e2e_test.go:794: the box was ticked and the tag records no hallmark
```

Unticked means unticked, and the second test says so: a screen that assumed a hallmark would put the doubt hanging over 739 old pieces into every new one too.

Defect 50's table has a wrong column

Checking the population turned up an error in the entry this came from. Every figure in defect 50 reproduces today to the paisa -- except one column:

	no hallmark	value	"On the shelf"	really
18kt	411	5,95,69,420.89	539	505
22kt	217	4,20,34,082.23	352	319
14kt	104	1,37,93,189.74	196	165
9kt	7	67,169.00	38	34

The four "On the shelf" figures sum to **1,125**, and the sentence above them says **1,023** -- which is the right number and the one today's shelf gives. The column matches neither the tags in stock nor all tags ever made for those purities (547/361/247/41).

It matters in the direction that flatters: 739 of 1,125 reads as **66%** unrecorded, and 739 of 1,023 is **72%**. Corrected in place.

What is still the shop's to say

Whether the 739 are unhallmarked or hallmarked-and-unrecorded is unchanged, and no measurement here can settle it. But the ask is narrower now: the flag IS in live use and rising, so "is hallmarking recorded in the ERP at all" has an answer -- **yes, on 52.5% of this year's gold** -- and what is left is the older stock and the paper trail behind it.

343. Eighty-three orders carried forward from earlier years all say they were placed on 1 April 2026

Found while checking whether defect 28's four-year-overdue order was real. It is, and the thing that made it look wrong is a different defect.

83 order lines promise delivery before the order was taken -- the earliest due 9 August 2022 against an order dated 2026. That reads as a broken delivery date and it is not. The order NUMBERS carry the financial year:

FY in order_no	orders	ordered_on
2122	2	all 2026-04-01
2223	5	all 2026-04-01
2324	6	all 2026-04-01
2425	14	all 2026-04-01
2526	56	all 2026-04-01
2627	201	2026-04-01 to 2026-08-19

The delivery dates are right and the order dates are the artefact. Eighty- three orders still open at the year end were carried forward and stamped with the first day of the new financial year. Exactly one order genuinely numbered in 2026-27 also falls on that date, so 84 orders share it.

Checked against the legacy before blaming the loader, which is faithful on both sides:

legacy.customer_order	284 orders, every date1 in 2026
legacy.customer_order_main	delivery dates 2022:1 2023:2 2024:6 2025:39 2026:318
mcj_customer_order_lines	the same distribution exactly

So the legacy did this, and the capture reproduces it.

What it costs

Every screen that shows an order's date shows **01 Apr 2026** for 83 of the shop's 284 orders -- 29% of them - as though a third of a year's book was taken in one morning. `statistics.go` goes further and FILTERS on it: a statistics run covering April 2026 counts all 83 as orders taken that month, when the oldest of them was taken four financial years earlier.

Defect 28 is unaffected and was right. It measures against `delivery_date`, which is the sound column, and its headline -- `RP/2223/00194` due 9 August 2022, 1,474 days over -- is a real order from financial year 2022-23, not a typo.

Not fixed, and why

The true order date is not in the capture. The financial year is recoverable from the order number and nothing finer, so the honest repair is to show "*carried forward from 2022-23*" rather than a date the shop never recorded -- and inventing 1 April 2022 to replace 1 April 2026 would be trading one wrong date for another that merely looks better. Which of those the shop wants is a question, and it is now on the list.

344. There is no making percentage over 6 g, and the shop's own shelf shows why

Defect 47 ends with three overlapping bands and no way to choose:

6 - 100 g	12%
6 - 250 g	15%
6 - 300 g	14%

The question went on the list as "*what is the making percentage on a gold ornament over 6 g?*" **The shelf answers it, and the answer is that there is no such number.** Of the 186 gold jewellery tags over 6 g priced as a percentage of the metal:

12%	52 tags	1,71,22,284.58
18%	44 tags	58,10,166.60
10%	32 tags	1,06,88,365.48
11%	31 tags	45,54,552.00
15%	15 tags	21,50,634.63
14%	10 tags	53,81,403.10
20%	2 tags	91,968.00

Seven rates, not three -- and the two commonest, 12% and 18%, are one that IS in the ladder and one that is not in it anywhere above 6 g.

The ladder's PRINCIPLE holds, though, past where it stops

A heavier piece costs proportionally less to make, and the shelf keeps doing that after the table runs out:

band	tags	average	commonest	range
6 – 10 g	41	15.24%	18%	10–20
10 – 20 g	46	14.17%	18%	10–20
20 – 50 g	81	12.07%	11%	10–18
over 50 g	18	12.61%	12%	10–18

The average falls from 15.24% to 12.07% across the range, which is the ladder carrying on. **The spread does not narrow with it** -- every band runs 10% to 18% or 20% -- so the rate is chosen piece by piece and the band only moves where the middle sits. The rise at over-50 g is 18 tags and not worth reading.

So the question dissolves and a smaller one is left

Nothing prices from the ladder: making comes from the tag, as defect 47 already says. Nobody needs to pick between 12, 14 and 15 because nothing consults them.

What could be asked instead is a design question, not a data one: **should the tag screen suggest a making percentage from the piece's weight**, given the shop plainly has a habit -- about 15% just over 6 g easing to about 12% by 20 g -- and given that whatever it suggested would be overridden a third of the time by the look of that spread.

Recorded rather than built. A suggestion nobody asked for, on a screen that prices real stock, is worth a conversation first.

345. Six of the tag prices that "differed" differ by the tax and nothing else

Is a printed tag price a fixed price or a starting point? went on the decision list with **24 of 28 flat-priced sales differed from it; 9 were above**. The figures reproduce. What they mean does not survive looking at the ratios:

ratio of billed to tag price		
0.718 – 0.839	8 sales	all SILVER
0.952	1 sale	
0.97087	6 sales	all COINS
1.000	4 sales	
1.003 – 1.163	6 sales	
2.98769	2 sales	the same piece twice

0.97087 is 1/1.03. Every one of those six is a coin, and the ratio is exact to five places on five of them. The tag price is GST-INCLUSIVE and the invoice line's amount is the taxable value, so those pieces sold at their printed price to the paise -- $140,534.00 \times 1.03 = 144,750.02$ against a tag of 144,750.00. Nothing was discounted.

	sales	amount
tax, not a discount	6	15,966.00
sold at the tag exactly	4	--
genuinely below the tag	9	6,005.00
sold above the tag	9	4,864.00

93% of the 17,107.00 that looked given away is tax. The real movement is 6,005.00 off on nine pieces against 4,864.00 on top on nine others -- **net 1,141.00 across 28 sales**, which is a shop haggling, not a shop leaking.

So the answer is different for a coin and for silver

coins	sold at the printed price, every time, 6 of 6
silver	sold 16-28% under it on 8 of the 9 that moved at all

For a coin the tag is a price. For silver it is a starting point, and the starting point is about a fifth above where the piece actually goes. Ten of the 28 honoured the tag, not four.

The one that is not either

Two sales of SILVER EXCLUSIVE PCS billed **1,942.00 against a 650.00 tag** -- 2.98769 times, identical on both. Three at the tag price would be 1,950.00, eight rupees off, so a three-piece sale against a per-piece tag is the likely reading and not a certain one. Worth a look at that tag rather than a rule.

346. The 4,000 a gram placeholder is on 360 tags, five purities, and 4,391 grams

Defect 36/58 put *"re-rate the tags carrying placeholder rates"* on the decision list with **154 tags at exactly 4,000/g at 18kt**. Measured today the shelf holds more than twice that, and the reason it is certainly a placeholder is not the round number.

purity	tags at exactly 4,000/g	tagged on
18	307	5 distinct days, 2024-04-01 to 2025-11-21
22	39	3 days
14	8	3 days
95	5	2 days
9	1	1 day
360 tags, 4,391.116 g net		

22kt and 9kt cannot share a metal rate. They differ by more than double in gold content, and here they carry the same figure to the paisa, on a handful of batch-entry days. That is a default being written into a rate field, not a price anybody set.

It priced the tags, it did not merely sit beside them

The tell that matters is whether the figure was USED. For every one of the 307 at 18kt:

amount = rate x net_weight + the tag's own stones	307 of 307, to the rupee
---	--------------------------

So the stock value on those tags was computed from 4,000 a gram. This was not obvious: on 234 the amount is exactly rate x weight and on the other 73 it is not, which reads as two different populations until the stones are added back -- and then it is one.

What it is worth

the 360 tags, valued as tagged	1,75,64,464.00
the same metal at today's rate per purity	5,69,65,495.93
difference	3,94,01,031.93

18kt alone is 2,50,96,717.57 of that. The shelf stands at 14,23,61,188.20, so this is **28% of it** sitting on a number that was never a rate.

The difference is not all error. These were tagged between April 2024 and November 2025 and gold has risen since, so some of the gap is the market. But none of it can be *read* from the tag, because a single figure covering 9kt and 22kt cannot be a historical rate either.

Not re-rated here

Re-rating stock is the shop's decision and not a repair -- it moves the value of the shelf by 3.94 crore, and which basis to use (the rate the day it was tagged, today's, or a stock-take) is exactly the sort of choice this project does not make on its own. The ask on the list is now a number and a basis rather than a count.

347. The opening stock that is not moving is not the old stock

8.5 crore of opening stock has sold 1.7% in five months -- is that expected? went on the decision list beside **"251 pieces date from 2023 or earlier"**. Both figures reproduce exactly:

	tags	still held	sold	turned
opening stock	1,571	1,545	26	1.65%
everything else	1,245	1,030	215	17.27%

Ten times faster, as stated, and the opening shelf stands at **8,51,58,772.85**.

The two halves of the sentence describe different stock. The 251 pieces from 2023 or earlier are real and they are worth **14,00,024.17** -- 1.6% of the figure in front of them:

tagged	on the shelf	value	at the 4,000 placeholder
2016	2	3,910.00	--
2021	106	1,36,530.00	--
2022	45	1,52,639.38	--
2023	98	11,06,944.79	--
2024	196	1,64,44,212.40	129
2025	692	6,65,62,696.93	229
2026	406	7,51,839.35	--

8,30,06,909.33 of the 8,51,58,772.85 was tagged in 2024 and 2025. The stock that is not moving is one to two years old, not ancient. The ancient stock is 1.6% of it and could be given away without changing the answer.

And more than half of it is priced at the placeholder

Of the 360 tags carrying exactly 4,000 a gram (defect 346), **358 are opening stock**, and they hold **4,69,17,081.28** -- **55% of the opening shelf value**.

So the question as asked cannot be answered from this data. "Is 1.7% expected for 8.5 crore of stock" assumes the 8.5 crore means something, and the majority of it rests on a figure that spans 9kt and 22kt alike.

What is worth asking

Is 1.7% expected for stock tagged in 2024 and 2025? -- which is what this actually is. The 26 pieces that did sell went for 1,08,995.31 between them, so what moved was the cheap end and what stayed is the value.

Turnover is counted by PIECES here, as the original figure was. By value it is worse, and saying so precisely needs the re-rating in defect 346 first, which is the shop's decision and not a measurement.

348. The shop does honour an order's locked rate, and its own bills show the rule

Do you honour an order at the rate it was taken? has sat on the decision list with the exposure attached -- 60,14,200.75 to honour the live book. Nobody asked the 47 orders that have already been delivered and billed.

order lines matched to their bill by barcode	47
billed at EXACTLY the locked rate	33
billed BELOW it	12
billed ABOVE it	2

Seventy per cent to the paisa, and the exceptions have a shape. All twelve billed below the lock; between them the customers paid 21,855.52 less than the locked rate would have charged. Four were billed at exactly that day's rate where the day's rate had fallen -- 18kt locked at 12,045 and billed at 11,075, silver locked at 275.00 and billed at 231.50, both the market that morning.

So the working rule is not "the rate it was taken". It is the better of the two for the customer, which is a more generous rule than the question assumes and one the shop follows without being asked.

Two went the other way

barcode	purity	locked	billed	net wt	the customer paid
38323709	22	13,610.00	14,618.00	2.100	2,116.80
95831681R1	22	13,583.00	13,758.00	1.440	252.00

Both were billed at exactly that day's rate, which was higher than the lock. On the reading above -- the customer gets the better of the two -- these are the two where it did not happen, and together they are 2,368.80.

Whether that is a mistake or a renegotiation the data cannot say. It is two bills out of 47 and under two and a half thousand rupees, so it is worth a glance and not an investigation.

What this does to the question

The 60,14,200.75 still stands: it is what honouring every LIVE order would cost if metal stays where it is. But the question behind it -- do you honour the lock -- is answered by the shop's own counter, 33 times exactly and 12 times better.

What is left to ask is narrower: **the two that were charged the day's higher rate.** And a design question that follows from it -- should the sale screen warn when a tag reserved against an order is being billed above the rate that order locked? Nothing does that today, and it is the only place these two would have shown up before the customer paid.

349. The "second system" in defect 66 is called `test_bill` in the schema

Defect 66 is a BLOCKER and its arithmetic holds. What it never says is what the second numbering IS, and the column name says it plainly:

```
legacy.sale_bill_link (id, mcj_bill, TEST_BILL, rt_code, comp_code, ac_yr)
```

Everything else about the pair is consistent with a parallel system that raised MORE documents than the real one over the same window:

	links	distinct	lowest	highest
mcj_bill	135	132	1	220
test_bill	135	134	1	338
real bills on file	224		1	224

The gap widens monotonically and never reverses. Not one link has the test number BEHIND the shop's, and the offset grows from 0 at bill 1 to 119 by bill 220 -- the two series start together and the test one pulls away by about one bill in two.

bill 1-2	offset 0
bill 4-6	offset 3
bill 27-39	offset 21
bill 99-113	offset 79
bill 212-220	offset 118-119

That is what a practice or staging environment looks like beside a live one: the same series, the same retail and company codes, and extra documents in the test one that the shop never issued.

It does not dissolve the blocker, it aims it

Defect 66's danger is exact and unchanged: **test number GST/2627/00007 is also a real bill of the shop's**, belonging to ANITA ROY, 4 April 2026, 32,886.00 -- while the link says it stands for the shop's GST/2627/00004, which is MANISH KUMAR SHARMA on 2 April for 20,65,000.00. If anybody ever took away an invoice printed by the test side, the number on it names a different customer's sale in the shop's own books.

Whether that ever happened is the whole question, and it is one sentence long. The old ask -- *"which bill numbering is the shop's own, and do customers hold invoices carrying the other one?"* -- has a first half the schema answers: the shop's own is `mcj_bill`, and the other is labelled test. What is left is the second half.

Nothing here proves no such invoice left the counter. A column name is evidence and not a guarantee, and 83 collisions are 83 whatever the environment was called.

350. D599 holds 57% of the shelf, has never made anything, and is not in the database

What is D599? has sat on the decision list as an open question about a maker code. Searched properly this time -- **every text column in the legacy capture, 1,877 of them across 149 tables:**

D599 appears in barcode_main.karigar_code 470 times
and nowhere else at all

No ledger, no address book, no master of any kind. The same scan run against the seven next-largest codes finds them in exactly two places each, `barcode_main.karigar_code` and `stock_conversion_main.karigar_code` -- **except D599, which is absent from the second.**

code	tags	conversions	value on the shelf
D599	470	0	8,09,58,545.00
D142	112	4	1,63,442.40
DL236	104	2	1,36,366.55
d76	87	7	1,88,271.90
A031	51	37	1,40,42,503.07

A karigar makes things. AO31 holds 51 tags and appears on 37 conversions -- metal going out and coming back, which is what the code is for. D599 holds nine times the tags, **57% of the whole shelf by value**, and has never been party to a single one.

What it looks like instead

D599 tags on the shelf	470
carried in as opening stock	410
priced at the 4,000-a-gram placeholder	316
tagged across	15 days, Oct 2021 to Jan 2026

316 of the 360 placeholder tags in defect 346 are D599's. Against that, A031 is 16 opening of 51 and still being tagged in August 2026 -- a code in live use. D599 looks like a marker on stock that arrived rather than a person who made it.

The question that replaces it

"What is D599" cannot be answered from the capture -- the scan settles that. But it can be asked much better:

Is D599 a goldsmith, or is it how stock carried in from before the system was marked?

8,09,58,545.00 of the shelf turns on the answer, and if it is the second, then 470 tags name a maker that never existed and the manufacturing history everyone has been looking for was never missing.

Not decided here. A code that has never touched a conversion is evidence and not proof -- a supplier who only ever sold finished pieces would look the same.

351. The raw stock says who has it, in a column nobody had read

Defect 133 left the question *"where is the raw metal now, and who has it?"* and established why the second half is hard: the 146 issues and 14 receipts live in a scratch report, not a table, so there is no movement history to load.

The opening position answers the "who" outright. `mcj_raw_stock` carries a `holder` column, taken straight from `barcode_main_rawstone.type`:

holder	side	rows	net weight
sh	Dr	10	27,326.609 g

One holder, ten rows, every gram. The loaders elsewhere use the same three-letter vocabulary -- `sh` own, `kr` karigar or dealer, `co` party -- and the legacy table contains **no** `kr` **and no** `co` **row at all**, so this is not a subset that was loaded selectively:

<code>legacy.barcode_main_rawstone</code>	<code>type='sh'</code>	10 rows	27,326.609 g	(the whole table)
---	------------------------	---------	--------------	-------------------

And the two gold figures are not the same figure

opening raw gold, from the table	12,304.524 g
the June report, after 146 issues and 14 receipts	11,998.005 g
difference	306.519 g

Defect 133 quotes the second and the shop's own raw stock screen shows the first. They differ by 306.519 g because one is the opening and the other is where the report left it in June -- which is the point that entry was making, and is worth having as two numbers rather than one.

18kt	5,566.291	22kt	4,973.818
14kt	1,073.049	995	524.720
22 coin	84.490	9kt	82.156
		gold	12,304.524 g
		silver etc	15,022.085 g

So the ask narrows to one half

"Who has it" has an answer on file: the shop, all of it, at opening. Nothing was ever recorded as out with a karigar or a party -- which sits oddly beside a report showing 146 issues, and that oddity is now the question rather than the whole thing.

Where is it now is still unanswerable here, for defect 133's reason. Nothing new was found that changes it.

352. Every scheme is still marked active, and 235 of them have already matured

B23 asks *how scheme settlements were recorded*. Measured, the answer is that nothing in the capture records one.

enrolments	239
still flagged is_active	239
already past their maturity date	235
registered between	2017-04-07 and 2026-02-11
maturing between	2017-07-25 and 2027-01-11

Not one enrolment is marked closed, settled or paid out, including schemes that matured in 2017. `is_active` is true on all 239 and there is no other column that could carry the fact.

What the customers did

fully paid up	174
part paid	64
never paid a rupee	1
	239
paid in, across 2,144 payments	87,90,900.00
payable in total	93,01,000.00
still owed BY customers	6,55,100.00 on 65 schemes
matured AND fully paid	173 schemes
their maturity value	77,37,300.00

77,37,300.00 of gold or credit is owed by the shop on schemes the customers finished and nothing says whether it was handed over.

Two readings, and the capture cannot separate them

Settlement might have been recorded as a SALE -- the customer takes jewellery against the scheme. Of the 173 parties, **23 have a bill on file**. That looks damning and is not: the invoice capture runs April to August 2026 while these schemes matured from 2017 onward, so almost every settlement would predate the bills that survive. **The test is not available, not failed.**

What can be said is that the scheme tables themselves carry no settlement of any kind, and that whatever happened is not recoverable from them.

And nothing posts a scheme payment

Bank	1,287 payments	46,99,400.00
Cash	528	25,49,300.00
Card	329	15,42,200.00

87,90,900.00 came in, by three named modes, and **no controller writes a voucher for any of it** -- which is defect 233's question (*which account does an instalment credit?*) seen from the money's side rather than the ledger's. The payments are loaded, reportable, and absent from the books.

Both asks now have a number. B23 is 77,37,300.00 owed on 173 finished schemes; defect 233 is 87,90,900.00 received and unposted. Neither is answerable here.

353. Two customers' stones are being held against orders the shop cancelled

Return the 14.43 carats of customers' own stones has sat on the decision list as one lot. It is not one lot. Joining each receipt to the order it came in against splits it cleanly:

receipt	customer	carats	held	the order
CG/2627/00003	RASHMI SARAF	2.750	58 d	CANCELLED
CG/2627/00005	POOJA PRASAD	2.850	34 d	CANCELLED
CG/2627/00005	POOJA PRASAD	1.300	34 d	CANCELLED
CG/2627/00006	TAHA.M. TAMBAWALA	6.250	30 d	Urgent
CG/2627/00007	PRABHA BHANSALI	0.930	7 d	Urgent
CG/2627/00008	PRABHA BHANSALI	0.170	7 d	Process
CG/2627/00009	P. K. MUNDRA	0.180	6 d	Urgent

6.90 carats belong to two customers whose orders were cancelled. RASHMI SARAF's stone has been here 58 days and POOJA PRASAD's four have been here 34, in both cases against work the shop is no longer doing. Nothing is owed to the shop on either; there is no reason to still have them.

The other **7.53 carats across three customers are against live orders** -- Urgent or in Process -- and holding those is the job. They are not the same question and should not be on the same line of a to-do list.

The screen can do it

`partysample.go` writes `returned`, `returned_on` and `return_note`, so marking these back is a counter action and not a build. Nothing is blocked.

What is actually being asked

Not *"return 14.43 carats"* but:

Two customers' stones are held against cancelled orders, 58 and 34 days on. Are they being kept back deliberately, or did the cancellation just not reach the counter?

The second is likelier and neither is knowable from here. Cancelling an order does not prompt anybody to hand back what came in with it -- there is no link from the order's status to the goods receipt, and defect 335 found the same shape on the sales side, where a status change left another record standing.

354. The vendor's login out-ranks the super admin, on the one right the super admin was denied

Should the vendor's rps login keep all four override rights? sat on the decision list with the four named and nothing about who else has them. `users` carries them as four flags -- `barcode_change`, `back_date_entry`, `due_date_change`, `home_rate_change` -- and across 24 logins:

can post to a closed period	9
can change the gold rate	6
can edit a tag	4
can move a due date	4
ALL FOUR	2

The two are `rps` (ADMIN) and `Shibananda` (SUB ADMIN).

And the super admin is not one of them

login	role	tag	back-date	due date	rate
GKS	SUPER ADMIN	1	0	1	1
Shibananda	SUB ADMIN	1	1	1	1
rps	ADMIN	1	1	1	1

GKS , the super admin, cannot post to a closed period. rps can. Whoever set these up withheld back-dating from the highest role in the shop and left it with the vendor's login and the sub admin.

That is either deliberate -- the vendor needed it to fix data and the owner did not want it -- or it is what happens when a support login is set up once and never revisited. Nothing here can tell which.

Why back-dating is the one that matters

The other three are visible in their own records: an edited tag, a moved due date and a changed rate all leave the thing they changed. Back-dating does not announce itself -- a voucher posted into a closed period looks exactly like one posted at the time, and this rebuild has already found what a wrong date does in defect 325, where a bill dated before the rate feed gave the gold away.

Nine logins can do it, so it is not a vendor question alone. But the vendor's login is the one on the list, and the answer to "should it keep all four" now has a shape: it holds a right that the owner's own login does not.

355. The eight Thursday orders have all already passed, and Thursday is the day the shop avoids

Eight live orders are promised for a Thursday, when the shop is closed went on the decision list as something to reschedule. Both halves check out and the conclusion does not.

Thursday is the closing day, and the evidence is strong:

holidays recorded	109	
on a Thursday	108	2016-01-07 to 2024-12-26
on a Monday	1	2017-05-01, May Day

The eight are real. They are also all in the past:

live lines with a Thursday delivery date	8
falling after today	0
earliest	2024-02-15
latest	2026-07-30

Nothing is coming up to be missed. These are past-due lines that happen to carry a Thursday date, and they are already counted among defect 28's overdue work.

The practice is sound

Across the 149 live lines that carry a delivery date at all:

Friday	35	Monday	19
Sunday	27	Tuesday	12
Saturday	25	THURSDAY	8
Wednesday	23		

Thursday is the least-used day of the week, by a clear margin -- 5.4% of live promises against Friday's 23%. The counter is already avoiding it. Eight over two and a half years is not a habit, it is the exceptions.

What is actually worth noting

the holiday list was last written 2024-12-26

Twenty months dead, so **nothing in this system can tell anybody today whether a date is a closing day**. The rule is recoverable -- 108 of 109 rows say Thursday -- but no screen applies it, and a delivery date typed for a Thursday gets no warning.

That is the only part of this worth doing anything about, and it is small: a warning on the order screen, or a standing report. The eight need nothing.

356. Correcting a legacy tag's making rewrites its value on a different basis

Found by testing an invariant the code's own formula implies. `pricing.Calculate` ends:

```
Total: rupees.Sum(amount, making, l.OtherMaking)
```

so for anything this system prices, `total = amount + making + other`. Across the shelf:

tags	2,816
where total = amount + making + other	1,678
where it does not	1,138

All 2,816 are legacy rows, and certification closes **none** of the 1,138. The legacy simply did not compute `total` the way this system does. Worked through on the largest:

29522987	DIAMOND SET, on the shelf		
rate 11,800.00 x net 64.138	=	7,56,828.40	metal
amount		19,76,308.40	metal AND stones
making		64,138.00	
total on the tag		8,20,966.40	= metal + making

The tag's total excludes its stones and its amount includes them. The gap is 12,19,480.00 and it is exactly the stone value.

Why that is a defect and not a curiosity

`tagcorrect.go` reprices whatever it touches and writes the result:

```
SET ... making = $6, total = $7
```

total there is price.Total, which is amount + making + other -- and the repricing rebuilds amount from the tag, stones included. So **correcting the making on 29522987 would move its value from 8,20,966.40 to 20,40,446.40**, a silent 12,19,480.00 added to the shelf by a screen the user opened to fix a making charge.

tags on the shelf that would move	1,038
by more than 10,000.00	62
by more than 1,00,000.00	3
worst single tag	12,19,480.00
the shelf if every one were corrected	-4,10,891.26

The net is small and that is the trap. Positives and negatives nearly cancel, so a bulk revaluation would look almost free while individual pieces moved by lakhs in both directions.

Not fixed here, and the reason is which basis is right

The screen is not wrong to price the way the rest of the system prices. What is wrong is that it changes a figure the user did not ask it to change and does not say so. Three repairs are possible -- leave total alone unless the making actually moved it, carry the legacy's own basis per tag, or re-state every tag on this system's basis and accept the shift -- and the third is defect 346's re-rating question again, which is the shop's.

What can be said without deciding: 1,038 tags on the shelf carry a total that this system would compute differently, and nothing warns anybody.

357. The companion invariant holds on every row, which is what makes 356 measurable

Defect 356 rests on amount being trustworthy. If amount were computed on two bases as well, there would be nothing solid to measure total's drift against and the finding would be two unknowns instead of one. So it was tested:

tags	2,816
where amount = rate x net weight + the tag's stones	2,816
where it is not	0

No exceptions at all -- across legacy and every purity, stone-bearing and plain. amount means one thing.

Pinned as the 25th rule in check_invariants.py, so it stays that way. Proven able to fail rather than assumed: tightening its tolerance from a rupee to a millionth surfaces 172 rows, which is the rounding in a numeric(18,2) against a product of two numerics and exactly why the tolerance is there.

x a tag's amount is its metal and its stones 172 tag(s) ... -- 172 more than the 0 allowed

What the pair says together

amount	metal + stones	2,816 of 2,816	one basis
total	two bases	1,678 of 2,816	the other 1,138 exclude stones

The problem in 356 is not that the legacy data is arbitrary. It is that ONE field carries two conventions and the field beside it does not, so nothing in the tag itself says which convention any given row follows -- and a screen that reprices has no way to tell.

358. The same two-basis problem is on the sold lines, and there it is harmless

Defect 356 found `total` carrying two conventions on the shelf. The sale side was checked for the same thing, because the sale writes its line amount from the same formula:

```
amount = price.Total = rate x net weight + stones + making + other making
```

sold lines	367
satisfying that	138
not	229

The same split, and largely explainable: of the 229, **122 carry a certification charge** -- which `pricing.Calculate` deliberately leaves out (defect 47) and a certified piece is billed at a price agreed across the counter (defect 300) -- **38 are flat-priced** with rate 0, and **68 are untagged** lines typed by hand.

Why it does not matter here and does on the shelf

```
writers that change a bill after it is raised:  
creditnote.go:572  UPDATE mcj_sales_invoices ... SET is_returned = true
```

That is the only one, and it sets a boolean. Nothing in this system ever recomputes a sold line's amount, so a line whose composition follows the legacy's basis simply stays as the shop billed it. It is a record, not an input.

A tag is the opposite: `tagcorrect.go` reprices it and writes the answer back, which is exactly why 356 is a defect and this is not.

And the aggregate was already guarded

```
bills 224  
goods_amount equals the sum of its lines 224
```

Checked, clean, and it turns out this is already the first rule in `check_invariants.py` -- *"every bill's lines add up to its goods"*, from defect 220. **Re-derived a guard that already existed**, which is a fair use of an hour: the rule now has a second pair of eyes on it and the answer did not change.

359. Old gold was the one document type with no rule saying it agrees with its voucher

Listing the invariants out rather than adding to them found a gap in a family. Three of the four documents this shop raises had a rule:

a bill agrees with its own voucher	defect 308
a purchase agrees with its own voucher	defect 309
a return agrees with its own voucher	defect 332
old gold	-- nothing

That is the sort of gap that survives because nobody writes the family down. Each rule was added when its own defect was found, and old gold never had a defect, so it never got a rule -- which is not the same as being checked.

Checked now:

old gold documents	25
whose lines sum to what the party was credited	25
that disagree	0

Exact on every one. The party is CREDITED here, unlike a sale: the shop buys metal across the counter and owes for it.

Proven able to fail, on the second attempt

The first falsification was no falsification. Removing the `ROUNDED OFF` exclusion should have broken the arithmetic and did not -- **old gold vouchers carry no rounding line at all**, so the clause it removed was doing nothing on this data. The rule still passed and that proved nothing.

Flipping the comparison to the debit side did it:

x old gold agrees with its own voucher	25 ... -- 25 more than the 0 allowed
--	--------------------------------------

A test that passes when you break it has not been run, which is the same lesson as defects 298 and 302 arriving by a different road: this time the break itself was wrong rather than the assertion.

Twenty-six rules now, and the four document types are symmetric.

360. Thirteen orders say nothing about what was ordered, and now a fourteenth would be caught

The family check that found defect 359 was run across the other documents too -- does each carry at least one line?

purchases with no contents	0 of 43
old gold vouchers with no lines	0 of 25
conversions with no pieces	0 of 859
orders with no lines	13 of 284

All thirteen are the shop's own. Every one has nought rows in `legacy.customer_order_main` as well, so nothing was lost loading them:

C0/2122/00013	C0/2223/00054	C0/2324/00536	C0/2425/00053	C0/2425/00432
C0/2122/00334	C0/2223/00950	C0/2324/00783	C0/2425/00208	C0/2425/00489
RP/2223/00008	RP/2223/00116	RP/2627/00029		

Twelve are carried-forward orders stamped 2026-04-01 (defect 343). **Ten are still undelivered** -- work that cannot be finished because nothing on file says what it is. Two are repairs, which is a customer's own gold.

Both live paths were already right

```
the order screen    refuses an empty order: "Say what the customer has asked for."  
the outstanding list shows the thirteen instead of dropping them (defect 260)
```

So this is not a new hole. Defect 260 found the same ten from the other side -- an inner join had made them invisible on the one screen that exists to list what is outstanding -- and fixed it.

What was missing is the rule underneath. The sale side has had one since defect 220; the order side never did, so both guards were code and neither was data. Pinned now as the 27th, with the thirteen allowed, and the allowance proven exact rather than assumed:

```
x  an order says what was ordered    13 order(s) ... -- 1 more than the 12 allowed
```

Set the allowance to twelve and it fails by one, which is the only way to know the number is the number.

361. A rule that would have caught a bug I wrote, proven by writing it again

Continuing the family sweep: does every document name a party that exists?

bills	224	0	without one
credit/debit notes	6	0	
orders	284	0	
old gold lines	25	0	
purchases	43	0	
customer goods	7	0	

Clean everywhere. **Added anyway, because it would have caught defect 333's loader bug** --

`load_credit_notes.sh` joined ledgers on `legacy_id` where every other loader in that directory joins on `code`, nought of six notes resolved, and the loader's own guard checked the VOUCHER join and not the ledger one. It printed success. I found it by measuring afterwards, which is luck dressed as method.

Proven by reproducing the bug

Rather than argue the rule would have caught it, the bug was written back in -- the wrong join restored, the guard removed -- and the loader run:

```
loaded 6 return document(s, every one tied to its voucher and its party  
x  every document names a party that exists    3 document(s) naming no party
```

The loader still says "and its party". That sentence is now false and the script cannot tell; the rule can.

And the wrong join half worked

Three of six, not nought. Some ledgers' `legacy_id` happens to equal another ledger's `code`, so the broken join resolves a few rows **to the wrong party** rather than to none. That is worse than failing cleanly and it is why a spot check of one row is not a test: pick the right row and the bug is invisible.

Loader and data restored; 28 of 28 rules hold.

362. No other loader has 333's bug, and the rule that catches it only catches half of it

Two questions came out of defect 361. Both answered.

Is the wrong join anywhere else?

Every loader that reaches a ledger was read:

```
load_bank_accounts    load_carry_forward    load_customer_goods    load_invoices
load_estimates        load_oldgold            load_orders             load_schemes
load_payroll          load_payroll_history    load_purchase_invoices load_vouchers
repair_opening_balances
```

All of them join on `lg.code`. The three that appear to join on an id -- `ON lg.id = l.ledger_id` -- are second-stage joins against a UUID this system already resolved, which is correct. `load_credit_notes.sh` was the only one, and it is fixed.

How dangerous is the wrong key?

ledgers	785
whose code equals their own <code>legacy_id</code>	0
whose <code>legacy_id</code> is ANOTHER ledger's code	43

The two keys never coincide by accident on the same row, so a wrong-key join never quietly works. **But 43 times in 785 it resolves to a DIFFERENT ledger** -- 5.5% -- which is why the reproduction in defect 361 attached 3 of 6 notes rather than nought.

A join on the wrong key does not fail. It picks somebody else.

And that is the half rule 361 cannot catch

every document names a party that exists finds a document with NO party. The 43-collision case gives the document a party -- the wrong one -- and the rule passes it, because the ledger it names is perfectly real.

So the new rule caught 3 of the 6 broken notes and would have missed the other 3. **Stated here rather than left implied**, because a guard that catches half a bug is more dangerous than none if nobody knows which half.

The only thing that catches mis-attachment is a loader comparing its output to its source, party by party -- which is what a loader's own guard is for, and what `load_credit_notes.sh` now does on both its joins.

363. Seven loaders join a ledger without checking it resolved, and one of them has 272 that did not

Defect 362 ended by saying the loader's own guard is the load-bearing check. So which loaders have one?

```
guard their ledger join      carry_forward, credit_notes, customer_goods,  
                             estimates, vouchers  
do NOT                      bank_accounts, invoices, orders, payroll,  
                             payroll_history, purchase_invoices, schemes
```

Seven join and do not check. Six of those seven are clean anyway -- invoices, orders, purchase invoices, schemes and both payroll loaders resolve every row, and rule 361 now watches four of them from the data side.

(Wrong about one of them. payroll_history has 37 rows with no ledger -- found in defect 364 by measuring each TABLE rather than reading each loader. The other five are as stated, and the count of clean ones is five, not six.)

`load_bank_accounts.sh` **is not.**

```
mcj_bank_accounts rows      398  
  with a ledger             126  
  WITHOUT one               272
```

It is the shop's data, and the ledger load is complete

The obvious suspicion is 333's again -- a wrong key, or a partial ledger load. Both were checked and both are innocent:

```
legacy.ledger_master rows      785  
mcj_ledgers rows               785      every one with a code and a name  
  
legacy.ledger_bank_details rows 1,071  
  whose party_ldgr exists in ledger_master 349  
  whose party_ldgr does NOT              722
```

722 of the legacy's own 1,071 bank-detail rows name a ledger code that is not in the legacy's own ledger master. The bank details outlived the accounts they belong to. Nothing was lost in the migration; the rubble is original.

That is the mechanism behind defect 284's warning -- *"398 rows sounds operational and 272 name no ledger at all, so a screen over it would mostly be built over rubble"* -- which said what and now has a why.

What is worth doing

Not a guard on `load_bank_accounts.sh`: a guard that fails on 272 rows the shop genuinely has would stop the load for a fact nobody can fix. **The other six should have one**, because for them nought is the true number and a guard would hold it there. That is a small change and it is not made here, because six loaders' worth of new failure modes on the evening a rule was added is how a green check becomes a habit of ignoring red ones.

364. A correction: one of the two payroll loaders does not resolve every row

Defect 363 said *"invoices, orders, purchase invoices, schemes and both payroll loaders resolve every row"*. Five of those six are right. `payroll_history` **is not**, and the way to find that out was to measure each rather than group them:

sales invoices	224	0 without a ledger
customer orders	284	0
scheme enrolments	239	0
purchases (supplier resolves)	43	0
payroll history	209	37

I had checked the loaders and not the tables. Four of the five were clean, and "both payroll loaders" was a sentence written about a pair when only one had been looked at.

The 37 are the shop's data, checked the same way as the bank details

rows with no ledger	37	
distinct staff codes	4	8827, 4023, 8855, 8828
months covered	2025-04 to 2026-03	
components on those rows	4,39,640.00	
rows in legacy.staff_salary_sheet_payroll_detail naming a code that is NOT in legacy.ledger_master	37	

Exactly the same 37. The four codes return nothing from the legacy's own ledger master, so this is not a join failure: the payroll history outlived the staff ledgers, as the bank details outlived their accounts (defect 363).

Four people, a year of months each, and 4,39,640.00 of PF, ESI, professional tax, HRA and dearness allowance recorded against ledgers the shop has since deleted. Whether those staff left and their accounts were cleaned up is a question the data cannot answer and the shop can in a sentence.

What it means for the guard

`load_payroll_history.sh` cannot carry the guard 363 proposed either: nought is not its true number, 37 is. It joins on the right key and gets the right answer, and the answer includes 37 rows whose party no longer exists.

Two of the seven unguarded loaders turn out to have a real non-zero, which is the reason the sweep was worth doing at all -- and the reason not to have added six guards on the assumption they would all pass.

365. Four loaders now check their own ledger join, and the first version of one guard tested the wrong column

Defects 363 and 364 established which loaders could carry the guard: five of the seven unguarded ones resolve every row, and two -- `bank_accounts` (272) and `payroll_history` (37) -- have a real non-zero that is the shop's own data. `load_payroll.sh` writes no ledger column at all, so four remained.

load_invoices.sh	mcj_sales_invoices	ledger_id IS NULL
load_orders.sh	mcj_customer_orders	ledger_id IS NULL
load_schemes.sh	mcj_scheme_enrolments	ledger_id IS NULL
load_purchase_invoices.sh	mcj_purchase_invoices	supplier_code resolves

All four pass on the real data. **Each was then broken on purpose**, its ledger join switched from `code` to `legacy_id` -- 333's bug, exactly:

load_invoices	ERROR: 219 row(s) in mcj_sales_invoices match no ledger
load_orders	ERROR: 267 row(s) in mcj_customer_orders match no ledger
load_schemes	ERROR: 227 row(s) in mcj_scheme_enrolments match no ledger

And not one of them catches every row. 219 of 224 bills, 267 of 284 orders, 227 of 239 schemes -- the remainder resolved through the wrong key to the WRONG ledger, which is defect 362's 43 collisions doing exactly what was predicted.

The fourth guard was decoration and the break proved it

`load_purchase_invoices.sh` passed with its join broken. The reason is that the column the guard reads is not the column the join feeds:

supplier_code	<- si.ldgr	straight from the legacy
supplier_name	<- lg.name	through the join

So *"does supplier_code resolve to a ledger"* is a real check and a blind one: it cannot notice the join failing, because its input never went through the join. A second guard was added on the column that does:

```
ERROR: 42 purchase(s) name a real ledger and carry no supplier name
```

Both guards are kept. The first catches a legacy code that is not in the chart; the second catches the join that fetches its name breaking. They fail on different things and the first one's blindness is why it looked sufficient.

The test method failed once too, before any of this

The first attempt to break all three used `sed` with `\\|` alternation, which BSD `sed` does not support, so nothing was substituted and all three "passed". Reported as *"guard is decoration"* for a moment, when the truth was that the test had not run. **Twice in three days a falsification has been the thing that was broken** (defect 359 was the other), which is its own small lesson: when a break does not break anything, suspect the break.

366. Re-running the purchase loader emptied the register hanging off it

Found by triggering it. After adding the ledger guards in defect 365 I re-ran the four loaders, and the next full check came back with two failures that had nothing to do with ledgers:

```
x a purchase says what it bought 43 purchase bill(s) with no contents
x report (no data) /mcj/reports/mrp-purchases
x report (no data) /mcj/reports/purchase-contents
```

`load_purchase_invoices.sh` DELETes its 43 invoices and reloads them. `mcj_purchase_contents` hangs off them with `ON DELETE CASCADE`, so every re-run takes all 60 content rows with it -- and the loader printed OK `mcj_purchase_invoices` 43 rows, input tax ties at 521364.93 while it did.

This is defect 294's shape and 294's own fix was never applied here. `load_invoices.sh`, `load_conversions.sh` and `load_orders.sh` each carry a guard that PUTS THE ROWS BACK when they cascade something away. This one had none, so the family was three of four -- the same gap that defect 359 found in the invariants and 363 found in the loader guards.

Fixed the same way as its siblings: count the contents, reload them if they are gone, and fail loudly if the reload does not take. Run three times in a row to prove the repair is idempotent -- 43 invoices and 60 contents, every time.

Two mistakes inside the fix

The repair called `"${HERE}/load_purchase_contents.sh"`, copied from `load_invoices.sh` where `HERE` exists. **This loader does not define it** -- it `cd` s to the repository root instead -- so the first run died with `HERE: unbound variable` **after** it had already emptied the table. A guard that crashes is worse than no guard: it leaves the damage and stops.

And the check that caught all this was `check_all.sh`, run out of habit at the end of an unrelated piece of work. **The loaders' own output said OK.**

367. The cascade family, swept to the end: eleven of eleven now survive a re-run

Defect 366 was found by tripping over it. The obvious next question is how many more are waiting, so the whole family was listed from the database rather than from memory:

```
SELECT ... FROM pg_constraint WHERE contype='f' AND confdeltype='c'
```

Eleven cascading foreign keys between mcj tables. Three of them -- `mcj_voucher_lines`, `mcj_sales_invoice_lines`, `mcj_customer_order_lines` -- are written by the same statement that writes their parent, so a re-run rebuilds both and there is nothing to lose. The other eight split two ways:

child written by ANOTHER script	parent repairs it?
<code>mcj_ordered_stones</code>	yes (defect 294)
<code>mcj_sold_stones</code>	yes (defect 294)
<code>mcj_conversion_stones</code>	yes
<code>mcj_stock_conversion_pieces</code>	yes
<code>mcj_purchase_contents</code>	yes (defect 366, yesterday)
child written by the SAME script	reloaded in the same run?
<code>mcj_design_stones</code>	proven
<code>mcj_estimate_lines</code>	proven
<code>mcj_scheme_payments</code>	proven

The three "same script" ones were proven, not assumed

Reading a script and concluding it reloads what it deleted is how defect 366 survived:

`load_purchase_invoices.sh` looked fine too. So all three were run and counted on both sides:

	before	after
design_stones	3,416	3,416
estimate_lines	2	2
scheme_payments	2,144	2,144

Unchanged. Then the whole check: 9 of 9, after six loaders had been re-run in one sitting.

The family is closed. Eleven cascades, eleven that survive a re-run, and the one that did not was found by running the check out of habit rather than by reading anything.

368. Every loader verifies its own output, and it took three wrong greps to establish it

The family after the cascades: does each loader check what it wrote? Swept by grepping for the pattern the loaders I had written use --

loaders	39
with SRC/DST comparison	26
without	13

-- and the thirteen looked like a real gap. They are not one.

The second grep was wrong too. Looking for `|| { echo "FAIL` found assertions in all twelve real ones (`load_all.sh` is an orchestrator), between one and five each, which suggested the opposite conclusion: verified, just differently.

`load_masters.sh` **looked worst and is the most strictly checked file here.** One assertion by that grep, seventeen tables written -- and a `check()` function it calls once per table with an EXACT expected count:

<code>check mcj_items 322</code>	<code>check mcj_metal_rates 2820</code>	
<code>check mcj_packets 205</code>	<code>check mcj_ledger_groups 96</code>	
<code>check mcj_states 36</code>	<code>check mcj_value_bands 43</code>	<code>... 17 in all</code>

A pinned absolute is stronger than a source-to-destination comparison, because it survives the source changing underneath it.

`load_codelists.sh` looked like it wrote nothing -- it builds `INSERT INTO ${target}` dynamically, so a literal grep cannot see the table names -- and it compares src to dst per list, sets `fail=1`, and ends `[ "$fail" = "0" ] || exit 1`.

What is actually true

All 38 loaders verify their output. No gap, nothing to fix, and the family is closed like the four before it.

The part worth keeping

Three greps, three wrong answers, and the truth only when the files were opened. Each grep encoded "a check looks like the checks I write" -- SRC/DST, then `|| { echo "FAIL`, and both missed a `check()` helper and a dynamic loop.

That is the same error as defect 359's inert falsification and 365's `sed` that did not substitute: **the measurement passed and measured nothing**. Third time this week, and the tell is the same every time -- a sweep that returns a suspiciously tidy gap, in files nobody has read.

369. Three returned pieces never went back on the shelf, one of them worth 18.28 lakh

Testing the complement of an existing rule. Rule 19 says *a tag the trail sold is off the shelf*; nothing said the opposite for returns. Every tag that is off the shelf was checked for what its LAST movement was:

last movement	tags	value	
SALE-GST	196	56,60,136.67	sold, as it should be
NEW-BAR	33	29,89,769.63	defect 314's known 42, with
OP-STK	9	400.00	the 400.00 defect 317 could not place
SALE-RET	3	18,53,671.28	returned -- and still gone

A return puts a piece back. These three did not come back:

36568032	21,274.08	sold GST/2627/00099, returned SR/2627/00002 -- TWICE, ten minutes apart on 16 June
22157192	4,271.20	sold GST/2627/00013, returned SR/2627/00002
70630587	18,28,126.00	sold GST/2627/00004, returned SR/2627/00001

The largest is the one from defect 333: **GST/2627/00004 is MANISH KUMAR SHARMA's 20,65,000.00 bill, returned in full on SR/2627/00001** -- and the piece that came back is still marked out of stock.

The shop's own data, checked before blaming the load

legacy.barcode_main.stk_status	
70630587, 36568032, 22157192	0 out of stock
the 2,575 on the shelf	1

The legacy recorded the return movement and left the flag down. Nothing was lost loading it.

It cannot happen again, which is why the allowance is three

`creditnote.go` sets `in_stock` back to true and writes the SALE-RET movement in the same transaction (defect 192), so a return raised here restocks or neither happens. The three are historical and the rule is pinned at exactly three -- proven by setting it to two and watching it fail by one.

What is not knowable here is whether the pieces are physically in the shop. The trail says they came back; the flag says they are gone; and 18.28 lakh of diamond set is a thing somebody can walk over and look for.

370. Fifty-one pieces left the shelf on a sale that no bill line names

The complement of rule 8, tested both ways. Every tagged sale line has its movement -- **256 of 256, clean**. The other direction is not:

SALE-GST movements	324	
whose barcode appears on no invoice line	51	
referencing a bill not in the capture	38	before the invoice window
referencing a bill we hold	13	across 9 bills

Seven of the nine bills explain themselves

The bill wrote the piece as an UNTAGGED line and the shelf still knew which tag went. GST/2627/00200 is the shape of it:

```
billed  27119951 GOLD CHAIN | 28873579 DIAMOND TOPS | 35220094 DIAMOND BRACELET
        (blank) GOLD ORNAMENTS x3
moved   27119951, 28873579, 35220094, 61583434, 69669400, 78534314
```

Three tagged lines, three untagged, and six pieces off the shelf. **The invoice cannot say which pieces the customer took and the trail can** -- which is a one-way link, not a contradiction, and it is how a quarter of this shop's lines are billed.

Two bills name a different piece from the one that left

```
GST/2627/00207  98,300.00
billed  69343561R1  DIAMOND RING      95,437.00  off the shelf, correct
ALSO moved 53541166  DIAMOND RING      52,445.17  STILL IN STOCK

GST/2627/00081  1,800.00
billed  69005805   SILVER BRACELET    1,748.00  off the shelf, correct
ALSO moved 55112377R2                                NO TAG ON FILE AT ALL
```

Neither bill has an untagged line to hold the extra. **A 52,445.17 diamond ring carries a sale movement against a bill it is not on, and is sitting in the case.**

Both are inside existing allowances -- 53541166 is one of the nine that rule 19 permits, 55112377R2 one of the 1,298 movements naming no tag -- so nothing fails today. An allowance is a number, not an explanation, and these two now have one.

The trail's orphans are large and correctly handled

movements	9,031	
naming a barcode with no tag	1,298	across 499 barcodes

Fourteen per cent, which is what a trail spanning years looks like beside a table of current tags. Every reader was checked: `movements.go` uses a LEFT JOIN LATERAL and carries `(t.id IS NULL) AS tag_gone` to say so on the row; its total counts the movements alone; `backdatestock.go` joins to stock on purpose, to value what was held; `hallmarking.go` only counts. **Nothing drops them silently.**

371. The payments report counted bill numbers and called them bills

Testing the complement of rule 13 -- it says a payment names a bill that exists, with 8 allowed; nothing said whether a bill was paid.

bills (not cancelled)	224	
with no payment row at all	83	1,82,16,133.00
paid in full	115	
underpaid	26	
overpaid	0	

The 83 look alarming and are mostly not. Followed through:

of the 83		
the customer has a RECEIPT voucher	44	
no receipt, and the customer owes nothing	26	40,62,806.00
no receipt, and the customer owes	13	6,28,034.00

The 26 were settled in old gold. Their ledgers are credited **38,77,608.00 on PURC-OG vouchers** -- the customer brought jewellery in and took new against it, which is the exchange this trade runs on. Another 21,08,211.00 came from returns and 12,44,421.00 from journals.

So only **13 bills, 6,28,034.00**, are a customer who simply still owes.

The defect is in the caveat, not the data

payments.go already says the right thing -- *"THIS IS NOT A RECEIVABLES REPORT ... a partial record of how payment arrived, not of whether it did"* -- and then sizes it wrongly:

said	149 bills of 224, 52,52,946.00
true	141 bills of 224, 51,93,365.00

It counted DISTINCT BILL NUMBERS in the payment tables. Eight of those numbers name no bill this shop has, and 59,581.00 came in against them -- the same eight rule 13 allows, which is why nothing failed. A number that matches an allowance is not thereby explained.

Corrected on the screen, with the old figures and why they were wrong kept beside them. The caveat now also says what a missing payment row usually means, since the answer took four queries to establish and the next person should not need them.

372. A rule called "names a document" that 8,165 movements passed by containing a word

Third turn running, an allowance turned out to hide something. This time the allowance was fine and the RULE was not.

```
("a stock movement names a document", 470 allowed)
SELECT count(*) FROM mcj_stock_movements WHERE btrim(reference) = ''
```

It checks that a text field is not blank. Across the 9,031 movements:

6,395	"New Create"	a word
1,770	"Opening"	a word
470	nothing	the allowance
396	a document number	SALE-GST 324, CST-ORD 44, SALE-RET 17, ISS-HM 4, REC-HM 4, EST 2, PURC-IN 1

Only 396 movements in 9,031 name a document, and that is correct. A piece being created or carried in has no document to name -- "New Create" and "Opening" are honest labels for events that have none. Nothing here is wrong with the data.

The title was wrong. It promised that every movement is traceable to a document, and 8,165 of them satisfied it by containing a word. Anybody reading the rule list -- which is the point of the rule list -- would have taken the trail to be far more documented than it is.

Renamed to "**a movement's reference is filled in**", which is what it does, with the four-way split written above it so the next reader does not have to run the query.

And the 470 is two events missing their label

278	OP-STK	should say "Opening"
192	NEW-BAR	should say "New Create"

161 of those 192 barcodes are still on the shelf, worth **57,41,225.14** -- and it does not matter, because the label they are missing carries no information either. A tidiness allowance, not a hole, and now recorded as one.

The pattern across three turns

defect 370	an allowance held a diamond ring sold on a bill it is not on
defect 371	an allowance made a report overstate its own coverage
defect 372	an allowance was fine and the rule above it was misnamed

A number that passes is not a fact that was checked. Three different shapes of the same lesson, from the same habit: reading what a green line claims rather than what it measures.

373. Fifty-three of the fifty-seven pieces with no beginning have one, on the tag

Continuing the allowance sweep. *Every piece has a beginning* allows 57 tags that have movements but no NEW-BAR arrival. Split by where the tag says it came from:

created_from	tags	name an arrival document	name none
purc	43	43	0
ret	7	7	0
rec	7	3	4

53 of the 57 are fully documented and only the TRAIL is missing an entry. The tag carries `created_from` and `voucher_no` -- a purchase bill, a sale return -- so the arrival is recorded, just not as a movement. That is a gap in one record, not in the shop's knowledge.

The four that are not

82342508B	GOLD BANGLE	no karigar	2022-09-30	6,099.60
22128188	GOLD LOCKET	karigar "1"	2016-04-01	3,750.00
45365399	MALA	karigar D602	2016-04-01	160.00
62038342A	GOLD BANGLE	no karigar	2023-02-27	13,455.00

All four still on the shelf, **23,464.60** between them. Received from a karigar, no document, and two of them naming no karigar either -- so nothing on the tag or in the trail says where they came from.

Two are dated 1 April 2016, which is an opening date from a migration two systems before this one. A piece that arrived in an opening balance nine years ago has no arrival document because there was never one to have.

The allowance is right and now says what it holds

Nothing changed but the comment above the rule, which now carries the split. The number 57 was correct and meant three different things; it now says which.

That is the fourth allowance opened in as many turns -- 470, 8, 9, and 57 -- and the fourth time the number was accurate and uninformative.

374. The 31 uncleared-date vouchers are two different things, and neither can happen again

Fifth allowance opened. *Clearing a cheque sets both columns* permits 31 vouchers marked `bank_cleared` with no `bank_cleared_on`.

```
all 31    type PMNT, dated 13 May to 5 Aug 2026
all 31    carry a legacy_v_id -- not one was raised here
all 31    touch a Bank Account; none touches cash

12    carry a cheque number, a cheque date and a bank name    3,30,462.00
19    carry none of the three                                8,37,549.97
```

The 19 are not missing a cheque -- they are not cheques. A bank transfer has no number to record, so the only thing absent is the clearing date, and for a transfer that is the same day as the payment.

The 12 are cheques, and there the date is the whole point. A cheque clears days after it is written; without `bank_cleared_on` the lag cannot be measured, which is what a reconciliation is for.

It cannot grow

```
reconcile.go
var clearedOn *time.Time
if want { clearedOn = &on }
UPDATE mcj_vouchers SET bank_cleared=$1, bank_cleared_on=$2 ...
```

The date is written in the same statement that sets the flag, so a clearance made through this system always has one. The 31 are closed history and the allowance should never move.

Five allowances, five different answers

470	two events missing a label that carries no information	(372)
8	payments naming bills that do not exist, and a report that counted them as coverage	(371)
9	one of them a diamond ring sold on a bill it is not on	(370)
57	53 documented on the tag, 4 with nothing anywhere	(373)
31	12 cheques with no clearing date, 19 that are not cheques at all	(374)

Not one of the five was a single population, and not one of the numbers said so.

375. Eight of the eighteen duplicate movements are one bill, saved twice a day apart

Sixth and last sizeable allowance. *One voucher moves a tag once* permits 18 events recorded twice under one voucher number. Split by how far apart the two rows sit:

6	within the hour	worst 21 minutes	a save made twice
3	same day	worst 4 hours	
9	days apart	worst 28 days	

The nine are not nine accidents. Eight of them are a single bill:

GST/2627/00150			
2026-07-03 09:25	9 pieces	BAPI	
2026-07-04 14:06	8 pieces	BAPI	the same eight, again

A bill re-saved the next afternoon, by the same person, writing its stock movements a second time instead of replacing them. Eight of the nine pieces moved twice; one did not, which is what makes it a re-save of an edited bill rather than a duplicate submission.

The ninth is 84188080R1 on **SR/2526/00005**, 28 days apart and from the financial year before -- a different event entirely, and the only one of the eighteen that is genuinely on its own.

It cannot happen here

sale.go writes one movement per line inside the voucher's own transaction, and nothing in this system re-posts a saved bill: there is no edit path that reaches the movement writer twice. The 18 are legacy behaviour and the allowance is closed history, like the 31 in defect 374.

Six allowances, and the sweep is done

470	(372)	two events missing a label that carries no information
57	(373)	53 documented on the tag, 4 with nothing anywhere
31	(374)	12 cheques with no clearing date, 19 that are not cheques
18	(375)	one bill saved twice, six double-saves, one from last year
9	(370)	one of them a diamond ring sold on a bill it is not on
8	(371)	payments naming bills that do not exist, counted as coverage

Six numbers, none of which was one population. Every rule now carries its split in the comment above it, so the next reader gets the meaning without running a query -- which is the whole difference between a check that passes and a fact that is known.

376. The last two allowances are a paisa and a pair of malas

Finishing the sweep. Of the nine rules that allow anything, six were opened in defects 370-375 and the coin one (14) was itemised when it was written. Two were left, both of size one.

Every voucher balances -- allowed 1

```
JRNL 312, 16 May 2026
  debits    509.15
  credits   509.14
  out by    0.01
```

One paisa, in 1,848 vouchers. A rounding remainder rather than a missing line: nothing is absent, the two sides simply do not meet at the last decimal. There is no adjustment worth making and none is proposed.

A barcode names one tag -- allowed 1

```
86065435, on TWO tags
  both MALA, both 900.00, both karigar D603,
  both created 2024-03-31, both still on the shelf
```

Scanning that barcode finds two pieces. The sale screen resolves it with

```
ORDER BY s.in_stock DESC LIMIT 1
```

-- a piece still on the shelf, and one of them. **With two rows identical in every visible respect the choice cannot be wrong:** 900.00 leaves the shelf and one of the two malas is marked sold, whichever it is. That is only true because the tie-break prefers a piece in stock; without it the screen could pick a tag already sold and refuse a sale of something the shop is holding.

The sweep, complete

```
470 (372)  a label carrying no information, missing twice
57  (373)  53 documented on the tag, 4 with nothing anywhere
31  (374)  12 cheques with no clearing date, 19 that are not cheques
18  (375)  one bill saved twice, six double-saves, one from last year
14  (324)  itemised when written -- an extra or a missing zero, each one
9   (370)  one of them a diamond ring sold on a bill it is not on
8   (371)  payments naming bills that do not exist, counted as coverage
1   (376)  a paisa
1   (376)  two identical malas under one barcode
```

Nine allowances, 609 rows, all now described rather than counted. Six needed opening, two were a sentence each, and one had been done properly the first time.

377. Fifteen reports are named by no test, and one of them is mine

A coverage family, prompted by defect 269 having once found seventeen of them:

report specs	81	
named by no test	15	
BackDatedSubgroupStock	ConversionStones	DesignCatalogue
ItemCatalogue	PacketStockSummary	Quarterly
RatioAnalysis	RawStockTally	SchemeBook
StockJournal	StockValuation	SubgroupStockTally
SuspectRates	UntaggedItemStock	UserMaster

`ConversionStones` **is the one I built in defect 327**, and it went in without a test while its own entry described reconciling 2,60,68,682.20 three ways. The reconciling was real and it was done by hand, once, in a terminal that is gone.

A render check is not a test, and this is the difference

`check_screens_render.py` fetches all 81 on every run and has always returned green for this one. It proves the query runs and returns rows. **It cannot tell 2,60,68,682.20 from 26,06,868.20.**

Three tests written, and both of the report's real risks proven catchable:

```
the summary equals the lines, and the lines are 3 not 6
  broken by joining shapes on the tenant instead of the id --
x  the report lists 6 stone line(s), want 3

a zero says whose it is
  broken by giving the customer's stone the shop's wording --
x  the customer's stone is marked "no price recorded"
```

The first break is the exact fault the report was written to avoid: four lookups per row, any one of which fanning out doubles the money with no other symptom.

And the first run failed on the seed

The customer-stone tests failed before the report was touched, because the test tenant has no item named CUSTOMER STONE and the join found nothing. **The seed was wrong, not the screen** -- the fourth time this week a measurement failed before the thing it measures, after defect 359's inert break, 365's `sed` and 368's three greps. It is worth noticing that all four were caught by the result looking wrong rather than by any check.

Fourteen reports still carry no test. That is a list, not a claim that they are broken.

378. The valuation screen revalues metal and not stones, and now something says so

Second from defect 377's list of fifteen. `StockValuation` is the screen that answers what the shelf is worth at today's rate rather than the rate on each tag -- which is the question defect 346 raised about the 4,000-a-gram placeholder, and the one that would decide a 3.94 crore re-rating.

It had no test.

The arithmetic it must not get wrong

Only the metal revalues. Stones and making were agreed when the piece was tagged and do not move with the gold rate:

```
SUM(s.total + (c.sale_rate - s.rate) * s.net_weight)
```

Three tags seeded -- two on the shelf, one sold -- 30 g at 6,000 rising to 7,000, with 5,000 of stones on one of them:

tags counted	2	the sold one absent
net weight	30.000	
book value	1,85,000.00	
value today	2,15,000.00	
difference	30,000.00	30 g x 1,000, and nothing else

Broken on purpose by revaluing the whole tag total instead:

```
x value today 215833.33, want 215000.00 -- only the metal revalues
x difference 30833.33, want 30000.00
```

833.33 on a three-tag seed, from 5,000 of stones. The error scales with the stones in the case, and this shop's shelf carries them in quantity.

And the rate today is the LATEST, not the first:

```
x rate today reads "6,000.00", want 7,000.00 -- the 1 August rate
```

Reversing one `ORDER BY` turns a revaluation into a restatement of what the tags already say, and every difference to nought. Nothing else in the screen would look wrong.

Why this one mattered more than most

The 346 question -- on what basis is the shelf valued -- gets answered from this screen. **An untested screen was the evidence for a 3,94,01,031.93 decision.** It is right, and now it is checked.

Thirteen of the fifteen remain.

379. The scheme book is tested, and the seed failed three times before it did

Third from defect 377's list. `SchemeBook` is the screen behind blocker B23 and defect 352 -- 239 enrolments, all still flagged active, 235 matured, and **77,37,300.00 of gold owed on schemes the customers finished.** It had no test.

Three written, and both real risks proven catchable:

```
the paid total is one sum per enrolment
  broken by joining payments on the tenant instead of the enrolment --
x PAID UP shows "17" instalments, want 12
x HALF WAY shows "17" instalments, want 5

a shortfall appears only when there is one
  broken by dropping the IsPositive guard --
x a scheme paid in full shows a shortfall of 0.00
```

The first break is the one that matters. A customer five instalments into twelve, reading as seventeen and paid up, is a debt that leaves the screen -- and on a book of 239 enrolments nobody would notice which.

The seed failed three times, and each failure was a different mistake

- 1 "inconsistent types deduced for parameter \$2"
the instalment date built as ``$2::int * INTERVAL '1 month'``.
Cast it again -- no. Moved the arithmetic into Go -- no.
- 2 the SAME error, from a statement I had not looked at: \$2 appearing
in a SELECT list and again in a WHERE, one of them bare.
THAT is the defect this project already knows (see the note kept in
pkg-contracts.md), and I spent two attempts on the wrong statement
because the error names a parameter and not a query.
- 3 "NEVER PAID is not on the scheme book" -- correct behaviour. The screen's
default view is schemes past maturity, and I had dated the unpaid one in
the future.

Two of the three were the test being wrong about the screen, which is the fifth and sixth time this week (359, 365, 368, 377, and these). The tell is constant: the failure said something surprising about code that had not changed.

Twelve of the fifteen remain.

380. The ratio screen prints +Inf and NaN without its guard, and a group name is nearly a synonym

Fourth from defect 377's list. `RatioAnalysis` is mostly restraint: it refuses gross margin, stock turnover and return on stock because all three need closing stock inside the ledgers and blocker B19 is precisely that it is not there. What it prints is liquidity and gearing, which need only balances.

Seeded a shop with 1,00,000 in the till, a customer owing 50,000, a 30,000 overdraft and 20,000 owed to suppliers:

quick ratio	$(1,00,000 + 50,000) / (30,000 + 20,000)$	3.00
cash to overdraft	$1,00,000 / 30,000$	3.33
receivable to payable	$50,000 / 20,000$	2.50

Three tests, and each risk proven catchable.

A division can be inverted and still look like a ratio:

```
x Cash to overdraft reads "0.30", want 3.33
```

0.30 is a perfectly plausible number for a jeweller. Nothing about it says the division went the wrong way round, which is why the numerator and denominator are shown beside it and now checked.

And the zero guard is load-bearing:

```
x Quick ratio reads "+Inf" with a zero denominator, want a dash
x Receivable to payable reads "NaN"
```

A shop with no overdraft and no supplier debt is not an error; it is a shop having a good month. Without `if b.IsZero()` this screen prints `+Inf` and `NaN` at a reader.

The seed was wrong in a way worth writing down

The first run gave a quick ratio of 5.00 against a hand-worked 3.00, and a receivable-to-payable of "--". The cause was one word:

```
seeded      'Sundry Creditors'
counted     'Sundry Creditors Expenses'
```

The screen names three supplier groups explicitly and the shop's chart uses the longer one. My seed's supplier existed, had a credit balance, and was counted as nothing -- so payable was nought, and the quick ratio quietly divided by the overdraft alone.

A group name that is a near-synonym of a real one fails silently, because nothing joins to it: there is no foreign key to violate and no row to be missing. The same shape as defect 371's coverage figure and 362's wrong key -- something that resolves to nothing rather than to an error.

Eleven of the fifteen remain.

381. The screen that finds tags priced off the rate cannot see the 360 that are

Fifth from defect 377's list, and the one that found something.

`SuspectRates` exists to catch a tag carrying a figure that was never a rate. Its two structural tests are exactly right for the job:

```
a round hundred          what somebody types meaning to come back
a rate equal to the net weight  a transposition
```

And a pre-filter runs before them: `s.rate < c.sale_rate * 0.25`. Only tags under a QUARTER of today's rate reach the structural tests at all.

	today's rate	a quarter	tags at exactly 4,000	reaching the test
18kt	12,398.00	3,099.50	307	0
22kt	15,153.00	3,788.25	39	0
14kt	9,643.00	2,410.75	8	0
95	7,500.00	1,875.00	5	0
9kt	6,199.00	1,549.75	1	0
			360	0

Not one of defect 346's 360 placeholder tags reaches the test that would catch it --
3,94,01,031.93 of understatement, invisible to the screen built to find it, because 4,000 is above a quarter of every rate in the shop. The 22kt case misses by 212 rupees.

The screen currently flags **18 tags, 6,33,930.93.**

The filter is not a mistake, which is what makes this hard

It was set deliberately (defect 36) after a first version that used the fraction AS the finding and was 90% noise: 159 of 177 rows were silver tagged when silver was genuinely a quarter of today's price. The captured rate history starts 1 April 2026, so a tag made before then cannot be checked against the rate of its own day, and being far below today's is evidence of nothing.

Removing the filter entirely and letting the structural tests stand alone:

831 tags flagged, of which				
18kt	400 (2,83,69,578.84)	22kt	187 (3,25,63,979.47)	the signal
999	112 (39,855.45)	SLV	22 (39,544.67)	the noise

134 tags of noise worth about 79,400 against 631 of signal worth 6.28 crore. That looks decisive and it is not mine to decide: it changes what a screen shows, the threshold was chosen on evidence, and a jeweller may have reasons for pure gold and silver that this reading does not see.

What was done instead

Four tests, and one of them pins the exclusion:

```
TestSuspectRates_TheQuarterFilterExcludesSomeRealOnes
```

It seeds a round-hundred tag at 40% of today's rate and asserts it does NOT appear. If somebody changes the fraction the test fails and says why -- so the change becomes a decision instead of a side effect.

Ten of the fifteen remain.

382. The stock journal is tested, including the label it must not print

Sixth from defect 377's list. `StockJournal` is the metal side of a conversion -- 859 documents, 1,242 pieces -- and its companion `ConversionStones` got tests in defect 377 while this one had none.

Four written. Two of the breaks are worth keeping.

The summary must cover the selection, not the page. The list stops at 500 rows and the summary counts separately, which is the guard defect 145 earned after a screen presented a partial page as a total:

```
x the summary counts "2 piece(s)", want 3 piece(s)
x the summary says 0.00 and the lines say 1,14,500.00
```

And the direction flag must stay a flag. The screen shows 0 or 1 exactly as the shop recorded it, because the capture does not settle which means stock in -- defect 323 is still on the decision list asking, and every measurement points one way while the whole case for the other is an absence. Labelling it would answer the question by assertion:

```
x the flags shown are map[in:true out:true], want both 0 and 1
x the flag reads "out" -- defect 323 has not been answered and this screen
  must not answer it by labelling
```

That is a test against a plausible improvement. "in" and "out" read better than "0" and "1", and somebody tidying this screen in a year would have every reason to make that change. The test is there to tell them what it costs.

The other two cover a negative piece being flagged rather than hidden -- the shop's own books carry two, the same silver bangle -- and the direction filter selecting rather than decorating.

Nine of the fifteen remain.

383. The screen that warns against adding gold to silver was adding gold to silver

Seventh from defect 377's list, and the second of the fifteen to turn up a fault rather than just a gap.

`RawStockTally` carries this comment, above the code that classifies each purity:

```
// Gold and silver are kept apart. Adding them gives a number that is
// neither, which is the mistake this record made four times before defect
// 78 was corrected -- so the tally never prints a combined weight.
```

Thirty lines below it, the total row:

```
"net": fmt.Sprintf("%.3f", goldWt+silverWt+otherWt),
```

On the shop's own books that read 27,326.609 -- gold plus silver plus a silver article -- in the Net wt column, one column to the left of a note explaining that such a number is neither.

```
before  10 line(s) | | | 27,326.609 | Gold 12304.524 g, silver 15022.085 g. Kept
apart...
after   10 line(s) | | |           | Gold 12304.524 g, silver 15022.085 g. Kept
apart...
```

The eye goes to the figure in the column, not to the prose beside it. A reader taking one number off this screen was taking the wrong one, and the screen was telling them so in the next cell.

Fixed and pinned

The total row now carries no weight at all. The split stays in the note, where it is true, and anything that is neither gold nor silver is named there too -- conditionally, because a standing "other 0.000 g" would invent a third metal for symmetry.

Three tests, and the break is the change somebody would make to tidy this:

```
x the total row carries a net weight of 350.000 -- gold and silver added
   together is the number defect 78 corrected four times
```

The same shape, a fourth time

```
defect 372  a rule whose NAME promised more than it checked
defect 371  a caveat that sized its own coverage wrongly
defect 358  a comment describing four screens, right about two
defect 383  a comment saying "never prints a combined weight", printing one
```

Prose and code drift, and the prose is what gets believed. Every one of these was found by reading the two against each other rather than by anything failing.

384. Two live comments still said 39 bills have no line detail, four months after the last one was recovered

Defect 383 found a comment contradicting the code beneath it. That is four of these now, all stumbled on, so this time the class was swept: code comments that state a checkable figure, checked.

Most hold. 2,401 of the shop's 2,816 tags came in as `opening stock is exact( created_from = 'stk' )`. 2,816 captured tags carry `R001 is exact`. **One did not, in three places:**

```
invoice.go          "39 of the shop's 224 bills carry no line detail at all,
                    and for those this is the only place the tax exists"
invoice_e2e_test.go the same claim, opening the test
hsn_e2e_test.go     "39 of the shop's 224 are like that in the capture itself"
```

There are none. 0 of 224, and invariant rule 2 has been asserting it on every run. The figure came from blocker B21, which read `sales_vou_main` alone; the loader also reads `sales_vou_detail`, and together they cover all 224. Defect 155 recovered them -- the lines went from 328 to 367.

The docs had already corrected it

```
data-defects.md: "### B21 said 39 bills have no line detail. There are none"
gstr.go:         "That was true when written. Defect 155 recovered those 39"
```

`gstr.go` rewrote its own paragraph to say so and kept the old text as a correction. Two other files did not hear about it, and one of them is a comment justifying a real design decision -- taking tax from the voucher -- on a fact that stopped being true.

The decision is right and did not need the 39. Its comment now says so without them, which makes it stronger: a reason resting on a number that moved is a reason that expires.

What the sweep says about the class

```
383  a comment saying "never prints a combined weight", printing one
384  a comment citing 39 bills, of which there are none
```

Both were true when written. **Neither was wrong in a way any test could catch**, because a comment cannot fail. The only thing that finds these is reading the prose against the data on purpose, which is now done twice and worth doing again.

385. The rest of the comment claims hold, and the two that looked wrong were my arithmetic

Finishing the sweep defect 384 started. Every code comment stating a checkable figure, checked against the shop's data:

2,401 tags came in as opening stock	exact	(created_from = 'stk')
2,816 captured tags carry R001	exact	
2,797 tags name the document they came on	exact	
699 tags are certified	exact	
6,587 NEW-BAR movements, of 9,031	exact	
6,395 of those name a person	exact	
366 order lines	exact	
2,68,52,217 estimated across them	exact	
50,71,370 of old gold bought in	exact	
39 bills with no line detail	WRONG	-- there are none (defect 384)

Ten of eleven hold to the digit. One was wrong and is fixed.

Two of the ten looked wrong first, and were not

certified tags	I counted certificate_no <> ''	678
	the comment says certificate_no <> '' OR	
	certification_charge > 0	699
naming a person	I counted done_by across ALL movements	6,789
	the comment says of the NEW-BAR ones	6,395

Both comments state their own definition in the sentence before the number, and both times I measured something adjacent and got ready to file a defect against prose that was exactly right.

That is the seventh measurement error this week -- after 359's inert break, 365's sed , 368's three greps, 377's seed, 379's three seeds, 380's group name. Every one had the same shape: a number that disagreed, and a check I had not read closely enough to know what it was counting.

The lesson is not "be careful". It is that **the comment told me how to measure it and I measured something else** -- so the fix is mechanical: when prose and data disagree, re-read the prose for its definition before believing the disagreement.

What the class is worth

Two real findings in eleven claims (383, 384), both invisible to every test, both true when written. A comment cannot fail, and there is no guard to build here -- only the habit of reading the two against each other, which has now paid twice and cost seven false starts.

386. The back-dated shelf is tested, and it disagrees with `in_stock` on purpose

Eighth from defect 377's list. `BackDatedSubgroupStock` answers what was on the shelf on a chosen day, which is the question anybody reconciling a physical count a week later actually has. There is no snapshot, so it rolls the shelf back from two facts:

```
created on or before the date
its LAST sale-or-return movement up to that date was not a sale
```

Three tests, seeded with four tags that each take one branch:

KEPT	never sold	held on 30 June
SOLD	sold 10 May	gone by 30 June
RETURNED	sold 10 May, returned 20 May	held on 30 June
LATER	created 1 July	not yet made on 30 June

The return is the case worth having a test for. Read the FIRST movement instead of the last and a piece that came back stays sold:

```
x on 25 May the shelf holds "1", want 2 -- the return brought one back
```

And dropping the created-date clause puts a piece on a shelf it was not made for yet:

```
x on 30 June the shelf holds "3", want 2 -- LATER is not made yet
```

It disagrees with the tag's own flag, and should

Defect 369 found three pieces whose last movement is a return while the tag still reads out of stock. **This screen counts them as held and `in_stock` says gone.** That is not a contradiction to fix: `in_stock` is a fact about now, maintained by whatever last touched the tag, and this screen is a reconstruction from the trail for a date in the past. Where they differ, the trail is the thing with dates in it.

Worth stating because the natural instinct on finding two answers is to make them agree, and here that would mean throwing away the only record that can be asked about June.

Seven of the fifteen remain.

387. Twenty-nine silver articles the shop owns and no screen showed

Ninth from defect 377's list. `UntaggedItemStock` is the loose metal read by item rather than by purity — how much GOLD ORNAMENTS is loose, not how much 18kt. Four tests to start, and they hold: one item held at two purities is ONE row that names both (grouping by purity as well splits it: *shows "1" line(s), want 2 ...*

gross is "40.000", want 100.000), a line whose item was never named gets (item not on file) rather than an empty label (an unnamed item is shown under an empty label), and the summary counts items and lines and no weight, for the same reason the tally's does not.

Then the seed needed the shape of the real table, and the real table had something in it.

Nine rows are weighed. The tenth is counted.

The whole untagged holding is ten rows in barcode_main_rawstone :

id 7	item 916 SILVER ARTICLE	purity ARTIC(PCS)	gw 0.000	nw 0.000
		qty 29	nag 29	

Twenty-nine silver articles, counted in pieces, weighing nothing. qty and nag are kept in parallel by the legacy and both read 29 — the second reading this figure was checked against. mcj_raw_stock had no column for either, the loader took neither, and so on both raw-stock screens that line showed **0.000 g**: a row that looks like something somebody forgot to delete, and is stock the shop owns.

Built: migration mcj-1787417000 adds pieces, the loader carries GREATEST(qty, nag) with a guard on the 29, and both screens grow a Pieces column. **Blank, not nought, on the nine weighed rows** — a column of noughts against nine lines held by weight says "none" where the truth is "not counted that way", and that falsifies cleanly:

x	SILVER ORNAMENT is held by weight and shows 0 piece(s)
x	GOLD ORNAMENTS is held by weight and shows 0 piece(s)

The summary **does** total pieces, and still totals no weight. A piece is a piece whatever metal it is made of; it is the one column on this screen that can be added down.

What this is not

The obvious guess on finding one dropped column is that its neighbours went too. rate, amount, making and total sit right beside qty in that table — and they are **0.00000 on all ten rows**. So defect 78's finding that the untagged holding carries no rate and no value is a fact about the shop's books, not a column this migration forgot. Checked before the column was added, because the two cases want opposite responses and look identical from a distance.

Also corrected: the screen's own comment said "Ten rows describe the whole of it". Ten lines, five rows — it groups them.

Six of the fifteen remain.

388. gofmt rewrites doc comments, and one of them was a SQL predicate

Found while formatting after 387. gofmt -l flagged 13 files, and the diff on one of them was not whitespace:

-	// `certificate_no <> '' OR certification_charge > 0`, and a certified piece is
+	// `certificate_no <> " OR certification_charge > 0`, and a certified piece is

Two single quotes become one closing smart quote. Go 1.19 taught gofmt to reformat doc comments, and part of that is the old troff convention: ```` opens a quotation and `''` closes one. In prose that is a kindness. In an empty string it is a rewrite from SQL into punctuation — true of the typography and false of the code, and the reader who copies it out gets a syntax error.

Reproduced minimally to be sure it was gofmt and not something local:

```
// F decides it with `certificate_no <> '' OR certification_charge > 0`  
→  
// F decides it with `certificate_no <> " OR certification_charge > 0`
```

Exactly **one line** in this repository would have been corrupted, and it is the certified-piece rule in `daystrade_e2e_test.go` — the same predicate defect 385 corrected one turn earlier, which would have been silently un-corrected by anybody's editor on save.

Fixed by indenting the predicate. gofmt leaves an indented block in a doc comment verbatim, so the rule now reads:

```
//      certificate_no <> '' OR certification_charge > 0
```

The explanatory sentence beneath it had to be reworded too: my first draft said what gofmt does to a bare `''` **by writing one**, and gofmt duly ate it. It now says "the two single quotes of the empty string" in words.

The tree is now gofmt-clean, and `tools/check_gofmt.sh` keeps it so — check 11. Not for tidiness: a formatted tree means that rewrite happens under a check rather than unnoticed in an editor. `tag.go` carries the same predicate in an ordinary comment inside a function body, which gofmt does not touch — only doc comments are reformatted.

389. The tally's unweighed count was 218. It is 369, and the extra 151 are a different thing

Tenth from defect 377's list. `SubgroupStockTally` is the sheet somebody carries around the shop with a pen: ordered by where the goods sit, leading with the count and the weight, because those are the two figures a person can check against a tray. Five tests, and the three rules that matter all falsify:

```
x MIXED says "counted by the piece, not weighed", want it to count the one unweighed tag  
x WEIGHED holds "3" tag(s), want 2 -- TAL-WSOLD is sold  
x the ungrouped tag is not on the sheet at all
```

A count of zero and a weight of zero are different facts

The screen's own comment said *"218 imitation pieces carry no weight at all"*. The real figure is **369 of the 2,575 tags**, and `gross_weight = 0` picks out the identical 369 — two columns agreeing, which is the second reading:

IMITATION JEWELLERY	218 of 218	counted by the piece
SILVER ARTICLE (PCS)	128 of 129	counted by the piece
22KT JEWELLERY	16 of 606	GOLD RAKHI
SILVER JEWELLERY	5 of 736	SILVER BANGLE
CUSTOMER GOODS	2 of 2	the customer's own

The 218 was true and was not the whole of it. The extra 151 divide into two kinds, and the difference is the whole reason this screen has two different notes.

The 21 in weighed subgroups are all priced at exactly 2,000

A gold tag with no weight looks like a weighing somebody skipped. These are not that. Sixteen GOLD RAKHI at one piece each and five SILVER BANGLE at two, and **every one of the 21 is 2,000.00 exactly** — min and max both — so they are goods sold at a flat price sitting in a subgroup that is otherwise weighed.

Worth stating plainly because the screen already handles it correctly: a subgroup where NOTHING is weighed is described ("*counted by the piece, not weighed*"), and a subgroup where SOME of it is has them counted ("*16 of these carry no weight*"). Collapsing the two into one message is the break that fails above.

What a zero weight does cost: those 21 cannot be re-priced when the metal rate moves, because there is no weight to multiply. That is correct for a flat-priced rakhi and would be wrong for anything else, so it is worth knowing that the set is exactly 21 and exactly two descriptions.

One dead branch, and why it is not dead code

'(no subgroup on the item)' cannot be reached through an ungrouped item: `mcj_items.group_id` is NOT NULL, so every item has a group. The only route is a tag whose `item_id` is null — **and the shop has eight of those**, all out of stock, so the row never appears today and appears the moment one of them comes back. Seeded and tested at defect 390, which is what those eight turned out to be.

Five of the fifteen remain.

390. Eight barcodes carried forward twice with nothing in them

Found through 389's dead branch. Eight tags have no `item_id` at all:

19684380/SR	30223979/SR	30683902	45174064
55380544	65910625/SR/SR/SR/SR	67141215	69375796

No item, no description, no purity, no weight, no value, `is_opening` true and `in_stock` false. Each carries a `legacy_id` and **exactly two movements**, and the two are the same on all eight:

OP-STK	2024-04-01	0.000	from barcode_wise_all_transaction
OP-STK	2025-04-01	0.000	from retail_all_stock

Two successive financial-year openings, from two different legacy tables, each rolling an empty barcode forward.

Not a migration loss. The legacy row is empty too — checked before writing any of the above, because a dropped column and an empty column look identical from a distance and want opposite responses (defect 387, one entry earlier):

tran_type	Opening	item_id	0	qty	0	nw	0.000000	gw	0.000000
rate	0.00000	purity	' '	ac_yr	2025-2026			status	N

`item_id 0` is why they arrive here as null. So the shop's own opening-stock table carries these eight as barcodes with no payload, and has done for two years running.

`65910625/SR/SR/SR/SR` is the one to look at: the legacy appends `/SR` to a barcode each time the piece comes back, so that one was returned **four times** before ending as an empty shell. Two of the other seven carry a single `/SR`.

What it costs, and what it does not

Nothing on the shelf: all eight are out of stock and worth 0.00, so the ₹14,23,61,188.20 and the 2,575 are untouched. What they do sit inside is the counts — 8 of the 2,816 tags ever created, and **16 of the 9,031 movements** are openings of nothing. Any tally of movements by year picks up sixteen rows that describe no goods.

Left in place, not cleaned. They are what the shop's books say, and a barcode that was returned four times is a history somebody may want to ask about.

391. The packet stock summary counted stones that had left the shop

Eleventh from defect 377's list, and the first of these to be wrong rather than merely untested.

A packet is one envelope of loose diamonds of a grade, drawn on as pieces are made. `PacketStockSummary` says how much of each is in the building. Its title says Stock, its doc comment said *"drawn on by anything on the shelf"*, and its own empty-state line says *"No packet is drawn on by anything in stock."*

The query never mentioned `in_stock`. It joined packets to stones and totalled the lot, so a diamond set into a ring sold last year still counted as stock in the packet it came from.

	stones	tags	stone value
as it read	107	104	13,80,534.60
on the shelf	85	82	11,91,170.00
difference	22	22	1,89,364.60

Every one of the 22 is on a tag that exists and has been sold — no orphans, so this is a filter that was never written rather than data that does not join.

One packet carries almost all of it

DMIX	96 stones drawn	76 held	6,15,580.00 → 4,28,290.00
DRF1	1 stone	0 held	1,720.00 → nothing
IPRD1	1 stone	0 held	354.60 → nothing

DMIX is the shop's working mixed packet and 20 of its stones have gone out set into pieces. The other two are emptied entirely: the screen reported ten packets "in use" when eight hold anything.

Fixed, and the emptied packets keep their rows

The stone columns now count what is on the shelf. **A packet whose whole draw has been sold keeps its row**, saying `all 1 sold`, because "which packets does this shop draw on" and "what is left in them" are both real questions and dropping the row answers the first one wrong. Partly-drawn packets say `20 more sold`, so the gap between 96 and 76 is on the row rather than left for somebody to find.

Five tests, and both halves falsify:

```
x PKT-PART holds "3" stone(s), want 2 -- the third went out on a sold piece
x PKT-PART is worth 80000.00, want 40000.00
x PKT-GONE has vanished from the summary along with its stones
```

The summary now separates the two: totals for the shelf, and a line saying how many stones went out and how many packets have nothing left. The 195 packets nobody has drawn on stay off the screen, which was already right — a packet master is a catalogue of grades the shop buys, and listing them would bury the ten that matter.

Four of the fifteen remain.

392. "The edge of what was captured" was a guess the screen could not make

Twelfth from defect 377's list. `Quarterly` is trading by quarter read from the vouchers, and it already refuses to call its bottom line a profit, because closing stock sits outside the ledgers entirely (blocker B19).

Its per-row note did not have that discipline. Any quarter holding a single voucher was labelled:

```
one voucher only – the edge of what was captured
```

That is true of both of the shop's:

```
2026 Q1      1 voucher      the 1,399.00 journal of 9 March
2026 Q2      1,196
2026 Q3        650
2026 Q4      1 voucher      the post-dated cheque of 5 December
```

Both are first and last. So the sentence has never yet been wrong — and it is not something the row can know. **A single-voucher quarter in the MIDDLE of the range is a quiet quarter**, which is a fact about the shop's trade, and calling it an edge of the capture explains away the very thing a reader should stop at. The same blind spot as defect 381: a claim that happens to hold on today's data and is stated as though it were derived from it.

Now only the first and last rows may say it. The middle says `one voucher only` and leaves the reader to ask why. On today's data the screen reads exactly as before; the change is what it will say the first time a quiet quarter appears.

```
x 2026 Q3 says "one voucher only – the edge of what was captured"
-- it holds one voucher and it is not an edge
```

Two rules that were already right, now held down

A voucher is counted once however many lines it has. The join to lines and ledgers multiplies rows, so a count without DISTINCT counts postings and calls them vouchers. Seeded a four-line voucher to prove it:

```
x 2026 Q2 holds "6" voucher(s), want 2
```

Income and expense take opposite natural sides — income is credit less debit, expense is debit less credit — so both read positive and the gap is one subtraction. Taking expense on the income side does not merely flip a sign:

```
x 2026 Q2 expense is -4000.00, want 4000.00
x 2026 Q2 gap is 14000.00, want 6000.00
```

A 6,000 gap reported as 14,000. The error inflates the result, which is the direction nobody queries.

Also seeded: a voucher touching neither income nor expense, which is the shop's December cheque. It still counts, because counting it is how a reader learns December has anything in it at all.

Three of the fifteen remain.

393. The item catalogue holds, and the one HSN worth watching is a watch

Thirteenth from defect 377's list, and the first in a while with nothing wrong in it. Six tests, three falsifications, no fault.

Its headline claim checks out exactly: **221 of the 322 items have no stock at all**, 101 do, and the stock those 101 carry sums to ₹14,23,61,188.20 — the shelf baseline, arrived at from the item side rather than the tag side, which makes it a second reading of a figure this project has quoted often.

The rules that hold:

- **A dormant row shows nothing, not a nought.** 221 noughts in a column of 322 rows hide the 101 that are not. Printing them: *NEVER shows 0 in the tag column, want it left empty.*
- **A sold tag does not make an item look stocked** — defect 391's trap, one screen over, and this screen has the `in_stock` filter 391 was missing. Removing it: *SOLDOUT shows 1 tag(s) -- its only piece has been sold*, and the "only items with none" filter drops from 2 rows to 1.
- **The stocked filter cuts both ways**, because "show me what is dormant" and "show me what is working" are the two questions this screen is opened for.

A filter that cannot quietly drop off

Breaking the group filter did not produce a wrong answer. It produced:

```
item catalogue: expected 1 arguments, got 2
```

The clause and its parameter are appended together, so removing one without the other is an arity error at the database rather than a full-table listing that looks like a filter finding everything. Worth naming as a property: the failure mode of a hand-built WHERE is usually silence, and here it is not.

Four items carry no HSN, and the legacy says the same four

ALLOYS	stone	gm	a making input
Old Gold	raw	gm	metal taken in exchange
Old Silver	raw	gm	metal taken in exchange
WATCH	metal	pcs	a thing the shop sells

legacy.item_master records exactly 4 blank hsn_no against 318 set, so nothing was dropped in loading.

None of the four has ever been on a bill — 0 of the 367 sale lines carries an item without an HSN — so nothing is wrong today.

Three of them are inputs and would not normally go out on a sale line. WATCH would. If one is ever sold, the invoice has no HSN to print, and the GST return has nothing to group it under.

Not raised as a blocker: one unsold item code is a thing to notice, not a thing to stop for. Named here so that when a watch is finally billed, the reason the line looks wrong is already written down.

Two of the fifteen remain.

394. Three design numbers name two designs each, and a status flag nobody reads

Fourteenth from defect 377's list. DesignCatalogue is what the shop can make: 5,221 designs, their weights, their making charge and their stones. Six tests, and defect 145's rule is the one they exist for.

The cap and the summary, held apart

The list stops at 500 rows and the summary does not. Totalling the 500 dearest and printing it under the column would give a number that answers nothing — neither the page nor the catalogue. Seeded 504 designs and broke it properly:

```
x the summary counts "500 design(s)", want 504
x the summary making is 250000.00, want 252000.00
x the summary stone value is 3000.00, want 3800.00 -- DBNG0002 is off the
  page and its stones are still the shop's
```

That last line is why the rule matters: the design pushed off the page by the cap is a cheap one, so capping the summary loses the stones of the designs **least** likely to be looked at, and the error is invisible from the page.

My first attempt at that break was inert, and it is worth writing down. I added LIMIT 500 to the summary query — which is SELECT count(*), SUM(...) with no GROUP BY, so it returns exactly one row whatever the limit says. The test passed and for a moment that looked like the test failing to bite. The real break wraps the cap in a subquery and aggregates over that. Same shape as defect 359: a falsification that changes nothing proves nothing, and reads exactly like one that proves the code right.

The other two rules bite first time: a design search matches a **family** prefix (DBNG0103, DBNG0104 — a jeweller asks for the family), and every filter reaches the summary as well as the list, which is defect 145 wearing a different hat.

Three design numbers are ambiguous

GBLI0001	1.500 g, making 400.00,	estimate 2,650.00	(legacy 395)
GBLI0001	1.320 g, making 1,050.00,	estimate 4,350.00	(legacy 4347)
GBLI0002	3.250 g, making 585.00,	estimate 5,460.00	(legacy 865)
GBLI0002	1.620 g, making 729.00,	estimate 4,779.00	(legacy 4462)
GBLI0003	1.640 g, making 900.00,	estimate 3,360.00	(legacy 1516)
GBLI0003	1.310 g, making 622.25,	estimate 3,897.25	(legacy 4463)

Not a double load: `design_transaction_main` has exactly **3** duplicated design numbers of its own, and the six rows carry six distinct legacy ids. It is the shop's own catalogue.

A design number is how somebody asks for a design. "Make me a GBLI0001" has two answers, and on the dearest of the three they differ by 64% of the estimate. Three out of 5,221 is small; the reason to write it down is that the catalogue screen searches by that number and will show both without comment.

5,242 rows saying which designs are which, and nothing reads them

`legacy.design_no_status` is one of the 53 tables never opened, and it covers the same design set exactly — every design here has a row there and every code there has a design here, so it is not a stray:

status 0	5,066 rows
status 1	176 rows

The 176 are not a random slice. Their average estimate is **₹49,286.68** against ₹28,618.81 for the rest, and **68% of them carry stones** against 48%. Something selected them.

What the flag means is not in the data — active, discontinued, featured, photographed, sent to a karigar. **Added to the ask list**, because loading a column whose meaning is guessed is worse than leaving it: a design shown as discontinued when the flag meant "featured" is a wrong answer with a confident face.

(The duplicate rows in that table — DWAT0016 appears five times — all carry the same status, so they are repeats rather than disagreements.)

One of the fifteen remains.

395. Three people worked the books who were never in the user master

The last of defect 377's fifteen. `UserMaster` is who could sign in to the old system and what each was trusted with — the question anybody setting up real accounts has to start from.

Four tests. Two bite hard:


```
x POWERFUL is not shown as able to back-date a voucher: "things, things, things, things"
x POWERFUL has "1 screen(s) granted", want 2
x PLAIN has "no screens granted to this login", want 1
x UNGRANTED has "1 screen(s) granted" -- the row keyed '10' is a role, not a login
```

That second break is the bug this screen was born with, reproduced: grants live in `mcj_legacy_access` keyed by **login name** in a column called `role_code`, while the user master's own `role_code` is a number. Joining the two columns that share a name matches almost nothing and shuffles the rest onto the wrong people — note that it does not merely undercount, it hands UNGRANTED a screen it never had.

The shop's 24 logins, as they stand:

```
9 can back-date a voucher      6 can change a rate
4 can edit a barcode           4 can move a due date
11 of 24 can change something after the fact
2,755 screen grants, and every login has some
```

`rps` and `Shibananda` hold **all four**. `rps` is the login defect 354 asks about; this is what the old system says it could do.

Two of the nine back-daters are desks, not people

```
CASH      90 screen grants, can back-date
COUNTER   18 screen grants, can back-date
```

A shared till login that can back-date a voucher means a back-dated voucher cannot be attributed to anybody. Not a fault in this data — it is how the shop ran — but it bounds what any audit of back-dating can conclude.

And three names on the books are in no master at all

```
ADMIN      68 vouchers entered, 1,837 movements
RANAJIT    438 movements
AMIT       39 movements
```

2,314 of the 6,789 movements that name a doer — 34% — name somebody the user master does not have. (The other 2,242 movements name nobody at all.)

Not a loader filter losing them: `load_users.sh` skips rows with a blank name, and `legacy.users` holds exactly **24 rows, none blank**. The master really is 24 people, and these three are not among them. ADMIN reads like a system account the old application knew about without storing; RANAJIT and AMIT read like people.

So the screen answers "who had which powers" for the 24 it has, and cannot answer it for a third of the stock movements. Worth knowing before anybody uses this screen as the basis for real accounts — which is the whole reason it exists.

B15 is held by the schema, not by care

`mcj_legacy_users` has **no password or security-code column at all**. The credentials were never loaded, so no query written later can leak one by accident. There is now a test that fails if they are added back for completeness — which is exactly how a protection like this gets undone.

All fifteen of defect 377's untested reports are now tested. Four of them turned up real faults: 381, 383, 391, and 392.

396. Two of the five tables worth opening turned out to be empty of meaning

With defect 377's fifteen reports done, the next work is the 53 legacy tables no loader has ever read — 21,436 rows. Five of them looked substantive enough to open. Two are now closed, and neither should be loaded.

`item_adjustment` — 922 rows, five of which say anything

Per-item making percentage, making rate per gram, and a gold plus-minus. Exactly the parameters that would explain a price the pricing formula cannot reproduce, which is why it was opened first.

```
922 rows          460 distinct item ids
 5 rows carry a making percentage
 2 carry a rate per gram
 3 carry a plus-minus
```

917 of the 922 rows are entirely zero. And the five that are not do not agree with each other:

1066	GOLD ORNAMENTS	8% making	250.00/g	10 plus-minus
1066	GOLD ORNAMENTS	15% making	—	—
1066	GOLD ORNAMENTS	8% making	250.00/g	10 plus-minus

The same item, twice at 8% and once at 15%. Two more rows (items 657 and 735) name item ids the master does not have — and **150 of the 460 ids in this table are in no item master at all**, with 67 ids appearing more than once.

Nor is it live: across 367 sale lines, `making = 250` appears **once** and `making = 15` **once**. Whatever these values were meant to drive, the billing did not use them.

A table that is 99.5% empty, whose non-empty rows contradict each other, and a third of whose keys point at nothing, cannot be a source of truth about anything. **Examined and deliberately not loaded**, on the same footing as `sales_quotes` and `barcode_detail_rawstone`.

`voucher_hsn` — 756 rows, every HSN zero

An HSN per voucher would be an independent check on the GST returns: what the old system actually stamped, against what this one derives from the item. That was the hope.

```
756 rows over 608 vouchers, 1 April to 5 December 2026
hsn_no: min 0, max 0, non-zero rows 0
v_ttyp: PMNT 592, RCPT 164 — and nothing else
```

Every row is zero, and the only voucher types present are **payments and receipts** — the two kinds that move money rather than goods, and so the two kinds an HSN could never apply to. Not one sale or purchase voucher appears.

So it is a table that was created and then filled by whatever code ran on payments and receipts, recording a zero each time. The shop's real HSN data is where it always was: 21 codes on the item master, which is what the returns already use.

Examined and deliberately not loaded.

What this leaves

Three of the five still stand: `barcode_history` (3,083 rows — per-tag bill and date), `order_rate_master` (2,520 — the rate agreed per order), and `design_no_status` (5,242 — the flag defect 394 put on the ask list).

Worth saying plainly: two of the five most promising unread tables held nothing. That is a real result about how much is left, not a wasted hour — the estimate of remaining work should come down by their row counts, and the 53 unopened tables should be assumed emptier than they look until each is opened.

397. ₹9,26,698 was sold by barcode against tags that never existed

`barcode_history` was the third of defect 396's five tables to open. It is not worth loading — and it pointed straight at something that is.

The table itself adds nothing

3,083 rows, 2,824 barcodes, seven transaction types, 1 April 2024 to 19 August 2026. Every type it has, `mcj_stock_movements` already has more of:

	history	ours
NEW-BAR	2,533	6,587
OP-STK	278	2,048
SALE-GST	242	324
CST-ORD	19	44
SALE-RET	7	17

A partial log of the same events, from a system that already gave us the fuller one. **Examined and deliberately not loaded**, like `item_adjustment` and `voucher_hsn` before it.

What it did do was name ten barcodes we hold no tag for

Ten of its 2,824 barcodes are not in `mcj_stock_items`, and eight of those ten sit on one bill. Following that thread:

```
499 distinct barcodes in mcj_stock_movements have no tag
0 of the 499 are in legacy.barcode_main
```

The shop's own tag master never had them. Not a loader gap — `barcode_main` holds 2,816 rows and all 2,816 were loaded. Every one of the 499 arrives through `barcode_wise_all_transaction`, the movement log, and 488 of them carry a NEW-BAR movement: the old system recorded these barcodes being created

and its tag master does not know them.

Twenty-four of them were sold

24 sale lines across 14 bills	₹9,26,698.00
-------------------------------	--------------

The worst is **GST/2627/00150, 3 July 2026 — ₹7,11,438.00 over nine lines, eight of them tagless:**

66357120	GOLD SET	31.900 g	18kt	4,20,793.00
69879664	DIAMOND LOCKET	2.774 g	18kt	88,922.00
70729576	GOLD MANG TIKA	4.230 g	18kt	57,444.00
53147276	GOLD RING	3.180 g	18kt	43,252.00
19486670	DIAMOND RING	2.368 g	14kt	39,791.00
30244386B	GOLD MANGAL SUTRA	2.700 g	18kt	36,887.00
33455507	SILVER SET	50.730 g	SLV	16,500.00
92255551	DIAMOND NOSEPIN	0.142 g	14kt	7,849.00

The invoices are not damaged. All 24 lines carry a description, a net weight, a purity and an amount — none blank. What is missing is the piece's history: where it came from, what it cost, when it was made. The bill is right and the provenance is absent.

Ruled out before writing this: these are not tags under a slightly different number. None of the 24 matches a tag with a trailing letter stripped (30244386B → 30244386) or as a prefix. They are absent, not renamed.

For scale, the three states a sale line can be in:

232 lines	barcode resolves to a tag
111 lines	no barcode at all — untagged goods sold by weight, expected
24 lines	a barcode, and no tag anywhere

Added to the ask list. The likeliest innocent explanation is goods bought in and sold the same day without ever being tagged — which the shop would know and the data cannot say. The reason to ask rather than assume: if that is what happened, ₹9.27 lakh of purchases should appear somewhere, and if it is not, then eight pieces left the shop on one bill with no record of how they arrived.

Three of defect 396's five tables are now closed. `order_rate_master` and `design_no_status` remain.

398. The rate card is loaded now, reversing defect 59's decision — and the code already knew more than I did

`order_rate_master` was the fourth of defect 396's five tables. It is not a discovery: defects 35 and 59 both read it, and **defect 59 ended with "recorded, not loaded — the rates belong to a decision the shop has not made."**

That decision is reversed here, and the reason is defect 59's own correction. Its headline figure moved from ₹42.85 lakh to **₹60,14,200.75** when the card was opened by hand, because the earlier number had been computed without it. A second reading that has to be re-derived by hand each time it is wanted is not a second reading — it is a thing somebody remembers to do.

Loading the card does not act on the rates. Whether to honour them is still defect 35 and still an ask.

What it holds

2,520 rows over exactly the shop's 284 orders
0 orders unmatched in either direction, 0 rates zero

22	284	18	284	995	284	14	284	95	284
SLV	284	SLV999	280	999	268	9	266	ARTIC(WT)	2

Ten rows per order against the order's one line: the whole day's rate card was frozen, not only the purity that was ordered. `mcj_order_rates` keeps it that way, keyed by order and purity, because the purities nobody ordered are the answer when a line is added later.

`DISTINCT ON` in the loader, lowest id first: the legacy repeats a purity for an order in a few cases, and the rate struck first is the one the customer was told.

The two readings agree 340 times and differ 21

The order line carries a `metal_rate` of its own. Where the two disagree, one is wrong — and the Customer Order Book already computes "costs extra to honour" from the line alone. So the card now appears on the row **only where it differs**, under `Rate card says`. Silent on the 340 that agree, because a column filled on every row is a column nobody reads.

Nineteen of the 21 differ by ₹13 to ₹866 a gram — the rate moving between when the order was struck and when the line was priced.

The near-miss worth writing down

The other two are the largest by far, and I had them wrong:

CO/2627/00041	995	line 1,52,000	card 15,548	ratio exactly 10.000
CO/2526/00402	SLV	line 24,600	card 242.50	ratio about 100

An exact $\times 10$ with no making charge reads as a decimal slip, and that is what I was about to write. **The comment already sitting in `orders.go` had worked it out properly:** CO/2627/00041 is a **COIN-10 GM** order, and 1,52,000 is a rate *per coin* — 15,200 a gram against the day's 15,482, and rate $\times$ pieces is exactly the ₹3,04,000 estimate on the line. Not an error at all. The same comment already records that the card is per gram for both, and that the unit is not consistent even among coin orders, and that the shortfall is therefore deliberately not computed for them.

I found the ratio, read it as a fault, and was saved by prose somebody had written for exactly that reader. Recorded because the lesson is not "check the comments" but that **a ratio is not a diagnosis** — $\times 10$ fits a slipped decimal and a ten-gram coin equally well, and only the unit tells you which.

Four of the five tables are closed. `design_no_status` remains, and it is on the ask list at defect 394.

399. A return you cannot itemise is a return you cannot restock

The returns built at defect 333 are headers. They say **₹20,04,854.00 came back from MANISH KUMAR SHARMA against GST/2627/00004** and not one word about what came back. Four legacy tables held the answer and no loader had read any of them:

sale_return_main	3 rows	the metal line of each sale return
sale_return_detail	4 rows	its stones, keyed by main_id
purchase_return_main	3 rows	the metal line of each purchase return
purchase_return_detail	2 rows	its stones

One line per return in both directions, and **every line total reconciles to its header exactly** — 5,45,749.83 / 1,60,219.39 / 1,35,020.30 against the three debit notes, 20,04,854.00 / 16,505.00 / 41,952.00 against the three credit notes. That reconciliation is the loader's guard, not a remark: a line that does not sum to the note it hangs off has found the wrong note.

Built: `mcj_credit_note_lines`, its loader, and a *What came back* column on both return registers. The shop's six returns now read:

SR/2627/00001	DIAMOND BRACELET	76.264 g 18,	10.630 ct	(70630587 → 70630587R1)
SR/2627/00002	DIAMOND RING	0.928 g 14,	0.060 ct	(36568032 → 36568032R1)
SR/2627/00003	GOLD CHAIN	2.790 g 22		
PR/2627/00001	DIAMOND ORNAMENTS	25.709 g 14,	7.150 ct	
PR/2627/00002	GOLD ORNAMENTS	22.481 g 9		
PR/2627/00003	GOLD ORNAMENTS	10.030 g 22		

The re-tag number is written on the stones

`sale_return_main.new_barcode` is blank on all three sale returns. The number the piece was re-tagged under is on the **stone rows** instead. So the loader takes it from whichever record carries it — the piece was re-tagged once, and which table the clerk's cursor was in when he typed it is not a fact about the piece.

And neither re-tag was ever made

70630587R1	is a tag: no
36568032R1	is a tag: no

Not here, and **not in** `legacy.barcode_main` **either**, so nothing was dropped in loading. Meanwhile the original barcodes 70630587 and 36568032 both read out of stock. A ₹20 lakh diamond bracelet came back on 15 April, the shop wrote down the number it would wear, and no tag was ever created under it.

The convention itself is real and mostly works: `barcode_main` holds **263** R-suffixed barcodes and all 263 were loaded. These two are where it broke. The loader prints them every run, because the day one of them resolves is the day those goods come back on the books.

For scale, of the 499 tagless barcodes defect 397 found, **30 carry an R-style suffix**, and their conventions do not agree with each other:

R1	R2	R4	/SR2	BR1	AR1	and 52013986r4R1
----	----	----	------	-----	-----	------------------

That last one has been returned twice under two different spellings of the same idea.

Two things that cost time, written down

`sql-migrate` parses `--` inside a **string literal** as the start of a comment, so `'... never created anywhere -- defect 399.'` in a `COMMENT ON COLUMN` swallowed the closing quote and the migration failed with *"The last statement must be ended by a semicolon"*. It fails loudly, which is the good kind of trap.

And the same figure has different column names on the two sides: the metal line's total is `amount` on a sale return and `value` on a purchase return, and the stone line's total is `value` on both. Written into the loader rather than remembered.

400. Eighty-three bill numbers mean two different sales, and now the bill says so

Defect 349 read `sale_bill_link` and worked out what defect 66's "second system" is: the column is literally called `test_bill`, and the test series ran ahead of the shop's, reaching **GST/2627/00338 while the book reached GST/2627/00224**. That analysis was correct and it stayed on paper.

The consequence is not documentary. **83 of the shop's 224 bill numbers are ambiguous** — each is a real bill belonging to one customer *and* the number a test print carries for a different customer's sale:

number on paper	our books say	a test print means	whose sale
GST/2627/00007	ANITA ROY	GST/2627/00004	MANISH KUMAR SHARMA
GST/2627/00008	SUMAN CHATURVEDI	GST/2627/00005	ASHIM MAJUMDER
GST/2627/00009	NEETU JAISWAL	GST/2627/00006	SHIREEN AHMED
GST/2627/00013	PRABHA BHANSALI	GST/2627/00008	SUMAN CHATURVEDI
GST/2627/00014	PANKAJ GOYEL	GST/2627/00008	SUMAN CHATURVEDI

Thirty-seven percent of the book. A clerk typing a number off a customer's piece of paper had no way to know which sale he had landed on, and the system said nothing.

Built: `mcj_bill_number_links`, its loader, and a notice on the bill itself:

This number is shared. A copy printed by the old test system and numbered GST/2627/00007 is in fact bill GST/2627/00004, sold to MANISH KUMAR SHARMA. Check which system produced the paper in front of you — see defect 66.

It appears on the invoice sheet, not beside it, because the person who needs it is the one holding the paper. Silent on the other 141.

The loader guards the three figures that matter — 134 links, every mapped bill one we actually hold, and **83 ambiguous numbers** — so a reload that quietly changed the exposure would stop rather than print a different number.

What this does and does not settle

It does not answer defect 66. Whether a test-printed invoice ever left the counter is still a question only the shop can answer, and it is still on the ask list.

What it does is remove the way that question hurts somebody who never reads the ask list. The hazard was one sentence in a defect file; it is now a line on the document.

Loaded as recorded, duplicates and all: `GST/2627/00207` appears twice against the same test number and collapses, because a repeat of the same fact is not a second fact. Three real bills map to two different test numbers each and keep both rows, because that is two facts and one of them is surprising.

401. The shop set reorder levels and nothing has ever shown them

Four more of the unread tables, and one of them pointed at something that was already loaded and invisible.

Three closed, with their numbers

<code>ledger_ref_details</code>	1,071 rows	2 carry a reference name
<code>barcode_create_from_purchase_detail</code>	21 rows	all 21 already have stones here
<code>item_merge</code>	40 rows	1 of the 40 is still an item

`ledger_ref_details` is `item_adjustment` again: a table with a thousand rows and two facts in it. `barcode_create_from_purchase_detail` is stone lines for 21 tags, and every one of those tags already carries the same stones in `mcj_stock_stones` — fully redundant. `item_merge` is a copy of the item master's own shape for 40 items, **39 of which no longer exist in the master**; it explains **none** of the 150 orphan item ids `item_adjustment` pointed at, which was the reason to open it.

All three examined and deliberately not loaded.

The fourth had a finer number than we hold

`item_wise_tagging` is 158 rows over **34 items** — a reorder level per item AND CATEGORY, where the item master holds one per item. An item whose master level is 150 has 33 rows here summing to 456. Two different grains of the same idea, and the finer one belongs to a screen this rebuild does not have.

Not loaded either, but it is what sent me to look at the coarser one.

And the coarser one was already here, read by nothing

`mcj_items.minimum_quantity` is loaded. **31 items carry a level**, set by somebody at the shop who decided how many GOLD RINGS a counter should never fall below. Nothing in the old menu displayed it, and nothing here did either:

GOLD RING	wants 200	has 38
DORI	wants 100	has 0
GOLD TOPS	wants 150	has 57
GOLD MANGAL SUTRA	wants 25	has 0
DIAMOND RING	wants 200	has 129

26 of the 31 are below their level, 7 have nothing at all, and the shelf is 873 pieces short across all of them. That is not a fault in the data. It is the data doing its job the first time anybody asked it.

Built: a `Reorder Levels` report, under REPORTS › Stock Tally Report. Five tests, and the two rules that matter falsify:

x	FULL shows a shortfall of -1 while holding 3 against a level of 2
x	EMPTY is priced at 0.00 off a shelf that holds nothing

A NOUGHT IN "SHORT BY" IS NOT A MEASUREMENT, and the break shows why the guard is `short <= 0` and not `short == 0`: an item stocked ABOVE its level reads as minus one short, which is worse than a nought. A fully stocked row says `stocked` in words. And an item with nothing on the shelf cannot be priced from the shelf, so it carries no money figure at all — a nought there would be a claim that filling it is free.

The money column is the shortfall at the average of the pieces already there, and the row says so. It is an indication and not a quotation: the pieces still to buy are not those pieces.

Two things this cost

A report needs a **navigation entry** as well as a registry entry — registering it in `reports.go` alone gave 404 Not Found from the render check. And a screen with no `LegacyFile` needs `Top`, `Subgroup` and `Item`, because inventing a plausible `.php` name would read as evidence about the old system.

Then it needed an **i18n key**: `MCJMenu.Item.reorder-levels` missing panicked every page, not just this one. The same trap as defect 331, which is why the render check catches it in eleven seconds and a person would not.

402. ₹43,88,746 of bank items the reconciliation cannot see, and looks right without

`brs1` is the old system's bank reconciliation working list. Twenty-six rows, and **every one of them is uncleared** — no clearance date anywhere in the table — and every one stamped **2025-2026**:

24 payments	25,16,608.00
1 receipt	15,28,530.00
1 journal	3,43,608.00

	43,88,746.00

Not one of the 26 resolves to a voucher in this database, and it could not: `voucher_main` holds 1,840 vouchers for 2026-2027 and **none at all for 2025-2026**. The year those items belong to was never captured.

Why it belongs on a screen and not only here

The Bank Reconciliation computes *balance as per bank statement* by adding back the withdrawals it knows are unrepresented and subtracting the deposits not yet credited. That arithmetic is right and it is tested. What it cannot do is add back items it has never seen — so a reconciliation covering a period these span comes out **up to ₹43,88,746.00 away from the real statement and looks complete while doing it**. A screen that is confidently wrong is worse than one that refuses.

Built: `mcj_prior_uncleared`, its loader, and a line beneath the statement:

This statement cannot see everything. 26 item(s) totalling 43,88,746.00 stood uncleared when 2025-2026 closed, and the vouchers behind them are not in this capture, so they cannot be ticked off here and are not in the figures above. Which bank account each belongs to was not recorded.

Beneath the table, not inside it. A line in the statement reads as a figure that has been accounted for, and this is the opposite of that.

The figure is read from the table on every render rather than written into the sentence, so it cannot drift away from the data it describes.

Which bank is not recorded

`brs1` has no ledger column. So the caveat shows against **every** bank account, including the two small ones, rather than being attributed to UNION BANK OF INDIA on the grounds that it is the big one. That guess would probably be right and it would still be a guess printed as a fact.

The loader carries only uncleared rows — there are none of the other kind today, and the filter is there so a fuller dump does not quietly bring settled items in. It also warns, without failing, if one of the 26 ever resolves to a voucher here: that would mean the missing year had arrived, and this table would no longer be the right answer.

Three more tables closed while here

<code>currency_master</code>	108 rows	countries and currency codes
<code>dept_user</code>	103 rows	which users are in which departments
<code>ledger_wise_document_upload</code>	104 rows	scanned KYC images, paths only

The first is a reference list of the world's currencies in a shop that trades in rupees. The third is 104 file paths under `../ledger_doc/` and the files were not captured — the same shape as `barcode_image` and blocker B24. `dept_user` is real and small, and belongs with defect 395's user picture rather than on its own.

403. Twelve hundred contacts attached to customers who do not exist, and a default bank ledger that is a customer

A sweep of what is left. Nothing loaded this time, and two of the findings are worth as much as a table would have been.

`address_book_sub_name` — **1,247 names, 107 of them attached to anybody**

The second name on an account: the spouse who collects the order, the son who pays for it. 1,247 rows across 1,039 ledger codes, with 399 mobile numbers and 84 email addresses on them.

rows	1,247
rows whose ledger code is a customer we hold	107
codes present in <code>legacy.address_book_master</code>	90 of 1,039

Not a loader gap. `address_book_master` is clean — 40,519 rows, 40,519 distinct codes, not one blank name — and our copy of it matches row for row. The 107 that resolve here are the same 107 that resolve in the legacy. The other 1,140 rows name a customer number **the shop's own address book does not have**.

Examined and deliberately not loaded, for two reasons and the second matters more. A contact list where 91% of the entries point at nobody is not a contact list. And loading 1,140 personal names, 399 mobile numbers and 84 email addresses that attach to **no account at all** would be adding personal data with no owner and no purpose to a database blocker **B18** already says is holding more of it than it should.

The 107 that do resolve are real and readable — KETAN CHOPRA against NIRAJ KAYATHWAL's account, KALA TULSYAN against ASHOK TULSYAN's. If a second name on an account is ever wanted, 107 of them are there to load and the rest are not.

`default_settings` — three defaults confirmed, two pointing at nothing

One row, holding the ledgers the old system posted to by default:

sale	1016	SALE	Sales Account	resolves
purchase	1011	PURCHASE	Purchase Account	resolves
cash	1422	CASH	Cash in Hand	resolves
bank	2636	—	—	NOT A LEDGER
stock transfer	1438	—	—	NOT A LEDGER

The first three are worth something on their own: this system hard-codes `ledgerSale = "1016"`, and here is the shop's own configuration file saying the same number. A constant that agrees with a second source is a different kind of constant.

The other two resolve to nothing — not here, and not in `legacy.ledger_master` either. 2636 does exist in the address book, where it is **ASHA PAREEK**, a customer. So the old system's configured default bank account named a person, and its default stock-transfer account named nothing at all.

Nothing depends on them here — this rebuild never read that file — but it is worth knowing that two of the five defaults the old system was configured with were already broken when the dump was taken.

`retail_wise_tax` — confirmatory, and that is a result

```
SALE, SALE-0, EST, SALE-APPR-0:  SGST 1.50  CGST 1.50  IGST 3.00
```

Exactly the 3% the sale screen defaults to, split the way it splits it. The screen already derives its rates from the bills themselves (3% on 202 of 224, 1.5% on 19, 0.25% on 2), so this adds nothing — which is the useful answer. A configuration that agrees with the transactions is one less thing to wonder about.

The rest, closed

barcode_image	3,965	barcode and path; the files are gone (B24)
user_online_status	1,190	session log, page-in-progress, lock status
user_wise_order_report_setup	401	per-user report column preferences
exception_sub_menu / sub_menu /		
main_menu / ip_table	263	the old menu's own structure
item_wise_wastage	1	one wastage percentage
br_rt_tag / working_calc /		
payment_amount / sales_customer* / barcode_category_tagging		

None is business data this system needs. `dealer_wise_making` (7 rows: making rate bands per supplier and purity) is the one with a real idea in it, and seven rows is not a pricing policy.

Where the unopened list actually stands

```
47 tables, 18,746 rows never opened
```

Of those, **13 have now been examined and declined** across defects 396, 397, 401, 402 and this one; **7 are the credential tables blocker B15 keeps shut**; one is `design_no_status`, which is on the ask list at defect 394. The genuine remainder is `barcode_image` and a tail of one-and-two-row tables.

Which is worth saying plainly after eight entries of opening things: **the unopened tables are close to exhausted, and most of what they held was empty.**

404. The biller was the last person in the shop who could see what a customer owed

Back to the goal itself: a real day's trade, end to end. Five tests already cover rate → tag → sell → settle, a piece returned and resold, the year's numbering, a certified piece, and a sale reaching the GST return. Walking the same path looking for what is missing turned up something at the counter.

Choosing a customer on the sale screen showed their name and nothing else.

	accounts	with a balance	owe the shop	in credit
Customer	485	28	73,96,553.00	21,41,852.00
Dealer	76	56	8,80,38,622.00	6,15,02,878.56

Ten customers owe the shop ₹73,96,553.00 between them and eighteen more have ₹21,41,852.00 sitting on account. A biller settling a bill at the counter could not see either. The figure is in the ledger, three screens away, and nobody walking a customer to the till goes and looks.

Built: the chosen customer's standing, beside the tax basis, where the biller is already reading facts about this customer rather than about this bill:

HARSIT ANAND SULTANIA owes 32,30,681.00 on account before this bill.
DIAMOND MELA JEWELS LTD is in credit by 92,66,009.46 before this bill.

Silent on a square account, which is 477 of the 561. A line that appears on every bill is a line nobody reads, and the break shows exactly what the rule prevents:

```
x ledger 7003 reads "SQUARE is in credit by 0.00 before this bill.", want ""
```

Dr and Cr as words rather than a sign, the way the registers do it — a customer in credit is ordinary here and a minus sign reads as a mistake.

What sent me there, and what it was not

The path I was actually checking was **order** → **advance** → **bill**, because a jeweller takes money up front. `mcj_customer_orders.advance` exists and is **zero on all 284 orders** — and zero in `legacy.customer_order` too, so the loader dropped nothing and the shop never used that column.

It does take advances. **69 vouchers carry "advance" in the narration:**

PMNT	32	60,91,163.00	mostly against purchases, to suppliers
RCPT	48	20,65,154.00	received from parties
JRNL	58	6,01,476.00	

They are posted to the parties' ledgers properly, which is why the accounting is right and the order screen looks empty. **The advance is a ledger fact, not an order fact** — which is how a jeweller's books actually work, and exactly why the balance belongs on the bill rather than a link between an order and a receipt.

So the gap was not the missing link I went looking for. It was that the number which already existed had nowhere to be seen.

405. A tag made here was the only kind whose certification could not be read back

`check_unwritten_columns.py` lists fields the shop filled that nothing in this system writes. Three sit on `mcj_stock_items` and are large:

<code>certification_type</code>	2,801 of 2,816 filled
<code>lot_no</code>	2,059
<code>making_sales</code>	1,825

`making_sales` **is a duplicate, and that is the whole answer**

It looked like the missing half of the pricing puzzle — a second making rate, stock against sales, which would explain prices the formula cannot reproduce.

It is not. Of the 1,825 tags carrying both, `making_sales` **equals** making **on all 1,825**. Not one differs. Closed.

`certification_type` **is not which certificate — it is how the charge was reached**

Two values, and both were measured rather than guessed:

<code>by_val</code>	2,683 tags	253 with a charge
<code>per_ct</code>	118 tags	118 with a charge

`by_val` **means the rate IS the charge**. `certification_rate` equals `certification_charge` on all 253. My first three formulas assumed a percentage — rate against the tag total, against the stone value, against the goods amount — and all three matched nothing, which is what sent me to look at the simplest possibility last.

`per_ct` **is a rate a carat**, and `rate × the tag's stone carats` reproduces the charge on **111 of 118**.

The seven that do not are all at ₹900 a carat and all charged for **fewer carats than the tag holds**:

23664936R1	$900.00 \times 0.710 = 639.00$	charged 270.00
70630587	$900.00 \times 10.630 = 9567.00$	charged 9252.90
89558954	$900.00 \times 1.670 = 1503.00$	charged 378.00

Consistent with only some of the stones being certified — the solitaire and not the melee — which the tag does not record. Stated as consistent-with, not as the cause: three of the seven carry an R-suffix and were re-tagged after a return (defect 399), and I cannot separate the two explanations from this data.

Built

The tag screen takes one figure, a charge — which is exactly what `by_val` means. It now writes `certification_type = 'by_val'` and `certification_rate` equal to the charge, the way the shop's own 253 `by_val` tags are written, and **says nothing at all when there is no charge**:

```
x an uncertified piece claims certification mode "by_val"
x the rate is 0 and the charge 900; on a by_val tag they are the same figure
```

Both falsify. A tag made here can now be read back the way every other tag can.

`per_ct` is deliberately not offered: the screen has no rate-a-carat field, and adding one would mean deciding which stones are certified, which is the question the seven exceptions above are about.

And the cast, again

`$22` is now written once and read twice more in the same statement. All three occurrences are cast. A placeholder used more than once with only some casts fails with *"inconsistent types deduced for parameter \$22"* — the same trap as defects 379 and 389, and the comment in the INSERT now says so where the next person will be standing.

(`lot_no` is next, and it is not a lot number: 59 distinct values, all of them category names — SILVER JEWELLERY 557, NORMAL 413, D GENERAL 386, UTENSILS 159. A grouping label, on 2,059 of 2,816 tags.)

406. "D GENERAL" is spelled six ways, and ₹1.97 crore sits under it

The last of the three large unwritten columns from defect 405. `lot_no` is not a lot number: **2,059 of the shop's 2,816 tags carry one**, and the 58 distinct values are a merchandise classification —

by kind	SILVER JEWELLERY 557	UTENSILS 159	SILVER MURTI 20	RAKHI 18
by making	CASTING 57	MACHINE 15	ANTIQUE 5	
by source	KOLKATA EXCLUSIVE 103	RAJKOT 22	BOMBAY 11	
by grade	D GENERAL 386	DIA GENERAL 23	DIA IMPORTED 7	

Nothing in this system writes it, and nothing reads it. That is a gap worth closing — but not yet, and the reason is the finding.

Six spellings of one label

Squashing every value to its letters alone, three labels turn out to be written more than one way:

DGENERAL	6 spellings	D GENERAL (386), DGENERAL (4), D GEN ERAL (1), D G E NERAL (1), D GENE RAL (1), d GENERAL (1)
MACHINE	2 spellings	MACHINE (15), Machine (1)
RAJKOT	2 spellings	RAJKOT (22), Rajkot (1)

394 tags in the D GENERAL family, worth ₹1,97,05,407.34 on the shelf, and anybody grouping by this field gets six of them. Three of the six carry a space dropped into the middle of the word, which is a keyboard slip nobody would defend; `DGENERAL` and `d GENERAL` are the same word differently punctuated.

That much is arithmetic. What is not arithmetic:

SILVER JEWELLERY (557)	and	SILVER JEWELERY (5)	one L apart
UTENSILS (159)	and	UTENSIL (4)	singular and plural
KOLKATA EXCLUSIVE (103)	and	KOLKATA (8)	narrower or the same?
D GENERAL (386)	and	DIA GENERAL (23)	same grade or two?

The first two look like slips and the last two might be real distinctions. **No normalisation can tell them apart**, and a screen that guessed would merge two grades of diamond or split one.

Why nothing was built

The obvious fix is to offer `lot_no` on the tag screen as a list of the values already in use, so a new tag joins a category instead of inventing a seventh spelling. That is right, and doing it now would put six D GENERALS in a dropdown and make them permanent.

Added to the ask list: which of these are the same label. With that answer the list is a dropdown and the shelf can be grouped by it; without it, a tidy screen would preserve the mess in a nicer typeface.

The three large unwritten columns are now all accounted for: `making_sales` a duplicate (405), `certification_type` built (405), and `lot_no` waiting on one answer.

407. I rebuilt defect 366's trap twice in one session

The full suite passes — eleven checks, the whole Go suite included. So the question was what the checks cannot see, and the answer was in the loaders I had just written.

`mcj_credit_note_lines` (defect 399) and `mcj_order_rates` (defect 398) both hang off their parents with `ON DELETE CASCADE`. Both parents' loaders `DELETE` and `reload`. **So running `load_credit_notes.sh` on its own emptied every return's line detail, and running `load_orders.sh` on its own emptied the whole rate card** — silently, and with the loader printing success.

This is exactly defect 366, which is exactly defect 294:

```
294  load_invoices, load_conversions, load_orders had this and were fixed
366  load_purchase_invoices did not, and cascaded 43 bills' contents away
407  two tables I added this session, in loaders whose guards predate them
```

The guard is never wrong when it is written. It goes missing when a **child arrives after its parent's loader** — nobody edits a working script to protect a table that does not exist yet. Three times now, and the mechanism has been the same all three.

Fixed the way 294 and 366 fixed it: put the rows back, do not report them. A note on `stderr` is lost by any caller that filters output, and the shop is left with an empty register either way.

Proved rather than reasoned. Running each parent loader alone now prints:

```
-- mcj_credit_note_lines was cascaded away; reloading it --
OK    mcj_credit_note_lines          6 rows

-- mcj_order_rates was cascaded away; reloading it --
OK    mcj_order_rates                2520 rows
```

Both lines appeared on the first run after the guard went in, which is the proof the hazard was real and not hypothetical: the rows had genuinely been cascaded away by the parent's DELETE moments earlier.

The order in load_all.sh was already right

Checked before running anything: all four new loaders sit after their dependencies — bill links after invoices, return lines after return documents, rate cards after orders, prior uncleared after nothing. A full reload would have repaired itself and hidden the bug. **The hazard only shows when a loader is run on its own**, which is what somebody debugging one table does.

`mcj_bill_number_links` (defect 400) is not exposed: it stores the bill number as text and holds no foreign key, so nothing cascades it. That was luck rather than judgement, and it is worth writing down as such — the same table with an `invoice_id` would have been the third.

408. The fourth time is a check, not a fix

Defect 407 was the third appearance of one mechanism. Defect 294 fixed three loaders, 366 found a fourth, 407 found two more — and all three times the cause was identical: **a child table arrives after its parent's loader, and nobody edits a working script to protect a table that does not exist yet.**

A mechanism that has recurred three times will recur a fourth. So it is now `tools/check_cascade_guards.py`, **check 11**.

What it asks

It does not read a list anybody maintains. It asks the database which children exist *today*:

```
every FK with ON DELETE CASCADE whose parent is an mcj_ table
→ which loader runs a bulk DELETE on that parent
→ does that loader name the child anywhere at all
```

Thirteen pairs qualify today, and all thirteen pass. A parent nothing deletes is not a finding — nothing cascades it.

The guard test is deliberately loose: the child's **name appearing anywhere in the parent's loader**. A loader that mentions the child has had somebody think about it; one that never names it has not. A stricter test would encode what a guard looks like, and defect 368 is three greps that each encoded "a check looks like the checks I write" and proved nothing.

It catches the thing it was written for

Removing the guard I had added to `load_orders.sh` an hour earlier:


```
FAIL: a loader deletes a parent and never names the child it cascades:
load_orders.sh: deletes mcj_customer_orders, and mcj_order_rates hangs
off it with ON DELETE CASCADE
Put the rows back rather than reporting them -- see defects 294, 366 and 407.
```

Which is the whole point: the next table to hang off a loaded parent will be caught the first time anybody runs the checks, rather than the first time somebody re-runs one loader on its own and wonders where a register went.

What it cannot do

It cannot tell a guard that works from a mention in a comment. A loader that merely says the child's name in prose passes. That is the cost of the loose test, and it is the right trade: this check exists to catch the case where nobody thought about the child at all, which is what happened all three times.

409. Two checks not written, and the measurements that said not to

Defect 408 turned a mechanism that had recurred three times into a check. The obvious next move was to do the same for the other repeat offenders. Two candidates, and **both were declined on measurement** — which is the point of measuring first.

The repeated-placeholder cast

"A placeholder used twice needs `::type` on every occurrence, not just one" has bitten at defects 379, 389, and again in the tag INSERT this session. Three times, which is the 408 threshold.

```
SQL literals in non-test Go with a placeholder used more than once    59
of those, at least one occurrence uncast                               47
```

Forty-seven of fifty-nine, and the code works. So "cast every repetition" is not this codebase's rule and never has been; a check enforcing it would flag forty-seven correct sites on its first run. The rule that actually holds is narrower — cast when the same placeholder is deduced in **different** type contexts, one column `text` and another `int` — and that is not decidable from the SQL text without knowing the column types it lands in.

Left as a comment where it bites, which is where it already is.

And that measurement was wrong the first time. My regex asked for `$1` and matched the first two characters of `$12`, giving 71 and 59 instead of 59 and 47. Caught by looking at it again before writing it down, which is the only reason the smaller pair is in this entry.

Every loader verifies its own output

Defect 368 established this at 38 of 38 and nothing keeps it true. There are 43 loaders now, four of them mine from today.

The first measurement said **41 of 43**, with `load_masters.sh` verifying nothing. That would have been a real finding.

It is wrong. `load_masters.sh` verifies **seventeen** tables through a `check()` helper that compares a count against an expected figure and exits 1 when they differ. My measurement split each file at `COMMIT;` and looked only at what came after — and the helper is **defined before the load**, so I threw it away and then reported its absence.

All 43 verify. No check is needed for a rule that is not decaying, and the two minutes it took to open the file were the difference between that sentence and a false accusation against a loader that is more careful than most.

Three in one session

```
394  a falsification that changed nothing, and looked like proof
398  a ratio read as a diagnosis when it was a per-coin quote
409  a regex that matched $1 inside $12, and a grep that discarded a helper
```

Different mistakes, one shape: **the measurement was the broken thing, and a broken measurement reads exactly like a finding**. Written down together because the pattern is easier to see in a list than one entry at a time.

410. Twenty bills charged IGST on a supply their own record places in the state

`check_empty_columns.py` lists columns that exist and are never filled. `mcj_sales_invoices.interstate` is one: **null on all 224 bills**. Defect 302 already knew that and handles it — the GST return derives the classification from the place of supply against the shop's own state, which is the GST rule.

What nobody had checked is whether the derivation agrees with the tax the bill actually charged.

	bills
agree, inter-state	18
agree, within the state	186
derived inter-state, no IGST	0
CHARGED IGST, PLACE OF SUPPLY IN THE STATE	20

Twenty. ₹1,56,108.86 of IGST on ₹52,04,145.00 of goods — 64% of all the IGST the shop charged in the year.

GST/2627/00027	LK THAKUR	party Delhi	place of supply West Bengal
GST/2627/00057	PRIYANKA AGARWAL	party Assam	place of supply West Bengal
GST/2627/00067	JANKI KUMAR	party Haryana	place of supply West Bengal
GST/2627/00034	SHISH JEWELS LTD	party Gujarat	place of supply West Bengal

The pattern is plain: the tax follows the **customer's state** and the place of supply says **West Bengal**, the shop's own. Three of the twenty name no party state at all and charged IGST anyway.

Not a loader fault, checked first: `legacy.sales_vou.place_of_supply` reads West Bengal on 206 bills and Bihar on 18, and the 18 are exactly the 18 that agree. The loader carried it faithfully.

What was built, and what deliberately was not

A counter sale to a Delhi customer who carries the goods away is a West Bengal supply and takes CGST and SGST; the same goods shipped to Delhi take IGST. Which happened is not in this data, and **the shop is the only place that knows**.

So the return still files what the place of supply says — that is the rule, and guessing the other way would be inventing inter-state supplies. But it no longer does so silently:

```
the bill charged IGST; its place of supply is in the state
```

Marked, not corrected. A return that quietly overrides the invoice is a return nobody can reconcile against the bills, and this is 64% of the year's IGST.

```
x the row says "", want it to say the bill charged IGST
```

Added to the ask list

Because the two answers differ by ₹1,56,108.86 of tax and by which return those supplies belong in — and because if the IGST was charged in error, twenty customers were charged the right total under the wrong heads.

The measurement that was wrong on the way

My first attempt at the money joined invoices to voucher lines and read **80 bills and ₹1,56,108.86 of IGST against a ledger total of ₹53,244.71**. The join fanned out. Caught by comparing against the ledger figure — the corrected reading is 20 bills, and the ₹53,244.71 is the year's IGST *net of* ₹1,89,406.41 of journals and a ₹495.15 return, which reconciles exactly to the ₹2,43,146.27 charged on sales.

Fourth measurement error this session, and the first one caught by a total I already had rather than by rereading the query.

411. Four hundred and fifty mobile numbers and a hundred and ten GSTINs were being read from the wrong columns

Still working `check_empty_columns.py`. `mcj_ledgers.mobile`, `.address` and `.gstn` are empty on all 785 ledgers, and the first two are selected by the loader — which is the shape of a column read from the wrong place rather than one the shop never filled.

```
legacy.ledger_master, 785 rows
mobile      0 filled      <- what the loader reads
mobile1     450 filled     <- where the numbers are
address      0 filled     <- genuinely empty; not a fault
gst_no      110 filled     <- the loader never asked
```

Four hundred and fifty mobile numbers, lost to a column of the same name sitting empty beside the one with the data in it. `mobile2` and `mobile3` exist too and are left alone: one number is what the ledger has room for.

`address` really is empty in the source, so that one is the shop's and not ours — worth stating, because the three looked identical from the report.

The GSTIN is the one that stops a return being filed

GSTR-1's B2B section is **keyed on the buyer's GSTIN**. Without it a B2B invoice cannot be filed at all. The shop has 20 B2B bills, and:

ledgers behind those 20 bills	3
of those, carrying a GSTIN in the legacy	3
carrying one in this system, before today	0

All three had one all along. Every B2B row on that return was unfileable, and nothing said so, because a blank column reads as a customer who has no GSTIN rather than as a question never asked.

Built: the loader now takes `mobile1` and `gst_no`, and the outward return grows a **Buyer's GSTIN** column. 450 mobiles and 110 GSTINs are in, and all 20 B2B bills carry their buyer's — three distinct numbers across them.

```
x GST/2627/00001 shows GSTIN "", want the buyer's
```

The falsification is the interesting one: pointing the lookup at `mcj_parties.gstin` instead — the obvious other place, and empty on all 40,519 rows — puts the column back to blank and every test but this one still passes.

A fixture that was less real than the data

The test seed for the outward return never set `ledger_id` on its invoices, so nothing read off the party could be exercised at all. **Every one of the shop's 224 bills has one.** Fixed in the fixture rather than worked around in the test: a seed that cannot reach the party is a seed that will hide the next defect of this kind too.

The loader could not be run to prove it

`load_ledgers.sh` cannot run on its own — the vouchers reference the ledgers it deletes, and its own header says to run `load_all.sh`, which empties the referencing tables first. So the change was verified by running its corrected SELECT against the legacy (785 rows, 450 mobiles, 110 GSTINs) and then bringing the database into line with a targeted UPDATE on the same two columns.

Stated plainly because "the loader was changed and the numbers appeared" would imply the loader had run, and it has not. It will on the next full reload.

412. The empty-columns list, walked to the end

`check_empty_columns.py` names 49 columns that exist and are filled in no row. Defects 410 and 411 came out of the first two. The rest were walked to the end so nobody has to walk them again, and **every one is faithful to the source.**

is_cancelled	224 bills	legacy cancel_status = 0 on all 224
is_returned	3 bills	legacy ret_status = 1 on exactly 3 – and this system marks exactly those 3
discount	224 bills and 367 lines	legacy discount = 0 on all 328 lines
making_rate	5,221 designs	legacy making_rate = 0 on all 5,221
metal_rate	5,221 designs	legacy rate = 0 on all 5,221
coin_rate	2,820 rates	a column the shop never used – defect 271 and data defect 54 already say so
address	785 ledgers	legacy address = 0 on all 785

The shop cancelled no bill in the captured year, gave no recorded discount on any of 328 lines, and never wrote down the rate a design's making was computed at. Those are facts about the books, not gaps in the migration, and each was checked against the source table before being called one.

The one that is a fact worth having

4,288 designs carry a making amount and 5,022 an estimate, and not one carries the rate either was computed at. The same shape as defect 405's `by_val` certification: a figure with no derivation. Nobody can ask what a design's making would be at today's rates, because the rate it was struck at was never stored.

Not raised as a defect — the column is empty in the shop's own books and there is nothing to recover. Recorded because it bounds what the design catalogue can ever answer, and somebody will eventually ask it to reprice.

What the sweep is worth

Two real defects out of 49 columns, and 47 confirmations. The confirmations are the point:

`check_empty_columns.py` is a list of *suspensions*, and until each one is opened it is 49 open questions. It is now two fixed and 47 answered.

The tool's own header says it over-matches — a tick under `read` means the column name appears in non-test Go, which catches columns of the same name on other tables. That warning earned its place here:

`mcj_sales_targets.pieces` is listed as read by fifteen files, none of which reads that table.

413. A check that says "no screen" and never looks at screens

`check_unwritten_tables.py` ends with a section headed *"read by no screen and no report"*. It listed three tables. **Two of them were read by screens** — by the invoice (defect 400) and by the bank reconciliation (defect 402), wired in this same session.

The wording was right and the search was narrower than the wording:

```
read = mentions(table, REPORTS, False)
```

`REPORTS` is one directory, `presentation/reports`. A table read by a **controller** — which is what a screen is — was invisible to it.

That matters more than a wrong line of output. This tool's own header warns that *"a tool that overstates gets believed once and then discounted"*, and it had started overstating in exactly the way it warns about. Two thirds of that section was wrong.

Fixed: readers are now reports *and* controllers, and the summary line says "read by a screen or report" because now it means it. The count of tables read but never written by the app moves 21 → 23: my two joined it, correctly, being read by a screen and written only by a loader.

And then one true positive was left

```
mcj_financial_years    0 rows, no loader, no reader, no writer, no FK in
```

Created by migration `mcj-1787321060` to make *"is this date in the open year?"* a comparison rather than string surgery. That was the right idea and it was answered a different way: the financial year is a **value object**, `domain/value_objects/finyear`, with its own tests. The table was superseded before anything wrote to it.

Dropped, with a down migration that puts it back exactly as it was — the table, both indexes including the partial one that allows a single open year, and the range constraint. The Down SQL was executed inside a transaction and rolled back, to prove it is valid rather than to assume it.

Nothing is lost: it has never held a row.

The reason to drop rather than leave it is the same reason the tool's wording mattered. A check that names one dead table every run teaches the reader to skim past the line — and the next thing to appear in that section will be real.

414. A bill raised here records a different place of supply from every bill on the books

Defect 410 asked the shop which of two readings is right for twenty bills. While that sits with them, the sale screen has been answering it in code.

```
the shop's 224 bills    place_of_supply = West Bengal (206) or Bihar (18)
                        - the state the supply happened in
a bill raised here      place_of_supply = the party's state
                        - Delhi, for a Delhi customer at the counter
```

One line does it: `VALUES ($1,$2,$3,$4,$5,$6,$7,$7,...)` — `$7` is the party's state and it fills both `party_state` and `place_of_supply`.

Defect 302's comment states the fact plainly and treats it as background to the `interstate` fix rather than as a question. Read beside defect 410 it is the question: **the two conventions cannot both be right**, and which one is depends on whether those sales were carried away from the counter or shipped — which is exactly what the shop has been asked.

Left as it is, deliberately. Changing it now would answer 410 by writing code. And the direction is not obvious: if those sales were shipped, this screen is right and the shop's own records are wrong; if they were carried away, this screen is wrong and a bill raised today would disagree with the 224 already on the books

rather than match them.

Marked in the code where the next reader will be standing, pointing at 410.

Nothing else changed. The `interstate` flag is recorded explicitly (defect 302), so a bill raised here classifies its own tax correctly whatever the place of supply says; what is at stake is the place-of-supply field itself, which is what GSTR-1 files and what prints on the customer's copy.

Two more that were checked and are not defects

`mcj_customer_order_lines.making` is filled on 122 of 366 lines and the order screen writes only `making_amount . making` is a rate (4,690.000, 4,020.000 — three decimals, repeated across lines) and `making_amount` is money (25,425.97); all 122 that have one have both and they are never equal. **I could not derive one from the other** — not per gram of net weight, not on any of the six rows I tried — so the unit is unknown and no field was added. Guessing it would be defect 398's mistake with a form control attached.

`mcj_sales_invoices.party_mobile` is on 196 of 224 bills and the sale screen does not capture it. Now that ledgers carry 450 mobiles (defect 411) the obvious fix is to take it from the chosen customer — except **not one of the 28 bills without a mobile has a ledger that has one**. The gap is real for new bills and there is nothing to backfill.

And a third quoting trap in one session

<code>sql-migrate</code>	<code>--</code> inside a string literal ends the statement	(399)
<code>bash</code>	backticks inside an unquoted heredoc execute	(399)
<code>Go</code>	a backtick inside a raw string literal ends it	(414)

Writing `legacy.sales_vou.place_of_supply` in backticks inside a SQL comment inside a Go raw string closed the string and broke the build. Three containers, three different characters, one shape: **a comment is not outside the thing it is written in**.

415. The first bill raised here would have been the first unattributed one on the books

`check_unwritten_columns.py` lists `mcj_sales_invoices.department` — filled on all 224 bills and written by nothing in this system.

Defect 293 is the reason that matters. `dept_code` sits on twenty-eight legacy tables, every sale and order and conversion records the department that raised it, and **the legacy ERP had no screen that read it back**. This rebuild built that screen: `DepartmentWork`, whose own comment says *"every one of them is attributed — there is no 'not recorded' row to show, which is unusual in this data."*

It was going to stop being unusual. The tag screen offers a department picker; **the sale screen had none**, so the first bill raised here would have arrived on that report under `(no department recorded)` — a row the report has carried since defect 295 and the shop's own data has never needed.

Built: a *Raised by* picker on the sale screen, beside *Sold by*, from `mcj_legacy_departments` — the same list the tag screen offers, offered the same way.

```
x the bill was raised by department "", want 5
```

Optional, and that is a decision. A required box the counter cannot answer is worse than a bill the report already knows how to display, and the shop's own salesperson field is blank on 93 of 224 for the same reason. Both cases tested: one bill that names CASH, one that names nobody.

The list shows the department's **name**, not its code — the counter sees CASH, not 5, which is the same choice the salesperson list makes.

Not done, and why

The **order screen** has the same gap: `mcj_customer_orders.department` is filled on all 284 and written by nothing. It is the same shape and the same fix, and the order form is a different template with a different props type, so it is left as a separate piece of work rather than half-done here. Named so it is not rediscovered as a surprise.

416. The order screen's half of it, closed

Defect 415 built the department picker on the sale screen and named the order screen as the same gap, left as separate work. This is that work, done rather than left to be rediscovered.

```
mcj_customer_orders.department filled on all 284, written by nothing
```

Built: a *Taken for* picker on the order screen, from `mcj_legacy_departments`, beside *Taken by* — the same list the sale and tag screens offer, offered the same way, storing the code and showing the name.

```
x the order was taken for department "", want 2
```

Optional, for the third time and the same reason: a required box the counter cannot answer would stop an order the shop has always been able to take. Both cases tested here too.

Why the order half was worth the second trip

The department report's most useful reading is about orders, not bills. Its own comment: *"ORDER raises 19 bills averaging over three lakh each while CASH's 185 average under a lakh, and WORKSHOP takes 117 of the 284 orders. That is a picture of how work flows through the shop, not a league table."*

117 of 284 is WORKSHOP. An order taken here with no department would not just be an unattributed row — it would quietly thin the one figure that shows where the shop's work actually goes.

Three screens now agree

tag	Department	since defect 293/295
sale	Raised by	defect 415
order	Taken for	defect 416

All three read `mcj_legacy_departments`, all three store the code, all three show the name, and all three allow blank. The next screen that writes a departmental record has a pattern to copy rather than a decision to re-make.

Conversions — the third source the department report reads, 859 of them — are loaded and not written by any screen at all, so there is no gap there to close yet. When a conversion screen is built, this is the fourth place it goes.

417. A stone set here belonged to no envelope

`mcj_stock_stones.packet_id` — 107 of the shop's 819 stone rows carry one, across 104 tags, and **no screen in this system could set it.**

That is not a dormant column. Defect 391 rebuilt the Packet Wise Stock Summary on it eight entries ago, and taught it to count only what is still on the shelf: ₹11,91,170.00 of stones in ten packets. A diamond set into a piece tagged here would have been invisible to that screen for ever — not wrong, not missing, just not part of any envelope.

Built: a *Packet* column on the tag screen's stone grid, between Colour and Pieces, using the same `stonePick` control shape and colour already use.

```
x the stone came from packet "", want DMIX
```

Optional, on the same terms as shape and colour. The shop names a packet on 107 of 819 rows — seven eighths of its own stone work does not — so requiring one would stop the majority of tagging to record a minority practice. Both cases tested: a stone from DMIX, and a stone from nowhere.

The list is long and mostly dormant, and stays that way

The master holds **205 packets and only ten are drawn on** (defect 391). The picker shows all 205. Hiding the 195 would be the tidier screen and would make it impossible for a packet to ever become drawn-on: the first pick is exactly the moment a dormant packet stops being dormant.

Sorted by the number written on the envelope, which is what the person at the bench is holding.

The pattern, four screens deep now

tag	Department	293/295
sale	Raised by	415
order	Taken for	416
tag	Packet, per stone	417

Same shape each time: a column the shop fills, a screen that could not fill it, a list read from the shop's own master, the code stored and the name shown, and blank allowed because the shop's own data has blanks. Four of these in four entries is not four discoveries — it is one gap that `check_unwritten_columns.py` has been naming all along, worked through a column at a time.

418. A customer asking for a size 14 could not be answered from a tag made here

Two more from `check_unwritten_columns.py`, both on `mcj_stock_items`:

```
size      99 of 2,816 filled
remarks1  451 of 2,816 filled, 308 of them distinct
```

size **is the ring or bangle size** and it is the one a customer says out loud. The shop's own values run 8 to 24 — ring sizes — and 2.3 to 2.8, which are bangle sizes, **in the same column**. Free text here for that reason: a numeric field would have to decide which scale it was on, and the shop never did.

remarks1 **is a notepad**, not a code: 451 tags carry one and 308 of the strings are distinct. What is in them is mixed — MRP 2500 , 92.5 , 01A , bare numbers — which is what a free note looks like after a few years.

Built: both as text fields on the tag screen, Size beside the certificate number and Remarks after the certification charge.

```
x the tag is size "", want 14
x the bangle is size "", want 2.6 -- a decimal size is not a number to round
```

The bangle case is tested separately and deliberately. A field that quietly rounded 2.6 to 3 would be wrong in a way nobody notices until a bangle does not fit.

ref_no **was looked at and left**

260 tags carry one and **261 distinct values** — near enough one per tag, so it is an identifier rather than a category. What it identifies is not in the data: the values are bare numbers and short codes with no series and no obvious counterpart in any other table. Adding a box labelled *Reference* that nobody at the shop could fill correctly would be worse than the blank column.

Left, and named here so the next reader of the unwritten-columns list does not spend the same twenty minutes.

The list is now **45 columns**, down from 52 when defect 405 opened it. What is left is mostly the gemological detail on mcj_packets — culet, polish, symmetry, crown angle — filled on two to five packets each, which is a diamond grader's screen this shop has never had.

419. Four fields say whether a sale left the state, and no two of them agree

Defect 410 found two readings disagreeing on twenty bills. Chasing mcj_customer_orders.bill_type — 284 filled, written by nothing — turned up the invoice's bill_type , which is written, and made the count worse.

On the orders it is a false lead: bill_type there holds **CO (179) and RP (105)**, the order kind, which the order screen already takes and writes into the order number. C0/2627/00001 says CO. Redundant, not missing.

On the invoices it holds local and central , and that is a fourth opinion on defect 410's question:

bill_type	place of supply	IGST	bills
local	in state	none	184
local	in state	CHARGED	19
local	ANOTHER STATE	CHARGED	18
central	in state	none	2
central	in state	CHARGED	1

Not one bill has all three agreeing that it left the state. The eighteen the place of supply calls inter-state are all marked `local`. The three marked `central` include one whose party state, place of supply and bill type are all West Bengal — GST/2627/00214, local by every measure the shop recorded except the one that says otherwise.

A fourth reading exists and matches none of them: **29 bills name an out-of-state party**, and 128 name no party state at all.

IGST charged	38
place of supply another state	18
party out of state	29
bill_type central	3

What this changes

Defect 410 asked whether twenty counter sales were carried away or shipped. That question stands and this does not answer it. What it does is remove the last temptation to answer it from the data: **there is no field to appeal to**. Each of the four was filled by somebody at a different moment for a different purpose, and the shop is the only place that knows which describes the goods leaving.

The GST return files by place of supply, marks the disagreement with the tax charged (defect 410), and now has nothing further to consult. That is the correct state for a return whose classification is genuinely open — it files a defensible answer and says where it is uncertain, rather than picking a fourth field and sounding confident.

`bill_type` on invoices is written by the sale screen already, from the biller's interstate choice. **Left alone**. Making it agree with one of the other three would be answering 410 by writing code, which defect 414 declined to do for the same reason.

420. The loader chain, run end to end

Defect 411 changed `load_ledgers.sh` and could not run it: the vouchers reference the ledgers it deletes, so it only runs inside `load_all.sh`. That entry says so plainly and says the change would take effect on the next full reload.

This is that reload. It was safe to do because the database holds **no application-written rows** — `load_all.sh` checks six tables for rows with no legacy id before it clears anything, and every one is zero, so the reload rewrote exactly what was already there.

```
exit=0, no FAIL, no ERROR, no REFUSING TO RUN
```

Every baseline came back to the rupee

tags	2,816	in stock	2,575		
shelf	14,23,61,188.20				
vouchers	1,848	movements	9,031		
bills	224	orders	284	order lines	366
conversions	859	enrolments	239		
ledgers	785	purchases	43		

And the six things this session added or changed, all of them now written by a loader run rather than by hand:

mcj_order_rates	2,520	
mcj_credit_note_lines	6	
mcj_bill_number_links	134	
mcj_prior_uncleared	26	
ledger mobiles	450	<- defect 411, previously an UPDATE
ledger GSTINs	110	<- defect 411, previously an UPDATE
raw stock pieces	29	<- defect 387

Defect 411's caveat is closed. The mobiles and GSTINs are there because the loader put them there, which is what that entry said would have to happen before the claim was whole.

What this proves that the checks cannot

`check_all.sh` runs against whatever is in the database. It cannot tell a table loaded correctly from a table loaded once and since edited, and it has no opinion on whether the loaders would produce the same thing again. Four loaders were added this session and two changed; until now the only evidence they composed was that each ran alone.

Defect 407 is the reason this matters: two of those four hang off parents whose loaders delete, and the cascade guards were missing. The order in `load_all.sh` would have hidden it. This run exercises the order, the guards and the arithmetic together, and the fact that every figure returned unchanged is the result.

Nothing was found. That is worth an entry: a full reload that changes nothing is the only evidence that a session of loader work changed nothing it should not have.

421. Twenty-six equally-weighted rows is not a list

The goal names `docs/STATUS.md` as the single place that says what is done, what is left, **and what needs the shop's answer**. The third of those had stopped working.

rows in "Waiting on you"	26 open
longest row	1,374 characters
ordering	the order they were written in

Everything in it is true and none of it is prioritised. A reader looking for what to do on Monday has to read four thousand words and rank them himself, which is the work the list exists to save. Four of the twenty-six were added today, and each addition made it slightly less usable — an ask list that grows without being ordered eventually costs more attention than it saves.

Built: two short tables above the existing one, and the existing one untouched below.

The first names **three things that cost an hour each and cannot be undone if skipped** — get the capture off the laptop (B18: the only copy of 40,519 customer records), rotate the exposed credentials (B15), and check whether one tag photo still loads (B24: 889 tags worth ₹10,73,39,445.03, **75.4% of the shelf**, photographed only on hosts the shop does not control).

The second names **the four questions with the most money on them** — the scheme booking and settlements (₹87,90,900 collected, ₹77,37,300 owed), the twenty IGST bills (₹1,56,108.86), the untagged pieces sold (₹9,26,698.00), and the basis the shelf is valued on (₹3,94,01,031.93).

Then one sentence saying the rest is real and none of it is urgent, and that the table below is in the order things were written rather than the order they matter.

The ranking is not by money, and that is the point

B18, B15 and B24 carry no rupee figure between them and they are first. The three sort by **what cannot be recovered**: a failed disk, a live token, a photo host that stops answering. Everything below sorts by money because the money is already spent and will still be there next month.

Every figure quoted was checked against a second source before it went in — the 75.4% recomputed from ₹10,73,39,445.03 against the shelf's ₹14,23,61,188.20, the ₹3,94,01,031.93 read back from where defect 346 derived it. A summary that gets a number wrong is worse than no summary, because a summary is the part people read.

Nothing was deleted. The twenty-six rows are all still there and all still exact.

422. Two things the shop did last week, and "What it cannot do" did not mention either

Defect 421 made the ask list usable. The section above it — **What it cannot do** — turned out to be the one with the real gap: five rows, two of them struck through as fixed, and neither of the two biggest omissions listed.

A stock conversion cannot be recorded

859 conversions, 1,242 pieces
most recent: 18 August 2026 — the day before the books end

Nothing in this system writes `mcj_stock_conversions` or `mcj_stock_conversion_pieces`. The shop melts and remakes pieces, did so the day before the capture ends, and this system can display every one of those 859 and record none.

The awkward part is how it was counted. The menu has exactly one entry for this:

```
"RETAIL","(direct)","Stock Journal","stock_journal.php"
```

RETAIL, not **REPORTS** — the area where transactions are entered. This rebuild registered `stock_journal.php` as a read-only report under `ReportRead`, and `check_todo_screens.py` counts it among the 126 built, correctly by its own rule: the screen is answered for. But "built" here means the reading half of something whose place in the menu says it also wrote.

I cannot prove from the capture that `stock_journal.php` was the entry screen — the file is not in the dump. What is certain is that the shop entered 859 conversions, that no other menu entry is about conversions, and that this system cannot enter one.

A scheme instalment cannot be taken

2,144 instalments, ₹87,90,900
most recent: 10 August 2026

Defect 233 established this and it is on the ask list, where it has sat since. It was not in **What it cannot do**, which is the table somebody reads to find out what this system will not let them do on Monday.

It is not a screen anybody forgot. There is **no scheme, chit or deposit ledger among the 785**, so a screen that took the money would book it nowhere — which is the question 233 asks and the shop has not answered.

Both added, and the honest reading of the count

The two rows are in the table now. Neither changes the 126-of-193 figure and neither should: that count is against the old menu and the old menu is answered. What it does not measure is whether a rebuilt screen does everything the original did, and this is the second time that gap has been the finding — defect 311 was the first, where "no trace at all" of manufacturing turned out to be too strong.

A screen counted as built can still be half a screen. The count is honest about what it counts and silent about what it does not, and the way to keep it honest is to say the missing half out loud in the table next to it.

423. The menu entry called GST Sale opened a list of sales already made

Defect 422 found a screen counted as built that was the reading half of one that also wrote. That is a mechanism, so I looked for others: every menu entry in a **transaction** area that this rebuild serves with a read-only report.

Twenty-seven came back, and **most are false positives**. ACCOUNTS holds rpt_cashbk.php , rpt_trl_balance.php , t_formbalancesheet.php — reports by name and by nature, in an area that mixes vouchers with reports. A heuristic that reads the menu area cannot tell a list screen from an entry screen.

One is not a false positive, and reading the submenu settles it.

RETAIL > Sales Voucher		
Sales Approval Voucher	vou_sales_appr.php	
GST Sale	sales_voucher_gst.php	<- pointed at /reports/sales-register
Credit Sale	sales_voucher.php	
Export Sale	sales_voucher_export.php	
Repair Sale	sales_voucher_repair.php	
Estimate Sale	sales_voucher_estimate.php	
Exhibition Sale	sales_voucher_exhibition.php	
Credit Note	vou_sale_return.php	
Sales Return REGISTER	sales_ret_reg.php	

Every sibling is an entry screen. And one item in that list is named **Sales Return Register** — a shop that takes the trouble to call its register a register did not mean *GST Sale* to be one.

A biller following the menu to make a bill arrived at a list of bills already made. The screen that actually makes one was registered against `add_sales_voucher_combined.php` — a real legacy file that **is not in the menu at all**, placed by hand under an invented item called "Sales Bill".

Fixed

`sales_voucher_gst.php` now opens the sale screen, under `VoucherRead` rather than `ReportRead` , and the hand-placed duplicate is gone. The register keeps a home of its own under `REPORTS > Sales Report`, because a report nobody can reach is worse than one in a slightly different place.

The screen count is unchanged at 126 of 193 — the same menu entry is answered, by a screen that does what its name says. What moved is the count of built screens that are **not** menu entries: 6 → 5.

The i18n trap, third time today

```
MCJMenu.Item.sales-register (/mcj/reports/sales-register)
```

A new menu item with no translation panics **every** page, not just its own. Defect 401 hit it, defect 415 and 416 avoided it by reusing existing items, and this one hit it again. It was caught by `TestNavigation_EveryMenuTitleHasATranslation` before the render check ran — which is the test earning its place: it names the missing key and the path, so the fix is one line and no debugging.

What the sweep is worth

One real finding out of twenty-seven candidates. The other twenty-six are the cost of a heuristic that had to be written to find the one, and they are recorded rather than hidden: nobody should run this query again expecting a clean list.

424. The navigation audited the other way, and a near-miss on the way

Defect 423 swept transaction-area entries served by reports and found one real defect in twenty-seven candidates. This is the mirror sweep and the leftovers.

Report-sounding names served by an entry screen: two, both right

ACCOUNTS	Bank Book Clearance	brs_cler_rpt.php	-> ReconcilePath
HR	Address Book	address_book.php	-> PartyPath

The first has `_rpt` in its file name and **Clearance** in its title, and the bank reconciliation is where entries are ticked as cleared — a screen that shows and acts, correctly served by one that does both. The second is a master list that is also where a customer is edited. Neither is a defect.

So the mapping is sound in both directions apart from 423.

The five screens that are not menu entries

label_print.php	Print Tag Labels
day_book.php	Day Book
print_sales_vocher_register_gst.php	Print a Tax Invoice
add_gold_rate_fix.php	Quote a Price
set_rate_gold_opening.php	Metal Rates

All five are real legacy files placed by hand because the menu never listed them, and each carries a comment saying so. The last is the interesting one.

The old menu had no way to set the day's metal rate. Seven entries mention gold or metal and every one is a purity master, a karigar issue or a report; not one sets a rate. Yet the rate is the first thing a jeweller does each morning — it is where the day's-trade test starts, and 2,820 rates were recorded.

That is a fact about the old menu, not a gap here: the screen exists and is reachable. Recorded because "126 of 193 menu screens" invites the reading that the menu describes the shop's work, and on the single most-used action of the day it does not.

The near-miss

I read `coinrate.go` writing to `mcj_coin_rates`, saw `update_item_rate.php` registered at `ReportsPath + "/coin-rates"`, and had most of an entry written about a screen that changes rates being served at a report path under a read permission.

It is two different screens.

<code>/mcj/coin-rates</code>	<code>coinrate.go</code> , writes, <code>ItemRead + ItemUpdate</code>
<code>/mcj/reports/coin-rates</code>	the report, reads

Caught by opening the path constant instead of assuming the controller file owned the path its name resembles. **Fifth measurement error of the session**, and the first where the mistake was in reading the code rather than the data:

394	a falsification that changed nothing
398	a ratio read as a diagnosis
409	a regex matching \$1 inside \$12; a grep discarding a helper
410	a join that fanned out 20 bills into 80
424	two paths that share a name and are not the same screen

425. A table that says it was counted mechanically was not

`docs/screens.md` opens with "**117 of 187 legacy menu items**". The true figure is **126 of 193**, and the paragraph directly beneath that number says:

the count still has to be taken rather than edited by hand, which is how this table came to say 20 long after it was 117

It has now done it again, and nothing caught it: `status_figures.py --check` validates STATUS.md and has never read screens.md.

And STATUS.md's own area table has the same disease

Remaining, by area carries the line **"Counted from the menu CSV against the navigation registry, not by hand."** Two of its twelve rows were wrong:

	said	the menu says
MIS REPORTS	6 of 6	7 of 7
REPORTS	26 of 49	25 of 48

The two errors cancel. One screen counted under REPORTS instead of MIS REPORTS leaves the total at 126 of 193 either way — which is exactly the figure the check does validate, so it passed every run while the table beneath it was wrong in two places.

The deduplication that makes it 193

Counting rows gives 195. `holiday_calender.php` appears twice in the CSV, under SETUP > Payroll SetUp. `status_figures.py` builds a **set** of links, so its headline was right; the area table was built by hand and counted the row twice.

That is the whole mechanism: a figure taken mechanically at the top of a file and by eye a hundred lines further down.

Fixed, and checked

Both tables corrected, and `status_figures.py --check` now validates:

- every area row in **both** documents against the deduplicated menu count
- screens.md's own headline against the same source as STATUS.md's

x docs/STATUS.md: MIS REPORTS says 6 of 6, the menu says 7 of 7 x docs/screens.md: headline says 117 of 187, taken as 126 of 193

Both falsify by restoring the exact error that was there this morning.

This is defect 408's lesson in a different room. A figure that has drifted twice in the same file — the preamble records the first time — does not need a third correction; it needs the thing that corrects it to be a program.

426. The other four documents, swept for the same disease

Defect 425 found two guarded-looking tables that were not guarded. The obvious next question is whether the four documents nothing checks are carrying the same kind of stale figure.

DECISIONS.md	no count claims at all
pkg-contracts.md	no count claims at all
pricing-formula.md	three, all correct
migration-profile.md	one stale cross-reference

One line, and it is a cross-reference rather than a figure:

```
> Superseded in part by data-defects.md, which carries the  
> full 18-defect register
```

There are 425. Written when there were eighteen, in the present tense, and left.

Fixed by saying what it meant — *"18 entries when this line was written and several hundred since"* — and by adding what the file had never said out loud: **this is a dated snapshot, not a live document**. It records the 2026-08-21 dump as first loaded and its figures are deliberately not maintained. A reader arriving at "451 tables, 128,176 rows" needs to know that before they compare it to anything.

That is the better fix. A number corrected today drifts again; a sentence saying the file is not maintained stays true.

The near-miss in pricing-formula.md

```
`diamond_rate_master` (774 rows, ...)  
  
legacy.diamond_rate_master    774  
mcj_diamond_rates              1,416
```

A loader producing nearly twice its source is the shape of a join fanning out — which I had done to myself sixteen entries earlier at defect 410, so I went looking for it with some confidence.

There is no fan-out. The loader reads **two** tables:

```
diamond_rate_master      774   the retail grid  
dealer_wise_diamond_rate  642   the dealer grid  
                        1,416
```

and its own guard checks that exact sum, which is why it has never complained. `mcj_diamond_rates` splits retail 774 / dealer 642 on inspection. The document's 774 describes the retail table it names and is correct.

Sixth near-miss of the session, and the second in two entries where the suspicion came from a real pattern I had recently been burned by. A pattern learned is a pattern over-applied for a while; the answer is the same either way — open the loader and read what it selects from.

427. Sixteen thousand of input credit against a supplier with no GSTIN anywhere

Defect 411 put the **buyer's** GSTIN on the outward return. The mirror question is the supplier's, which is what an input credit rests on.

The purchase side is in better shape than the sale side. On the tax classification it is exactly right, which is worth saying after defects 410 and 419:

```
43 purchases  
11 marked interstate, 11 charged IGST  
0 marked interstate with no IGST  
0 charging IGST while marked local
```

interstate comes from the shop's own `gst_type = 'gst2'` flag and agrees with the tax on every bill. The sales-side tangle is a sales-side tangle.

Where it is not right

purchases from a supplier with no GSTIN in the ledger	22
of those, carrying no tax at all	19
of those, carrying input tax	3

Nineteen are unregistered-supplier or old-gold purchases with no GST on them, and a missing GSTIN is the correct state for those. **Three are not.** All three are the same supplier:

ANJALI LABTECH LIMITED, ledger 12168					
GDP/2627/00005	18 Apr 2026	1,44,019.00	IGST 4,320.57	B2B	
GDP/2627/00023	22 Jul 2026	3,43,545.00	IGST 10,306.35	B2B	
GDP/2627/00024	5 Aug 2026	57,579.00	IGST 1,727.37	not B2B	

₹16,354.29 of input tax, all interstate, from a limited company whose GSTIN is recorded nowhere — not in `mcj_ledgers` and **not in** `legacy.ledger_master` **either**, so nothing was dropped in loading. The shop never wrote it down.

An input credit needs the supplier's GSTIN on the invoice. The paper invoices may well carry it; this system cannot show it, and neither could the old one.

And the third bill is marked differently from the other two

Same supplier, same interstate basis, same month-to-month trade — and `GDP/2627/00024` is not flagged B2B while the other two are. That flag is the shop's own `b2b_purchase`, carried faithfully, so it is the shop's inconsistency rather than the loader's.

It matters because the inward return sections split on it: two of these bills appear in the B2B section and the third does not, and they are the same relationship.

Not added to the ask list

Deliberately. The ask list was cut back to what is urgent at defect 421, and ₹16,354.29 against a supplier the shop can telephone is a bookkeeping tidy, not a decision only they can make. It is recorded here, where the accountant preparing a return will find it, rather than competing with the ₹87 lakh questions above.

428. The half that claims money was the poorer of the two

Defect 411 put the buyer's GSTIN on the outward return and stopped there. The inward return had none — which is the wrong way round, because an outward GSTIN identifies who was billed and an **inward one is what an input credit rests on**.

Built: a *Supplier's GSTIN* column on the inward return, read from the ledger the purchase's `supplier_code` names, on invoices and debit notes alike. 21 of the shop's 43 purchases now show one.

x	none of the 2 inward rows carries the supplier's GSTIN
---	--

The falsification is the useful one: looking the supplier up by **name** instead of code blanks the column and every other test still passes. A join on the wrong key does not fail — it finds nothing, or worse, finds somebody else, which is defect 362's mechanism and the reason this one is keyed on the code.

It makes defect 427 visible instead of merely recorded

Yesterday's entry found ₹16,354.29 of input tax on three bills from ANJALI LABTECH LIMITED, a supplier whose GSTIN is recorded nowhere. That was a query I ran and wrote down. It is now a blank cell on the return an accountant is reading, next to the tax being claimed.

A finding needs somewhere to appear. The column had to exist before the gap in it could be seen by anybody but me.

The fixture, less real than the data again

`seedInward` set `supplier_name` and no `supplier_code`, so nothing read off the supplier could be exercised — the same shape as defect 411's outward fixture, which set no `ledger_id`. **All 43 of the shop's purchases carry a supplier code.** Fixed in the fixture, with the two suppliers seeded as ledgers, rather than worked around in the test.

Twice in eighteen entries the test seed has been the thing standing between a column and its coverage. Both times the seed was simpler than the data it stood for, and both times that simplicity read as "there is nothing to test here".

429. Fifteen lakh of making, written by the screen and asserted by nothing

Defects 411 and 428 were both found because a test fixture was simpler than the data. Twice is a pattern, so I asked the question mechanically: **which columns does the shop fill, with more than one value and never a null, that no test in this repository names?**

Fifty. Most are inert — the gemological detail on `mcj_packets` is twenty of them, filled with two to seven distinct values across 205 rows, which is a grader's screen this shop never had (defect 418).

`mcj_parties.website` is another. A column being untested is not by itself a fault.

One is not inert:

```
mcj_sales_invoices.making_charge    224 bills, 161 with a figure, 157 distinct
                                     ₹15,20,734.00 in total
```

The sale screen writes it. **No test named it.**

What it is, and why that makes it worse rather than better

It is a memo. `goods_amount` carries the whole value of the goods with making inside it, and `making_charge` says how much of that was making — the shop's own convention, measured on all 224 bills when it was implemented. Nothing in this system reads it back.

A driver that goes wrong shows up on a screen. **A memo that goes wrong shows up nowhere** — the bill still totals correctly, the ledgers still balance, and the only symptom is that the answer to "how much of last year was making?" is quietly false.

Pinned

```
x the bill's making memo says 0.00 and its lines carry 1000.00
```

Two assertions, and the second is the one that would have caught the older mistake this convention replaced: the memo must not **exceed** the goods figure, because a bill where making is added beside the goods rather than counted inside them is the error the comment above that INSERT records having made once.

On the other forty-nine

Not tested, and mostly not worth testing. The list is recorded here rather than acted on, because the useful thing about it is the shape: **a column the shop fills on every row, carrying real variety, that no test mentions, is where the next 411 will be.** `mcj_stock_items.wastage_percent` (14 distinct across 2,816) and `mcj_stock_items.design_code` (94 distinct) are the two I would look at next.

430. Five thousand designs and no way to ask which had ever been made

Following defect 429's own list. `mcj_stock_items.wastage_percent` was the first name on it and is a dead end — **2,790 of the 2,816 tags carry 0.0000**, and the sale screen already says out loud that it has no field for the 14 bill lines that use one and "does not invent" it. Honest and already handled.

The second name is not a dead end.

tags carrying a design code	109 of 2,816
naming a design that exists	70 (69 of them on the shelf, worth ₹10,38,715.52)
naming nothing	39
designs in the catalogue	5,221
screens or reports joining the two	0

The shop links a finished piece back to the design it was made from. It has done so 109 times. **Nothing in this system had ever joined those two tables**, so a catalogue of 5,221 designs could not answer "*have we ever made this one, and is there one in the case now*" — which is the question somebody flipping through a design book is actually asking.

Built: *Made* and *On the shelf* columns on the Design Catalogue.

```
x DBNG0001 shows "2" made and "2" held, want 2 and 1 -- one was sold
x DBNG0002 shows 0 made, and nothing was ever made from it
```

Blank rather than nought on a design never made, which is **5,152 of the 5,221**. A column of noughts down five thousand rows would bury the sixty-nine that carry a number, which is the same rule as defect 401's reorder levels and defect 418's piece counts.

The thirty-nine that name nothing

0	13 tags
L	2
2	2
s	2
RJ JEWELLERY	1
Y	1

Not design codes at all. Thirteen tags carry the literal `0`, several carry a single letter, and one carries **RJ JEWELLERY** — which is a `lot_no` value (defect 406), typed into the wrong box.

Left as they are. They are what the shop recorded, they name no design so they count towards none, and inventing a rule to clean them would be guessing at what somebody meant to type in 2023.

431. Every stock movement records the address it was made from, and nobody can see any of it

Third name on defect 429's list. `mcj_stock_movements.from_address` is not an address in the postal sense:

```
8,561 of 9,031 movements carry one
34 distinct values, every one an IP address
24 of the 34 appear ten times or fewer

122.163.127.22    5,307 movements
49.37.41.119      1,770      (all of them OP-STK, the year-opening load)
115.187.57.184    535
115.187.57.108    483
```

This system writes it too: the tag screen records the client IP on every new tag. Nothing anywhere displays it.

Sales came from seven different machines

122.163.127.22	278 sales	May 2024 – Aug 2026
115.187.43.78	25	Aug 2024 – Apr 2026
203.171.244.119	9	all on 21 July 2026
152.58.177.69	5	all on 5 August 2024
122.168.89.218	4	1 April 2026
122.168.92.11	2	3 April 2026
122.168.93.47	1	15 April 2026

One address is plainly the counter. The rest are small and clustered on single days, which is what a mobile connection, a home broadband or a backup line during an outage all look like.

What I am not going to say

It is tempting to read this against **defect 66** — whether an invoice printed by the test system ever left the counter. Nine sales from one address on one day is the shape of an answer.

It is not one. A different IP is not a different system, single-day clusters are what stand-in connections look like, and I have no evidence tying any address to the test numbering. Saying otherwise would be defect 398's mistake — a pattern read as a diagnosis — with somebody's bookkeeping on the end of it.

And why no screen was built

The data is here and it is an audit trail the shop already keeps. Surfacing it would take an afternoon. **It is not built because a screen showing IP addresses to somebody who does not know which is the counter is a screen that manufactures suspicion.**

One sentence from the shop — *"122.163.127.22 is the shop, 49.37.41.119 was the old line"* — turns thirty-four opaque numbers into a usable trail, and until then the honest thing is to record what is there.

Available on request, and deliberately not on the ask list: defect 421 reserved that for what is urgent, and this is not.

432. Nearly half the shelf was tagged by somebody the user master has never heard of

Fourth name on defect 429's list, and it turns defect 395 from a curiosity into a number.

`mcj_stock_items.legacy_user_id` is misnamed: it holds a **name**, not an id. 2,808 of the shop's 2,816 tags carry one, across 19 names.

tags naming who made them		2,808			
by one of the user master's 24		1,510			
BY SOMEBODY ELSE		1,298		– 46%	
names not in the master		12			
RANAJIT	614	MEHUL	40	Sourav	2
AKASH	249	RAJNISH SINGH	40	DINESH	1
ARINDAM	212	sohan	15	PUNAM SHARMA	1
admin	113	AMIT	8	ANUPAM	3

Defect 395 found three names on the books that the user master does not have — ADMIN, RANAJIT and AMIT — reading vouchers and movements. **This is the same question asked of the tags, and it finds twelve**, nine of which 395 never saw.

AKASH, ARINDAM, MEHUL, RAJNISH SINGH, sohan, ANUPAM, Sourav, DINESH and PUNAM SHARMA between them tagged **556 pieces**, and the system that is meant to say who could do what in this shop has no record that any of them existed.

What it does and does not mean

It does **not** mean 1,298 tags are suspect. The likeliest explanation is ordinary: staff come and go, and a user master captured in August 2026 holds who had a login **then**, not everyone who ever had one. RANAJIT's 614 tags run across the whole capture; that is an employee, not an anomaly.

What it means is that **defect 395's screen answers "who had which powers" for 24 people and the shelf was built by at least 31**. Anybody reading that screen as a complete account of who touched this shop's stock would be wrong by twelve names and nearly half the tags.

Recorded on the User Master's own entry rather than raised as an alarm, because the screen is right about what it shows and wrong only about what a reader might assume it covers.

The column name

`legacy_user_id` holding `PRIYANKA` is the sort of thing that costs somebody an hour. Left as it is — renaming a loaded column to match its contents means a migration, a loader change and a reload for a field nothing reads — but noted here, which is where the hour would otherwise be spent.

433. The rest of that list, closed out

Defect 429 produced fifty columns the shop fills that no test names, and said the useful thing about it was the shape rather than the list. Four were worth following: 430 built the design link, 431 found the IP trail, 432 turned defect 395 into a number. These are the rest, and none of them is a defect.

<code>mcj_items.barcode_short_code</code>	319 distinct over 322 items and NOT ONE of the shop's 2,816 barcodes begins with its item's code
---	--

A code the item master carries and the barcodes do not use. The shop's barcodes are plain eight-digit numbers. Whatever the short code was for, generating a barcode was not it.

<code>mcj_designs.designed_on</code>	5,221 designs, 12 distinct dates 4,991 of them on 5 July 2016
--------------------------------------	--

Not a design date. It is the day the catalogue was loaded into the old system, with a few hundred added over the following months. Anything reading it as "when this was designed" would be reading a data-entry date.

<code>mcj_stock_items.order_sl</code>	four values: 0, 1, 2, 3
<code>mcj_stock_items.remarks2</code>	three: blank, `2.4``, `MRP 1250`

A serial within an order, and a second remarks box used twice in 2,816 tags — once for a size and once for a price, both of which now have fields of their own (defect 418).

The whole suite, after ten entries of change

12 checks, the Go suite included, all green

Defects 423 through 432 changed the navigation registry, two report queries, the sale screen, the tag screen, the order screen, the locale file, the figures tool and six test files. This is the second full run of the session; the first was at defect 420, before any of it.

Nothing broke. Recorded because ten entries of "eleven checks green" from the quick run is not the same claim: the quick run skips the Go suite, which is where a navigation change or a report column would break something the render check cannot see.

What 429's sweep was actually worth

```
50 columns named
3 worth building or investigating (430, 431, 432)
4 closed as inert (this entry)
43 never opened, and correctly so - packet gemology, party contact
                                   details, and the like
```

Two of the three finds were real gaps a person would want closed, and the third turned a 24-name screen into a 31-name question. That is a good return on one query, and the reason to write the query down rather than the answers: the next person to add a column will want to run it again.

434. The bill can say who to ring, now that the ledgers know

Defect 411 fixed the ledger loader, which had been reading `mobile` — empty on all 785 rows of the source — instead of `mobile1`, which holds 450. That entry put the numbers in the database. It did not put them anywhere a person sees.

```
bills carrying a mobile          196 of 224
written by this system           0
customer and dealer ledgers with one 424 of 561
```

The shop records the customer's number on seven bills in eight. A bill raised here carried none — not because nobody thought of it, but because until yesterday there was **nothing to copy**: no ledger in this database had a mobile on it.

Built: the bill takes the number from the chosen customer's ledger. Nobody types it twice, and a repair or an order can be telephoned about from the bill itself.

```
x the bill carries mobile "", want the customer's 9830012345
```

Blank where the ledger has none, and tested separately. A bill that invented a number would be worse than one that admits it does not know, and 137 of the 561 ledgers have none.

A fix that was only available because of an earlier one

Worth naming as a sequence. Defect 411 looked like a loader tidy — one column read from the wrong name. It was the precondition for this, for the GSTIN on the outward return, for the supplier's GSTIN on the inward (428), and for defect 427's finding being visible at all.

None of those were possible while the column was empty, and the column being empty read as "the shop never filled this in" rather than "we asked the wrong question". That is the whole reason `check_empty_columns.py` exists and the reason its list was worth walking to the end (defect 412).

The backtick, again

```
sale.go:1641:56: syntax error: unexpected name mobile in argument list
```

Writing `mobile1` in backticks inside a SQL comment inside a Go raw string closed the string. **Defect 414 recorded this exact trap** — *"a comment is not outside the thing it is written in"* — and I did it again in the same file eleven entries later, in a comment explaining the fix that entry made possible.

Recorded rather than quietly repaired. A lesson written down and not learnt is worth knowing about: the note in 414 was accurate and did not change my hands.

435. Three sales the trail records and the bills did not make

`check_invariants.py` carries eleven allowances — known exceptions a rule is permitted, each tracked so it cannot grow unnoticed. One has read **9 allowed, unchanged** since defect 314 and nobody had asked which nine.

```
tags the trail says were sold and the shelf still counts: 9
worth ₹99,492.23
```

Six name bills in the **2025-26** series, a year whose vouchers were never captured (defect 402). Five of those six are GOLD RAKHI sold on one bill on 23 August 2025 — five of the sixteen flat-priced rakhis defect 389 found. Nothing here can adjudicate them.

Three name bills this system holds, and those can be settled.

	the movement says sold on	the bill's lines carry
53541166 ₹52,445.17	GST/2627/00207	69343561R1
67675269 ₹9,512.97	GST/2627/00193	45623887 and one untagged
15551589 ₹1,866.09	GST/2627/00219	one untagged line

Different pieces entirely. And the bills are not short:

GST/2627/00193	goods 5,643.00 = lines 5,643.00	complete
GST/2627/00207	goods 95,437.00 = lines 95,437.00	complete
GST/2627/00219	goods 2,136.00 = lines 2,136.00	complete

Every one balances to the rupee against its own lines — the property defect 429 pinned. **A complete bill that does not carry the piece did not sell the piece.**

So the movement is the wrong record and the shelf is right: **₹63,824.23 is correctly on the shelf**, and three SALE-GST movements describe sales that never happened.

What was done

The allowance stays at nine — the rule is about a disagreement and the disagreement is real. What changed is that its comment now says **which record is wrong for three of them, and why**, with the test that settles it: a bill whose goods equal its lines has nothing missing.

Not "fixed". Deleting three movements to make a check tidier would be editing the shop's trail to match a conclusion, and the trail is evidence. The shelf figure — which is what the money rests on — was never wrong.

Why this took until now

The allowance had been sitting as a number since defect 314. It says a rule is knowingly broken nine times, which reads as settled. **An allowance is a suspended question, not an answered one**, and eleven of them are suspended in that file. This is the first to be reopened.

436. One absence, three exceptions

Defect 435 reopened one of eleven suspended allowances. Two more, and both come back to the same thing.

a payment names a bill that exists — 8 allowed, all one cause

GST/2526/00017	bank	18,500.00	5 Apr 2025
GST/2526/00066	bank	1,300.00	29 Apr 2025
GST/2526/00080	bank	5,100.00	3 May 2025
GST/2526/00094	bank	5,000.00	14 May 2025
GST/2526/00100	bank	1,200.00	20 May 2025
GST/2526/00104	bank	800.00	21 May 2025
GST/2526/00246	bank	27,069.00	1 Aug 2025
GST/2526/00622	bank	612.00	29 Oct 2025

		59,581.00	

Every one names a GST/2526 bill — the 2025-26 series. The capture holds 2026-27 and nothing before it. The payments are real, the bills were real, and the dump begins after them.

Nothing to fix and nothing to chase. The allowance is now annotated to say so.

The missing year, counted in one place

This is the third exception in this session tracing to the same absence:

defect 402	26 bank items uncleared at the year end	₹43,88,746.00
defect 435	6 tags the trail sold on a 2025-26 bill	₹35,668.00
defect 436	8 payments against 2025-26 bills	₹59,581.00

Three separate rules, three separate allowances, one cause. Each was recorded on its own as a known exception, and none of the three said what it had in common with the others. **The year before the capture leaves a mark in every register that references a document by number**, and this is the first entry to add them up.

every piece has a beginning — 57 allowed, and mostly not a mystery

53 of the 57 carry an OP-STK movement
3 reference a prior-year document

They came in as **opening stock** rather than being newly tagged. The rule looks for a NEW-BAR and finds none, which is correct as far as it goes — but an opening entry is a beginning. That is not the missing year; it is the ordinary shape of a shop that existed before its stock system did.

Annotated rather than changed. Narrowing the rule to accept OP-STK would make the number smaller and the check weaker: the four that are neither opening stock nor prior-year are the ones worth keeping visible, and they are only visible while the rule counts all 57.

Nine still suspended

a movement's reference is filled in	470	
clearing a cheque sets both columns	31	
one voucher moves a tag once	18	
a coin price is not out by a digit	14	
an order says what was ordered	13	
a tag the trail returned is back	3	(defect 369)
every voucher balances	1	(B4, a rounding paisa)
a barcode names one tag	1	(B8)

437. The largest allowance is one table with no column to fill

Two more of the nine suspended allowances, and the biggest of them turns out not to be a defect at all.

a movement's reference is filled in — **470, and every one from one source**

barcode_wise_all_transaction	8,561 movements	0 without a reference
retail_all_stock	470 movements	470 without a reference

The stock trail is merged from two legacy logs. One of them has **no document column at all**:

```
tran_id, date1, item_id, barcode, status, item_type, tran_type,  
qty, purity, nw, gw, rate, active, dept_code, comp_code, ac_yr, rt_code
```

It is a stock snapshot — status, quantity, purity, weights — not a transaction log. There is nothing in it to put in a reference, and there never was.

So the largest allowance in the file is not 470 missing values. It is **470 rows from a table that does not have the field**, and the other 8,561 are complete.

Left unscoped on purpose. Restricting the rule to `barcode_wise_all_transaction` would make the allowance zero and read better, and it would stop noticing if the `retail_all_stock` side grew. An exact allowance of 470 catches that; a scoped rule with a clean zero would not. The number is ugly and it is doing work.

clearing a cheque sets both columns — **31, and the money is fine**

```
31 payment vouchers marked cleared with no clearing date  
13 May – 5 August 2026, all PMNT, 12 carrying a cheque number
```

This year, not the missing one, so it is not defect 436's cause. It is a data-entry habit: the box was ticked and the date was not typed.

The bank reconciliation is unaffected — a cleared item is excluded from the unpresented list whether or not it says when it cleared, so the statement arithmetic of defect 402 is right either way. What is lost is the date: an item settled *sometime* cannot be placed in a statement month, which is exactly what somebody reconciling last month's page needs to do.

Where the eleven stand now

settled or explained	5	(435, 436 ×2, 437 ×2)
known and documented	3	(B4 rounding, B8 barcode, defect 369)
still unopened	3	one voucher moves a tag once 18
		a coin price is not out by a digit 14
		an order says what was ordered 13

Three left. None of them large, and each is a question somebody suspended rather than answered.

438. The last three allowances, and one bill at the centre of everything

Eleven allowances, and these are the last three unopened.

an order says what was ordered — **13, and twelve are the carried-forward book**

C0/2122/00013	C0/2223/00054	C0/2324/00536	C0/2425/00053
C0/2122/00334	C0/2223/00950	C0/2324/00783	C0/2425/00208
RP/2223/00008	RP/2223/00116		C0/2425/00432
			C0/2425/00489

All twelve are dated 1 April 2026 and numbered in earlier financial years — 2021-22 through 2024-25. They are defect 343's carried-forward orders, and their line detail belongs to years the capture does not hold. Third variant of defect 436's missing year, reaching back four of them rather than one.

The thirteenth, **RP/2627/00029**, is this year's: a repair taken on 31 May 2026, department 2, marked delivered, with nothing on it saying what was repaired. Genuinely empty and not explained by anything.

a coin price is not out by a digit — **14, and nobody paid them**

COIN-8 GM	15,00,400	usual 1,14,860	13.1x	26-31 May 2026
SILVER COIN 1KG	29,33,000	usual 2,56,000	11.5x	25 May
SILVER COIN-25 GM	73,330	usual 6,400	11.5x	25 May

Not a bad day at the keyboard: on each of those dates **52 coins are priced and only one or two are wild**, the same coins repeating across consecutive days.

Not one coin was sold in that window. 32 coin lines were billed across the whole year, ₹26,11,606.00, and none between 25 May and 1 June. The prices sat on the list and charged nobody — which is the only part of this that matters, and it took a query to know rather than an assumption.

one voucher moves a tag once — **18, and this is where it all converges**

Defect 375 found that eight of the eighteen are one bill: **GST/2627/00150** posted nine pieces on 3 July at 09:25 and eight of them again on 4 July at 14:06, same user, a bill re-saved rather than replaced.

Defect 397, sixteen entries later and from a completely different direction, found that bill selling **₹7,11,438.00 of goods against barcodes with no tag.**

They are the same eight barcodes.

19486670	30244386B	33455507	53147276	
66357120	69879664	70729576	92255551	posted twice, no tag
46372798				posted once, no tag, and on no invoice line

All nine barcodes that bill moved have no tag in the master, and one of them never appears on the invoice at all. Two allowances, one defect and a ₹9.27 lakh finding all point at one document and one afternoon in July.

I am not going to say what happened on 3 and 4 July 2026. The records are consistent with a bill saved twice for pieces sold before they were tagged, and consistent with other things. What can be said is that **it is one event, not three**, and the ask list already carries the question (defect 397).

All eleven now opened

explained by the missing year	3	(402, 435, 436)
explained by the carried-forward book	1	(438)
explained by their own source	2	(437 ×2)
explained and harmless	2	(438 ×2)
known and documented before	3	(B4, B8, defect 369)

Not one turned out to be a fault in this system. Four turned out to be the same fact about the capture wearing different clothes, and the largest was a table with no column to fill.

439. Two rules for the failure this shop has already had

Defect 438 ended on a question it did not ask: GST/2627/00150 was saved twice and wrote its stock movements twice. **Did it double anything else?**

duplicate invoice lines	(same bill, same barcode, same amount)	0
duplicate voucher lines	(same voucher, same ledger, same figures)	0
bills whose goods equal the sum of their own lines		224 of 224

No. The double-save reached the stock trail and stopped there. **The money was never doubled** — not on that bill and not anywhere in the shop's books.

That was a query I ran once. It is now two rules.

no bill counts a piece twice	0 allowed
no voucher posts a line twice	0 allowed
31 of 31 rules hold	

Why they are not redundant

Forcing a real duplicate in — inserting one, running the checks, removing it — fires **three** rules at once:

```
x every bill's lines add up to its goods
x the line detail sees every taxable rupee
x no bill counts a piece twice
```

which looks like the new rule adding nothing. The cases it exists for are the ones that move no total:

- a doubled line of **zero amount** — some lines are, and they carry a piece
- a doubled **debit and credit pair** in a voucher, which leaves the voucher balancing perfectly, every ledger correct, and every total unchanged

Both of those pass every other rule in this file. A bill that re-saves is exactly the mechanism that would produce them, and this shop has one on record.

Falsified for real, and put back

The duplicate was inserted into the live table rather than a transaction that rolls back, because a rule that only fails against a rolled-back row has not been shown to work. Three rules fired, the row was removed, and the table is back to its 367 lines with 31 of 31 holding.

Defect 359's lesson applied deliberately: a falsification that changes nothing proves nothing, and the safest way to prove a check works is to break the thing it checks and watch it complain.

440. A bill that names the wrong customer would balance perfectly

Defect 439 added two rules for failures that move no total. That framing is worth following: **what else could go wrong in this shop's books without a single figure changing?**

A bill that names one customer while its voucher charges another.

bills whose ledger matches their voucher's debtor	224 of 224
purchases whose supplier matches their voucher's	43 of 43
rules checking it	0

Everything about such a bill is arithmetically perfect. The voucher balances, the sales register totals, the GST return files, the trial balance ties. The only thing wrong is **who owes the money** — and no total anywhere would move.

It matters more since yesterday

Defect 404 put the customer's standing on the sale screen: *"HARSIT ANAND SULTANIA owes 32,30,681.00 on account before this bill."* That figure is read off the **ledger**. The name on the bill comes from the **invoice**.

They are the same today on all 224. If they ever came apart, the biller would be shown one customer's debt beside another customer's name — and would have no way to tell, because both numbers are individually correct.

Built: two rules, 0 allowed, one for each direction of trade.

a bill names the party its voucher charges
a purchase names the party its voucher pays

33 of 33 rules hold

Falsified against the live table

Pointing GST/2627/00001 at a different customer and running the checks:

x a bill names the party its voucher charges
1 bill(s) naming a different party from their voucher
32 of 33 rules hold

Then restored from the voucher itself — the voucher is the record that decides, which is the whole premise of the rule — and back to 33 of 33.

Live table again rather than a rolled-back transaction, for defect 439's reason: a check that has only ever failed against a row that was never really there has not been shown to work.

441. Ten thousand rupees a customer is recorded as having paid twice

Two more in defect 440's class — failures that move no total — and one of them is a real figure with a customer's name behind it.

a sold line is the purity the tag says — **232 of 232, and nothing checked**

The bill line carries its own `purity_code` and the tag carries a `purity_id`. The money comes from the line, so a line reading 22 against an 18kt tag **bills correctly and describes the wrong metal**. It also feeds the fine-weight figure — net weight at the purity's fineness — so the customer's copy would state a fine weight the piece does not contain.

All 232 lines with a resolvable tag agree. Now a rule at 0 allowed, falsified against the live table and restored from the tag, which is the record that decides.

one instalment number, one payment — **1, and it is worth ₹10,000**

party 9588
contracted 10 instalments $\times 10,000.00 = 1,00,000.00$
recorded 11 payments = 1,10,000.00, across 10 distinct numbers
the twin instalment 2, recorded twice on 20 November 2022

The books say this customer paid a whole instalment more than the scheme is worth.

Same day, same amount, same instalment number is the signature of a double entry. It could also be an eleventh payment misnumbered — a customer who paid ahead. Only the shop's receipt book for 20 November 2022 can say which.

It matters at maturity. A scheme settled on "paid ₹1,10,000" either pays out on ₹10,000 that never arrived, or short-changes a customer who really did pay it.

Not added to the ask list as its own row — the scheme questions are already there (defect 233, blocker B23) and ₹10,000 does not outrank the ₹87,90,900 sitting above it. Recorded in the invariant itself, allowed at one **so that a second one fails**, which is the part that protects the shop going forward.

The 34 enrolments with gaps are not a finding

34 enrolments skip 41 instalment numbers between them, worst 3

A chit customer who misses March and pays in April leaves a hole in the numbering. That is what a savings scheme looks like, not a defect, and it is the reason the rule counts repeats rather than gaps.

35 of 35 rules hold

442. The third document type, and a rule for what this system writes next

Defect 440 built the party-agreement rule for sales and purchases. There are **three** document types that carry both a party and a voucher, and it left the third out.

credit notes whose ledger matches their voucher's party 6 of 6

Now covered. A return crediting the wrong customer is the same silent failure as a bill charging one: perfect arithmetic, wrong person.

A weight rule for 2.7% of the trail, and all of the future

movements carrying a weight	246 of 9,031
NEW-BAR	185
OP-STK	61
sales, orders, returns	0
movements whose weight differs from their tag's	0

The legacy trail mostly does not record weights, so a rule over it covers almost nothing today. It is worth having for what comes next: `tag.go` **writes the net weight on every NEW-BAR movement it creates**, so every piece tagged in this system from now on lands in the 2.7% this rule covers.

It guards the path this system uses, measured against the path the legacy left — and 246 of 246 agreeing to the milligram is what makes it safe to assert at zero.

37 of 37 rules hold

Six rules in three entries, and what they have in common

439	no bill counts a piece twice	the money doubled
439	no voucher posts a line twice	the money doubled
440	a bill names the party its voucher charges	the wrong person owes
440	a purchase names the party its voucher pays	the wrong person is owed
441	a sold line is the purity the tag says	the wrong metal described
441	one instalment number, one payment	a customer credited twice
442	a return names the party its voucher credits	the wrong person refunded
442	a movement states the tag's weight	the trail contradicts the shelf

Every one is a failure where **each figure is individually correct and the relationship between them is wrong**. None would be caught by a total, a balance or a reconciliation, because nothing is out by a rupee.

That is the shape worth looking for, and this session found eight of them by asking one question repeatedly: *what could be wrong here that no arithmetic would show?*

The data is untouched

Both falsifications ran against the live table and both were restored from the record that decides — the tag for the weight, the voucher for the party. Verified after: 9,031 movements, 367 sale lines, 224 bills, shelf ₹14,23,61,188.20.

443. A barcode is not a piece — it is a number the shop lends out

Chasing defect 442's question along a new axis: **chronology**. Can a piece move before it exists?

movements whose tag records a created_date	7,725	
dated BEFORE that date	3,326	– 43%

Forty-three percent looked like a serious finding for about ten minutes. It is not one, and what it actually is matters more.

NEW-BAR	2,114 of 5,488 predate their tag, by 131 days on average
OP-STK	1,176, by 597 days
SALE-GST	25, by 493 days

A tag's own *creation* movement dated before the tag was created is a contradiction on its face. The worst are 46225943C , 46225943D , 86883721D , 11157159 — and one barcode, 74472468 , carries two NEW-BAR movements thirteen days apart.

The shop re-issues barcodes, and the trail remembers every life

barcodes with more than one NEW-BAR movement	2,062
of those, on DIFFERENT days	2,060
same day (the doubling of defect 375)	2
distinct barcodes in the trail	3,314
tags	2,816

Two thirds of the barcodes in the trail have been tagged more than once, on different days, over years. A barcode is retired and re-issued; the movement log is keyed by that number and accumulates every piece that ever wore it.

So `created_date` preceding a movement is not an error — it is the current tag's date sitting above an older piece's history. The 3,326 are explained, and the rule I was about to write would have been wrong.

What it costs, and where

Any report that joins movements to tags by barcode reads one number's whole history as one piece's.

BackDatedSubgroupStock (defect 386) does exactly that: it rolls the shelf back by finding each tag's last sale-or-return movement up to a chosen date.

```
sale or return movements predating their tag's created_date    28
barcodes involved                                              27
all of them on tags still in stock                             27
```

Twenty-seven of 2,575 tags could be put on the wrong side of a past date. That is the whole exposure, and it is now written into that report's header rather than left for somebody to discover.

Not corrected, because it cannot be from this data. Nothing distinguishes *"sold in 2024 while a different piece wore this number"* from *"sold in 2024"*. A report that guessed would be worse than one that says what it cannot see.

And one clean result

```
sale lines whose bill predates the tag's creation    0 of 232
```

Not one piece was billed before it was tagged. The chronology that would actually be alarming holds perfectly.

444. The same exposure, in the screen most likely to be trusted with it

Defect 443 measured barcode re-use and wrote the limit into one report. The obvious next question is which others carry it.

```
joins the movement trail to a tag by barcode
backdatestock.go      annotated at defect 443
movements.go          StockHistory – NOT annotated

joins a document line to a tag by barcode
mrpsales.go           sale lines
ordertobarcodes.go     order lines
```

StockHistory is the one that matters

Its own header calls it **"the only multi-year history that survived"** — 1 April 2024 to 19 August 2026, against vouchers covering a single year. It is the screen somebody opens to trace one barcode through time, which is exactly the thing a re-issued number cannot support.

It labels each movement with the piece wearing that barcode **now**, through a lateral join. So:

movements shown with a description belonging to a later piece	3,326
---	-------

A 2024 movement carrying a 2026 piece's name, on a screen headed *"what happened to a tag"*. The report already flags `tag_gone` when no tag holds the barcode at all — it knows the two can come apart — and had nothing to say when they come apart the other way.

Now annotated: **this is the history of a NUMBER, and a number can have been worn by more than one piece**. Nothing can say which piece a past movement belonged to, because the trail does not record it.

The two document joins are clean, and measured rather than assumed

sale lines whose bill predates the tag's creation	0 of 232
order lines whose tag was created after the order	31 of 55

Zero on sales, which defect 443 established: a piece is billed after it is tagged, always.

Thirty-one on orders, and **that is the correct answer, not an exposure**. A customer orders a piece, the shop makes it, the tag is created weeks later and its barcode written back onto the order line.

`ordertobarcode.go` exists to show exactly that pairing. Twenty-three were made after the due date as well, which is lateness, not a data fault.

The difference between the two cases is worth stating: a movement's barcode is a number the trail happened to see, while a document line's barcode was written down when the piece was in somebody's hand. Only the first can drift.

Where this leaves the pair

443	BackDatedSubgroupStock	27 of 2,575 tags could sit on the wrong side of a date
444	StockHistory	3,326 movements labelled with a later piece

Both stated in the code, neither corrected, because the data does not contain the answer. Two reports that were quietly confident are now explicitly bounded.

445. Barcode re-use is not a fault to fix — and the tag table keeps only the last life

Defects 443 and 444 described what barcode re-use costs two reports and deliberately did not correct it. That leaves a design question open: **should the system stop allowing it?**

barcodes last re-tagged in 2024	9
in 2025	1,029
in 2026	1,022

About a thousand a year. This is not legacy residue; it is how the shop works. A number is retired when its piece is sold and issued again to the next one. Blocking it would break a daily practice to protect a report.

And the system already draws the line in the right place:

```
barcodeInUse: SELECT count(*) FROM mcj_stock_items
                WHERE btrim(barcode) = btrim($2) AND in_stock
```

A barcode **on the shelf** cannot be issued again; one whose piece has gone can be. That is exactly the shop's own rule, and `TestTag_RefusesABarcodeAlreadyOnTheShelf` already holds it down.

So the answer is no. 443 and 444 chose correctly: describe the consequence, do not prevent the practice.

Why the history can never be disambiguated

tag rows	2,816	
distinct barcodes among them	2,815	(the one repeat is blocker B8)
distinct barcodes in the trail	3,314	

The tag table keeps one row per barcode — the current life. When a number is re-issued the previous tag's row is not retained beside it; there is one row and it describes whatever wears that number now.

That is what makes defect 444's 3,326 mislabelled movements permanent rather than merely unfixed. The trail knows 3,314 numbers and the tag table can describe 2,815 pieces, and the 499 difference (defect 397) plus every retired life are gone. **There is no record to join to**, so no report could do better however it was written.

the one barcode on two tag rows B8, known and open

What was measured and not built

No guard, no migration, no screen. Three questions asked and answered:

is re-use current?	yes, ~1,000 a year
does the system limit it right?	yes, and it is tested
can the history be recovered?	no, the previous life is not kept

An entry that builds nothing is worth writing when the alternative was building the wrong thing — and "stop the shop re-using barcodes" would have been exactly that.

446. Four hundred and forty-five entries, two hundred and ninety-nine of them cited

This register is now large enough to be a reference rather than a diary. Sixty entries were added today and they lean on each other heavily — 443 leads to 444 leads to 445; 439 through 442 are one argument in four parts.

A citation is only worth anything if it lands.

defect entries on file	445
distinct numbers cited elsewhere	299
citations pointing at nothing	0

Checked across every `.go`, `.templ`, `.py`, `.sh` and `.sql` in the tree plus all four documents. Not one comment, loader or entry sends a reader to a defect nobody wrote.

`check_blocker_citations.py` has asked this question of blockers since defect 318 — whether live text cites a blocker that has been closed. It has never asked it of the register, which is twenty times the size.

Built: the same question, of defects, in the same tool.

```
x docs/screens.md cites defect <a number nobody wrote>, which is not
  in the register
```

(The message is quoted with the number removed on purpose. Writing a real example of a dangling citation *into* the register makes it a dangling citation: the check flagged this entry the moment it carried one, which is the honest limitation — **it cannot tell a quoted example from a real reference.**)

It caught the comment that introduced it

The check's first run reported **one** dangling citation before I had broken anything on purpose: the docstring of the function itself, which says "*Defect 446*" — this entry, not yet written.

That is the correct answer. A comment referring forward to an entry that does not exist yet is exactly the state the check exists to catch, and it caught its own author within a minute of being written.

And a bug in the check, caught by falsifying it

The first version appended its findings to the list the blocker half already uses. That list holds four-part tuples and mine were strings, so the reporting loop would have crashed — **on the first real finding, and never before it.** It passed cleanly while there was nothing to report.

```
findings += dangling_defect_citations() # crashes when it finds something
```

Defect 359's lesson, met again from the other side: a check that has only ever been run against a clean tree has not been shown to work. This one now has a list of its own, its own reporting, and a falsification that ran end to end — insert a bad citation, watch it exit 1, remove it, watch it pass.

447. I emptied a document with a one-line idiom, and got it back by luck

While falsifying defect 446's new check I appended a line to `docs/screens.md`, then removed it with:

```
open(p, 'w').write(open(p).read().replace(marker, ''))
```

Python opens the left-hand file before it evaluates the argument. `open(p, 'w')` truncates on open, so `open(p).read()` then read an empty file and wrote it back. A 1,998-line document, 105 KB, gone in one statement that looks like a substitution.

It was caught within the minute — not by noticing, but by `status_figures.py`:

```
Screens written up in `screens.md`: STATUS.md says 69, taken as 0
```

The figures check compares STATUS.md against the filesystem, and a document with no `## Screens N` headings left in it reads as zero written up. **The check that defect 425 extended to guard against drift caught a deletion instead**, which is not what it was for and is the second time today a check has earned its keep on a failure nobody designed it for.

Correction, made at defect 448: it was not only luck

This IS a git repository — the environment banner says otherwise and it is wrong. `docs/screens.md` is tracked, and `git checkout` would have recovered it.

It would have recovered the version committed on **22 August**, three days stale: without defect 425's headline correction, without its area-table fix, without anything else that has touched the file since. A safety net that returns the wrong version is better than nothing and is not the same as a backup.

The scratchpad copy was two hours old and correct. That part was still luck.

Recovered from a backup made for something else

```
scratchpad/sc.bak    105,584 bytes, taken at 08:33
```

I had copied `screens.md` two hours earlier to falsify defect 425 — break the headline, watch the check fail, restore. That copy was made to prove a check worked, not to protect the file, and it happened to be the corrected version rather than a stale one.

Restored and verified identical, 1,998 lines, 41 screen headings, the 126-of-193 headline and the area table intact. `status_figures.py` passes, which is a stronger statement than "the file is back": it says the file agrees with the menu CSV and the navigation registry, so it is the right version and not merely a version.

The recovery was luck. Nothing in this session's method would have saved that file if the falsification at 425 had happened to use a different document.

What changes

That idiom is never safe and I have used it repeatedly today — every `python3 - <<'PY'` block that rewrites a file in place has the same shape, and they only worked because they read into a variable first:

```
s = open(p).read()      # read completes
...
open(p, 'w').write(s)   # then truncate and write
```

That is the safe form and it is what the rest of this session used. The one-liner collapses the two steps and inverts their order.

Written down rather than quietly corrected because the failure mode is invisible: the statement is shorter, reads naturally, and destroys the file every time.

448. Three days of work exists in one place, and it is the same laptop as B18

Defect 447 said the recovery of `screens.md` was luck. Checking that claim turned up something larger: **the environment banner says this is not a git repository and it is wrong.**

```
git rev-parse --show-toplevel  /Users/rahul/nimbus/wingoldv2
last commit                    22 August 2026
```

So there was a safety net, three days stale. That corrects 447, and the correction is now written into it.

The larger thing is what the net is holding.

```
tracked files modified      104
insertions                  35,672
deletions                   1,417
untracked files            270
  reports                   120
  migrations                45
  controllers               37
  legacy loaders            28
  tools                     13
```

Every migration this session wrote is untracked — mcj-1787416000 through mcj-1787422000, the credit-note table, the order rate card, the bill-number links, the prior uncleared items, the raw-stock pieces column, and the drop of mcj_financial_years. So are all four new loaders, reorder.go, and check_cascade_guards.py.

Three days of work. Sixty-three defect entries. Five tables, four loaders, eleven checks, and the whole of today.

Why this is blocker B18 wearing different clothes

B18 sits at the top of the ask list: *the only copy of the legacy data is on one laptop*. The same sentence is now true of the work done to it. A failed disk takes the dump **and** the rebuild, and the rebuild is the part that cannot be downloaded again from anywhere.

the dump	irreplaceable, one copy, on this laptop	(B18)
the rebuild	three days uncommitted, on the same laptop	(this)

Not committed, and that is deliberate

Committing is not mine to do. The standing instruction is to commit when asked, and I have not been asked — a session that ends with sixty-three entries and an unrequested commit is a session that decided something it was not asked to decide.

Added to the ask list, beside B18, because it is the same risk and the same one-line fix, and because the ask list is where things that need the shop's hand belong (defect 421).

```
git add -A && git commit
```

is thirty seconds. The measurement above is what it is protecting.

449. Auditing the session's own diff, and the file that looked worst was fine

Defect 448 established there is a git baseline from 22 August. That makes something possible that was not before: **reading the session's own diff and accounting for every file in it.**

104 tracked files are modified. One of them should not be:

```
modules/mcj/presentation/navigation/legacy_menu.csv    196 +, 190 -
```

That file is the capture of the legacy menu. It is the source of the 193-screen count, of `check_todo_screens.py`, and of the area tables defect 425 corrected. It is evidence, not something to edit — and every one of its 190 lines had changed.

It is two changes, and neither is a loss

```
rows removed since the commit    0
rows added                        6
committed file carriage returns  0
working copy carriage returns    196
```

Every line differs because the line endings do. The working copy is CRLF and the committed version is LF, so a whole-file diff reports 386 changed lines and means six.

The six are all REPORTS entries:

```
Karigar Repair Report  Issue From User
Karigar Repair Report  Karigar Issue/Receive Register
Karigar Repair Report  Shape Wise Issue Receive Repair
Manual Stock Tally     Packet Wise Stock Tally
Manual Stock Tally     Rawstock Tally
Manual Stock Tally     Subgroup Stock Tally Back Date
```

These are defect 175's recoveries — screens found in the database's own `sub_menu` table that the captured HTML menu had missed. Three of them were tested for the first time this session: the packet stock summary (defect 391, which turned out to be counting sold stones), the raw stock tally (383), and the back-dated subgroup stock (386).

Nothing was lost and nothing was invented. The menu grew by six screens the shop's own database proved it had, which is why the count reads 193 and not 187.

What the audit was worth

One file out of 104 looked wrong, and looking took ten minutes to answer completely. The answer is reassuring, and the reason to write it down is that the next person to read this diff will stop at the same line — a 386-line change to the file that defines what "built 126 of 193" means is exactly where a reader should stop.

The line-ending difference is left alone. Normalising it would produce a second whole-file diff on top of the first and make the six real rows harder to find, not easier.

450. Five files I could not remember editing, and the register remembered for me

Defect 449 accounted for the one modified file that looked wrong. The rest of the audit is the other 103, and the useful cut is by date:

```
last touched 22 Aug    7
              23 Aug   38
              24 Aug   28
              25 Aug   31  - this session
```

Of the 31 from today, 26 are ones I can name: the docs, the locale, `links.go` and `reports.go` (defects 401 and 423), the sale screen (404, 415, 434), `gstr.go` (410, 411, 428), `designs.go` (430), `movements.go` (444), the reconciliation (402), `orders.go` (398), three loaders (398, 407, 411), and five `_templ.go` files regenerated by `templ generate`.

Five I could not place at all.

```
tools/legacy/load_invoices.sh    95 +,  20 -
tools/legacy/load_schemes.sh     11 +,   3 -
modules/mcj/.../checks.go        126 +,   4 -
modules/mcj/.../conversions.go    65 +,   2 -
modules/mcj/.../schemes.go        1 +,    1 -
```

Two are shell scripts, so no formatter of mine rewrote them. And the changes are not cosmetic:

```
- LEFT JOIN mcj_parties p ON p.id = e.party_id
+ LEFT JOIN mcj_ledgers p ON p.id = e.ledger_id
```

That is a model change — scheme enrolments moving from the 40,519-row address book to the 785-row chart of accounts — sitting in files with today's date and no memory attached to them.

The register answered it

```
docs/data-defects.md:6109  the three loaders that set `ledger_id` in their own ...
docs/data-defects.md:19325 load_schemes.sh    mcj_scheme_enrolments  ledger_id IS NULL

mcj_scheme_enrolments has ledger_id and no party_id
```

Logged, guarded, and matching the live schema. The work was done earlier today and written down at the time; only my recollection of it is missing.

That is what the register is for. Sixty-five entries went in today and the value of the ones written hours ago is not that they explain the code — the code explains itself — but that they let somebody with no memory of a change confirm it was deliberate.

The audit is closed

104 modified files: one explained in detail (449), 26 recognised, five reconstructed from the register, and the remaining 72 last touched before this session and belonging to work already logged on their own dates.

Nothing in the diff is unaccounted for. That is worth establishing **before** the commit defect 448 asks for, not after — a commit is where an unexplained change stops being visible.

451. The commit defect 448 asks for would have swept in 26 MB of binaries

Defect 450 audited the 104 modified files and concluded nothing in the diff was unaccounted for. That claim was true and incomplete: `git add -A` **commits the untracked files too, and there were 270 of them.**

```
go      166
sql     45
sh      30
py      11
other   3
```

The three are not source.

```
reconcile    13,114,658 bytes  Mach-O 64-bit arm64 executable
seed-admin   13,249,506 bytes  Mach-O 64-bit arm64 executable
tools/__pycache__/          Python bytecode
```

Twenty-six megabytes of compiled binaries, left in the working directory by `go build` naming its output after the package. And `.gitignore` was four lines long:

```
.env
.env.local
*.log
/tmp/
```

Nothing covering build output. So the thirty-second commit at the top of the ask list would have put two Mach-O executables into the history permanently — a binary cannot be removed from git without rewriting every commit after it, which is a far worse afternoon than this was a minute.

Fixed by extending `.gitignore`, which commits nothing and prevents the mistake:

```
/seed-admin /reconcile /wingold /wingold-cli /specgen /legacy-extract
__pycache__/ *.pyc

untracked files: 270 -> 267, and all 267 are source
```

Named individually rather than by a pattern. A bare `reconcile` in a `.gitignore` matches a *directory* of that name too, and `reconcile.go`, `reconcile.templ` and `reconcile_templ.go` all exist already — a pattern that swallowed a future `reconcile/` package would be a subtler bug than the one being fixed. The leading slash pins each to the repository root.

Checked afterwards that no tracked file became ignored: `git ls-files -i -c --exclude-standard` returns nothing.

The shape of this one

449 audited what changed. 450 audited who changed it. Neither asked what would happen when the change is *committed*, which is the only step that makes any of it permanent.

Three entries to audit one commit that has not happened. That is the right ratio when the commit is three days of work and the alternative to checking is finding out afterwards.

452. A check that was right about the code and wrong about its own reach

`check_failed_saves.py` guards a class that has bitten three times — defects 206, 319 and 320 — where a handler renders the reason for a failed save without setting a status, `atomic` sees a 2xx, and whatever the save had already written is committed.

It passes. It has always passed. **It was watching four branches out of thirteen.**

```
SAVE = if err := c.(save|apply|record|reserve|cancel|post)...(
```

Six of those six verbs are guesses about naming; two are used. The controllers hold seven more calls whose failure leaves a write possibly done:

<code>writeSaleDetail</code>	<code>sale.go</code>	– edited today, defects 415 and 434
<code>settle</code>	<code>sale.go</code>	
<code>writeCreditNote</code>	<code>creditnote.go</code>	
<code>writePurchaseInvoice</code>	<code>creditnote.go</code>	
<code>writeOldGoldLines</code>	<code>oldgold.go</code>	
<code>addRawStock</code>	<code>rawstock.go</code>	
<code>removeRawStock</code>	<code>rawstock.go</code>	

None of them is rendering a failure on a 200. The check was right about the code. It was wrong about how much of the code it was looking at, and that is the worse of the two errors: a passing check reads as coverage.

```
branches watched    4 -> 13
```

Falsified by breaking one of the newly-covered paths

Making `writeSaleDetail`'s failure branch render without a status:

```
renders: templ.Handler(voucherpage.Sale(props)).ServeHTTP
atomic commits on 2xx, so whatever that save wrote is kept
1 screen(s) tell the wrapper a failed save succeeded
```

Before the widening that would have passed silently — the exact shape of defects 206, 319 and 320, on a function I had edited twice today.

Restored, and the tree builds.

The pattern this belongs to

```
408  a rule three times repaired became a check
413  a check that said "no screen" and never looked at screens
446  a check that asked its question of blockers and not of the register
452  a check that watched four of the thirteen places its class can occur
```

Four checks written to catch a recurring fault, each one narrower than the sentence describing it. **A check is a claim, and its reach is part of the claim** — "no screen renders a failed save on a 200" is a different statement from "no screen whose save is spelled `save` or `apply` does".

The docstring said so honestly: *"what it cannot see: a save spelled some other way."* Nobody had measured how many other ways there were.

453. Auditing my own check, written today, for the fault I had just named

Defect 452 named a class: **a check whose claim is broader than its search**. Four instances, one of them mine. So I audited the rest of the eleven, and the first one I looked at was also mine, written this morning at defect 388.

```
check_gofmt.sh: gofmt -l modules/ tools/ pkg/ cmd/
its message:     "every Go file is gofmt-clean"
```

Four directories named by hand. Two are not in the list:

```
e2e/smoke_test.go      NOT gofmt-clean
internal/assets/assets.go  clean, and also unwatched
```

The check has been passing all day while a Go file in this repository was not formatted. Not a serious file and not a serious difference — one import line — but the check's sentence was false and its output was green, which is the combination 452 was written about.

Fixed by searching rather than listing

```
DIRTY=$(find . -name '*.go' -not -path './.git/*' -print0 | xargs -0 gofmt -l)
```

A directory list goes stale the day somebody adds a directory. `find` cannot.

Formatted the one file — checking the diff first, because defect 388's whole point is that **gofmt rewrites doc comments** and can turn `' '` into a smart quote. The diff was one import line and no quote changes, so it was safe.

```
FAIL: gofmt would rewrite these files:
./e2e/smoke_test.go
```

Falsified afterwards by appending a badly formatted function, which it caught — in the directory it could not see an hour ago.

Five now, and the shape is consistent

```
408  a rule three times repaired became a check
413  a check that said "no screen" and never looked at screens
446  a check that asked its question of blockers, not the register
452  a check watching four of the thirteen places its class occurs
453  a check saying "every Go file" while naming four directories
```

Every one was written carefully by somebody who then described it in a sentence slightly larger than the code. **The sentence is what the next reader believes.**

Two of the five are mine and both were written today. That is not carelessness accumulating — it is what happens when a check is written to catch a specific thing that just happened and then summarised in general terms.

454. Two reports that net credit notes were outside the check that guards footers

Continuing defect 453's audit of the eleven checks for reach. Four remained. Three are clean, and each derives its targets rather than listing them:

```
check_screens_render  discovers 39 Path constants; 35 navigation entries
                      resolve to a path and 0 are missed
check_empty_columns   `table_name LIKE 'mcj\_%'` over information_schema,
                      which is all 83 mcj_ tables
check_todo_screens    states its own covering: 126 built + 67 stood in = 193
```

The fourth is not.

```
check_report_footers  scans for  Key: "... "      found 82
report specs that exist                                84

missed: sales-returns, purchase-returns
```

Why those two and no others

`ReturnRegisters()` builds both from a struct:

```
key: "sales-returns",  title: "Sales Returns"
key: "purchase-returns", title: "Purchase Returns"
...
Key:                  k.key,
```

The literal is spelled with a lower-case `key`: a few lines above the spec, and the spec's own `Key`: is a variable. A scan for `Key:\s*` cannot see either.

They are the two reports where a doubled footer would matter most. Defects 332 and 334 rebuilt them to net credit notes against sales and debit notes against purchases — the halves that carry negated figures. A footer that added a summary row on top of negated rows is precisely defect 143's fault in the place it would be hardest to notice.

Neither is doubling. The check was right about the code and blind to two of the places its sentence covers.

Fixed: both spellings, 82 -> 84.

The check that caught me writing this

Adding the comment above made `check_blocker_citations.py` fail:

```
docs/... cites defect 454, which is not in the register
```

Defect 446 built that. It has now caught a forward reference twice — once to the entry introducing it, and once here, to the entry recording the fix it was guarding. A check that fires on its own author twice in one session is a check that works.

The audit is finished

```
eleven checks
  narrower than their claim, now widened    4  (413, 446, 452, 453, 454 – five fixes
across four tools)
  reach measured and correct                6
  deliberately loose and documented         1  (cascade guards, defect 408)
```

Two of the fixes were checks I wrote today. The lesson from 453 stands and is now measured: **a check written to catch one incident describes itself in general terms, and the general terms are what everybody afterwards believes.**

455. The two reports nobody had ever fetched

Defect 454 fixed a blind spot in `check_report_footers` and I did not think to ask whether the same pattern appeared anywhere else. The full suite answered it without being asked:

```
39 screen(s) and 82 report(s) render ...
```

Eighty-two, against the eighty-four specs 454 had just counted.

`check_screens_render.declared_reports()` uses the identical regex:

```
re.finditer(r'Key:\s*"([^\"]+)"', src)
```

So `sales-returns` and `purchase-returns` — built from a struct, their spec reading `Key: k.key` — **have never been fetched by the check that proves a screen works at all.**

This is worse than 454's version. A footer check that misses two reports fails to notice one class of arithmetic error. **A render check that misses two reports would not notice them returning 500 for a year.** Its sentence is *every screen and every report*, and it had been fetching all but two since it was written.

They were fine

```
39 screen(s) and 84 report(s) render, including over a range that
matches nothing, and 84 of them show real data
```

Both render, both over an empty date range, both with real data. Nothing was broken. **The exposure was that nobody could have known.**

Two tools, one regex, two entries

454	check_report_footers	82 of 84	fixed
455	check_screens_render	82 of 84	fixed

Same pattern copied into two tools, and finding it in the first did not make me look in the second — the full suite did, by printing a number I could compare against one I had measured an hour earlier.

That is the argument for a check printing **counts** rather than a verdict. "No report doubles a total" tells you nothing about how many it looked at; "82 report(s) render" is a number that can disagree with another number. Defect 279 made that change to the footer check for its own reasons, and it is the reason this was catchable at all.

The citation check, a third time

```
docs/... cites defect 455, which is not in the register
```

Defect 446's work has now fired on its own author three times in one session, each time on a comment written to explain a fix. It is the most consistently useful thing built today.

456. The tool behind three of today's builds, audited last and found sound

Defect 455 found the same narrow regex in a second tool. The obvious question is how many tools derive a list by scanning source, and which of them I have relied on today without checking.

status_figures.py	9 regexes	audited, defect 425
check_blocker_citations.py	7	audited and extended, defect 446
check_unwritten_columns.py	5	NOT AUDITED
check_todo_screens.py	2	audited, defect 454
check_invariants.py	1	explicit SQL, no scanning
check_report_footers.py		was narrow, fixed at 454
check_screens_render.py		was narrow, fixed at 455

`check_unwritten_columns.py` is the one that matters most and the one I had not looked at. **Defects 405, 406 and 418 are built on its output** — the certification mode, the `lot_no` question now on the ask list, and the size and remarks fields added to the tag screen. If its scan were narrow, that list of 52 columns was short and those three entries drew conclusions from a partial picture.

```
GO_ROOTS = ["modules", "cmd"]
```

```
Go files anywhere in the tree that write an mcj_ table: 16
of those, under modules/ or cmd/:                      16
missed:                                                  0
```

Sixteen writers, from `cmd/seed-admin/main.go` through the fourteen controllers and `voucher_repository.go`. Every one is inside the two roots it walks.

It also handles the case a regex over SQL would miss: columns written through the CRUD layer, matched separately as `crud.NewIntField("...")` and as JSON-ish `"name":` keys. That is the mechanism defect 239 recorded — fifteen tables writing through a `Schema()` no grep could see — and this tool was written knowing it.

Sound. The three entries stand.

What the whole audit came to

tools that derive a list	7	
found narrower than claimed	4	(413, 446, 452, 453, 454, 455 – six fixes)
measured and correct	3	

Six fixes across four tools, and two of the four were tools written today. Not one of the six was a wrong answer about the code: every check was telling the truth about what it looked at. **All six were wrong about how much that was.**

The one that finally stopped the chain was `check_unwritten_columns.py`, and the reason it is sound is visible in its own source: it walks roots with `os.walk` rather than naming directories, and it looks for two spellings of a write rather than one. Both are the fixes 453 and 455 had to make by hand.

457. The ask list re-measured, and one row that invited a wrong subtraction

Defect 421 rebuilt the ask list into three things to do first and four questions with money on them. It ordered the rows; it did not re-verify the figures in them, several of which were written days ago.

Every one holds.

B24	889 tags on the shelf with a photograph	₹10,73,39,445.03
B18	customer records in one place	40,519
410	IGST on bills whose place of supply is local	₹1,56,108.86
397	sold by barcode against tags that never existed	₹9,26,698.00
346	the shelf revalued at a different basis	₹3,94,01,031.93
233	scheme instalments collected	₹87,90,900.00
B23	owed on matured enrolments	₹77,37,300.00

B24 recomputed from `legacy.barcode_image` joined to the shelf: 927 tags carry a photograph, 889 of them are still in stock, and those 889 are worth ₹10,73,39,445.03 — the figure the blocker has carried, to the rupee.

The row that could be read wrongly

₹87,90,900 collected · ₹77,37,300 owed
--

Both true, and side by side they invite a subtraction that means nothing. ₹87,90,900 is **every instalment ever collected**, across 2,144 payments and 239 enrolments. ₹77,37,300 is the maturity value of the **173 enrolments that are both fully paid and matured** — and those 173 paid in **₹71,01,000**, not ₹87,90,900.

173 matured and fully paid	
paid in	71,01,000.00
maturity value	77,37,300.00
the gap	6,36,300.00

the scheme's uplift, and the shop's liability

Reading the original pair as a difference gives "₹10.5 lakh spare", which is not a quantity that exists. **The row now says which figure belongs to which population**, and names the ₹6,36,300 as what the uplift actually costs.

Why this is worth an entry

Defect 421 made the list usable by ranking it. Ranking a figure does not check it, and the summary was written from the entries rather than from the data. Every number survived the re-measurement — but the one thing that needed changing was not a wrong number. **It was two right numbers placed where a reader would subtract them.**

That is the failure mode of a summary: not error, but adjacency.

458. The same trap, one section up: an asset and a liability joined by a comma

457 found two right numbers placed where a reader would subtract them, and said the failure mode of a summary is not error but adjacency. That is a claim about a class, so the class was swept rather than left as a moral.

STATUS.md holds 987 rupee figures on 218 lines that carry two or more. Two sweeps over them:

```
stated arithmetic ("leaving", "short by", "of which", "explains")
single-claim lines      5    all compute
multi-clause bullets   33    figures from unrelated sentences

bare adjacency: two figures, short line, no relation stated
lines                9    8 sound, 1 not
```

The eight sound ones name their relation in words — "0.5% apart", "tagged in 2025 and in 2024", "taxable / input tax". The ninth did not.

The line

What each customer owes, and what the shop is holding for them
 – ₹73,96,553.00 owed in, ₹27,17,486.77 held

Both figures are right. Both were recomputed against the live data through the report's **own** group list, `['Customer', 'Other Debtor']`:

10 accounts owe the shop	73,96,553.00
20 accounts the shop owes	27,17,486.77
461 accounts flat	
491 customer accounts	

A comma joins them, and the subtraction it invites gives ₹46,79,066.23 — a "net customer position" that does not exist. The two sets are **disjoint**: no account can be on both sides, because a balance is one sign or the other. One customer's advance cannot settle a different customer's debt. **These are an asset and a liability**, and netting them across 10 accounts and 20 different accounts is the netting-off error, not a shortcut.

The row now says which population each figure belongs to, and that they are opposite sides of the balance sheet.

The screen was already right

`customerbalances.go` puts `owingTotal` in the balance slot and states the other in prose — *"the shop is holding ₹27,17,486.77 of customers' money"* — with separate counters that never meet. **The defect was in the summary of the screen, not the screen.** A summary compresses, and compression is where populations get dropped.

A measurement error inside this entry

The first recomputation returned **18 accounts and ₹21,41,852.00**, and for a moment that read as the figure being stale. It was not. `group_name LIKE '%ustomer%' OR '%Sundry Debtor%'` is not the list the report passes; with `['Customer', 'Other Debtor']` both figures reproduce to the rupee. **Seventh measurement error recorded** — and the rule that caught it is the one from defect 409: read the query the screen actually runs before believing a disagreement with it.

What is now guarded

`tools/check_stated_arithmetic.py` — **check 12**. Every single-claim line that asserts a relation between figures must have that relation resolve.

Adjacency itself is **not** mechanically checkable: whether two figures invite a wrong subtraction is a judgement about populations, and a checker that guessed would cry wolf on all nine lines to catch the one. The checkable half is guarded; the rest stays a reading. The limit was measured — at 300 characters five lines qualify and all five compute, at 400 the first five false positives appear — and **the check prints the 33 lines it skips**, because a checker that hides its own blind spot reads as coverage it does not have.

459. A correction that never reached the screens it corrects, and ₹1.35 crore missing from the ask list

`check_todo_screens.py` proves every one of the 193 menu entries is either built or has a stand-in. It has never once asked whether a stand-in's reason is **true** — it checks that a justification exists, not that it holds. Sixty-seven justifications, none of them tested. Three claims carry 61:

```
34  "no karigar master and no issue-or-receive table"
21  "no table for this exists among the 149"
6   "only two bill types were ever raised"
```

The first was tested against the capture, and the walk went somewhere unplanned.

The claim tested, and then the claim about the claim

legacy does hold karigar tables: `karigar_gold_stock` (2 rows) and `karigar_wip_bal` (1). Ledger 11699 is **MA TARA JEWELLERS**, `ac_grp` **Karigar**, `ledger_type` **KR** — and it is the **only** Karigar ledger in 40,519 accounts. The two rows issue **869.850 g of 995 fine** on 16 Sep 2025, bills C4/00001 and C4/00002, at ₹11,417.

at the rate on the issue	99,31,077.45
at today's 995 rate of 15,482	1,34,67,017.70

Every `receive_from_karigar` table is empty, `karigar_wip_bal` is zero at every purity, the C4 bills appear in **none** of the 20 other bill-numbered tables, and MA TARA JEWELLERS carries **₹0.00 opening and not one posting**.

All of which defect 164 established months ago, to the rupee. This walk reproduced it independently and landed on the same two figures — confirmation, not discovery. What was new is what 164 failed to change.

The correction did not propagate

Defect 164 is titled *"the record dismissed it as an empty table"*. Its whole argument is that **those two rows are not an empty table**.

The stand-in shown on all 34 manufacturing screens opened:

The capture holds no karigar master and no issue-or-receive table.

That is the dismissal 164 exists to correct, still on screen, 34 times. It did go on to name the two rows — so it was not concealing them — but a reader meets the denial first and the rows second, and `todo_screens.go` **cites no defect number anywhere in the file**, so nothing points at the entry holding the ₹1.35 crore.

Corrected in all 34: the shop has **one** karigar, named, and those two rows are an issue of 869.850 g that never came back — **with defect 164 cited**.

The part that actually matters

The ₹1.35 crore was not on the ask list. Twenty-nine rows, including *"Where is the raw metal now?"* for defect 78 / 133 / 351 — and nothing for the gold that left the building. The goal names STATUS.md as the single list of what needs Rahul's answer, and the largest single quantity of metal unaccounted for in the capture was not in it.

Added, next to the raw-metal question it belongs beside. The question is put without an inference either way: eleven months on this is **either** metal still out with the goldsmith **or** the residue of a module the shop trialled and dropped — C4/00001 and C4/00002 being the first two numbers of a series that never reached a third. Defect 78 guessed at exactly this kind of gap and defect 133 had to correct it. **Only Rahul can say.**

What this says about the check

A stand-in reason is evidence, and evidence goes stale. `check_todo_screens.py` counts justifications; it cannot read them. **33 of the 67 remain untested** — the 21 "no table exists" and the 6 bill-type claims are next.

460. Nineteen of twenty-one held; the two that did not said "none at all" about a table with ten rows

Finishing 459's sweep. Sixty-seven stand-in reasons, of which 459 tested the 34 karigar ones. The remaining claims, each put to the database:

6	"only two bill types were ever raised"	HOLDS, exactly
21	"no table for this exists among the 149"	19 hold, 2 do not

The six

`sales_vou` carries `bill_type`, and it holds:

local	221	
central	3	

	224	every bill in the capture, and all of them sale_type SALE-GST

No export, repair, exhibition, SC or CO bill exists anywhere. **Credit Sale, Export Sale, Repair Sale, Exhibition Sale, Purchase Voucher - SC and Purchase Voucher - CO have nothing to show, and the claim is exact** — the 221 and the 3 reconcile to the 224-bill baseline.

The nineteen

No table matching `discount`, `studded`, `approv`, `split`, `broken`, `tds`, `set_rate` or `sub_item` exists in the 149. Two near-misses were checked and both are correctly dismissed:

- `item_merge` (**40 rows**) is a *catalogue* merge — the loser of two merged item codes — not a barcode merge. Already recorded twice, at 940 and 2479.
- `barcode_detail_rawstone` / `_final` are the detail of the raw opening stock, not a studded one.

The two that were wrong

Party Raw Stock and Karigar / Dealer Raw Stock both said:

No table for this exists among the 149 the capture contains.
Not an empty table — none at all.

`barcode_main_rawstone` **exists and holds 10 rows**. All ten are `type='sh'` — the shop's own metal, the 27,326.609 g of defect 78. What is missing is not the table but the `co` and `kr` rows, of which there are none.

The distinction is the whole point of the sentence. *"No table at all"* says the old system never had the feature; *"a table with no rows of this type"* says it had the feature and the shop never used it. **The second is true**, and it is what this project's own documentation already said — defect 78 closes with *"raw stock exists only as*

`type='sh'` — *there are no co or kr rows, so Party Raw Stock and Karigar/Dealer Raw Stock have nothing*", which is exactly right.

So the docs had it correct and the screens did not, **the same shape as 459**: a finding recorded in `data-defects.md` and never carried into the text a user actually reads. Twice now in two entries, from two unrelated sweeps.

Both corrected, and both now name `barcode_main_rawstone`, its ten rows, and the defect that measured them.

The sweep is closed

All 67 stand-in reasons have now been tested against the data: **64 held, 3 did not** — the 34 karigar ones opened with a dismissal that defect 164 had already overturned (459), and these two claimed a table does not exist when it has ten rows. Nothing in the remaining 64 was found to overstate.

No screen was found that has data behind it and is not built. That is the goal's second clause, and it now rests on 67 tested reasons rather than 67 present ones.

461. Nine hundred and fifty numbers are printed on the screens, and nothing had ever checked one

459 and 460 both found the same shape: something true in `data-defects.md` that never reached the text a user reads. Twice from unrelated sweeps is a pattern, so the whole class was swept — **every absolute or numeric claim in on-screen text**.

The claims that name a table

187 strings in the presentation layer assert *no, never, none, not one*. Only **6** name a table that can be looked up, and all six were put to the database. Four were false alarms — HTML markup and doc comments citing legacy `.php` filenames, not data claims. Two were real, and **both hold**:

- `temp_pl`, cited on the Gross Profit screen as showing a loss of ₹93,72,540.55. `legacy.temp_pl` **does not exist**, which briefly read as a broken citation. It is not: defect 131 records `temp_pl` as one of five `temp_*` scratch tables in the ERP's manifest deliberately **not staged**. The arithmetic reproduces exactly — income ₹2,11,39,143.00 against expenses ₹2,62,33,491.29 + ₹42,66,733.51 + ₹11,458.75 = ₹3,05,11,683.55, a loss of **₹93,72,540.55**. **Eighth measurement near-miss**, caught by the rule from 409: read the source before calling the claim wrong.
- Pricing Pending**, whose comment says *"four tags at the time of writing"* and explains that 952 of the shelf's tags carry no metal rate legitimately, 951 of those having a value, and that reporting on the rate would flag 968. Measured:

no rate	968
of those, making_type 'val'	952
of those, carrying a value	951
total = 0, the report's own	4

All four exact. The phrase *"at the time of writing"* turned out to be modesty rather than a hedge.

The size of what nobody was checking

numeric claims in report doc comments	950
files carrying them	73

Nine hundred and fifty numbers, read by staff as fact, and **not one of them was verified by anything**. A reload moves a baseline under all 73 files at once — defect 420 performed exactly such a reload — and every stale figure would go on reading as true.

Nine were sampled at random and each put to the database independently:

```
2,816 tags · 239 scheme customers · 816 conversion-stone lines
95 band rows over 3 items · 149 bill numbers and 52,52,946.00
52 coin items across 115 days, 3 Apr – 19 Aug 2026
101 items on HSN 711319 · 129 vouchers reading high by 35.30, worst 1.00
4 tags priced at nothing
```

Nine of nine, exact. Including the rounding reconciliation, reproduced to the paisa. The text is in good order; it was simply unguarded.

What is now guarded, and what is not

`tools/check_report_figures.py` — **check 13**. Six whole-population baselines are read from the database and matched against the figures the reports print.

It cannot check all 950 and does not claim to. A subset count — *"692 tags priced by value"* — is true only of its own report's query. **The signature of drift is a figure CLOSE TO but not equal to its baseline:** a subset sits far away (692 against 2,816), while a figure that was right before a reload sits one or two off. So a claim within 5% of a baseline must match exactly, and anything further is left alone.

matching a baseline exactly	13
within the drift band	0
clear subsets, left alone	32

Falsified before being trusted. One comment was edited from 2,816 to 2,815; the check named the file, the line, the claim and the truth, and exited 1. The file was restored and confirmed identical to its backup. Defect 446 was a check that would only have crashed on a real finding — every check since is made to fail on purpose first.

462. One screen disagreed with four others about a bank balance, and both figures were already written down

Auditing the goal's first clause — a real day's trade, end to end. Two gaps found were already on the "What it cannot do" list (**take a scheme instalment**, defect 233; **record a stock conversion**, defect 422), so the survey moved to what the shop did *recently*: the most recent row in every dated legacy table.

Fifty tables, and one date stood out. `voucher_main` runs to **5 December 2026** — three and a half months into the future.

It is a post-dated cheque, and that part was known

```
PMNT 576    5 Dec 2026    1,35,370.00
Ch. No.435590, Dt.05.12.2026 Issued Against Their Dues
  DAYANIDHI TRADE PVT LTD  (Other Loan)      Dr 1,35,370.00
  UNION BANK OF INDIA      (Bank Accounts)   Cr 1,35,370.00
```

Ordinary practice, properly balanced, and already recorded: *"rightly excluded from an as-on-today balance sheet."*

It is not excluded everywhere. Of the six reports that compute a balance as opening plus movement, **five carry no date predicate at all** — `customerbalances`, `supplierbalances`, `ledgermerge`, `customermetal` and `ratio`. Four are harmless here: the one post-dated voucher touches Other Loan and Bank Accounts, none of which those screens cover. (This entry first said four. The fifth, `customermetal.go`, was missed by a grep of mine and found by the check built afterwards — defect 463.)

`ratio.go` covers Bank Accounts.

The contradiction was already on file, in two places

STATUS.md:1113	ratio screen	4,92,97,479.87	owed to Union Bank
STATUS.md:1631	four screens	4,91,62,109.87	"and they all agree"
	difference	1,35,370.00	

Line 1631 records that the trial balance, ledger book, reconciliation and direct SQL agree on `UNION BANK OF INDIA`. Line 1113 quotes the Ratio screen's figure. **They differ by exactly the post-dated cheque**, and both were written down as fact in the same document without the gap being noticed.

`ratio.go`'s own comment said it classified ledgers *"the way the balance sheet does it"*. The balance sheet has four date references. This had none.

The fix, and what it does not change

The subquery now joins `mcj_vouchers` and takes `v.date <= CURRENT_DATE`, which is the idiom `trialbalance.go` already uses. The screen takes no date parameter — it reads as of now — so now is the cut-off.

The ratios themselves do not move. Cash to overdraft is 0.01 before and after; the quick ratio stays 0.07. ₹1.35 lakh on ₹4.93 crore is 0.27%, invisible at two decimals. What was wrong is the **money column printed beside them**, which said ₹4,92,97,479.87 where four other screens said ₹4,91,62,109.87.

That is worth fixing precisely because the ratio hid it. A wrong ratio would have been argued with; a wrong figure next to a right ratio is read straight past. STATUS.md line 1113 now carries the corrected figure.

What is left, deliberately

`customerbalances`, `supplierbalances` and `ledgermerge` still have no date cut-off. **They are correct today by the shape of the data, not by design** — no future-dated voucher currently names a customer or a supplier, and nothing stops one: there is no domain validation against a future date, and the shop

demonstrably post-dates cheques. The voucher register compounds it from the other side, defaulting `to = now`, so a post-dated voucher can be saved and then not appear in the register that would show it.

Left rather than fixed because the right cut-off for those screens is a business question — a customer's post-dated cheque may well belong in "*what they owe*" — and guessing is how defect 78 got its inference wrong. **Added to the ask list.**

463. The check built to replace a measurement found the case the measurement missed

462 was written from a grep. To stop the same gap reopening, the rule was made into `tools/check_balance_date_policy.py` — **check 15**: a report computing opening-plus-movement must carry a date cut-off, or be named with the entry that asks why it has none.

It failed on its first run, naming a screen 462 does not mention: `customermetal.go`.

The grep was wrong, not the check

462's survey counted date references per file with this pattern:

```
v\.date|\.date <=|CURRENT_DATE|date BETWEEN|\$2::date|from.*to
```

The last alternative is the mistake. `from.*to` **matches any line containing "from" before "to"** — SQL and English both, constantly. It scored `customermetal.go` as carrying one date reference when it carries none, and the file was set aside as filtered.

`carryforward.go`, also scored 1, was re-checked and is genuine: `AND ($2::date IS NULL OR v.date <= $2::date)`. So the pattern produced one false positive out of two positives.

Five screens computed a balance with no cut-off, not four. 462 has been corrected in both documents rather than quietly amended — it now names the fifth and says which of its statements was wrong and why.

```
ninth measurement error recorded
```

The rule from defect 409 was that a regex matching `$1` inside `$12` invented a finding. This is its mirror: **a regex too loose to be false hid one**. Both come from writing the pattern and not testing what else it matches.

What customermetal.go does

It reports metal held against a customer — goods, payments out, receipts — and it filters `lg.group_name = 'Customer'` over voucher types `PURC-OG`, `SALE-GST`, `PMNT` and `RCPT`. `PMNT` **is the type of the post-dated cheque**. No customer holds a future-dated voucher today, so nothing is wrong on screen; the exposure is identical to `customerbalances`, and it now carries the same open question rather than a false clean bill.

The check, and its falsification

6 screens compute a balance	
2 carry a date cut-off	ratio (462), carryforward
4 are open questions	customerbalances, supplierbalances, ledgermerge, customermetal

Falsified before being trusted: 462's cut-off was stripped out of `ratio.go` and the check named that file and exited 1; restoring it returned the file identical to its backup and the check to green.

What this says about the last five entries. 459 and 460 found documentation that had not reached the screens. 461 found screen text nobody verified. 462 found two screens disagreeing. This one found the tool built to catch 462 catching something 462 itself had missed — **which is the argument for building the check rather than only fixing the case.** A fix closes one instance. A check reads every instance, including the ones the person who wrote the fix walked past.

464. Every check made to fail on purpose, and one that answered a typo with a stack trace

463 turned on the difference between a check that holds and a check that has never been made to speak. Three checks written this week were falsified as they were built. The other eight never had been. **All of them now have.**

4 the shop's own rules	fires	planted a rule that must fail
5 STATUS.md matches reality	fires	planted a wrong defect count
6 blockers and the ask list	fires	planted a citation of an entry nobody wrote
7 every screen is built	fires	renamed a stand-in's .php
8 no failed save says ok	fires	planted a save branch that renders
9 cascades are put back	fires	stripped a child from load_orders.sh
13 every screen renders	fires	pointed a query at a missing table
14 no report doubles a total	fires	planted a subtotal row in the cash book

Each plant was restored from a backup taken first, and each file confirmed byte identical afterwards. Nothing was left behind.

The defect this found

`check_invariants.py` handles a rule whose SQL **errors** — it prints the rule and carries on. A rule whose SQL **succeeds with the wrong shape** was a different story:

```
ValueError: not enough values to unpack (expected 2, got 1)
```

Every rule must select a count *and* a description of what it counted. Select only the count and the check dies on an unhandled exception, with no rule named. Inside `check_all.sh` that is indistinguishable from a violation: both print FAIL. **A typo in a rule looked exactly like the shop's data being wrong.**

Fixed in the style the tool already used for a failing query — the rule is named and reported as malformed, and the run continues:

```
? PLANTED malformed rule      is malformed: its SQL must select a count
                                AND a description, and it returned '2816'
```

Two false alarms of my own, both from the same habit

Falsifying check 14 took three attempts, and the first two failures were mine:

- **The plant went into a report with no footer.** `customer-balances` renders no `<tfoot>` at all, and the detector needs one. The check was right to say nothing; there was nothing to say.
- **The manual fetch was never logged in.** Checking that page by hand, the login form was posted as `email / password`. The application's fields are `Email / Password`. The request returned **200 with a login page** — no table, no rows — and for several minutes that read as *"Customer Balances renders nothing and check 13 tolerates it"*. Reproducing the tool's own login showed 84 of 84 reports speaking, exactly as claimed.

Tenth measurement error recorded. A 200 that is not the page you asked for is the worst kind of wrong answer, because every test for *"did it work"* passes.

Both mistakes are the same habit as 463's: **a probe built by hand and not itself checked.** The rule that keeps holding is to reproduce the tool's own request rather than approximate it.

What this leaves

```
15 checks defined
11 proven to fire on a planted fault
 4 fire trivially or by construction  build, vet, gofmt, the Go suite
```

The 8 tested here join the 3 falsified as they were written. `check_all.sh` closes with *"Nothing is wrong that this repository can detect"* — a sentence worth only as much as the checks behind it, and now every one of them has been watched saying otherwise.

465. A tool nobody runs, a filter that can never match, and a message that blamed the dates

464 tested every check `check_all.sh` runs. This one asks the other question: **what is in `tools/` that it does not run?**

```
check_empty_columns.py      referenced by nothing
```

Its own header says to run it *"after a data load, or when adding a report that reads a column you have not checked."* The last full reload was defect 420. It had not been run since — a check whose findings reach nobody is not a check.

It over-reported, in the way 463 warned about

Run with `--read`, it named **36 columns**. The first one opened was `mcj_prior_uncleared.particulars`, attributed to `statement.go` and `daybook.go`. **Neither file mentions that table.** They use the word *particulars* for a column of their own, and a match on the column name alone was enough.

The tool knew: every line ended "*(CHECK THE TABLE, not just the name)*". A warning printed 36 times is a warning nobody reads. Attribution now requires the report to name **the table and the column**:

```
36 -> 18      exactly half were the name colliding with itself
```

(`mcj_prior_uncleared.particulars` is genuinely empty, and so is `legacy.brs1.perticular` on all 26 uncleared rows. The loader is faithful and nothing displays the column. No defect there — the attribution was the fault.)

Of the eighteen, one was live

Most are booleans used as filters, where empty means the filter correctly selects nothing.

`mcj_salary_sheets.days_worked` is **defect 218's own column** and its fix holds: still selected, then `_ = days // read from the sheet, not shown`, and no Days field on the screen.

The Barcode Edits screen offers a **Changed** dropdown. Of 156 edits in the capture:

```
item_changed      31
units_changed     119
making_changed     7
gross_changed     0
net_changed       0
stone_added       0
stone_removed     0
other_making_changed 0
```

The **"Weight or stones"** choice filters on the four that are false on every row. **It can never match.** And the screen answered with:

```
No tag was edited in this period.
```

Which blames the dates. **156 tags were edited in that period** — none of them touched a weight. A clerk widens the date range, gets the same sentence, and concludes the screen is broken.

It now answers the question that was asked:

```
No weight or stone was ever corrected.
Not in this period – in the whole capture. Of 156 edits, 31 changed the
item, 119 the unit count and 7 the making, and none touched a gross
weight, a net weight or a stone.
```

One dead end of my own

The first fix was appended at the end of the query function, after the footer. It never ran: `len(out) == 0` is already handled forty lines earlier, which returns before reaching it. The code built, the tests passed, and the screen rendered the old message — the only thing that caught it was **fetching the page and reading what came back**. Reverted and done at the existing branch instead.

A test would not have found this. **The screen still said something sensible, so nothing failed.** That is what makes a wrong empty-state message the hardest kind to catch: an empty table looks the same whatever the reason.

`check_empty_columns.py --read` now runs as a note in `check_all.sh`, so its 18 are read after every load rather than never.

466. Two hundred and thirteen choices in one dropdown that could never match anything

465 fixed one filter that could never match. The obvious question is how many others there are, so **every select filter on every report was probed against the whole of the shop's history**: read each report's own dropdowns from the rendered page, then request every value and see what comes back.

```
84 reports
55 select filters
6 blaming a period in their empty-state message
```

Five of the six were sound. `mrp-sales`, `purchase-contents`, `sale-cost`, `gstr-sales` and `gstr-purchases` return rows for **every** value they offer, and `metal-rates` cannot go wrong by construction: `ratePurityOptions` is `SELECT DISTINCT purity_code FROM mcj_metal_rates`, so it can only offer a purity that has rates.

The sixth was Tag-wise Stock, and it was worse than the one 465 fixed.

The dropdown offered the whole item master

items offered in the Item list	322
items that have ever carried a tag	109
items with no tag in stock	221
items with no tag EVER	213

`itemOptions` was `SELECT id, name FROM mcj_items` — the entire master. But the master holds stones, alloys, broken lots and coin types the shop **buys and consumes rather than tags**: AMETHYST, ALLOYS, BEADS, BROKEN DIA, 999COIN. A tag is never made for them, so they can never appear on a stock report.

213 of the 322 choices could return nothing under any combination of filters. That is two in every three.

And it said nothing at all

Picking AMETHYST gave an empty table. Not a message — **no rows whatsoever**. `stock.go` had no empty-state branch, so the screen's answer to *"show me the amethyst on the shelf"* was a blank page indistinguishable from a broken report.

Fixed the way rates.go already does it

The Item list is now built from the population the report draws on:

```
SELECT i.id, i.name FROM mcj_items i
WHERE EXISTS (SELECT 1 FROM mcj_stock_items s WHERE s.item_id = i.id)
```

323 options -> 110 (109 items and the blank)
dead choices -> 0 every offered item returns rows

An unreachable choice is now impossible to offer, rather than merely absent today. The rule is the one `metal-rates` follows and `barcode-edits` did not: a filter must be built from the data it filters.

The filters still combine, so a narrow value range or a purity with none left can empty the list legitimately. That case now answers:

Nothing on the shelf matches that.
The filters are combined, so a narrow value range or a purity with none left will empty the list. Every item offered has carried a tag at some point – if this one shows nothing, its tags have all been sold.

A miscount of mine, and what it cost

The first sweep reported **9** dead values here. The true number is 213. The probe counted a filter dead when it returned `<= 1` row – a threshold lifted from `check_screens_render.py`, where rows include a footer. This probe counted only `<td>` rows, so `<= 1` also swallowed every **correct** single-row answer, and `user-rights` filtered to one person was flagged 24 times over.

Re-run against *zero* data rows, the same sweep found 213. **Eleventh measurement error recorded** – and its shape is now familiar: a threshold that was right in the tool it came from and wrong in the one it was copied into. 463 was a regex too loose, 464 a login posted with the wrong field names, this a constant carried across a border it did not survive.

The rule that keeps earning its place: **reproduce the measurement independently before believing a count**, especially one small enough to be reassuring.

467. The form dropdowns held; a database failure wearing a missing rate's clothes did not

466 swept every select filter on every **report**. The same trap in a **form** is worse: a report offers a choice that shows nothing, a form offers a choice the clerk picks and the save refuses. So the entry screens were swept the same way.

The forms held, and here is why

```
tagPurityOptions    only purities that HAVE a rate, joined to mcj_metal_rates
rateForPurity        rate_date <= the date, ORDER BY rate_date DESC LIMIT 1
tag.go               if err != nil || rate.IsZero() -> refuses, and says which
                    purity and which date
```

Nine forms already carry a test for exactly this — `OldGold_DoesNotOfferStamped Purities`, `PartySample_OffersOnlyCustomersHoldingSomething`, `Reconcile_OffersOnlyBankAccounts`, `CoinRate_ListsTheCoinsAndNotOtherItems` and others. **This concern was designed for; 466's fault was the exception.**

A gap in the rate book that costs nothing

rate days	141	1 April to 21 August 2026
calendar days	143	
missing	2	Saturday 2 May, Thursday 6 August

Every day that has rates has all 20 purities, so the gap is whole days, not partial ones. **The shop traded on 2 May** — bills GST/2627/00048, 00049 and 00050, ₹40,388.00, ₹2,427.00 and ₹3,204.00 — with no rate set.

It costs nothing, and that is checkable rather than assumed: 22ct was **14,304.00 on 1 May and 14,304.00 on 3 May**, so the carry-forward reaches the identical figure the shop would have used. Recorded because the next gap may not fall between two equal rates.

The defect

rateForPurity ended:

```
if err != nil {  
    return rupees.Zero, on, nil  
}
```

Every failure returned as a zero rate and no error. No row is genuinely not an error — nobody priced that metal, and both callers test `rate.IsZero()` for precisely that, which `sale.go` documents at length. But a database that is unreachable, a query that will not parse, a context that has been cancelled all came back the same way, and the clerk was told:

There is no rate for 22 on 2 May 2026.

A sentence about the shop's rate book, for a fault in the connection to it. Somebody would open the rate book, find 22ct priced, and have nowhere to go.

Now `pgx.ErrNoRows` returns zero as before, and anything else is wrapped and returned — the idiom `order.go` and `tagcorrect.go` already use.

The shape this keeps taking

464	a malformed rule crashed instead of naming itself
465	an unmatchable filter blamed the period
467	a database failure blamed the rate book

Three entries, one fault: **a failure reported as a different, more ordinary condition**. Each is harmless while nothing goes wrong, and each sends the person who has to fix it to the wrong place. That is the cost — not a wrong number, but an afternoon spent looking where the answer is not.

468. The same disguise twice more: a quote priced at zero and a receipt series restarted

467 named the shape — **a failure reported as a different, more ordinary condition** — and counted three instances across 464, 465 and itself. Naming a class is a claim about more than one case, so the class was swept.

The Go form of it is exact enough to search for:

```
if err != nil {  
    return <zero>, nil        // every failure becomes "nothing found"  
}
```

Two more, both live.

A quote priced at zero and called flat-priced goods

`quote.go` carried its own copy of `rateForPurity`, down to the comment:

```
// No rate is not an error: flat-priced goods never had one.
```

True of one error. The caller tests `rate.IsZero()` and treats a zero as a piece priced at a flat figure rather than by weight — so **an unreachable database would have produced a quote with nothing charged for the metal**, and called it flat-priced. The Quote screen is the one whose output is read out to a customer standing at the counter.

A receipt series restarted

`partysample.go` numbers customers' own goods. On error it returned:

```
CG/<year>/00001
```

the **first number in the series**. The reasoning was right — nothing received yet means the series starts here — and it was reached by every other failure too. A database that was merely unreachable would have **restarted the receipt numbering**, and `PartySample_RefusesADuplicateReceiptNumber` would then refuse the save and tell the counter that receipt number was already used.

Which is true. And is not the reason. A clerk would go looking for `CG/2627/00001`, find it in the book from April, and conclude the screen had lost its place.

Both now forgive `pgx.ErrNoRows` by name and wrap everything else — the shape `order.go` and `tagcorrect.go` already used.

What the sweep says about the codebase

```
206 files scanned  
3 branches swallowed every failure    sale.go (467), quote.go, partysample.go  
0 remain
```


Three in a codebase this size is not a habit; it is the same helper written three times, each with the same reasonable comment about the one error that is not an error. **The comment was right and the branch was wider than the comment.**

Guarded, and falsified

`tools/check_swallowed_errors.py` — **check 16**. A branch returning a nil error must first name the error it forgives. Falsified by collapsing 467's fix back to its original single branch: the check named `sale.go:1514` and exited 1, and the file was restored byte identical.

```
464  a malformed rule crashed instead of naming itself
465  an unmatchable filter blamed the period
467  a database failure blamed the rate book
468  a database failure priced a quote at zero, and restarted a receipt series
```

Four entries, one fault. It is now the only one of the four with a check behind it, which is the difference between having fixed something and having stopped it.

469. A check for the accident that broke every page twice, and it missed the accident on its first run

Defects 401 and 423 were the same accident four weeks apart: a screen was added, its menu label was not, and **every page in the application panicked** — the sidebar is drawn on all of them, so one absent key takes the whole thing down. Both were found by loading a page and watching it die.

Nothing could have found them by searching. These keys are never written down:

```
"MCJMenu.Top."    + slug(top)
"MCJMenu.Group." + slug(top) + "__" + slug(sub)
"MCJMenu.Item."  + slug(item)
```

`tools/check_menu_keys.py` — **check 17** — rebuilds every one the way `navigation.go` does and asks whether `en.json` has it.

It was falsified with the real accident, and failed

The obvious test is to delete the exact key that broke everything at 401 — `MCJMenu.Item.reorder-levels` — and see the check say so.

It said nothing. 236 keys built, all present, exit 0.

Reorder Levels is not in `legacy_menu.csv`. It has no place in the old menu at all — defect 401 added it because the item master carried a minimum quantity for 31 items and no legacy screen ever read it. Such a screen registers itself in `modules/mcj/links.go`, and `navigation.go` has a **third pass** for exactly these, keying them identically at line 405.

The check read the CSV and the stand-in list, and stopped one source short — **and the source it missed is the one both historical defects came from.** A check for an accident that could not have caught either instance of it.

```
236 -> 261 keys once links.go is read
```

Falsified again, against both

MCJMenu.Item.reorder-levels	wanted by screen 'Reorder Levels'	(defect 401)
MCJMenu.Item.sales-register	wanted by screen 'Sales Register'	(defect 423)

Each deleted in turn, each named by the check with the screen that wants it and what to do; `en.json` restored and confirmed byte identical after both.

No key is missing today — 261 of 261 resolve, including 12 top menus, 32 sub-groups, 192 items and 25 screens with no legacy menu place.

The lesson, which is not the one it looks like

This is the fourth time in nine entries that a measurement was narrower or looser than its claim — 463, 464, 466, and now this. The difference here is what caught it: **not a reviewer, not a second pass, but insisting the check fail on the real historical case before trusting it.**

Had it been falsified with a made-up key — inventing `MCJMenu.Item.nonsense` — it would have passed the test and shipped, because a made-up key is missing from `en.json` *and* from the sources the check reads. It would have looked correct and covered nothing that had ever actually gone wrong.

Falsify with the accident you are trying to prevent, not with an accident you invented.

470. Four documents could be emptied and every check would still pass

469 ended on a rule: falsify with the accident you are trying to prevent. The obvious next question is which past accidents are still unguarded, so the record was read back and the faults it describes were tried again against today's checks.

The first one tried was **defect 447**, which is not hypothetical — it happened here, in this project, four days ago:

```
open(p, "w").write(open(p).read().replace(...))
```

Python opens the file for writing, **emptying it**, before evaluating the argument that reads it.

`docs/screens.md` lost 1,998 lines, and the only reason they came back is that a backup existed for an unrelated reason two hours earlier.

What is guarded, and what turned out not to be

`status_figures.py` catches a truncated `screens.md` and `data-defects.md`, because it checks a headline figure in each and a truncated file cannot produce it. That was verified rather than assumed: cutting `screens.md` to 60 lines fails check 5.

The other four documents have no figure anyone checks. Each was cut to five lines in turn and the suite run:

DECISIONS.md	16 check(s) passed
migration-profile.md	16 check(s) passed
pkg-contracts.md	16 check(s) passed
pricing-formula.md	16 check(s) passed

910 lines of decisions, contracts and formulas could vanish between two commits and every check would report the repository sound. Including `DECISIONS.md`, which is the record of what Rahul has already answered.

A floor, not a size

`tools/check_doc_floors.py` — **check 18**. Each document declares a minimum line count and the headings that must survive it.

A floor rather than an exact size on purpose: documents grow, and a check that had to be edited every time one did would be edited into uselessness. It fires on what truncation looks like — a file falling through its floor or losing a heading — and stays quiet for ordinary editing. Raise a floor when a document has really grown; never lower one to make this pass.

7 documents, 30,117 lines, each above its floor

Falsified with 447's own line of code

Not with a made-up truncation. **Defect 447's exact expression** was run against each of the four unguarded documents:

DECISIONS.md	0 lines, check exit 1
migration-profile.md	0 lines, check exit 1
pkg-contracts.md	0 lines, check exit 1
pricing-formula.md	0 lines, check exit 1

Each restored from a backup taken first and confirmed byte identical.

That the accident empties a file **completely** is worth noticing: it is not a subtle corruption, and it still went undetected for two hours and would have gone undetected forever in four files out of seven. **The size of a fault has nothing to do with whether anything is watching for it.**

471. The migrations were never once applied to an empty database, and they are correct

Continuing 470's method: read the record back, try each past fault against today's checks. **Defect 399** was a migration carrying `--` inside a SQL string literal — `sql-migrate` treats it as a comment marker and cuts the line there, so the migration applied something other than what was written.

Nothing checks the migrations. `status_figures.py` counts the files without reading them, and `check_all.sh` has never applied one. A malformed migration sits quiet until somebody builds a database from scratch — **the moment it is most expensive to find.**

So it was done, once, properly

```
CREATE DATABASE wingold_migcheck
DB_NAME=wingold_migcheck migrate up
-> Applied 134 migrations, exit 0
```

Every migration applies to an empty database. That had not been demonstrated before.

And the schema it builds is the one that exists

The stronger question is not whether they apply but whether they **reproduce** the database being used. Both schemas were dumped column by column, with types, and compared:

```
live      145 tables  1,544 columns
from empty 145 tables  1,544 columns
difference none
```

Identical, column for column and type for type. The migrations rebuild the database exactly, and the database has not drifted from them — no hand-applied column, no forgotten ALTER, nothing carried in a loader that should have been a migration. The scratch database was then dropped, and only `wingold` remains.

For a schema assembled across 134 files and several weeks, that is the result worth having and the one nobody had checked.

The cheap part, made permanent

`tools/check_migrations.py` — **check 19** — reads every migration for the two faults that are silent:

```
no '-- +migrate Up'      sql-migrate applies nothing and reports success
'--' inside a string     sql-migrate cuts the line -- defect 399 exactly
```

Both clean across 134 files. The full apply is not in `check_all.sh`: it needs a database of its own and minutes to run. What is written down instead is that it was done and what it proved, so the next person does not have to wonder.

Falsified with 399's own fault

Not an invented one. A line of the shape that broke it was planted into a real migration — `SELECT 'brs1 -- the bank reconciliation carry-forward'` — and the check named the file, the line and the string. The missing-Up case was planted separately and named too. The file was restored and confirmed byte identical.

```
470 falsified with 447's line of code
471 falsified with 399's fault
```

Two entries running, the check was proven against the accident it exists for rather than one invented to suit it — which is the only version of that test that means anything, and 469 is why.

472. Nineteen tables could empty with nothing failing, and the check written for it lost the digits off a name

470 asked which documents could vanish with every check still passing, and found four. The same question of the data:

populated mcj_ tables	83	
read by a report	62	check 13 fails if one goes silent
named in a check directly	28	
neither	19	

The nineteen are almost all **masters** — `mcj_states`, `mcj_religions`, `mcj_card_types`, `mcj_ledger_groups` (96 rows), `mcj_value_bands`, `mcj_weight_bands`, `mcj_professions`. No report reads them because they fill the **dropdowns on forms**, and 466 and 467 are two entries about how much a dropdown matters. Emptying `mcj_states` would leave the sale form unable to name a place of supply, and the suite would have said the repository was sound.

The rule taken from the load chain, not from a list

A list somebody maintains goes stale. `tools/check_loaded_tables.py` — **check 20** — reads `INSERT INTO` and `DELETE FROM` out of `tools/legacy/load_*.sh` and requires every table named there to have at least one row.

65 tables the loaders fill, 94,826 rows, none empty

That is the right rule because of how these loaders are built: **each one begins by DELETING what it is about to replace**. A loader that deletes and then loads nothing is not a hypothetical failure mode; it is the failure mode, and defects 294, 366 and 407 are three occasions when a variant of it happened.

Not-empty rather than a row count, deliberately. A count fails the day the shop legitimately deletes a card type, and a check that cries wolf gets edited until it says nothing. Zero rows in a table the shop has data for is never right.

The falsification found a bug in the check

There was no empty `mcj_` table to point at, so one was made — `mcj_falsify_471` — named from a loader, and the check was run. It fired, and said:

`mcj_falsify_` is written by a loader and does not exist

`mcj_falsify_`. The digits fell off. The regex was `mcj_[a-z_]+`, which stops at the first number, so a table whose name carries one would be looked up under a name that does not exist and reported missing while sitting there populated.

No `mcj_` table has a digit today — the query says zero — so nothing was wrong. The regex was wrong regardless, and **the only reason it was ever read closely was that the falsification printed it back**. A check that passes prints a summary nobody examines; a check that fails prints the thing it found.

```
463 a regex too loose hid a screen
466 a threshold carried across a border it did not survive
472 a regex too narrow lost the digits off a name
```

Third time in ten entries. The pattern is not carelessness about regexes; it is that **a pattern is only ever tested against the input you had in mind**, and the falsification is where a different one finally arrives.

Fixed to `mcj_[a-z0-9_]+`, and the scratch table dropped.

473. B15 was a decision nobody was checking, and both existing guards were narrower than it

Blocker **B15** is Rahul's decision: five legacy tables were deliberately never opened, because they hold the shop's messaging credentials.

```
api_details      password, key_name
email_master
sms_profile      pwd
whats_app_url
whatsapp_profile password
```

They are staged in `legacy` only because the whole dump was staged. Nothing loads, reads or reports on them, and the point of the decision is that it stays that way as the system grows.

Nothing was checking that it had.

Two guards existed, and both are narrower than the decision

```
load_users.sh      asserts no credential column exists
                   -- on mcj_legacy_users and mcj_legacy_access only,
                   and only when a loader is run
usermaster_e2e_test.go asserts the same
                   -- on mcj_legacy_users only
```

Both were verified still working. Neither would notice the way this would actually go wrong: **not somebody adding a password column to the users table, which is obvious, but a loader for message history quietly bringing the profile along with it**. A new table built from `sms_profile` passes both guards without touching either.

The check, over everything

`tools/check_no_credentials.py` — **check 21** — asks both halves across the whole system:

```
no mcj_ table anywhere carries a column reading pass, pwd, scode,
secret, token, otp or api_key
none of the five sealed tables is named by any loader, migration,
controller, report or template
```

Comments are stripped before the second search, so an entry explaining the decision is not mistaken for a use of it — the decision has to be discussable without tripping the check that enforces it.

Both clean.

Falsified in both directions

```
a loader reading legacy.sms_profile    named the file and the table, exit 1
ALTER TABLE mcj_legacy_users ADD scode    named the column, exit 1
```

The column was added to a real table and dropped again; the loader was restored from a backup taken first and confirmed identical. The users table has its original column count.

Why this one is different from the twenty before it

Every other check here enforces something the code should be true to. This one enforces something **Rahul said**, and the only record of it was a blocker paragraph and two guards that had drifted narrower than the sentence they came from.

A decision that lives only in prose is kept by whoever remembers it. **Sixteen entries have now been written since B15 was recorded, and the decision survived on memory alone.** It no longer has to.

474. The check written to keep credentials out could not spell two of them

473 built check 21 against blocker B15 and called it done. One entry later, a different thread — auditing the settled decisions in `DECISIONS.md` — walked past this line in **defect 21**, written months ago:

```
Ten columns store credentials this way, including `branch_master.br_psw`
(`b001`) and `retail_master.rt_psw` (`r001`).
```

`psw`. Check 21's list was `pass`, `pwd`, `scode`, `secret`, `token`, `otp`, `apikey`, `api_key`. It does not contain `psw`, and neither `br_psw` nor `rt_psw` matches any of the others.

```
credential-looking columns in legacy    15
spellings check 21 could recognise      13
spellings it could not                  2    br_psw, rt_psw
```

And these are not on tables nobody touches. `branch_master` **and** `retail_master` **are both read by** `load_masters.sh` — they are where `mcj_branches` and `mcj_retail_points` come from. They are, of the fifteen, the two columns most likely to be carried across by accident, and the two the check was blind to.

Nothing came across: the loader names its columns one at a time and never asks for either. **The check simply would not have said so** — one entry after being written to say exactly that.

Widened, and falsified with the real name

`psw` and `key_name` added. Falsified per 469's rule with the historical name rather than an invented one:

```
ALTER TABLE mcj_branches ADD COLUMN br_psw text
-> mcj_branches.br_psw looks like a credential, exit 1
```

Column dropped; `mcj_branches` has its original column count.

Four times now, and the reason is not the regexes

```
463 a regex too loose hid a screen
466 a threshold carried across a border it did not survive
472 a regex too narrow lost the digits off a name
474 a list of spellings that did not include the one on file
```

The common cause is not carelessness with patterns. It is that **a pattern is written from what the author is picturing**, and the author was picturing `password`. The two real names had been written down in this repository since defect 21 — the check's own subject matter, in the entry it exists to enforce — and were not read while writing it.

What breaks the cycle is not more care. **It is reading the record for the actual names before writing the pattern, and falsifying with those names rather than with something invented.** That rule has now caught its own author four times.

475. A settled decision the schema does not keep: branch is modelled on two tables out of eighty-three

473 said a decision living only in prose is kept by whoever remembers it. That was about B15. The same question put to `docs/DECISIONS.md`, which is the record of what Rahul has already answered, found decision #4:

```
Scale | Multi-branch is definitely coming.
      | Branch modelled properly from day one; inter-branch transfers...

mcj_ tables carrying tenant_id      83 of 83
mcj_ tables carrying a branch       2 of 83
```

The two are `mcj_bank_accounts` and `mcj_retail_points`. **Not one operational table has a branch:**

<code>mcj_sales_invoices</code>	224 bills	no branch
<code>mcj_stock_items</code>	2,816 tags	no branch
<code>mcj_vouchers</code>	1,848 vouchers	no branch
<code>mcj_stock_movements</code>	9,031 movements	no branch
<code>mcj_customer_orders</code>	284 orders	no branch
<code>mcj_stock_conversions</code>	859 conversions	no branch

`tenant_id` is not this. A second shop under the same owner is one tenant and two branches, and every one of those rows would be unattributable.

The legacy did record it

This is not a column the old system lacked. **Defect 5**, one of the first written:

```
voucher_detail.br_code holds B001 (the code), 1 (the numeric brid), and
blank. 495 rows carry 1. voucher_main.br_code is always B001.
```


Its instruction was "*normalise 1 → B001 and blank → header value during the transform*". The transform instead dropped the column. Every voucher in the capture belongs to branch **B001**, and that fact is now nowhere.

Why nothing noticed

`check_unwritten_columns.py` reports "*44 columns across 14 tables — each a field the legacy recorded and this system cannot*", which sounds like it would catch this. It does not: it iterates `mcj_ tables` and finds columns no Go code writes. **It never reads the legacy schema.** A column dropped entirely is invisible to it, because there is no new column to be found unwritten.

Nothing anywhere compares the two schemas. A first measure put it at **511 column names with no counterpart anywhere in the mcj schema** — a figure that turned out to be badly inflated, because it matched by NAME and the loaders rename as they go: `ac_name` becomes `name`, `mobile1` becomes `mobile`, `d_o_b` becomes `date_of_birth`. **Defect 476 redid it against what the loaders actually select and found 66**, of which `br_code` is one. The correct figure is there, not here.

What it costs, and when

Today it costs nothing. The shop has one branch, `branch_master` has one row, and every voucher says `B001`. Nothing on any screen is wrong.

It costs on the day the second shop opens. Then 2,816 tags, 9,031 movements and 1,848 vouchers have no branch and no way to acquire one — the information was thrown away at load, not merely left unmodelled — and every query, report and screen has to change at the same time as the shop is opening a branch.

This is Rahul's call, and it is on the ask list

Two honest options, and the cost is very different:

- **Add branch now.** 83 tables, but the data is small and every row is `B001`, so backfilling is one statement per table. The work is in the screens.
- **Accept it and do it later**, knowing the historical rows cannot be attributed afterwards.

Not decided here. Defect 78 guessed at something like this and defect 133 had to correct it — and unlike 78, this one has a settled decision on file saying which way it should go, which is exactly why it should be asked rather than assumed.

476. Five hundred and eleven was wrong; sixty-six is right, and one customer lost their only phone number

475 said nothing compares the legacy schema with the new one, and put the size of that unexamined space at **511 column names with no counterpart**. That figure was published one entry ago and it is wrong.

It matched by name, and the loaders rename as they go.

legacy	->	mcj_parties
ac_name	->	name
mobile1	->	mobile
adrs_off	->	office_address
d_o_b	->	date_of_birth
company	->	company_name

Every one of those was counted as dropped. `mobile1` has 39,399 distinct values and sits in `mcj_parties.mobile` — it is carried, and the comparison said it was gone.

Measured again, three times, because twice was wrong

columns dropped by NAME, with 50+ rows and 2+ distinct values	388
of those, named somewhere in a loader (i.e. renamed, carried)	322
named by NO loader at all	66

The second attempt gave 90, not 66. The identifier pattern was `[a-z][a-z0-9_]{2,}` — **three characters minimum** — so `gw` and `nw`, the gross and net weights, could never be recognised as carried and were counted as lost on four tables at once. **Twelfth measurement error, and the second in this one entry.**

511	matched by name, ignored renames
90	identifier pattern too long for a two-letter column
66	correct

The sixty-six, spot-checked at the top

`address_book_master.panno` — there are two PAN columns and the loader takes the other one, `pan_no`.

both filled and identical	227
both filled and DIFFERENT	1
PAN only in panno	0
PAN only in pan_no	91

Nothing is lost: no customer's PAN exists only in the dropped column. One disagrees — **ARUNABH DAS**, `BXRPD7554D` loaded against `BXPRD7554D` dropped, a transposition of two letters. The fourth character of a PAN is the holder type and must be `P` for a person: the **loaded** value has it and the dropped one does not. The loader took the valid one, and it took it by luck rather than by checking.

`address_book_master.phone2` — filled on 3,033 rows, and `mcj_parties` has no second landline.

a different number from phone1	2,997
the only landline they have	974
and no mobile either	1

So 2,997 second numbers are genuinely not carried, and the harm is one customer. **One person in 40,519 cannot be telephoned from this system and could be from the old one.**

What this entry is really about

The finding is small. The correction is not: **a figure went into the record with a hedge on it — "a proxy rather than a defect list" — and the hedge did not make it true.** 511 read as an alarming number and was quoted as one. Hedged prose around a wrong number still leaves a wrong number, and the reader keeps the number.

475 has been corrected in both documents to point here rather than quietly amended, and 66 is the figure that stands.

477. The sixty-six triaged, and the order book's header disagrees with its own lines on 78 orders

476 reduced the dropped-column question to a real list of **66**. Triaged, they fall into three groups:

internal legacy keys	tran_id, new_vid, rcpt_vid, stkid, sl_id, pmnt_vid, jrnل_vid, barcode_detail_id, ... -> replaced by this schema's own UUIDs
denormalised copies	tot_amt, sale_amount, due_amount, actual_gw, actual_nw -> recomputable from the lines
real content	phone2 (476), picture_sketch (5,046 design sketches), amount_received, item_size, catg

Most of the 66 are the first group and are right to be gone.

The one that looked worst was not a loss

`customer_order.amount_received` is filled on **273 of 284 orders** while `mcj_customer_orders.advance` sums to **zero**. That reads as every advance the shop ever took being dropped.

It is not. Compared against each order's own lines, `amount_received` **equals the line total** — ratio 1.00 on the large majority. It is what the order is worth, not what was paid on it. The genuine `advance` column is zero on all 284 rows: the shop never used it, and the loader carries it faithfully.

Nothing is lost. The order's value is in its lines, and the lines are loaded.

What the comparison did find

orders with lines	271
header equals the sum of its lines	193
header DISAGREES	78
all 78 have the header HIGHER, none lower	
total	1,06,00,834

A one-directional gap of **₹1.06 crore**. Not scattered either way, which random error would be — the header is never lower. That points at something the header includes and the lines do not, or at lines removed after the header was written.

This system computes an order's worth **from the lines**, which is the itemised source and the defensible choice. But if the header is right, the order book is understating by ₹1.06 crore across 78 orders. **Added to the ask list**, because which one is right is a question about how the shop wrote orders, not one the data settles.

A bug of my own that hid the largest row

The first pass at this ordered by a column alias:

```
SELECT to_char(recv, 'FM9,99,99,999') AS received, ...  
ORDER BY received DESC
```

The alias **shadowed the numeric column it was computed from**, so the sort was alphabetical over formatted text — and 9,95,038 sorted above 7,38,00,000. The single largest order in the shop's book, worth ₹7.38 crore, was invisible on page one of its own report while two of my queries quietly disagreed.

Thirteenth measurement error. It joins the pattern the last several entries keep circling: 463 a regex too loose, 466 a threshold from another tool, 472 a regex too narrow, 474 a spelling not on the list, 476 a name comparison blind to renames, and now **a sort that was never numeric**. Every one produced a confident, plausible, wrong number — and every one was caught by asking the same question a second way.

478. Thirty-eight columns of the shop's data that no screen reads, and two checks that both missed them

477's list of 66 named `design_transaction_main.picture_sketch` as real content never loaded. Following it found something larger, and one file over.

`picture_final` is loaded — into `mcj_designs.picture_path`, filled on **5,104 rows**. Nothing reads it. No screen names the column, and:

```
check_unwritten_columns.py    finds columns nothing WRITES    -- this is written  
check_empty_columns.py       finds columns that are EMPTY    -- this is full
```

It fell between both tools. They sit either side of the question and neither asks it: a column that is written, full of the shop's data, and read by nothing.

What is in the gap

```
38 columns across 13 tables    (understated -- 40 once comments stop
                                counting as reads, see defect 490)

mcj_packets                    17 columns    crown_angle, crown_height, depth_mm,
                                width_mm, length_mm, table_percent...
                                NOT 205 packets. Between 2 and 6 rows
                                each -- the figure first written here
                                was wrong, see defect 479
mcj_bank_accounts              3 columns    account_no and ifsc on 397 accounts
mcj_scheme_payments            2 columns    payment_mode and instalment_no on all
                                2,144 instalments
mcj_designs                    2 columns    designed_on (5,221) and picture_path
mcj_stock_items                5 columns    lot_no (2,059), making_sales (1,825)
mcj_scheme_enrolments          1 column     instalment_amount (239)
```

Some of it is rightly unread. `legacy_v_id` and `legacy_user_id` are tracing keys. `picture_path` points at `../image/...` — a **relative path on the shop's own ERP server**, which 404'd, so those files are gone and the column is a record of what was lost rather than a way to find it.

The rest is not plumbing. (The diamond measurements are **not** part of it — defect 479 recounted them and they are filled on a handful of packets, not all 205.) **Every bank account's number and IFSC** are recorded and shown nowhere. **How each of 2,144 instalments was paid** — cash, card, transfer — is known and never displayed, on a scheme book that is already on the ask list twice.

The two image populations, which no document separates

```
tag photos    http://tag09.com/mcj2/images/bar_pic/...    B24, 927 tags
               a third-party host, still answering, behind a bot check
design images  ../image/14017640.jpg                    5,105 sketches
               the shop's own server, 404 since 21 August
```

B24 asks Rahul to *"open one tag photo in a browser"* — a live test, because that host is still there. **The design images cannot be tested that way at all.** They were never on a host anyone can try. `designs.go` says *"the pictures are gone"*, which is right about `picture_final`; there are **two** images per design and the sketch, 5,105 of them, is recorded nowhere.

Now a standing note

`tools/check_unread_columns.py` prints the 38 after every run, beside the two notes it sits between. Not a failing check: **a column carried across and not yet shown is a decision, not a fault.** But it is the shortest description of what this system holds and does not surface, and it should be read rather than rediscovered by hand a third time.

479. "The full gemmological measurement of every diamond lot" was four packets

478 was written one entry ago. Acting on it — the goal says build every screen that has data behind it, and 205 diamond lots with crown angles and table percentages is exactly that — meant opening the packet screen. The data is not there.

mcj_packets	205 rows
crown_angle	4
table_percent	4
depth_percent	4
crown_height	4
depth_mm	4
pavilion_angle	4
average_weight_ct	6
length_mm	3
width_mm	2
rapaport_discount	2

Every one of the seventeen is filled on between two and six packets.

The census counted zero as a value

```
btrim(COALESCE(col::text, '')) NOT IN ('', '0', '0.00', 'false')
```

crown_angle is numeric, and Postgres renders its zero as 0.000 — three decimals, matching none of the three spellings in that list. 201 zeros were counted as data, and the column reported 205 filled out of 205.

stored value	rows
0.000	201
34.300	1
34.900	1
35.300	1

Fourteenth measurement error, and the second time in four entries that a figure published with confidence had to be corrected in the entry after it (476 corrected 475's 511).

What the correction changes

Not much of the finding, and all of the emphasis. The 38 unread columns are real and so is most of the list — **designed_on on 5,221 designs, instalment_no and payment_mode on all 2,144 instalments, account_no and ifsc on 397 bank accounts, lot_no on 2,059 tags**. Those counts were re-derived type by type and hold.

What does not hold is the sentence that made the entry sound important: *"205 packets, the full gemmological measurement of every diamond lot."* The shop recorded those measurements for **four stones** and left the rest at zero. A screen for it would be a heading over an empty table — the exact thing defect 460 was about — and it will not be built.

The tool now carries the numbers

check_unread_columns.py printed names without counts, so it could not have been wrong; it also could not have caught this. It now counts each column **the way its type requires** — numeric zero is not a value, a false boolean is not a value — and prints the figure beside the name:

```
mcj_designs: designed_on (5,221), picture_path (5,104)
mcj_scheme_payments: instalment_no (2,144), payment_mode (2,144)
mcj_packets: average_weight_ct (6), fluorescence (5), crown_angle (4), ...
```

A note that prints a name invites a guess about the quantity, and the guess was mine. The figures are in the output now, so the next reading of this list starts from what is there rather than from what its heading suggests.

480. The scheme book now says how each instalment was paid, and how far the customer got

479 corrected 478 and closed the packet measurements: four stones, not 205, and no screen worth building. The rest of that list survived the recount, and the largest genuinely useful item was on a screen already built.

`mcj_scheme_payments` carries `payment_mode` and `instalment_no` on **all 2,144 payments**, and the Scheme Book read neither.

Derived twice before touching anything

Fourteen measurement errors say to reproduce the figure from the data before acting on it. Both columns were counted first, and the modes reconcile:

Bank	1,287	46,99,400.00
Cash	528	25,49,300.00
Card	329	15,42,200.00

	2,144	87,90,900.00

₹87,90,900.00 is defect 233's figure for every instalment ever collected, to the rupee, arrived at from a column nothing had read. `instalment_no` runs 1 to 10, ten distinct values, which is what a scheme of ten instalments should have.

That is the check 479 was written to enforce, applied before writing rather than after: **the data agreed with a figure derived independently months ago.**

Two columns added

Up to no.	the highest instalment number reached			
Paid by	the modes used, e.g. "Bank, Cash"			
KIRAN ASWANI	1 instalment	up to no. 1	Cash	5,000.00
P. K. MUNDRA	2 instalments	up to no. 2	Cash	10,000.00
RESHMI AGARWAL.	3 instalments	up to no. 3	Cash	15,000.00

Both are aggregated in the LATERAL subquery already there, so the screen makes no extra query. 236 rows render, and `mcj_scheme_payments` has left the unread list: **38 columns, 13 tables → 36 columns, 12 tables.**

Why this and not the others

The remaining 36 are not all worth a screen and this entry does not claim they are. `legacy_v_id` is a tracing key. `picture_path` points at a dead server. `designed_on` on 5,221 designs is a real candidate; `account_no` and `ifsc` on 397 bank accounts are real but belong on a master, not a report.

The scheme book was chosen because it is the one place where unread data sat **underneath a question already on the ask list twice** — defect 233 asks how scheme settlements were recorded and B23 asks about ₹77,37,300 owed on matured enrolments. Whoever answers those will want to know whether the money came in by bank, cash or card, and now the screen says so.

481. The design date is the day the ERP was installed, and the catalogue has not moved since 2017

480 took the largest useful column off 478's unread list. The next one down was `mcj_designs.designed_on` — filled on **all 5,221 designs**, with a Design Catalogue screen already built to put it on.

It was derived first, which is the rule fourteen measurement errors bought.

```
5,221 designs
  12 distinct dates between them
4,991 of them share ONE date: 5 July 2016    (95.6%)

2016-07-05    4,991
2016-11-07     135
2016-09-06     51
2017-01-10     18
2016-07-29      9
```

Both sides agree: `legacy.design_transaction` gives the same 12 days and the same range. **This is not when the designs were drawn. It is the day the old ERP was loaded**, with a handful of later entries after it.

So the column was not added

A column headed "**Designed**" showing 5 July 2016 against 4,991 designs would be read as fact by anyone who opened the screen, and it would be an installation timestamp wearing a design date's label. That is the family 465 and 467 are about — **a value reported as something more ordinary than it is** — and the cheapest place to stop it is before the column exists.

The reasoning now sits in `designs.go`, above the function, where the next person tempted by an unread column will meet it. **459 and 460 were both about findings that never reached the text a user reads; this one starts there.**

What the dates do say

```
the last design in the catalogue    20 January 2017
```


Nothing has been added in nine years. The shop has gone on trading to August 2026 — 224 bills, 284 orders, 2,816 tags — against a design catalogue that stopped receiving entries in the January after it was installed. Whether the shop stopped recording designs or stopped using the screen, the catalogue is a historical document rather than a working one, and no screen said so.

Also checked, already known

mcj_designs holds 5,221 rows under 5,218 distinct design numbers. The three repeats — GBLI0001, GBLI0002, GBLI0003, each twice with two different estimates (₹2,650 against ₹4,350; ₹4,779 against ₹5,460; ₹3,360 against ₹3,897.25) — are **defect 394**, already on file. Checked before writing rather than logged twice.

482. Six customers with dates that have not happened, and a data quality screen that called them complete

481 found the design catalogue frozen since January 2017. That was one dataset's staleness; the obvious question is which others stopped, so the most recent date in every loaded table was taken.

mcj_designs	2017-01-20	5,221 rows	
mcj_holidays	2024-12-26	109	
mcj_estimates	2025-05-23	2	
mcj_scheme_enrolments	2026-02-11	239	
...			
mcj_metal_rates	2026-08-21	2,820	
mcj_vouchers	2026-12-05	1,848	the post-dated cheque, 462
mcj_parties	9888-12-09	40,519	

Nine thousand eight hundred and eighty-eight.

JAINAT KUMAR BEZBARUAH	born 9888-12-09	
SHARMIN KHAN	born 4582-12-12	
SARITA SAFAR	born 2098-08-18	
SUSHIL KUMAR AGARWAL	born 2026-08-30	(five days from now)
R. D. KHANDELWAL	anniversary 2027-10-13	
DR. BISHWAJIT MODAK	anniversary 2028-04-07	

Typed years, almost certainly: 9888 for 1988, 2098 for 1998.

The screen that exists for this counted them as good

The Data Quality screen offers the six checks the legacy dropdown had:

Mobile number · Address · Email address · Anniversary date · Date of birth · Duplicate name
--

Every one asks whether a field is PRESENT. None asks whether it is possible. A birthday in the year 9888 is not missing, so all six customers passed the very checks that exist to find bad customer records.

This matters more than a typo usually would because of what the field is for. A jeweller greets customers on their birthday and their anniversary — B18 is on the ask list precisely about having customer records in one place. **A birthday list built from this either never reaches those four or greets one of them in the year 9888.**

Two checks added, and two rules

The screen now offers "**Born in the future**" and "**Anniversary not yet reached**", so the shop can find and fix them rather than being told the records are complete.

`check_invariants.py` gains two rules at the counts that exist today — 4 and 2, allowed and recorded, in the codebase's own pattern for a known quantity that must not grow:

```
nobody is born in the future          holds (4 allowed, unchanged)
no anniversary is still to come       holds (2 allowed, unchanged)

39 of 39 rules hold
```

Falsified against the database: a fifth impossible birthday was written to a row that had none, and the rule reported *"5 customer(s) with a date of birth after today"* and failed the suite at 38 of 39. The row was set back to NULL and the count is 4 again.

One more thing the census said

`mcj_holidays` stops at **26 December 2024**. The shop has traded through all of 2025 and 2026 with no holiday list for either year — 109 rows describing which days the shop was shut, none of them covering the last twenty months.

483. Three hundred and seventeen birthdays with no year, and two sentinels behind them

482 ended on the distinction between a field being **present** and being **possible**. Sweeping every date column in the database for values outside any plausible window turned up exactly two columns — both on `mcj_parties`, and both much larger than the six future dates 482 found.

```
date_of_birth      317 before 1900      4 in the future
anniversary_date   283 before 1900      2 in the future
```

Every other date in every other table is within a sane window. **The customer table is the only place this lives**, which fits: it is the only table filled in by hand at a counter.

They are not junk, and three different things are mixed together

```
40,519 customers
32,842  no date of birth at all
 7,356  a plausible one   (14 Apr 1900 to 10 Aug 2026)
   247  year 0001
    60  year 1000
    10  a three-digit year: 199, 934, 975, 979, 983, 996, 997
     4  in the future                                     (defect 482)
```

Year 0001 is a birthday recorded without a year. The 247 spread across **204 distinct day-and-month pairs covering all twelve months** — that is a real birthday list with the year left out, not corruption. The 60 at year 1000 are the same thing behind a second sentinel: 54 distinct day-months.

Only the **ten three-digit years** are typing: 975 for 1975, 983 for 1983, 996 for 1996.

Why the distinction decides what to do

For a jeweller the day and month are the whole point — a birthday greeting does not need the year. So **307 of these 317 are usable for the thing the field exists for**, and would be lost by any rule that treated a pre-1900 date as garbage.

What they cannot support is arithmetic. Any age calculation on them returns two thousand years, and any screen printing the date raw shows *"14 Dec 0001"*.

So the Data Quality screen now offers **"Birthday with no usable year"** rather than anything that calls them invalid, and the invariant is a **floor that must not grow**, in the codebase's own pattern:

```
no birthday loses its year      holds (317 allowed, unchanged)

40 of 40 rules hold
```

What this says about 482's fix

482 added *"Born in the future"* and caught 4. This sweep found **317 in the same column**, seventy-nine times as many, by asking the question the other way round.

Looking for the impossible found four. Looking for **everything outside the possible** found three hundred and twenty-one, and then required the harder work of deciding that most of them are fine. **The cheap version of a plausibility check is the one that finds the smallest problem.**

484. Thirty-one designs priced below nothing, and the same alias bug I logged eight entries ago

483 swept every date column for values outside the possible. This is the other half: **every numeric column on every populated table**, 256 of them, asked for its minimum, maximum and negative count.

Six hold a negative value, and three of those are right:

mcj_sales_invoices.round_off	115 rows	rounding down is negative
mcj_vouchers.round_off	129 rows	the same
mcj_purchase_invoices.round_off	12 rows	the same
mcj_credit_notes.round_off	2 rows	the same
mcj_stock_conversion_pieces.total	2 rows	-40.00
mcj_designs.estimate	31 rows	down to -3,97,488.00

A design estimate is a price. It cannot be less than nothing.

DBNG0100	-3,97,488.00
DBNG0097	-3,10,655.00
DBNG0094	-1,83,940.00
DBNG0099	-1,56,760.00

31 designs	-12,86,354.00

The loader is faithful — `legacy.design_transaction_main.estimate_amount` holds the same 31 negatives and the same minimum.

No cause found, and one hypothesis killed

DBNG0100 has `rate = 0`, which looked like the answer: an estimate computed against a metal rate of nothing. It is not. **All 5,221 designs have rate = 0**, so a zero rate is universal and explains nothing. The 31 are spread across four item families — 19 rings, 6 lockets, 5 bangles, 1 top — with no shared shape. They read as individual entry errors.

What it costs

`designs.go` sums estimates for the catalogue's footer:

shown today	14,95,13,508.73
positives only	15,07,99,862.73
the difference	12,86,354.00

The Design Catalogue reports the shop's design book as ₹12.86 lakh smaller than the sum of its designs, because thirty-one of them subtract. A negative row in a list of prices is visible if you look; a footer that quietly nets it is not.

Now `no design is priced below nothing` — holds (31 allowed, unchanged), a floor in the codebase's pattern. **41 of 41 rules hold.**

The bug in the middle of this, which I have made before

The first listing of the negatives sorted them wrongly and showed -107.00 as the most negative of thirty-one. The query was:

```
SELECT to_char(estimate,'FM-9,99,999.00') AS estimate ... ORDER BY estimate
```

The alias `estimate` **shadowed the numeric column it was computed from**, so the sort ran over formatted text. It hid -3,97,488.00, the largest of them.

That is defect 477 exactly — same shadowing, same ORDER BY, same consequence of hiding the biggest row — written up eight entries ago, in this document, by me. **Fifteenth measurement error, and the first that is a repeat.**

Writing the lesson down did not prevent it. What caught it both times was the same habit rather than the same memory: **two numbers from the same data disagreed, and the disagreement was chased instead of picked between.** That is worth more than the note, because the note demonstrably did not work.

485. The order book values the shop's orders at a fifth of what its own lines say

484 swept every numeric column for negatives. The other side of that sweep — the maxima — pointed here, by a route worth recording because the first two things it suggested were both wrong.

An order line of **5,000.000 g** looked impossible. It is not: the line is **SLV**, and five kilos of silver is an ordinary article. Checking the purity before writing killed that. But the largest order in the book, **CO/2526/00402**, showed a line estimate of **.00** while defect 477 had recorded its value as **₹7.38 crore**. Two figures for the same order, from the same capture.

Two columns, and the loader takes the empty one

`legacy.customer_order_main` carries **both** `estimate` and `amount`.

```
366 order lines
176  estimate is set
185  estimate is ZERO while amount is not
    -- more than half of the order book

SUM(estimate)    2,68,52,217.00
SUM(amount)      13,37,70,389.00
```

`load_orders.sh` line 59 takes `m.estimate`. So the Order Book values every one of those 185 lines at **nothing**, and the shop's order book reads:

```
shown            2,68,52,217.00
its own amounts  13,37,70,389.00
```

A fifth of what the lines say they are worth.

Which column is right, and how that is known

Not by preference. Defect 477 compared each order's header `amount_received` against the sum of its lines and found **193 of 271 agree exactly**. That comparison used `amount`. The header corroborates `amount` on three orders in four; **nothing corroborates** `estimate`, and it is empty on more than half the lines it is supposed to describe.

The loader's own reconciliation passes

```
SRC = SUM(estimate) FROM legacy.customer_order_main
DST = SUM(estimate) FROM mcj_customer_order_lines
OK: estimates tie at 2,68,52,217.00
```

It ties, and it always will. **A reconciliation confirms that a column arrived, never that it was the right column.** Every loader here is built this way, which is correct for catching a broken transfer and blind by construction to a wrong choice of source. That is the more useful half of this entry: the check that was supposed to make the order book trustworthy cannot see this at all.

Not changed, asked

Switching the loader would move a headline figure by **₹10.69 crore**, and which column the shop treated as the order's worth is a question about how they wrote orders. Defect 477 already asks a smaller version of it — the ₹1.06 crore gap between headers and lines — and this belongs beside it.

The evidence points one way and the entry says so plainly: `amount` **is corroborated by the shop's own headers on 193 orders and** `estimate` **is empty on 185 lines**. But defect 78 inferred and 133 had to correct it, so it is asked.

486. The vouchers agree to thirty-one rupees, and the column that looks like their key is a third of one

485 found a loader taking the wrong column and a reconciliation that could not see it. The same question was put to the vouchers, which are the accounting backbone: `legacy.voucher_main.tot_amt` is a header total on all 1,848 vouchers and no loader takes it.

legacy header totals	13,60,64,186.61
loaded lines, summed	13,60,64,218.37

difference	31.76

Thirty-one rupees and seventy-six paise across 1,848 vouchers, ₹13.6 crore of them. The shop's own voucher headers and the lines this system rebuilt them from agree to within a rupee per fifty-eight vouchers. Some of it is rounding — `round_off` accounts for ₹17.62 across 243 vouchers — and the rest is the paisa of defect 337.

That is the answer 485's question deserved and did not get: **here the loader took the right column, and it can be shown**. Unlike the order book, nothing had demonstrated it before.

The trap found on the way

Checking it per voucher rather than in aggregate gave a completely different answer: 1,817 of 1,848 disagreeing, an average gap of ₹3,17,188 and a net of **₹57.6 crore**. Both cannot be true.

<code>legacy.voucher_main</code>	1,848 rows
<code>distinct v_id</code>	648
<code>distinct (v_id, v_typ)</code>	1,840
<code>distinct (v_id, v_typ, ac_yr)</code>	1,848

`v_id` **is not a key**. A legacy voucher is identified by id, type and year together; `v_id` alone repeats because numbering restarts each year and runs per type. Joining on it fans each row into about three, which is where fifty-seven crore of nothing came from.

`mcj_vouchers.legacy_v_id` stores that first third. It reads like a back-reference and cannot identify a legacy voucher — anyone tracing one back lands on roughly three candidates. **Nothing reads it today** (it is on 478's unread list), so there is no live error, and the type and date are both on `mcj_vouchers`, so the full key can be rebuilt when something needs it.

The warning now sits in `load_vouchers.sh` , above the INSERT that writes the column, because **484 established that recording a lesson in this document does not stop it happening again.**

Sixteenth measurement error, and the second repeat

Defect 410: *"a join fanned 20 bills into 80."* This is that, on vouchers, at scale — and the second time in three entries that a fault already written up here was committed again (484 repeated 477's alias shadowing).

Both repeats were caught the same way, and it is worth naming because it is now three for three: **an aggregate and a per-row measurement of the same quantity disagreed, and the disagreement was chased rather than resolved by preference.** The aggregate was right both times. **Two figures from one dataset that do not match are not a nuisance; they are the only free error detection available.**

487. Eleven purchase bills tax the metal and not the stones, and ₹28.9 lakh sits outside the taxable base

486 ended on two independent measurements of one quantity being free error detection. Applied across the loaded ledgers, three of the four pairs agree and the fourth does not.

bills	legacy total_gross_amount	2,40,66,454.00	exact, three ways
	loaded goods_amount	2,40,66,454.00	
	sum of loaded lines	2,40,66,454.00	
vouchers	legacy header totals	13,60,64,186.61	31.76 apart (486)
	loaded lines	13,60,64,218.37	
orders	header vs lines		1.06 crore (477) and 485 open
purchases	invoice taxable	2,34,04,751.50	28,94,996.21 apart
	sum of contents	2,62,99,747.71	

It is not scattered — it is one rule

invoices with metal only	9	agree exactly
invoices with stone only	23	agree exactly
invoices with BOTH	11	every one differs

On all eleven, `taxable` **equals the metal value to the paisa** and the stone value — **₹28,94,996.21** — falls outside it.

That is a rule, not an error rate. Somebody decided the taxable base of a mixed purchase is its metal.

Whether that is right is not answerable from the capture

Gold is taxed at 3% and diamonds are not; a mixed bill legitimately has more than one taxable base, and the stones may sit on a separate bill or a separate line of the same return. The blended rates say nothing either way — **2.061% of the taxable, or 1.241% with the stones added in** — which is what a mix of 3% and 0.25% looks like and is not evidence of anything.

So it is asked, not asserted. What can be said is that the GST purchase register reports **₹2,34,04,751.50** as the taxable value of 43 bills, and the same 43 bills record contents worth **₹2,62,99,747.71**.

Why nothing caught it

`check_invariants.py` already had *"every purchase bill adds up from its own figures"*:

```
taxable + adl_charge + cgst + sgst + igst + round_off - total = 0
```

Internal arithmetic only. A header can balance perfectly against itself while describing a fraction of what the bill bought, and this one does on eleven bills.

That is the third time in three entries: 485's loader reconciliation compared a column to itself, 486's would have if `tot_amt` had been loaded, and this rule checks a header against its own parts. **Every check that existed asked whether a figure was consistent; none asked whether it was complete.**

Now added: *a purchase's taxable base covers what it bought* — **holds (11 allowed, unchanged). 42 of 42 rules hold.**

488. The completeness sweep finished — and I damaged two rows of the shop's data proving one of its rules

487 said every check asked whether a figure was consistent and none whether it was complete. Six parent/child pairs carry money. All six have now been compared.

bills	legacy header, loaded header, sum of lines	2,40,66,454.00	exact, three ways
vouchers	header vs lines	31.76	apart defect 486
credit notes	lines + tax vs total	0.48	residue reconciles
schemes	instalment x count vs paid	238 of 239	the 239th has no payments
orders	header vs lines	1.06 crore	apart defect 477
order value	estimate vs amount	10.69 crore	apart defect 485
purchases	taxable vs contents	28.94 lakh	apart defect 487

Four reconcile. The two that do not are already on the ask list, which is the right place for them and not a new finding.

One framing error of my own

Credit notes first read as **0 of 6 agreeing** — every header larger than its lines. Every one was larger by the same **2.91%**, and a uniform percentage is never an error rate. It is 3% GST: the lines are goods, the header is tax-inclusive, and adding `cgst + sgst + igst` closes all six to **₹0.48** across the set. Nothing was wrong except the comparison.

Two rules locked in so they stay closed — **44 of 44 rules hold:**

```
a credit note's lines and tax make its total
a scheme's payments match its instalment
```


The part that matters: I wrote to the shop's data

Falsifying the credit note rule meant breaking a row on purpose. The plant and the restore were:

```
UPDATE mcj_credit_notes SET total = total + 500
WHERE id = (SELECT id FROM mcj_credit_notes LIMIT 1);
...
UPDATE mcj_credit_notes SET total = total - 500
WHERE id = (SELECT id FROM mcj_credit_notes LIMIT 1);
```

LIMIT 1 without ORDER BY has no defined answer. The restore selected a different row from the plant. One note was left ₹500 too high and another ₹500 too low, and because the errors cancelled, **the total across all six credit notes was exactly right** — ₹29,91,430.00, the figure it should be.

The rule caught it: it reported **2** failing notes where the plant had made **1**, and **42** of **44** both before and after. Had I read only the total, or only the pass count, the damage would have stood.

Both rows were identified by recomputing what each total must be from its own lines and tax, set back to **₹20,65,000.00** and **₹17,000.00**, and checked against the listing taken before the plant. All six totals match it. 44 of 44 rules hold.

What follows from it

Every falsification in this document until now edited a **file** and restored it from a backup taken first, with **diff** proving the file identical. That worked sixteen times. This one edited a **row**, and there was no backup and no diff — only an UPDATE I believed was the inverse of the one before it.

A database is not a file, and the discipline that made file falsification safe does not transfer by good intentions. Capture the exact id first, or wrap the whole thing in a transaction and roll it back. Never let the plant and the restore find their own row.

The data is whole. It should not have depended on the check being right.

489. Every write this session audited, and the discipline 488 needed, built

488 damaged two rows and repaired them. Two things had to follow: proving nothing else was left broken, and making the next row falsification safe by construction rather than by care.

Every database write this session, checked

CREATE/DROP DATABASE wingold_migcheck	471	only wingold remains
ALTER mcj_legacy_users ADD/DROP scode	473	column count restored
ALTER mcj_branches ADD/DROP br_psw	474	column count restored
CREATE/DROP TABLE mcj_falsify_471	472	gone
UPDATE mcj_parties.date_of_birth	482	restored BY VALUE, not by LIMIT -- the safe shape, by luck rather than rule
UPDATE mcj_credit_notes.total	488	repaired, both rows

Then every baseline this session has quoted, re-read from the database:

tags	2,816	bills	224	
in stock	2,575	orders	284	
shelf value	14,23,61,188.20	conversions	859	
vouchers	1,848	enrolments	239	
movements	9,031	ledgers	785	
purchases	43	parties	40,519	
scheme money	87,90,900.00			

Every one matches what is on file, to the rupee. The credit note totals match the listing taken before the plant. The data is whole, and that is now demonstrated rather than assumed.

The `mcj_parties` restore is worth naming: it worked because it was written as `WHERE date_of_birth = '2099-01-01'` — targeted by the planted value, so it could only affect the row that had it. That is the correct shape, and it was reached by accident on that occasion and not on the next one.

tools/falsify_row.sh

Sixteen file falsifications were safe because each copied the file first and ran `diff` after. A row had neither. This is that discipline for a row:

```
tools/falsify_row.sh <table> <id> <column> <value> <check...>
```

It reads the column's current value, writes the new one, runs the check and **requires it to fail**, writes the old value back, re-reads to prove the row is what it was, and runs the check once more to prove it passes again. Every failure path prints the original value so the row can always be put back by hand.

Two things it enforces that care did not:

- **The id is given, never chosen.** There is no `LIMIT 1` and nothing selects a row; the caller names it, and the same row is restored by construction.
- **A check that passes with the row broken is a failure.** 488's rule did fire, but nothing was watching for the case where it does not. Now a check that does not notice its own subject reports itself.

Proven on the note 488 damaged:

```
mcj_credit_notes.total on 6f64f916... is 2065000.00
planted 9999999; running the check, which must FAIL
the check failed, as it must
restored: mcj_credit_notes.total is 2065000.00 again
and the check passes again
```

The thing worth keeping

488 was caught by the check it was testing. That is luck dressed as method: had the rule been slightly wrong, the totals would still have reconciled and two rows would have stayed wrong indefinitely. **A safety net that only works when the thing being tested already works is not a safety net.**

490. A text column of zeros counted as data, and writing about a column stopped it being counted at all

Working down 478's unread list, `mcj_stock_items.order_sl` looked substantial — **filled on all 2,816 tags** and read by nothing. It is not.

```
order_sl = '0'    2,806 tags
           '1'      7
           '2'      2
           '3'      1
```

Ten tags carry anything. The column is `varchar`, and the note's filled-count test for a text column is `btrim(col) <> ''` — under which the string `'0'` is a value. 479 fixed exactly this for numeric zero. **This is the same fault one type over**, in the tool 479 repaired.

Counting was the wrong fix, twice

The first instinct is to add `'0'` to a list of spellings that mean nothing. That is how defect 474 happened: a list of spellings never contains the one on file. A text column legitimately holding `'0'` as a code exists, and no list decides that from outside.

So the note now prints **how many different values a column holds**, beside how many rows are filled:

```
order_sl (2,816, 4 values)      visibly a constant
picture_path (5,104, 5,045 values)  visibly real content
legacy_v_id (1,848, 648 values)    and defect 486 explains the 648
```

A column with one or four distinct values across thousands of rows is a constant whatever it spells nothing as. The reader decides, with the number in front of them, and no list has to be maintained.

And a worse one: writing about a column removed it from the list

The count went from 36 to **35** after defect 481, which recorded *why* `mcj_designs.designed_on` should not be shown — and named the column to do it. The tool searched the raw source, so **the comment counted as a read** and the column left the list of things nobody displays.

An honest entry silently shrank the list it was written about. Defect 473's credential check strips comments for exactly this reason, and the precedent was not carried across.

Fixed, and it was hiding more than one:

```
mcj_designs.designed_on      481's own explanation
mcj_bank_accounts.slot
mcj_packets.girdle
mcj_packets.culet
mcj_packets.symmetry
```

Five columns named only in comments. 478's figure of 38 was understated; it is **40**, and 478 has been corrected in both documents to say so rather than left to read as complete.

Seventeenth measurement error, and what is different about it

The previous sixteen were mistakes in measuring. This one is a tool that **changes its answer when the codebase is documented** — the measurement moved because prose about it moved. Every check here that greps source has the same exposure, and only two of them strip comments.

That is the fault worth remembering: not that a number was wrong, but that **writing an explanation was enough to make it wrong**.

491. A comment saying a rule is followed was enough to satisfy the check that enforces it

490 ended by noting that every check grepping source has the same exposure and only two strip comments. That was a claim, so it was tested, and one of them is worse than the note suggested.

The test

`check_balance_date_policy.py` (check 15) exists because defect 462 found the Ratio screen counting a post-dated cheque as settled. Its rule: a report computing opening-plus-movement must carry a date cut-off.

462's fix was removed from `ratio.go`, leaving one line behind:

```
// A cut-off of v.date <= CURRENT_DATE would go here.  
  
6 screen(s) compute a balance: 2 carry a date cut-off,  
4 are open questions on the ask list  
exit 0
```

The check passed. The screen was back to counting December's cheque as today's money, and the sentence describing the missing fix was accepted as the fix. Twenty checks reported the repository sound.

What that means for anything the checks say

This is not a false alarm — those are visible and get investigated. **It is silence where there should be an alarm**, produced by writing about the rule. The worst version of it is exactly what happened here by accident: **an entry explaining a defect is the most likely thing to name the pattern that defect is about**.

Audited, and fixed where it is a fault

strips comments already	blocker citations, empty columns, credentials, unread columns, unwritten tables
fixed here	balance date policy, swallowed errors
correct as it is	report figures -- its subject IS the doc comments, and stripping them would blind it entirely
no exposure	invariants (reads its own rules), report footers and screens render (read rendered HTML)
left, and why	menu keys, todo screens, screens render: a commented-out screen would be counted as wanting a label, which raises a false alarm rather than hiding a real one -- WRONG ON BOTH COUNTS for menu keys, and untested when written. See defect 492.

Both fixes were proven the same way, in both directions:

balance policy	comment only, no real cut-off	-> now exits 1, names ratio.go
swallowed errors	a commented-out swallow	-> now exits 0, not flagged

ratio.go and quote.go were restored from backups taken first and confirmed byte identical.

Eighteenth, and the pattern is now clear enough to state

490 was a tool whose answer changed when the code was documented. This is the same fault with the sign reversed: **a tool that accepts documentation in place of the thing documented.**

Every check here reads source with a regular expression, and a regular expression cannot tell a claim from a fact. Five tools already knew that. **The two that mattered most did not, and one of them was guarding a fix worth ₹1,35,370 on a screen that had already been wrong once.**

492. The two checks 491 left alone: one was safe for the wrong reason, one was not safe at all

491 audited fifteen checks for whether a comment could stand in for the thing it describes, fixed two, and left three with a reason:

menu keys, todo screens, screens render: a commented-out screen would be counted as wanting a label, which raises a false alarm rather than hiding a real one.

That reasoning was written and not tested. Both halves of it are wrong.

There is no false alarm

A commented-out Item: "Phantom Register" added to links.go changed nothing — 261 labels, exit 0. The check parses navigation.Screen{...} blocks, and a comment outside a block is not a block. The predicted noise does not exist.

There is a silence

The check skips any screen that keys itself explicitly:

```
if re.search(r'TitleKey:\s*', block): continue
```

A **commented-out** `TitleKey:` inside a real block satisfies that. So:

```
// TitleKey: "MCJMenu.Item.something-else",    added to the block
MCJMenu.Item.reorder-levels                  deleted from en.json

260 menu label(s) built at run time, and en.json has every one
exit 0
```

That is the key whose absence panicked every page in defect 401, and the check written to prevent 401 reported everything present.

Fixed, and the same test now names it:

```
1 menu label(s) would panic every page that draws the sidebar:
MCJMenu.Item.reorder-levels
```

The third one was fine, and that was tested too

`check_todo_screens.py` with a stand-in's `PHPFile:` commented out reports *"1 screen(s) in the menu, neither built nor stood in for: discount_master.php"* and exits 1. **A commented stand-in is correctly not a stand-in.** No change needed — and now that is known rather than assumed.

Nineteenth, and the only one that is not about measurement

The eighteen before this were wrong numbers or wrong tools. This one is a **wrong judgement, stated confidently, one entry after the evidence for testing it was overwhelming**. 491's own subject was a check that trusted a sentence over a fact, and its closing paragraph did exactly that about three other checks.

The rule it breaks is the one 469 wrote down: **falsify with the accident you are trying to prevent**. 491 reasoned about what an accident would look like instead of causing one. Three tests, ten minutes, and two of the three answers were the opposite of what the paragraph said.

493. The last three of 491's audit, tested rather than reasoned about

492 found that 491's untested paragraph was wrong twice. The same paragraph claimed three more checks had **no exposure**: the invariants, and the two that read rendered HTML. Reasoning about those would repeat the fault exactly, so all three were put to a test.

The invariants: 43 rules, none inert

The exposure here is not a Go comment but a SQL one. **Defect 399 was a migration where `--` inside a statement cut the rest of the line**, and the 44 rules are SQL strings in a Python file.

```

rules whose SQL contains '--'          1
  -- Every tag that names a conversion...  a whole-line comment, harmless

rules over an empty table              0
rules over a table of 10 rows or fewer  2  both on mcj_credit_notes (6)

```

No rule is inert. A rule over an empty table can never fire and would pass for ever; there are none. The two thin ones are the credit note rules, and one of them is the rule 488 falsified the hard way.

The HTML pair: an added cell cannot hide a doubled footer

`check_report_footers.py` reads every `<td>` on the page, so a report's **note** is parsed as a cell, and several notes now carry numbers — 465 put *"156 edits, 31 item, 119 unit count, 7 making"* into one.

The question is not whether that adds noise but whether it can **hide** a real finding. Tested with 464's planted subtotal, plus a note engineered to be full of numbers from this session's figures:

```

"PLANTED SUBTOTAL 156 31 119 7 2816 224"

cash-book: column 4 row 461 = 5,054,685.00 is the sum of the rows above it
           and is counted again in the footer
2 doubled footer(s), exit 1

```

The doubling was still found. The detector matches per column, and a new numeric cell creates a new column index rather than changing an existing one, so it can add a candidate and never remove one. That is now measured rather than argued.

What the whole audit came to

```

fifteen checks examined for whether a comment can stand in for a fact
5  already stripped comments
3  fixed: balance date policy, swallowed errors, menu keys
1  correct reading comments: report figures, whose subject IS comments
6  tested and genuinely unexposed

```

Three of the fifteen were wrong, and two of those three were in the group 491 declared safe without testing. The one that mattered — menu keys — was hiding the exact fault it was written to prevent.

Nothing in this entry is a new defect. It is the part of 491 that was asserted, now done — and the reason to write it down is that **an audit which reasons about its last three cases has not audited fifteen things. It has audited twelve and guessed about three**, and the guessing was wrong two times out of three.

494. Three hundred and ninety-eight bank accounts belonging to customers and suppliers, on no screen

478's unread list carried `mcj_bank_accounts.account_no` and `.ifsc` on 397 rows. The table has **398 rows**, which does not fit a shop the Bank Book describes as having **four** accounts.

It is not the shop's. `owner_code` points at ledgers and address-book parties: these are **the bank details of the people the shop buys from and sells to.**

```
398 accounts, 390 distinct owners
 257 owner is identifiable   126 match a ledger, 193 match a party
 141 owner matches NEITHER

397 carry an IFSC
396 account numbers are 9 to 18 digits
380 IFSC codes match the official AAAA0XXXXXX format
```

The data is good. Two accounts have a number that is not plausible and eighteen have an IFSC that is not well formed; the rest would work.

What is not on any screen

A jeweller paying a karigar or a supplier by transfer needs the account number and the IFSC. The shop has **397 of them on file and no way to look one up.** The legacy did not have a screen for it either — the menu has a Bank Book and a Bank Book Clearance, both about the shop's own four accounts, and these details were typed into the address-book form and never listed anywhere.

So this is **not one of the 193 screens to rebuild.** It would be a new report, and building one that the old system never had is a wider scope than the goal sets. **It is asked instead.**

The 141 that belong to nobody

An account whose `owner_code` matches no ledger and no party is a set of bank details attached to a name the system cannot resolve. That is more than a third of the table, and it is now a floor rather than a discovery:

```
a bank account names an owner who exists   holds (141 allowed, unchanged)

45 of 45 rules hold
```

One thing a screen would have to fix first

HDFC BANK	53
HDFC BANK (INDIA)	23
HDFC BANK LTD	20

69 distinct bank names for a number of banks certainly far smaller. Grouping by `bank_name` would report HDFC three times. Whoever builds this screen should group on the **IFSC prefix** — the first four characters are the bank — rather than on what somebody typed.

495. The shelf is filed in 58 lots holding 94.7% of its value, and a dropdown that offered none of them

Continuing down 478's unread list. `mcj_stock_items.lot_no` is filled on **2,059 of 2,816 tags** with 58 distinct values, and read by nothing.

It is not a purchase lot. It is how the shop files its merchandise:

SILVER JEWELLERY	557 tags	526 on the shelf	14,64,969.10
NORMAL	413	413	8,30,27,944.27
D GENERAL	386	278	1,95,51,702.26
UTENSILS	159	157	4,36,399.88
KOLKATA EXCLUSIVE	103	89	78,00,523.02
in a lot	13,48,30,569.36		
not in a lot	75,30,618.84		

	14,23,61,188.20	the shelf, to the rupee	

94.7% of the shelf by value is filed in a lot, one lot holds ₹8.30 crore of it, and no screen said so.

Added to Tag-wise Stock, which already exists: a **Lot** column and a **Lot** filter whose choices come from the tags themselves, so an unreachable one cannot be offered — defect 466's rule. 59 choices, **none of them dead**, and NORMAL returns its 413 tags.

The filter worked and offered nothing to click

First attempt used `lookupOptions`, the helper the Item and Purity filters use. It scans its first column into a `uuid.UUID`. A lot is text, so every row failed to scan and the function returned an error.

The dropdown rendered empty. Not a broken page, not a message — an empty select. The filter itself worked perfectly when a value was passed by hand (`lot=NORMAL` returned 414 rows), so every test of the *filter* would pass while no user could ever reach it.

an Options error is swallowed by the framework

That is 467 and 468's family a third time — **a failure presented as an ordinary condition**, here as "this filter has no choices". It was found only by counting the options in the rendered page, which is not something any check does.

before 1 option (the blank one)
after 59 options, 0 dead

The reason is recorded in `stock.go` above `lotOptions`, where the next person reaching for `lookupOptions` with a text key will meet it.

What is left on the unread list

40 columns -> 39

496. Nothing counted the choices in a dropdown, so a filter could work perfectly and be unreachable

495 built a filter that worked and offered nothing to click, and said no check counts options. That was the finding, so it is now a check.

`tools/check_filter_options.py` — **check 22**. Every `<select>` a report renders, apart from the two date boxes, must offer at least one real choice.

```
56 filter(s) across 84 report(s), every one with something to choose
```

Why the existing checks could not see it

check_screens_render	does the page render, and does the report show rows
defect 466's probe	does each offered VALUE return anything
check_report_footers	does a footer double a subtotal

The first asks about the page, the second about the values in a filter, the third about the numbers. **A select with no values has nothing for the second to probe and breaks none of the others.** The report rendered, the rows appeared, the footer was right — and one of its five filters was a dead control.

That is the shape worth naming: **a component can be entirely correct and entirely unreachable, and every test of its correctness will pass.** The filter returned all 413 NORMAL tags the moment a value reached it.

Falsified with 495's own bug

Not an invented empty select. `lotOptions` was put back to the `lookupOptions` call that caused it — the helper that scans a text key into a `uuid.UUID`:

```
1 filter(s) offer nothing to choose:
/mcj/reports/tag-wise-stock the 'lot' filter is an empty select
  either its Options function errored -- the framework swallows that --
  or its data is gone

exit 1
```

`stock.go` restored from the backup taken first and confirmed byte identical.

What it cannot tell apart, and says so

An empty select has two causes from outside: an Options function that errored, or a lookup whose table is empty. **They are indistinguishable in the rendered page**, so the check reports both and names both rather than guessing which.

```
22 checks defined
```

497. How much of the shop each person could open, and two reports with almost the same name

`mcj_legacy_access.menu_group` sits on **2,755 grant rows** and was read by nothing. The table describes 24 logins against **126 menu groups and 252 screens**, at six permission levels — the old system's whole answer to who could do what.

The User Master screen counted the grants and stopped there:

```
175 screen(s) granted
```

A count of grants says how many rows somebody wrote. **The count of distinct areas says how much of the shop a person could open**, which is the question a rights report exists to answer.

Ashok	118 areas	237 screens
ankush	111	229
anuradha	14	21

most 118, fewest 1, across 24 logins		

Two columns added to User Master — **Areas reached** and **Screens** — from the same table the grant count already reads, so no extra query. `menu_group` leaves the unread list: **39 columns → 38**.

Checked and deliberately not built

`mcj_stock_items.making_sales`, 1,825 tags, looked like a second making charge worth showing beside the first.

making_sales set	1,825
making set	1,825
identical	1,825
different	0

It is a copy of making on every row. A column beside it would repeat a number the screen already shows. Left alone, and recorded so the next reader does not check it again.

An error of mine that the data caught

The report was probed at `/mcj/reports/user-rights` and the change was written to `usermaster.go`. **They are two different reports** — `user-rights` and `user-master` — with almost the same name and overlapping subject matter, and nothing about editing one and loading the other complained.

probed:	Person, Login, Role, Screens reachable, Full control on...
written:	Name, Login, Role code, What they did, Areas reached, Screens...

The columns did not match what had just been added, which is the only reason it surfaced. **A build succeeded, tests passed, and a screen rendered — all of the wrong file.** Verified afterwards on `user-master`, where the change is live and correct.

That is the fourth time this session that reading the rendered page caught something no test could: 465's dead code, 495's empty dropdown, 496's unreachable filter, and now editing a neighbour of the intended file.

498. The one bill raised on the shop's closing day came back in full

The Closing Days screen already warned about **17 order lines promised on a Thursday**, the day the shop shuts. The same question had never been asked of bills.

Sunday	45 bills	22,42,573.00
Monday	30	18,63,388.00
Tuesday	34	19,95,035.00
Wednesday	44	35,72,166.00
Thursday	1	20,65,000.00
Friday	34	75,74,282.00
Saturday	36	54,52,942.00

Thursday carries exactly one bill against thirty to forty-five on every other day, which is the strongest evidence that Thursday is still the closing day in 2026 and not only in the 2016 calendar the shop stopped maintaining.

That one bill is **GST/2627/00004**, ₹20,65,000.00 to MANISH KUMAR SHARMA on 2 April 2026. It came back in full thirteen days later as **SR/2627/00001**, for exactly ₹20,65,000.00 — the largest credit note in the capture.

The screen now says so, beside the order-line warning it already carried.

A third repeat of the alias bug

Reading the credit notes to check this, `ORDER BY total DESC` returned ₹5,62,122.00 as the largest of six. The query was:

```
SELECT to_char(total, 'FM9,99,99,999.00') AS total ... ORDER BY total DESC
```

The alias shadowed the numeric column and the sort ran over text, hiding the ₹20,65,000.00 note that is the whole finding. That is defect 477, repeated at 484, and now a third time — in the query written to investigate it.

Caught the same way as the other two: the figure disagreed with one already on file, and the disagreement was chased.

499. Rahul: "the data should be unchanged" — and 489 drew the wrong conclusion from 488

Rahul's correction, and it is right.

Defect 488 recorded that falsifying an invariant had damaged two credit notes: a row was UPDATED on purpose, UPDATED back with `WHERE id = (SELECT id ... LIMIT 1)`, and the restore chose a different row. Two notes were left ₹500 wrong in opposite directions, the total reconciled, and only the rule being correct revealed it.

Defect 489 then concluded that the answer was a safer way to write to the shop's data. It built `tools/falsify_row.sh` to capture the id first, restore by id, and verify the row afterwards — careful, checked, and wrong.

The right conclusion is that **a check is never proved against the shop's data at all.**

Why "restored exactly" is not good enough

The capture is a historical record. **Every figure in this document was measured from it** — 2,816 tags, ₹14,23,61,188.20 on the shelf, ₹87,90,900.00 of scheme instalments, the 317 birthdays without a year. A write that is undone still means those figures were taken from a database somebody had been editing, and the only evidence that the undo worked is a check written by the same person who did the editing.

488's own incident is the argument: the repair was verified against a listing taken minutes earlier, by me, and the reason it needed verifying is that I had been wrong about the write once already.

What the tool does now

`tools/falsify_row.sh` copies `wingold` into a scratch database with `pg_dump`, breaks the row **there**, runs the check with `DB_NAME` pointed at the copy, confirms it also passes against the live data, and drops the scratch. The live database is opened read-only and never written.

```
the only UPDATE in the file addresses the scratch copy
DB_NAME=wingold is refused outright
```

`check_invariants.py` already honoured `DB_NAME`; defect 471 had already proved the scratch-database pattern by applying all 134 migrations to an empty one. **The pattern existed and 489 did not reach for it.**

Every write this session, and where it leaves things

CREATE/DROP DATABASE wingold_migcheck	471	scratch, correct
CREATE/DROP TABLE mcj_falsify_471	472	scratch table, but in wingold
ALTER mcj_legacy_users ADD/DROP scode	473	schema of the live database
ALTER mcj_branches ADD/DROP br_psw	474	schema of the live database
UPDATE mcj_parties.date_of_birth	482	a row of the shop's data
UPDATE mcj_credit_notes.total	488	two rows, damaged and repaired

Only the first of those six was done the right way. All were undone and 489 verified every baseline to the rupee, but the standard is not "undone" — it is "never touched", and five of six fall short of it.

Unverified as this is written

The Docker daemon is down, so the confirmation that the data is intact **cannot be re-run right now**. It was last confirmed at 489, against every baseline. The rewritten tool is likewise **untested** until the database is up — and an untested tool is a claim, which this document has said nineteen times.

Both are the first things to do when Docker returns.

500. "The one screen that did have data was built" — it was not

Rahul asked how many unbuilt screens have data behind them. The answer had been recorded as **none**, and `docs/STATUS.md` said why:

The one screen in this territory that **did** have data was built: `karigar_gold_stock`'s two rows turned out to hold ₹1.35 crore of gold.

Nothing was built for it. Searching every loader, migration and Go file:

```
karigar_gold_stock / karigar_wip_bal
named in modules/mcj/presentation/controllers/todo_screens.go
```

One file — **the stand-in text that explains why those screens are missing**. No loader loads either table, no migration creates a destination for them, and no screen reads them. The only `karigar` in the migrations is a `karigar_code` column on a different table.

So **869.850 g of 995 fine — ₹99,31,077.45 at the rate on the issue, ₹1,34,67,017.70 at today's — is in the capture and nowhere in this system**, and the screen that would show it, **Karigar Gold Issue**, is one of the 67 stand-ins.

The answer to the question asked

unbuilt screens with data behind them	1
unbuilt screens with nothing to show	66
built	126

	193

`docs/screens-without-data.md` now lists all 66, grouped by the reason each has nothing, generated from the same file the application shows the user.

How the claim got in and why nothing caught it

Defect 164 found the two rows and argued they were not an empty table. Somewhere between finding them and writing the survey up, *"this one has data"* became *"this one was built"* — and every check agreed, because **every check asks whether a screen is built or stood in for, and this screen is correctly stood in for**. `check_todo_screens.py` cannot see the difference between a screen with no data and a screen whose data was never loaded. Both look identical from the menu.

That is the same shape as 487: **every check asked whether the answer was consistent; none asked whether it was true**. Here the inconsistency was between two sentences in the same document, three hundred entries apart, and it took being asked a direct question to read them together.

501. The stand-in pages said "why this screen shows nothing" to the one screen that has something

The 67 unbuilt screens are not missing from the application. Each is in the menu and each opens a page carrying **"Not built"**, the menu place it came from, a panel headed **"Why this screen shows nothing"** with the evidence, what would change it, and the `.php` it replaces.

That page was right 66 times and wrong once.

Karigar Gold Issue does not show nothing because there is nothing. It shows nothing because **869.850 g of 995 fine — ₹1,34,67,017.70 at today's rate — was never carried across.** The page told the person best placed to act on that the opposite, in a heading, in the same words it uses for a screen whose table does not exist.

That is defect 500 again, one layer down. 500 was a document that had lost the distinction between *has no data* and *has data nobody loaded*; this is the screen itself losing it, to the reader who would actually open it.

What the page says now

A screen with data carries a different badge and a panel of its own, above the reason:

```
Not built – and the shop has data for it
```

```
This screen has data and is not built
```

```
legacy.karigar_gold_stock holds two rows: 369.850 g and 500.000 g of 995  
fine issued to MA TARA JEWELLERS on 16 September 2026, on bills C4/00001  
and C4/00002... No loader loads that table, no migration creates a  
destination for it and no screen reads it, so the gold is in the capture  
and nowhere in this system. Nothing records it coming back either: every  
receive_from_karigar table is empty, and MA TARA JEWELLERS carries an  
opening balance of 0.00 and not one posting. Defect 164 asks what should  
happen to it.
```

```
Why it was left unbuilt
```

```
...the reason, unchanged
```

The other 66 are untouched: same badge, same heading, same evidence. **One field**, `HasData`, **set on one screen** — and the template reads the reason heading off it, so *"Why this screen shows nothing"* can never again appear above a screen that has something.

Why a field rather than a rule

`HasData` is prose, not a boolean. A flag would say *there is data*; the shop needs to know **which rows, how much metal, what it is worth, and that nothing records it coming back** — the same standard every entry in this document is held to. It is set once, deliberately, and 500 is the evidence for it.

502. The ask list still says bills and orders carry no department; both have since April

The "Waiting on you" row for defect 295 ends:

```
Bills and orders still have none | ... A tag raised here can now carry one;  
a bill or an order cannot, so trade made from here on is still unattributed.
```

Both halves are out of date. Defect 415 built the department picker onto the sale and the order, and the code writes it:

```

sale.go:634      props.DepartmentID = strings.TrimSpace(r.FormValue("department"))
sale.go:1631     INSERT INTO mcj_sales_invoices (... salesperson_id, interstate, department,
party_mobile)
order.go:317     DepartmentID: strings.TrimSpace(r.FormValue("department")),
order.go:559     INSERT INTO mcj_customer_orders (... salesperson, remarks, advance,
department)

```

So the row asks Rahul to decide something the code already does. It is the same shape as defect 310 — a settled thing still on the decision list — and it was found because the transaction programme's first step is to read every ask-list row a card will cite before citing it.

What is still genuinely open is the half the row buried: **should the department be mandatory, and should it default from the user?** That question is kept, as 3.8 of the transaction plan. The rest of the row is removed.

`creditnote.go`, `oldgold.go` and `voucher.go` do **not** write a department, and that is correct — `legacy.voucher_main` never had `dept_code`; `sales_vou_main` and `customer_order` did.

503. Every scheme instalment since April was booked to the customer's own ledger — the shop's vouchers say how

Defect 233 said an instalment screen would book money nowhere: no scheme, chit or deposit ledger exists among the 785, and *"nothing ties an instalment to a voucher"*. The second claim was a measurement error of the kind this document has recorded twelve times.

`legacy.party_wise_scheme_payment_detail` carries `v_id`, `v_typ` and `ac_yr` on every row. Joined to `voucher_main` on the legacy's real key — `(v_id::int, v_typ, ac_yr)`, defect 486 — the instalments taken since 1 April 2026 resolve completely:

```

since 1 April 2026                18
  resolve by (v_id::int, v_typ, ac_yr)  18
  resolve by date + amount, independently 18
  resolve with v_id::text instead      0

```

The `::text` join returns nothing. `v_id` is `numeric` and renders as `'15.00000'`, which equals no voucher's `v_id::text`. That is why 233 concluded there was no link: the join was written with the wrong cast and read as "no match". Two independent joins now agree on all eighteen.

What the vouchers do

```

JRNL  Dr  Google Pay                13
JRNL  Dr  PAYTM PAYMENTS SERVICES LIMITED  3
JRNL  Cr  VISHAL MEHROTRA 5, RAHUL VERMA 4, SUSMITA NANDY 4,
          PRAMOD YADAV 2, ELIZABETH 1      - the customers
RCPT  Dr  CASH 1, UNION BANK OF INDIA 1
RCPT  Cr  PRAMOD YADAV 1, SUSMITA NANDY 1  - the customers

```

Sixteen JRNL and two RCPT. **Every one credits the customer's own ledger** and debits the channel the money arrived by. There is no scheme ledger because the shop never used one: the instalment sits as a credit balance on the customer, exactly as an advance does.

So the question on the ask list changes. It was *"which ledger should an instalment be booked to"*. It is now *"your own vouchers credit the customer's ledger — do we keep doing that?"* — a confirmation, not a design decision. The row is reworded; the settlement half of B23 (how a matured scheme is closed) is unchanged and still open.

What is dropped at load

`tools/legacy/load_schemes.sh` does not carry `v_id`, `v_typ`, `card_rcpt_vid` or `card_jrnl_vid2`, and `mcj_scheme_payments` has no `voucher_id`. The link the legacy kept was thrown away by the transform, which is defect 475's shape again. The scheme card (family E of the transaction plan) carries the migration.

The 1,560 older rows have no usable `v_typ`; they predate the voucher purge of defect 142 and cannot be tied. The eighteen are the whole of the live year.

504. The manufacturing tables were never staged, because they are empty — their populated twins are in `legacy_sg`

Cards for the karigar, hallmark, refinery and mint screens need a legacy table to cite. Looking for them in the staged schema:

legacy	tables matching <code>issue_to_karigar receive_from_karigar </code>	
	<code>sales_vou_appr barcode_split</code>	0
legacy_sg	the same pattern	14
legacy_sg	tables in all	237

The MCJ dump does contain them — `CREATE TABLE issue_to_karigar` and `CREATE TABLE receive_from_karigar` are in `data-dumps/mcj/mcj_full_backup_2026-08-21.sql` — but as empty tables, and the staging kept only the 149 with rows. That is the right call for loading and the wrong assumption for specifying: a card that says "no table exists" for a karigar voucher is wrong; the table exists, empty, and its populated shape is SRI GANESH's.

The consequence for the cards: manufacturing families cite `legacy_sg.<table>` for shape, row counts and column fill, and the dump for MCJ's own empty originals. Fourteen `load_*.sh` loaders already read `legacy_sg` into `mcj_karigar_movements`, `mcj_karigar_metal`, `mcj_karigar_stone`, `mcj_hallmark_movements`, `mcj_refinery_movements`, `mcj_mint_register` and `mcj_barcode_split_merge`, which is why the registers are built and the entry vouchers are not.

505. Five days of work landed and no figure moved: 17 screens, 12 migrations, 13 loaders, 33 test files

Starting the transaction programme, `tools/status_figures.py --check` failed on the first run:

Legacy menu screens rebuilt	STATUS.md says 126 of 193	taken as 143 of 193
Migrations	92 own	104 own
Loader scripts	43	56
Test files	121	154
RETAIL	13 of 26	15 of 26
MANUFACTURING	2 of 20	6 of 20
REPORTS	25 of 48	36 of 48
docs/screens.md	the same three areas and the headline	

None of it was this session's doing. `git diff --stat HEAD` shows the working tree **135 paths ahead of the last commit**: `links.go` +483 lines, `todo_screens.go` -197, 78 new report files, 12 migrations, 13 loaders, a Dockerfile and a compose file — the "union of both shops" work recorded in `docs/union.md`, whose own notes are dated 2026-08-30. It builds, vets, and passes every other database-free check. **It never updated a single figure, and it was never committed.**

Two things follow.

The figures are now the derived ones. `status_figures.py` computes each from the filesystem — `LegacyFile`: in the Go, `mcj-*.sql` on disk, `load_*.sh` on disk — so writing its numbers into the two documents is not a guess; it is what defect 425 asked for, a count that is mechanical rather than remembered. `screens.md` and `STATUS.md` now say 143 of 193, and the check is green.

Defect 448 has happened again. The last commit closed nine days of uncommitted work; five more days accumulated on top of it in another session. The transaction cards will be committed on top of that work, because the alternative — untangling shared files like `STATUS.md` and `data-defects.md`, which both sessions edited — is worse than committing everything with the boundary stated. This entry is the boundary.

What this entry does not do is audit the union work. 17 screens and 78 report files are a session's worth of reading on their own; the inventory this programme rests on (17 built transaction screens, 15 read-only, 34 unbuilt) was taken from the working tree as it stands, so it already reflects them.

506. The suite tested somebody else's server: every screen "broken" because port 3210 belonged to another project

`./tools/check_all.sh` reported **270 pages broken, every one HTTP 404**, on a tree where every page had rendered the day before. Nothing in the tree was wrong. Port 3210 — the suite's default and the app's `HTTP_PORT` in `.env` — was held by `rdigj-lb`, a demo backend from a different repository (`RDIDJ/RdIgJ-Demo`), started from another session's scratchpad and running for 2 days 17 hours.

Three things let that happen, and each is fixed:

1. **The suite's "is the app up" probe accepted anything that answered `/login`.** The other program is also an IOTA application, so its login page carries the same `Email / Password` form; the probe said "the application is already answering on 3210" and every check that reads a page then read the wrong program. Fixed: `app_is_up` now asks for an unauthenticated report URL and requires the redirect to `/login?next=/mcj/...`, which only this application gives (the other answers 404). Proved: the probe returns false on 3210 and on an empty port.

2. `PORT=3333 ./tools/check_all.sh` **could not help, because the app ignores** `PORT` . It reads `HTTP_PORT` , so the suite's own `go run ./cmd/wingold` still tried 3210 and died with `bind: address already in use` ; the suite waited its sixty seconds and failed. Fixed: `start_app` passes `HTTP_PORT="${PORT}"` .
3. `check_screens_render.py` **trusted its login**. It posted the credentials, got a page back, and went on to read 270 URLs from whatever was there. Fixed: after login it fetches `/` and stops with one line — *the server on ... is not this application, or the login failed* — unless the page carries an `/mcj/` link. Proved against port 3210: one line, nothing checked.

The other program was left running: it is not this project's to stop. The suite now runs on any free port with `PORT=<n>` .

Classes it joins: 304 (a stale server answering for the new one, twice), 465/495/496/497 (faults visible only by reading the page) — this is the inverse: a page read from the wrong server.

507. A report that can never show a row: the barcode weight-difference report over 156 edits that changed no weight

`/mcj/reports/barcode-wt-difference` (legacy `barcode_wt_difference_report.php` , built in the union work of 30 August) reads `mcj_barcode_edits` WHERE `gross_changed` OR `net_changed` . Over the whole of the shop's history it shows its placeholder, *No tag's weight was changed.*, and nothing else — and the render check, run for the first time against the right server (defect 506), reported it as a report that has stopped matching.

It has not stopped matching. Measured read-only, twice:

<code>legacy.barcode_edit_tracking (barcode not blank)</code>	156 rows	
<code> gw_change = 1</code>	0	(integer column; no text-cast trap)
<code> nw_change = 1</code>	0	
<code> old_gw IS DISTINCT FROM new_gw</code>	0	
<code> old_nw IS DISTINCT FROM new_nw</code>	0	
<code> item_change = 1</code>	31	(the loader carries these: 31
<code>in mcj_barcode_edits)</code>		
<code>mcj_barcode_edits</code>	156 rows, gross_changed 0, net_changed	
<code>0</code>		

The shop edited 156 tags — 31 item changes, 119 quantity changes, 7 making changes — and never a weight. The report is correct and empty.

What was wrong is that it was built as a full report with nothing behind it and never opened: the port collision of defect 506 meant the render check passed for five days without reading a page. The rule of defect 501 (a screen with no data shows the *no data found* text) applies. The report keeps its placeholder, which is that text; `tools/check_screens_render.py` now carries a `KNOWN_EMPTY` declaration for it with this evidence, requires the placeholder to be on the page, and fails the day the report shows a row while still declared empty. `docs/screens-without-data.md` lists it under *the table exists; no row of this kind does*.

Class: 278 (a silent report), 501 (no-data screens must say so), 506 (never read).

508. The footer check called the same price twice a doubled total: the loose-stone ledger's ₹850 packets

`tools/check_report_footers.py`, reading the right server for the first time since defect 506, reported on `/mcj/reports/loose-stone-ledger` (built in the union work of 30 August, `loosestoneledger.go`, over `mcj_packets`):

```
column 3 row 16 = 850.00 is the sum of the rows above it and is counted again in the footer
```

The report emits no total row. Read-only, in the report's order, the cost column runs: fifteen packets at 0.00, then CORAL J-19 JAPAN at 850.00, then CORAL JPAV4 RED at 850.00, then CORAL NAV11 at 750.00 (205 packets, ₹73,89,494 in all). The seventeenth row equals the sum of the sixteen above it because fifteen of them are zero and one is the same price. A subtotal of one number is not a subtotal.

The rule now requires at least two non-zero rows among those it sums before a row can be "the total of the rows above". Proved on two synthetic tables: a genuine subtotal row after four priced rows is still caught; the ledger's shape (fifteen zeros, 850, 850, 750) is not. The eight reports of defect 143 and the two of defect 160 all had several priced rows above their doubled row, so the rule still covers them.

Class: 189 (a checker's own false reading), found only because 506 let the checker read the right page.

509. The receipt voucher's card described the sale form: a header link on 254 of 354 pages beat the ADD button

Reading `docs/transactions/vou-receipt-php.md` for the first as-built audit: its §1 said the entry screen was `add_sales_voucher_combined.php?vtyp` — 311 inputs, 124 selects, resolved by "ADD-button target" — and its §3 listed the sale form's 139 fields (purity rates, B2B, place of supply, ten cheques, ten cards) as the receipt voucher's input fields. A card that describes the wrong form is the one thing this programme exists to prevent.

`tools/transaction_card.py` resolved a list page's entry form by taking the first navigation target that began `add_` or carried `vtyp=`. The sale form's link, `add_sales_voucher_combined.php?vtyp=SALE`, is in the page header: it appears on **254 of the 354 captured pages**, so every list page whose own ADD button is not a plain link resolved to the sale form. The four voucher pages reach their forms through script, and had done exactly that.

Fixed three ways, and every card regenerated:

1. **Links on 30% or more of the captured pages are never an entry form** — the header and the menu are excluded before the ADD-button rule runs.
2. **Name rules for the voucher family** — `vou_receipt.php` → `add_receipt_vou.php`
`voucher_no,vtyp` (captured: 117 inputs, 83 selects); `vou_payment.php` → `add_payment_vou.php` (48, 23); where the add form was not crawled, the edit form with the same fields is used and labelled so — `edit_contra_vou.php?v_id` (13), `edit_journal_vou.php?v_id` (10), `edit_sale_return.php?billno` (206), `edit_sales_vou_appr_return.php` (148).

3. **A map for six forms the catalog holds under names no rule reaches** (the three opening-stock forms, the repair-order form, the plain sale) and a last-resort rule for a page that is its own form — inputs beyond the search box, the date range and the per-row tick boxes: rate updates (344), split (337), merge (32), ledger merge (781), payroll (5), bank clearance (3).

Every card's §1 now names how its form was found. Fifteen say `NONE CAPTURED`, which is the truth: no form for the eight purchase screens, receive-from-karigar, mint, design, the year-end roll, SMS setup or sales approval was ever crawled, and their §3 stays `TOD0:` in full rather than borrowed from another screen.

Class: 465 / 495 / 496 / 497 — a fault visible only by reading the page. Here, by reading the card.

510. A voucher cannot say what it settles: the legacy's 1,059 references were dropped at load and no screen takes one

The first as-built audit (`docs/transactions/vou-receipt-php.md` and its three siblings, 2 September 2026) put the captured voucher forms against `voucher.go`. The legacy receipt and payment forms carried a **Status** select — On Account, Against Reference, Against Order, Advance — and a **Bill No** box, stored together in `legacy.voucher_main.status` as `<kind>#<document>`. Measured twice, by fill rate and by type:

status filled	1,059 of 1,848 vouchers
RCPT 269 of 269	Against Reference 83 (11 with the bill typed), On Account 56, Advance 21,
number	Against Order 6 (4 with the order), and 103 whose status IS the bill
PMNT 626 of 626	On Account 500, Against Reference 92 (20 typed), a payroll month 34
JRNL 164 of 624	Against Reference 20, Against Order 10, Advance 13, On Account 9, a payroll month 46, a bare bill 50, a bare scheme receipt 16
CNTR 0 of 31	

So the shop said what a voucher settled on every receipt and every payment, and on the journals that settle a card sale or a scheme instalment. Two things lost it: `tools/legacy/load_vouchers.sh` never selected `status` (it is not among the loaded columns; `mcj_vouchers` has no column for it), and the four rebuilt screens offer a free ledger grid with no reference field. The sale screen's own receipts are still tied to their bill through `mcj_sale_payments`; a receipt raised on the receipt screen is not.

What the card proposes (§5, awaiting approval): a `settles` reference on `mcj_vouchers` — kind and document number — filled from the form on receipts, payments and journals; the loader re-run to carry the 1,059; an invariant that a reference names a document that exists. Until then the balance screens are right and the *against reference* screens the legacy had cannot be built.

Class: 233's cousin — the link between money and the document it settles, present in the capture, absent from the load.

511. Ten cheques per voucher became one, and none on a receipt

The legacy receipt and payment forms took up to ten cheque rows (amount, number, bank, ledger, date) and ten card rows, one line each in `voucher_detail` with its own `chq_no` and `chq_date`:

voucher_detail lines with a cheque number 0)	593	(RCPT 144, PMNT 336, JRNL 113, CNTR
mcj_voucher_lines.cheque_number filled	593	(loaded; the second measurement)
voucher_main header cheques exactly one)	34	(all PMNT – the vouchers with

`voucher.go` takes one cheque number, date and bank per voucher, at the header (`HasCheque`), and only on payments. A receipt by cheque — 144 legacy lines — has nowhere to write the cheque number; a payment by three cheques writes one. `mcj_voucher_lines.cheque_number` and `cheque_date` exist, are loaded, and are written by no screen; the bank reconciliation reads them (`voucherprint.go` , `reconcile.go` fall back to the line's reference), so the data model already expects them per line.

Proposed on the cards: cheque number and date per grid line, on every kind; the header pair stays for the single-cheque payment. No new column.

512. Card receipts lost their commission: 231 legacy lines on the swipe ledgers, and nothing computes the bank's cut

The legacy receipt form's card rows carried the swipe machine, the card type (rate#tax#name from `card_wise_bank_commission` , 25 rows), the bank commission and its service tax, and booked them. In the capture, measured by type and by ledger:

lines on swipe ledgers	231 = RCPT 86 + JRNL 145
(6)	231 = Google Pay 11836 (150) + PAYTM 12560 (75) + CASHFREE 12489

The rebuilt grid can put a line on a swipe ledger, and the scheme instalments of answer 233 do exactly that, but nothing computes or books the commission the legacy took per card row. Severity MEDIUM: the money is not lost — the bank's statement shows the net — but the commission expense is unbooked until the bank statement is journalled by hand.

Proposed on the cards: a card block per line (machine, card type from a commission master, commission and tax computed) — after 510 and 511, and after `card_wise_bank_commission` has a loader (it has none today).

Class: 415/416 — a field the legacy had and the rebuild's generic form does not. \n

513. A returned tag comes back on the shelf unguarded by count: the credit note's stock flip

The trade family's audit (`docs/transactions/vou-sale-return-php.md` , 2 September 2026). `creditnote.go` restores returned tags with

```
UPDATE mcj_stock_items SET in_stock = true ... WHERE tenant_id = $1 AND barcode = ANY($2)
AND NOT in_stock
```

and then inserts a SALE-RET movement for every barcode named, whether or not the UPDATE touched it. `checkTags` refuses a tag still on the shelf before the save, so in one request the two agree; between two requests they need not, and the sale screen's own flip checks `RowsAffected()` against the number of tags for

exactly this reason. Severity LOW: the movement would be the only trace, and the invariant *every piece has a beginning* would not notice a second "returned by customer" on a tag that never left again. The card requires `TestCreditNote_ATagAlreadyOnTheShelfIsNotReturnedTwice` and the count check.

514. Credit and debit notes record no line: the legacy priced each returned item, the rebuild takes one amount

The legacy sale-return form is the sale form turned around: item grid, stone grid, rate panel, making, certification — and its three returns fill them (`sale_return_main` rate, making, purity, weights on 3 of 3; `sale_return_detail` stones on 4 of 4). The three purchase returns do the same (`purchase_return_main` item, purity, weights, value on 3 of 3). The rebuilt credit note and debit note take **one amount** (`goods`) and, for a sale return, the tags coming back; `mcj_credit_note_lines` was loaded with the six legacy lines and is written by no screen (the same shape as 511's cheque columns).

What is lost is the per-item value on a note of several items, and the weight and purity of a returned item without a tag. Severity LOW: the tag carries weight and purity, and the money is right. Recorded on both cards; the line grid is a card decision for the reviewer, not a build.

515. A purchase makes no tag here: the legacy's 288 tags were born from a purchase bill

`legacy.barcode_main.voucher_no` names a GDP bill on 287 tags and a GCP bill on 1 tags (288 in all; `barcode_create_from_purchase_detail` holds 21 more rows of the same link), so the legacy's purchase screen — never captured — created the tags for what it bought, and the tag knew its bill. The rebuilt purchase screen writes the bill and its contents and no tag; tags are made afterwards on the tag screen with no link back to the bill, and `mcj_stock_items` has no column for one. Severity LOW today (43 purchases, all migrated with their tags already in place), and the card records the gap rather than proposes the build: linking a tag to the bill it came from is a column and a step on the tag screen, decided with the tag card (family H).

Class: 510 — a link between a document and what it produced, present in the capture and absent from the rebuild.

Corrected the same day, before the tag card was written. The tag screen does link a tag to the document it came in on: `tag.go` takes `source` (`rec / purc / stk / og`, the legacy's `created_from`) and `came_on` — a picker over the documents of that source (`arrivalDocuments`) — and writes `mcj_stock_items.voucher_no` from it, exactly as the legacy did on 2,797 of 2,816 tags (STK 2,401, GDP 287, SR 45, RV 33, DP 28, OGP 2, GCP 1). What this entry stands for is narrower than it first said: the purchase screen does not create the tags for what it bought, so tagging a purchase is a second step on the tag screen, where the legacy's screen (never captured; 21 rows in `barcode_create_from_purchase_detail`) made them in one. No column is missing and no link is lost. Severity: none — a design difference, recorded on the purchase and tag cards.

516. A registered dealer's sale cannot be raised: twenty of the shop's bills went to SALE_B2B, this screen sends every sale to SALE

The GST sale's as-built audit (docs/transactions/sales-voucher-gst-php.md , 2 September 2026). The legacy sale form had a **B2B Sale** tick and a sales ledger select; the capture shows what they did, twice over:

legacy.sales_vou.b2b_sale = 1	20 of 224 bills
legacy.sales_vou.sale_ldgr = 1014	20 of 224 (SALE_B2B) – the same twenty
legacy.sales_vou.sale_ldgr = 1016	204 of 224 (SALE)
mcj_sales_invoices.is_b2b	20 of 224 (loaded)

sale.go has no B2B field, writes is_b2b as the literal false (mcj_sales_invoices INSERT), and credits 1016 on every bill (ledgerSale). The purchase side of the same rebuild does it right: a b2b tick and the 1012 PURCHASE_B2B ledger, "without exception" on the 43 bills (creditnote.go). The sale side was built before that pattern and never caught up. Severity MEDIUM: the GSTR outward report separates B2B from B2C on is_b2b , so a dealer's sale raised here files as a retail sale.

Proposed on the card: a b2b tick (as on the purchase), is_b2b from it, goods to 1014 when ticked, a ledgerSaleB2B constant added to the mandatory accounts, and a rule *a B2B bill posts to 1014* (allowed o).

517. A bill cannot name the order it delivers: 33 of 224 bills and 57 lines did

legacy.sales_vou.order_no is filled on 33 bills and sales_vou_main.order_no / order_main_id on 57 lines (measured by fill rate and by count). The legacy sale form took **Order No** and carried the order's agreed rate and making per line (h_or_rate , h_or_making , h_or_value). sale.go reads no order field and mcj_sales_invoices has no column for one; the customer order screen (order.go) closes an order on its own deliver action, unlinked to the bill that did it. Severity MEDIUM: the two documents that are one event in the shop's practice are two unrelated rows here, and the rate agreed on the order is not what prices the bill (the bill prices at the day's rate or agreed_ typed again).

Proposed on the card: an order_no on the bill, validated against an open order of the same customer, the order's agreed rate offered as agreed_ , and the order closed by the bill. Family F's card owns the other half.

518. One instrument per bill: eight of the shop's 149 paid bills took two to four

The legacy sale form took cash, up to ten cheques and up to ten cards on one bill; sale_pay_cash (101), sale_pay_card (50) and sale_pay_bank (8) hold 159 rows for 149 bills — **8 bills paid by two, three or four instruments** (the widest, four). sale.go takes one (pay_mode , pay_amount , pay_account , pay_instrument , pay_card_type) and posts one RCPT; the rest of a split payment must be a receipt voucher raised afterwards, which cannot name the bill (defect 510) — so the bill shows a balance due that the receipt has already paid. Severity LOW: 8 of 149, and the money is right on the ledger.

Proposed on the card: the payment block as a small grid (mode, amount, account, instrument, card type) posting one RCPT with a line per instrument, mcj_sale_payments already being one row per instrument.

519. The sale has no failed-save test

`TestTrade_AFailedSaveDoesNotReportSuccess`, `TestOrder_...` and `TestTagCorrect_...` exist; the sale — the screen with the most writes (eight statements across seven tables, defect-231 class) — has none. `sale.go` does return `failedSaveStatus` (500) through `renderAt`, and `check_failed_saves.py` reads that; but the check reads the code, not the behaviour, and the guarantee the test gives (the reason survives on the page, no "current transaction is aborted" leaks) is untested here. Severity LOW; the card requires `TestSale_AFailedSaveDoesNotReportSuccess` before `tested`.

520. A tag cannot say which lot it belongs to: 2,059 of 2,816 did

The tag screen's as-built audit (`docs/transactions/barcode-opening-stock-php.md`). The legacy tag form took **Lot No.**; `legacy.barcode_main.lot_no` is filled on 2,059 of 2,816 tags (73%), naming 59 distinct lots — the batch a piece was made or received in, which is what the karigar and hallmark registers work by. `load_stock.sh` carried it into `mcj_stock_items.lot_no`; `tag.go` reads no lot field, so every tag made here has none, and the column joins the class of 511 and 514 — loaded, unwritten. Severity MEDIUM: the hallmark and refinery screens of family K issue by lot.

Proposed on the card: a `lot` field, optional, offered as the last lot used for the same source; `TestTag_RecordsTheLot`.

521. A customer's goods are held by carats alone: their weight and purity were dropped

The party-sample audit (`customer-own-goods-php.md`). The legacy receipt took, per line, pieces, carats, gross weight, metal purity and diamond purity, and its seven lines fill all five (`customer_own_goods_receive_detail`: pcs, ct, gw, metal_purity, diamond_purity 7 of 7; rate, value, shape, colour, loss, wastage, fine weight, packet 0 of 7). `mcj_customer_goods` has pieces and carats; `partysample.go` takes those two. A customer's gold piece left for setting is therefore recorded with a carat weight and no gram weight and no purity — the two things a dispute about it would turn on. Severity MEDIUM.

Proposed on the card: `gross_weight`, `metal_purity` and `stone_purity` on `mcj_customer_goods`, the loader re-run for the seven, the receive grid taking them; `TestPartySample_RecordsTheWeightAndPurity`.

Corrected on review, the same day. Read again with the seven rows in front of me: `metal_purity` and `diamond_purity` are **0 on all seven lines**, and `gw` equals `ct × 0.2` on all seven — the carat weight restated in grams, not a second measurement. The seven lines are stones, and the rebuild's pieces-and-carats record holds everything the legacy held for them. Nothing was dropped. What remains true is narrower: the legacy form could also take a gold piece with a gram weight and a purity, and the rebuilt screen cannot — a limit to record on the card, not a loss in the data. Severity: none for the seven; LOW for the day a customer leaves a gold piece.

522. A diamond's clarity was dropped at load and cannot be tagged: 416 of 819 stone lines carried one

`legacy.barcode_details.diamond_purity` — the clarity grade (IF, VVS, VS, SI ...) of a stone set into a piece — is filled on 416 of 819 stone lines (51%); shape on 555, colour on 416, cut on 28, packet on 109.

`mcj_stock_stones` has shape, colour, cut, sieve and packet columns and **no clarity column**, so `load_stock.sh` could not carry it; the tag screen's stone grid takes item, pieces, carats, rate, shape, colour and packet; and `mcj_sold_stones`, which copies the stone at sale, cannot carry it either. Severity MEDIUM: clarity is half of a diamond's price, and the GSTR and stock registers that print "D / VS" from the legacy cannot from a tag made here.

Proposed on the card: `clarity_id` on `mcj_stock_stones` and `mcj_sold_stones` against the existing clarity master, the loader re-run for the 416, a `stone_clarity` field on the tag screen.

523. "No table for this exists" was wrong for Opening Stock Rate: its table is dealer_wise_diamond_rate, 642 rows, loaded and on a built screen

The stand-in for `set_rate_sub_item.php` (SETUP > Opening Stock > Opening Stock Rate) said, since defect 459 tested the stand-in reasons: *No table for this exists among the 149 the capture contains*. The transaction programme's rule for a not-built card — check the loader first — caught it. The captured page is its own form (98 inputs beyond the search box: `shape_id[]`, `color_id[]`, `cut_id[]`, `clarity_id[]`, `rate[]`), and those are the columns of `legacy.dealer_wise_diamond_rate`:

<code>dealer_wise_diamond_rate</code>	642 rows, 2 dealers, 1 item, ₹35,500 – ₹60,500 a carat
<code>load_diamond_rates.sh</code>	reads it, with <code>diamond_rate_master</code> (774), into
<code>mcj_diamond_rates</code>	
<code>mcj_diamond_rates</code>	1,416 rows = 642 + 774 (the second measurement)
<code>reports/diamonddates.go</code>	shows them — the built Diamond Rates report

The 459 test asked whether a table matching `set_rate` or `sub_item` existed; none does by name, and the reason stood. Corrected: the stand-in now says the table exists, is loaded and is shown, and that this is the entry screen of a rate master; `docs/screens-without-data.md` moves the row from *no table exists* (17 → 16) to *already shown by a screen that is built* (1 → 2); the transaction card records `decision: not built` as a master, not a transaction.

Class: 459 / 460 — a stand-in reason put to the data and found wrong; the fourth of 67.

524. The salary card's posting ledger cited 47 journal lines and ₹10,65,016 that no query reproduces; the books say 285 lines and ₹62,00,227

The Monthly Payroll card (`docs/transactions/staff-salary-sheet-php.md`, §6) named the expense ledger the posting journal should debit as *SALARY AND BONUS — expense (SRI GANESH used exactly that name, 47 JRNL-SAL lines, ₹10,65,016)*. The end-to-end review of the 66 cards put the parenthesis to the data and no reading of it gives those numbers:

legacy_sg.voucher_detail, v_typ JRNL-SAL, ledger SALARY AND BONUS	
285 lines, 285 vouchers, ₹62,00,227 debit – all in 2025–2026, one per staff per month	
the same ledger on JRNL	64 lines, ₹4,47,253 Dr / ₹1,40,605 Cr
the same ledger on PMNT	1 line, ₹44,014
lines dated on or after 1 April 2026	0

Nothing gives 47, and nothing gives ₹10,65,016: the figure was written without a second measurement, the failure the project's own rule exists to catch (*verify every headline figure against a second source*). What the data does say is more useful than the wrong number: MCJ already holds the ledger the posting needs — **2662 SALARY AND BONUS** (one line, ₹4,000) — beside 3381 TDS ON SALARY (13 lines, ₹2,19,600), so the posting journal's debit side has a code, not only a name.

Corrected on the card: §6 line 1 now reads *2662 — SALARY AND BONUS (MCJ's own ledger, one line of ₹4,000 so far; SRI GANESH's ledger of the same name carries 285 JRNL-SAL lines in 2025-2026, ₹62,00,227 — one per staff per month)*. The figure was on no other page.

Class: 459 / 460 — a number written into the record from memory rather than from a query; caught by reading the card against the data.

525. The carry-forward card put B19's ₹15.32 crore on the 186 rolled rows; those rows are out by ₹16.44 crore

The Closing & Opening Balance card (docs/transactions/transfer-opening-balance-php.md , §2) said the 186 rows the legacy's last roll left for 2027-2028 were *out of balance by B19's ₹15.32 crore*. Summed, they are not:

legacy.ldgr_open_balance_final	186 rows: debit ₹10,62,76,061.42, credit
₹27,06,82,293.35	
credit – debit	₹16,44,06,231.93
mcj_carry_forward	the same 186 rows, the same sums (the second
measurement)	

₹15,32,28,603 is B19's own figure — the gap Gate 3 measures on the opening trial balance — and it belongs to that sentence, not to these rows. The two gaps have the same cause (closing stock was never a ledger, so the debit side is short) and different sizes, and the card now says so: *out of balance by ₹16.44 crore on their own ... the fault B19 measures on the opening trial balance at ₹15.32 crore*.

Class: 459 / 460 — a right number attached to the wrong set; caught by the end-to-end read of the cards.

526. A Run button on 73 of the 134 registers, submitting an empty form to fetch the same page

The register shell renders one parameter bar for every report: the parameters, then Run, then the export link. 73 of the 134 registered reports declare no parameters at all, so what those screens drew was a bordered box holding a single button, and pressing it submitted an empty GET and re-fetched the identical page.

Counted from the registry itself, not from the source:

reports registered	134
declaring at least one parameter	61
declaring none – Run could not act	73

Among the 73: `stock-valuation`, `top-10-items-sold`, `negative-stock`, `all-stock-summary`, `user-master`, `suspect-rates`, and the whole karigar family. Somebody arriving at a register reaches for the control that looks like the way to get the numbers, and on more than half of them it did nothing.

The bar now renders only where there is something to choose; a report with no parameters carries its export link alone. `TestReports_RunAppearsOnlyWhereThereIsSomethingToChoose` sweeps every report in the registry and fails if either arm is empty, so a shell that never draws the button cannot pass it either.

527. GST Input Register and GST Output Register were the same URL

Both menu entries pointed at `/mcj/reports/gst-register` with no parameter between them, so the two screens the old menu named separately opened on a byte-identical page. The register reads both halves on purpose — reading output tax without input tells you what you charged and not what you owe — but nothing on the screen said which of the two menu items had been clicked.

```
links.go, gst_input_register_new.php    -> /mcj/reports/gst-register
links.go, gst_output_register_new.php  -> /mcj/reports/gst-register
GSTRegister().Params                  from, to – no side
```

The register now takes a `side` parameter and still shows both by default; the menu entries carry `?side=input` and `?side=output`, and `Spec.TitleFor` names the screen from its arguments so the heading says which half you are on. The electronic credit ledger appears on either half: it is what the input tax has become, and a person reading either half is reading towards it.

The same fault, one degree milder, on the order book: Customer Order Book and Customer Repairing differ by `?kind=RP`, which was honoured, but both headings read *Customer Order Book*, so the second menu entry looked like the first arrived at by mistake. It now names itself.

528. Print a Tax Invoice and Voucher Print asked for a query string to be typed by hand

Both are reached from the menu with no query string, and both answered with the shape of one:

```
/mcj/invoice      "Give a bill number, for example ?bill=GST/2627/00001"
/mcj/voucher-print "Give a voucher, for example ?type=RCPT&number=3&year=2026-2027"
```

That asks the shop to know a bill number — or a voucher type, number *and* financial year, because a voucher number alone is not a key — before it can look one up. The person printing is holding the goods or the counterfoil, not the number. Reported by Sudipta on 3 September as two screens that are "completely empty".

Each now offers a box for a known number and the 25 most recently written documents to pick from. The invoice list marks a bill that is cancelled or returned before the paper comes out; the voucher list names the counterparty by the rule the printed voucher itself uses — the line that is neither cash nor bank. A number

that matches nothing comes back with the box still filled and the list still offered, rather than a dead end.

529. "Work it out" refused to price a purchase until its contents were entered

On Purchase Voucher GSC and GCO — the two trade kinds with `WritesInvoice` — the pricing action ran the contents check as well as the save. A biller who entered the party, the date and the amount and pressed the only button that speaks to those three was answered, in red, *Say what the bill bought*. The arithmetic it offers had in fact been computed and was on the page below the error, which is the worst of both readings: the button appeared to have refused.

```
TradeKinds() with WritesInvoice: purchase (purchase_vou_gsc_new.php),  
                                   purchase-order (purchase_vou_gco_new.php)  
credit-note and debit-note: unaffected, and neither was reported
```

The contents are required to **save** — a purchase without them is absent from the purchase register entirely, which is defect 223 and still enforced — and not to price. `readContents` no longer reports an empty table as a malformed one, because only the caller knows which button was pressed; while pricing, the outstanding step is said in plain type beside the total instead of in red instead of it.

530. Tag Print Setup read as a list with its Add button missing

`mcj_tag_print_setup` carries a unique index on `tenant_id`: one settings record per shop, and the screen is registered `WithoutCreate` and `WithoutDelete` because a shop with two conflicting printer configurations has nothing to say which one wins. What a visitor met was a list of exactly one row, no Add control, and no explanation — a complete screen that reads as a broken one. Reported by Sudipta on 3 September.

It now opens on the record it exists to edit. The redirect makes the same read- permission check the SDK's List makes inside its own handler; without that it would hand the record's id to any authenticated user, which is the shape of defect 107. Where no record exists the list still renders, so the screen degrades to what it was rather than to nothing.

Beside it, from the same report: **Barcode Category Master opened on a page headed "Design Categories"** (`mcj_categories.List.Title`), and **the three rate boxes on Daily Metal Rates sat hard left under headings sitting hard right** — the headings are `text-right`, the boxes a fixed `w-32` in a `w-full` table, and the cell between them carried no alignment at all. On the one screen the shop retypes every morning.

531. Three tests on the merged branch had never passed, and three generated files did not match their source

Found merging PR #2 (`SudiptaPaul-31:sudipta/new-task` , 9 commits, 45 files) on 4 September. The merge itself was clean; the branch was not.

<code>TestCRUDSearch_HSNFiltersRows</code>	seeded "SEARCH-HSN-ALPHA" (16 chars) into <code>mcj_hsn_codes.number: varchar(8)</code> , CHECK length IN (4,6,8)
<code>TestOrder_RendersRowControls</code>	counted 3 add-row controls, asserted 2
<code>TestTag_RendersCompactLayoutAndStoneRowControls</code>	counted 2 stone rows, asserted 1

The first could not have run once. The other two count a bare `data-` marker across the whole response — and the script that adds and deletes rows names the same marker in a `querySelector`, so one rendered row reads as two and two buttons read as three. They now count against the page with its inline scripts stripped, which is what the browser draws.

Separately, `partysample_templ.go`, `creditnote_templ.go` and `sale_templ.go` were stale against their own `.templ` sources — source line numbers in the error paths off by four to six lines, script blocks still emitted as backtick raw strings — so the pages the browser drew were not the pages the source described. The other 413 generated files in the tree already agreed with `templ vo.3.857`. Regenerated; the emitted string literals compared before and after, whitespace-normalised, gain and lose nothing — same tags, same `data-` attributes, same script bodies.

532. Three menu entries opened a page another entry already owned, and one of the three showed the wrong rows

A sweep of all 253 registered navigation screens for entries sharing a URL, run after defect 527 fixed the GST pair. Three more:

<code>/mcj/reports/order-book?kind=RP</code> <code>(customer_order_reg_all.php)</code>	REPORTS > Customer Report > All Order
<code>(customer_order_list_new.php?bill_type=RP)</code>	MANUFACTURING > Customer Repairing
<code>/mcj/reports/sales-register</code> <code>(sale_register.php)</code>	REPORTS > Sales Report > Sale Register
legacy file)	REPORTS > Sales Report > Sales Register (placed, no
<code>/mcj/reports/user-rights</code> <code>(user_access_right.php)</code>	SETUP > User Management > User Access Right
<code>(user_right_report.php)</code>	MIS REPORTS > User Right Report

All Order was the one that lied. Its name and its place in the menu both say every order, and it pointed at the repairs filter:

<code>mcj_customer_orders</code>	284 rows	C0 179, RP 105
----------------------------------	----------	----------------

So the single entry that promises the whole book showed 105 of 284 and hid the 179 custom orders, while Customer Repairing — two entries below it — showed the same 105. It carries `?status=all` now, the kind unfiltered; the status matters too, because the order book opens on live work and a register called All Order that drops the 214 cancelled lines is the same fault in the other column.

Sales Register was a duplicate placed on a false premise. Its comment said the register "lost its menu slot to the screen that slot was named for" — but the slot GST Sale took was `sales_voucher_gst.php` 's, not the register's, and `sale_register.php` still places it under REPORTS > Sales Report. The second entry was placed in that same subgroup, so "Sale Register" and "Sales Register" sat adjacent and opened one page. Removed.

User rights is left as it is, and asked. One screen answers both legacy names on purpose — it carries the whole 2,755-row permission record — but there is no rights editor here, so the SETUP entry named for one leads to a read-only report. On the ask list rather than reclassified: moving a screen out of the built count is a

coverage decision, not a fix.

`TestNavigation_NoTwoEntriesOpenTheSameURL` now fails on any pair that is not declared, and the user-rights pair is the only declaration.

Also checked in the same sweep and clean: every one of the 133 report constructors is registered, every one of the 40 controllers is mounted, every form field rendered by a page is read by a controller, and `templ generate` rewrites none of the 417 generated files.

533. Every image the application asks for answered 404, so the sidebar drew a broken icon where the logo belongs

Reported from the running app: the logo slot showed the browser's broken-image glyph. Measured against the live server, signed in:

```
/assets/css/*      200    (5 stylesheets)
/assets/fonts/*    200    (3 faces)
/assets/js/*       200
/assets/images/logo-<hash>.webp      404
/assets/images/logo-compact-<hash>.webp 404
/assets/images/favicon.ico              404
```

Every other asset class serves. Only `images/` is missing, and for the reason `internal/assets` already exists: the SDK's `modules/core/presentation/assets` is a build output, gitignored upstream, so it is absent from the published Go module. The stylesheet was lost this way once before — that is why `internal/assets` and `TestLoginPageStylesheetsResolve` exist — and the images were the other half of the same hole, never noticed because every page still returned 200.

Vendoring the files alone does not fix it. `hashfs.FS.open` strips the hash from a request only when it matches *its own* hash of that file:

```
base, hash := fsys.ParseName(name)
if hash != "" && fsys.HashName(base) == name { name = base }
```

The SDK hashes `images/logo.webp` from its own embed; a copy of a different file in our embed hashes differently, so the SDK's URL still misses. The application therefore provides the logo component itself — `assets.Logo()`, naming the files by our hash — through an after-middleware on `constants.LogoKey`, which runs later than the SDK's `Provide(LogoKey, DefaultLogo())` and earlier than `middleware.ProvideDynamicLogo`, so a tenant that uploads its own logo still wins. The favicon needed no component: the SDK asks for it unhashed.

The files are Nimbus Systems' own mark rather than a placeholder, built from `Logo1.jpg` in the repository root: the white surround flood-filled away from the four corners so the white strokes inside the mark survive, then fitted to the boxes the SDK asks for — 96×96 for the collapsed sidebar, 300×100 for the expanded one, where the square mark is centred on a 3:1 canvas because the img carries `width="150" height="50"` as attributes and would otherwise be stretched to them. The mark alone, with no wordmark, is what lets one image serve both themes: the sidebar's surface tokens flip with the theme, and the mark's green and blue both hold on white and on near-black.

`TestSidebarImagesResolve` in the smoke suite now fetches every image the sidebar draws, the favicon included, and fails on a 404 or an empty body.

534. The captured menu's own note, "(direct)", was read as a subgroup and became 52 sidebar entries

Reported from the running app: the sidebar carried a group called (*direct*).

`legacy_menu.csv` writes (`direct`) in the subgroup column for an item that hangs straight off its top-level group — a note about the capture, not a name. `loadLegacyMenu` took the column verbatim, and every `if e.Subgroup != ""` in the tree builder then treated it as a real subgroup:

<code>legacy_menu.csv</code> rows carrying "(direct)"	32
<code>links.go</code> screens copying it into their Subgroup	3
occurrences on one rendered page	52

Fifty-two because the SDK repeats a group's name in its label, its tooltip and its `aria-label`, so a screen reader announced "(direct)" as well. Seven groups had one — Manufacturing, MIS Reports, Retail, Audit, GST Reports, DSS Report, Imp Reports — and under each, screens that belong one level up sat one level down inside a folder named after a note.

Why no check caught it. `check_menu_keys.py` asks whether every menu label has an `i18n` entry. Each of the seven did:

```
"manufacturing__direct": "(direct)"
"mis-reports__direct":  "(direct)"
...
```

The key existed, so the check passed; nothing looked at the value. The seven entries are deleted with the groups they named.

`ownSubgroup` now maps the sentinel to "" at both boundaries — the CSV parse and `Register`, so a screen placed by hand cannot reintroduce it either — and two tests in the navigation package hold it there, one for a legacy screen (MIS Reports > User Right Report) and one for a placed screen.

535. The sidebar gave a top-level menu the same rhythm as another menu's children, so nothing said where one ended

Reported from the running app: the menu has no clear division between one entry and the next.

Read off the rendered page, the sidebar is:

<code>nav#sidebar-navigation > div > ul</code>	the top-level menus
<code>> div > li</code>	one menu each
<code>> details.group > ul.ml-4</code>	that menu's children

Both lists carry the same `flex flex-col gap-2`, both rows the same `btn btn-sidebar btn-md`, and the children are indented one rem and marked in no other way. With Administration open, its seven children ran into Retail's header at the same spacing as everything else, so Users and Roles read as peers of Setup and

Retail rather than as theirs.

Two rules in `internal/assets/css/mcj.css`, the application's own stylesheet, loaded after the SDK's:

- a rule between one top-level menu and the next, suppressed when the sidebar is collapsed to a column of icons (Alpine puts `px-2` on the list in that state), where a line straight across reads as a divider between sections that are not there;
- a rail down the left of a group's children, at every depth the menu nests to — it nests three deep in places.

Nothing about what is in the menu, or the order it comes in, changes.

Getting the stylesheet loaded needed the same move as the logo in defect 533: the SDK builds its whole head in one component with no slot to add to, so `assets.Head()` renders `layouts.DefaultHead()` and follows it with the one link, installed on `constants.HeadKey` by an after-middleware that runs later than the SDK's own `Provide`.

The selectors are anchored to the nav rather than written against the classes, because a child list carries the same classes as the top-level one; both were checked against the live page's DOM before being written — 28 top-level entries under the first list, 76 nested lists under `details.group`.

536. The landing page said 144 of 194 screens where the menu holds 193, and STATUS.md said 143 of 193

Reported from the running app: why does it show 144?

`statistics_report.php` is listed twice in the captured menu — MIS REPORTS › Statistics Report, and REPORTS › Sales Report › Statistics Report. One screen, reachable from two places.

`navigation.Progress()` counted a link once per SECTION and then added the sections up, so that screen was two in the headline:

menu rows carrying a link	195
distinct links	193
rebuilt	143
Progress() said	144 of 194
STATUS.md said	143 of 193

The landing page was contradicting the status document in public, and nothing compared them.

The opposite error was hiding behind it. `status_figures.py`'s `by_area` deduplicated globally, so it credited Statistics Report to whichever section came first in the file and not the other — REPORTS read **36 of 48** for a menu that offers 49. One tool double-counted and the other under-counted, and the two faults were in different figures so neither check saw the other.

The two counts now have their own rule, because they answer different questions:

- a SECTION counts what that section offers, so a screen two menus offer is counted by both — REPORTS is 37 of 49, corrected in `STATUS.md` and `screens.md`;
- the HEADLINE counts screens, so it is 143 of 193.

They therefore differ by one, and the page says so outright rather than leaving a reader who adds the cards up to decide which figure to distrust:

```
143 of 193 screens rebuilt from the previous system.  
One screen is counted in both the menus that offer it, so the cards add to 194.
```

`TestProgressCountsScreensNotMenuRows` counts the distinct links straight from the CSV rather than through Progress, because a check that reuses the arithmetic under test agrees with it however wrong it is.

537. There was no search on the page at all below 1024px, and never one in the sidebar

The SDK has a search — the spotlight — and it indexes the `Keywords` every screen registers, which is what somebody hunting for "karigar" or "hallmark" needs across 193 screens. It sits in the top bar inside `hidden lg:block`, so below 1024px the page carried no search at all, and the sidebar — where a person looks for one, and where all 193 screens are listed — never had it.

`internal/assets/js/mcj.js` adds a Search button as the first entry of the sidebar list. It opens the SDK's own panel rather than being a second search: the spotlight's Alpine component listens for `ctrl/cmd-K` on the `WINDOW` (`@keydown.window="handleShortcut($event)"`), so dispatching that reaches it wherever it is and whether or not the top-bar trigger is hidden.

Checked in a browser against the real script: the button is inserted, it is first in the list, clicking it reaches a window keydown listener, and running the installer again after an `htmx` swap leaves one button rather than two.

538. Eleven stand-ins said "not built" and did not say the data was on screen under another name

Of the fifty screens the old menu has and this system had not rebuilt, eleven are covered by a screen that IS built — the register exists and only the entry form is missing, or the same list is built under a newer legacy file name. Each one's own `WouldChangeIt` said so:

```
Nothing to view here -- see the screen named above.
```

but the page a visitor reached said only *not built*, and sent them away from the screen that would have answered them.

Five where BOTH the viewing and the entry are covered are claimed against that screen, the way `gst_input_register_new.php` is claimed against the GST register: `ledger_setting.php` → Ledger Master, `sales_voucher.php` → GST Sale, `purchase_vou_sc_new.php` and `purchase_vou_co_new.php` → the trade vouchers built for PURC-GSC and PURC-GCO, `karigar_full_ledger_detail.php` → the karigar ledger built under its `_new.php` name.

Three of the five were then reverted, by this repository's own gate. `check_transaction_cards.py` refuses a screen built against a card marked `decision: not built` whose `approved_by` is empty:

sales-voucher-php.md:	decision: not built, approved_by: (blank)
purchase-vou-sc-new-php.md:	decision: not built, approved_by: (blank)
purchase-vou-co-new-php.md:	decision: not built, approved_by: (blank)

The gate is the point — a screen becomes built against an APPROVED card, and approval is the shop's, not the builder's. Ledger Setting and the old Karigar Ledger slot have no card and landed; the other three wait on three signatures. Coverage 143 → **145 of 193**, and RETAIL stays 15 of 26 until they are signed.

The remaining six keep their stand-in and carry a new `SeeInstead` link — *This is on screen elsewhere*, with a button straight to the built register. Five of those six are the karigar and mint entry vouchers, which additionally wait on questions 3.11–3.14; the sixth, Opening Stock Rate, waits on nothing but a rate-master screen over `mcj_diamond_rates`.

Two of the five claims made a second menu entry share a URL, which

`TestNavigation_NoTwoEntriesOpenTheSameURL` (defect 532) caught on the next run. Both are declared in its allow-list with the reason, alongside user-rights: `/mcj/ledgers` and `/mcj/reports/karigar-ledger-detail` each answer two of the old menu's names in full.

`mcj_problems.md` in the repository root lists all 48 remaining in plain language, generated from `todo_screens.go` so the document cannot drift from the list the application serves.

539. Nine legacy registers had no table and so had no screen, when what they were for was recoverable and their shape was not the missing part

Defect 538 left 48 screens standing in, each showing a page that said it was not built and why. That is honest and it is nearly useless: a reader learns that the screen is absent and nothing about what it was for, what it would have shown, or whether they need it. Sixteen of the 48 stood in for the same reason — *no table for this exists among the 149 the capture contains* — and that reason was taken to settle the matter.

It does not. **A register's columns and filters are most of what a register tells you**, and for nine of the sixteen both survive.

What was measured. Every table the capture's own reconstruction proposes for these screens was searched for among all 149 recovered tables:

purchase_approval	absent
purchase_approval_return	absent
container_receive_detail	absent
temp_purchase_register	absent
temp_work_order_wise_stone_report	absent

Not empty — **absent**. So the reason the stand-ins gave was correct, and it was also not the whole story: the parameter bars are in the captured HTML, which is server-rendered, so the from/to pair each of these offered is **evidence, not inference**. Only the columns are reconstructed, because the legacy filled these tables by AJAX and the JavaScript did not survive.

Built as shells under decision 13: real columns, real filters, real export, no rows, and a sentence on the page saying there is no table behind it *and that the columns are a reconstruction*. Purchase Approval Register, Sales Approval Register, Approval Pending Report, Purchase During Receive, Broken Stone Ledger, TDS On

Job Charge Register, Work Order Report, Work Order Wise Stone Report, Work Order Wise Stone Balance. **148 → 157 of 193**; REPORTS 38 → 46 of 49 and IMP REPORTS 2 → 3 of 3, which completes that area.

The seven not built are the ones where the capture does not say what the columns were: the two barcode screens, the four studded and sample opening-stock screens, and Discount Scheme Master. A screen that looked authoritative about a system nobody can confirm is worse than the gap.

Two things this got wrong on the way, both caught by the repository's own checks rather than by me:

One — I recorded in the plan that empty shells were safe with the render check. They are not. `check_screens_render.py` fails a report that shows no data over the whole of the shop's history, on the reasoning that *a report that has stopped matching looks like one with nothing to say* — and all nine failed it on the first run. The mechanism for this already existed and I had not read far enough to find it: `KNOWN_EMPTY` takes a report key, **the placeholder the page must show**, and the evidence. All nine are declared there. That is stricter than an exemption would have been: a shell that stopped *saying* it has no table now fails, and any of them that ever shows a row must come off the list.

Two — `mcj_problems.md` claimed to be generated from `todo_screens.go` and was written by hand. It drifted the moment a screen landed. It is now genuinely generated (`tools/gen_mcj_problems.py` , checked by `check_all.sh`), and the first honest run of the generator **dropped two screens** — `karigar_gold_issue` and `set_rate_sub_item` — because their fields are written as concatenated Go strings across a dozen lines and the parser read line by line. The screen it silently lost is **the one with ₹1.35 crore of gold behind it**. Reconciling the generator's count against `check_todo_screens.py` is what caught it; the generator alone would have printed a confident 34 and been wrong.

540. The screen called Backup returned the whole database to anyone who opened it, and this project's own copy of the data is that page's output

`back_up.php` sat under AUDIT and stood in here as *"a utility rather than a screen with data behind it"*. That was true of the stand-in. It was not true of the original.

What opening that URL returned. The entire database as SQL, in the HTTP response body:

```
1,28,628  INSERT statements
37,389,767  bytes
149  tables, including the five credential tables blocker B15 is about
40,519  customer records
```

It is not an inference, and the evidence is uncomfortable: this project's own

`gopiji/mcj_full_backup_2026-08-21.sql` **IS that page's output.** The two files have the same md5 over their first 200 bytes, end on the same bytes, and differ in length by **121 bytes** — 37,389,767 against 37,389,646. The dump every figure in this repository is measured from was obtained by fetching a web page.

That is the finding twice over. **For the legacy:** any account that could reach one URL could take the whole shop — the customer list, the ledgers, and the SMS and WhatsApp credentials — with no export permission, no audit record, and nothing on any screen that would show it had happened. **For this rebuild:** a faithful port of that screen would put it straight back, and "faithful to the legacy" is the argument that would have carried it.

Built instead as the half worth having. The screen reports the row count of every `mcj_` table — scoped to the tenant where the table carries one, whole where it does not, in one query rather than a round trip each — and says plainly at the top that it does not send the database to a browser, and why. **Checking a backup is the real need:** a restore that brings back 83 tables where the shop has 102 is a bad backup, and nothing else on any screen would have said so. Taking the backup stays a server operation, where the file can be written to a disk that is not the database's.

AUDIT is now **4 of 4**, complete.

541. There was no way for anybody in the shop to change a password

`change_psw.php`, SETUP › Retail Management, stood in for the same reason as Backup and with the same mistake in it. It is not a utility. It was the only way anybody in the shop could change a password without somebody opening the database — and **blocker B15 is a list of passwords that need changing**.

The SDK does not provide one: its account controller covers the profile and the session list and stops there, while `CheckPassword` and `SetPassword` sit on the user aggregate unused by any screen.

Built, with the security decisions written down rather than assumed:

- **It changes only the password of whoever is signed in.** An administrator resetting somebody's forgotten password is a different screen against a different permission. A screen that can change any password is worth attacking; this one is worth nothing to an attacker already signed in as you.
- **It is the one screen here with no permission gate**, on purpose. Every other screen asks whether this user may see this data; this one only ever acts on the requester, so a permission could only take away somebody's ability to change their *own* password.
- **The current password is checked first and alone.** Validating the new one first would let somebody at an unattended counter tell a correct password from a wrong one by which complaint came back.
- **Nothing typed in is echoed back into the page.** A password in the HTML is in the page source, the browser's back cache and any proxy log between the counter and the server, which is the whole of what the screen protects.

Tested five ways, and the harness had to be corrected to do it. `itf.User` builds a user with **no password at all**, and `CheckPassword` on an empty password is false for every input — so the "a wrong password is refused" test would have passed against a screen that refused nothing. The suite now signs in a user whose password is known and asserts the user recognises it before proceeding. `itf` also never writes the user to the database, so the save failed on a foreign key that had nothing to do with the screen; the test seeds the signed-in user. Falsified by making the screen save the OLD password: the test reports *the OLD password still works after the change*.

542. Twenty-five screens were "not built" for four different reasons, and only one of them was a reason not to build

Asked to complete the thirty screens that still needed a transaction card, the first useful thing was to stop treating them as one list. Every one carried a `decision: not built` card, and the cards' own §12 sections divide them sharply:

16 of 30	nothing open – buildable now
13 of 30	blocked by 3.9 / defect 164 alone
1 of 30	blocked by 3.14 (does the shop use a mint at all)

The thirteen are one question, not thirteen. Defect 164 asks whether MA TARA JEWELLERS still holds 869.850 g of 995 fine — ₹1.35 crore at today's rate — and its third answer, *an abandoned trial*, makes every karigar screen decision: not built. It is one of the five money questions decision 15 excludes from assumption, so building those thirteen would have meant asserting that the gold is outstanding. **One answer unblocks fourteen screens** (the thirteen plus Karigar / Dealer Studded Stock, which is karigar entry under another name).

Of the sixteen, eleven were built and five were not, for reasons worth separating:

Three were aliases nobody had noticed. A studded piece is a tag with stones, and the built Tag Opening Stock screen writes `mcj_stock_stones` — all 819 stone lines the shop holds were entered that way. A party's studded stock is the party's own stones held by the shop, which is `mcj_customer_goods`, 233 pieces and 14.43 carats. Neither needed building; both needed noticing.

Two were a filter. `raw_opening_stock.php` appears three times in the old menu and is one screen over one table, separated by `holder`. Two were stand-ins — and **the read was unfiltered**, so Own Raw Stock would have shown a party's metal the day any existed. Correct today because all ten rows are `sh`; wrong the moment the screens it was missing were used.

Six were genuinely new — Opening Stock Rate, the four approval vouchers, and barcode split/merge — and each is recorded with what is measured and what is borrowed. Three of the six take their shape from SRI GANESH because MCJ has never performed the workflow, and every one of those cards says so in an assumption: line rather than leaving it to be discovered.

Five could not honestly be built, and the reasons differ:

- **SEND WHATSAPP and SMS ORDER SETUP** — this repository's own `check_no_credentials.py` forbids any `mcj_` column whose name reads like a secret, which is blocker B15 enforced in code. A credential settings screen cannot store credentials, so it is not a screen; it is deployment work.
- **Export Sale and Exhibition Sale** — no such sale type exists in either capture, and an export sale's GST treatment (zero-rated under LUT, or IGST paid and refunded) is a tax decision. Now asked rather than assumed.
- **Discount Scheme Master** — its captured page holds **no discount fields at all**, only the shared chrome: the export widget, the logout field and the state selector. The form was drawn by JavaScript that did not survive. Its columns are unrecoverable, and inventing them is what the sixteen no-table screens were spared for the same reason.

Two findings inside the work:

A screen that was already built had an unfiltered read. Own Raw Stock read every row of `mcj_raw_stock` whatever its `holder`. Nothing was wrong with the output because every row is the shop's own — the fault was invisible precisely because the two screens that would expose it did not exist. Both the

read and the **delete** are scoped now: three screens sharing a table need the filter that separates them on the way in to separate them on the way out, or the party screen could remove the shop's own metal by being handed its line id.

Adding a permission needs `seed-admin` re-run, and nothing said so. The Opening Stock Rate screen answered **403 to an administrator holding every permission**, because a permission declared in the binary is not a permission anybody holds. It is not a visible failure — the menu entry is simply absent — so it is easy to deploy and believe the screen was never built. Now in `deploy/README.md`.

Three failures the quick suite could not have caught, found by the full one. `tools/check_all.sh --quick` skips the Go suite, so eleven screens were built and committed against 23 green checks that never ran a Go test. The full suite found three:

- `reports/approval.go` **cast \$3 and \$4 on one use and not the other.** That is the rule this project has already written down twice — `TestSQL_ARepeatedParameterIsCastConsistently` exists because of defect 486 — and it was reintroduced anyway. Postgres deduces a parameter's type from the uncast use and then fails at run time; the register would have worked empty and broken on its first row.
- **Change Password's POST is not behind a permission, and its menu entry has no visibility policy.** Both were deliberate and neither was declared, which is the same thing as an accident to a checker. They are declared now, in the tests themselves, with the reasoning: the screen acts only on the user making the request, so a permission could only take away somebody's ability to change their own password, and a visibility policy could only hide the one screen every signed-in user may open. Any other ungated POST or policy-less entry still fails.

A fourth, on the next run, and it was mine twice over. Moving the three approval registers onto a real table left my own phase-2 test still asserting they say *there is no table behind this screen*. The test was right to fail: the registers had changed what they claim and the test had not been told. It is split now — six shells assert the no-table wording, three approval registers assert the merely-unused wording, and **each asserts the OTHER wording is absent**, so the two can never quietly converge on one message.

What this says about the checks: the 23 that pass in two minutes are worth having and they are not the suite. A phase that ends green on `--quick` has not been tested — it has been linted. The Go suite belongs at the end of every phase that adds a controller, not at the end of the day's work. **Four real failures accumulated behind that gap in a single day's work**, and three of them were in code I had just written and believed verified.